Appunti di Data Mining

Appunti del Corso

Anno Accademico 2026/2027

Indice

1 Introduzione al Data Mining e il Processo KDD **1**
1.1 Fondamenti, Motivazioni e Definizione Formale 1
1.2 La Confluenza di Discipline e il Processo KDD 4
1.3 Tassonomia dei Task di Data Mining 7
1.4 Sfide Aperte del Data Mining Moderno 10

2 Data Understanding I: Tipologie di Dati, Attributi e Statistica Descrittiva **12**
2.1 Conoscere i Dati e Tassonomia dei Dataset 12
2.2 Tipologie di Attributi e Proprietà di Scala 15
2.3 Problemi di Qualità del Dato 17
2.4 Ispezione delle Distribuzioni Monovariate 17
2.5 Misure di Tendenza Centrale e Dispersione 19
2.6 Five-Number Summary, Quartili e Box Plot 20

3 Data Understanding II – Analisi Multivariata, Correlazione e Rilevamento Outliers **22**
3.1 Tecniche di Visualizzazione Multidimensionale 22
3.2 Analisi di Dipendenza Bivariata: Covarianza e Correlazione . . 26
3.3 Tassonomia e Rilevamento degli Outlier 31
3.4 Gestione dei Valori Mancanti 33

4 Data Preparation & Cleaning – Riduzione, Discretizzazione e Trasformazione **35**
4.1 Differenze tra Data Understanding e Data Preparation 35
4.2 Aggregazione e Campionamento 36
4.3 Riduzione della Dimensionalità 38
4.4 Analisi delle Componenti Principali (PCA) 40
4.5 Gestione dei Valori Mancanti 43
4.6 Tecniche di Discretizzazione e Binarizzazione 44

4.7 Trasformazioni Funzionali di Attributi e Stabilizzazione della Varianza . . . 47
4.8 Algoritmi di Normalizzazione . . . 49
4.9 Trattamento Univariato degli Outlier . . . 50

5 Laboratorio Python: Data Understanding e Data Preparation 53
5.1 Ambiente di Lavoro, Ingestione e Data Integration . . . 53
5.2 Diagnosi e Imputazione dei Valori Mancanti . . . 56
5.3 Esplorazione Multivariata e Analisi di Correlazione . . . 57
5.4 Feature Transformation, Discretizzazione e Encoding . . . 59
5.5 Riduzione Dimensionale: PCA e t-SNE . . . 60

6 Misure di Prossimità, Teoria dell'Informazione e Cluster Analysis 64
6.1 Fondamenti di Similarità, Dissimilarità e Prossimità . . . 64
6.2 Prossimità Unidimensionali per Singolo Attributo . . . 65
6.3 Distanze Vettoriali e Metriche di Minkowski . . . 67
6.4 Esempio di Calcolo delle Matrici di Distanza . . . 68
6.5 Proprietà Matematiche di Metriche e Similarità . . . 69
6.6 Prossimità tra Vettori Binari: SMC vs Indice di Jaccard . . . 70
6.7 Similarità Coseno per Dati Documentali . . . 71
6.8 Ponderazione, Dati Mancanti e Correlazione tra Oggetti . . . 72
6.9 Teoria dell'Informazione: Entropia e Informazione Mutua . . . 73
6.10 Principi Fondamentali della Cluster Analysis . . . 75
6.11 Tassonomie dei Metodi e Strutture di Raggruppamento . . . 77
6.12 Tipologie di Cluster e Formulazione Matematica . . . 78
6.13 Caratteristiche dei Dati di Input e Famiglie Algoritmiche . . . 80

7 Cluster Analysis: Algoritmi K-Means e Clustering Gerarchico 82
7.1 Introduzione alla Cluster Analysis Partizionale e Gerarchica . 82
7.2 K-Means: Algoritmo di Lloyd e Geometria di Voronoi . . . 83
7.3 Funzione Obiettivo SSE e Dimostrazione di Convergenza . . . 88
7.4 Scelta del Parametro K, Minimi Locali e Inizializzazione . . . 90
7.5 Euristiche di Inizializzazione e Varianti di K-Means . . . 92
7.6 Limitazioni Strutturali di K-Means e Tecniche di Adattamento 94
7.7 Fondamenti del Clustering Gerarchico . . . 96
7.8 Algoritmo Agglomerativo e Criteri di Collegamento (Linkage) 97
7.9 Clustering Gerarchico Divisivo basato su Minimum Spanning Tree (MST) . . . 104
7.10 Complessità, Limiti e Confronto Sinottico . . . 105

8 Cluster Analysis: Clustering Basato su Densità (DBSCAN) 107
8.1 Introduzione al Paradigma Basato su Densità 107
8.2 Fondamenti della Densità e Tassonomia dei Punti 108
8.3 Le Relazioni di Connettività per Densità 111
8.4 Architettura Operativa dell'Algoritmo DBSCAN 114
8.5 Analisi Fenomenologica su Dati Spaziali Continui 120
8.6 Punti di Forza e Capacità Geometriche 121
8.7 Limitazioni Critiche e Sfide Applicative 122
8.8 Metodologia Euristica per la Scelta dei Parametri (Eps e MinPts) 124
8.9 Quadro Comparativo Globale: K-Means vs Gerarchico vs DBSCAN 127

9 Varianti di K-Means, Criterio BIC, Modelli Mistura ed EM 128
9.1 Inquadramento Scientifico e Limiti del K-Means Standard . . 128
9.2 L'Algoritmo Bisecting K-Means 130
9.3 L'Algoritmo X-Means e la Scelta Automatica di K 134
9.4 Il Criterio d'Informazione Bayesiano (BIC) 136
9.5 Modelli di Mistura ed Algoritmo EM 139
9.6 Distribuzioni Normali e Fondamenti di Stima Statistica 143
9.7 Caso di Studio: Deconvoluzione di una Miscela di Due Gaussiane 144
9.8 Confronto Architetturale e Sistematico: K-Means vs. EM . . . 146

10 Validazione del Clustering: Misure, Teoria e Metodologie 149
10.1 Fondamenti e Sfide della Validazione del Clustering 149
10.2 Tassonomia delle Misure di Validazione 151
10.3 Tendenza al Raggruppamento e Statistica di Hopkins 153
10.4 Validazione Interna mediante Matrici di Prossimità 156
10.5 Coesione e Separazione: Teoria ed Equivalenza Algebrica . . . 158
10.6 Il Coefficiente di Silhouette 164
10.7 Metriche Interne: Davies-Bouldin e Caliński-Harabasz 167
10.8 Misure Esterne: Entropia, Purezza e Corrispondenza 168
10.9 Quadro Statistico Inferenziale e Modelli Nulli 171
10.10 Laboratorio Python: Metriche di Validazione e Selezione di K 173

11 Analisi delle Associazioni: Concetti di Base e Algoritmi 178
11.1 Introduzione al Mining delle Regole di Associazione 178
11.2 Definizioni Formali: Itemset, Supporto e Frequenza 180
11.3 Regole di Associazione e Metriche Fondamentali 181
11.4 Il Compito di Mining e la Trattabilità Computazionale 183
11.5 Disaccoppiamento del Problema: L'Approccio a Due Fasi . . . 184
11.6 Reticolo degli Itemset e Complessità di Generazione 185

11.7 Il Principio Apriori e la Proprietà di Anti-Monotonia 187
11.8 Esecuzione Tracciata Passo-Passo dell'Algoritmo Apriori . . . 188
11.9 Pseudocodice e Tecniche di Generazione dei Candidati 190
11.10Conteggio del Supporto con Hash Tree 193
11.11Generazione delle Regole dagli Itemset Frequenti 195
11.12Fattori che Influenzano la Complessità Computazionale 197
11.13Rappresentazioni Compatte: Itemset Massimali e Chiusi . . . 198
11.14Valutazione dei Pattern e Limiti della Confidenza 201
11.15Misure Statistiche di Interesse Oggettive 203
11.16Trattamento di Attributi Categorici e Continui 205
11.17Gerarchie Concettuali e Regole Multi-Livello 206

12 Sequential Pattern Mining: Modelli, Definizioni ed Esercizi 209
12.1 Introduzione e Motivazioni al Sequential Pattern Mining . . . 209
12.2 Dal Paradigma Transazionale al Paradigma Sequenziale 212
12.3 Tassonomia e Rappresentazione dei Dati Sequenziali 215
12.4 Formalizzazione Matematica delle Sequenze 218
12.5 Relazione di Contenimento e Definizione di Sottosequenza . . 220
12.6 Supporto dei Sequential Patterns e Istanze Multiple 224
12.7 Esercizio Guidato: Matching Completo ed Enumerazione delle Istanze 226

13 Mining di Regole di Associazione (FP-Growth), Simulazioni di Clustering e Pattern Sequenziali (GSP) 231
13.1 Limiti dell'Approccio Level-Wise e Motivazione di FP-Growth 231
13.2 Architettura della Struttura Dati FP-Tree 233
13.3 Costruzione Dettagliata dell'FP-Tree con Ordinamento Lessicografico 234
13.4 Dimensione dell'FP-Tree ed Euristica 237
13.5 Estrazione dei Pattern Frequenti con FP-Growth (Divide and Conquer) 239
13.6 Traccia di Esecuzione Completa sul Suffisso E (minsup $= 2$) . 241
13.7 Esplorazione Ricorsiva per il Suffisso E 243
13.8 Elaborazione del Suffisso Successivo: Suffisso D 245
13.9 Proprietà e Complessità dell'Algoritmo FP-Growth 246
13.10Esercizio 1: Simulazione Geometrica e Analitica di K-Means . 246
13.11Concetto Introduttivo di DBSCAN su Spazio Metrico Continuo 249
13.12Esercizio 2: Simulazione di K-Means con Fenomeno di Migrazione dei Punti 250
13.13Fondamenti sui Vincoli Temporali nelle Sequenze (Min-Gap e Max-Gap) 252

13.14Esercizio 1: Pattern Sequenziali con Vincolo Min-Gap 252
13.15Esercizio 2: Pattern Sequenziali con Vincolo Max-Gap 254
13.16Algoritmo GSP: Simulazione del Livello $k = 1$ su Dataset . . . 256
13.17Algoritmo GSP: Fase di Pruning dei Candidati (C_4) 257

14 Serie Temporali: Similarità, Distanze, Trasformazioni e Clustering 261
14.1 Introduzione e Fondamenti delle Serie Temporali 261
14.2 Metriche di Similarità e Trasformazioni di Segnale 265
14.3 Dynamic Time Warping (DTW) 267
14.4 Studio di Caso ed Esercizi Analitici su DTW 270
14.5 Valutazione Sperimentale e Limitazioni Computazionali 275
14.6 Similarità Strutturali e Modellistiche 277
14.7 Tecniche di Approssimazione e Riduzione della Dimensionalità 279
14.8 Clustering di Serie Temporali 282

15 Serie Temporali: Motif Discovery e Matrix Profile 285
15.1 Introduzione al Motif Discovery nelle Serie Temporali 285
15.2 La Struttura Dati Matrix Profile: Fondamenti e Componenti . 288
15.3 Interpretazione della Matrix Profile: Motifs e Discords 294
15.4 Algoritmo di Calcolo della Matrix Profile e Complessità 296
15.5 Estrazione dei Motifs e Geometria nello Spazio m-Dimensionale 300
15.6 Quadro Sinottico e Riferimenti Bibliografici 303

16 Laboratorio Python: Analisi, Trasformazioni e Mining di Serie Temporali 306
16.1 Ambiente di Lavoro, Stack Software e Dataset del Laboratorio 306
16.2 Trasformazioni, Preprocessing e Scaling delle Sequenze 309
16.3 Metriche di Distanza ed Allineamento Elastico (DTW) 311
16.4 Rappresentazioni Discrete: PAA, SAX e 1D-SAX 315
16.5 Clustering di Serie Temporali: Paradigmi a Confronto 316
16.6 Motif Discovery ed Anomaly Detection con Matrix Profile . . 318
16.7 Quadro Comparativo delle Librerie e Sintesi Operativa 320

17 Laboratorio di Pattern Mining e Social Network Analysis in Python 322
17.1 Introduzione e Obiettivi del Laboratorio 322
17.2 Parte I: Frequent Pattern Mining e Regole di Associazione . . 323
17.3 Parte II: Network Science e Social Network Analysis 332
17.4 Quadro Sinottico e Buone Pratiche Metodologiche 340

18 Laboratorio di Clustering per Dati Eterogenei e Multi-View 342
18.1 Introduzione e Obiettivi del Laboratorio 342
18.2 Parte I: Clustering di Dati Eterogenei (Categorici e Misti) . . 343
18.3 Parte II: Il Paradigma Multi-View e Benchmark UCI Multiple Features 347
18.4 Parte III: Strategie di Fusione: Early vs Late Fusion 349
18.5 Parte IV: Joint Multi-View Clustering (Apprendimento Congiunto) 352
18.6 Parte V: Valutazione Sperimentale e Quadro Sinottico 354

19 Laboratorio di Topic Modeling con BERTopic 357
19.1 Introduzione e Obiettivi del Laboratorio 357
19.2 Dal Bag-of-Words ai Sentence Transformers 358
19.3 Riduzione Dimensionalità con UMAP 360
19.4 Clustering Gerarchico basato su Densità con HDBSCAN . . . 362
19.5 Rappresentazione Lessicale con Class-based TF-IDF 365
19.6 La Pipeline Completa con BERTopic 366
19.7 Raffinamento Avanzato: Topic Reduction e KeyBERT 368
19.8 Quadro Sinottico e Buone Pratiche Metodologiche 369

20 Classificazione: Concetti di Base e Alberi di Decisione 371
20.1 Introduzione e Fondamenti della Classificazione 371
20.2 Il Modello ad Albero di Decisione 375
20.3 Induzione degli Alberi: L'Algoritmo di Hunt 377
20.4 Specificazione delle Condizioni di Test 379
20.5 Misure di Impurità e Selezione dello Split Ottimale 382

21 Alberi di Decisione Avanzati: Complessità e Potatura 388
21.1 Introduzione e Motivazione 388
21.2 Criteri di Arresto, Vantaggi e Complessità Computazionale . . 388
21.3 Trattamento Rigoroso dei Valori Mancanti (*Missing Values*) . 391
21.4 Limiti Strutturali degli Alberi di Decisione 394
21.5 Teoria dell'Errore, Underfitting e Overfitting 396
21.6 Selezione del Modello e Stima dell'Errore di Generalizzazione . 399
21.7 Tecniche di Potatura (*Pruning*) 402

22 Valutazione dei Modelli di Classificazione 405
22.1 Introduzione e Metriche Fondamentali di Prestazione 405
22.2 Metodologie di Partizionamento e Stima Affidabile 410
22.3 Confronto tra Modelli e Analisi ROC 414
22.4 Test di Significatività Statistica e Confronto tra Modelli 418

23 Metodi Ensemble, Reti Neurali Artificiali e Backpropagation 422
23.1 Architettura Generale e Fondamenti Teorici degli Ensemble . . 422
23.2 Bagging (*Bootstrap Aggregating*) 426
23.3 Boosting e l'Algoritmo AdaBoost 429
23.4 Random Forests 432
23.5 Ispirazione Biologica, Neurone Artificiale e Percettrone 434
23.6 Reti Neurali Multistrato e Funzioni di Attivazione 439
23.7 Addestramento delle Reti: Discesa del Gradiente e Backpropagation 440
23.8 Esercitazione Completa: Backpropagation Passo-Passo 442

24 Laboratorio di Machine Learning: Codifica, Alberi di Decisione, Dati Sbilanciati e Generazione Sintetica 449
24.1 Introduzione e Motivazione 449
24.2 Parte I: Laboratorio Empirico su Dataset Tabellare Adult . . . 451
24.3 Parte II: Trattamento dei Dati Sbilanciati (Imbalanced Data) 456
24.4 Parte III: Modelli Generativi per Dati Sintetici Tabellari . . . 464
24.5 Riepilogo e Conclusioni 471

Bibliografia 472

Capitolo 1

Introduzione al Data Mining e il Processo KDD

1.1 Fondamenti, Motivazioni e Definizione Formale

1.1.1 Che cos'è il Data Mining?

Il **Data Mining** (estrazione di conoscenza dai dati) è l'applicazione di algoritmi e tecniche computazionali efficienti per l'analisi di collezioni di dati di grandi dimensioni, al fine di scoprire pattern, regolarità, correlazioni e strutture informative finora sconosciute, non banali, valide e potenzialmente utili.

Dal punto di vista concettuale, il Data Mining non si limita alla semplice esecuzione di query su una base di dati (come avverrebbe in un sistema relazionale tramite SQL), né si riduce al calcolo di indicatori statistici descrittivi elementari. L'obiettivo centrale è l'estrazione di **conoscenza nascosta** (*hidden knowledge*): modelli impliciti, strutture di dipendenza latenti e relazioni predittive che non emergerebbero da un'ispezione manuale o tramite analisi tradizionali.

Definizione 1.1.1 (Data Mining)**.** Il **Data Mining** è il processo computazionale di scoperta automatica o semi-automatica di pattern, regolarità, correlazioni o strutture latenti all'interno di moli massive di dati (*Big Data*), producendo modelli validi, inediti, potenzialmente utili e comprensibili per il decisore umano.

1.1.2 Il Contesto Applicativo: Driver Commerciali e Scientifici

L'esigenza di sviluppare discipline computazionali dedicate all'estrazione di pattern nasce dalla crescita esponenziale della quantità di dati generati, archiviati e gestiti a livello globale:

- **Punto di vista commerciale:** Le transazioni e-commerce, i log di navigazione web, le registrazioni di acquisto della grande distribuzione organizzata (GDO) e i flussi finanziari (pagamenti tramite carte di credito, bonifici, transazioni POS) generano flussi continui di dati nell'ordine dei terabyte o petabyte al giorno. Piattaforme digitali e istituti bancari non possono trarre vantaggio strategico dal semplice stoccaggio passivo; necessitano di trasformare questi log in insight comportamentali per ottimizzare il *Customer Relationship Management* (CRM), personalizzare offerte, prevenire l'abbandono dei clienti (*churn analysis*) e contrastare le frodi finanziarie in tempo reale.

- **Punto di vista scientifico:** La sensoristica remota e la strumentazione moderna producono moli di dati senza precedenti. Sistemi satellitari come il programma *NASA EOSDIS* riversano nei database petabyte di misurazioni ambientali su base annua; le telescopie astronomiche digitali mappano l'universo scansionando miliardi di corpi celesti; la genomica genera sequenze di nucleotidi ad altissima frequenza; simulazioni fisiche, climatiche e biologiche computazionali producono terabyte di output in poche ore. In tale contesto, l'analisi empirica e l'elaborazione di ipotesi scientifiche richiedono processi di automazione avanzata capaci di sintetizzare matrici multidimensionali complesse.

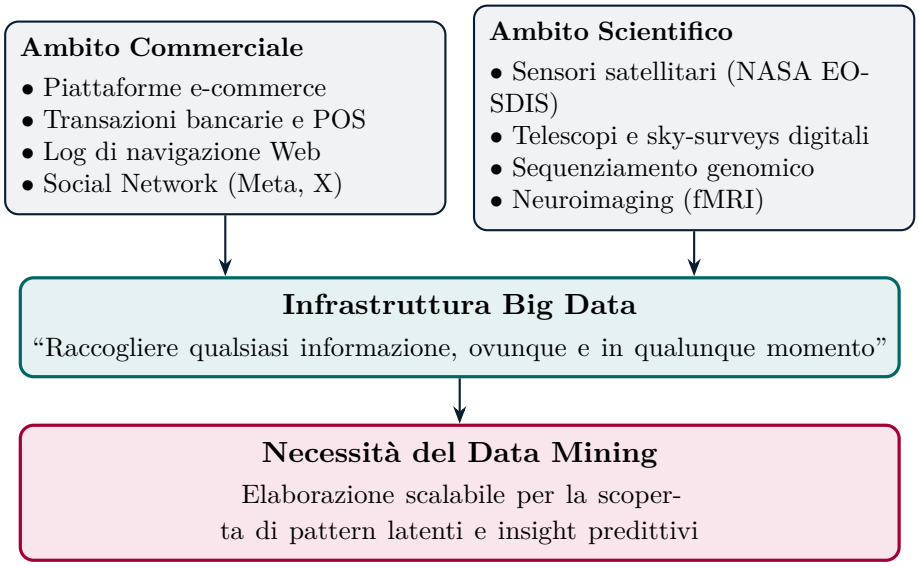


Figura 1.1: Genesi ed evoluzione della necessità del Data Mining tra contesto commerciale e scientifico.

1.1.3 Big Data come Proxy della Vita Sociale e Sfide Globali

Le tracce digitali generate da dispositivi connessi, terminali mobili e ambienti interattivi costituiscono a tutti gli effetti un *proxy quantitativo* dei comportamenti e delle interazioni umane:

1. **Pattern di acquisto e stili di vita:** Lo scontrino della spesa, i registri fedeltà e il carrello virtuale tracciano le preferenze alimentari, le risposte ai prezzi e i comportamenti economici.

2. **Relazioni e reti sociali:** I grafi di interazione ricavati dalle piattaforme social descrivono la topologia dei contatti interpersonali e le modalità di propagazione delle informazioni e delle opinioni.

3. **Desideri, opinioni e sentimenti:** Le query immesse nei motori di ricerca, i post testuali e i contributi a enciclopedie aperte (es. Wikipedia) forniscono una misura in tempo reale degli interessi e dello stato emotivo collettivo.

4. **Mobilità e spostamenti:** I dati GPS, le connessioni alle celle telefoniche e i flussi del traffico aereo permettono di ricostruire la mobilità sia a livello locale (analisi della congestione urbana) sia globale (mappatura

delle rotte e previsione della diffusione di patogeni pandemici, come studiato tramite piattaforme quali *BioDiaspora*).

Questi strumenti offrono l'opportunità di affrontare problemi complessi di rilevanza sociale: ottimizzazione dei sistemi sanitari, transizione verso fonti di energia rinnovabile tramite reti elettriche intelligenti (*smart grid*), previsione degli effetti del cambiamento climatico e pianificazione della produzione agricola su scala globale.

1.2 La Confluenza di Discipline e il Processo KDD

1.2.1 Data Mining: Confluenza Multidisciplinare

Il Data Mining è un'area di ricerca e applicazione intrinsecamente interdisciplinare. Si colloca all'incrocio tra diversi settori:

- **Sistemi di Basi di Dati (Database Systems):** Forniscono l'infrastruttura per l'archiviazione, l'indicizzazione, l'accesso efficiente e il recupero dei dati su memoria secondaria e distribuita.
- **Statistica (Statistics):** Fornisce i fondamenti formali per la quantificazione dell'incertezza, l'inferenza, il campionamento, la verifica delle ipotesi (*hypothesis testing*) e la modellazione probabilistica.
- **Machine Learning (Apprendimento Automatico):** Introduce architetture computazionali e algoritmi ottimizzati capaci di apprendere regole direttamente dai dati per generalizzare su istanze mai osservate.
- **Teoria degli Algoritmi (Algorithms):** Garantisce che le procedure di calcolo siano computazionalmente sostenibili dal punto di vista della complessità spaziale e temporale, gestendo dataset con milioni di righe e migliaia di colonne.
- **Visualizzazione (Visualization):** Permette l'esplorazione percettiva e intuitiva dei dati e la comunicazione dei risultati all'analista o all'esperto di dominio.

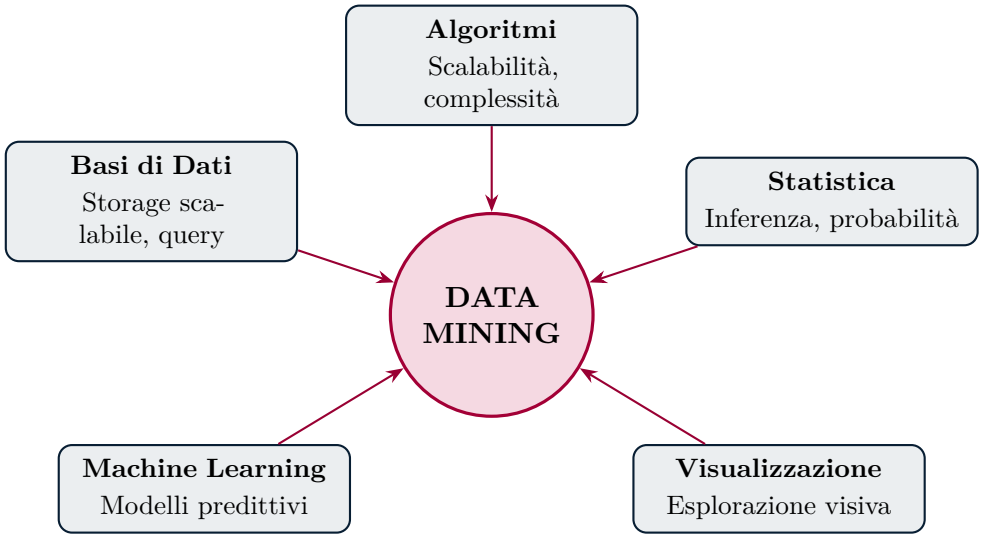


Figura 1.2: La natura multidisciplinare del Data Mining alla confluenza di discipline informatiche e matematiche.

1.2.2 Il Processo KDD (Knowledge Discovery in Databases)

Il Data Mining non costituisce un'attività isolata, bensì la fase algoritmica cardine di un processo iterativo, ciclico ed esteso noto come **KDD** (*Knowledge Discovery in Databases*). L'intero ciclo di vita del dato si articola in sei fasi sequenziali con cicli di retroazione:

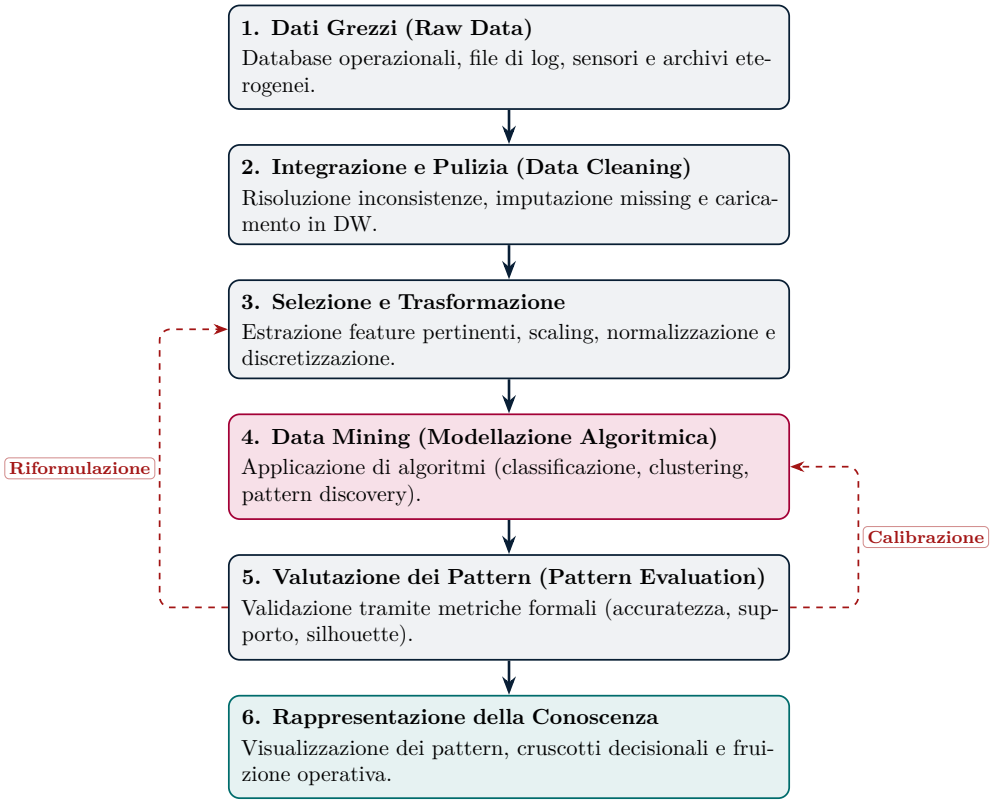


Figura 1.3: Architettura e passaggi sequenziali del processo KDD con percorsi di raffinamento iterativo.

Dettaglio delle Fasi del Processo KDD:

1. **Comprensione del Business / Obiettivo (*Business Understanding*):** Formalizzazione del problema aziendale o scientifico, definizione degli indicatori di prestazione (KPI) e comprensione dei vincoli operativi.

2. **Integrazione e Pulizia dei Dati (*Data Integration & Cleaning*):** Le sorgenti informative originali, spesso frammentate, incomplete e piene di rumore, vengono conciliate, ripulite dagli errori di sintassi e caricate in un repository integrato (spesso un *Data Warehouse* strutturato per query analitiche complesse).

3. **Selezione e Trasformazione (*Selection & Transformation*):** Estrazione del sottoinsieme di record e attributi pertinenti al target (*Task-relevant Data*); normalizzazione, aggregazione e discretizzazione delle feature.

4. **Data Mining:** Selezione e calibrazione degli algoritmi analitici appropriati (classificazione, clustering, regole associative, rilevamento anomalie) per estrarre regolarità strutturali dai dati.

5. **Valutazione dei Pattern (*Pattern Evaluation*):** Validazione dei risultati estratti sulla base di metriche formali (accuratezza, precisione, supporto, confidenza, silhouette score) e valutazione dell'effettiva utilità o novità per il contesto reale.

6. **Rappresentazione della Conoscenza (*Knowledge Representation*):** Sintesi e restituzione dei pattern validati tramite report, grafici e cruscotti decisionali, pronti all'uso per i decisori umani o all'integrazione in pipeline automatizzate.

1.3 Tassonomia dei Task di Data Mining

I task di Data Mining si dividono in due macrocategorie principali:

1. **Metodi Predittivi (*Predictive Methods*):** Mirano a costruire un modello statistico/matematico a partire da un insieme di variabili indipendenti (predittori) al fine di prevedere il valore incognito, continuo o discreto, di una specifica variabile dipendente (target).

2. **Metodi Descrittivi (*Descriptive Methods*):** Hanno come scopo l'individuazione di schemi, associazioni, raggruppamenti e proprietà intrinseche ai dati, senza che esista a priori una variabile target definita; il modello serve a rendere il dato comprensibile all'utente umano.

1.3.1 Classificazione (Predictive Modeling)

La classificazione assegna un record $x \in \mathcal{X}$ a una di un numero finito di classi predeterminate $y \in \mathcal{Y} = \{c_1, c_2, \ldots, c_k\}$. L'algoritmo riceve un insieme di addestramento (*Training Set*) formato da coppie ordinate:

$$\mathcal{D}_{\text{train}} = \{(x_1, y_1), (x_2, y_2), \ldots, (x_N, y_N)\} \tag{1.1}$$

L'obiettivo dell'apprendimento è derivare una funzione di classificazione (modello o classificatore):

$$f : \mathcal{X} \to \mathcal{Y} \tag{1.2}$$

che minimizzi l'errore di classificazione atteso su dati non visti (*Test Set*).

Esempio: Valutazione del Merito Creditizio (*Credit Worthiness*)
Si consideri il dataset per la concessione di prestiti bancari:

TID	**Occupato**	**Livello Istruzione**	**Anni Residenza**	**Merito Creditizio** (Y)
1	Sì	Laurea	5	**Sì**
2	Sì	Diploma	2	**No**
3	No	Triennale	1	**No**
4	Sì	Diploma	10	**Sì**

Tabella 1.1: Dataset giocattolo per l'induzione di un classificatore di merito creditizio.

Un algoritmo di apprendimento supervisionato induce un albero di decisione strutturato come segue:

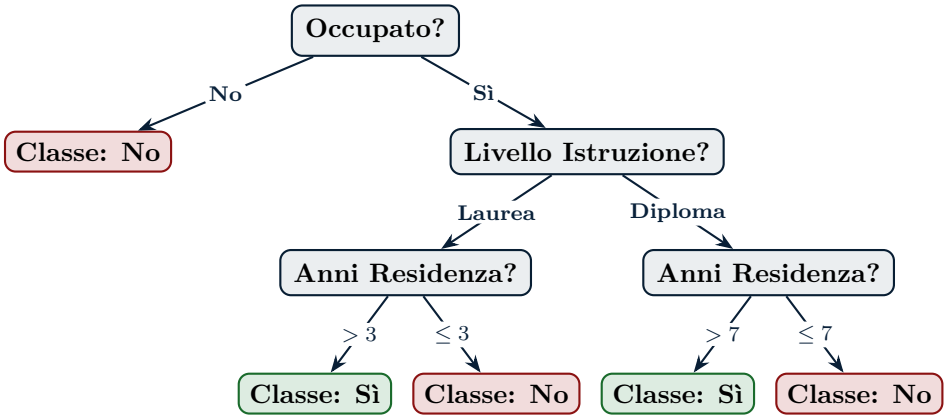


Figura 1.4: Albero di decisione dedotto dal dataset di merito creditizio.

Applicazioni Tipiche della Classificazione:

- *Credit Card Fraud Detection:* identificazione di transazioni fraudolente tramite feature quali importo, posizione, timestamp e frequenza.
- *Churn Prediction:* predizione della probabilità che un cliente passi alla concorrenza.
- *Classificazione Biomedica:* distinzione istologica tra neoplasie benigne e maligne a partire da biopsie.
- *Categorizzazione Testuale:* smistamento automatico di documenti e notizie in categorie tematiche.

1.3.2 Raggruppamento (Clustering)

Il clustering consiste nel ripartire un insieme di oggetti non etichettati $\mathcal{D} = \{x_1, x_2, \ldots, x_N\}$ in un numero k di gruppi o partizioni disgiunte $\mathcal{C} = \{C_1, C_2, \ldots, C_k\}$ tali che:

1. **Distanza intra-cluster minimizzata:** Gli oggetti appartenenti allo stesso cluster C_i presentano un'elevata somiglianza reciproca.

2. **Distanza inter-cluster massimizzata:** Gli oggetti collocati in cluster distinti C_i e C_j $(i \neq j)$ sono tra loro eterogenei o geometricamente distanti nello spazio delle metriche.



Figura 1.5: Rappresentazione concettuale del clustering: omogeneità interna e separazione esterna.

1.3.3 Estrazione di Regole di Associazione (Association Rules)

L'estrazione di regole associative individua dipendenze tra insiemi di elementi presenti all'interno di un database transazionale. Sia $\mathcal{I} = \{i_1, i_2, \ldots, i_m\}$ l'insieme degli item e $\mathcal{T} = \{t_1, t_2, \ldots, t_n\}$ l'insieme delle transazioni, dove ogni transazione $t_j \subseteq \mathcal{I}$.

Una **regola di associazione** è un'espressione logica del tipo:

$$X \implies Y \quad \text{con } X, Y \subset \mathcal{I} \text{ e } X \cap Y = \emptyset \tag{1.3}$$

Il significato formale indica che, nei record in cui compare l'insieme di elementi X (antecedente), si osserva sistematicamente anche la presenza dell'insieme Y (conseguente).

TID	Items Acquistati
1	Pane, Coca-Cola, Latte
2	Birra, Pane
3	Birra, Coca-Cola, Pannolini, Latte
4	Birra, Pane, Pannolini, Latte
5	Coca-Cola, Pannolini, Latte

Tabella 1.2: Esempio di database transazionale per Market Basket Analysis.

Dall'analisi delle co-occorrenze si estraggono regole come:

$$\{\text{Latte}\} \implies \{\text{Coca-Cola}\}, \qquad \{\text{Pannolini}, \text{Latte}\} \implies \{\text{Birra}\} \tag{1.4}$$

1.3.4 Rilevamento delle Anomalie (Anomaly Detection)

L'identificazione di anomalie consiste nell'individuare oggetti, pattern o sequenze temporali che si discostano marcatamente dalla distribuzione o dal comportamento tipico del resto del dataset. I campi applicativi includono la rilevazione di intrusioni su reti di calcolatori (*Network Intrusion Detection*), il blocco di frodi su carte di credito, l'individuazione di avarie nei sensori industriali e il monitoraggio del disboscamento tramite serie temporali satellitari.

1.4 Sfide Aperte del Data Mining Moderno

L'impiego dei metodi tradizionali di statistica e machine learning su dataset complessi incontra vincoli sistemici:

1. **Scalabilità (Scalability):** Molti algoritmi classici hanno una complessità computazionale non lineare ($\mathcal{O}(N^2)$ o $\mathcal{O}(N^3)$ rispetto al numero di campioni N), rendendoli impraticabili su insiemi di miliardi di record senza parallelizzazione o approssimazioni.

2. **Alta Dimensionalità (High Dimensionality):** L'incremento del numero di attributi p genera problemi geometrici legati alla densità dello spazio (la cosiddetta *maledizione della dimensionalità*), che rende le distanze uniformi e disperde i dati nello spazio.

3. **Eterogeneità e Complessità del Dato:** I dati aziendali e scientifici non sono composti unicamente da tabelle piatte, ma includono grafi, serie temporali, testi liberi, sequenze biologiche, dati georeferenziati e flussi continui in streaming (*data streams*).

4. **Distribuzione e Proprietà dei Dati:** I dati sono spesso distribuiti su nodi remoti appartenenti a organizzazioni diverse, con vincoli legali di privacy, segreto industriale e disponibilità limitata di banda.

5. **Analisi Non Tradizionali:** Esigenza di gestire distribuzioni sbilanciate, derive dei concetti nel tempo (*concept drift*) e flussi informativi asincroni ad altissima frequenza.

Capitolo 2

Data Understanding I: Tipologie di Dati, Attributi e Statistica Descrittiva

2.1 Conoscere i Dati e Tassonomia dei Dataset

Il **Data Understanding** è la fase in cui l'analista esamina a fondo i dati a disposizione per determinare la qualità delle misurazioni, comprendere il significato applicativo delle feature e individuare preliminarmente la presenza di rumore, outlier o valori mancanti.

Obiettivi Fondamentali del Data Understanding

- Quali tipologie di attributi compongono il dataset?
- Qual è la qualità generale del dato e quali errori sintattici/semantici sono presenti?
- Sono presenti outlier o misurazioni anomale univariata e multivariata?
- Esistono correlazioni e dipendenze tra coppie di attributi?
- Come si distribuiscono i valori mancanti e quale meccanismo li genera?
- È necessario trasformare o ingegnerizzare nuove feature prima del mining?

2.1.1 Struttura Fondamentale: Oggetti e Attributi

Un dataset è una collezione formale di **oggetti di dati** (noti anche come *record, punti, campioni, entità, istanze o casi*) descritti da un insieme di **attributi** (noti anche come *variabili, feature, campi, caratteristiche o dimensioni*).

- Un **oggetto** rappresenta una singola entità del mondo reale (un cliente, una transazione, una lettura di un sensore).
- Un **attributo** denota una proprietà o caratteristica misurabile di tale oggetto (l'altezza, il prezzo, la temperatura, la categoria).

2.1.2 Classificazione delle Strutture Dati

I dataset si differenziano per la loro conformazione strutturale:

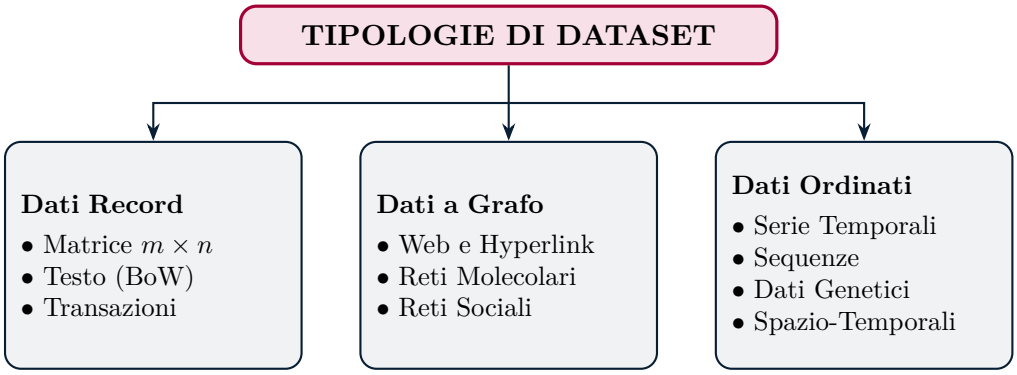


Figura 2.1: Tassonomia strutturale delle macro-tipologie di dataset.

1. Dati di tipo Record

- **Matrice Dati (*Data Matrix*):** Se tutti gli m oggetti sono descritti dallo stesso insieme fisso di n attributi numerici continui, il dataset si modella come una matrice di dimensione $m \times n$:

$$D = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1n} \\ x_{21} & x_{22} & \cdots & x_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ x_{m1} & x_{m2} & \cdots & x_{mn} \end{bmatrix} \in \mathbb{R}^{m \times n} \tag{2.1}$$

  Ogni riga i corrisponde a un punto immerso nello spazio vettoriale continuo $\mathbb{R}^n$, dove ogni colonna rappresenta un asse coordinato.

- **Dati di tipo Documento (*Document-Term Matrix*):** Ogni testo viene rappresentato con il modello vettoriale *Bag-of-Words*. Le colonne corrispondono al vocabolario dei termini distinti e i valori registrano la frequenza:

Documento	team	coach	play	ball	score	game	win	lost	timeout	season
Doc 1	3	0	5	0	2	6	0	2	0	2
Doc 2	0	7	0	2	1	0	0	3	0	0
Doc 3	0	1	0	0	1	2	2	0	3	0

Tabella 2.1: Rappresentazione Document-Term Matrix per testi non strutturati.

- **Dati Transazionali (*Transaction Data*):** Ogni riga è identificata da un TID ed è formata da un insieme a cardinalità variabile di item discreti (articoli del carrello della spesa).

2. Dati a Grafo (*Graph Data*)

Modellano esplicitamente le relazioni tra entità: la topologia del Web (nodi = pagine, archi = collegamenti ipertestuali), molecole chimiche (es. benzene C_6H_6 con atomi e legami covalenti) e grafi di citazione bibliografica.

3. Dati Ordinati (*Ordered Data*)

Le relazioni di ordinamento determinano il significato del dato:

- **Serie Storiche:** Misurazioni indicizzate dal tempo con forte autocorrelazione.
- **Dati Sequenziali:** Sequenze ordinate di eventi (es. carrelli temporali: $(A, B) \rightarrow (D) \rightarrow (C, E)$).
- **Sequenze Genomiche:** Catene di basi nucleotidiche ('A', 'C', 'G', 'T').
- **Dati Spazio-Temporali:** Misurazioni indicizzate congiuntamente da coordinate geografiche e tempo (es. mappe globali delle temperature marine e terrestri).

2.2 Tipologie di Attributi e Proprietà di Scala

Le proprietà matematiche di un attributo dipendono dalle operazioni formali lecite sui valori del suo dominio:

- **Distinzione ($=$ o $\neq$):** Confronto di uguaglianza/disuguaglianza tra valori.
- **Ordinamento ($<$ o $>$):** Relazione d'ordine naturale totale o parziale.
- **Differenza ($+$ o $-$):** Intervalli quantificabili con metrica costante.
- **Rapporto ($*$ o $/$):** Zero assoluto naturale (totale assenza della quantità misurata).

Scala	Dist.	Ord.	Diff.	Rapp.	Operatori Statistici Ammessi
Nominale	✓	–	–	–	Moda, entropia, test χ^2
Ordinale	✓	✓	–	–	Mediana, quantili, Spearman ρ
Intervallo	✓	✓	✓	–	Media, dev.std, correlazione Pearson
Rapporto	✓	✓	✓	✓	Media geometrica, coeff. variazione

Tabella 2.2: Gerarchia e proprietà formali delle scale di misurazione statistica.

2.2.1 Attributi Categorici (Qualitativi)

1. **Nominali:** Permettono solo il test di uguaglianza ($=, \neq$). Esempi: colore degli occhi, CAP, codice fiscale, ID.

2. **Binari:** Casi particolari di attributi nominali con esattamente due stati $\{0, 1\}$.

   - *Binari Simmetrici:* Entrambi gli esiti hanno pari importanza (es. genere Maschio/Femmina).
   - *Binari Asimmetrici:* Uno dei due esiti è raro o di interesse primario (1 = positivo a malattia rara, 0 = negativo).

3. **Ordinali:** Possiedono un ordinamento intrinseco ($<, >$), ma l'intervallo tra valori contigui non è costante numericamente. Esempi: scala di durezza di Mohs, giudizi $\{\text{scarso}, \text{medio}, \text{buono}\}$, voti in lettere $\{A, B, C, D, F\}$.

2.2.2 Attributi Numerici (Quantitativi)

1. **A Intervallo (*Interval-Scaled*):** La differenza tra due valori è costante, ma la scala **non possiede uno zero assoluto**. Esempi: temperatura in gradi Celsius o Fahrenheit (20°C non è "il doppio" di 10°C, poiché in Kelvin $293.15\,\text{K} \neq 2 \cdot 283.15\,\text{K}$); date di calendario.

2. **A Rapporto (*Ratio-Scaled*):** Possiedono uno **zero assoluto naturale** indicante l'assenza della grandezza. Esempi: lunghezza, peso, età, temperatura in Kelvin (0 K assenza di energia termica), importi monetari, tempo trascorso. Ha senso affermare che una partita durata 3 ore è il 50% più lunga di una durata 2 ore ($3/2 = 1.5$).

2.2.3 Variabili Discrete vs Continue

- **Attributo Discreto:** Possiede un insieme finito o numerabile di valori possibili, tipicamente rappresentato da numeri interi (CAP, conteggi di frequenza, parole).
- **Attributo Continuo:** Possiede un dominio coincidente con i numeri reali $\mathbb{R}$, rappresentato in virgola mobile.

2.3 Problemi di Qualità del Dato

L'impiego di dati degradati inficia qualunque algoritmo analitico (principio *Garbage In, Garbage Out*). Studi industriali (Redman, 2004) evidenziano che la scarsa qualità incide per il $10\% \div 20\%$ del fatturato aziendale complessivo.



Figura 2.2: Dimensioni fondamentali della qualità dei dati.

2.4 Ispezione delle Distribuzioni Monovariate

2.4.1 Asimmetria e Multimodalità

La forma della densità di frequenza di un attributo continuo rivela le proprietà intrinseche del fenomeno indagato:

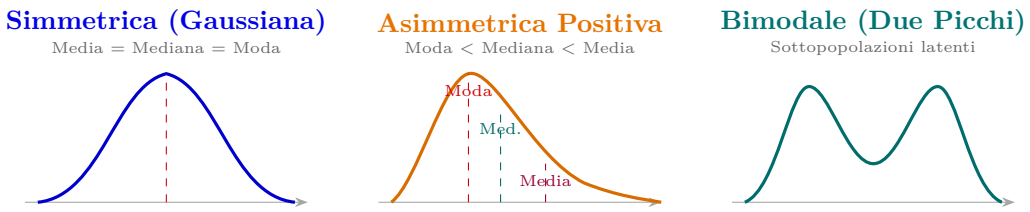


Figura 2.3: Forme caratteristiche delle distribuzioni univariata: simmetria, asimmetria e multimodalità.

- **Distribuzione Simmetrica:** Densità speculare attorno al centro. Coincidono Media = Mediana = Moda.
- **Asimmetrica Positiva (*Right-Skewed*):** Coda allungata verso valori alti (es. reddito pro capite). Pochi valori elevati spostano la media:

$$\text{Moda} < \text{Mediana} < \text{Media} \tag{2.2}$$

- **Asimmetrica Negativa (*Left-Skewed*):** Coda allungata verso valori bassi:

$$\text{Media} < \text{Mediana} < \text{Moda} \tag{2.3}$$

- **Bimodale o Multimodale:** Segnala la sovrapposizione di sottopopolazioni disgiunte non ancora separate da variabili latenti (es. peso corporeo aggregato senza distinzione di genere).

2.4.2 Strumenti Visuali: Bar Chart vs Istogrammi

- **Grafico a Barre (*Bar Chart*):** Utilizzato per variabili **categoriche/discrete**. Le barre sono visivamente separate per evidenziare l'assenza di continuità metrica sull'asse orizzontale.
- **Istogramma (*Histogram*):** Utilizzato per variabili **continue**. Il dominio viene partizionato in intervalli contigui (*bin*) e l'area di ciascun rettangolo è proporzionale al numero di osservazioni.

Scelta del Numero di Bin e Regola di Sturges: Un numero insufficiente di bin appiattisce la distribuzione nascondendo la bimodalità; un numero eccessivo introduce oscillazioni casuali. Per distribuzioni approssimativamente normali si adotta la **Regola di Sturges**:

$$k = \lceil \log_2(n) + 1 \rceil \tag{2.4}$$

dove n è la dimensione campionaria e $\lceil \cdot \rceil$ è l'arrotondamento all'intero superiore.

2.5 Misure di Tendenza Centrale e Dispersione

2.5.1 Misure di Tendenza Centrale

- **Media Aritmetica Campionaria:**

$$\bar{x} = \frac{1}{m} \sum_{i=1}^{m} x_i \tag{2.5}$$

  È estremamente sensibile agli outlier: una singola osservazione estrema falsa il valore medio.

- **Mediana:** Valore che occupa la posizione centrale nel campione ordinato:

$$\text{mediana} = \begin{cases} x_{\left(\frac{m+1}{2}\right)} & \text{se } m \text{ è dispari} \\ \dfrac{x_{\left(\frac{m}{2}\right)} + x_{\left(\frac{m}{2}+1\right)}}{2} & \text{se } m \text{ è pari} \end{cases} \tag{2.6}$$

  È una misura **robusta**, invariante rispetto all'entità numerica dei valori estremi.

- **Moda:** Il valore a frequenza empirica massima. È l'unica misura applicabile anche ad attributi nominali.

- **Media Troncata (*Trimmed Mean*):** Calcolata escludendo una percentuale α di campioni minimi e massimi per combinare efficienza e robustezza.

2.5.2 Misure di Dispersione dei Dati

- **Range (Campo di Variazione):** Range $= x_{\max} - x_{\min}$.

- **Varianza Campionaria (s^2):**

$$s^2 = \frac{1}{m-1} \sum_{i=1}^{m} (x_i - \bar{x})^2 \tag{2.7}$$

  con correzione di Bessel a denominatore $(m-1)$ per garantire uno stimatore non distorto.

- **Deviazione Standard Campionaria (s):** $s = \sqrt{s^2}$, espressa nella medesima unità di misura della variabile originaria.

Misure di Dispersione Robuste:

- **Deviazione Assoluta Media (*Absolute Average Deviation - AAD*):**

$$\mathrm{AAD}(x) = \frac{1}{m} \sum_{i=1}^{m} |x_i - \bar{x}| \tag{2.8}$$

- **Mediana degli Scostamenti Assoluti (*Median Absolute Deviation - MAD*):**

$$\mathrm{MAD}(x) = \text{mediana} \left(\left\{ |x_i - \text{mediana}(x)| \right\}_{i=1}^{m} \right) \tag{2.9}$$

2.6 Five-Number Summary, Quartili e Box Plot

2.6.1 Il Five-Number Summary

Il **Five-Number Summary** sintetizza la distribuzione statistica tramite cinque parametri posizionali:

$$\left\{ x_{\min},\, Q_1,\, \text{Mediana } (Q_2),\, Q_3,\, x_{\max} \right\} \tag{2.10}$$

- Q_1 (Primo Quartile): 25% delle osservazioni sono inferiori o uguali a esso.
- Q_2 (Mediana): 50% delle osservazioni.
- Q_3 (Terzo Quartile): 75% delle osservazioni.
- **Range Interquartilico (IQR):** $\mathrm{IQR} = Q_3 - Q_1$.

2.6.2 Anatomia del Box Plot

Il **Box Plot** traduce visivamente la sintesi a cinque numeri:

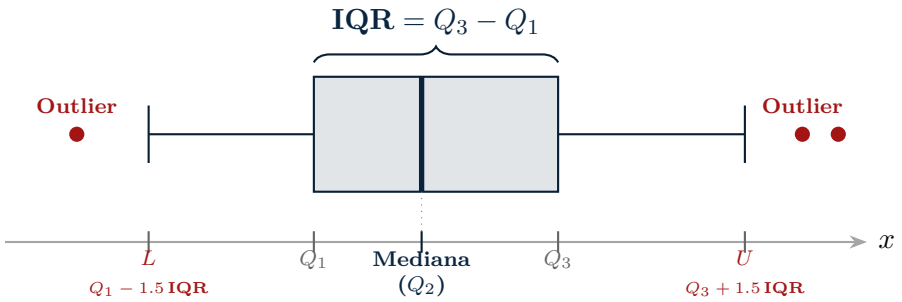


Figura 2.4: Anatomia formale del Box Plot con recinzioni di Tukey per l'identificazione degli outlier.

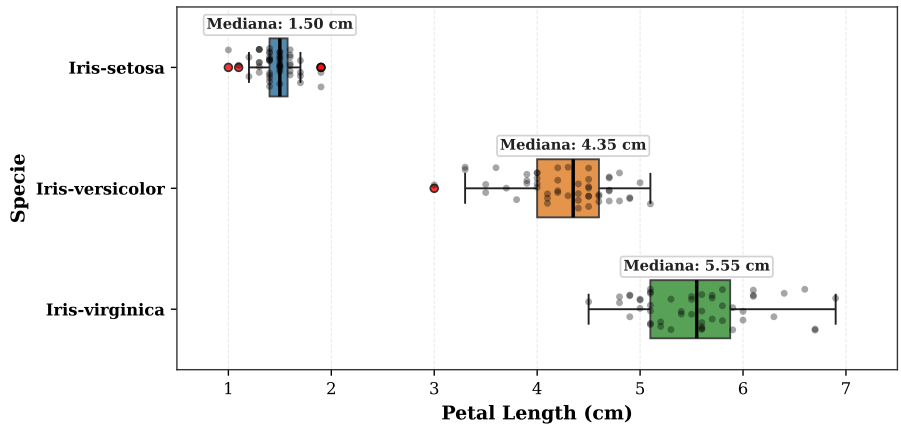


Figura 2.5: Box Plot condizionale calcolato sui dati reali del dataset Iris: distribuzione empirica della lunghezza del petalo (*Petal Length*) stratificata sulle tre specie, con evidenza della mediana, dei quartili e dei singoli campioni osservati.

Limiti del Box Plot rispetto agli Istogrammi: Sebbene il box plot sintetizzi efficacemente posizione e dispersione, **non permette di identificare la multimodalità**. Due distribuzioni, una marcatamente bimodale e una unimodale simmetrica, possono presentare lo stesso Five-Number Summary $(x_{\min}, Q_1, Q_2, Q_3, x_{\max})$ generando lo stesso identico box plot; solo l'istogramma consente di visualizzare la presenza di una concavità o di picchi multipli tra i quartili.

Capitolo 3

Data Understanding II – Analisi Multivariata, Correlazione e Rilevamento Outliers

3.1 Tecniche di Visualizzazione Multidimensionale

L'analisi multivariata studia le relazioni simultanee tra due o più attributi del dataset, superando i limiti delle descrizioni univariata isolate. Quando l'analisi si estende a configurazioni bivariate e multivariate, la rappresentazione grafica deve consentire la percezione immediata della separabilità geometrica tra classi, dell'addensamento in cluster naturali e della presenza di anomalie strutturali.

Il caso di studio impiegato come riferimento applicativo è il benchmark **Iris** (introdotto dal botanico Edgar Anderson e reso celebre da Ronald Fisher nel 1936), composto da 150 osservazioni distribuite in modo bilanciato su 3 classi da 50 campioni ciascuna (*Iris setosa*, *Iris versicolor*, *Iris virginica*) e descritto da 4 feature continue espresse in centimetri:

- Lunghezza del sepalo (*Sepal Length*)
- Larghezza del sepalo (*Sepal Width*)
- Lunghezza del petalo (*Petal Length*)
- Larghezza del petalo (*Petal Width*)

3.1.1 Scatter Plot e Scatter Matrix

Lo **Scatter Plot (Grafico a Dispersione)** traccia ogni coppia di attributi continui (X, Y) proiettando le osservazioni come punti con coordinate

cartesiane $(x_i, y_i) \in \mathbb{R}^2$.

- **Caratteristiche**: Permette di evidenziare visivamente cluster, tendenze di correlazione diretta o inversa e la presenza di outlier bivariati che risultano invisibili dall'analisi dei singoli istogrammi.
- **Codifica cromatico-categoriale**: Assegnando a ciascun punto un colore o un simbolo corrispondente alla classe di appartenenza, l'analista può valutare la separabilità geometrica indotta dalla coppia di feature.
- **Evidenza sul dataset Iris**: Incrociando *Petal Length* e *Petal Width*, le classi mostrano un elevato grado di separabilità: la classe *Setosa* forma un cluster compatto e isolato, mentre *Versicolor* e *Virginica* presentano solo una ridotta sovrapposizione marginale (Figura 3.1). Al contrario, il piano formato da *Sepal Length* e *Sepal Width* mostra una forte sovrapposizione tra *Versicolor* e *Virginica*.

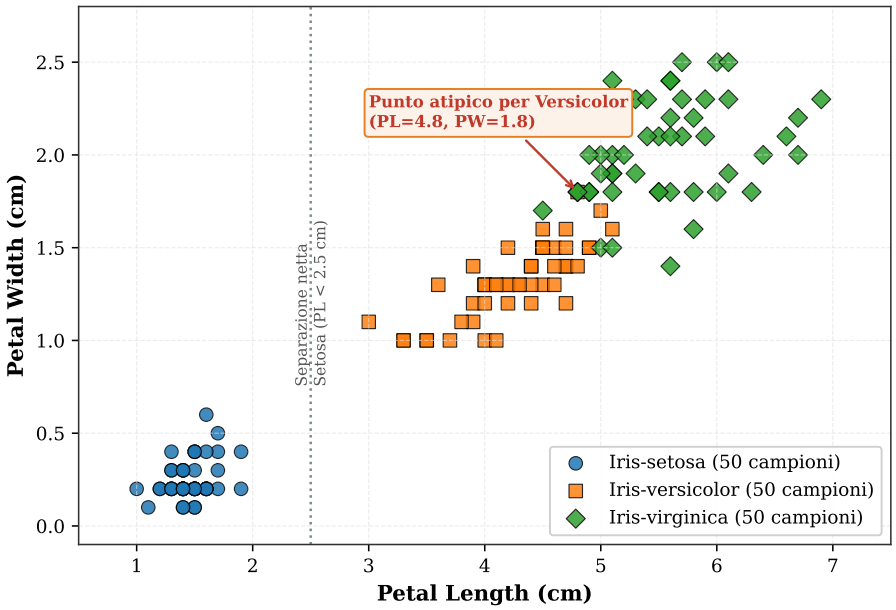


Figura 3.1: Scatter plot bivariati sul dataset Iris: confronto tra la separabilità generata da sepalo e petalo.

Quando il numero di attributi numerici p è moderato ($3 \leq p \leq 10$), si ricorre alla **Scatter Matrix (Scatter Plot Matrix - SPLOM)**: una griglia simmetrica di dimensione $p \times p$ contenente tutti i $\frac{p(p-1)}{2}$ grafici a dispersione a coppie distinte. Sulla diagonale principale vengono tipicamente inseriti gli

istogrammi univariati o le stime di densità (KDE) della singola variabile, consentendo una diagnosi simultanea di forma e correlazione (Figura 3.2).

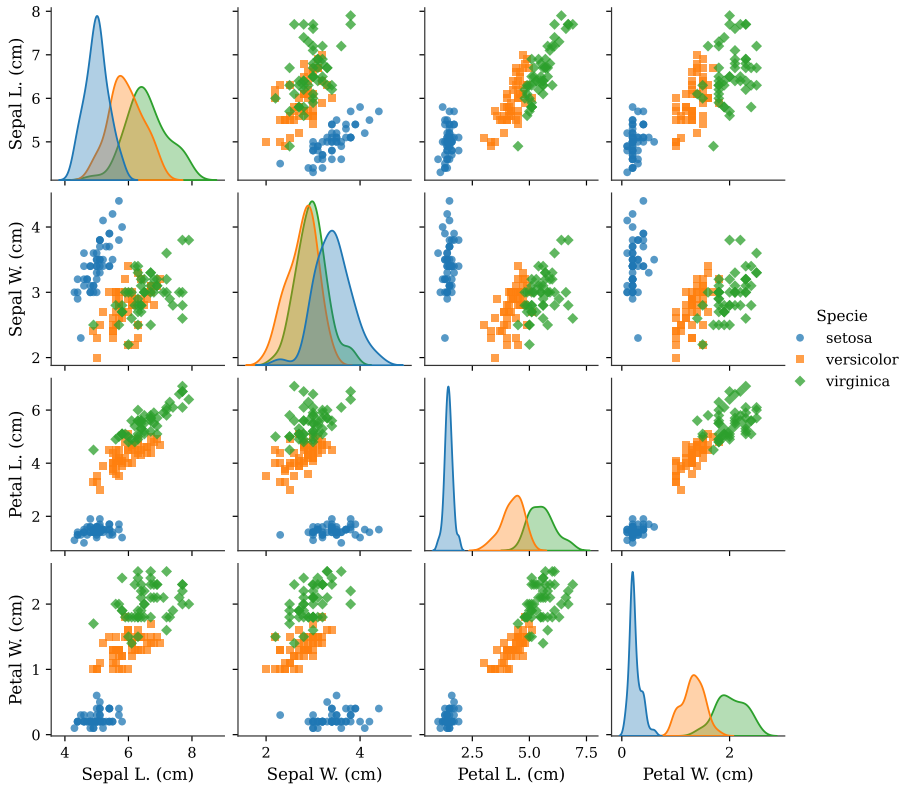


Figura 3.2: Scatter Plot Matrix (SPLOM) completa per le quattro feature del dataset Iris.

Lo **Scatter Plot Tridimensionale (3D)** estende la proiezione a tre coordinate cartesiane simultanee (X_1, X_2, X_3). Consente di identificare outlier tridimensionali non rilevabili nelle proiezioni bidimensionali marginali, sebbene la fruizione su supporto bidimensionale risenta delle occlusioni e dell'angolazione di visuale.

3.1.2 Visualizzazione di Spazi ad Alta Dimensionalità

Negli spazi a dimensione elevata ($p > 3$), la geometria euclidea cartesiana classica non consente proiezioni dirette esaustive. Si adottano schemi grafici alternativi basati su geometrie coordinate non ortogonali.

Coordinate Parallele (Parallel Coordinates)

Nelle coordinate parallele gli assi non sono mutuamente perpendicolari, ma disposti come p rette verticali equi-spaziate e parallele, ciascuna tarata sull'intervallo di valori dell'attributo corrispondente.

- **Mappatura geometrica**: Un'osservazione p-dimensionale, definita dal vettore $\mathbf{x}_i = (x_{i1}, \ldots, x_{ip}) \in \mathbb{R}^p$, viene rappresentata non come un punto, ma come una **polilinea (linea spezzata)** che attraversa tutti gli assi, intersecando l'asse j-esimo alla quota x_{ij}.
- **Proprietà diagnostiche**: I record appartenenti alla stessa classe formano fasci compatti con andamento coerente. Una polilinea isolata con traiettoria discordante segnala un potenziale outlier multivariato.
- **Rilevanza dell'ordinamento degli assi**: La percezione dei legami statistici dipende dall'ordinamento orizzontale degli assi. Collocando attributi fortemente correlati su assi contigui, fasci di segmenti paralleli indicano correlazione positiva, mentre fasci che si incrociano a X indicano correlazione negativa (Figura 3.3).

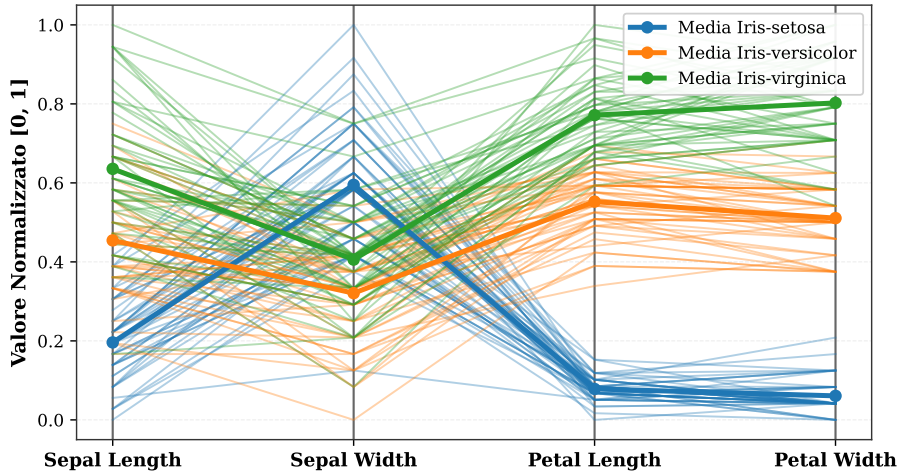


Figura 3.3: Coordinate parallele per il dataset Iris con curve differenziate per classe.

Radar Plot (Grafico Radar o Diagramma a Stella)

Il **Radar Plot** condivide il principio delle coordinate parallele, con la variante che gli assi dimensionali non sono disposti linearmente, ma **irradiano a**

raggiera da un polo centrale comune, distanziati da angoli equi-spaziati:

$$\theta_j = \frac{2\pi(j-1)}{p}, \quad j = 1, \ldots, p \tag{3.1}$$

Per ciascun record, il valore assunto dalla variabile definisce la distanza del vertice dal centro lungo il corrispondente raggio; congiungendo i vertici si genera un poligono chiuso. Quando il radar plot viene replicato in piccoli riquadri separati per ciascuna osservazione, prende il nome di **Star Plot**, agevolando il confronto visivo di profili complessi (Figura 3.4).

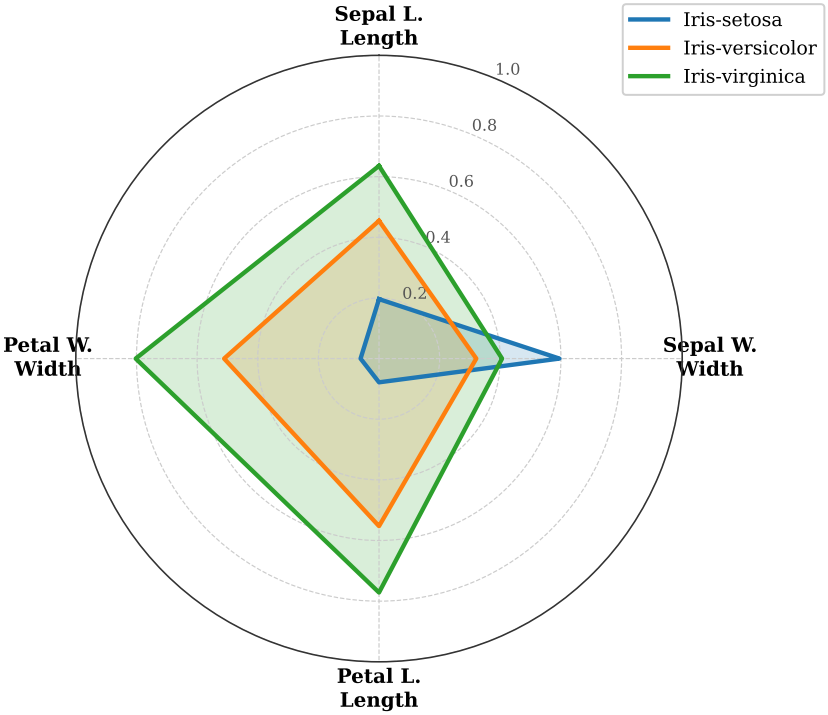


Figura 3.4: Profili medi delle classi del dataset Iris visualizzati tramite Radar Plot.

3.2 Analisi di Dipendenza Bivariata: Covarianza e Correlazione

L'analisi quantitativa della dipendenza studia l'intensità e la direzione con cui due variabili casuali variano congiuntamente.

3.2.1 Da Varianza a Covarianza

Mentre la varianza misura la dispersione di una singola variabile rispetto al proprio baricentro, la **Covarianza Campionaria** quantifica la tendenza di due variabili numeriche X e Y a manifestare scarti concordi o discordi rispetto alle rispettive medie:

$$\text{Cov}(X, Y) = s_{xy} = \frac{1}{n-1} \sum_{i=1}^{n} (x_i - \bar{x})(y_i - \bar{y}) \tag{3.2}$$

Nel caso speciale $X = Y$, la covarianza coincide con la varianza campionaria:

$$\text{Cov}(X, X) = \frac{1}{n-1} \sum_{i=1}^{n} (x_i - \bar{x})^2 = s_x^2 \tag{3.3}$$

Matrice di Covarianza

Per un insieme di p attributi continui, la struttura complessiva di variabilità congiunta è sintetizzata dalla **Matrice di Covarianza** $V \in \mathbb{R}^{p \times p}$:

$$V = \begin{bmatrix} \text{Cov}(X_1, X_1) & \text{Cov}(X_1, X_2) & \cdots & \text{Cov}(X_1, X_p) \\ \text{Cov}(X_2, X_1) & \text{Cov}(X_2, X_2) & \cdots & \text{Cov}(X_2, X_p) \\ \vdots & \vdots & \ddots & \vdots \\ \text{Cov}(X_p, X_1) & \text{Cov}(X_p, X_2) & \cdots & \text{Cov}(X_p, X_p) \end{bmatrix} \tag{3.4}$$

- Gli elementi diagonali $V_{jj} = \text{Var}(X_j) = s_j^2$ rappresentano le varianze marginali.
- Gli elementi extra-diagonali $V_{jk} = \text{Cov}(X_j, X_k)$ rappresentano le covarianze a coppie.
- Poiché $\text{Cov}(X_j, X_k) = \text{Cov}(X_k, X_j)$, la matrice V è **simmetrica** ($V = V^T$) e semi-definita positiva ($V \succeq 0$).

Interpretazione Geometrica dei Prodotti degli Scarti

Il segno della covarianza dipende dalla posizione delle osservazioni rispetto al baricentro $(\bar{x}, \bar{y})$ nel piano cartesiano, che divide lo spazio in quattro quadranti:

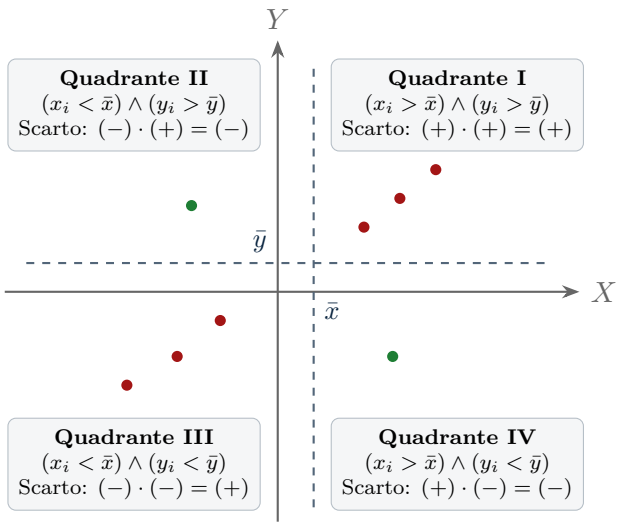


Figura 3.5: Scomposizione geometrica degli scarti in quattro quadranti centrati sul baricentro $(\bar{x}, \bar{y})$.

- Punti nei quadranti I e III apportano contributi positivi: le deviazioni sono concordi $\implies \text{Cov}(X, Y) > 0$.
- Punti nei quadranti II e IV apportano contributi negativi: le deviazioni sono discordi $\implies \text{Cov}(X, Y) < 0$.
- Se le osservazioni si disperdono uniformemente nei quattro quadranti, i termini positivi e negativi si bilanciano, portando a $\text{Cov}(X, Y) \approx 0$.

Limite Strutturale: Dipendenza dall'Unità di Misura

La covarianza non è invariante per trasformazioni di scala. Se la variabile X esprime un reddito in euro e viene convertita in migliaia di euro ($X' = X/1000$), la covarianza si riduce di un fattore 1000, nonostante la correlazione statistica rimanga immutata. Essa non consente quindi di confrontare coppie di attributi aventi ordini di grandezza o unità differenti.

3.2.2 Coefficiente di Correlazione Lineare di Pearson

Il **Coefficiente di Correlazione Lineare di Pearson** (r_{xy}) normalizza la covarianza dividendola per il prodotto delle deviazioni standard campionarie:

$$r_{xy} = \frac{\text{Cov}(X, Y)}{s_x s_y} = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2}\sqrt{\sum_{i=1}^{n}(y_i - \bar{y})^2}} \tag{3.5}$$

Proprietà Matematiche del Coefficiente r_{xy}

1. **Limitazione dell'intervallo**: Per la disuguaglianza di Cauchy-Schwarz nello spazio dei vettori centrati, il coefficiente è rigorosamente limitato nell'intervallo:
$$-1 \leq r_{xy} \leq +1 \tag{3.6}$$

2. **Invarianza di scala**: È un indice adimensionale, invariante rispetto a qualunque trasformazione lineare affine positiva ($X' = aX + b, Y' = cY + d$ con $a, c > 0$).

3. **Correlazione lineare perfetta**:

   - $r_{xy} = +1$: indica una perfetta relazione lineare positiva crescente; tutti i punti giacciono su una retta a pendenza positiva ($y = ax + b$, con $a > 0$).
   - $r_{xy} = -1$: indica una perfetta relazione lineare negativa decrescente ($a < 0$).

4. **Assenza di relazione lineare**: Un valore $r_{xy} \approx 0$ denota **assenza di associazione lineare**, ma non implica l'indipendenza stocastica. Relazioni funzionali non lineari deterministiche (es. $Y = X^2$ con X distribuita simmetricamente attorno all'origine) generano $r_{xy} = 0$, pur in presenza di una totale dipendenza.

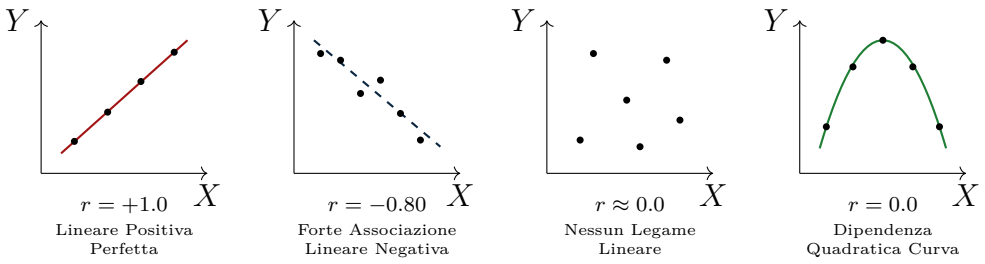


Figura 3.6: Configurazioni tipiche di dispersione bivariata e corrispondenti coefficienti r di Pearson.

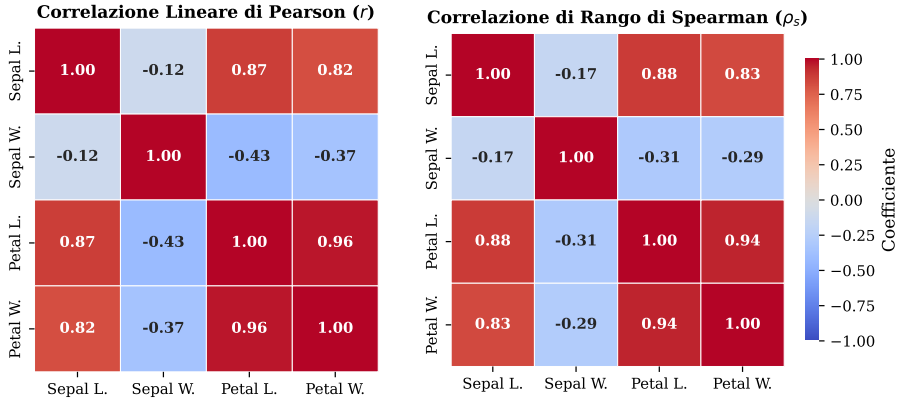


Figura 3.7: Heatmap di correlazione per il dataset Iris complessivo e disaggregato per singola specie.

3.2.3 Fallacia Causale e Variabili Concomitanti

Un principio fondante dell'analisi dati stabilisce che **la correlazione statistica non implica causalità** (*correlation does not imply causation*). Un valore elevato di $|r_{xy}|$ può originare da tre dinamiche:

1. X è causa diretta di Y.
2. Y è causa diretta di X.
3. X e Y sono entrambe causate da una terza variabile non osservata o latente, definita **variabile di confondimento (confounding factor)**.

Esempio Classico: Vendita di Gelati e Annegamenti

Durante i mesi estivi si registra una forte correlazione lineare positiva ($r > 0.85$) tra le vendite di gelati (X) e il numero di decessi per annegamento (Y). Dedurre che l'assunzione di gelati aumenti il rischio di annegamento costituisce una fallacia causale: entrambe le grandezze sono influenzate da un fattore di confondimento esogeno, la temperatura ambientale (*hot weather*), che incentiva simultaneamente il consumo di gelati e l'afflusso di bagnanti in mare e nei fiumi.

3.2.4 Coefficiente di Correlazione per Ranghi di Spearman

Quando i dati violano l'ipotesi di normalità bivariata o presentano relazioni non lineari ma strettamente monotone, si impiega il **Coefficiente di Correlazione per Ranghi di Spearman (ρ)**.

La procedura converte ciascun valore continuo x_i e y_i nella rispettiva posizione d'ordine o rango ordinale $r(x_i), r(y_i) \in \{1, \dots, n\}$. In assenza di valori paritari (*ties*), la formulazione algebrica esplicita risulta:

$$\rho = 1 - \frac{6 \sum_{i=1}^{n} \Big(r(x_i) - r(y_i) \Big)^2}{n(n^2 - 1)} \tag{3.7}$$

- $\rho = +1$: indica una perfetta relazione monotona crescente (al crescere di X, Y cresce strettamente, anche secondo andamenti esponenziali o sigmoidei).
- $\rho = -1$: indica una perfetta relazione monotona decrescente.
- **Robustezza**: Essendo basato sui ranghi ordinali anziché sui valori metrici effettivi, Spearman è molto meno sensibile a valori anomali isolati situati nelle code.

3.3 Tassonomia e Rilevamento degli Outlier

Un **outlier** (valore anomalo) è definito come un'istanza che devia marcatamente dalla distribuzione della maggior parte delle osservazioni, suggerendo che sia stata generata da un meccanismo generatore differente o affetta da distorsioni di misura.

3.3.1 Ruolo degli Outlier: Rumore vs Target di Analisi

- **Outlier come Rumore (Noise)**: L'anomalia è conseguenza di un errore materiale di digitazione, di un malfunzionamento momentaneo della sensoristica o di un'alterazione spuria. In questa veste, l'outlier degrada gli stimatori e deve essere isolato, corretto o rimosso in fase di pre-processing.
- **Outlier come Obiettivo Primario (Analysis Goal)**: L'indagine analitica è finalizzata precisamente all'identificazione di eventi eccezionali, che veicolano il massimo valore informativo.
  - Rilevamento di intrusioni informatiche (*Intrusion Detection*).

  - Identificazione di frodi finanziarie o transazioni bancarie illecite.
  - Manutenzione predittiva su macchinari industriali tramite rilievo di micro-vibrazioni anomale.

3.3.2 Metodologie di Identificazione degli Outlier

La tassonomia degli approcci di identificazione distingue tra indagini su singola feature e indagini congiunte multidimensionali:

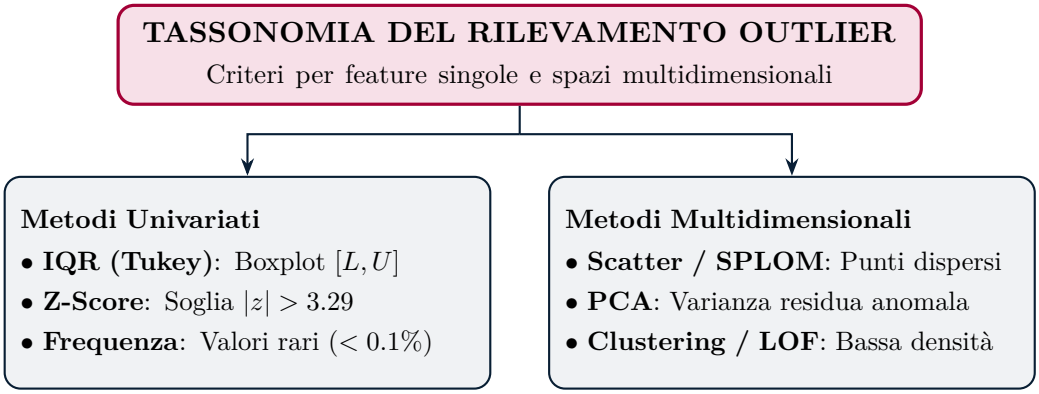


Figura 3.8: Tassonomia delle metodologie di rilevamento delle anomalie.

- **Attributi Categorici**: Un'osservazione anomala presenta una categoria associata a una frequenza empirica prossima allo zero ($< 0.1\%$).
- **Attributi Numerici Univariati**:
  - *Regola di Tukey*: Valori esterni all'intervallo $[Q_1 - 1.5\,\mathrm{IQR},\ Q_3 + 1.5\,\mathrm{IQR}]$.
  - *Criterio Z-Score*: Valori con scostamento standardizzato $|z| > 3.29$ (code estreme della normale).
- **Spazi Multidimensionali**:
  - *Scatter Plot bivariati*: Identificazione ottica di punti lontani dal corpo centrale della nube.
  - *Proiezioni PCA*: Punti che presentano valori anomali lungo le componenti principali secondarie.
  - *Clustering e Local Outlier Factor (LOF)*: Riconoscimento di punti isolati a bassa densità locale rispetto al vicinato di k elementi.

3.4 Gestione dei Valori Mancanti

Un record manifesta **valori mancanti (Missing Values)** quando per una o più feature non è presente alcuna informazione registrata (formalmente denotata con simboli nulli come `NaN`, `NULL`, `?` o mediante valori convenzionali impliciti come `999` o `0`).

3.4.1 Cause di Insorgenza

1. **Omissione volontaria per riservatezza**: L'intervistato si rifiuta di comunicare attributi sensibili (es. reddito personale, orientamento politico o religioso, abitudini sanitarie).

2. **Inapplicabilità logica del campo**: La variabile non è definita per una specifica sottopopolazione (es. il numero di gravidanze per individui di sesso maschile, o l'anzianità lavorativa per minori in età scolare).

3. **Avarie strumentali o caduta di connettività**: Malfunzionamento di sensori di telemetria che generano valori nulli o trasmettono serie fittizie costanti (es. sensore anemometrico bloccato che registra sistematicamente velocità del vento pari a $0\,\mathrm{m/s}$).

Mentre nella fase di *Data Understanding* l'obiettivo è identificare la natura e l'incidenza dei dati mancanti, le strategie operative di bonifica, cancellazione ed imputazione statistica sono approfondite nel Capitolo 4 (sezione 4.5) e applicate in ambiente Python nel Capitolo 5 (sezione 5.2).

3.4.2 Checklist Operativa per il Data Understanding

A completamento della fase di esplorazione preliminare, l'analista deve verificare sistematicamente la seguente checklist:

Ambito di Controllo	Verifica Diagnostica da Eseguire
Coerenza Sintattica	Verificare i tipi di dato, la codifica di formati numerici/stringa e i vincoli di dominio.
Valori Anomali (Outlier)	Ispezionare la presenza di valori estremi mediante Boxplot univariati e Scatter Plot bivariati.
Valori Mancanti (Missing)	Rilevare incidenza percentuale dei campi nulli e identificare valori sentinella impliciti (`0`, `-1`, `999`).
Associazioni e Ridondanze	Calcolare le correlazioni di Pearson e Spearman per isolare coppie di attributi collineari o ridondanti.
Assunzioni Distributive	Verificare simmetria, normalità e omoschedasticità prima di impostare la fase di preparazione e pulizia.

Tabella 3.1: Checklist diagnostica operativa per la chiusura della fase di Data Understanding.

Capitolo 4

Data Preparation & Cleaning – Riduzione, Discretizzazione e Trasformazione

4.1 Differenze tra Data Understanding e Data Preparation

Mentre la fase di Data Understanding si concentra sulla diagnosi delle anomalie e sulla comprensione delle strutture latenti del dataset, la **Data Preparation** costituisce la componente operativa e trasformativa del ciclo KDD. In questa fase i dati grezzi vengono bonificati, compressi, riscalati e ricodificati per soddisfare i requisiti algebrici e computazionali degli algoritmi di data mining.

La transizione tra la fase diagnostica e la fase trasformativa è sintetizzata dal passaggio dalle evidenze empiriche alle azioni di pulizia:

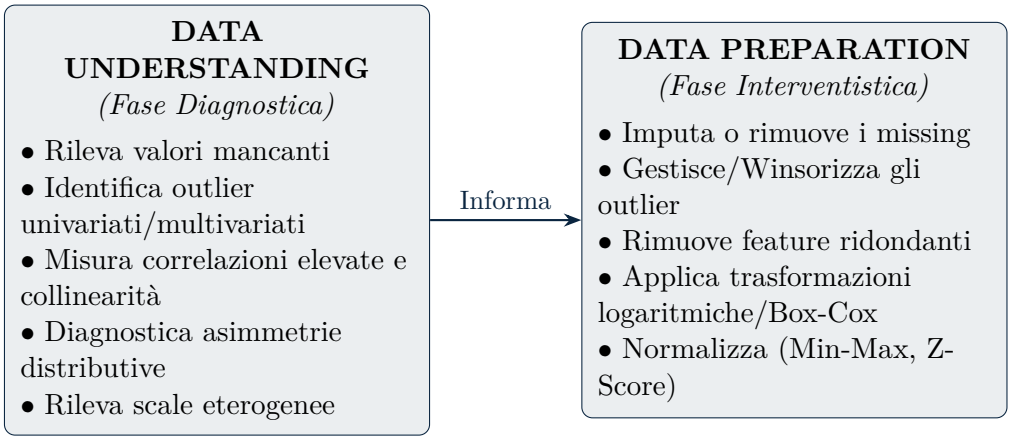


Figura 4.1: Relazione sequenziale tra Data Understanding (diagnosi) e Data Preparation (intervento).

4.2 Aggregazione e Campionamento

La riduzione del volume dei dati si persegue mediante la combinazione di osservazioni o attraverso la selezione statistica di sottoinsiemi rappresentativi.

4.2.1 Aggregazione dei Dati

L'aggregazione sintetizza due o più record o attributi in una singola misura aggregata.

- **Obiettivi primari**:
  1. *Riduzione dimensionale*: Riduce il numero complessivo di record o attributi, alleggerendo il carico computazionale.
  2. *Riconfigurazione di scala e granularità*: Adatta l'analisi al livello concettuale idoneo (ad esempio, aggregare le transazioni orarie in totali settimanali o i singoli punti vendita a livello regionale).
  3. *Stabilizzazione della varianza*: Le grandezze aggregate mediano il rumore casuale, producendo indicatori a varianza inferiore e relazioni più stabili.
- **Caso Studio: Precipitazioni in Australia (1982–1993)**: Nelle serie storiche pluviometriche australiane, la deviazione standard delle precipitazioni medie mensili evidenzia un'elevata variabilità con code fino a 15 cm. Al contrario, aggregando i dati su base annua, la deviazione standard si concentra su valori bassi con dispersione contenuta

(Figura 4.2). Per l'analisi dei cicli stagionali è necessaria la scala mensile disaggregata, mentre per la comparazione dei regimi climatici regionali la scala annua aggregata risulta più robusta e priva del rumore a breve termine.

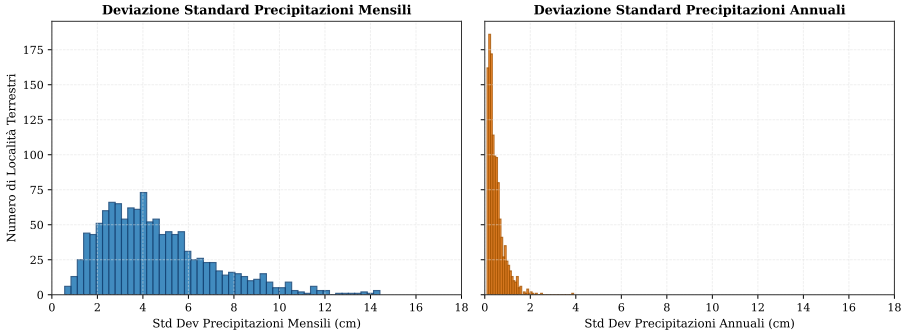


Figura 4.2: Confronto di dispersione tra precipitazioni medie mensili e precipitazioni medie annue aggregate in Australia.

4.2.2 Tecniche di Campionamento (Sampling)

Il campionamento estrae un sottoinsieme rappresentativo di dimensione n da una popolazione complessiva di cardinalità N ($n \ll N$). Un campione è **rappresentativo** se riflette fedelmente le medesime proprietà distributive e statistiche della popolazione sorgente.



Figura 4.3: Albero delle strategie di campionamento nel data mining.

1. **Campionamento Casuale Semplice (Simple Random Sampling)**: Ciascuna istanza possiede la medesima probabilità a priori di selezione:

$$P(x_i) = \frac{1}{N} \tag{4.1}$$

- *Senza reinserimento (without replacement)*: L'elemento estratto viene rimosso dal pool e non può essere riselezionato.
- *Con reinserimento (with replacement)*: L'elemento estratto rimane disponibile per successive estrazioni (stesso record estratto > 1 volta).

2. **Campionamento Stratificato (Stratified Sampling)**: La popolazione viene suddivisa in strati omogenei disgiunti (ad esempio per etichetta di classe o per percentili). Da ciascun strato viene estratto un campione proporzionale alla sua cardinalità relativa, garantendo che le classi rare non vengano omesse dal campione estratto.

La scelta della dimensione campionaria ottimale impone un compromesso tra costo computazionale e accuratezza statistica, come illustrato nella curva di trade-off di Figura 4.4.

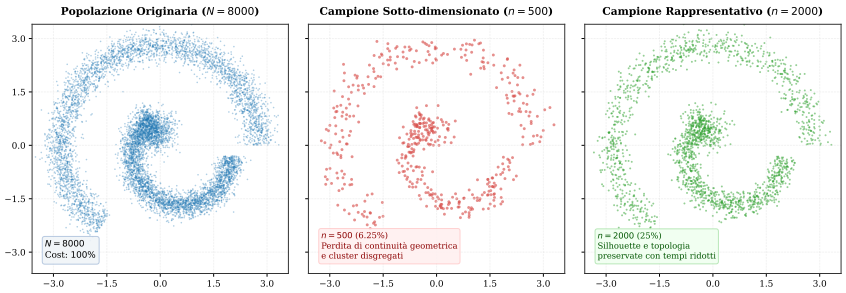


Figura 4.4: Trade-off tra dimensione campionaria e accuratezza empirica di stima.

4.3 Riduzione della Dimensionalità

La presenza di un numero elevato di feature determina rilevanti criticità computazionali e geometriche.

4.3.1 La Maledizione della Dimensionalità (*Curse of Dimensionality*)

All'aumentare della dimensionalità p, il volume dello spazio $\mathbb{R}^p$ cresce esponenzialmente rispetto al numero di osservazioni disponibili. Questo fenomeno causa:

1. **Sparsità estrema**: Le osservazioni si disperdono nello spazio multidimensionale; la densità locale tende a zero e lo spazio diventa prevalentemente vuoto.

2. **Concentrazione delle distanze**: All'aumentare di p, il contrasto relativo tra la distanza minima e la distanza massima rispetto a qualsiasi punto di riferimento tende a vanificarsi:

$$\lim_{p \to \infty} \frac{\text{dist}_{\text{max}} - \text{dist}_{\text{min}}}{\text{dist}_{\text{min}}} \to 0 \tag{4.2}$$

   Ciò compromette le prestazioni degli algoritmi metrica-dipendenti (k-NN, clustering K-Means).

3. **Instabilità delle stime**: La stima della matrice di covarianza $p \times p$ richiede una numerosità campionaria che scala esponenzialmente con p per scongiurare singolarità numeriche.

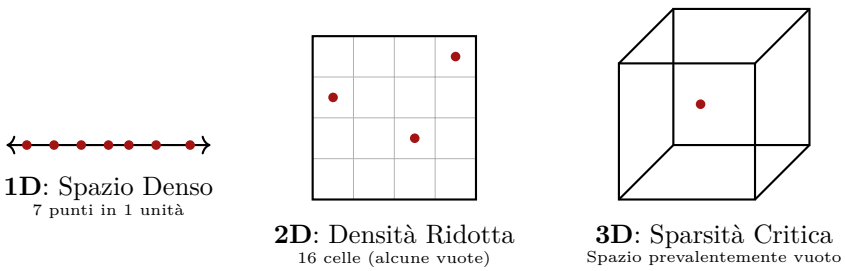


Figura 4.5: Illustrazione della dispersione geometrica dei campioni all'aumentare delle dimensioni (Curse of Dimensionality).

4.3.2 Selezione delle Feature (Feature Selection)

La **Feature Selection** seleziona un sottoinsieme ottimale di feature originali, eliminando gli attributi irrilevanti (non informativi) e ridondanti (duplicati di informazione già presente). I metodi si suddividono in tre categorie:

Paradigma	Meccanismo Operativo	Vantaggi / Limiti
Filter	Valutano la significatività statistica (t-test, ANOVA, correlazione di Pearson, χ^2) a priori, indipendentemente dal modello.	Computazionalmente velocissimi; non considerano l'interazione tra feature.
Wrapper	Utilizzano il modello predittivo come oracolo di valutazione iterativo (Forward Selection, Backward Elimination).	Ottimizzano l'accuratezza finale; computazionalmente gravosi (2^p combinazioni teoriche).
Embedded	La selezione è integrata nel processo di addestramento dell'algoritmo (es. alberi decisionali, regolarizzazione Lasso L_1).	Eccellente compromesso tra accuratezza e tempi computazionali.

Tabella 4.1: Confronto sinottico tra approcci Filter, Wrapper ed Embedded per la Feature Selection.

4.3.3 Creazione e Proiezione di Feature

- **Costruzione di Feature (*Feature Construction*)**: Generazione di nuove variabili combinando attributi esistenti tramite conoscenza del dominio (ad esempio, calcolo della densità come massa/volume o dell'efficienza come ore effettive/ore previste).
- **Proiezione di Feature (*Feature Projection*)**: Mappatura lineare o non lineare dello spazio originario $\mathbb{R}^p$ in un sottospazio ridotto $\mathbb{R}^k$ ($k \ll p$), massimizzando la varianza conservata o la separabilità tra classi (PCA, SVD, Linear Discriminant Analysis - LDA, Autoencoder).

4.4 Analisi delle Componenti Principali (PCA)

L'**Analisi delle Componenti Principali (PCA)** è una tecnica lineare non supervisionata che trasforma un insieme di p variabili originarie correlate in un nuovo insieme di p variabili ortogonali (scorrelate), ordinate in modo decrescente rispetto alla quota di varianza spiegata.

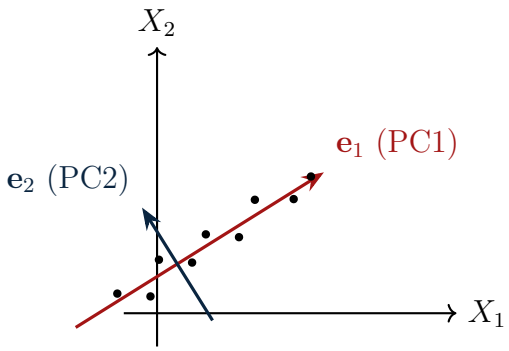


Figura 4.6: Allineamento delle componenti principali ortogonali lungo le direzioni di massima varianza dei dati.

4.4.1 Procedura Dettagliata per il Calcolo della PCA

La procedura analitica per il calcolo della PCA si articola in cinque passi formali:

Passo 1: Centratura e Standardizzazione della Matrice Dati

Data la matrice iniziale $D \in \mathbb{R}^{n \times p}$, si calcola la media campionaria per ciascuna colonna:

$$\bar{x}_j = \frac{1}{n} \sum_{i=1}^{n} D_{ij} \tag{4.3}$$

Si genera la **matrice centrata** $C \in \mathbb{R}^{n \times p}$ sottraendo il vettore delle medie da ciascuna riga:

$$C_{ij} = D_{ij} - \bar{x}_j \tag{4.4}$$

Qualora le variabili presentino unità o scale eterogenee, si calcola la **matrice standardizzata** $Z \in \mathbb{R}^{n \times p}$ dividendo ciascuno scarto per la deviazione standard campionaria s_j:

$$Z_{ij} = \frac{D_{ij} - \bar{x}_j}{s_j} \tag{4.5}$$

Passo 2: Calcolo della Matrice di Covarianza

Si determina la matrice di covarianza empirica $V \in \mathbb{R}^{p \times p}$ mediante il prodotto matriciale:

$$V = \frac{1}{n-1} C^T C \tag{4.6}$$

Passo 3: Decomposizione Spettrale (Autovalori e Autovettori)

Si risolve il problema agli autovalori per la matrice reale simmetrica V:

$$V\mathbf{e}_j = \lambda_j \mathbf{e}_j, \quad j = 1, \ldots, p \tag{4.7}$$

- Ogni **autovettore** normalizzato ad ampiezza unitaria $\|\mathbf{e}_j\|_2 = 1$ identifica la direzione del j-esimo asse principale. Poiché V è simmetrica, autovettori relativi ad autovalori distinti sono mutuamente ortogonali ($\mathbf{e}_j^T \mathbf{e}_k = 0$ per $j \neq k$).
- Il corrispondente **autovalore** $\lambda_j \geq 0$ quantifica la varianza spiegata lungo la direzione dell'asse principale $\mathbf{e}_j$.

Passo 4: Ordinamento e Selezione delle Componenti

Si ordinano autovalori e autovettori in senso decrescente rispetto alla magnitudo di λ:

$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_p \geq 0 \tag{4.8}$$

La percentuale di varianza spiegata dalla singola componente j è calcolata come:

$$\text{Varianza Spiegata (\%)} = \frac{\lambda_j}{\sum_{m=1}^{p} \lambda_m} \times 100 \tag{4.9}$$

Si selezionano le prime k componenti ($k < p$) che garantiscono una soglia cumulativa desiderata (es. 85% o 95%), scartando le restanti componenti a bassa varianza (Figura 4.7). Si assembla la matrice di trasformazione $F \in \mathbb{R}^{p \times k}$:

$$F = \begin{bmatrix} \mathbf{e}_1 & \mathbf{e}_2 & \cdots & \mathbf{e}_k \end{bmatrix} \tag{4.10}$$

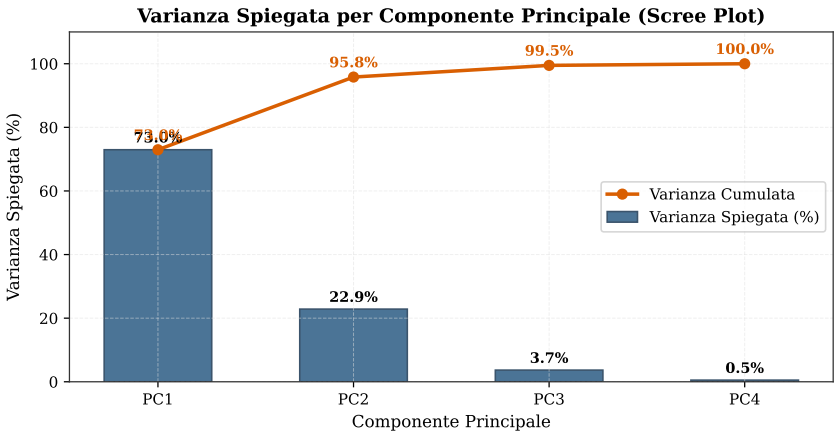


Figura 4.7: Scree plot degli autovalori e curva di varianza cumulativa spiegata.

Passo 5: Proiezione nel Nuovo Sottospazio

La matrice finale proiettata $Z \in \mathbb{R}^{n \times k}$ nello spazio ridotto k-dimensionale si calcola mediante moltiplicazione matriciale:

$$Z = CF \tag{4.11}$$

In Figura 4.8 è visualizzata la proiezione dei campioni del dataset Iris sul piano bidimensionale definito dalle prime due componenti principali (PC1 e PC2), che catturano oltre il 95% della varianza originaria complessiva.

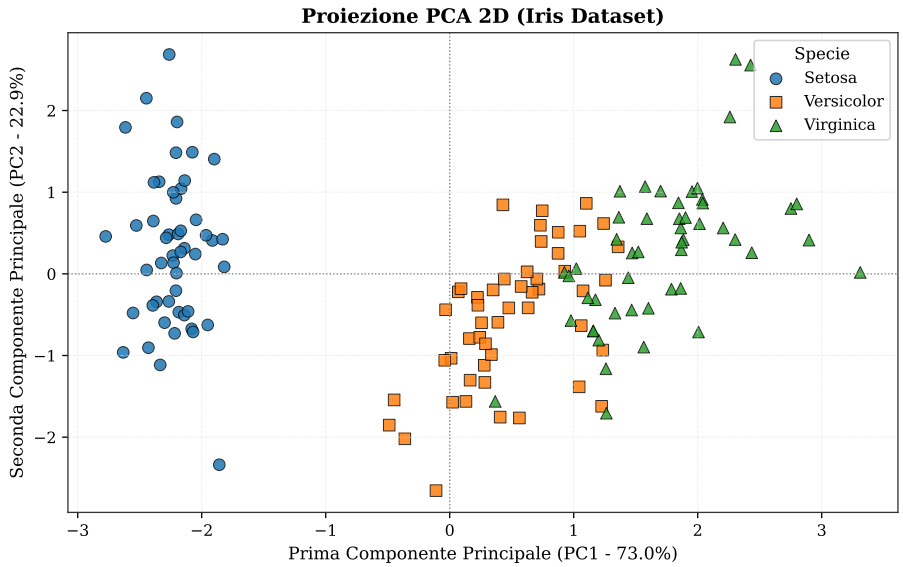


Figura 4.8: Proiezione bidimensionale dei campioni Iris lungo le prime due componenti principali.

4.5 Gestione dei Valori Mancanti

Le strategie operative per il trattamento dei record incompleti comprendono:

1. **Eliminazione Completa del Record (Listwise Deletion)**: Rimozione integrale di tutte le righe con almeno un campo nullo. Applicabile in sicurezza solo quando l'incidenza dei missing è trascurabile ($< 3-5\%$) e i dati mancano in modo puramente casuale (*Missing Completely at Random - MCAR*); altrimenti introduce gravi distorsioni selettive.

2. **Sostituzione con Indici di Tendenza Centrale**:

- *Media*: Per variabili numeriche simmetriche prive di outlier.
- *Mediana*: Per variabili numeriche asimmetriche o in presenza di code pesanti.
- *Moda*: Per attributi categorici qualitativi.
- *Criticità*: Riduce artificialmente la varianza della feature e attenua l'intensità delle correlazioni con gli altri attributi.

3. **Imputazione Condizionale tramite Segmentazione**: Ripartizione dei campioni in cluster o classi omogenee sulla base delle feature complete e imputazione del valore mancante impiegando la media o mediana calcolata esclusivamente all'interno del rispettivo cluster.

4. **Imputazione Predittiva Basata su Modelli**: Addestramento di un modello supervisionato (regressione lineare, k-NN, Random Forest) dove la variabile da stimare funge da target e gli attributi completi fungono da predittori.

4.6 Tecniche di Discretizzazione e Binarizzazione

La **discretizzazione** trasforma una variabile quantitativa continua in un attributo ordinale discreto, suddividendo il dominio continuo in un insieme finito di intervalli contigui (detti *bin* o *classi*).

4.6.1 Discretizzazione Non Supervisionata

1. Natural Binning (Equal-Width Binning)

Suddivide l'intervallo $[x_{\min}, x_{\max}]$ in k classi aventi la medesima **ampiezza costante**:

$$\delta = \frac{x_{\max} - x_{\min}}{k} \tag{4.12}$$

L'i-esimo intervallo ($i = 0, \ldots, k-1$) è delimitato da:

$$I_i = \Big[x_{\min} + i\delta,\ x_{\min} + (i+1)\delta\Big) \tag{4.13}$$

Esempio Numerico: Prezzo di 12 Birre

Dato l'insieme di $N = 12$ prezzi compresi tra $x_{\min} = 100$ e $x_{\max} = 160$, impostando $k = 4$ classi otteniamo ampiezza $\delta = \frac{160-100}{4} = 15$. Gli intervalli risultano:

- Classe 1: $[100, 115) \rightarrow \{100, 110, 110\}$ (Conteggio: 3)
- Classe 2: $[115, 130) \rightarrow \{120, 120, 125\}$ (Conteggio: 3)
- Classe 3: $[130, 145) \rightarrow \{130, 130, 135, 140\}$ (Conteggio: 4)
- Classe 4: $[145, 160] \rightarrow \{150, 160\}$ (Conteggio: 2)

2. Equal-Frequency Binning (Equal-Depth Binning)

Ordina i campioni in senso crescente e assegna a ciascun intervallo lo **stesso numero di osservazioni**:

$$f = \frac{N}{k} \tag{4.14}$$

Sugli stessi 12 prezzi ordinati:

$$\{100, 110, 110, 120, 120, 125, 130, 130, 135, 140, 150, 160\}$$

con $k = 4$, la frequenza per classe è $f = 12/4 = 3$:

- Classe 1: $\{100, 110, 110\}$
- Classe 2: $\{120, 120, 125\}$
- Classe 3: $\{130, 130, 135\}$
- Classe 4: $\{140, 150, 160\}$

Questo approccio uniforma la densità e mitiga l'impatto degli outlier, sebbene possa collocare valori contigui identici in classi distinte qualora cadano a cavallo della soglia.

Formule per il Numero e l'Ampiezza Ottimale dei Bin

Oltre alla Regola di Sturges, si utilizza la formulazione logaritmica decimale:

$$C = 1 + \frac{10}{3} \log_{10}(N) \tag{4.15}$$

Per ottimizzare direttamente l'ampiezza h combinando variabilità e numerosità campionaria, si applica la **Regola di Scott (1979)**:

$$h = \frac{3.5 \times s}{\sqrt[3]{N}} \tag{4.16}$$

dove s è la deviazione standard campionaria e N la numerosità del dataset.

4.6.2 Discretizzazione Supervisionata Basata sull'Entropia

Quando è disponibile una variabile di classe target $Y \in \{c_1, \ldots, c_m\}$, le soglie di taglio vengono ottimizzate per massimizzare la purezza informativa. L'entropia di un insieme di campioni S è data da:

$$\text{Entropy}(S) = -\sum_{j=1}^{m} p_j \log_2(p_j) \tag{4.17}$$

dove p_j è la proporzione di istanze in S appartenenti alla classe c_j. Se un punto di split T partiziona S nei sottoinsiemi S_1 ed S_2, l'entropia combinata pesata è:

$$E(S,T) = \frac{|S_1|}{|S|}\text{Entropy}(S_1) + \frac{|S_2|}{|S|}\text{Entropy}(S_2) \tag{4.18}$$

Il punto di split ottimale T^* massimizza il guadagno informativo $\text{Gain}(S,T) = \text{Entropy}(S) - E(S,T)$.

4.6.3 Binarizzazione (Encoding)

Converte variabili qualitative con m modalità in attributi binari $\{0,1\}$:

- **Codifica One-Hot (Asymmetric Binary Encoding)**: Alloca un indicatore binario dedicato per ciascuna modalità. Genera m variabili asimmetriche ed evita di introdurre relazioni d'ordine artificiose:

Valore Categorico	**Codice Intero**	x_1	x_2	x_3	x_4	x_5
awful	0	1	0	0	0	0
poor	1	0	1	0	0	0
OK	2	0	0	1	0	0
good	3	0	0	0	1	0
great	4	0	0	0	0	1

Tabella 4.2: Esempio di codifica One-Hot per un attributo a 5 livelli qualitativi.

- **Codifica Binaria Compatta**: Mappa ciascuna modalità nella sua rappresentazione binaria posizionale numerica, richiedendo $n = \lceil \log_2(m) \rceil$ bit.

Valore Categorico	Codice Intero	x_1	x_2	x_3
awful	0	0	0	0
poor	1	0	0	1
OK	2	0	1	0
good	3	0	1	1
great	4	1	0	0

Tabella 4.3: Codifica binaria compatta su 3 bit per 5 livelli categorici.

Criticità: Induce correlazioni fittizie tra i singoli bit privi di riscontro empirico nel fenomeno reale.

4.7 Trasformazioni Funzionali di Attributi e Stabilizzazione della Varianza

Una **trasformazione funzionale** applica una funzione T continua e strettamente monotona a ciascun valore di un attributo X: $Y = T(X)$.

- **Stabilizzazione della varianza**: Rende omogenea la dispersione lungo tutto il dominio (*omoschedasticità*).
- **Normalizzazione della forma**: Riconduce code pesanti o asimmetrie a distribuzioni approssimativamente normali.
- **Linearizzazione delle relazioni**: Rende lineari dipendenze polinomiali o esponenziali.

4.7.1 Famiglia delle Trasformazioni Esponenziali e Logaritmiche

La trasformazione parametrica generale è definita come:

$$T_p(x) = \begin{cases} ax^p + b & \text{se } p \neq 0 \\ c\log(x) + d & \text{se } p = 0 \end{cases} \tag{4.19}$$

con $x > 0$.

- **Trasformazione Lineare ($p = 1$)**: Modifica unicamente origine e scala (es. euro in lire con $a = 1936.27, b = 0$; gradi Fahrenheit in Celsius con $a = 5/9, b = -160/9$).

- **Trasformazione Logaritmica** ($p = 0$): $T(x) = c\log(x) + d$. Comprime i valori elevati ed espande la scala dei valori piccoli, contrastando l'asimmetria positiva destra.

Bar	**Birra**	**Gain Originario** (X)	**Gain Trasformato** $\log_{10}(X)$
A	Bud	20	1.3010
A	Becks	10000	4.0000
C	Bud	300	2.4771
D	Bud	400	2.6021
D	Becks	5	0.6990
E	Becks	120	2.0792
E	Bud	120	2.0792
F	Bud	11000	4.0414
G	Bud	1300	3.1139
H	Bud	3200	3.5051
H	Becks	1000	3.0000
I	Bud	135	2.1303

Tabella 4.4: Campione di 12 osservazioni di profitto (Gain) prima e dopo trasformazione logaritmica.

Parametro Statistico	**Scala Originaria** (X)	**Scala Logaritmica** ($Y = \log_{10} X$)
Media Aritmetica	2481.82	2.5956
Deviazione Standard	4079.02	1.0651
Minimo	5	0.6990
1° **Quartile** (Q_1)	120	2.0792
Mediana (Q_2)	400	2.6021
3° **Quartile** (Q_3)	2250	3.3095
Massimo	11000	4.0414

Tabella 4.5: Indicatori statistici descrittivi prima e dopo la trasformazione logaritmica.

Nella scala originaria la deviazione standard (4079.02) eccede abbondantemente la media (2481.82), trascinata dai picchi a 10000 e 11000. Dopo

la conversione in $\log_{10}$, media (2.5956) e mediana (2.6021) coincidono quasi perfettamente, ripristinando la simmetria.

4.7.2 Altre Trasformazioni di Stabilizzazione

- **Radice Quadrata ($p = 1/2$)**: $T(x) = \sqrt{x}$. Stabilizza la varianza nei dati di conteggio poissoniani ($\sigma^2 = \mu$).
- **Reciproca ($p = -1$)**: $T(x) = 1/x$. Utilizzata quando la varianza cresce in modo quadratico o superlineare rispetto alla media.

4.8 Algoritmi di Normalizzazione

La normalizzazione riconduce attributi misurati su scale numeriche disomogenee all'interno di intervalli standard comparabili, scongiurando il dominio distorto di feature con valori numerici elevati negli algoritmi basati sulle distanze metriche (k-NN, K-Means, SVM, reti neurali).

4.8.1 Normalizzazione Min-Max

Mappa linearmente l'intervallo originario $[\min_A, \max_A]$ nell'intervallo target $[\text{new_min}_A, \text{new_max}_A]$ (frequentemente $[0, 1]$):

$$v' = \frac{v - \min_A}{\max_A - \min_A}\Big(\text{new_max}_A - \text{new_min}_A\Big) + \text{new_min}_A \tag{4.20}$$

Preserva esattamente le relazioni lineari originarie, ma risulta sensibile a outlier estremi che possono comprimere i valori ordinari in un sottointervallo minuscolo.

4.8.2 Standardizzazione Z-Score

Riconduce la variabile ad avere **media nulla e varianza unitaria**:

$$v' = \frac{v - \bar{x}_A}{s_A} \tag{4.21}$$

dove $\bar{x}_A$ è la media campionaria e s_A la deviazione standard campionaria. È l'approccio raccomandato per distribuzioni approssimativamente normali e resiste meglio alla presenza di valori sparsi nelle code.

4.8.3 Normalizzazione per Scalamento Decimale (Decimal Scaling)

Sposta la virgola decimale dividendo per una potenza di 10:

$$v' = \frac{v}{10^j} \tag{4.22}$$

dove j è il minimo intero positivo tale che:

$$\max(|v'|) < 1 \iff 10^{j-1} \leq \max(|v|) < 10^j \tag{4.23}$$

Esempio Numerico di Decimal Scaling

Dato il vettore $V = \{-10, 201, 301, -401, 501, 601, 701\}$:

1. Il massimo in valore assoluto è $m = \max(|v|) = 701$.
2. Il minimo intero j tale che $10^j > 701$ è $j = 3$ ($10^3 = 1000$).
3. Dividendo per 1000 si ottiene:

$$V' = \{-0.01, 0.201, 0.301, -0.401, 0.501, 0.601, 0.701\} \tag{4.24}$$

Tutti i valori trasformati appartengono rigidamente all'intervallo aperto $(-1, +1)$.

4.9 Trattamento Univariato degli Outlier

Richiamando i criteri diagnostici fondati su Boxplot, Range Interquartilico (IQR) e Z-Score formalizzati nel Capitolo 3 (sezione 3.3), la fase di *Data Preparation* definisce le regole operative per neutralizzare l'influenza distorsiva dei valori anomali isolati senza necessariamente scartare l'intero record:

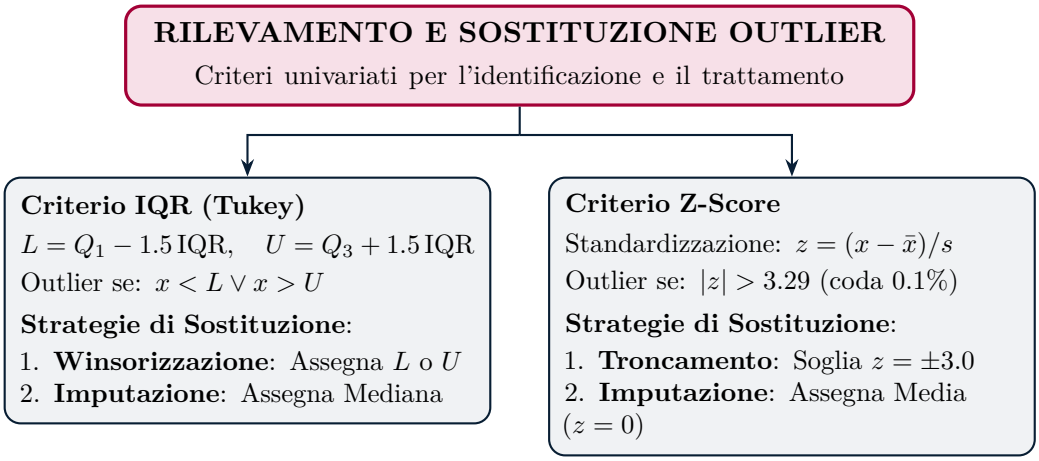


Figura 4.9: Quadro decisionale per l'identificazione e la sostituzione univariata degli outlier.

4.9.1 Trattamento Basato sul Range Interquartilico (Tukey)

Individuate le recinzioni di Tukey $[L, U] = [Q_1 - 1.5\,\mathrm{IQR},\ Q_3 + 1.5\,\mathrm{IQR}]$, le opzioni di bonifica per i valori esterni $x \notin [L, U]$ prevedono:

- **Winsorizzazione (Sostituzione al bordo)**: Si imposta $x' = L$ se $x < L$, e $x' = U$ se $x > U$. Questa trasformazione preserva la monotonicità ordinale e la numerosità campionaria originale, annullando la magnitudo distorsiva del picco estremo;
- **Imputazione con la Mediana**: Si rimpiazza il valore estremo con la mediana calcolata sul sottoinsieme dei dati regolari interni all'intervallo $[L, U]$.

4.9.2 Trattamento Basato sullo Z-Score

Per attributi a distribuzione approssimativamente normale con anomalie identificate dalla condizione $|z| > 3.29$, si adottano due strategie di sostituzione:

1. **Troncamento a 3σ (Clamping)**: Il valore estremo viene convertito nel valore corrispondente al punto di soglia $z = \pm 3.0$ ($x' = \bar{x} \pm 3.0\,s$). Questa scelta riconduce il record entro un intervallo plausibile senza cancellare l'informazione che l'osservazione apparteneva all'estremo della coda;

2. **Imputazione con la Media Campionaria**: Il valore anomalo viene rimpiazzato con il baricentro $z = 0$, corrispondente alla media campionaria originaria $\bar{x}$.

Capitolo 5

Laboratorio Python: Data Understanding e Data Preparation

5.1 Ambiente di Lavoro, Ingestione e Data Integration

Il laboratorio fa riferimento all'attività svolta con lo stack scientifico di Python (`numpy`, `pandas`, `scipy`, `matplotlib`, `seaborn` e `scikit-learn`), impiegando il dataset **Tips** (arricchito con la tabella **Tips_Age**), relativo alle abitudini di spesa e di mancia dei clienti di un ristorante, per illustrare l'intero flusso operativo: dall'ingestione e integrazione dei dati, alla diagnosi e imputazione dei valori mancanti, fino all'ingegnerizzazione delle feature, normalizzazione e riduzione dimensionale con PCA e t-SNE.

Ambiente Interattivo e Riproducibilità del Laboratorio

Tutti gli esempi, le pipeline di trasformazione e i grafici presentati in questo capitolo sono riproducibili ed eseguibili interattivamente tramite i notebook Jupyter inclusi nella cartella `Data-Mining/notebooks/`.
Avvio rapido con Docker:

- Da terminale (nella radice del progetto): `docker compose up jupyter`
- Oppure con doppio clic su Windows: `scripts/start-jupyter.cmd`
- Interfaccia web: `http://localhost:8888/lab` (senza password né token).

Per eseguire direttamente gli script senza interfaccia grafica da riga di comando:

- `docker compose run -rm app python3 Data-Mining/notebooks/generate_dispensa_figures.py`

Nota 5.1.1 (Riferimento al Notebook Interattivo)**.** Il codice delle sezioni 5.1, 5.2 e 5.3 corrisponde al notebook Jupyter interattivo **Notebook 1 (Data Understanding)** (file `1_basics_and_ understanding.ipynb`).

Lo stack software fondamentale per l'analisi esplorativa dei dati in Python comprende le seguenti librerie:

```
import numpy as np
import pandas as pd
import scipy.stats as stats
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.impute import SimpleImputer, KNNImputer
from sklearn.preprocessing import StandardScaler,
    MinMaxScaler, KBinsDiscretizer, OneHotEncoder,
    TargetEncoder
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
```

5.1.1 Caricamento e Ispezione Strutturale

Il dataset principale viene acquisito tramite file delimitato da tabulazione (`tsv`) impostando l'indice sulla prima colonna:

```
tips = pd.read_csv('tips.csv', sep='\t', index_col=0)
```

```
print(tips.head())
tips.info()
```

Il dataset contiene inizialmente 244 osservazioni e 7 colonne: `total_bill` (importo totale del conto), `tip` (mancia), `sex` (genere del pagante), `smoker` (presenza di fumatori al tavolo), `day` (giorno della settimana), `time` (pranzo o cena), `size` (numero di coperti al tavolo).

5.1.2 Data Integration e Join Relazionale

Nel laboratorio, l'età dei clienti è memorizzata in una sorgente secondaria (`tips_age.csv`). L'integrazione delle due tabelle avviene tramite una join sull'identificativo primario `ID`:

```
tips_age = pd.read_csv('tips_age.csv', sep='\t', index_col=0)
df = tips.join(tips_age, on='ID')
# Rimozione di eventuali colonne ridondanti generate dalla sorgente
df = df.drop(columns=['gender'], errors='ignore')
```

5.1.3 Rilevamento e Rimozione di Record Duplicati

L'ispezione delle righe duplicate richiede la verifica dell'uguaglianza tra tutti gli attributi del record:

```
# Visualizzazione di tutti i duplicati (tutte le copie)
duplicated_rows = df[df.duplicated(keep=False)]
print(f"Righe duplicate rilevate: {len(duplicated_rows)}")

# Rimozione mantenendo la prima occorrenza
df = df.drop_duplicates()
```

5.1.4 Bonifica di Valori Non Validi e Cast di Tipo

Spesso i dati contengono errori di trascrizione o stringhe non valide inserite al posto di campi numerici (ad esempio il valore fittizio `'aaaaa'` nella colonna `total_bill`). Ciò costringe Pandas a tipizzare la colonna come generico tipo `object`. La bonifica avviene mediante sostituzione puntuale e conversione esplicita di tipo:

```
# Sostituzione della stringa anomala con zero o NaN
df["total_bill"] = df["total_bill"].replace("aaaaa", np.nan)
```

```
3
4 # Cast esplicito a float numerico
5 df["total_bill"] = pd.to_numeric(df["total_bill"])
```

5.2 Diagnosi e Imputazione dei Valori Mancanti

5.2.1 Audit dei Missing Values

Il conteggio dei valori mancanti (`NaN`) per ciascun attributo si ottiene combinando `isnull()` e `sum()`:

```
1 # Riepilogo dei valori nulli per colonna
2 null_counts = df.isnull().sum()
3 print(null_counts[null_counts > 0])
4
5 # Selezione dei soli record incompleti
6 incomplete_records = df[df.isnull().any(axis=1)]
```

5.2.2 Strategie di Rimozione vs Imputazione Statistica

- **Listwise Deletion**: Eliminazione integrale di tutti i record con almeno un valore nullo:

```
1 df_cleaned = df.dropna()
```

- **Imputazione con la Mediana Condizionale**: Quando l'attributo `age` presenta valori mancanti o codificati con la convenzione sentinella 0, l'imputazione mediante la mediana calcolata sull'intero dataset potrebbe appiattire la variabilità tra fasce orarie distinte (*Lunch* vs *Dinner*). Si esegue pertanto un'imputazione stratificata:

```
1 df['age'] = df['age'].replace(0, np.nan)
2 # Imputa con la mediana di gruppo calcolata sulla
      fascia oraria (time)
3 df['age'] = df['age'].fillna(df.groupby('time')['age'
      ].transform('median'))
```

5.2.3 Imputazione Avanzata con Scikit-Learn

Il modulo `sklearn.impute` mette a disposizione trasformatori integrabili nelle pipeline di machine learning:

```python
# 1. SimpleImputer per feature numeriche (media o mediana)
imp_mean = SimpleImputer(strategy='mean')
df['tip'] = imp_mean.fit_transform(df[['tip']])

# 2. SimpleImputer per feature categoriche (moda/frequenza massima)
imp_mode = SimpleImputer(strategy='most_frequent')
df['smoker'] = imp_mode.fit_transform(df[['smoker']]).ravel()

# 3. KNNImputer (imputazione multivariata basata su k-Nearest Neighbors)
knn_imp = KNNImputer(n_neighbors=2, weights='distance')
num_cols = ['total_bill', 'tip', 'size', 'age']
df[num_cols] = knn_imp.fit_transform(df[num_cols])
```

5.3 Esplorazione Multivariata e Analisi di Correlazione

5.3.1 Tabelle di Contingenza e Distribuzioni Condizionate

Per studiare le associazioni tra variabili categoriche si utilizzano tabelle di contingenza normalizzate (*cross-tabulation*):

```python
# Tabella di frequenza congiunta tra genere e abitudine al fumo
smoker_xt = pd.crosstab(df['smoker'], df['sex'])

# Normalizzazione per riga (proporzioni percentuali)
smoker_pct = smoker_xt.div(smoker_xt.sum(1).astype(float), axis=0)
smoker_pct.plot(kind='bar', stacked=False, title='Genere per Categoria Fumatore')
plt.ylabel('Frequenza Relativa')
plt.show()
```

5.3.2 Matrici di Correlazione e Heatmap

Pandas consente di stimare simultaneamente i coefficienti di correlazione lineare (Pearson) e di rango (Spearman, Kendall) sui soli attributi numerici:

```python
# Calcolo della matrice di correlazione di Pearson
```

```
corr_pearson = df.corr(numeric_only=True, method='pearson')

# Calcolo con il coefficiente di Spearman (monotonia robusta)
corr_spearman = df.corr(numeric_only=True, method='spearman')

# Visualizzazione grafica tramite heatmap con annotazioni numeriche
plt.figure(figsize=(8, 6))
sns.heatmap(corr_pearson, annot=True, cmap='coolwarm', fmt='.2f', vmin=-1, vmax=1)
plt.title('Matrice di Correlazione di Pearson (Dataset Tips)')
plt.show()
```

5.3.3 Scatter Matrix e Pairplot

L'ispezione visiva incrociata delle relazioni bivariate si realizza tramite la funzione `scatter_matrix` di Pandas o `pairplot` di Seaborn:

```
# Scatter Matrix di Pandas
pd.plotting.scatter_matrix(df[['total_bill', 'tip', 'size', 'age']], figsize=(9, 9), diagonal='kde')
plt.show()

# Pairplot con codice colore basato sul momento del pasto (time)
sns.pairplot(df, vars=['total_bill', 'tip', 'size'], hue='time', palette='Dark2')
plt.show()
```

5.3.4 Feature Engineering: Costruzione di Indicatori di Dominio

La conoscenza del dominio consente di combinare variabili grezze per sintetizzare nuovi attributi ad alto valore interpretativo:

```
# 1. Percentuale effettiva di mancia rispetto al conto
df['tip_ratio'] = df['tip'] / df['total_bill']

# 2. Spesa media per persona (Bill Per Person - BPP)
df['bpp'] = df['total_bill'] / df['size']

# Ispezione con Boxplot condizionato per genere
df.boxplot(column=['bpp'], by='sex')
```

```
plt.title('Spesa Media per Persona per Genere')
plt.suptitle('')
plt.show()
```

5.4 Feature Transformation, Discretizzazione e Encoding

Nota 5.4.1 (Riferimento al Notebook Interattivo)**.** Il codice delle sezioni 5.4 e 5.5 corrisponde al notebook Jupyter interattivo **Notebook 2 (Feature Engineering)**:

`2_feature_engineering_and_data_representation.ipynb`

5.4.1 Discretizzazione con KBinsDiscretizer

La libreria Scikit-Learn fornisce `KBinsDiscretizer` per convertire variabili continue in classi ordinali discrete secondo tre strategie:

```
# Discretizzazione a 4 intervalli con strategie diverse
disc_uniform = KBinsDiscretizer(n_bins=4, encode='ordinal',
    strategy='uniform')     # Equal-Width
disc_quantile = KBinsDiscretizer(n_bins=4, encode='ordinal'
    , strategy='quantile')  # Equal-Frequency
disc_kmeans = KBinsDiscretizer(n_bins=4, encode='ordinal',
    strategy='kmeans')      # Basata su Cluster

df['total_bill_bins'] = disc_uniform.fit_transform(df[['
    total_bill']])
print(df['total_bill_bins'].value_counts())
```

5.4.2 Normalizzazione dei Dati: StandardScaler vs MinMaxScaler

- **StandardScaler**: Trasforma la feature per avere media zero e deviazione standard unitaria ($\mu = 0, \sigma = 1$).
- **MinMaxScaler**: Riscala linearmente l'intervallo originario entro $[0, 1]$.

```
num_cols = ['total_bill', 'tip', 'age']

# Standardizzazione Z-Score
scaler_std = StandardScaler()
```

```
df_standardized = df.copy()
df_standardized[num_cols] = scaler_std.fit_transform(df[
    num_cols])

# Normalizzazione Min-Max
scaler_minmax = MinMaxScaler()
df_normalized = df.copy()
df_normalized[num_cols] = scaler_minmax.fit_transform(df[
    num_cols])

# Ripristino dei valori originari tramite trasformazione
    inversa
df_recovered = scaler_minmax.inverse_transform(
    df_normalized[num_cols])
```

5.4.3 Codifica di Variabili Categoriche: One-Hot vs Target Encoding

```
# 1. One-Hot Encoding (evita di creare ordinalita fittizia)
ohe = OneHotEncoder(handle_unknown='ignore', sparse_output=
    False)
encoded_days = ohe.fit_transform(df[['day']])
encoded_df = pd.DataFrame(encoded_days, columns=ohe.
    get_feature_names_out(['day']))

# 2. Target Encoding (sostituisce la categoria con la media
    della target)
target_enc = TargetEncoder(smooth='auto')
# Codifica 'day' in base all'importo medio della mancia '
    tip'
df['day_target_encoded'] = target_enc.fit_transform(df[['
    day']], df['tip'])
```

5.5 Riduzione Dimensionale: PCA e t-SNE

5.5.1 Principal Component Analysis (PCA)

Prima di applicare la PCA è indispensabile standardizzare le feature continue affinché attributi con varianze ampie non dominino la decomposizione spettrale:

```
# Selezione attributi quantitativi
X = df[['total_bill', 'tip', 'size', 'age', 'tip_ratio', '
    bpp']].dropna()
```

```
X_scaled = StandardScaler().fit_transform(X)

# Esecuzione della PCA
pca = PCA()
X_pca = pca.fit_transform(X_scaled)

# Autovalori (quota di varianza spiegata per componente)
eigenvalues = pca.explained_variance_
# Percentuale di varianza spiegata
explained_var_ratio = pca.explained_variance_ratio_
# Varianza cumulata
cumulative_var = np.cumsum(explained_var_ratio)
```

Costruzione dello Scree Plot

Lo Scree Plot consente di individuare visivamente il "gomito" (*elbow*) oltre il quale le componenti spiegano quote trascurabili di varianza:

```
plt.figure(figsize=(8, 4))
plt.plot(range(1, len(eigenvalues) + 1), eigenvalues, 'o-',
    color='navy', label='Autovalori singoli')
plt.title('Scree Plot degli Autovalori (PCA)')
plt.xlabel('Numero Componente Principale')
plt.ylabel('Autovalore (Varianza spiegata)')
plt.grid(True, linestyle='--', alpha=0.5)
plt.legend()
plt.show()
```

5.5.2 Proiezione Non Lineare con t-SNE

Il **t-Distributed Stochastic Neighbor Embedding (t-SNE)** è una tecnica non supervisionata non lineare specificamente progettata per preservare le strutture di vicinato locale nei dati ad alta dimensionalità:

```
tsne = TSNE(
    n_components=2,
    perplexity=30,      # Equilibrio tra granularita locale
        e globale (tipicamente 5-50)
    learning_rate=200,  # Passo di ottimizzazione della
        discesa del gradiente
    max_iter=1000,      # Iterazioni di minimizzazione
        della divergenza KL
    random_state=42
)
X_tsne = tsne.fit_transform(X_scaled)
```

5.5.3 Confronto Metodologico tra PCA e t-SNE

Caratteristica	PCA (Principal Component Analysis)	t-SNE (t-Distributed SNE)
Tipologia	Lineare, globale, deterministica.	Non lineare, locale, probabilistica/stocastica.
Funzione Obiettivo	Massimizza la varianza globale proiettata (decomposizione agli autovalori).	Minimizza la divergenza di Kullback-Leibler tra distribuzioni di probabilità di vicinato.
Interpretabilità	Gli assi rappresentano combinazioni lineari degli attributi (pesi/loading noti).	Gli assi non hanno significato fisico; contano solo le distanze relative tra punti.
Distanze Lontane	Preserva fedelmente le distanze euclidee su larga scala.	Non preserva le distanze globali tra cluster distanti (la grandezza dei cluster non riflette la densità originaria).
Uso Tipico	Riduzione preliminare di feature per modelli predittivi, compressione, eliminazione di collinearità.	Visualizzazione esplorativa avanzata e separazione visiva di cluster complessi (2D/3D).

Tabella 5.1: Confronto metodologico tra PCA e t-SNE nell'analisi dei dati.

```
# Visualizzazione comparata delle due proiezioni bidimensionali
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# Scatter PCA
ax1.scatter(X_pca[:, 0], X_pca[:, 1], c='navy', alpha=0.7, edgecolors='k')
ax1.set_title('Proiezione Lineare PCA (PC1 vs PC2)')
ax1.set_xlabel('Prima Componente Principale')
ax1.set_ylabel('Seconda Componente Principale')
ax1.grid(True, linestyle='--', alpha=0.5)

# Scatter t-SNE
ax2.scatter(X_tsne[:, 0], X_tsne[:, 1], c='crimson', alpha=0.7, edgecolors='k')
ax2.set_title('Proiezione Non Lineare t-SNE')
```

```
14  ax2.set_xlabel('Dimensione t-SNE 1')
15  ax2.set_ylabel('Dimensione t-SNE 2')
16  ax2.grid(True, linestyle='--', alpha=0.5)
17
18  plt.tight_layout()
19  plt.show()
```

Capitolo 6

Misure di Prossimità, Teoria dell'Informazione e Cluster Analysis

6.1 Fondamenti di Similarità, Dissimilarità e Prossimità

Nelle discipline di Data Mining, Machine Learning e Information Retrieval, il confronto tra coppie di entità informative costituisce il nucleo operativo di molteplici paradigmi analitici: dalla classificazione supervisionata mediante istanze vicine (k-Nearest Neighbors), all'estrazione di cluster naturali, fino alla stima di densità e anomalie.

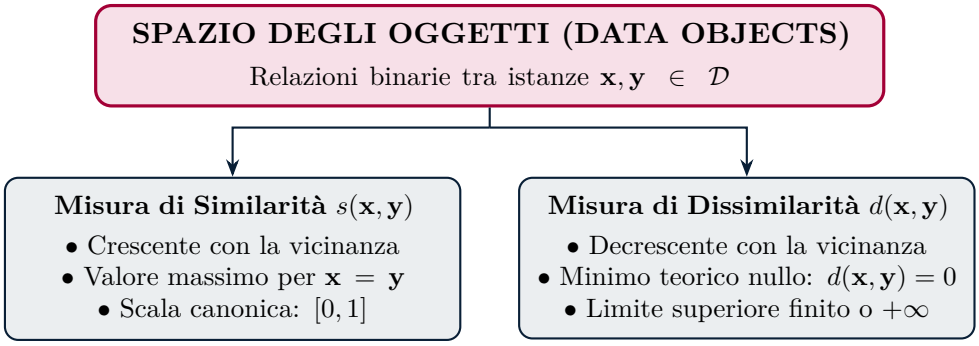


Figura 6.1: Quadro concettuale delle misure di prossimità nello spazio dei dati.

Definizione 6.1.1 (Misura di Similarità)**.** Una **misura di similarità** è una funzione reale $s : \mathcal{D} \times \mathcal{D} \to \mathbb{R}$ che quantifica il grado di affinità tra

due oggetti $\mathbf{x}$ e $\mathbf{y}$. Tale valore è monotonicamente crescente all'aumentare della vicinanza tra gli oggetti: assume il suo valore massimo quando i due oggetti sono identici e decresce man mano che divergono. Nella maggior parte dei domini, la funzione è normalizzata nell'intervallo $[0, 1]$, in cui 1 denota coincidenza assoluta e 0 disgiunzione totale.

Definizione 6.1.2 (Misura di Dissimilarità)**.** Una **misura di dissimilarità** è una funzione reale $d : \mathcal{D} \times \mathcal{D} \to \mathbb{R}$ che quantifica la differenza o discrepanza tra due oggetti $\mathbf{x}$ e $\mathbf{y}$. Assume valori più bassi quando gli oggetti sono tra loro affini o identici, e cresce monotonicamente all'aumentare della loro diversità. Il limite inferiore è convenzionalmente fissato a $d(\mathbf{x}, \mathbf{y}) = 0$ quando $\mathbf{x} = \mathbf{y}$, mentre il limite superiore può essere limitato (1) oppure illimitato ($+\infty$).

Il termine ombrello **prossimità** (*proximity*) è impiegato nella letteratura per riferirsi indistintamente a una funzione di similarità o di dissimilarità.

6.2 Prossimità Unidimensionali per Singolo Attributo

Dati due oggetti $\mathbf{x}$ e $\mathbf{y}$, sia p il valore assunto da $\mathbf{x}$ su un dato attributo e q il valore assunto da $\mathbf{y}$ sul medesimo attributo. Il calcolo della dissimilarità elementare $d(p, q)$ e della duale similarità $s(p, q)$ dipende dalla scala di misurazione dell'attributo.

Scala Attributo	**Dissimilarità** $d(p, q)$	**Similarità Duale** $s(p, q)$
Nominale	$d = 0$ se $p = q$; $d = 1$ se $p \neq q$	$s = 1$ se $p = q$; $s = 0$ se $p \neq q$
Ordinale	$d = \dfrac{\lvert p - q \rvert}{n - 1}$ (con stati mappati in $\{0, \ldots, n-1\}$)	$s = 1 - d = 1 - \dfrac{\lvert p - q \rvert}{n - 1}$
Intervallo o Rapporto	$d = \lvert p - q \rvert$	$s = -d$, $s = \dfrac{1}{1+d}$, oppure $s = 1 - \dfrac{d - d_{\min}}{d_{\max} - d_{\min}}$

Tabella 6.1: Regole di calcolo della prossimità elementare per singoli attributi.

6.2.1 Attributi Nominali

Poiché gli attributi qualitativi nominali (es. colore, tipologia di conto bancario) non presentano relazioni di ordinamento o metrica intrinseca, le uniche operazioni lecite sono il test di identità e diversità:

$$d(p,q) = \begin{cases} 0 & \text{se } p = q \\ 1 & \text{se } p \neq q \end{cases}, \qquad s(p,q) = \begin{cases} 1 & \text{se } p = q \\ 0 & \text{se } p \neq q \end{cases} \tag{6.1}$$

6.2.2 Attributi Ordinali

Gli attributi ordinali presentano un ordinamento intrinseco composto da n livelli discreti. Per calcolare distanze coerenti, si esegue preliminarmente la **mappatura posizionale** degli stati discreti nell'insieme degli interi $\{0, 1, \ldots, n-1\}$.

La differenza assoluta $|p - q|$ varia da 0 a un massimo di $n - 1$. Normalizzando sullo scarto massimo ammissibile, si ottiene:

$$d(p,q) = \frac{|p-q|}{n-1}, \qquad s(p,q) = 1 - d(p,q) = 1 - \frac{|p-q|}{n-1} \tag{6.2}$$

Esempio 6.2.1 (Similarità tra Giudizi Ordinali)**.** Si consideri una scala di valutazione a $n = 5$ livelli:

$$\{\text{insufficiente} = 0,\ \text{sufficiente} = 1,\ \text{buono} = 2,\ \text{distinto} = 3,\ \text{ottimo} = 4\} \tag{6.3}$$

Dati due individui con valutazioni $p = 1$ (*sufficiente*) e $q = 4$ (*ottimo*):

$$d(1,4) = \frac{|1-4|}{5-1} = \frac{3}{4} = 0.75 \implies s(1,4) = 1 - 0.75 = 0.25 \tag{6.4}$$

6.2.3 Attributi ad Intervallo e a Rapporto

Per gli attributi numerici continui o discreti, la metrica elementare coincide con lo scarto assoluto lineare $d(p,q) = |p - q|$. La riconduzione a una misura di similarità normalizzata può seguire tre approcci:

1. **Opposto Aritmetico** (illimitato superiormente): $s(p,q) = -|p - q|$;
2. **Inversione Asintotica**: $s(p,q) = \dfrac{1}{1 + |p-q|}$, con $s \to 1$ quando $d \to 0$, e $s \to 0$ quando $d \to \infty$;
3. **Normalizzazione Min-Max Lineare**: $s(p,q) = 1 - \dfrac{d(p,q) - d_{\min}}{d_{\max} - d_{\min}}$, in cui $d_{\min} = 0$ e $d_{\max}$ denota la massima estensione osservata.

6.3 Distanze Vettoriali e Metriche di Minkowski

Quando gli oggetti sono descritti da vettori multidimensionali $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$, la dissimilarità viene calcolata aggregando gli scarti dimensionali lungo le n coordinate ortogonali.

Definizione 6.3.1 (Distanza di Minkowski (Norma L_r))**.** Dati due vettori $\mathbf{x} = (x_1, \ldots, x_n)$ e $\mathbf{y} = (y_1, \ldots, y_n)$ in $\mathbb{R}^n$, la **distanza di Minkowski** con parametro $r \geq 1$ è definita come:

$$d(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|_r = \left(\sum_{k=1}^{n} |x_k - y_k|^r \right)^{1/r} \tag{6.5}$$

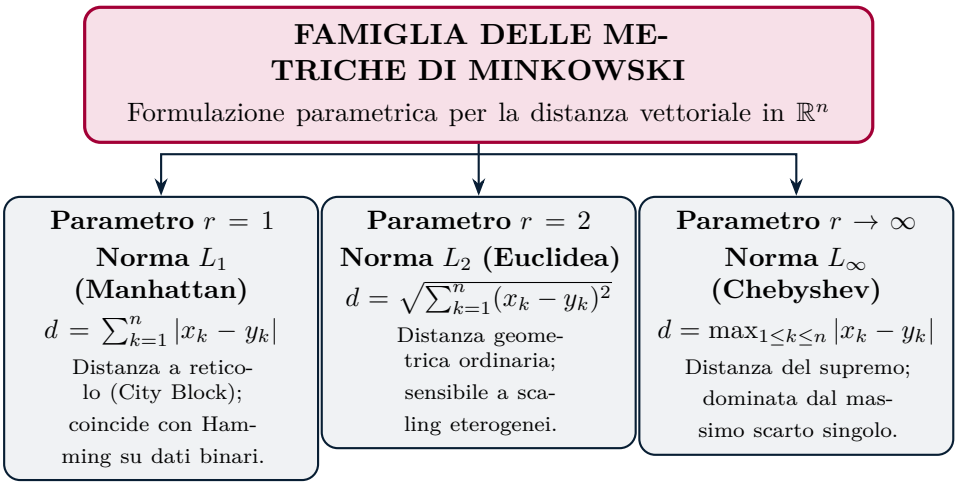


Figura 6.2: Struttura della famiglia delle metriche di Minkowski al variare del parametro r.

1. **Distanza di Manhattan ($r = 1$, Norma L_1, City Block)**:

$$d_{L_1}(\mathbf{x}, \mathbf{y}) = \sum_{k=1}^{n} |x_k - y_k| \tag{6.6}$$

   Modella il tragitto percorso lungo percorsi ortogonali. Su vettori binari coincide rigorosamente con la **distanza di Hamming**.

2. **Distanza Euclidea ($r = 2$, Norma L_2)**:

$$d_{L_2}(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{k=1}^{n} (x_k - y_k)^2} \tag{6.7}$$

Corrisponde alla lunghezza del segmento retto nello spazio euclideo. Se le variabili presentano ordini di grandezza eterogenei, è essenziale standardizzarle preventivamente con z-score per evitare la dominanza delle feature ad alta varianza.

3. **Distanza del Supremo ($r \to \infty$, Norma L_∞, Distanza di Chebyshev)**:

$$d_{L_\infty}(\mathbf{x}, \mathbf{y}) = \lim_{r \to \infty} \left(\sum_{k=1}^{n} |x_k - y_k|^r \right)^{1/r} = \max_{1 \le k \le n} |x_k - y_k| \tag{6.8}$$

Estrae la massima deviazione osservata lungo una singola componente.

6.4 Esempio di Calcolo delle Matrici di Distanza

Si consideri l'insieme di 4 punti bidimensionali analizzato didatticamente:

$$\mathbf{p}_1 = (0, 2), \quad \mathbf{p}_2 = (2, 0), \quad \mathbf{p}_3 = (3, 1), \quad \mathbf{p}_4 = (5, 1) \tag{6.9}$$

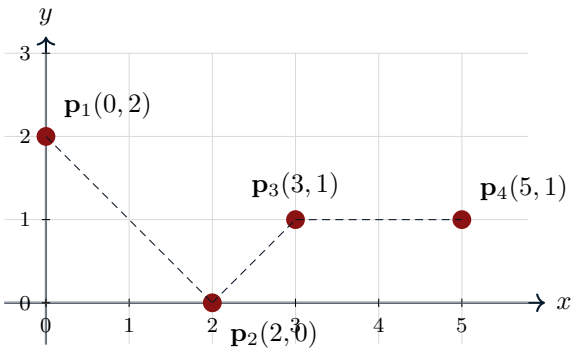


Figura 6.3: Disposizione geometrica dei quattro punti campione nel piano cartesiano.

Il calcolo delle distanze analitiche per tutte le coppie distinte genera i seguenti valori:

- $(\mathbf{p}_1, \mathbf{p}_2)$: $\Delta x = 2$, $\Delta y = 2 \implies L_1 = 4$, $L_2 = \sqrt{8} \approx 2.828$, $L_\infty = 2$;
- $(\mathbf{p}_1, \mathbf{p}_3)$: $\Delta x = 3$, $\Delta y = 1 \implies L_1 = 4$, $L_2 = \sqrt{10} \approx 3.162$, $L_\infty = 3$;
- $(\mathbf{p}_1, \mathbf{p}_4)$: $\Delta x = 5$, $\Delta y = 1 \implies L_1 = 6$, $L_2 = \sqrt{26} \approx 5.099$, $L_\infty = 5$;
- $(\mathbf{p}_2, \mathbf{p}_3)$: $\Delta x = 1$, $\Delta y = 1 \implies L_1 = 2$, $L_2 = \sqrt{2} \approx 1.414$, $L_\infty = 1$;

- $(\mathbf{p}_2, \mathbf{p}_4)$: $\Delta x = 3$, $\Delta y = 1 \implies L_1 = 4$, $L_2 = \sqrt{10} \approx 3.162$, $L_\infty = 3$;
- $(\mathbf{p}_3, \mathbf{p}_4)$: $\Delta x = 2$, $\Delta y = 0 \implies L_1 = 2$, $L_2 = \sqrt{4} = 2.000$, $L_\infty = 2$.

Poiché ogni metrica soddisfa riflessività ($d(\mathbf{p}_i, \mathbf{p}_i) = 0$) e simmetria ($d(\mathbf{p}_i, \mathbf{p}_j) = d(\mathbf{p}_j, \mathbf{p}_i)$), le matrici risultanti assumono forma simmetrica a diagonale principale nulla:

$$\mathbf{D}_{L_1} = \begin{bmatrix} 0 & 4 & 4 & 6 \\ 4 & 0 & 2 & 4 \\ 4 & 2 & 0 & 2 \\ 6 & 4 & 2 & 0 \end{bmatrix}, \quad \mathbf{D}_{L_\infty} = \begin{bmatrix} 0 & 2 & 3 & 5 \\ 2 & 0 & 1 & 3 \\ 3 & 1 & 0 & 2 \\ 5 & 3 & 2 & 0 \end{bmatrix}, \tag{6.10}$$

$$\mathbf{D}_{L_2} = \begin{bmatrix} 0 & 2.828 & 3.162 & 5.099 \\ 2.828 & 0 & 1.414 & 3.162 \\ 3.162 & 1.414 & 0 & 2.000 \\ 5.099 & 3.162 & 2.000 & 0 \end{bmatrix} \tag{6.11}$$

6.5 Proprietà Matematiche di Metriche e Similarità

Definizione 6.5.1 (Assiomi di una Metrica di Distanza)**.** Una funzione di dissimilarità $d : \mathcal{D} \times \mathcal{D} \to \mathbb{R}$ è qualificata formalmente come **metrica** se e solo se soddisfa tre assiomi per ogni terna di punti $\mathbf{x}, \mathbf{y}, \mathbf{z} \in \mathcal{D}$:

1. **Definitezza Positiva**:

$$d(\mathbf{x}, \mathbf{y}) \geq 0 \quad \forall \mathbf{x}, \mathbf{y}; \qquad d(\mathbf{x}, \mathbf{y}) = 0 \iff \mathbf{x} = \mathbf{y} \tag{6.12}$$

2. **Simmetria**:

$$d(\mathbf{x}, \mathbf{y}) = d(\mathbf{y}, \mathbf{x}) \quad \forall \mathbf{x}, \mathbf{y} \tag{6.13}$$

3. **Disuguaglianza Triangolare**:

$$d(\mathbf{x}, \mathbf{z}) \leq d(\mathbf{x}, \mathbf{y}) + d(\mathbf{y}, \mathbf{z}) \quad \forall \mathbf{x}, \mathbf{y}, \mathbf{z} \tag{6.14}$$

Tutte le metriche di Minkowski ($r \geq 1$) soddisfano rigorosamente i tre assiomi (la disuguaglianza triangolare discende direttamente dalla disuguaglianza di Minkowski per integrali e sommatorie).

Analogamente, una funzione di similarità $s : \mathcal{D} \times \mathcal{D} \to \mathbb{R}$ soddisfa:

1. **Identità del Massimo**: $s(\mathbf{x}, \mathbf{y}) = 1$ (o valore massimo ammesso) se e solo se $\mathbf{x} = \mathbf{y}$;
2. **Simmetria**: $s(\mathbf{x}, \mathbf{y}) = s(\mathbf{y}, \mathbf{x})$ per ogni coppia $\mathbf{x}, \mathbf{y}$.

6.6 Prossimità tra Vettori Binari: SMC vs Indice di Jaccard

Negli spazi descritti da vettori binari asimmetrici (come le transazioni di acquisto o la presenza di vocaboli in documenti), la quantificazione della concordanza richiede la distinzione tra la presenza (1) e l'assenza (0) di attributi.

Dati due vettori binari **p** e **q**, le frequenze congiunte delle configurazioni di bit definiscono la tabella di contingenza 2×2:

		Oggetto q	
		1	0
Oggetto p	1	M_{11} (presenza congiunta)	M_{10} (presente solo in **p**)
	0	M_{01} (presente solo in **q**)	M_{00} (assenza congiunta)

Tabella 6.2: Tabella di contingenza per vettori di attributi binari.

Il numero totale di attributi binari considerati coincide con la cardinalità:

$$n = M_{11} + M_{10} + M_{01} + M_{00} \tag{6.15}$$

6.6.1 Definizione dei Coefficienti

1. **Simple Matching Coefficient (SMC)**:

$$\text{SMC} = \frac{M_{11} + M_{00}}{M_{01} + M_{10} + M_{11} + M_{00}} \tag{6.16}$$

   Assegna pari dignità informativa alle concordanze positive (M_{11}) e a quelle negative (M_{00}). È appropriato per attributi simmetrici (ad es. sesso biologico o risposte binarie bilanciate).

2. **Coefficiente di Jaccard (J)**:

$$J = \frac{M_{11}}{M_{01} + M_{10} + M_{11}} \tag{6.17}$$

   Tratta l'attributo come asimmetrico: ignora deliberatamente le concordanze negative (M_{00}).

Perché l'Indice di Jaccard è Indispensabile nei Dati Sparsi

Nei problemi ad elevata dimensionalità e forte sparsità (come l'analisi dei carrelli della spesa o il recupero testuale), due transazioni condividono l'assenza di decine di migliaia di articoli o vocaboli. Se si impiegasse il coefficiente SMC, il termine M_{00} dominerebbe numericamente la frazione, portando a stimare una similarità artificiosamente elevata (SMC $\to 1$) per il solo fatto che i due profili non presentano la quasi totalità degli item. L'indice di Jaccard evita questa distorsione vincolando l'analisi agli attributi effettivamente attivi.

Esempio 6.6.1 (Confronto SMC vs Jaccard su Vettori Sparsi)**.** Siano dati due vettori binari con $n = 10$ attributi:

$$\mathbf{p} = [1, 0, 0, 0, 0, 0, 0, 0, 0, 0] \tag{6.18}$$
$$\mathbf{q} = [0, 0, 0, 0, 0, 0, 1, 0, 0, 1] \tag{6.19}$$

Dall'ispezione delle posizioni si ottiene:

$$M_{11} = 0, \quad M_{10} = 1, \quad M_{01} = 2, \quad M_{00} = 7 \tag{6.20}$$

I coefficienti risultano rispettivamente:

$$\text{SMC} = \frac{0+7}{2+1+0+7} = \frac{7}{10} = 0.70, \qquad J = \frac{0}{2+1+0} = \frac{0}{3} = 0.00 \tag{6.21}$$

L'SMC riporta un'elevata affinità (70%) dovuta alla sola condivisione di zeri, mentre Jaccard certifica l'assoluta assenza di elementi comuni ($J = 0$).

6.7 Similarità Coseno per Dati Documentali

Nel modello a spazio vettoriale (*Vector Space Model*) tipico dell'Information Retrieval, i documenti vengono rappresentati come vettori multidimensionali di conteggio o pesi TF-IDF nello spazio lessicale dei termini.

In tale scenario, testi di lunghezze eterogenee che affrontano il medesimo tema risulterebbero distanti se valutati con la metrica Euclidea, a causa della diversa norma dei vettori. La **Similarità Coseno** isola l'orientamento angolare reciproco, prescindendo dalla magnitudine:

$$\cos(\mathbf{d}_1, \mathbf{d}_2) = \frac{\mathbf{d}_1 \cdot \mathbf{d}_2}{\|\mathbf{d}_1\|_2 \|\mathbf{d}_2\|_2} = \frac{\sum_{k=1}^{n} d_{1,k} d_{2,k}}{\sqrt{\sum_{k=1}^{n} d_{1,k}^2} \sqrt{\sum_{k=1}^{n} d_{2,k}^2}} \tag{6.22}$$

Esempio 6.7.1 (Calcolo Guidato su Vettori Documentali)**.** Siano dati i vettori dei termini:

$$\mathbf{d}_1 = [3, 2, 0, 5, 0, 0, 0, 2, 0, 0] \tag{6.23}$$

$$\mathbf{d}_2 = [1, 0, 0, 0, 0, 0, 0, 1, 0, 2] \tag{6.24}$$

1. **Prodotto Scalare**:

$$\mathbf{d}_1 \cdot \mathbf{d}_2 = (3 \times 1) + (2 \times 0) + (5 \times 0) + (2 \times 1) + (0 \times 2) = 3 + 2 = 5 \tag{6.25}$$

2. **Norme Euclidee**:

$$\|\mathbf{d}_1\|_2 = \sqrt{3^2 + 2^2 + 5^2 + 2^2} = \sqrt{9 + 4 + 25 + 4} = \sqrt{42} \approx 6.4807 \tag{6.26}$$

$$\|\mathbf{d}_2\|_2 = \sqrt{1^2 + 1^2 + 2^2} = \sqrt{1 + 1 + 4} = \sqrt{6} \approx 2.4495 \tag{6.27}$$

3. **Similarità Coseno**:

$$\cos(\mathbf{d}_1, \mathbf{d}_2) = \frac{5}{\sqrt{42} \times \sqrt{6}} = \frac{5}{\sqrt{252}} = \frac{5}{15.8745} \approx 0.3150 \tag{6.28}$$

I documenti condividono il 31.5% di orientamento tematico congiunto.

6.8 Ponderazione, Dati Mancanti e Correlazione tra Oggetti

6.8.1 Similarità Ponderata con Gestione di Dati Mancanti

Qualora le feature posseggano rilevanze differenziate o contengano valori nulli, si introduce un vettore di pesi non negativi $\{\omega_k\}_{k=1}^n$ e una variabile indicatrice binaria $\delta_k \in \{0, 1\}$ che vale 1 se e solo se la coordinata k-esima è definita sia per $\mathbf{x}$ sia per $\mathbf{y}$:

$$\text{similarity}(\mathbf{x}, \mathbf{y}) = \frac{\sum_{k=1}^n \omega_k \delta_k s_k(\mathbf{x}, \mathbf{y})}{\sum_{k=1}^n \omega_k \delta_k} \tag{6.29}$$

In modo analogo, la **distanza di Minkowski ponderata** è espressa da:

$$d(\mathbf{x}, \mathbf{y}) = \left(\sum_{k=1}^n \omega_k |x_k - y_k|^r \right)^{1/r} \tag{6.30}$$

6.8.2 Coefficiente di Correlazione tra Oggetti

Il coefficiente di Pearson può essere applicato trasversalmente per quantificare la prossimità tra coppie di vettori profilo $\mathbf{p}$ e $\mathbf{q}$:

$$\text{corr}(\mathbf{p}, \mathbf{q}) = \frac{\sum_{k=1}^{n}(p_k - \bar{p})(q_k - \bar{q})}{\sqrt{\sum_{k=1}^{n}(p_k - \bar{p})^2}\sqrt{\sum_{k=1}^{n}(q_k - \bar{q})^2}} = \mathbf{p}^* \cdot \mathbf{q}^* \tag{6.31}$$

dove $\mathbf{p}^*$ e $\mathbf{q}^*$ sono i profili standardizzati (a media nulla e varianza unitaria lungo le coordinate).

6.9 Teoria dell'Informazione: Entropia e Informazione Mutua

La teoria dell'informazione formalizzata da Claude Shannon (1948) offre gli strumenti analitici per quantificare l'incertezza, il disordine e la dipendenza non lineare tra variabili discrete o categoriche.

6.9.1 Informazione Propria ed Entropia di Shannon

L'informazione associata a un evento aleatorio x_i con probabilità $p_i = P(X = x_i)$ corrisponde alla sorpresa legata al suo verificarsi:

$$I(x_i) = \log_2\left(\frac{1}{p_i}\right) = -\log_2(p_i) \tag{6.32}$$

L'**Entropia di Shannon** $H(X)$ rappresenta il valore atteso dell'informazione propria:

$$H(X) = \mathbb{E}[I(X)] = -\sum_{i=1}^{n} p_i \log_2(p_i) \tag{6.33}$$

con la convenzione di continuità $\lim_{p \to 0^+} p \log_2(p) = 0$.

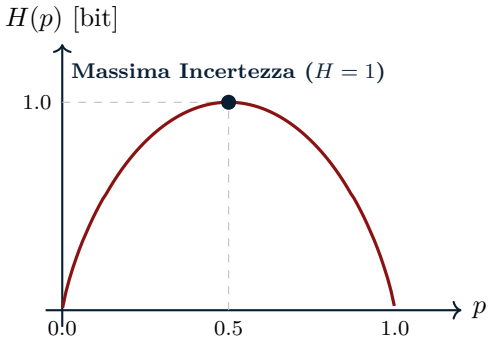


Figura 6.4: Curva dell'entropia di Shannon per una variabile di Bernoulli in funzione della probabilità di successo p.

L'entropia è limitata: $0 \leq H(X) \leq \log_2(n)$. Si annulla se e solo se la variabile è puramente deterministica ($p_k = 1$), mentre raggiunge il massimo teorico $\log_2(n)$ in presenza di distribuzione uniforme ($p_i = 1/n$).

6.9.2 Informazione Mutua (Mutual Information)

L'**Informazione Mutua** $I(X;Y)$ quantifica la quota di incertezza condivisa tra due variabili aleatorie (ossia la riduzione dell'incertezza su X derivante dalla conoscenza esatta di Y):

$$I(X;Y) = H(X) + H(Y) - H(X,Y) \tag{6.34}$$

in cui $H(X,Y)$ rappresenta l'entropia congiunta definita sulle probabilità congiunte $p_{ij} = P(X = x_i, Y = y_j)$:

$$H(X,Y) = -\sum_{i=1}^{n_X} \sum_{j=1}^{n_Y} p_{ij} \log_2(p_{ij}) \tag{6.35}$$

Proprietà fondamentali:

1. $I(X;Y) \geq 0$ e $I(X;Y) = I(Y;X)$ (non negatività e simmetria);
2. $I(X;Y) = 0 \iff P(X,Y) = P(X)P(Y)$ (annullamento sotto indipendenza statistica);
3. $I(X;Y) \leq \min\Big(H(X), H(Y)\Big) \leq \log_2(\min(n_X, n_Y))$.

6.9.3 Esempio Numerico di Calcolo dell'Informazione Mutua

Si consideri il campione di $m = 100$ studenti universitari analizzato didatticamente, in cui si confronta lo **Status Accademico** ($X \in \{\text{Undergrad}, \text{Grad}\}$) con il **Voto** conseguito ($Y \in \{A, B, C\}$).

Status (X)	**Voto A**	**Voto B**	**Voto C**	**Totale (X)**
Undergrad	5 (0.05)	30 (0.30)	10 (0.10)	45 (0.45)
Grad	30 (0.30)	20 (0.20)	5 (0.05)	55 (0.55)
Totale (Y)	35 (0.35)	50 (0.50)	15 (0.15)	100 (1.00)

Tabella 6.3: Distribuzione di frequenza congiunta e marginale per lo status e i voti accademici.

1. **Entropia Marginale di X**:

$$H(X) = -0.45 \log_2(0.45) - 0.55 \log_2(0.55) \approx 0.9928 \text{ bit} \tag{6.36}$$

2. **Entropia Marginale di Y**:

$$\begin{aligned} H(Y) &= -0.35 \log_2(0.35) - 0.50 \log_2(0.50) - 0.15 \log_2(0.15) \\ &\approx 0.5301 + 0.5000 + 0.4105 = 1.4406 \text{ bit} \end{aligned} \tag{6.37}$$

3. **Entropia Congiunta $H(X, Y)$**:

$$\begin{aligned} H(X,Y) &= -\sum_{i=1}^{2} \sum_{j=1}^{3} p_{ij} \log_2(p_{ij}) \\ &\approx 2(0.2161) + 2(0.5211) + 0.3322 + 0.4644 = 2.2710 \text{ bit} \end{aligned} \tag{6.38}$$

4. **Informazione Mutua**:

$$\begin{aligned} I(X;Y) &= H(X) + H(Y) - H(X,Y) \\ &= 0.9928 + 1.4406 - 2.2710 = 0.1624 \text{ bit} \end{aligned} \tag{6.39}$$

La conoscenza dello status dello studente apporta una riduzione netta dell'incertezza sulla votazione pari a 0.1624 bit (l'11.27% dell'entropia originaria della variabile voto).

6.10 Principi Fondamentali della Cluster Analysis

La **Cluster Analysis** (o analisi dei gruppi) è il processo computazionale non supervisionato preposto alla partizione di un dataset $\mathcal{D} = \{\mathbf{x}_1, \ldots, \mathbf{x}_N\}$ in sottoinsiemi omogenei (detti **cluster**) in base a due principi cardine:

1. **Massimizzazione dell'Omogeneità Intra-Cluster**: Gli oggetti allocati nel medesimo raggruppamento devono presentare elevata coesione interna e prossimità reciproca;

2. **Massimizzazione della Separazione Inter-Cluster**: Oggetti afferenti a cluster distinti devono presentare massima dissimilarità e separazione metrica.

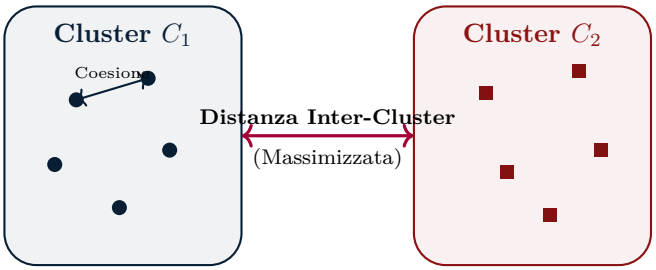


Figura 6.5: Dicotomia geometrica tra coesione interna e separazione esterna nella cluster analysis.

A differenza dei modelli supervisionati, il clustering non fa affidamento su alcuna variabile target Y prestabilita; l'architettura relazionale emerge autonomamente dalla geometria dello spazio campionario.

6.10.1 Ambiti Applicativi: Comprensione vs Sintesi

Le motivazioni applicative del clustering si distinguono in due polarità:

1. **Comprensione dei Dati (*Understanding*)**: Scoperta di strutture e tassonomie latenti:

   - *Information Retrieval*: Organizzazione tematica automatica di collezioni testuali;
   - *Bioinformatica*: Raggruppamento di geni con profili di coespressione funzionale affini;
   - *Finanza e Mercati*: Identificazione di cluster azionari con rendimenti covarianti (es. comparti tecnologici, finanziari o energetici).

2. **Sintesi dei Dati (*Summarization*)**: Compressione dimensionale quantitativa in cui milioni di misurazioni locali vengono rappresentate per mezzo dei rispettivi prototipi (centroidi o medoidi), preservando le proprietà statistiche complessive.

6.10.2 Cosa NON è la Cluster Analysis

Per evitare ambiguità epistemologiche, è cruciale distinguere il clustering da altri task:

- **Segmentazione Manuale**: La suddivisione di record imposta dall'esterno su criteri deterministici arbitrari (es. ripartizione alfabetica per cognome $A - L$ ed $M - Z$) non riflette pattern naturali;
- **Query su Database**: Clausole SQL dichiarative (es. `WHERE Age > 30`) filtrano sottoinsiemi vincolati a regole rigide specificate dall'utente anziché derivate dalla topologia dei dati;
- **Classificazione**: Minimizza l'errore empirico su etichette supervisionate $y \in \mathcal{Y}$ assegnate a priori;
- **Pattern Mining**: L'estrazione di regole di associazione ricerca correlazioni locali o regole implicative tra insiemi di item, mentre il clustering modella la struttura globale dell'intero spazio geometrico.

6.11 Tassonomie dei Metodi e Strutture di Raggruppamento

6.11.1 Clustering Partizionale vs Gerarchico

1. **Clustering Partizionale**: Suddivide lo spazio in k partizioni mutuamente disgiunte $\mathcal{C} = \{C_1, \dots, C_k\}$ che ricoprono integralmente il dataset:
$$\bigcup_{i=1}^{k} C_i = \mathcal{D}, \qquad C_i \cap C_j = \emptyset \quad \forall i \neq j \tag{6.40}$$
Ogni oggetto appartiene a una e una sola partizione. Il numero di gruppi k deve generalmente essere specificato a priori (es. K-Means).
2. **Clustering Gerarchico**: Costruisce una famiglia annidata di partizioni rappresentabile visivamente come un albero metrico denominato **dendrogramma**, eliminando il vincolo di dover fissare a monte il valore scalare di k.

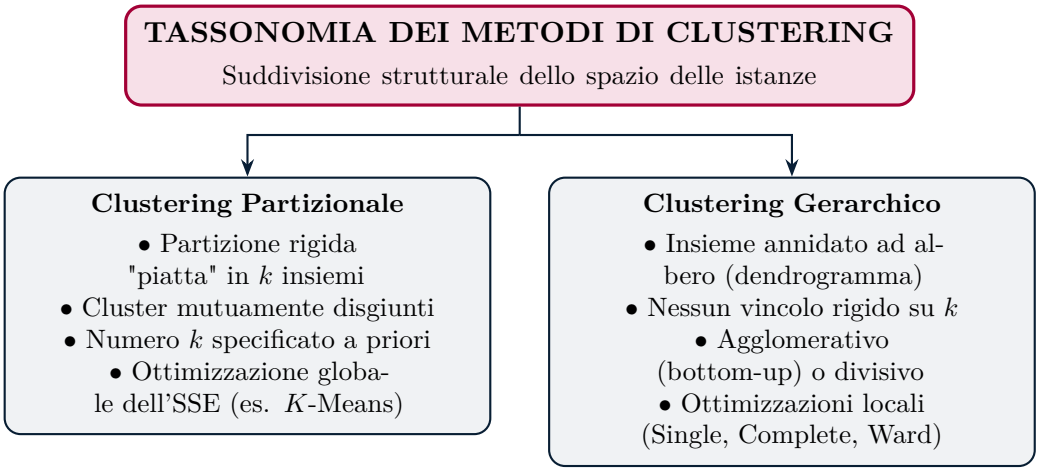


Figura 6.6: Confronto strutturale tra clustering partizionale e clustering gerarchico.

6.11.2 Ulteriori Dimensioni Tassonomiche

- **Esclusivo vs Non-Esclusivo (Overlapping)**: Nel clustering non-esclusivo un punto può appartenere simultaneamente a più raggruppamenti (utile per punti di frontiera o domini multi-argomento).
- **Fuzzy (Soft) vs Non-Fuzzy (Hard)**: Nel clustering fuzzy ogni punto $\mathbf{x}_i$ è associato al cluster C_k mediante un coefficiente di appartenenza continuo $w_{ik} \in [0, 1]$, soggetto al vincolo di conservazione $\sum_{k=1}^{K} w_{ik} = 1$.
- **Parziale vs Completo**: Gli algoritmi parziali isolano i nuclei ad alta coesione e scartano le osservazioni anomale come rumore o outlier.
- **Omogeneo vs Eterogeneo**: Gli algoritmi omogenei assumono a priori volumi e forme sferoidali confrontabili, mentre i metodi eterogenei ammettono forme arbitrarie e densità fortemente dissimili.

6.12 Tipologie di Cluster e Formulazione Matematica

La nozione semantica di "cluster" assume connotazioni analitiche profondamente distinte a seconda della proprietà geometrica prescelta:

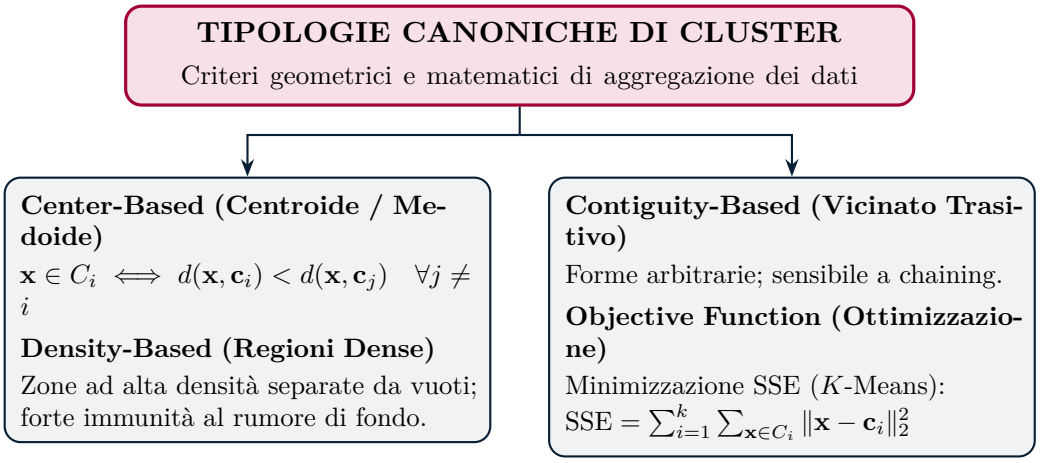


Figura 6.7: Quadro sintetico delle definizioni formali di cluster.

1. **Cluster Ben Separati (*Well-Separated*)**: Ogni punto interno al cluster è più vicino a tutti i membri del proprio gruppo rispetto a qualunque punto esterno:

$$\forall \mathbf{x}, \mathbf{y} \in C_i,\ \forall \mathbf{z} \notin C_i \implies d(\mathbf{x}, \mathbf{y}) < d(\mathbf{x}, \mathbf{z}) \tag{6.41}$$

2. **Cluster Basati sul Centro (*Center-Based*)**: Ciascun oggetto appartiene al cluster il cui punto centrale è più vicino:

$$\mathbf{x} \in C_i \iff d(\mathbf{x}, \mathbf{c}_i) < d(\mathbf{x}, \mathbf{c}_j) \quad \forall j \neq i \tag{6.42}$$

   Il centro può essere il **centroide** (media aritmetica continua $\mathbf{c}_i = \frac{1}{|C_i|} \sum_{\mathbf{x} \in C_i} \mathbf{x}$) oppure il **medoide**, ovvero l'istanza reale più centrale:

$$\mathbf{m}_i = \arg\min_{\mathbf{y} \in C_i} \sum_{\mathbf{x} \in C_i} d(\mathbf{x}, \mathbf{y}) \tag{6.43}$$

3. **Cluster Basati sulla Contiguità (*Contiguity-Based o Graph-Based*)**: Un punto appartiene al cluster se è più vicino ad almeno un elemento del cluster rispetto a qualsiasi punto esterno:

$$\forall \mathbf{x} \in C_i,\ \exists \mathbf{y} \in C_i \setminus \{\mathbf{x}\} \quad \text{t.c.} \quad d(\mathbf{x}, \mathbf{y}) < \min_{\mathbf{z} \notin C_i} d(\mathbf{x}, \mathbf{z}) \tag{6.44}$$

   Consente di modellare geometrie arbitrarie non convesse, ma soffre del grave fenomeno dell'**effetto catena** (*chaining effect*), in cui pochi punti di rumore fanno da ponte fondendo impropriamente cluster separati.

4. **Cluster Basati sulla Densità (*Density-Based*)**: Regioni dense di punti delimitate e separate da aree a bassa densità campionaria (es. DBSCAN). Sono naturalmente resistenti al rumore e non vincolati alla forma sferoidale.

5. **Cluster Definiti da una Funzione Obiettivo**: Formulati come problemi di ottimizzazione matematica. L'individuazione della partizione globale ottima su N punti in k gruppi richiederebbe di testare tutti i numeri di Stirling di seconda specie:

$$S(N,k) = \frac{1}{k!}\sum_{j=0}^{k}(-1)^{k-j}\binom{k}{j}j^N \approx \frac{k^N}{k!} \tag{6.45}$$

   Poiché il problema è intrinsecamente **NP-Hard**, si impiegano euristiche iterative di discesa come in K-Means per minimizzare la somma degli errori quadratici (SSE):

$$\text{SSE} = \sum_{i=1}^{k}\sum_{\mathbf{x}\in C_i}\|\mathbf{x}-\mathbf{c}_i\|_2^2 \tag{6.46}$$

6.13 Caratteristiche dei Dati di Input e Famiglie Algoritmiche

La stabilità e l'efficacia di un processo di clustering dipendono dalle caratteristiche dello spazio di input:

- **Maledizione della Dimensionalità**: Negli spazi $\mathbb{R}^n$ con n elevato, le distanze tra coppie di punti tendono a concentrarsi e ad uniformarsi, degradando il contrasto tra vicini e lontani nella metrica euclidea;
- **Sparsità ed Eterogeneità**: Vettori sparsi richiedono metriche asimmetriche (Jaccard, Coseno); attributi misti necessitano di metriche ibride (Gower);
- **Rumore e Outlier**: Alterano il calcolo dei baricentri nei metodi center-based e innescano fusioni anomale nei metodi contigui.

I tre paradigmi algoritmici fondamentali che derivano da tali premesse sono riassunti come segue:

1. **K-Means e Varianti**: Metodo partizionale, esclusivo e completo. Ottimizza iterativamente la funzione SSE su cluster sferoidali compatti a densità equivalente;

2. **Clustering Gerarchico Agglomerativo**: Metodo gerarchico e nestato. Fonde iterativamente le coppie di gruppi più vicine secondo i criteri di linkage (Single, Complete, Average, Ward), producendo un dendrogramma interpretabile;

3. **Clustering Density-Based (DBSCAN)**: Metodo basato su densità locale, parziale e non parametrico rispetto a k. Isola naturalmente i punti di rumore e identifica cluster di forma geometrica arbitraria.

Capitolo 7

Cluster Analysis: Algoritmi K-Means e Clustering Gerarchico

7.1 Introduzione alla Cluster Analysis Partizionale e Gerarchica

Come introdotto nel Capitolo 6 (sezione 6.10), la **Cluster Analysis** persegue la partizione di un dataset $\mathcal{D} = \{\mathbf{x}_1, \ldots, \mathbf{x}_n\} \subset \mathbb{R}^d$ massimizzando congiuntamente la coesione interna e la separazione esterna tra i gruppi. Sulle basi della tassonomia generale dei metodi delineata nella sezione 6.11, questo capitolo approfondisce i due capisaldi operativi dell'apprendimento non supervisionato: l'approccio partizionale fondato su prototipi (K-Means) e l'approccio gerarchico basato su matrici di prossimità e dendrogrammi.

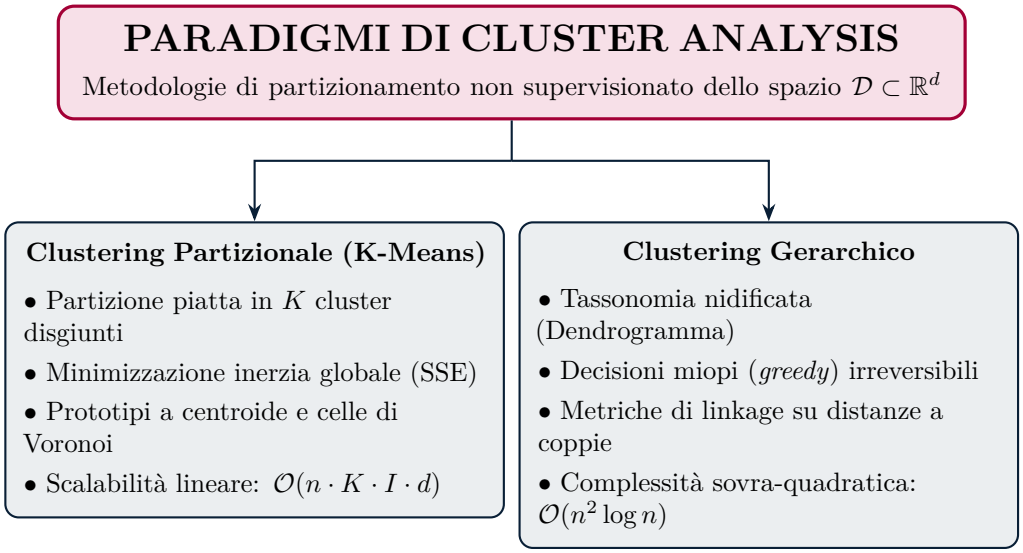


Figura 7.1: Tassonomia dei due principali paradigmi di clustering trattati nel capitolo.

I due paradigmi affrontano l'obiettivo del raggruppamento con filosofie strutturali differenti: il paradigma partizionale fissa a priori il numero di raggruppamenti K e persegue una tassellazione poligonale ottima dello spazio mediante prototipi; il paradigma gerarchico genera una sequenza continua di partizioni nidificate, delegando la determinazione di K a un'analisi a posteriori del dendrogramma.

7.2 K-Means: Algoritmo di Lloyd e Geometria di Voronoi

L'algoritmo **K-Means** rappresenta il capostipite dei metodi partizionali basati su centroide o prototipo. Introdotto storicamente da Hugo Steinhaus nel 1956, formalizzato da Stuart Lloyd (1957/1982) nei contesti di quantizzazione dei segnali, e denominato da James MacQueen (1967), il metodo realizza un partizionamento rigido (*hard clustering*) in cui ogni punto è assegnato univocamente a un solo cluster.

Definizione 7.2.1 (Partizione Hard e Centroide)**.** Data una collezione di n punti $\mathcal{D} \subset \mathbb{R}^d$ e un intero positivo fissato $K \in \mathbb{N}^+$, una partizione rigida è una famiglia di insiemi $\{C_1, \ldots, C_K\}$ tale che:

$$\bigcup_{i=1}^{K} C_i = \mathcal{D}, \qquad C_i \cap C_j = \emptyset \quad (\forall i \neq j), \qquad C_i \neq \emptyset \quad (\forall i \in \{1, \ldots, K\}) \tag{7.1}$$

A ciascun cluster C_i è associato un vettore prototipo denominato **centroide** $\mathbf{m}_i \in \mathbb{R}^d$ (indicato talvolta anche con $\mathbf{c}_i$), che per la distanza euclidea coincide con il baricentro aritmetico dei punti assegnati:

$$\mathbf{m}_i = \frac{1}{|C_i|} \sum_{\mathbf{x} \in C_i} \mathbf{x} \tag{7.2}$$

dove $|C_i|$ denota la cardinalità del cluster C_i.

7.2.1 Pseudocodice dell'Algoritmo di Lloyd

L'algoritmo di base opera alternando ricorsivamente due fasi deterministiche fino a convergenza, come illustrato nell'Algoritmo 1.

Algorithm 1: Algoritmo standard di K-Means (Lloyd).

Dati: Dataset $\mathcal{D} = \{\mathbf{x}_1, \ldots, \mathbf{x}_n\} \subset \mathbb{R}^d$, numero di cluster K, tolleranza $\varepsilon \geq 0$.

Risultato: Partizione finale $\{C_1, \ldots, C_K\}$ e centroidi $\{\mathbf{m}_1, \ldots, \mathbf{m}_K\}$.

Inizializza K centroidi iniziali $\{\mathbf{m}_1^{(0)}, \ldots, \mathbf{m}_K^{(0)}\}$ mediante estrazione casuale da $\mathcal{D}$

$t \leftarrow 0$

repeat

  $t \leftarrow t + 1$

  `// Fase 1: Assegnazione alle celle di Voronoi`

  Inizializza $C_i^{(t)} \leftarrow \emptyset$ per ogni $i \in \{1, \ldots, K\}$

  foreach $\mathbf{x} \in \mathcal{D}$ **do**

    Determina l'indice del centroide più vicino: $k^* = \arg\min_{j \in \{1,\ldots,K\}} \|\mathbf{x} - \mathbf{m}_j^{(t-1)}\|^2$

    Assegna $\mathbf{x}$ al cluster corrispondente: $C_{k^*}^{(t)} \leftarrow C_{k^*}^{(t)} \cup \{\mathbf{x}\}$

  end

  `// Fase 2: Ricalcolo dei centroidi (baricentri)`

  foreach $i \in \{1, \ldots, K\}$ **do**

    if $|C_i^{(t)}| > 0$ **then**

      $\mathbf{m}_i^{(t)} \leftarrow \dfrac{1}{|C_i^{(t)}|} \sum_{\mathbf{x} \in C_i^{(t)}} \mathbf{x}$

    end

    else

      Gestisci cluster vuoto riallocando un centroide orfano

    end

  end

until *i centroidi non variano significativamente:* $\sum_{i=1}^{K} \|\mathbf{m}_i^{(t+1)} - \mathbf{m}_i^{(t)}\|^2 \leq \varepsilon$

7.2.2 Interpretazione Geometrica: Diagramma di Voronoi

La fase di assegnazione partiziona lo spazio euclideo continuo $\mathbb{R}^d$ in K regioni convesse delimitate da iperpiani bisettori, note come **tassellazione di Voronoi** generata dai prototipi $\{\mathbf{m}_1, \ldots, \mathbf{m}_K\}$:

$$V(C_i) = \left\{ \mathbf{x} \in \mathbb{R}^d : \|\mathbf{x} - \mathbf{m}_i\|_2 \leq \|\mathbf{x} - \mathbf{m}_j\|_2 \quad \forall j \neq i \right\} \tag{7.3}$$

Ciascun iperpiano di separazione tra il cluster C_i e il cluster C_j è ortogonale al segmento congiungente $\mathbf{m}_i$ e $\mathbf{m}_j$ e passa per il loro punto medio. Ne consegue che ogni cluster individuato da K-Means è un poliedro convesso: punti appartenenti a regioni non convesse o concave non possono essere raggruppati correttamente senza frammentare la tassellazione.

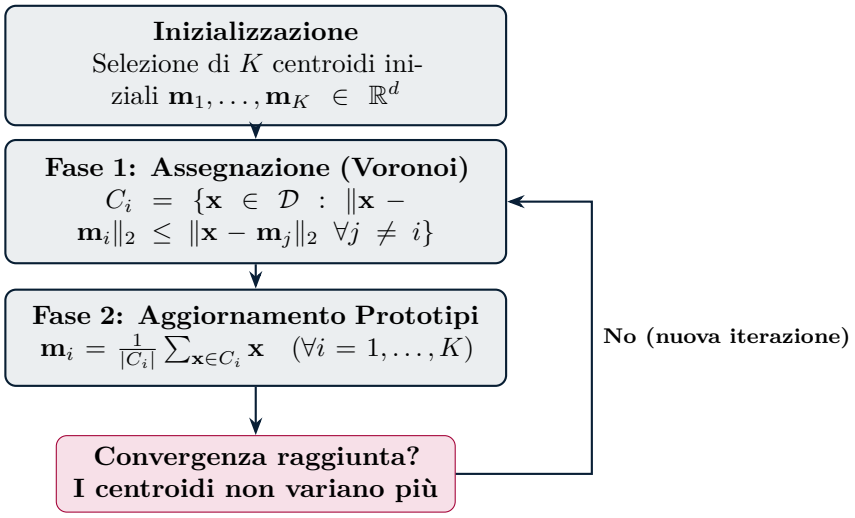


Figura 7.2: Ciclo iterativo di aggiornamento a due fasi dell'algoritmo K-Means.

7.2.3 Dinamica Temporale delle Iterazioni

Studiando l'evoluzione dei centroidi lungo l'asse delle iterazioni $t = 1, 2, \ldots$:

- **Iterazioni 1–3**: Si osserva la migrazione macroscopica più rilevante. I centroidi iniziali si allontanano rapidamente dalle configurazioni spurie e migrano verso i baricentri delle macro-regioni a maggiore densità.
- **Iterazioni 4–6 e successive**: Si entra nella fase di raffinamento fine. Pochi punti di confine vengono riassegnati lungo le superfici di Voronoi finché la variazione dei centroidi si annulla o scende sotto la soglia numerica prefissata ε.

7.2.4 Generalizzazione ad Altre Metriche di Prossimità

Sebbene K-Means sia formalizzato per la distanza euclidea, la struttura a due fasi può essere generalizzata ad altre misure di prossimità, a patto di ridefinire coerentemente la nozione di prototipo:

- **Distanza Euclidea (L_2)**: Prototipo = **Media** (baricentro). Minimizza l'errore quadratico (SSE).
- **Distanza di Manhattan (L_1)**: Prototipo = **Mediana** per componente (algoritmo **K-Medians**). Minimizza la somma degli errori assoluti (SAE = $\sum_i \sum_{\mathbf{x} \in C_i} \|\mathbf{x} - \mathbf{m}_i\|_1$).
- **Similarità Coseno**: Prototipo = **Media normalizzata** a norma unitaria ($\mathbf{m}_i / \|\mathbf{m}_i\|_2$). Massimizza la similarità coseno cumulativa (impiegato nel text mining).
- **Correlazione di Pearson**: Prototipo = **Vettore standardizzato** (media zero, varianza unitaria).

7.2.5 Analisi della Complessità Computazionale

L'algoritmo di Lloyd vanta un'eccellente scalabilità computazionale:

- **Complessità Temporale**: Ad ogni iterazione, la fase di assegnazione confronta ciascuno degli n punti con tutti i K centroidi in dimensione d, richiedendo $\mathcal{O}(n \cdot K \cdot d)$ operazioni. Il ricalcolo dei baricentri richiede la somma dei vettori, con costo $\mathcal{O}(n \cdot d)$. Indicando con I il numero totale di iterazioni fino a convergenza, la complessità asintotica totale è:

$$\mathcal{T}_{\text{K-Means}} = \mathcal{O}(n \cdot K \cdot I \cdot d) \tag{7.4}$$

  Poiché sperimentalmente $K \ll n$, $d \ll n$ e $I \ll n$ (spesso $I \leq 50$), il tempo cresce linearmente con la dimensione del dataset n.

- **Complessità Spaziale**: È sufficiente memorizzare la matrice dei dati $n \times d$ e i K vettori centroide di dimensione d, oltre al vettore degli indici di appartenenza $n \times 1$:

$$\mathcal{S}_{\text{K-Means}} = \mathcal{O}((n + K) \cdot d) \tag{7.5}$$

  rendendo l'algoritmo applicabile a contesti di Big Data in cui l'intero dataset risiede in memoria principale.

7.3 Funzione Obiettivo SSE e Dimostrazione di Convergenza

L'efficacia e la stabilità analitica del K-Means derivano dall'esistenza di una funzione obiettivo scalare rigorosamente minimizzata dall'alternanza delle due fasi di Lloyd.

Definizione 7.3.1 (Sum of Squared Errors (SSE))**.** Data una partizione $\{C_1, \dots, C_K\}$ e i rispettivi centroidi $\{\mathbf{m}_1, \dots, \mathbf{m}_K\}$, la **somma degli errori quadratici** (SSE, o inerzia interna del clustering) è definita come:

$$\text{SSE} = \sum_{i=1}^{K} \sum_{\mathbf{x} \in C_i} \|\mathbf{x} - \mathbf{m}_i\|_2^2 = \sum_{i=1}^{K} \sum_{\mathbf{x} \in C_i} \sum_{j=1}^{d} (x_j - m_{ij})^2 \tag{7.6}$$

7.3.1 Dimostrazione: Il Baricentro Minimizza Rigorosamente l'SSE

Teorema 7.3.1 (Ottimalità del Baricentro)**.** *Dato un cluster C_k contenente $|C_k|$ punti in $\mathbb{R}^d$, il prototipo $\mathbf{m}_k \in \mathbb{R}^d$ che minimizza la somma delle distanze euclidee al quadrato dei punti di C_k è univocamente il baricentro aritmetico dei punti stessi.*

Dimostrazione. L'errore quadratico associato al singolo cluster C_k può essere scomposto come somma indipendente sulle d dimensioni ortogonali:

$$\text{SSE}_k = \sum_{\mathbf{x} \in C_k} \sum_{j=1}^{d} (x_j - m_{kj})^2 = \sum_{j=1}^{d} \left(\sum_{\mathbf{x} \in C_k} (x_j - m_{kj})^2 \right) \tag{7.7}$$

Per trovare il punto di minimo rispetto a ciascuna componente scalare m_{kj} del centroide $\mathbf{m}_k$, calcoliamo la derivata parziale prima e la poniamo uguale a zero:

$$\begin{aligned} \frac{\partial \, \text{SSE}_k}{\partial m_{kj}} &= \frac{\partial}{\partial m_{kj}} \sum_{\mathbf{x} \in C_k} (x_j - m_{kj})^2 \\ &= \sum_{\mathbf{x} \in C_k} 2(x_j - m_{kj}) \cdot (-1) \\ &= -2 \sum_{\mathbf{x} \in C_k} x_j + 2 \sum_{\mathbf{x} \in C_k} m_{kj} = 0 \end{aligned} \tag{7.8}$$

Poiché m_{kj} è una costante scalare rispetto all'indice di sommatoria $\mathbf{x} \in C_k$, la seconda sommatoria restituisce $|C_k| \cdot m_{kj}$:

$$2|C_k| \, m_{kj} = 2 \sum_{\mathbf{x} \in C_k} x_j \implies m_{kj} = \frac{1}{|C_k|} \sum_{\mathbf{x} \in C_k} x_j \tag{7.9}$$

Verifichiamo la condizione di minimo calcolando la derivata seconda:

$$\frac{\partial^2 \mathrm{SSE}_k}{\partial m_{kj}^2} = 2|C_k| > 0 \tag{7.10}$$

La matrice Hessiana associata è una matrice diagonale scalare definita positiva:

$$\nabla^2 \mathrm{SSE}_k = 2|C_k|\,\mathbf{I}_d \succ 0 \tag{7.11}$$

Pertanto, il baricentro $\mathbf{m}_k = \frac{1}{|C_k|}\sum_{\mathbf{x}\in C_k}\mathbf{x}$ è l'unico minimizzatore globale stretto di SSE_k. □

7.3.2 Formulazione come Discesa di Coordinate e Convergenza

L'algoritmo di Lloyd può essere formalizzato matematicamente come un'ottimizzazione per **discesa di coordinate** (*Coordinate Descent*) su una funzione bivariata.

Definiamo la matrice di appartenenza binaria $\mathbf{W} \in \{0,1\}^{n\times K}$, dove $w_{ik} = 1$ se l'istanza $\mathbf{x}_i$ appartiene al cluster k e $w_{ik} = 0$ altrimenti, con il vincolo $\sum_{k=1}^{K} w_{ik} = 1$ per ogni $i \in \{1,\dots,n\}$. Sia inoltre $\mathbf{S} = (\mathbf{m}_1,\dots,\mathbf{m}_K) \in \mathbb{R}^{d\times K}$ l'insieme dei centroidi. La funzione obiettivo globale è:

$$G(\mathbf{W},\mathbf{S}) = \sum_{i=1}^{n}\sum_{k=1}^{K} w_{ik}\|\mathbf{x}_i - \mathbf{m}_k\|_2^2 \tag{7.12}$$

Teorema 7.3.2 (Convergenza Monotona a un Minimo Locale)**.** *L'algoritmo K-Means riduce monotonicamente la funzione obiettivo $G(\mathbf{W},\mathbf{S})$ ad ogni semi-passo e converge in un numero finito di iterazioni a un minimo locale o punto stazionario.*

Dimostrazione. L'esecuzione si articola in due passi di minimizzazione alternata:

1. **Passo 1 (Assegnazione)**: Fissata la matrice dei centroidi $\mathbf{S}^{(t-1)}$, si minimizza $G(\mathbf{W},\mathbf{S}^{(t-1)})$ rispetto a $\mathbf{W}$. Poiché ogni termine per l'osservazione $\mathbf{x}_i$ è indipendente, la funzione è minimizzata ponendo:

$$w_{ik}^{(t)} = \begin{cases} 1 & \text{se } k = \arg\min_j \|\mathbf{x}_i - \mathbf{m}_j^{(t-1)}\|_2^2 \\ 0 & \text{altrimenti} \end{cases} \tag{7.13}$$

   Ne consegue che:

$$G(\mathbf{W}^{(t)},\mathbf{S}^{(t-1)}) \le G(\mathbf{W}^{(t-1)},\mathbf{S}^{(t-1)}) \tag{7.14}$$

2. **Passo 2 (Aggiornamento)**: Fissata la nuova matrice di appartenenza $\mathbf{W}^{(t)}$, si minimizza $G(\mathbf{W}^{(t)}, \mathbf{S})$ rispetto a $\mathbf{S}$. Dal teorema precedente sull'ottimalità del baricentro, la scelta ottima per ogni $\mathbf{m}_k$ è:

$$\mathbf{m}_k^{(t)} = \frac{\sum_{i=1}^{n} w_{ik}^{(t)} \mathbf{x}_i}{\sum_{i=1}^{n} w_{ik}^{(t)}} = \frac{1}{|C_k^{(t)}|} \sum_{\mathbf{x} \in C_k^{(t)}} \mathbf{x} \tag{7.15}$$

Ne consegue che:

$$G(\mathbf{W}^{(t)}, \mathbf{S}^{(t)}) \leq G(\mathbf{W}^{(t)}, \mathbf{S}^{(t-1)}) \tag{7.16}$$

Combinando le due disuguaglianze, la successione dei valori assunti dalla funzione obiettivo è monotona non crescente:

$$G(\mathbf{W}^{(t)}, \mathbf{S}^{(t)}) \leq G(\mathbf{W}^{(t-1)}, \mathbf{S}^{(t-1)}) \tag{7.17}$$

Poiché $G(\mathbf{W}, \mathbf{S}) \geq 0$ è limitata inferiormente da zero, e il numero di possibili matrici di assegnazione discrete $\mathbf{W}$ è finito (pari al numero di partizioni di n elementi in K gruppi non vuoti, quantificato dai numeri di Stirling di seconda specie $S(n, K)$), l'algoritmo non può entrare in cicli infiniti e deve necessariamente convergere in un numero finito di passi a una configurazione stazionaria (minimo locale). □

7.4 Scelta del Parametro K, Minimi Locali e Inizializzazione

Nonostante la convergenza garantita, la superficie di errore dell'ottimizzazione K-Means è altamente non convessa e costellata da innumerevoli minimi locali sub-ottimali. Infatti, la determinazione della partizione che minimizza globalmente l'SSE è un problema **NP-hard** anche per soli $K = 2$ cluster in dimensione arbitraria, o per dimensione $d = 2$ con K arbitrario.

7.4.1 Determinazione del Numero di Cluster: Il Metodo a Gomito

Poiché all'aumentare di K la funzione SSE(K) decresce monotonicamente (fino ad annullarsi banalmente per $K = n$), non è possibile minimizzare l'SSE direttamente rispetto a K.

Si ricorre all'**Elbow Method** (Metodo a Gomito):

1. Si esegue K-Means per una sequenza crescente di valori candidati $K \in \{1, 2, \ldots, K_{\max}\}$;

2. Si traccia la curva di SSE in funzione di K;

3. Si individua il punto di flesso ("gomito") in cui il decremento marginale di errore subisce un drastico rallentamento.

Matematicamente, il gomito corrisponde al valore di K che massimizza la derivata seconda discreta dell'SSE:

$$\Delta^2 \mathrm{SSE}(K) = \mathrm{SSE}(K+1) - 2\,\mathrm{SSE}(K) + \mathrm{SSE}(K-1) \tag{7.18}$$

Oltre tale soglia, l'introduzione di ulteriori cluster frammenta aggregazioni naturali senza apportare reale guadagno informativo.

7.4.2 Sensibilità ai Centroidi Iniziali e Dimostrazione Combinatoria

La qualità della partizione finale di K-Means dipende criticamente dal posizionamento dei centroidi iniziali $\mathbf{m}_1^{(0)}, \ldots, \mathbf{m}_K^{(0)}$. Se due centroidi cadono all'interno dello stesso addensamento naturale, uno dei cluster vicini rimarrà orfano, intrappolando l'algoritmo in un minimo locale sub-ottimale.

Teorema 7.4.1 (Probabilità Combinatoria di Inizializzazione Perfetta)**.** *Si consideri un dataset ideale composto da K cluster naturali ben separati, ciascuno avente la medesima cardinalità N/K. Scegliendo K centroidi iniziali mediante campionamento casuale uniforme indipendente con reinserimento, la probabilità $P(K)$ che ciascun centroide iniziale appartenga a un cluster distinto è:*

$$P(K) = \frac{K!}{K^K} \tag{7.19}$$

Dimostrazione. Calcoliamo la probabilità come prodotto delle probabilità condizionate passo per passo:

1. Il primo centroide può cadere in uno qualsiasi dei K cluster: $P_1 = \frac{K}{K} = 1$.

2. Affinché il secondo centroide cada in un cluster distinto dal primo, deve essere estratto da uno dei restanti $K-1$ cluster non ancora occupati. Poiché tutti i cluster hanno cardinalità uguale, la frazione di punti ammissibili è $P_2 = \frac{K-1}{K}$.

3. Il terzo centroide deve cadere in uno dei restanti $K-2$ cluster: $P_3 = \frac{K-2}{K}$.

4. Iterando fino al K-esimo centroide, la frazione residua è $P_K = \frac{1}{K}$.

Moltiplicando le probabilità degli eventi indipendenti:

$$P(K) = \prod_{j=1}^{K} \frac{K-j+1}{K} = \frac{K \cdot (K-1) \cdot (K-2) \cdots 1}{K^K} = \frac{K!}{K^K} \tag{7.20}$$

□

Valutando la formula (7.19) per diversi valori di K:

- $K = 2 \implies P(2) = \frac{2}{4} = 0.50$ (50%);
- $K = 3 \implies P(3) = \frac{6}{27} \approx 0.222$ (22.2%);
- $K = 5 \implies P(5) = \frac{120}{3125} \approx 0.0384$ (3.84%);
- $K = 10 \implies P(10) = \frac{3\,628\,800}{10^{10}} \approx 0.00036$ (appena lo **0.036%**).

Applicando l'approssimazione asintotica di Stirling $K! \approx \sqrt{2\pi K}\left(\frac{K}{e}\right)^K$, otteniamo:

$$P(K) \approx \frac{\sqrt{2\pi K}\left(\frac{K}{e}\right)^K}{K^K} = \sqrt{2\pi K} \cdot e^{-K} \tag{7.21}$$

La probabilità di successo dell'inizializzazione casuale pura decade in modo **esponenzialmente veloce** verso zero al crescere di K. Con $K \geq 10$, l'inizializzazione uniforme non garantisce quasi mai la corretta convergenza.

7.5 Euristiche di Inizializzazione e Varianti di K-Means

Per superare la fragilità dell'inizializzazione casuale, la letteratura ha sviluppato potenti strategie correttive ed euristiche algoritmiche avanzate.

7.5.1 Tecniche Euristiche di Inizializzazione

1. **Restart Casuali Multipli (*Random Restarts*)**: Si esegue l'algoritmo R volte (es. $R = 50$) con semi casuali differenti, selezionando la soluzione che restituisce il valore minimo di SSE. Efficace per valori modesti di K, ma computazionalmente oneroso e inefficace per K elevato.

2. **Campionamento con Pre-clustering Gerarchico**: Si estrae un campione casuale ridotto del dataset ($\approx 1\% - 5\%$) e si applica un algoritmo gerarchico agglomerativo con Metodo di Ward per determinare K raggruppamenti stabili, i cui baricentri vengono usati come semi per K-Means.

3. **K-Means++ (D^2-Weighting)**: Introdotto da Arthur e Vassilvitskii (2007), sceglie il primo centroide a caso, e ciascun centroide successivo con probabilità proporzionale al quadrato della distanza minima dai centroidi già allocati:

$$p(\mathbf{x}) = \frac{D(\mathbf{x})^2}{\sum_{\mathbf{y}\in\mathcal{D}} D(\mathbf{y})^2}, \qquad D(\mathbf{x}) = \min_{j\in\{1,\dots,m\}} \|\mathbf{x} - \mathbf{m}_j\|_2 \tag{7.22}$$

Garanzia teorica: K-Means++ garantisce che il valore atteso dell'SSE finale sia entro un fattore logaritmico rispetto all'ottimo globale:

$$\mathbb{E}[\text{SSE}] \leq 8(\ln K + 2) \cdot \text{SSE}_{\text{opt}} = \mathcal{O}(\log K) \cdot \text{SSE}_{\text{opt}} \tag{7.23}$$

4. **Bisecting K-Means**: Algoritmo partizionale ibrido che opera ricorsivamente con bisezioni binarie ($K = 2$):

   - Inizia considerando l'intero dataset come un unico macro-cluster;
   - A ogni iterazione, seleziona il cluster con il più alto SSE interno (o con cardinalità massima);
   - Applica il K-Means standard con $K = 2$ su tale cluster per scinderlo in due sotto-gruppi;
   - Ripete il processo finché non si ottengono esattamente K cluster.

   Bisecting K-Means è nettamente meno sensibile all'inizializzazione rispetto a Lloyd e produce naturalmente una gerarchia parziale dei cluster.

7.5.2 Aggiornamento Incrementale (Online K-Means)

Nella variante di MacQueen (1967), i centroidi vengono aggiornati *immediatamente* dopo la riassegnazione di ogni singolo punto, invece di attendere la fine dell'epoca (*batch*):

Proprietà 7.5.1 (Formule di Aggiornamento Incrementale di MacQueen)**.** *Se il punto* $\mathbf{x}$ *viene rimosso dal cluster* C_j *e assegnato al cluster* C_i*:*

$$\mathbf{m}_j \leftarrow \mathbf{m}_j - \frac{\mathbf{x} - \mathbf{m}_j}{|C_j| - 1} \tag{7.24}$$

$$\mathbf{m}_i \leftarrow \mathbf{m}_i + \frac{\mathbf{x} - \mathbf{m}_i}{|C_i| + 1} \tag{7.25}$$

L'approccio incrementale garantisce che l'SSE decresca strettamente ad ogni singolo spostamento, azzera la possibilità che si generino cluster vuoti, ma introduce una dipendenza dall'ordine di presentazione dei punti.

7.5.3 Gestione dei Cluster Vuoti (*Empty Clusters*)

Nell'algoritmo batch di Lloyd, può accadere che durante la fase di assegnazione nessun punto dati sia più vicino al centroide $\mathbf{m}_k$ rispetto agli altri $K-1$ centroidi, portando a $|C_k| = 0$ (centroide orfano). Strategie di ripristino per mantenere inalterato K:

1. **Selezione a Massimo Errore Residuo**: Identificare il punto $\mathbf{x}^* \in \mathcal{D}$ che massimizza la distanza quadratica dal proprio centroide assegnato $\|\mathbf{x}^* - \mathbf{m}_{c(\mathbf{x}^*)}\|^2$ e promuoverlo a nuovo centroide isolato del cluster vuoto. Questa strategia produce la massima riduzione marginale istantanea dell'SSE.

2. **Scissione del Cluster a Massimo SSE**: Identificare il cluster che esibisce il più elevato SSE cumulativo ed estrarre il suo punto più periferico per popolare il cluster orfano.

7.6 Limitazioni Strutturali di K-Means e Tecniche di Adattamento

K-Means presuppone implicitamente che i cluster siano sferici, compatti, ben separati e dotati di volumi e densità omogenee. In presenza di configurazioni geometriche complesse, l'algoritmo fallisce sistematicamente.



Figura 7.3: Quattro scenari critici di fallimento strutturale dell'algoritmo standard K-Means.

7.6.1 Analisi delle Tre Limitazioni Geometriche

1. **Dimensioni Differenti**: Poiché l'SSE penalizza gli scarti quadratici senza considerare il volume intrinseco, il centroide assegnato al cluster

compatto attrae punti del cluster disperso per minimizzare l'errore globale, determinando confini incoerenti.

2. **Densità Differenti**: Iperpiani posizionati equidistantemente tra centroidi non tengono conto del gradiente di densità. Un cluster a bassissima densità richiederà più centroidi per essere coperto, causando l'unione impropria di cluster ad alta densità vicini.

3. **Forme Non Globulari (es. *Two Moons*)**: Le celle di Voronoi sono poliedri convessi. Se i dati seguono varietà curvilinee non convesse (due semicerchi intrecciati), un iperpiano lineare secante taglia inevitabilmente a metà entrambi i corpi continui, fallendo completamente la separazione.

Strategia di Risoluzione: Over-Clustering

Per modellare varietà curvilinee o densità eterogenee, si esegue K-Means con un numero elevato di micro-cluster ($K' \gg K$). Tanti piccoli centroidi formano un'approssimazione poligonale a pezzi (*piecewise linear approximation*) che segue il profilo della curva continua. Successivamente, algoritmi basati su grafi o collegamenti ricompongono i micro-cluster nelle forme non lineari desiderate.

7.6.2 Pre-processing e Post-processing: Il Paradigma ISODATA

Per mitigare questi limiti nell'analisi partizionale, si adottano routine di pre- e post-elaborazione formalizzate storicamente nell'algoritmo **ISODATA** (*Iterative Self-Organizing Data Analysis Technique*):

- **Pre-processing**:
  - Normalizzazione o standardizzazione delle feature (z-score) per impedire che dimensioni con scale numeriche elevate dominino il calcolo della distanza euclidea;
  - Identificazione e rimozione preliminare degli outlier prima dell'inizializzazione dei centroidi.

- **Post-processing (Dinamica ISODATA)**:
  - **Split (Scissione)**: Se la deviazione standard o l'SSE di un cluster supera una soglia massima θ_{split}, e la cardinalità è sufficiente ($|C_i| \geq 2\theta_{\text{min}}$), il cluster viene diviso in due lungo la coordinata di massima varianza.

– **Merge (Fusione)**: Se la distanza euclidea tra due centroidi $\|\mathbf{m}_i - \mathbf{m}_j\|_2$ è inferiore a una soglia minima θ_{merge}, i due cluster vengono fusi in un unico raggruppamento baricentrico.
– **Eliminazione**: Se $|C_i| < \theta_{\text{min}}$, il cluster viene soppresso e i suoi punti redistribuiti.

7.7 Fondamenti del Clustering Gerarchico

Il **Clustering Gerarchico** supera la rigidità della partizione piatta generando una tassonomia continua di partizioni nidificate (*nested clusters*).

Definizione 7.7.1 (Dendrogramma)**.** Il **dendrogramma** è una rappresentazione grafica ad albero binario radicato in cui:

- Le **foglie** al livello $h = 0$ rappresentano i singoli punti dati isolati (cluster singleton);
- I **nodi interni** rappresentano le unioni successive di coppie di cluster;
- L'**altezza** h a cui due rami si collegano corrisponde formalmente alla distanza o dissimilarità inter-cluster al momento della loro fusione.

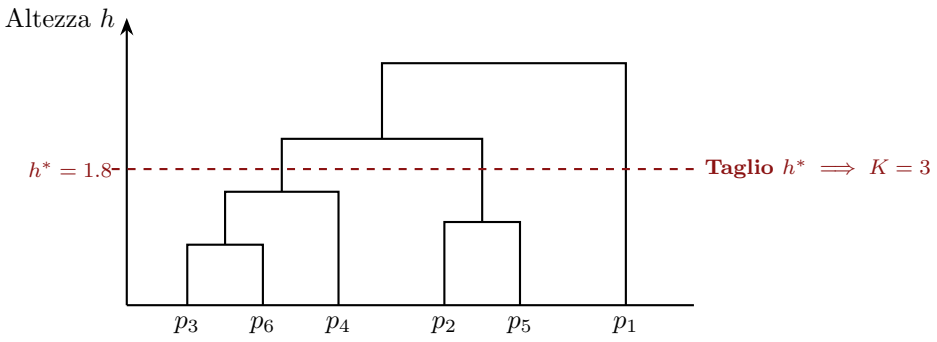


Figura 7.4: Struttura di un dendrogramma: le altezze delle giunzioni indicano la distanza di unione. Un taglio orizzontale induce una partizione piatta con $K = 3$ cluster.

7.7.1 Vantaggi Distintivi del Paradigma Gerarchico

- **Nessun vincolo a priori su** K: Non è richiesto specificare anticipatamente il numero di cluster. L'analista ottiene una visualizzazione completa dell'architettura dei dati ed estrae K a posteriori tracciando un taglio orizzontale alla quota desiderata h^*.

- **Modellazione di gerarchie biologiche e tassonomiche**: Strutture evolutive, tassonomie filogenetiche e relazioni semantiche possiedono una natura intrinsecamente gerarchica perfettamente rispecchiata dal dendrogramma.
- **Determinismo**: A differenza del K-Means, gli algoritmi gerarchici standard sono completamente deterministici (non dipendono da semi di inizializzazione casuali).

7.7.2 Tassonomia degli Algoritmi Gerarchici

- **Agglomerativi (*Bottom-up*)**: Iniziano considerando ogni punto come un cluster singleton (n cluster iniziali) e, a ciascun passo, fondono iterativamente la coppia di cluster più vicina fino a formare un unico grande raggruppamento radice in $n - 1$ passi.
- **Divisivi (*Top-down*)**: Iniziano considerando l'intero dataset come un unico mega-cluster e, ad ogni iterazione, partizionano ricorsivamente i gruppi in sotto-gruppi fino a ricondursi a n singleton.

7.8 Algoritmo Agglomerativo e Criteri di Collegamento (Linkage)

L'approccio agglomerativo standard richiede la definizione di una matrice di prossimità inter-cluster $\mathbf{D} \in \mathbb{R}^{N \times N}$ e segue il ciclo descritto nell'Algoritmo 2.

Algorithm 2: Algoritmo gerarchico agglomerativo standard.

Dati: Dataset $\mathcal{D} = \{\mathbf{x}_1, \ldots, \mathbf{x}_n\}$, metrica di distanza tra punti $d(\cdot, \cdot)$.
Risultato: Dendrogramma completo delle fusioni gerarchiche.

Inizializza n cluster singleton: $C_i = \{\mathbf{x}_i\}$ per ogni $i \in \{1, \ldots, n\}$
Costruisci la matrice di prossimità iniziale $\mathbf{D}^{(0)} \in \mathbb{R}^{n \times n}$ con $D_{ij} = d(\mathbf{x}_i, \mathbf{x}_j)$
for $t = 1$ **to** $n - 1$ **do**
    Trova la coppia di cluster (A, B) a minima distanza nella matrice: $(A, B) = \arg\min_{i \neq j} D_{ij}^{(t-1)}$
    Fondi i cluster A e B creando il nuovo cluster $A \cup B$ a quota $h_t = D_{AB}^{(t-1)}$
    Elimina le righe e colonne corrispondenti ad A e B dalla matrice di prossimità
    Calcola la distanza tra il nuovo cluster $A \cup B$ e ogni altro cluster residuo Q secondo il criterio di collegamento prescelto
    Aggiorna la matrice di prossimità $\mathbf{D}^{(t)}$ inserendo la nuova riga/colonna associata ad $A \cup B$
end

7.8.1 I Quattro Criteri di Collegamento (*Linkage*)

La definizione della distanza $D(A, B)$ tra due insiemi di punti A e B determina interamente la forma e le proprietà della gerarchia risultante.

1. MIN (Single Link)

La distanza tra due cluster coincide con la minima distanza tra qualsiasi coppia di punti:

$$D_{\text{MIN}}(A, B) = \min_{\mathbf{x} \in A,\, \mathbf{y} \in B} d(\mathbf{x}, \mathbf{y}) \tag{7.26}$$

- **Punti di forza**: Capace di catturare forme arbitrarie non convesse, filiformi o ad anello (è isomorfo alla costruzione del Minimum Spanning Tree).
- **Debolezze (*Chaining Effect*)**: Estremamente sensibile al rumore e agli outlier. Anche un singolo punto spurio situato tra due cluster compatti distinti funge da "ponte", inducendo la fusione prematura dei due macro-gruppi in un'unica lunga catena.

2. MAX (Complete Link)

La distanza tra due cluster coincide con la massima distanza tra le loro coppie di punti:

$$D_{\text{MAX}}(A, B) = \max_{\mathbf{x}\in A,\, \mathbf{y}\in B} d(\mathbf{x}, \mathbf{y}) \tag{7.27}$$

- **Punti di forza**: Robusto rispetto al rumore e immune al chaining effect; tende a produrre cluster iper-compatti e globulari limitando formalmente il diametro massimo del cluster combinato: $\text{diam}(A \cup B) \leq h$.
- **Debolezze**: Tende a frammentare ingiustificatamente grandi cluster allungati e soffre la presenza di outlier periferici estremi.

3. Group Average (Average Link / UPGMA)

La distanza tra due cluster è la media aritmetica di tutte le distanze a coppie:

$$D_{\text{AVG}}(A, B) = \frac{1}{|A| \cdot |B|} \sum_{\mathbf{x}\in A} \sum_{\mathbf{y}\in B} d(\mathbf{x}, \mathbf{y}) \tag{7.28}$$

Costituisce un compromesso matematicamente bilanciato tra la permissività di MIN e la severità di MAX. È relativamente robusto al rumore poiché l'effetto di singoli outlier viene mediato sull'intera cardinalità del gruppo.

4. Metodo di Ward (Minimizzazione della Varianza)

Introdotto da Joe H. Ward Jr. (1963), la prossimità tra due cluster è misurata tramite l'**incremento dell'errore quadratico totale** (ΔSSE) che deriverebbe dalla loro eventuale fusione:

$$\Delta\text{SSE}_{AB} = \text{SSE}_{A\cup B} - (\text{SSE}_A + \text{SSE}_B) \tag{7.29}$$

Teorema 7.8.1 (Formulazione Chiusa del Metodo di Ward)**.** *L'incremento dell'errore quadratico ΔSSE_{AB} dovuto alla fusione dei cluster A e B equivale alla distanza euclidea al quadrato tra i loro rispettivi centroidi, scalata dal prodotto armonico delle loro masse:*

$$\Delta\text{SSE}_{AB} = \frac{|A| \cdot |B|}{|A| + |B|} \|\mathbf{m}_A - \mathbf{m}_B\|_2^2 \tag{7.30}$$

Dimostrazione. Siano A e B due cluster disgiunti con cardinalità $|A|, |B|$ e centroidi $\mathbf{m}_A, \mathbf{m}_B$. Il centroide del cluster fuso $A \cup B$ è la media pesata dei baricentri:

$$\mathbf{m}_{A\cup B} = \frac{|A|\mathbf{m}_A + |B|\mathbf{m}_B}{|A| + |B|} \tag{7.31}$$

Calcoliamo lo scarto vettoriale tra il baricentro combinato e il baricentro originale $\mathbf{m}_A$:

$$\mathbf{m}_{A\cup B} - \mathbf{m}_A = \frac{|A|\mathbf{m}_A + |B|\mathbf{m}_B - (|A| + |B|)\mathbf{m}_A}{|A| + |B|} = \frac{|B|(\mathbf{m}_B - \mathbf{m}_A)}{|A| + |B|} \tag{7.32}$$

Simmetricamente per il cluster B:

$$\mathbf{m}_{A\cup B} - \mathbf{m}_B = \frac{|A|(\mathbf{m}_A - \mathbf{m}_B)}{|A| + |B|} \tag{7.33}$$

L'errore quadratico totale del cluster unificato è:

$$\mathrm{SSE}_{A\cup B} = \sum_{\mathbf{x}\in A} \|\mathbf{x} - \mathbf{m}_{A\cup B}\|^2 + \sum_{\mathbf{y}\in B} \|\mathbf{y} - \mathbf{m}_{A\cup B}\|^2 \tag{7.34}$$

Sviluppiamo il primo termine inserendo e sottraendo il baricentro originale $\mathbf{m}_A$:

$$\begin{aligned}\sum_{\mathbf{x}\in A} \|\mathbf{x} - \mathbf{m}_{A\cup B}\|^2 &= \sum_{\mathbf{x}\in A} \|(\mathbf{x} - \mathbf{m}_A) - (\mathbf{m}_{A\cup B} - \mathbf{m}_A)\|^2 \\ &= \sum_{\mathbf{x}\in A} \|\mathbf{x} - \mathbf{m}_A\|^2 + \sum_{\mathbf{x}\in A} \|\mathbf{m}_{A\cup B} - \mathbf{m}_A\|^2 \\ &\quad - 2\left(\sum_{\mathbf{x}\in A}(\mathbf{x} - \mathbf{m}_A)\right) \cdot (\mathbf{m}_{A\cup B} - \mathbf{m}_A)\end{aligned} \tag{7.35}$$

Poiché per definizione di centroide $\sum_{\mathbf{x}\in A}(\mathbf{x} - \mathbf{m}_A) = \mathbf{0}$, il doppio prodotto svanisce identicamente:

$$\sum_{\mathbf{x}\in A} \|\mathbf{x} - \mathbf{m}_{A\cup B}\|^2 = \mathrm{SSE}_A + |A| \cdot \|\mathbf{m}_{A\cup B} - \mathbf{m}_A\|^2 \tag{7.36}$$

Sostituendo l'espressione precedentemente ricavata per $(\mathbf{m}_{A\cup B} - \mathbf{m}_A)$:

$$\begin{aligned}\sum_{\mathbf{x}\in A} \|\mathbf{x} - \mathbf{m}_{A\cup B}\|^2 &= \mathrm{SSE}_A + |A| \left\| \frac{|B|(\mathbf{m}_B - \mathbf{m}_A)}{|A| + |B|} \right\|^2 \\ &= \mathrm{SSE}_A + \frac{|A| \cdot |B|^2}{(|A| + |B|)^2} \|\mathbf{m}_A - \mathbf{m}_B\|^2\end{aligned} \tag{7.37}$$

Ripetendo lo stesso sviluppo per il secondo termine relativo a B:

$$\sum_{\mathbf{y}\in B} \|\mathbf{y} - \mathbf{m}_{A\cup B}\|^2 = \mathrm{SSE}_B + \frac{|A|^2 \cdot |B|}{(|A| + |B|)^2} \|\mathbf{m}_A - \mathbf{m}_B\|^2 \tag{7.38}$$

Sommando i due termini per ottenere $\text{SSE}_{A\cup B}$:

$$\begin{aligned}\text{SSE}_{A\cup B} &= \text{SSE}_A + \text{SSE}_B + \left[\frac{|A|\cdot|B|^2 + |A|^2\cdot|B|}{(|A|+|B|)^2}\right]\|\mathbf{m}_A - \mathbf{m}_B\|^2 \\ &= \text{SSE}_A + \text{SSE}_B + \left[\frac{|A|\cdot|B|(|B|+|A|)}{(|A|+|B|)^2}\right]\|\mathbf{m}_A - \mathbf{m}_B\|^2 \\ &= \text{SSE}_A + \text{SSE}_B + \frac{|A|\cdot|B|}{(|A|+|B|)}\|\mathbf{m}_A - \mathbf{m}_B\|^2 \qquad (7.39)\end{aligned}$$

Sottraendo $(\text{SSE}_A + \text{SSE}_B)$, si ottiene la tesi:

$$\Delta\text{SSE}_{AB} = \text{SSE}_{A\cup B} - (\text{SSE}_A + \text{SSE}_B) = \frac{|A|\cdot|B|}{|A|+|B|}\|\mathbf{m}_A - \mathbf{m}_B\|^2 \qquad (7.40)$$

□

Il Metodo di Ward è l'analogo gerarchico esatto del criterio di costo del K-Means: genera cluster regolari, sferici e ben bilanciati, risultando straordinariamente robusto rispetto al rumore.

7.8.2 Esempio Pratico Guidato su 6 Punti: MIN vs MAX

Consideriamo un dataset di 6 punti bidimensionali $\{p_1, \ldots, p_6\}$ con matrice di distanza iniziale:

$$\mathbf{D} = \begin{pmatrix} & p_1 & p_2 & p_3 & p_4 & p_5 & p_6 \\ p_1 & 0.00 & 0.24 & 0.22 & 0.37 & 0.34 & 0.23 \\ p_2 & 0.24 & 0.00 & 0.15 & 0.20 & 0.14 & 0.25 \\ p_3 & 0.22 & 0.15 & 0.00 & 0.15 & 0.28 & 0.11 \\ p_4 & 0.37 & 0.20 & 0.15 & 0.00 & 0.29 & 0.22 \\ p_5 & 0.34 & 0.14 & 0.28 & 0.29 & 0.00 & 0.39 \\ p_6 & 0.23 & 0.25 & 0.11 & 0.22 & 0.39 & 0.00 \end{pmatrix} \qquad (7.41)$$

Risoluzione con MIN (Single Link)

1. Minimo assoluto globale: $d(p_3, p_6) = 0.11 \implies$ Unione di $\{p_3, p_6\}$ a quota $h = 0.11$.
2. Minimo successivo: $d(p_2, p_5) = 0.14 \implies$ Unione di $\{p_2, p_5\}$ a quota $h = 0.14$.
3. Distanza tra $\{p_3, p_6\}$ e p_4: $\min(d(p_3, p_4), d(p_6, p_4)) = 0.15 \implies$ Fusione di $\{p_3, p_6\}$ con p_4 a quota $h = 0.15$.

4. Distanza tra $\{p_3, p_6, p_4\}$ e $\{p_2, p_5\}$:

$$\min_{\mathbf{x}\in\{3,6,4\},\,\mathbf{y}\in\{2,5\}} d(\mathbf{x},\mathbf{y}) = \min(0.15, 0.25, 0.20, 0.28, 0.39, 0.29) = d(p_3, p_2) = 0.15 \tag{7.42}$$

I due blocchi si fondono istantaneamente a quota $h = 0.15$ sfruttando il ponte ravvicinato p_3–p_2.

5. Distanza da p_1: $\min(d(p_1, p_j)) = d(p_1, p_3) = 0.22 \implies$ Fusione finale a quota $h = 0.22$.

Risoluzione con MAX (Complete Link)

1. Primi due passi identici: fusione $\{p_3, p_6\}$ a $h = 0.11$, e $\{p_2, p_5\}$ a $h = 0.14$.

2. Distanza MAX tra $\{p_3, p_6\}$ e p_4: $\max(d(p_3, p_4), d(p_6, p_4)) = 0.22$.

3. Distanza MAX tra $\{p_3, p_6\}$ e $\{p_2, p_5\}$:

$$\max_{\mathbf{x}\in\{3,6\},\,\mathbf{y}\in\{2,5\}} d(\mathbf{x},\mathbf{y}) = \max(0.15, 0.25, 0.28, 0.39) = d(p_6, p_5) = 0.39 \tag{7.43}$$

Poiché $0.22 < 0.39$, l'unione preferita a questo passo è $\{p_3, p_6\}$ con p_4 a quota $h = 0.22$.

4. La distanza tra $\{p_3, p_6, p_4\}$ e $\{p_2, p_5\}$ rimane elevata (0.39). Invece, la distanza da p_1 a $\{p_2, p_5\}$ è $\max(0.24, 0.34) = 0.34$. Pertanto, MAX ritarda la fusione tra i due gruppi principali ed evita fusioni precoci indotte da collegamenti isolati.

7.8.3 Formula di Ricorrenza di Lance-Williams

La formula di ricorrenza proposta da Lance e Williams (1967) fornisce un quadro analitico unificato per aggiornare la matrice di prossimità ad ogni fusione senza dover riesaminare i singoli punti dati originari.

Teorema 7.8.2 (Formula di Lance-Williams)**.** *Dati due cluster A e B fusi nel nuovo cluster $A \cup B$, la distanza tra $A \cup B$ e un terzo cluster Q è determinata da:*

$$D(A\cup B,\, Q) = \alpha_A\, D(A,Q) + \alpha_B\, D(B,Q) + \beta\, D(A,B) + \gamma\, |D(A,Q) - D(B,Q)| \tag{7.44}$$

dove i parametri $(\alpha_A, \alpha_B, \beta, \gamma)$ dipendono univocamente dal criterio prescelto.

Criterio di Linkage	$\alpha_{\mathbf{A}}$	$\alpha_{\mathbf{B}}$	β	γ
MIN (Single Link)	$1/2$	$1/2$	0	$-1/2$
MAX (Complete Link)	$1/2$	$1/2$	0	$+1/2$
Group Average	$\frac{\lvert A\rvert}{\lvert A\rvert+\lvert B\rvert}$	$\frac{\lvert B\rvert}{\lvert A\rvert+\lvert B\rvert}$	0	0
Centroid Linkage	$\frac{\lvert A\rvert}{\lvert A\rvert+\lvert B\rvert}$	$\frac{\lvert B\rvert}{\lvert A\rvert+\lvert B\rvert}$	$-\frac{\lvert A\rvert\lvert B\rvert}{(\lvert A\rvert+\lvert B\rvert)^2}$	0
Metodo di Ward	$\frac{\lvert A\rvert+\lvert Q\rvert}{\lvert A\rvert+\lvert B\rvert+\lvert Q\rvert}$	$\frac{\lvert B\rvert+\lvert Q\rvert}{\lvert A\rvert+\lvert B\rvert+\lvert Q\rvert}$	$-\frac{\lvert Q\rvert}{\lvert A\rvert+\lvert B\rvert+\lvert Q\rvert}$	0

Tabella 7.1: Parametri della formula di ricorrenza di Lance-Williams per i principali metodi gerarchici.

Dimostrazione di Consistenza per Group Average. Verifichiamo che la sostituzione dei parametri nella formula di Lance-Williams riproduca esattamente la definizione originaria della media:

$$\begin{aligned}\sum_{\mathbf{x}\in A\cup B}\sum_{\mathbf{z}\in Q} d(\mathbf{x},\mathbf{z}) &= \sum_{\mathbf{x}\in A}\sum_{\mathbf{z}\in Q} d(\mathbf{x},\mathbf{z}) + \sum_{\mathbf{y}\in B}\sum_{\mathbf{z}\in Q} d(\mathbf{y},\mathbf{z}) \\ &= |A|\cdot|Q|\cdot D_{\text{AVG}}(A,Q) + |B|\cdot|Q|\cdot D_{\text{AVG}}(B,Q) \quad (7.45)\end{aligned}$$

Dividendo per $|A\cup B|\cdot|Q| = (|A|+|B|)\cdot|Q|$ e semplificando $|Q|>0$:

$$D_{\text{AVG}}(A\cup B,\, Q) = \frac{|A|}{|A|+|B|}D_{\text{AVG}}(A,Q) + \frac{|B|}{|A|+|B|}D_{\text{AVG}}(B,Q) \quad (7.46)$$

che coincide esattamente con la riga di Group Average in Tabella 7.1. □

7.8.4 Tabella Sinottica Comparativa dei Criteri di Collegamento

Criterio	Obiettivo	Geometria	Outlier	Proprietà
MIN (Single)	$\min_{\mathbf{x},\mathbf{y}} d(\mathbf{x},\mathbf{y})$	Arbitraria, concatenata	Altissima (*chaining*)	Isomorfo all'MST
MAX (Complete)	$\max_{\mathbf{x},\mathbf{y}} d(\mathbf{x},\mathbf{y})$	Sferica, compatta	Bassa al centro	Diametro $\leq h$
Group Average	Media $\frac{\sum\sum d}{\lvert A\rvert\lvert B\rvert}$	Globulare tollerante	Moderata	Bilanciato
Metodo di Ward	ΔSSE_{AB}	Sferica bilanciata	Molto bassa	Analogo K-Means

Tabella 7.2: Confronto analitico tra i quattro criteri di collegamento nel clustering gerarchico.

7.9 Clustering Gerarchico Divisivo basato su Minimum Spanning Tree (MST)

Mentre gli algoritmi divisivi generici risultano computazionalmente proibitivi (considerare tutte le possibili bisezioni di un insieme di n punti comporta $2^{n-1} - 1$ configurazioni), l'approccio basato su teoria dei grafi fornisce una soluzione elegante ed efficiente tramite il **Minimum Spanning Tree** (MST).

Definizione 7.9.1 (Minimum Spanning Tree dei Dati)**.** Dato il grafo completo $G = (V, E)$ in cui i vertici V rappresentano gli n punti dati e ogni arco $(u, v) \in E$ ha peso $w(u, v) = d(\mathbf{x}_u, \mathbf{x}_v)$, un MST è un albero ricoprente privo di cicli di peso complessivo minimo:

$$w(\text{MST}) = \sum_{e \in \text{MST}} w(e) \to \min \tag{7.47}$$

7.9.1 Algoritmo Divisivo basato su Taglio di Archi

Algorithm 3: Clustering divisivo basato su Minimum Spanning Tree.

Dati: Dataset $\mathcal{D} = \{\mathbf{x}_1, \ldots, \mathbf{x}_n\}$, numero di cluster desiderato K.
Risultato: Partizione $\{C_1, \ldots, C_K\}$.

```
Costruisci il grafo completo pesato sui dati G = (V, E)
Estrai il Minimum Spanning Tree T_MST mediante l'algoritmo di Prim
  o Kruskal
Ordina gli n − 1 archi dell'albero per peso decrescente:
  w(e_1) ≥ w(e_2) ≥ ··· ≥ w(e_{n−1})
for k = 1 to K − 1 do
    Rimuovi dall'albero l'arco di peso massimo residuo e_k
    L'albero si scinde in due componenti connesse disgiunte
end
Identifica le K componenti connesse risultanti come i K cluster della
  partizione
```

Proprietà 7.9.1 (Equivalenza Teorica con il Single Link)**.** *La partizione ottenuta rimuovendo i $K-1$ archi a peso massimo dal Minimum Spanning Tree è* ***identica*** *alla partizione ottenuta arrestando l'algoritmo agglomerativo con Single Link a K cluster. Ne consegue che la costruzione dell'*MST *(realizzabile in $\mathcal{O}(n^2)$ con Prim) fornisce una via computazionalmente diretta per ricavare la gerarchia del Single Link.*

7.10 Complessità, Limiti e Confronto Sinottico

7.10.1 Analisi della Complessità Asintotica del Clustering Gerarchico

- **Complessità Spaziale**: L'algoritmo deve memorizzare l'intera matrice di prossimità triangolare $n \times n$:

$$\mathcal{S}_{\text{Gerarchico}} = \mathcal{O}(n^2) \tag{7.48}$$

  Ciò costituisce un limite severo nei contesti di Big Data. Ad esempio, per $n = 100\,000$ punti a precisione standard (8 byte per float), la matrice richiede circa 40 GB di memoria RAM, rendendo il metodo inapplicabile senza partizionamenti preliminari.

- **Complessità Temporale Naive**: Il ciclo compie $n - 1$ iterazioni. A ogni iterazione, la ricerca lineare del minimo nella matrice di dimensione residua richiede $\mathcal{O}(n^2)$ confronti, portando a:

$$\mathcal{T}_{\text{Naive}} = \sum_{t=1}^{n-1} \mathcal{O}((n-t)^2) = \mathcal{O}(n^3) \tag{7.49}$$

- **Ottimizzazione tramite Code con Priorità (Heap)**: Mantenendo le distanze per ciascun cluster in una coda con priorità (*min-heap*), l'estrazione del minimo e l'aggiornamento richiedono $\mathcal{O}(n \log n)$ per iterazione, riducendo la complessità totale a:

$$\mathcal{T}_{\text{Heap}} = \mathcal{O}(n^2 \log n) \tag{7.50}$$

  (Per criteri specifici come Single Link e Complete Link, algoritmi ottimizzati come SLINK e CLINK operano in $\mathcal{O}(n^2)$ tempo e $\mathcal{O}(n)$ spazio ausiliario).

7.10.2 Limiti Strutturali del Clustering Gerarchico

1. **Irreversibilità delle Decisioni (*Greedy Nature*)**: Le decisioni di fusione o scissione sono permanenti e prive di backtracking. Se due punti vengono uniti al passo t per via del rumore locale o di un criterio inadeguato, non potranno mai essere separati nei passi successivi.

2. **Mancanza di una Funzione Obiettivo Globale**: A eccezione del Metodo di Ward (che minimizza l'incremento locale di varianza), i criteri di linkage non ottimizzano una funzione di costo globale scalare sull'intero dataset.

7.10.3 Schema Comparativo Finale: K-Means vs Clustering Gerarchico

A completamento della trattazione dei due paradigmi, la Tabella 7.3 sintetizza le divergenze architetturali, algebriche e operative essenziali che guidano la scelta del modello in contesti ingegneristici e scientifici reali.

Caratteristica Operativa	**K-Means Clustering**	**Clustering Gerarchico**
Tipo di Struttura	Partizionale piatta (*hard clustering*)	Tassonomica ad albero nidificato (Dendrogramma)
Parametro K	Obbligatorio a priori	Derivabile a posteriori tracciando un taglio
Funzione Obiettivo	Minimizzazione di SSE globale	Scelte miopi locali (*greedy*) senza costo globale
Complessità Temporale	$\mathcal{O}(n \cdot K \cdot I \cdot d)$ (Lineare con n)	$\mathcal{O}(n^2 \log n)$ fino a $\mathcal{O}(n^3)$ (Sovra-quadratica)
Complessità Spaziale	$\mathcal{O}((n + K) \cdot d)$ (Lineare con n)	$\mathcal{O}(n^2)$ (Quadratica, memoria critica)
Scalabilità sui Dati	Estremamente elevata (milioni di record)	Molto bassa ($n \leq 20\,000$ in RAM standard)
Sensibilità Inizializzazione	Elevatissima (intrappolamento in minimi locali)	Deterministico (nessun seme o partizione iniziale)
Flessibilità di Forma	Solo sferica/globulare (iperpiani Voronoi)	Arbitraria con Single Link; sferica con Ward
Reversibilità Decisioni	Sì (i punti migrano ad ogni iterazione)	No (fusioni e tagli permanenti senza riassegnazione)

Tabella 7.3: Confronto sinottico globale tra K-Means e Clustering Gerarchico.

Capitolo 8

Cluster Analysis: Clustering Basato su Densità (DBSCAN)

8.1 Introduzione al Paradigma Basato su Densità

Nelle sezioni precedenti sono stati esaminati due pilastri storici del clustering non supervisionato:

- I **metodi partizionali a prototipo** (es. K-Means), in cui ogni gruppo è rappresentato dal proprio baricentro $\mathbf{m}_k$ e lo spazio euclideo viene tassellato secondo le celle iper-piane di Voronoi. Tale modello assume implicitamente cluster a geometria iper-sferica, convessa e con varianza confrontabile, fallendo drammaticamente quando i pattern reali assumono forme allungate, concave, a mezzaluna, spirali o anelli concentrici. Inoltre, la minimizzazione globale della devianza entro i cluster (SSE) costringe ogni singola istanza — inclusi rumore e outlier isolati — ad appartenere a un cluster, distorcendone la posizione media.
- I **metodi gerarchici** (agglomerativi o divisivi), che costruiscono un dendrogramma basato su matrici di distanza globale. Sebbene il collegamento singolo (*single link*) sia capace in linea teorica di seguire catene di punti arbitrari, esso soffre del patologico fenomeno del concatenamento spurio (*chaining effect*), dove anche un singolo punto di rumore interposto tra due cluster distinti fa collassare l'intera gerarchia.

Il **clustering basato su densità** (*density-based clustering*) supera in radice tali assunzioni ridefinendo la nozione stessa di cluster non più in termini di distanza da un centroide o di prossimità globale, bensì come una **regione**

continua e contigua ad elevata densità locale, separata da altri cluster mediante **regioni a bassa densità o vuoto spaziale** (le quali ospitano il rumore di fondo e gli outlier).

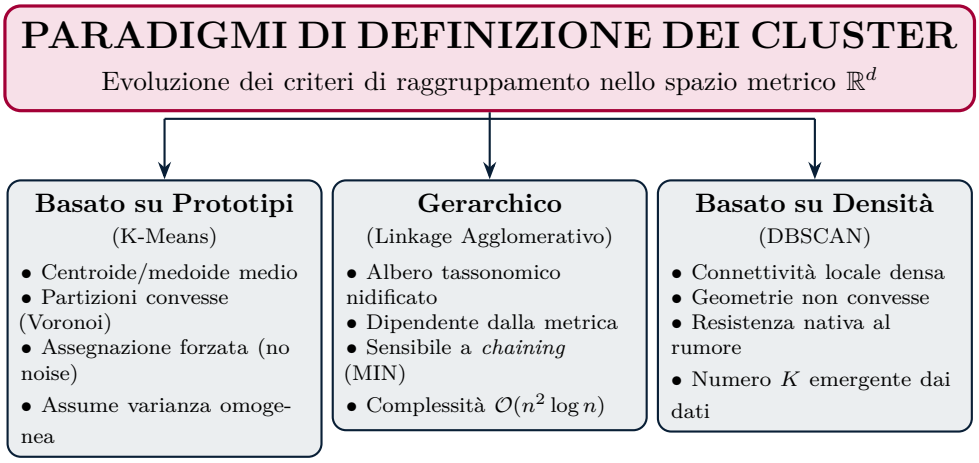


Figura 8.1: Inquadramento concettuale del clustering basato su densità rispetto ai paradigmi a prototipo e gerarchici.

Sotto questa ottica, l'algoritmo DBSCAN introduce un meccanismo di aggregazione locale in grado di:

1. Scoprire cluster con morfologie spaziali arbitrarie (concave, ad anello, filamentose);
2. Isolare ed espellere naturalmente le osservazioni non appartenenti ad alcuna aggregazione densa (*noise filtering*);
3. Non richiedere all'utente la specificazione a priori del numero K di cluster attesi, determinandolo come proprietà emergente della topologia dei dati.

8.2 Fondamenti della Densità e Tassonomia dei Punti

Sia dato un dataset multidimensionale $\mathcal{D} = \{\mathbf{p}_1, \mathbf{p}_2, \ldots, \mathbf{p}_n\} \subset \mathbb{R}^d$ dotato di una metrica di distanza $\text{dist} : \mathcal{D} \times \mathcal{D} \to \mathbb{R}_{\geq 0}$ (solitamente la distanza euclidea L_2, sebbene qualsiasi metrica o semimetrica sia formalmente applicabile).

La formulazione originale di DBSCAN parametrizza la densità spaziale attraverso due iperparametri globali:

- $\varepsilon \in \mathbb{R}^+$ (**Eps**): la soglia di raggio che definisce l'ampiezza dell'intorno sferico locale centrato su ciascun punto;
- $MinPts \in \mathbb{N}^+$: la soglia intera positiva indicante il numero minimo di punti che devono risiedere all'interno dell'intorno di raggio ε affinché l'area sia considerata ad elevata densità.

8.2.1 Definizione di Intorno e Densità Locale

Definizione 8.2.1 (ε-Intorno / Eps-Neighborhood)**.** Dato un punto generico $\mathbf{p} \in \mathcal{D}$ e un raggio scalare $\varepsilon > 0$, l'**ε-intorno** di $\mathbf{p}$, denotato con $N_\varepsilon(\mathbf{p})$ o $N_{\text{Eps}}(\mathbf{p})$, è il sottoinsieme delle osservazioni di $\mathcal{D}$ la cui distanza da $\mathbf{p}$ non supera la soglia ε:

$$N_\varepsilon(\mathbf{p}) = \left\{ \mathbf{q} \in \mathcal{D} \;\middle|\; \text{dist}(\mathbf{p}, \mathbf{q}) \leq \varepsilon \right\} \tag{8.1}$$

Nota 8.2.1 (Inclusione Riflessiva del Punto Stesso)**.** Poiché ogni metrica soddisfa la proprietà di identità degli indiscernibili $\text{dist}(\mathbf{p}, \mathbf{p}) = 0 \leq \varepsilon$, per qualunque scelta di $\varepsilon > 0$ si ha necessariamente:

$$\mathbf{p} \in N_\varepsilon(\mathbf{p}) \tag{8.2}$$

Pertanto, nel calcolo della densità locale il punto $\mathbf{p}$ viene **sempre conteggiato come elemento del proprio intorno** (*"counts the point itself"*).

Definizione 8.2.2 (Densità Locale)**.** La **densità locale** di un punto $\mathbf{p} \in \mathcal{D}$ rispetto al parametro ε è definita in modo puramente discreto dalla cardinalità del suo ε-intorno:

$$\text{Density}(\mathbf{p}) = |N_\varepsilon(\mathbf{p})| \tag{8.3}$$

In uno spazio continuo euclideo $\mathbb{R}^d$, la densità volumetrica corrispondente equivale a $|N_\varepsilon(\mathbf{p})|/V_d(\varepsilon)$, dove $V_d(\varepsilon) = \frac{\pi^{d/2}}{\Gamma(d/2+1)}\varepsilon^d$ è il volume dell'ipersfera di raggio ε. Fissato lo spazio $\mathbb{R}^d$ e la soglia ε, il volume $V_d(\varepsilon)$ è una costante e la cardinalità $|N_\varepsilon(\mathbf{p})|$ ne riflette direttamente la concentrazione di massa locale.

8.2.2 Tassonomia Esaustiva e Disgiunta dei Punti

Sulla base del confronto tra la densità locale $\text{Density}(\mathbf{p})$ e la soglia critica $MinPts$, DBSCAN partiziona univocamente l'intero dataset $\mathcal{D}$ in tre categorie mutuamente disgiunte:

Definizione 8.2.3 (Core Point / Punto Nucleo)**.** Un punto $\mathbf{p} \in \mathcal{D}$ è classificato come **Core Point** ($\mathbf{p} \in \mathcal{C}$) se e solo se il suo ε-intorno contiene almeno $MinPts$ punti:

$$\mathbf{p} \in \mathcal{C} \iff |N_\varepsilon(\mathbf{p})| \geq MinPts \tag{8.4}$$

I Core Point risiedono stabilmente all'interno (*interior*) delle regioni dense e fungono da nodi di propagazione strutturale del cluster.

Definizione 8.2.4 (Border Point / Punto di Bordo)**.** Un punto $\mathbf{p} \in \mathcal{D}$ è classificato come **Border Point** ($\mathbf{p} \in \mathcal{B}$) se non soddisfa la condizione di core point, ma cade nell'ε-intorno di almeno un core point esistente:

$$\mathbf{p} \in \mathcal{B} \iff \Big(|N_\varepsilon(\mathbf{p})| < MinPts\Big) \wedge \Big(\exists \mathbf{q} \in \mathcal{C} \text{ tale che } \mathbf{p} \in N_\varepsilon(\mathbf{q})\Big) \tag{8.5}$$

I Border Point non possiedono individualmente una densità locale sufficiente per generare un cluster, ma costituiscono la frontiera perimetrale esterna della componente densa a cui sono adiacenti.

Definizione 8.2.5 (Noise Point / Punto di Rumore o Outlier)**.** Un punto $\mathbf{p} \in \mathcal{D}$ è classificato come **Noise Point** ($\mathbf{p} \in \mathcal{N}$) se non appartiene né all'insieme dei core point né a quello dei border point:

$$\mathbf{p} \in \mathcal{N} \iff (\mathbf{p} \notin \mathcal{C}) \wedge (\mathbf{p} \notin \mathcal{B}) \tag{8.6}$$

Formulato in termini di distanze metriche:

$$\mathbf{p} \in \mathcal{N} \iff \Big(|N_\varepsilon(\mathbf{p})| < MinPts\Big) \wedge \Big(\forall \mathbf{q} \in \mathcal{C},\ \mathrm{dist}(\mathbf{p}, \mathbf{q}) > \varepsilon\Big) \tag{8.7}$$

I Noise Point rappresentano campioni sparsi, fluttuazioni casuali di campionamento o anomalie isolate situate nel vuoto inter-cluster.

Proprietà 8.2.1 (Partizione dello Spazio dei Punti)**.** *Per ogni coppia prefissata di parametri $(\varepsilon, MinPts)$, l'insieme dei dati $\mathcal{D}$ ammette una partizione esatta in tre classi disgiunte:*

$$\mathcal{D} = \mathcal{C} \cup \mathcal{B} \cup \mathcal{N}, \qquad \text{con } \mathcal{C} \cap \mathcal{B} = \emptyset,\ \mathcal{C} \cap \mathcal{N} = \emptyset,\ \mathcal{B} \cap \mathcal{N} = \emptyset \tag{8.8}$$

8.2.3 Analisi Geometrica: Esempio con $MinPts = 7$

La Figura 8.2 illustra la classificazione su un piano bidimensionale con il parametro di soglia $MinPts = 7$.

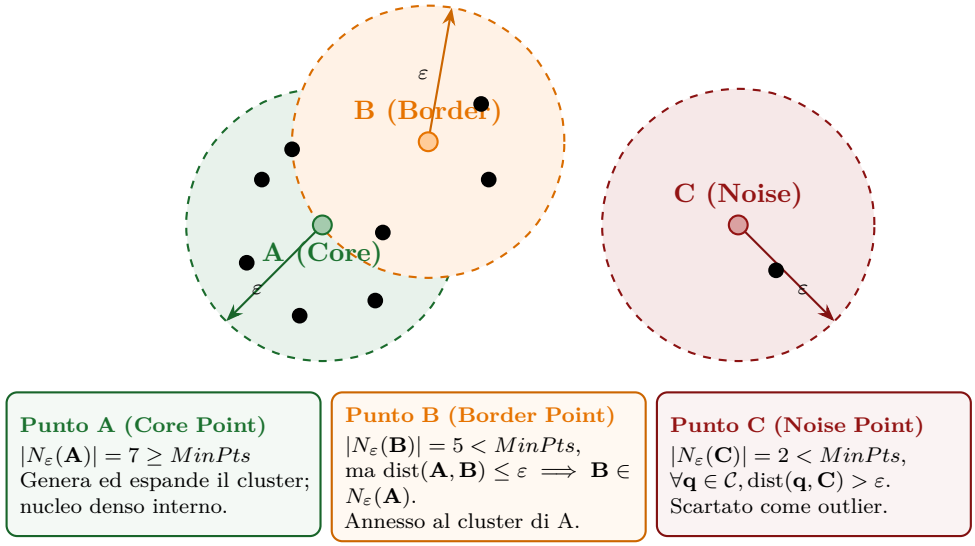


Figura 8.2: Rappresentazione geometrica della tassonomia di DBSCAN ($MinPts = 7$). Il punto A è Core ($|N_\varepsilon(A)| \geq 7$), B è Border ($|N_\varepsilon(B)| < 7$, ma entro distanza ε dal core A) e C è Noise (isolato e lontano da qualsiasi nucleo denso).

Dall'analisi geometrica emergono con chiarezza tre comportamenti:

1. **Punto A (Core Point)**: Tracciando il cerchio di raggio ε centrato in A, il conteggio comprensivo di A totalizza 7 osservazioni. Poiché $|N_\varepsilon(A)| \geq 7$, A è a pieno titolo un core point.

2. **Punto B (Border Point)**: Tracciando il medesimo cerchio di raggio ε centrato in B, si rinvengono soltanto 5 punti totali ($|N_\varepsilon(B)| = 5 < 7$). Tuttavia, la distanza euclidea tra A e B soddisfa $\text{dist}(A, B) \leq \varepsilon$, il che colloca $B \in N_\varepsilon(A)$. Essendo A un core point, B eredita la classificazione di punto di bordo associato ad A.

3. **Punto C (Noise Point)**: Il cerchio centrato in C cattura solamente 2 osservazioni ($|N_\varepsilon(C)| = 2 \ll 7$). Inoltre, la distanza tra C e qualunque core point supera rigorosamente ε. Di conseguenza, C viene classificato come rumore di fondo.

8.3 Le Relazioni di Connettività per Densità

Per costruire i cluster unendo i punti densi ed espandendo la frontiera senza incorrere in fallimenti geometrici, DBSCAN formalizza tre relazioni algebriche fondamentali tra coppie di punti:

8.3.1 Raggiungibilità Diretta per Densità (*Directly Density-Reachable*)

Definizione 8.3.1 (Raggiungibilità Diretta per Densità)**.** Un punto $\mathbf{q} \in \mathcal{D}$ è **direttamente raggiungibile per densità** da un punto $\mathbf{p} \in \mathcal{D}$ rispetto ai parametri ε e $MinPts$ (notazione $\mathbf{p} \rightarrow_{\text{dir}} \mathbf{q}$) se:

1. $\mathbf{p}$ è un Core Point, ossia $|N_\varepsilon(\mathbf{p})| \geq MinPts$;
2. $\mathbf{q}$ appartiene all'ε-intorno di $\mathbf{p}$, ossia $\mathbf{q} \in N_\varepsilon(\mathbf{p})$ ($\text{dist}(\mathbf{p}, \mathbf{q}) \leq \varepsilon$).

Proprietà 8.3.1 (Asimmetria della Raggiungibilità Diretta)**.** *La relazione di raggiungibilità diretta è in generale* ***asimmetrica****:*

$$\mathbf{p} \rightarrow_{\text{dir}} \mathbf{q} \implies \mathbf{q} \rightarrow_{\text{dir}} \mathbf{p} \tag{8.9}$$

Infatti, se $\mathbf{p} \in \mathcal{C}$ *e* $\mathbf{q} \in \mathcal{B}$*, allora* $\mathbf{q}$ *è direttamente raggiungibile da* $\mathbf{p}$*. Tuttavia, non essendo* $\mathbf{q}$ *un core point (*$|N_\varepsilon(\mathbf{q})| < MinPts$*), nessun punto può essere direttamente raggiungibile per densità da* $\mathbf{q}$*. La relazione diventa* ***simmetrica*** *se e solo se entrambi i nodi appartengono all'insieme dei core point:*

$$\Big(\mathbf{p} \in \mathcal{C} \ \wedge \ \mathbf{q} \in \mathcal{C} \ \wedge \ \text{dist}(\mathbf{p}, \mathbf{q}) \leq \varepsilon\Big) \iff \Big(\mathbf{p} \rightarrow_{\text{dir}} \mathbf{q} \ \wedge \ \mathbf{q} \rightarrow_{\text{dir}} \mathbf{p}\Big) \tag{8.10}$$

8.3.2 Raggiungibilità per Densità (*Density-Reachable*)

Definizione 8.3.2 (Raggiungibilità per Densità)**.** Un punto $\mathbf{q} \in \mathcal{D}$ è **raggiungibile per densità** da un punto $\mathbf{p} \in \mathcal{D}$ rispetto a ε e $MinPts$ (notazione $\mathbf{p} \rightsquigarrow \mathbf{q}$) se esiste una sequenza finita di punti in $\mathcal{D}$, denominata **catena di densità**:

$$\mathbf{p}_1, \mathbf{p}_2, \ldots, \mathbf{p}_m \tag{8.11}$$

tale per cui:

1. $\mathbf{p}_1 = \mathbf{p}$;
2. $\mathbf{p}_m = \mathbf{q}$;
3. Per ogni $i \in \{1, 2, \ldots, m-1\}$, il punto $\mathbf{p}_{i+1}$ è direttamente raggiungibile per densità da $\mathbf{p}_i$ ($\mathbf{p}_i \rightarrow_{\text{dir}} \mathbf{p}_{i+1}$).

Nota 8.3.1 (Ruolo dei Punti Intermedi della Catena)**.** Dalla definizione segue che tutti i punti intermedi della catena $\mathbf{p}_1, \mathbf{p}_2, \ldots, \mathbf{p}_{m-1}$ devono essere obbligatoriamente dei **Core Point**. Soltanto l'ultimo elemento $\mathbf{p}_m = \mathbf{q}$ può essere un Border Point. Ne consegue che la relazione $\mathbf{p} \rightsquigarrow \mathbf{q}$ è non-simmetrica quando l'estremo finale è un border point: da un core point si può raggiungere il bordo, ma da un punto di bordo la propagazione si interrompe istantaneamente.

8.3.3 Connettività per Densità (*Density-Connected*)

La mancanza di simmetria nella raggiungibilità per densità impedisce di utilizzarla direttamente come relazione di equivalenza per partizionare i punti in cluster. Per superare questa asimmetria, viene introdotta la nozione di connettività:

Definizione 8.3.3 (Connettività per Densità)**.** Due punti $\mathbf{p}, \mathbf{q} \in \mathcal{D}$ sono **connessi per densità** rispetto a ε e $MinPts$ (notazione $\mathbf{p} \sim_{\text{dens}} \mathbf{q}$) se esiste un terzo punto mediatore $\mathbf{o} \in \mathcal{D}$ (necessariamente un Core Point) da cui sia $\mathbf{p}$ sia $\mathbf{q}$ risultano raggiungibili per densità:

$$\mathbf{p} \sim_{\text{dens}} \mathbf{q} \iff \exists \mathbf{o} \in \mathcal{D} \quad \text{tale che} \quad (\mathbf{o} \rightsquigarrow \mathbf{p}) \ \wedge \ (\mathbf{o} \rightsquigarrow \mathbf{q}) \tag{8.12}$$

Proprietà 8.3.2 (Proprietà della Connettività per Densità)**.** *La relazione di connettività per densità* $\sim_{\text{dens}}$ *gode delle seguenti proprietà algebriche:*

1. ***Riflessività****: Per ogni punto* $\mathbf{p} \in \mathcal{C}$ *(o* $\mathbf{p} \in \mathcal{B}$ *raggiungibile da un core* $\mathbf{o}$*),* $\mathbf{p} \sim_{\text{dens}} \mathbf{p}$*;*
2. ***Simmetria****: Se* $\mathbf{p} \sim_{\text{dens}} \mathbf{q}$*, allora per commutatività della congiunzione logica* $\mathbf{q} \sim_{\text{dens}} \mathbf{p}$*;*
3. ***Transitività sui Core Point****: Se* $\mathbf{p}_1 \sim_{\text{dens}} \mathbf{p}_2$ *e* $\mathbf{p}_2 \sim_{\text{dens}} \mathbf{p}_3$ *con* $\mathbf{p}_2 \in \mathcal{C}$*, allora* $\mathbf{p}_1 \sim_{\text{dens}} \mathbf{p}_3$*.*

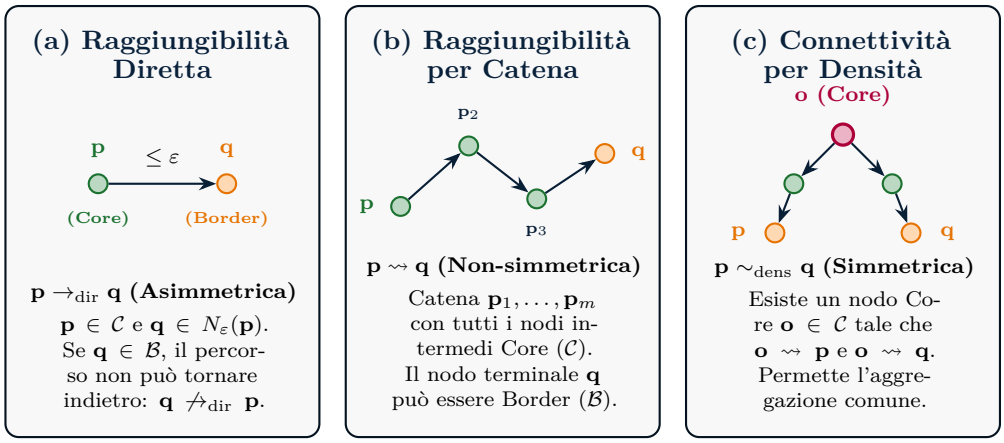


Figura 8.3: Formalizzazione algebrica delle tre relazioni spaziali di densità in DBSCAN.

8.3.4 Definizione Formale di Cluster in DBSCAN

Alla luce delle relazioni introdotte, è possibile formulare la definizione teorica rigorosa di cluster adottata da DBSCAN [1]:

Definizione 8.3.4 (Cluster Basato su Densità)**.** Dato un dataset $\mathcal{D}$, un **cluster** $\mathcal{C}_k \subseteq \mathcal{D}$ rispetto ai parametri ε e $MinPts$ è un sottoinsieme non vuoto di punti che soddisfa simultaneamente due requisiti assiomatici:

1. **Massimalità (*Maximality*)**: Se $\mathbf{p} \in \mathcal{C}_k$ e un punto $\mathbf{q} \in \mathcal{D}$ è raggiungibile per densità da $\mathbf{p}$ ($\mathbf{p} \rightsquigarrow \mathbf{q}$), allora $\mathbf{q} \in \mathcal{C}_k$;
2. **Connettività (*Connectivity*)**: Per ogni coppia di punti $\mathbf{p}, \mathbf{q} \in \mathcal{C}_k$, $\mathbf{p}$ e $\mathbf{q}$ sono connessi per densità ($\mathbf{p} \sim_{\text{dens}} \mathbf{q}$).

Teorema 8.3.1 (Caratterizzazione dei Cluster di DBSCAN)**.** *Sia $\mathcal{C}_k$ un cluster in $\mathcal{D}$ e sia $\mathbf{p} \in \mathcal{C}_k$ un qualunque suo Core Point. Allora l'insieme $\mathcal{C}_k$ è univocamente generato come l'insieme di tutti i punti raggiungibili per densità da $\mathbf{p}$:*

$$\mathcal{C}_k = \left\{ \mathbf{q} \in \mathcal{D} \;\middle|\; \mathbf{p} \rightsquigarrow \mathbf{q} \right\} \tag{8.13}$$

Il Teorema 8.3.1 stabilisce il principio operativo dell'algoritmo: per espandere e scoprire completamente un intero cluster è sufficiente individuare un singolo core point arbitrario $\mathbf{p}$ e tracciare la chiusura transitiva di tutti i punti raggiungibili per densità da esso.

8.4 Architettura Operativa dell'Algoritmo DBSCAN

L'algoritmo DBSCAN trasforma le proprietà teoriche di connettività in una procedura sequenziale ad alta efficienza. L'esecuzione si articola logicamente in tre macro-fasi sequenziali.

8.4.1 Le Tre Fasi Operative

1. **Fase 1: Classificazione Iniziale dei Punti (*Point Labeling*)**
   - Per ogni punto $\mathbf{p} \in \mathcal{D}$, viene eseguita una query spaziale di vicinato (*neighborhood query*) calcolando $N_\varepsilon(\mathbf{p}) = \{\mathbf{q} \in \mathcal{D} \mid \text{dist}(\mathbf{p}, \mathbf{q}) \leq \varepsilon\}$.
   - Se $|N_\varepsilon(\mathbf{p})| \geq MinPts$, il punto riceve lo stato `CORE`;
   - In caso contrario, riceve lo stato provvisorio `UNVISITED` o `NOISE` (in attesa di valutare se ricadrà nel raggio di un core point).

2. **Fase 2: Connessione dei Nuclei (*Core Point Graph Clustering*)**

    - Si considera il sottografo non orientato $G_{\text{core}} = (\mathcal{C}, E_{\text{core}})$ indotto dai soli punti marcati `CORE`, dove un arco non orientato $(\mathbf{u}, \mathbf{v}) \in E_{\text{core}}$ esiste se e solo se $\text{dist}(\mathbf{u}, \mathbf{v}) \leq \varepsilon$.
    - Le componenti connesse massimali di G_{core} identificano biunivocamente l'ossatura portante dei singoli cluster.

3. **Fase 3: Assegnazione dei Punti di Bordo ed Eliminazione del Rumore**

    - Per ogni punto $\mathbf{b} \in \mathcal{B}$ non-core che si trova nell'ε-intorno di almeno un core point $\mathbf{c} \in \mathcal{C}$, $\mathbf{b}$ viene aggregato al cluster di appartenenza di $\mathbf{c}$.
    - Tutti i punti che al termine dell'espansione non appartengono ad alcun intorno di un core point vengono marcati definitivamente come `NOISE` (contrassegnati tipicamente con il simbolo $\times$ ed espulsi dal partizionamento).

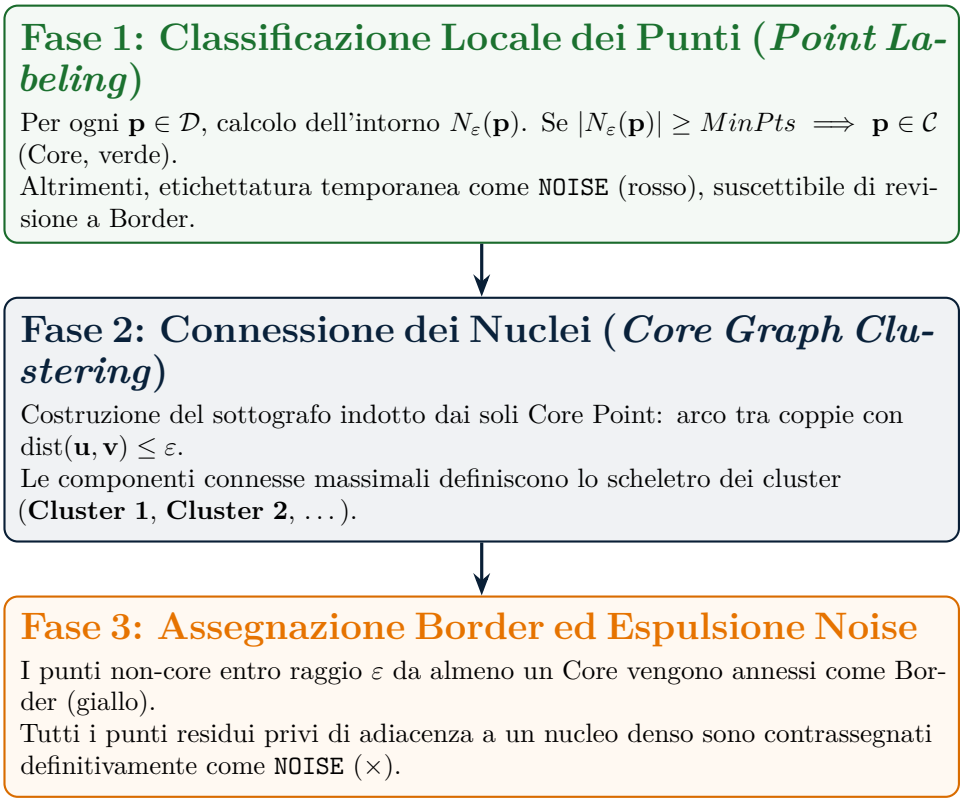


Figura 8.4: Sequenza delle tre fasi computazionali per la formazione dei cluster in DBSCAN.

8.4.2 La Questione della Non-Determinatezza dei Border Point Condivisi

Mentre la partizione dei Core Point e l'isolamento dei Noise Point sono operazioni rigorosamente deterministiche e indipendenti dall'ordine sequenziale con cui i dati vengono letti in memoria, l'assegnazione dei **Border Point contesi** presenta una sottile proprietà teorica:

Proprietà 8.4.1 (Non-Determinatezza dell'Assegnazione dei Border Point)**.** *Siano $\mathcal{C}_1$ e $\mathcal{C}_2$ due cluster distinti generati da componenti connesse disgiunte di core point. Se un border point $\mathbf{b} \in \mathcal{B}$ soddisfa simultaneamente:*

$$\begin{aligned} &\exists \mathbf{c}_1 \in \mathcal{C}_1 \cap \mathcal{C} \quad \textit{tale che} \quad \mathrm{dist}(\mathbf{b}, \mathbf{c}_1) \leq \varepsilon, \\ &\exists \mathbf{c}_2 \in \mathcal{C}_2 \cap \mathcal{C} \quad \textit{tale che} \quad \mathrm{dist}(\mathbf{b}, \mathbf{c}_2) \leq \varepsilon, \end{aligned} \tag{8.14}$$

*allora $\mathbf{b}$ è raggiungibile per densità sia dal cluster $\mathcal{C}_1$ sia dal cluster $\mathcal{C}_2$. Tuttavia, poiché $\mathbf{b}$ non è un core point ($|N_\varepsilon(\mathbf{b})| < MinPts$), esso **non funge***

da ponte di propagazione *tra* $\mathbf{c}_1$ *e* $\mathbf{c}_2$*, preservando l'indipendenza e la separazione dei due cluster.*

Di conseguenza, l'assegnazione finale di $\mathbf{b}$ *al Cluster 1 oppure al Cluster 2 dipende unicamente dall'****ordine di scansione sequenziale*** *dei punti nell'iterazione principale dell'algoritmo: il cluster il cui core point esamina per primo il punto* $\mathbf{b}$ *ne rivendicherà l'appartenenza esclusiva.*

In termini pratici, la frazione di border point contesi nei dataset reali è trascurabile e non altera la topologia macroscopica dei cluster individuati.

8.4.3 Pseudocodice dell'Algoritmo DBSCAN

Nelle implementazioni pratiche, le tre fasi teoriche vengono integrate elegantemente all'interno di un'unica procedura di visita guidata da code o liste di frontiera (equivalente a una visita in ampiezza BFS o in profondità DFS), come formalizzato nell'Algoritmo 4.

Algorithm 4: Algoritmo DBSCAN con espansione a coda di densità.

Dati: Dataset $\mathcal{D} = \{\mathbf{p}_1, \ldots, \mathbf{p}_n\}$, raggio metrico $\varepsilon > 0$, soglia di vicinato $MinPts \in \mathbb{N}^+$.

Risultato: Vettore di etichette $\mathbf{y} \in (\mathbb{N} \cup \{\texttt{NOISE}\})^n$.

Inizializza tutte le etichette: $y[\mathbf{p}] \leftarrow \texttt{UNVISITED}$ per ogni $\mathbf{p} \in \mathcal{D}$
cluster_id $\leftarrow 0$
foreach $\mathbf{p} \in \mathcal{D}$ **do**
    if $y[\mathbf{p}] \neq$ *UNVISITED* **then**
        continue
        // Punto già processato in una precedente espansione
    end
    $\mathcal{N}_\mathbf{p} \leftarrow \textsc{RangeQuery}(\mathcal{D}, \mathbf{p}, \varepsilon)$
    if $|\mathcal{N}_\mathbf{p}| < MinPts$ **then**
        $y[\mathbf{p}] \leftarrow \texttt{NOISE}$
        // Etichetta provvisoria: potrebbe essere un Border
    end
    else
        cluster_id $\leftarrow$ cluster_id $+ 1$
        $y[\mathbf{p}] \leftarrow$ cluster_id
        $\mathcal{S} \leftarrow \mathcal{N}_\mathbf{p} \setminus \{\mathbf{p}\}$
        // Insieme di espansione (coda o lista)
        while $\mathcal{S} \neq \emptyset$ **do**
            Estrai un punto $\mathbf{q}$ da $\mathcal{S}$
            if $y[\mathbf{q}] ==$ *NOISE* **then**
                $y[\mathbf{q}] \leftarrow$ cluster_id
                // Riconversione da Noise a Border Point
            end
            if $y[\mathbf{q}] \neq$ *UNVISITED* **then**
                continue
            end
            $y[\mathbf{q}] \leftarrow$ cluster_id
            $\mathcal{N}_\mathbf{q} \leftarrow \textsc{RangeQuery}(\mathcal{D}, \mathbf{q}, \varepsilon)$
            if $|\mathcal{N}_\mathbf{q}| \geq MinPts$ **then**
                $\mathcal{S} \leftarrow \mathcal{S} \cup \mathcal{N}_\mathbf{q}$
                // Propaga espansione: anche $\mathbf{q}$ è un Core Point
            end
        end
    end
end

8.4.4 Analisi della Complessità Computazionale

L'efficienza operativa di DBSCAN è dominata dal costo computazionale delle interrogazioni di vicinato RangeQuery$(\mathcal{D}, \mathbf{p}, \varepsilon)$, che vengono invocate per ogni punto del dataset.

- **Approccio Naive (Scansione Lineare Senza Indici Spaziali)**: In assenza di strutture dati di indicizzazione geometrica, la determinazione dell'intorno $N_\varepsilon(\mathbf{p})$ richiede di calcolare la distanza euclidea tra $\mathbf{p}$ e tutti gli altri $n-1$ elementi del dataset, con costo $\mathcal{O}(n \cdot d)$. Eseguendo la scansione per tutti gli n punti, la complessità temporale complessiva è puramente quadratica:

$$\mathcal{T}_{\text{naive}}(n) = \mathcal{O}(n^2 \cdot d) \tag{8.15}$$

- **Approccio Ottimizzato con Strutture Dati ad Albero Spaziale**: Qualora lo spazio presenti dimensionalità ridotta o moderata ($d \leq 10$), le osservazioni possono essere preventivamente indicizzate mediante strutture ad albero spaziale, quali **k-d tree** (per dati continui isotropi) oppure **R^*-tree** (per dati geometrici generali). In tale configurazione:

  1. La costruzione dell'albero richiede tempo $\mathcal{O}(d \cdot n \log n)$;
  2. Ciascuna query di raggio ε viene risolta mediamente in tempo logaritmico $\mathcal{O}(\log n)$;
  3. La complessità temporale totale dell'algoritmo si riduce a:

$$\mathcal{T}_{\text{tree}}(n) = \mathcal{O}(n \log n) \tag{8.16}$$

- **Alta Dimensionalità e la Maledizione della Dimensionalità**: Quando il numero di dimensioni cresce significativamente ($d \gg 10$), le prestazioni degli indici ad albero collassano (fenomeno del *dimensional curse*). Il partizionamento dello spazio richiede di esplorare quasi tutte le celle dell'albero per garantire l'esattezza della ricerca, facendo degradare il tempo per query da $\mathcal{O}(\log n)$ a $\mathcal{O}(n)$. In alta dimensionalità DBSCAN torna a richiedere tempo $\mathcal{O}(n^2)$.

- **Complessità Spaziale**: La memoria principale deve ospitare il dataset $\mathcal{D}$, l'array delle etichette di cluster y, la struttura ausiliaria per la coda di visita $\mathcal{S}$ e, se utilizzato, l'albero di indicizzazione. La complessità spaziale complessiva è dunque strettamente lineare:

$$\mathcal{S}(n) = \mathcal{O}(n \cdot d) \tag{8.17}$$

8.5 Analisi Fenomenologica su Dati Spaziali Continui

Per comprendere intuitivamente l'azione combinata dei parametri ε e $MinPts$, si consideri la distribuzione spaziale continua presentata nelle dispense di riferimento [2], caratterizzata da due corpi ad alta densità collegati da un sottile istmo (*ponte di connettività*) e immersi in una nube di punti sparsi e isolati.

8.5.1 Comportamento con Parametri $\varepsilon = 10$ e $MinPts = 4$

Impostando un raggio metrico $\varepsilon = 10$ e una soglia minima di 4 vicini ($MinPts = 4$):

1. **Struttura Portante (*Interior Backbone*)**: All'interno delle due masse principali, la concentrazione spaziale è tale che la distanza media tra vicini è marcatamente inferiore a 10. Di conseguenza, quasi tutti i campioni interni accumulano tra 8 e 25 vicini entro il raggio 10, eccedendo ampiamente la soglia $MinPts = 4$. Essi vengono marchiati come Core Point (blu scuro) e costituiscono l'ossatura densa del cluster.

2. **Frontiera di Bordo (*Boundary Definition*)**: Man mano che ci si allontana dal baricentro verso la periferia dei corpi densi, il gradiente di densità decresce. Lungo il perimetro esterno, i punti contano meno di 4 vicini nel proprio raggio, ma conservano una distanza ≤ 10 da almeno un core point interno. Questi punti assumono stabilmente il ruolo di Border Point (rossi), avvolgendo uniformemente la massa senza slabbrature.

3. **Trattamento del Ponte di Connettività**: Se l'istmo sottile che unisce le due masse contiene punti la cui distanza reciproca è ≤ 10 e ciascuno di essi ha almeno 4 vicini lungo il filamento, i punti del ponte risultano essere a loro volta Core Point. In tal caso, DBSCAN propaga la catena di raggiungibilità fondendo le due masse in un unico macro-cluster. Al contrario, se la densità del ponte scende al di sotto di 4 punti per raggio 10, i punti dell'istmo non sono Core Point e le due masse rimangono rigorosamente separate in due cluster indipendenti.

4. **Isolamento Naturale del Rumore**: I punti isolati dispersi nello spazio vuoto circostante distano mediamente più di 10 unità da qualunque

nucleo denso e presentano ≤ 2 vicini nel proprio raggio. Essi vengono identificati come Noise Point (grigio chiaro/crocette), impedendo qualsiasi distorsione della frontiera del cluster.

8.6 Punti di Forza e Capacità Geometriche

L'algoritmo DBSCAN ha acquisito un ruolo fondamentale nella letteratura di data mining grazie a proprietà geometriche e operative che superano molti dei limiti intrinseci di K-Means:

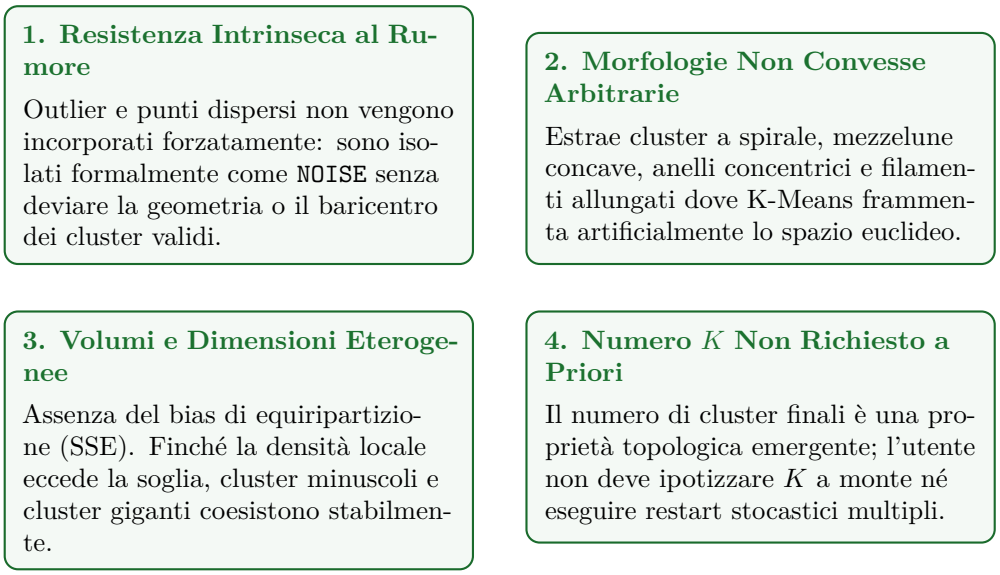


Figura 8.5: Quattro punti di forza architetturali di DBSCAN rispetto ai modelli basati su prototipo.

1. **Capacità di Individuare Cluster di Forma Arbitraria**: K-Means impone implicitamente cluster convessi delimitati da iperpiani bisettori (celle di Voronoi). DBSCAN definisce il cluster tramite percorsi di continuità topologica ad alta densità; pertanto, è perfettamente in grado di estrarre geometrie complesse come anelli concentrici, spirali intrecciate e distribuzioni a forma di mezza luna.

2. **Indipendenza dalla Cardinalità e dalla Dimensione dei Cluster**: K-Means, minimizzando l'errore quadratico totale $\text{SSE} = \sum_k \sum_{\mathbf{x} \in C_k} \|\mathbf{x} - \mathbf{m}_k\|^2$, manifesta una tendenza sistematica a generare cluster di dimensioni e volumi bilanciati, frammentando cluster reali estesi o accorpando cluster compatti. DBSCAN, al contrario, applica un criterio puramente

intensivo basato sulla densità locale $MinPts/V_d(\varepsilon)$: finché la densità locale eccede la soglia critica, un cluster di un milione di elementi convive stabilmente accanto a un cluster di venti elementi.

3. **Filtraggio Robusto di Rumore e Outlier**: A differenza di algoritmi che forzano una partizione rigida di tutto il dominio ($\bigcup_k C_k = \mathcal{D}$), DBSCAN ammette che sottoinsiemi arbitrari di punti siano classificati come rumore ($\mathcal{N} \neq \emptyset$). Ciò garantisce che campioni anomali o misurazioni corrotte non influenzino la struttura dei cluster validi.

4. **Nessuna Scelta a Priori del Parametro** K: L'utente non è costretto a formulare ipotesi a priori sul numero di raggruppamenti presenti. Il numero effettivo di cluster è un risultato emergente determinato dalla connettività delle regioni dense.

8.7 Limitazioni Critiche e Sfide Applicative

Nonostante i notevoli vantaggi, DBSCAN presenta due serie vulnerabilità strutturali che ne limitano l'impiego in determinati contesti applicativi.

8.7.1 1. Il Problema delle Densità Variabili (*Varying Densities*)

La limitazione più severa di DBSCAN risiede nel fatto che i parametri ε e $MinPts$ sono **costanti globali** applicate uniformemente all'intero dataset. Quando i dati presentano cluster dotati di densità naturali fortemente disomogenee, DBSCAN entra in una contraddizione insolubile:

Tentativo A: ε Eccessivo

- Cattura correttamente i cluster rarefatti (a bassa densità).
- **Criticità**: I cluster compatti adiacenti e le nubi dense collassano e vengono fusi in un unico macro-cluster amorfo.

Tentativo B: ε Troppo Ridotto

- Separa e isola i nuclei compatti ad altissima densità.
- **Criticità**: I cluster reali a densità medio-bassa scendono sotto la soglia di core point e vengono scartati come `NOISE`.

Figura 8.6: Il dilemma della soglia globale in presenza di cluster con densità eterogenee.

Si consideri l'esperimento comparativo discusso nelle dispense di riferimento [2]: un dataset composito contenente un'ampia ellisse rarefatta a sinistra,

una nube a densità intermedia al centro e tre nuclei iper-compatti nidificati all'interno di una nube densa a destra.

- Impostando $MinPts = 4$ ed $\varepsilon = 9.92$: DBSCAN cattura l'ellisse rarefatta, ma fonda l'intera porzione densa di destra in un solo megacluster indiscriminato, cancellando la separazione tra la nube e i nuclei compatti.
- Riducendo minimamente la soglia a $\varepsilon = 9.75$ ($MinPts = 4$): l'algoritmo riesce a isolare i tre micro-nuclei ad alta densità a destra, ma l'intera ellisse rarefatta di sinistra e la nube centrale decadono sotto la soglia critica di core point, venendo completamente espulse come rumore (`NOISE`).

Questo limite strutturale ha stimolato lo sviluppo di algoritmi avanzati basati su gerarchie di densità, quali:

- **OPTICS** (*Ordering Points To Identify the Clustering Structure*) [3]: genera un ordinamento lineare aumentato dei punti basato su distanze di raggiungibilità (*reachability-distance*), permettendo di visualizzare e sezionare strutture a densità variabile mediante un reachability plot;
- **HDBSCAN** (*Hierarchical DBSCAN*) [4]: integra la nozione di densità locale con il clustering gerarchico, estraendo automaticamente cluster a densità variabile massimizzando la stabilità (*excess of mass*) lungo il dendrogramma.

8.7.2 2. La Maledizione della Dimensionalità (*Curse of Dimensionality*)

La seconda criticità fondamentale si manifesta quando la dimensionalità delle feature cresce ($d \gg 1$). Come ampiamente discusso nel Capitolo 4 (sottosezione 4.3.1), la diluizione iper-volumetrica e l'uniformazione delle distanze relative in spazi ad alta dimensione fanno sì che qualunque ipersfera di raggio ε sia completamente vuota, oppure racchiuda quasi tutto il dataset con un incremento infinitesimo di ε. Di conseguenza, in alta dimensionalità diventa virtualmente impossibile identificare una soglia metrica ε che separi regioni dense da regioni rade, rendendo il concetto di densità locale euclidea privo di significato operativo.

8.8 Metodologia Euristica per la Scelta dei Parametri (Eps e MinPts)

Una delle problematiche pratiche più cruciali nell'applicazione di DBSCAN risiede nella determinazione ottimale della coppia di parametri $(\varepsilon, MinPts)$. Una scelta arbitraria o casuale porta inevitabilmente o al collasso di tutti i cluster in un unico blocco o alla classificazione della quasi totalità dei dati come rumore.

Per risolvere questo problema in modo sistematico e guidato dai dati, si adotta una metodologia euristica rigorosa basata sulla distribuzione delle **distanze dal k-esimo vicino più prossimo** (*k-nearest neighbor distance plot* o **k-distance plot**) [1, 2].

8.8.1 Principio Teorico Fondamentale

L'euristica sfrutta la divergenza strutturale tra campioni appartenenti a cluster densi e campioni di rumore isolati:

- **Punti interni a un cluster**: risiedono in aree ad alta concentrazione spaziale, per cui la distanza che separa ciascun punto dal proprio k-esimo vicino più prossimo è ridotta e sostanzialmente omogenea in tutto il cluster.
- **Punti di rumore e outlier**: risiedono in regioni rade o nello spazio vuoto inter-cluster, per cui la distanza che li separa dal k-esimo vicino è marcatamente superiore rispetto a quella tipica dei cluster.

8.8.2 Procedura Operativa Passo-Passo

La costruzione del k-distance plot segue quattro passaggi algoritmici:

1. **Fissazione del Parametro k ($MinPts$)**: Si imposta $k = MinPts$. Come regola empirica consolidata per spazi euclidei di dimensione d:

$$MinPts \geq d + 1 \tag{8.18}$$

   Per ridurre l'influenza di piccole fluttuazioni stocastiche di campionamento e rendere l'euristica più robusta, per dati bidimensionali ($d = 2$) si adotta frequentemente $MinPts = 4$ (o più in generale $MinPts \approx 2d$).

2. **Calcolo delle Distanze dai k-NN**: Per ciascun punto $\mathbf{p}_i \in \mathcal{D}$ ($i = 1, \ldots, n$):

- Si calcolano le distanze euclidee da tutti gli altri $n-1$ punti;
- Si ordinano le distanze in ordine crescente:

$$d_{(1)}(\mathbf{p}_i) \leq d_{(2)}(\mathbf{p}_i) \leq \cdots \leq d_{(k)}(\mathbf{p}_i) \leq \cdots \leq d_{(n-1)}(\mathbf{p}_i) \tag{8.19}$$

- Si estrae il valore $D_i = d_{(k)}(\mathbf{p}_i)$, che quantifica la distanza metrica di $\mathbf{p}_i$ dal suo k-esimo vicino più prossimo.

3. **Ordinamento Globale e Tracciamento della Curva**: Si ordina l'array delle distanze $\{D_1, D_2, \ldots, D_n\}$ in senso strettamente crescente:

$$D_{\pi(1)} \leq D_{\pi(2)} \leq \cdots \leq D_{\pi(n)} \tag{8.20}$$

dove π denota la permutazione di ordinamento. Si traccia quindi la funzione a gradino/curva discreta ponendo:

- Sull'**Asse delle Ascisse (X)**: l'indice dei punti ordinati $m \in \{1, 2, \ldots, n\}$;
- Sull'**Asse delle Ordinate (Y)**: il valore di distanza $D_{\pi(m)}$.

4. **Individuazione del "Knee Point" (Gomito della Curva)**: La curva risultante presenta tipicamente una morfologia caratteristica articolata in tre segmenti (Figura 8.7):

- **Segmento Piatto Iniziale ($D \leq 8$)**: corrisponde ai punti interni ai corpi densi. Le distanze crescono molto lentamente poiché i campioni sono densamente impaccati;
- **Il Gomito (*Knee Point*)**: punto di massima curvatura locale. Rappresenta la transizione di fase critica tra il regime di alta densità intracluster e il regime di rarefazione del rumore;
- **Segmento Asintotico Ripido ($D \geq 15$)**: coda terminale di punti isolati per cui la distanza dal k-esimo vicino esplode verso l'alto.

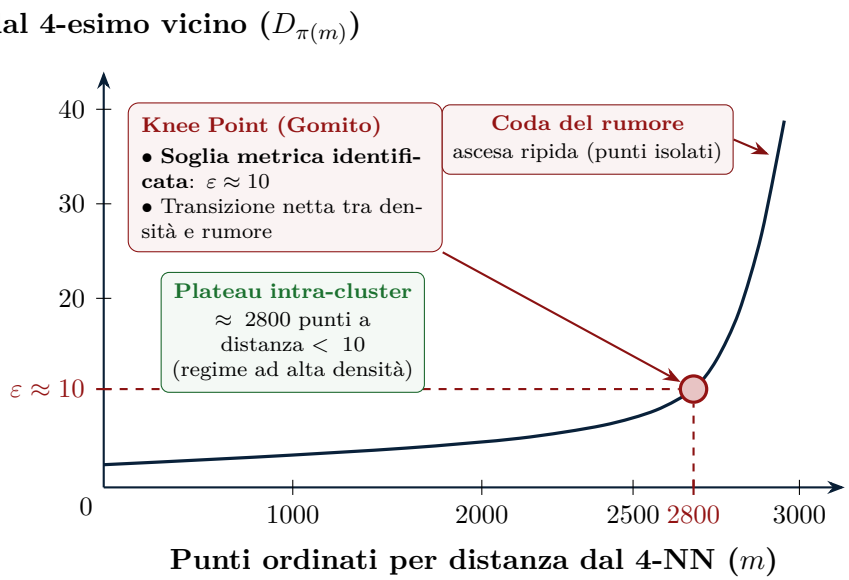


Figura 8.7: Grafico del k-distance plot ($k = 4$) per circa 3000 campioni. Il valore di soglia ottimale ε corrisponde alla quota sull'asse delle ordinate del punto di flesso (Knee Point), qui rilevabile a $\varepsilon \approx 10$.

8.8.3 Interpretazione e Selezione di $\varepsilon \approx 10$

Nel grafico della Figura 8.7:

- La quasi totalità dei punti (circa 2800 su 3000) possiede il proprio 4° vicino entro una distanza ridotta, compresa tra 2 e 8. Questi punti formano il plateau iniziale orizzontale.
- Il gomito della curva si colloca nitidamente a una quota sull'asse Y pari a $\varepsilon \approx 10$.
- Selezionando $\varepsilon = 10$, tutti i punti situati prima del gomito conterranno almeno 4 vicini entro il raggio 10, qualificandosi come candidati Core Point. Al contrario, i circa 200 punti collocati nella ripida ascesa asintotica avranno il loro 4° vicino oltre la soglia 10, venendo isolati correttamente come punti di rumore (`NOISE`).

Questa calibrazione analitica corrisponde esattamente ai valori empirici adottati nell'esempio applicativo della Sezione 8.5 ($\varepsilon = 10, MinPts = 4$).

8.9 Quadro Comparativo Globale: K-Means vs Gerarchico vs DBSCAN

A conclusione della trattazione dei tre paradigmi cardine dell'apprendimento non supervisionato analizzati nei Capitoli 7 e 8, la Tabella 8.1 fornisce un confronto sinottico esaustivo che ne sintetizza le proprietà topologiche, computazionali e ingegneristiche.

Proprietà Funzionale	**K-Means Clustering**	**Clustering Gerarchico**	**DBSCAN (Density-Based)**
Modello Fondamentale	Partizionale a prototipo (baricentroide)	Agglomerativo/Divisivo ad albero nidificato	Connettività locale ad alta densità
Numero di Cluster K	Obbligatorio specificarlo a priori	Derivabile a posteriori tagliando il dendrogramma	Determinazione automatica emergente dai dati
Geometria dei Cluster	Esclusivamente convessa/iper-sferica (Voronoi)	Arbitraria con Single Link; compatta/sferica con Ward	Arbitraria (concava, anelli, spirali, densità filamentose)
Sensibilità al Rumore	Molto elevata (gli outlier spostano i centroidi)	Dipendente dal linkage (Single Link soggetto a *chaining*)	Intrinsecamente immune (isolamento formale come `NOISE`)
Dimensioni/Densità	Assume dimensioni, volumi e varianze comparabili	Flessibile, ma decisioni miopi non revisionabili	Gestisce volumi e cardinalità eterogenee; fallisce su densità disomogenee
Complessità Temporale	$\mathcal{O}(n \cdot K \cdot I \cdot d)$ (Lineare con n)	$\mathcal{O}(n^2 \log n)$ fino a $\mathcal{O}(n^3)$ (Sovra-quadratica)	$\mathcal{O}(n \log n)$ con indici spaziali; $\mathcal{O}(n^2 \cdot d)$ naive
Complessità Spaziale	$\mathcal{O}((n + K) \cdot d)$ (Lineare)	$\mathcal{O}(n^2)$ (Matrice di dissimilarità globale)	$\mathcal{O}(n \cdot d)$ (Lineare)
Scalabilità sui Dati	Eccellente (gestisce decine di milioni di record)	Molto ridotta ($n \leq 20\,000$ vincolato da RAM)	Buona a moderata (eccellente per $n \leq 10^6$ a basse dimensioni)
Impatto Alta Dimensione	Soffre moderatamente per metrica euclidea	Distorsione globale delle distanze a coppie	Degenerazione severa (perdita di contrasto di densità locale)
Determinatezza	Non-deterministico (dipende dall'inizializzazione)	Rigorosamente deterministico	Deterministico per Core e Noise; ordine-dipendente per Border contesi

Tabella 8.1: Confronto sinottico omnicomprensivo tra i paradigmi di clustering: Partizionale (K-Means), Gerarchico e Basato su Densità (DBSCAN).

Capitolo 9

Varianti di K-Means, Criterio BIC, Modelli Mistura ed EM

9.1 Inquadramento Scientifico e Limiti del K-Means Standard

L'algoritmo K-Means standard (algoritmo di Lloyd), formalizzato nel Capitolo 7 (sezione 7.2), opera individuando K centroidi $\mathbf{C} = \{\mathbf{c}_1, \ldots, \mathbf{c}_K\}$ che minimizzano la devianza globale intra-cluster SSE (Equazione 7.6). Sebbene la procedura sia computazionalmente lineare ad ogni ciclo ($\mathcal{O}(N \cdot K \cdot d)$), le analisi condotte nella sezione 7.6 hanno evidenziato tre vulnerabilità strutturali fondamentali che ne limitano l'affidabilità nei contesti applicativi reali:

1. **Estrema sensibilità all'inizializzazione**: Il problema di minimizzazione dell'SSE è intrinsecamente non convesso ed è noto essere $\mathcal{NP}$-arduo persino nel piano bidimensionale. L'algoritmo di Lloyd garantisce unicamente la convergenza a un minimo locale: semi iniziali collocati casualmente o all'interno dello stesso addensamento naturale intrappolano il modello in configurazioni sub-ottimali;

2. **Necessità di specificare a priori il parametro** K: Il modello standard non possiede alcun meccanismo interno di selezione del modello. Poiché l'SSE decresce monotonicamente rispetto a K, non può guidare direttamente la scelta del numero naturale di cluster senza criteri di penalizzazione;

3. **Rigidità geometrica e partizionamento deterministico (*hard clustering*)**: Il K-Means costringe ogni istanza ad appartenere in modo esclusivo a un singolo cluster, definendo celle di Voronoi lineari. Tale

formulazione incorpora implicitamente l'ipotesi che i cluster siano sferici, convessi, di pari cardinalità e a varianza isotropa omogenea.

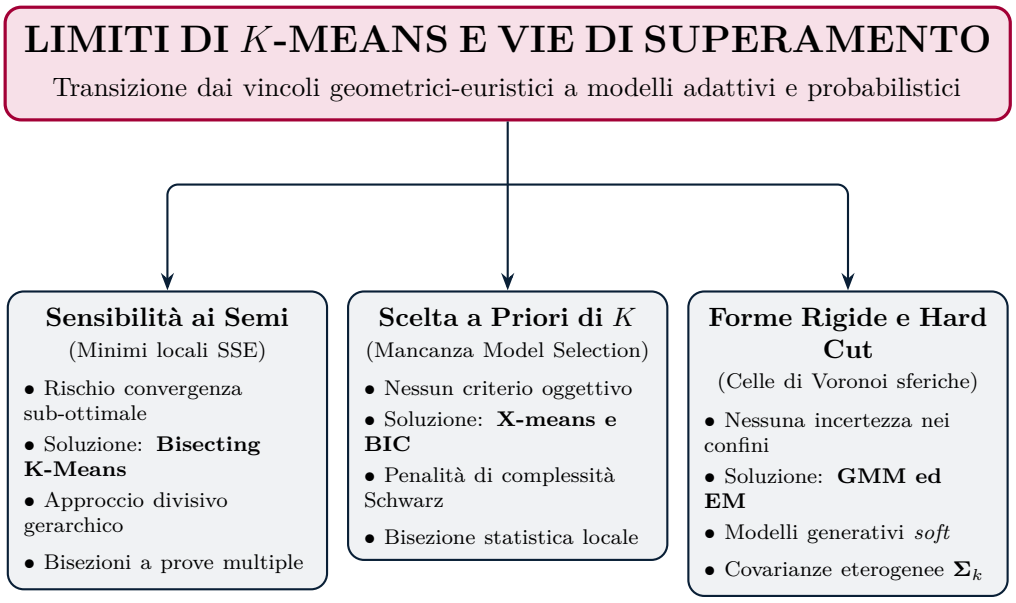


Figura 9.1: Panoramica concettuale dei limiti strutturali del K-Means standard e delle tre strategie metodologiche sviluppate per risolverli.

Per colmare tali lacune, la ricerca in data mining ha tracciato tre percorsi metodologici complementari illustrati in Figura 9.1:

- **Bisecting K-Means**: un paradigma gerarchico divisivo (*top-down*) che abbatte drasticamente la sensibilità all'inizializzazione riducendo l'ottimizzazione a una cascata di problemi a due vie ($K = 2$);
- **X-Means e Criterio BIC**: l'integrazione di un principio formale basato sulla teoria dell'informazione e sulla verosimiglianza regolarizzata per scoprire autonomamente la granularità ottimale dei dati;
- **Gaussian Mixture Models ed Expectation-Maximization**: l'abbandono delle partizioni geometriche deterministiche in favore della modellazione statistica generativa con assegnazioni probabilistiche continue (*responsabilità posteriori*).

9.2 L'Algoritmo Bisecting K-Means

L'algoritmo **Bisecting K-Means** (formalizzato come variante divisiva in [2]) affronta il problema dell'inizializzazione fragile sostituendo il partizionamento globale simultaneo in K gruppi con una procedura gerarchica divisiva (*top-down*) fondata sul principio del *divide et impera*. Invece di dover posizionare simultaneamente K semi nello spazio metrico ad alta dimensionalità, l'algoritmo decompone il problema in una sequenza deterministica di $K-1$ bisezioni binarie indipendenti ($K=2$).

9.2.1 Formalizzazione dell'Errore Quadratico e Strategia di Partizione

Sia dato un sottoinsieme di punti $\mathcal{C} \subseteq \mathcal{D}$ di cardinalità $|\mathcal{C}| = n_{\mathcal{C}}$. L'errore quadratico intra-cluster $\mathrm{SSE}(\mathcal{C})$ è calcolato rispetto al baricentroide $\mathbf{c} = \frac{1}{n_{\mathcal{C}}} \sum_{\mathbf{x} \in \mathcal{C}} \mathbf{x}$:

$$\mathrm{SSE}(\mathcal{C}) = \sum_{\mathbf{x} \in \mathcal{C}} \|\mathbf{x} - \mathbf{c}\|_2^2 \tag{9.1}$$

Se il partizionamento corrente dello spazio è rappresentato da una lista ordinata di cluster $\mathcal{L} = \{\mathcal{C}_1, \mathcal{C}_2, \ldots, \mathcal{C}_m\}$, l'errore quadratico totale della configurazione è dato dalla somma diretta dei singoli contributi:

$$\mathrm{SSE}_{\mathrm{total}}(\mathcal{L}) = \sum_{j=1}^{m} \mathrm{SSE}(\mathcal{C}_j) \tag{9.2}$$

Nel momento in cui un cluster $\mathcal{C}^* \in \mathcal{L}$ viene rimosso da $\mathcal{L}$ e bisezionato in due sotto-cluster disgiunti $\mathcal{A}$ e $\mathcal{B}$ (con $\mathcal{A} \cup \mathcal{B} = \mathcal{C}^*$ e $\mathcal{A} \cap \mathcal{B} = \emptyset$), il nuovo errore quadratico totale diventa:

$$\mathrm{SSE}_{\mathrm{total}}(\mathcal{L}') = \mathrm{SSE}_{\mathrm{total}}(\mathcal{L}) - \mathrm{SSE}(\mathcal{C}^*) + \Big(\mathrm{SSE}(\mathcal{A}) + \mathrm{SSE}(\mathcal{B})\Big) \tag{9.3}$$

Poiché per la proprietà geometrica di Huygens-Steiner la dispersione attorno a due baricentri indipendenti è strettamente non superiore alla dispersione attorno a un singolo baricentro comune, si ha sempre $\mathrm{SSE}(\mathcal{A}) + \mathrm{SSE}(\mathcal{B}) \leq \mathrm{SSE}(\mathcal{C}^*)$, garantendo una riduzione monotona della devianza residua ad ogni bisezione.

9.2.2 Pseudocodice e Analisi Procedurale

L'Algoritmo 5 descrive in dettaglio la procedura operativa di Bisecting K-Means.

Algorithm 5: Algoritmo Bisecting K-Means

Dati: Dataset $\mathcal{D} = \{\mathbf{x}_1, \ldots, \mathbf{x}_N\} \subset \mathbb{R}^d$, numero desiderato di cluster K, numero di prove di bisezione T

Risultato: Insieme finale di K cluster $\mathcal{L} = \{\mathcal{C}_1, \ldots, \mathcal{C}_K\}$

Inizializza la lista di cluster $\mathcal{L} \leftarrow \{\mathcal{D}\}$ contenente l'intero dataset come unico cluster
while $|\mathcal{L}| < K$ **do**
    Seleziona ed estrai un cluster $\mathcal{C}^*$ da $\mathcal{L}$ secondo una prefissata euristica di split (es. massimo SSE o massima cardinalità)
    $\mathcal{L} \leftarrow \mathcal{L} \setminus \{\mathcal{C}^*\}$
    best_sse $\leftarrow +\infty$
    $(\mathcal{A}^*, \mathcal{B}^*) \leftarrow (\emptyset, \emptyset)$
    for $i \leftarrow 1$ **to** T **do**
        Inizializza casualmente due centroidi all'interno di $\mathcal{C}^*$
        Esegui l'algoritmo K-Means standard con $K = 2$ su $\mathcal{C}^*$, ottenendo i sotto-cluster $(\mathcal{A}_i, \mathcal{B}_i)$
        current_sse $\leftarrow \text{SSE}(\mathcal{A}_i) + \text{SSE}(\mathcal{B}_i)$
        if *current_sse* $<$ *best_sse* **then**
            best_sse $\leftarrow$ current_sse
            $(\mathcal{A}^*, \mathcal{B}^*) \leftarrow (\mathcal{A}_i, \mathcal{B}_i)$
    Inserisci i due sotto-cluster ottimali nella lista: $\mathcal{L} \leftarrow \mathcal{L} \cup \{\mathcal{A}^*, \mathcal{B}^*\}$
return $\mathcal{L}$

9.2.3 Euristiche di Selezione del Cluster e Prove Multiple

L'algoritmo prevede due gradi di libertà algoritmici cruciali:

1. **Criterio di selezione del cluster candidato $\mathcal{C}^*$**:
   - *Massimo SSE individuale* ($\mathcal{C}^* = \arg\max_{\mathcal{C}\in\mathcal{L}} \mathrm{SSE}(\mathcal{C})$): Rappresenta la strategia standard raccomandata in letteratura. Bisezionando il gruppo con la massima dispersione interna, si massimizza la diminuzione locale del termine $\mathrm{SSE}(\mathcal{C}^*) - (\mathrm{SSE}(\mathcal{A}) + \mathrm{SSE}(\mathcal{B}))$, favorendo l'omogeneizzazione globale della varianza tra i cluster.
   - *Massima cardinalità* ($\mathcal{C}^* = \arg\max_{\mathcal{C}\in\mathcal{L}} |\mathcal{C}|$): Seleziona il gruppo più popoloso, utile qualora i dati contengano sbilanciamenti dimensionali macroscopici o cluster di ampiezza anomala.
2. **Ruolo delle prove multiple (T trials)**: Poiché il K-Means con $K = 2$ rimane un'euristica soggetta alla scelta casuale dei due semi iniziali all'interno di $\mathcal{C}^*$, eseguire T iterazioni indipendenti (solitamente $T \in [5, 20]$) e selezionare la partizione con il minor valore di $\mathrm{SSE}(\mathcal{A})+\mathrm{SSE}(\mathcal{B})$ abbatte esponenzialmente la probabilità di convergere verso un taglio sub-ottimale.

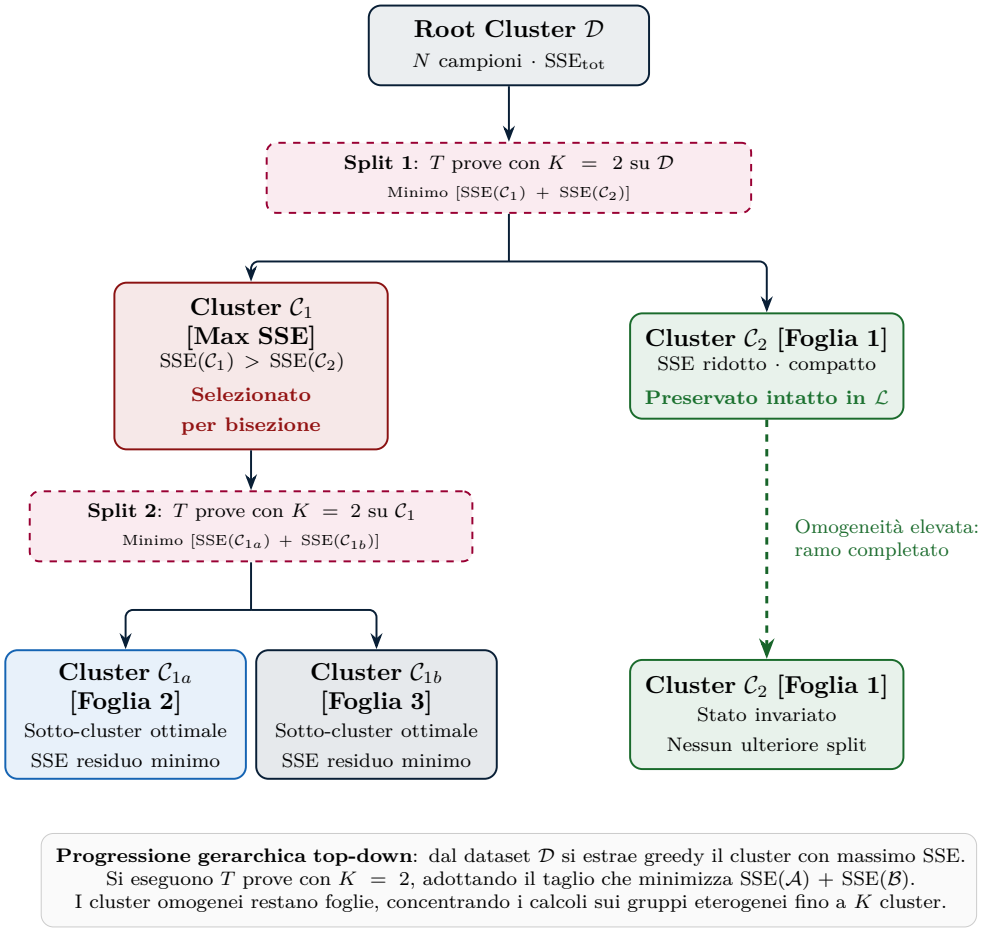


Figura 9.2: Rappresentazione ad albero binario gerarchico della progressione divisiva di Bisecting K-Means con prove multiple e selezione per minimo SSE.

9.2.4 Complessità Computazionale e Casi Limite

L'analisi di complessità computazionale evidenzia notevoli vantaggi operativi:

- **Costo per bisezione**: Eseguire 2-means su un cluster di n_k punti per I_2 iterazioni richiede tempo $\mathcal{O}(I_2 \cdot 2 \cdot n_k \cdot d)$. Poiché ad ogni livello dell'albero la somma delle cardinalità dei cluster è $\sum n_k = N$, se l'albero di divisione è bilanciato la profondità complessiva è $\mathcal{O}(\log K)$. Il costo totale ammonta a $\mathcal{O}(T \cdot I_2 \cdot N \cdot d \cdot \log K)$, risultando significativamente più rapido del K-Means globale standard qualora K sia elevato.

- **Casi degeneri e singleton**: Se un cluster candidato possiede cardinalità unitaria ($|\mathcal{C}| = 1$) oppure varianza nulla (tutti i punti coincidenti),

esso non può essere bisezionato. L'euristica deve intercettare tale eccezione saltando il cluster e selezionando il candidato successivo nella lista ordinata per SSE.

Raffinamento Globale di Lloyd (Global Lloyd Refinement)

Poiché le decisioni di partizionamento di Bisecting K-Means vengono assunte con un approccio greedy locale (i punti confinati nel cluster $\mathcal{C}^*$ vengono ripartiti senza poter interagire con i punti degli altri raggruppamenti già formati), la partizione finale può presentare confini di decisione leggermente disallineati rispetto all'ottimo globale dello spazio euclideo. È prassi consolidata utilizzare i K centroidi generati da Bisecting K-Means come **semi iniziali per una fase conclusiva di K-Means standard**. Questa raffinazione finale converge tipicamente in un numero ridottissimo di iterazioni (2–4 iterazioni), correggendo le assegnazioni di confine senza alcun rischio di cadere in minimi locali degeneri.

9.3 L'Algoritmo X-Means e la Scelta Automatica di K

L'algoritmo **X-Means**, introdotto da Dan Pelleg e Andrew Moore [5], risolve in modo rigoroso e interamente automatizzato il problema della scelta del parametro K. Invece di richiedere all'utente un valore fisso, X-Means richiede unicamente un intervallo di ricerca $[K_{\min}, K_{\max}]$, esplorando dinamicamente la complessità del modello attraverso una sequenza di test statistici locali guidati dal **Criterio d'Informazione Bayesiano** (*Bayesian Information Criterion*, BIC).

9.3.1 Flusso Operativo e Decisione di Bisezione Locale

L'algoritmo opera alternando ciclicamente una fase di clustering standard e una fase di verifica statistica della bisezione:

1. **Inizializzazione e Convergenza Globale**: Si inizializza l'algoritmo impostando $K = K_{\min}$ ed eseguendo K-Means convenzionale fino a convergenza.

2. **Perturbazione dei Centroidi lungo la Direzione Principale**: Per ciascun cluster $\mathcal{C}_k$ corrente (con centroide $\mathbf{c}_k$), si valuta l'ipotesi se $\mathcal{C}_k$ debba essere preservato come entità singola (ipotesi nulla $\mathcal{M}_1$) oppure

scisso in due sotto-cluster (ipotesi alternativa $\mathcal{M}_2$). A tal fine, vengono generati due semi candidati $\mathbf{c}_k^{(1)}$ e $\mathbf{c}_k^{(2)}$ perturbando il centroide lungo un vettore direzionale $\mathbf{v}$ proporzionale all'asse di massima varianza locale:

$$\mathbf{c}_k^{(1)} = \mathbf{c}_k - \alpha\mathbf{v}, \qquad \mathbf{c}_k^{(2)} = \mathbf{c}_k + \alpha\mathbf{v} \tag{9.4}$$

dove $\alpha = s \cdot \sqrt{\lambda_{\max}}$, con $\lambda_{\max}$ massimo autovalore della matrice di covarianza locale e s costante di scala.

3. **Bisezione Locale 2-means**: Si esegue un'istanza locale di 2-means confinata esclusivamente all'insieme di punti appartenenti a $\mathcal{C}_k$, ottenendo i due sotto-centroidi raffinati $\hat{\mathbf{c}}_k^{(1)}$ e $\hat{\mathbf{c}}_k^{(2)}$.

4. **Test di Scelta del Modello tramite BIC**: Si calcola il valore del criterio BIC per il modello a cluster singolo $\mathrm{BIC}(\mathcal{M}_1)$ e per il modello a cluster sdoppiato $\mathrm{BIC}(\mathcal{M}_2)$. Si calcola il guadagno informativo:

$$\Delta\mathrm{BIC} = \mathrm{BIC}(\mathcal{M}_2) - \mathrm{BIC}(\mathcal{M}_1) \tag{9.5}$$

   Se $\Delta\mathrm{BIC} > 0$, l'incremento di verosimiglianza giustifica l'introduzione di nuovi parametri: la scissione viene accettata e il centroide originale $\mathbf{c}_k$ viene rimpiazzato dalla coppia $(\hat{\mathbf{c}}_k^{(1)}, \hat{\mathbf{c}}_k^{(2)})$. Altrimenti, la bisezione viene rigettata.

5. **Verifica del Criterio d'Arresto**: Se nessun cluster viene scisso oppure se la cardinalità totale dei centroidi raggiunge il limite $K_{\max}$, l'algoritmo termina. In caso contrario, si riavvia il K-Means globale con il nuovo insieme espanso di centroidi e si ripete la procedura.

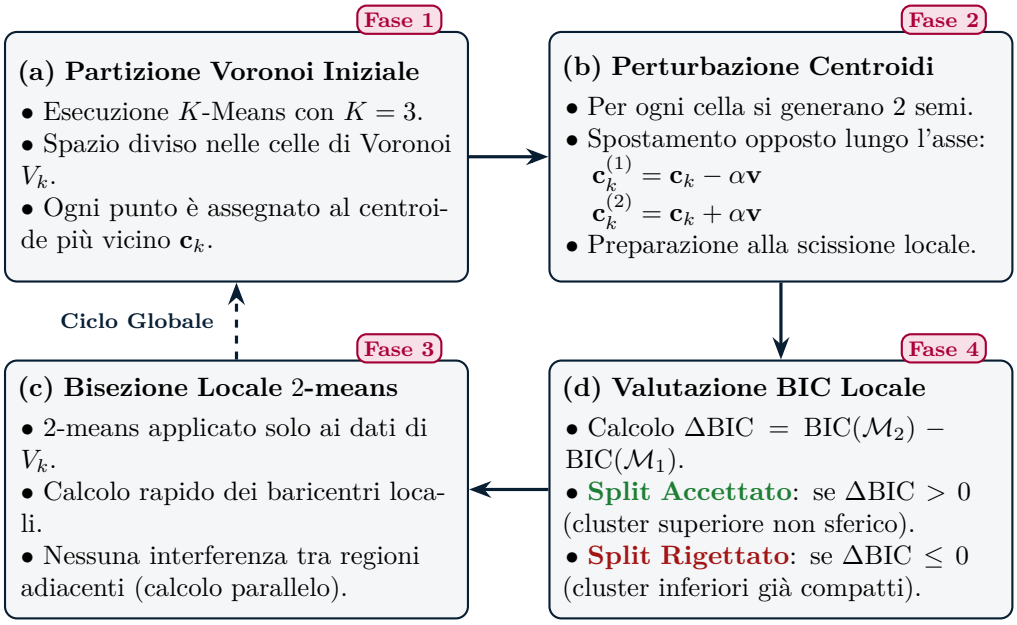


Figura 9.3: Ciclo operativo a quattro stadi di X-Means: partizione iniziale di Voronoi, perturbazione direzionale dei semi, bisezione locale e validazione statistica tramite il test informativo ΔBIC.

9.4 Il Criterio d'Informazione Bayesiano (BIC)

Il Criterio d'Informazione Bayesiano (BIC), formulato da Gideon Schwarz [6], affonda le proprie radici nella teoria asintotica della stima bayesiana del modello. Esso fornisce una misura quantitativa del compromesso fondamentale tra **bontà di adattamento ai dati** (*goodness of fit*) e **complessità parametrica del modello** (*parsimonia o rasoio di Occam*).

9.4.1 Derivazione Asintotica dall'Evidenza Marginale

Sia dato un insieme di dati osservati $\mathcal{D} = \{\mathbf{x}_1, \ldots, \mathbf{x}_R\}$ e una famiglia parametrica di modelli $\mathcal{M}$ indicizzata da un vettore di parametri $\boldsymbol{\theta} \in \mathbb{R}^p$. Nella formulazione bayesiana pura, la probabilità a posteriori del modello $\mathcal{M}$ dato il campione $\mathcal{D}$ è data dal teorema di Bayes:

$$p(\mathcal{M}|\mathcal{D}) = \frac{p(\mathcal{D}|\mathcal{M})\, p(\mathcal{M})}{p(\mathcal{D})} \propto p(\mathcal{D}|\mathcal{M})\, p(\mathcal{M}) \tag{9.6}$$

In assenza di preferenze a priori sui modelli ($p(\mathcal{M})$ uniforme), la scelta ottimale consiste nel massimizzare l'**evidenza marginale** (*marginal likelihood*):

$$p(\mathcal{D}|\mathcal{M}) = \int_{\Theta} p(\mathcal{D}|\boldsymbol{\theta}, \mathcal{M})\, p(\boldsymbol{\theta}|\mathcal{M})\, d\boldsymbol{\theta} \tag{9.7}$$

Poiché l'integrale (9.7) è analiticamente intrattabile nella quasi totalità dei casi non banali, Schwarz applicò il **metodo di Laplace** per approssimarlo asintoticamente quando la numerosità campionaria $R \to \infty$. Sviluppando in serie di Taylor la funzione di log-verosimiglianza $\ell(\boldsymbol{\theta}) = \ln p(\mathcal{D}|\boldsymbol{\theta}, \mathcal{M})$ attorno allo stimatore di massima verosimiglianza (*MLE*) $\hat{\boldsymbol{\theta}}$:

$$\ell(\boldsymbol{\theta}) \approx \ell(\hat{\boldsymbol{\theta}}) - \frac{1}{2}(\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})^T \mathbf{H} (\boldsymbol{\theta} - \hat{\boldsymbol{\theta}}) \tag{9.8}$$

dove $\mathbf{H} = -\nabla^2 \ell(\hat{\boldsymbol{\theta}})$ è la matrice Hessiana dell'informazione osservata. Sostituendo tale forma quadratica gaussiana nell'integrale e valutando il determinante asintotico per R grande, si ottiene:

$$\ln p(\mathcal{D}|\mathcal{M}) = \ell(\hat{\boldsymbol{\theta}}) - \frac{p}{2} \ln R + \mathcal{O}(1) \tag{9.9}$$

Definendo formalmente la statistica $\mathrm{BIC}(\mathcal{M})$ proporzionale al logaritmo dell'evidenza marginale, si giunge alla celebre formulazione:

$$\mathrm{BIC}(\mathcal{M}) = \ell(\hat{\boldsymbol{\theta}}) - \frac{p}{2} \ln R \tag{9.10}$$

dove:

- $\ell(\hat{\boldsymbol{\theta}}) = \sum_{i=1}^{R} \ln p(\mathbf{x}_i|\hat{\boldsymbol{\theta}})$ è la log-verosimiglianza massimizzata del modello sui dati;
- p è il numero totale di parametri liberi stimati dal modello;
- R è il numero di osservazioni empiriche;
- Il termine $-\frac{p}{2} \ln R$ rappresenta la **penalità di complessità di Schwarz**, che cresce logaritmicamente con la dimensione del campione e linearmente con il numero di parametri liberi.

Nota 9.4.1 (Convenzioni di Segno nella Letteratura Scientifica)**.** In letteratura statistica coesistono due definizioni formali del BIC:

1. **Convenzione della Verosimiglianza Regolarizzata** (usata in data mining e da Pelleg-Moore): $\mathrm{BIC} = \ell(\hat{\boldsymbol{\theta}}) - \frac{p}{2} \ln R$. In questo caso il criterio va **massimizzato** (punteggi più alti indicano modelli migliori).

2. **Convenzione della Devianza** (usata storicamente da Schwarz e in econometria): $\text{BIC}_{\text{dev}} = -2\ell(\hat{\boldsymbol{\theta}}) + p \ln R$. In questo caso il criterio va **minimizzato**. Le due formulazioni sono esattamente equivalenti poiché $\text{BIC}_{\text{dev}} = -2 \cdot \text{BIC}$.

9.4.2 Formalizzazione del BIC nel Contesto del Clustering Sferico

Nell'algoritmo X-Means [5], ciascun cluster $k \in \{1, \dots, K\}$ viene modellato come una distribuzione gaussiana d-dimensionale isotropa e sferica, con matrice di covarianza identica e diagonale $\boldsymbol{\Sigma}_k = \sigma^2 \mathbf{I}_d$. La densità di probabilità associata a un punto $\mathbf{x}$ generato dal cluster $\mathcal{C}_k$ è:

$$p(\mathbf{x}|\mathbf{c}_k, \sigma^2) = \frac{1}{(2\pi)^{d/2}\sigma^d} \exp\left(-\frac{\|\mathbf{x} - \mathbf{c}_k\|_2^2}{2\sigma^2}\right) \tag{9.11}$$

La log-verosimiglianza del sottoinsieme di R punti ripartiti nei K cluster è espressa analiticamente da:

$$\ell(\hat{\boldsymbol{\theta}}) = -\frac{R \cdot d}{2} \ln(2\pi) - \frac{R \cdot d}{2} \ln(\hat{\sigma}^2) - \frac{1}{2\hat{\sigma}^2} \sum_{k=1}^{K} \text{SSE}(\mathcal{C}_k) + \sum_{k=1}^{K} R_k \ln\left(\frac{R_k}{R}\right) \tag{9.12}$$

dove $R_k = |\mathcal{C}_k|$ è la cardinalità del cluster k, e lo stimatore non distorto della varianza comune residua $\hat{\sigma}^2$ è calcolato normalizzando per i gradi di libertà spaziali:

$$\hat{\sigma}^2 = \frac{1}{R - K} \frac{1}{d} \sum_{k=1}^{K} \text{SSE}(\mathcal{C}_k) \tag{9.13}$$

Sostituendo $\hat{\sigma}^2$ nell'equazione (9.12), il terzo termine si semplifica esattamente in $-\frac{(R-K)\cdot d}{2}$.

9.4.3 Quantificazione Rigorosa dei Parametri Liberi

Il conteggio dei parametri liberi p_K del modello con K cluster è strutturato come segue:

- $K \cdot d$ parametri per specificare le coordinate spaziali dei K vettori centroide $\mathbf{c}_k \in \mathbb{R}^d$;
- 1 parametro per la varianza isotropa condivisa $\hat{\sigma}^2$;
- $K - 1$ parametri liberi per i pesi a priori dei cluster $\pi_k = \frac{R_k}{R}$ (poiché $\sum_{k=1}^{K} \pi_k = 1$).

Il numero totale di gradi di libertà ammonta dunque a:

$$p_K = K \cdot d + 1 + (K - 1) = K(d + 1) \tag{9.14}$$

Qualora i pesi a priori siano considerati uniformi o fissati empiricamente dalle proporzioni campionarie, la letteratura computazionale adotta la formulazione ridotta $p_K = K \cdot d + 1$.

Struttura Covarianza	**Forma di $\boldsymbol{\Sigma}_k$**	**Parametri p**	**Proprietà Geometriche**
Sferica Condivisa	$\sigma^2 \mathbf{I}_d$	$K \cdot d + 1$	Sfere isotrope di raggio identico per tutti i cluster.
Sferica Distinta	$\sigma_k^2 \mathbf{I}_d$	$K(d + 1)$	Sfere isotrope con raggio e volume disomogenei tra cluster.
Diagonale Indipendente	$\mathrm{diag}(\sigma_{k1}^2, \ldots, \sigma_{kd}^2)$	$K \cdot 2d$	Iper-ellissoidi con assi allineati agli assi coordinati.
Piena Generale	$\boldsymbol{\Sigma}_k \succ 0$ libera	$K\left(d + \frac{d(d+1)}{2}\right)$	Ellissoidi ad orientamento, rotazione e scala arbitrari.

Tabella 9.1: Confronto del numero di parametri liberi p in funzione della modellazione della matrice di covarianza per K componenti gaussiane in dimensione d.

La Tabella 9.1 illustra chiaramente l'esplosione quadratica della complessità parametrica ($\mathcal{O}(K \cdot d^2)$) quando si passa da assunzioni sferiche a covarianze piene, evidenziando il motivo per cui X-Means adotta l'ipotesi sferica per garantire robustezza numerica e velocità di calcolo anche ad elevata dimensionalità.

9.5 Modelli di Mistura ed Expectation-Maximization (EM)

Mentre i metodi di partizionamento geometrico assegnano ciascun record a un cluster in modo binario, l'approccio generativo probabilistico fondato

sui **Modelli di Mistura** (*Mixture Models*) e sull'algoritmo **Expectation-Maximization** (EM) [7] modella il dataset come la realizzazione di una variabile casuale complessa generata da una combinazione convessa di distribuzioni di probabilità elementari.

9.5.1 Formulazione Generativa e Variabili Latenti

Si assume che ciascuna osservazione $\mathbf{x}_i \in \mathbb{R}^d$ $(i = 1, \dots, N)$ sia generata secondo il seguente processo stocastico:

1. Viene estratta una componente casuale latente $z_i \in \{1, \dots, K\}$ secondo una distribuzione categorica a priori:

$$p(z_i = k) = \pi_k, \qquad \text{con } \sum_{k=1}^{K} \pi_k = 1 \quad \text{e} \quad \pi_k \geq 0 \tag{9.15}$$

   I valori π_k sono denominati **pesi di miscelazione** o *mixing proportions*.

2. Condizionatamente alla scelta della componente k, l'osservazione $\mathbf{x}_i$ è campionata dalla corrispondente distribuzione continua $p_k(\mathbf{x}_i|\boldsymbol{\theta}_k)$:

$$\mathbf{x}_i \sim p(\mathbf{x}_i|z_i = k, \boldsymbol{\theta}_k) \tag{9.16}$$

La densità di probabilità marginale dell'osservazione $\mathbf{x}_i$ è data dalla legge della probabilità totale:

$$p(\mathbf{x}_i|\boldsymbol{\Theta}) = \sum_{k=1}^{K} p(z_i = k)\, p(\mathbf{x}_i|z_i = k, \boldsymbol{\theta}_k) = \sum_{k=1}^{K} \pi_k\, p_k(\mathbf{x}_i|\boldsymbol{\theta}_k) \tag{9.17}$$

dove l'insieme globale dei parametri da stimare è $\boldsymbol{\Theta} = \{\pi_1, \dots, \pi_K, \boldsymbol{\theta}_1, \dots, \boldsymbol{\theta}_K\}$.

9.5.2 Verosimiglianza Incompleta contro Verosimiglianza Completa

Dato il dataset $\mathbf{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$, assumendo le osservazioni i.i.d., la funzione di **log-verosimiglianza incompleta** (osservata) è data da:

$$\ln p(\mathbf{X}|\boldsymbol{\Theta}) = \sum_{i=1}^{N} \ln \left(\sum_{k=1}^{K} \pi_k\, p_k(\mathbf{x}_i|\boldsymbol{\theta}_k) \right) \tag{9.18}$$

La presenza del logaritmo di una sommatoria all'interno dell'espressione (9.18) impedisce di ricavare una soluzione analitica in forma chiusa per i gradienti

$\nabla_{\boldsymbol{\Theta}} \ln p(\mathbf{X}|\boldsymbol{\Theta}) = \mathbf{0}$, poiché i parametri delle diverse componenti risultano inestricabilmente accoppiati.

Se, per ipotesi teorica, fossero note anche le variabili indicatrici latenti non osservate $\mathbf{Z} = \{\mathbf{z}_1, \ldots, \mathbf{z}_N\}$, introducendo la codifica *one-hot* $z_{ik} = \mathbb{I}(z_i = k)$, la **log-verosimiglianza completa** dei dati congiunti $(\mathbf{X}, \mathbf{Z})$ assumerebbe la forma linearizzata:

$$\ln p(\mathbf{X}, \mathbf{Z}|\boldsymbol{\Theta}) = \sum_{i=1}^{N} \sum_{k=1}^{K} z_{ik} \left[\ln \pi_k + \ln p_k(\mathbf{x}_i|\boldsymbol{\theta}_k)\right] \tag{9.19}$$

Nell'equazione (9.19) il logaritmo agisce direttamente sulle densità individuali delle componenti, rendendo immediata l'ottimizzazione separata per ciascun parametro.

9.5.3 Fondazione Variazionale dell'Algoritmo EM

L'algoritmo Expectation-Maximization risolve l'ottimizzazione della verosimiglianza incompleta applicando la disuguaglianza di Jensen per definire una funzione limite inferiore (*evidence lower bound*) calcolata rispetto alla distribuzione a posteriori delle variabili latenti.

Sia $\boldsymbol{\Theta}^{(t)}$ la stima corrente dei parametri all'iterazione t. Si definisce la funzione ausiliaria $\mathcal{Q}(\boldsymbol{\Theta}, \boldsymbol{\Theta}^{(t)})$ come il valore atteso della log-verosimiglianza completa condizionata ai dati osservati e ai parametri correnti:

$$\begin{aligned}
\mathcal{Q}(\boldsymbol{\Theta}, \boldsymbol{\Theta}^{(t)}) &= \mathbb{E}_{\mathbf{Z}|\mathbf{X}, \boldsymbol{\Theta}^{(t)}} \left[\ln p(\mathbf{X}, \mathbf{Z}|\boldsymbol{\Theta})\right] \\
&= \sum_{i=1}^{N} \sum_{k=1}^{K} \mathbb{E}[z_{ik}|\mathbf{x}_i, \boldsymbol{\Theta}^{(t)}] \left[\ln \pi_k + \ln p_k(\mathbf{x}_i|\boldsymbol{\theta}_k)\right] \\
&= \sum_{i=1}^{N} \sum_{k=1}^{K} \gamma_{ik} \left[\ln \pi_k + \ln p_k(\mathbf{x}_i|\boldsymbol{\theta}_k)\right]
\end{aligned} \tag{9.20}$$

dove $\gamma_{ik} \equiv \mathbb{E}[z_{ik}|\mathbf{x}_i, \boldsymbol{\Theta}^{(t)}] = p(z_i = k|\mathbf{x}_i, \boldsymbol{\Theta}^{(t)})$ rappresenta la **responsabilità a posteriori** che la componente k attribuisce alla generazione dell'osservazione $\mathbf{x}_i$.

Teorema 9.5.1 (Monotonia della Verosimiglianza in EM)**.** *Per ogni successione di parametri $\boldsymbol{\Theta}^{(t)}$ e $\boldsymbol{\Theta}^{(t+1)}$ generata dall'algoritmo EM, vale la disuguaglianza di convergenza monotona:*

$$\ln p(\mathbf{X}|\boldsymbol{\Theta}^{(t+1)}) \geq \ln p(\mathbf{X}|\boldsymbol{\Theta}^{(t)}) \tag{9.21}$$

L'algoritmo converge monotonicamente verso un punto stazionario (minimo locale o punto di sella) della superficie di verosimiglianza incompleta.

9.5.4 Ciclo Operativo: E-step ed M-step

L'algoritmo itera ciclicamente due passaggi fino al raggiungimento della convergenza numerica:

1. Passo di Aspettazione (E-step, Expectation) Mantenendo costanti i parametri $\boldsymbol{\Theta}^{(t)}$, si applica la regola di Bayes per calcolare le responsabilità a posteriori per ogni punto i e per ogni componente k:

$$\gamma_{ik} = \frac{\pi_k^{(t)} \, p_k(\mathbf{x}_i|\boldsymbol{\theta}_k^{(t)})}{\sum_{j=1}^{K} \pi_j^{(t)} \, p_j(\mathbf{x}_i|\boldsymbol{\theta}_j^{(t)})} \tag{9.22}$$

Si noti che $\sum_{k=1}^{K} \gamma_{ik} = 1$ per ogni singolo punto i. Si definisce la **massa efficace** di punti assegnati al cluster k come:

$$N_k = \sum_{i=1}^{N} \gamma_{ik} \tag{9.23}$$

2. Passo di Massimizzazione (M-step, Maximization) Mantenendo costanti le responsabilità γ_{ik}, si ricavano i nuovi parametri $\boldsymbol{\Theta}^{(t+1)}$ massimizzando la funzione ausiliaria $\mathcal{Q}$:

$$\boldsymbol{\Theta}^{(t+1)} = \arg\max_{\boldsymbol{\Theta}} \mathcal{Q}(\boldsymbol{\Theta}, \boldsymbol{\Theta}^{(t)}) \tag{9.24}$$

Per i modelli di miscela gaussiana (GMM), le soluzioni in forma chiusa per i pesi, i vettori medi e le matrici di covarianza risultano:

$$\pi_k^{(t+1)} = \frac{N_k}{N} = \frac{1}{N} \sum_{i=1}^{N} \gamma_{ik} \tag{9.25}$$

$$\boldsymbol{\mu}_k^{(t+1)} = \frac{1}{N_k} \sum_{i=1}^{N} \gamma_{ik} \mathbf{x}_i \tag{9.26}$$

$$\boldsymbol{\Sigma}_k^{(t+1)} = \frac{1}{N_k} \sum_{i=1}^{N} \gamma_{ik} (\mathbf{x}_i - \boldsymbol{\mu}_k^{(t+1)})(\mathbf{x}_i - \boldsymbol{\mu}_k^{(t+1)})^T \tag{9.27}$$

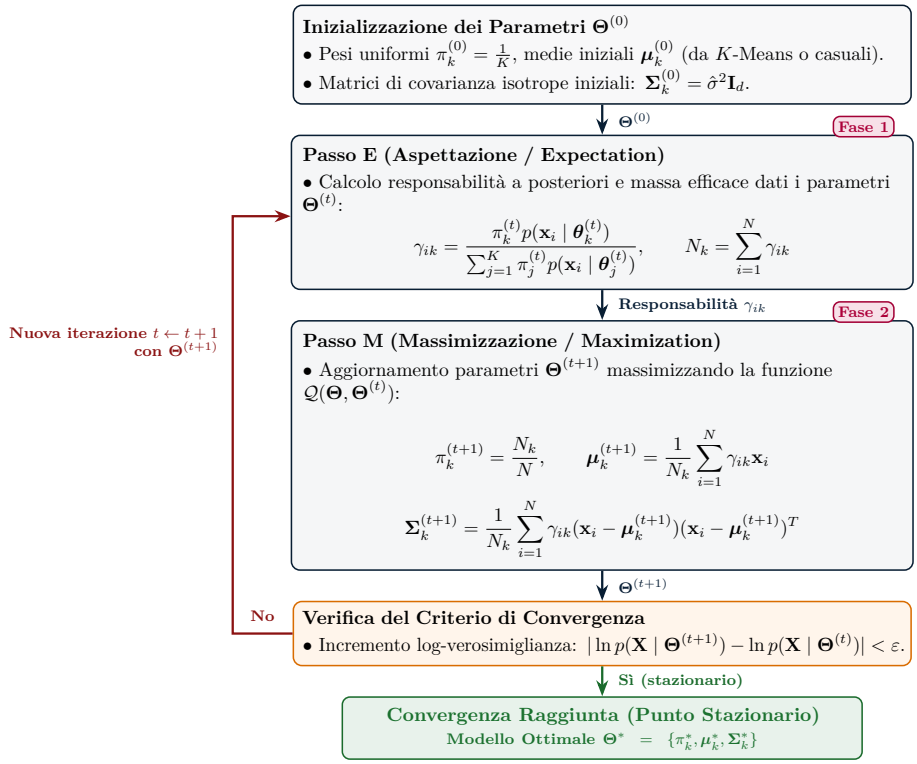


Figura 9.4: Diagramma operativo dell'algoritmo Expectation-Maximization (EM).

9.6 Distribuzioni Normali e Fondamenti di Stima Statistica

Per apprezzare appieno il funzionamento del passo di massimizzazione nei modelli GMM, è utile richiamare i fondamenti della stima a massima verosimiglianza (*Maximum Likelihood Estimation*, MLE) per una singola variabile casuale gaussiana unidimensionale.

La funzione di densità di probabilità di una variabile gaussiana $x \sim \mathcal{N}(\mu, \sigma^2)$ è espressa da:

$$p(x|\mu, \sigma^2) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) \tag{9.28}$$

Dato un campione i.i.d. di N osservazioni scalari $\{x_1, \ldots, x_N\}$, la funzione di log-verosimiglianza vale:

$$\ell(\mu, \sigma^2) = \sum_{i=1}^{N} \ln p(x_i|\mu, \sigma^2) = -\frac{N}{2}\ln(2\pi) - \frac{N}{2}\ln(\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^{N}(x_i-\mu)^2 \tag{9.29}$$

Annullando la derivata parziale rispetto al parametro di posizione μ:

$$\frac{\partial \ell}{\partial \mu} = \frac{1}{\sigma^2} \sum_{i=1}^{N} (x_i - \mu) = 0 \implies \hat{\mu}_{\text{MLE}} = \frac{1}{N} \sum_{i=1}^{N} x_i \tag{9.30}$$

si ottiene la media campionaria aritmetica. Sostituendo $\hat{\mu}$ e annullando la derivata parziale rispetto a σ^2:

$$\frac{\partial \ell}{\partial (\sigma^2)} = -\frac{N}{2\sigma^2} + \frac{1}{2(\sigma^2)^2} \sum_{i=1}^{N} (x_i - \hat{\mu})^2 = 0 \implies \hat{\sigma}^2_{\text{MLE}} = \frac{1}{N} \sum_{i=1}^{N} (x_i - \hat{\mu})^2 \tag{9.31}$$

Nel contesto dei modelli di miscela probabilistica (GMM), il passo M estende esattamente queste derivazioni classiche sostituendo la media aritmetica semplice con una **media ponderata** governata dalle probabilità a posteriori γ_{ik} (Equazioni (9.26) e (9.27)).

Fondamento Bayesiano MAP contro MLE Mentre la stima MLE massimizza puramente la verosimiglianza $\hat{\boldsymbol{\Theta}}_{\text{MLE}} = \arg\max \ln p(\mathbf{X}|\boldsymbol{\Theta})$, la stima di **Massima a Posteriori (MAP)** incorpora distribuzioni a priori coniugate $p(\boldsymbol{\Theta})$ sui parametri:

$$\hat{\boldsymbol{\Theta}}_{\text{MAP}} = \arg\max_{\boldsymbol{\Theta}} \Big[\ln p(\mathbf{X}|\boldsymbol{\Theta}) + \ln p(\boldsymbol{\Theta}) \Big] \tag{9.32}$$

Nei modelli di miscela ad alta dimensionalità, la stima MAP con distribuzioni a priori Normal-Inverse-Wishart sulle matrici di covarianza impedisce la singolarità del determinante ($|\boldsymbol{\Sigma}_k| \to 0$), fungendo da termine di regolarizzazione strutturale (*ridge shrinkage*).

9.7 Caso di Studio: Deconvoluzione di una Miscela di Due Gaussiane

Per comprendere concretamente l'azione del clustering probabilistico, consideriamo il problema didattico fondamentale presentato in [2], relativo alla modellazione dell'altezza in una popolazione mista eterogenea (ad esempio, la sovrapposizione di individui appartenenti a due etnie con statura media differente).

9.7.1 Definizione del Modello Sintetico Bimodale

Si consideri una miscela unidimensionale bilanciata ($K = 2$) composta da due distribuzioni normali identicamente disperse ma con baricentri traslati:

- **Componente 1** ($\mathcal{C}_1$): Media $\mu_1 = -4$, deviazione standard $\sigma_1 = 2$, peso $\pi_1 = 0.5$;
- **Componente 2** ($\mathcal{C}_2$): Media $\mu_2 = +4$, deviazione standard $\sigma_2 = 2$, peso $\pi_2 = 0.5$.

La funzione di densità marginale totale della popolazione risultante è data dalla combinazione convessa:

$$\begin{aligned} p(x) &= 0.5 \cdot \mathcal{N}(x \mid -4, 4) + 0.5 \cdot \mathcal{N}(x \mid +4, 4) \\ &= \frac{1}{2\sqrt{8\pi}} \left[\exp\left(-\frac{(x+4)^2}{8} \right) + \exp\left(-\frac{(x-4)^2}{8} \right) \right] \end{aligned} \tag{9.33}$$

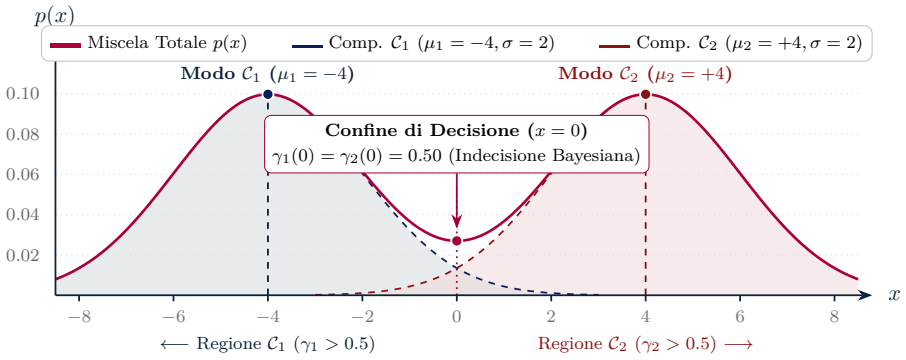


Figura 9.5: Deconvoluzione analitica di una miscela bimodale di due gaussiane con varianza identica ($\sigma = 2$) e baricentri separati ($\Delta\mu = 8$).

9.7.2 Analisi della Sovrapposizione e Responsabilità a Posteriori

La Figura 9.5 illustra le proprietà salienti della distribuzione di miscela:

1. **Presenza di due massimi locali marcati**: I picchi si collocano in stretta prossimità di $x = -4$ e $x = +4$.
2. **Punto di sella e incertezza nel punto** $x = 0$: Nel punto mediano $x = 0$, le densità condizionate delle due gaussiane coincidono perfettamente:

$$p_1(0) = \frac{1}{\sqrt{8\pi}} \exp\left(-\frac{16}{8} \right) = \frac{e^{-2}}{\sqrt{8\pi}}, \qquad p_2(0) = \frac{e^{-2}}{\sqrt{8\pi}} \tag{9.34}$$

Di conseguenza, applicando l'Equazione (9.22), le responsabilità a posteriori assumono il valore identico:

$$\gamma_1(0) = \frac{0.5 \cdot p_1(0)}{0.5 \cdot p_1(0) + 0.5 \cdot p_2(0)} = 0.5, \qquad \gamma_2(0) = 0.5 \tag{9.35}$$

In questo punto dello spazio, l'assegnazione rigida operata da un classificatore geometrico (che imporrebbe una partizione netta del tipo $\mathcal{C}_1$ se $x < 0$ e $\mathcal{C}_2$ se $x \geq 0$) distorcerebbe la natura probabilistica del fenomeno. L'algoritmo EM quantifica esattamente l'indecisione attraverso il valore continuo $\gamma_{ik} = 0.5$.

3. **Confronto con 20 000 campioni empirici**: Quando si generano $N = 20\,000$ campioni sintetici secondo la miscela (9.33), l'istogramma empirico delle frequenze si conforma alla curva teorica continua entro le deviazioni stocastiche standard previste dal teorema del limite centrale. L'algoritmo EM, inizializzato a partire da stime casuali, recupera i veri parametri generativi $(\mu_1, \mu_2, \sigma_1, \sigma_2, \pi_1, \pi_2)$ con errore trascurabile $(< 10^{-3})$.

9.8 Confronto Architetturale e Sistematico: K-Means vs. EM

L'algoritmo di Lloyd (K-Means) e l'algoritmo EM per modelli a miscela di gaussiane (GMM) condividono un'architettura iterativa formalmente isomorfa: entrambi partono da una stima iniziale dei parametri del modello e alternano una fase di determinazione dello stato dei punti a una fase di ricalcolo dei parametri dei cluster. Tuttavia, essi divergono radicalmente nelle loro assunzioni geometriche, energetiche e statistiche.

Dimensione	K-Means Standard (Lloyd)	Gaussian Mixture (EM)
Paradigma	**Hard clustering**: deterministico e disgiunto ($r_{ik} \in \{0, 1\}$).	**Soft clustering**: probabilistico e continuo ($\gamma_{ik} \in [0, 1]$).
Rappresentazione	Baricentroide geometrico $\mathbf{c}_k \in \mathbb{R}^d$.	Parametri $(\pi_k, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$.
Geometria Cluster	Iper-sferici, isotropi, a varianza omogenea.	Iper-ellissoidi con forma, volume e orientamento arbitrari.
Funzione Obiettivo	Minimizzazione errore quadratico intra-cluster (SSE).	Massimizzazione log-verosimiglianza incompleta ($\ln p(\mathbf{X}\|\boldsymbol{\Theta})$).
Fase 1 (Stato Punti)	Assegnazione Voronoi al centroide più vicino (L_2).	E-step: responsabilità a posteriori γ_{ik} via Bayes.
Fase 2 (Ricalcolo)	Media aritmetica semplice dei punti della cella.	M-step: media e covarianza ponderate con pesi γ_{ik}.
Costo Computazionale	$\mathcal{O}(N \cdot K \cdot d)$	$\mathcal{O}(N \cdot K \cdot d^2 + K \cdot d^3)$ (inversione di $\boldsymbol{\Sigma}_k$).
Punti di Debolezza	Sensibile ai semi, forme sferiche, nessun output probabilistico.	Singolarità per matrici degenerate, costo $\mathcal{O}(d^3)$, minimi locali.

Tabella 9.2: Quadro comparativo omnicomprensivo tra K-Means ed Expectation-Maximization (EM).

9.8.1 Vulnerabilità delle Singolarità della Covarianza in EM

Una vulnerabilità teorica fondamentale dell'algoritmo EM con covarianze piene $\boldsymbol{\Sigma}_k$ risiede nella presenza di **singolarità nella superficie di verosimiglianza**. Se durante le iterazioni una componente gaussiana concentra la propria responsabilità attorno a un singolo punto isolato $\mathbf{x}_m$, la covarianza campionaria collassa a zero:

$$\boldsymbol{\Sigma}_k \to \mathbf{0} \implies |\boldsymbol{\Sigma}_k| \to 0 \tag{9.36}$$

Poiché la densità gaussiana contiene al denominatore il fattore $|\mathbf{\Sigma}_k|^{1/2}$, si verifica che:

$$p_k(\mathbf{x}_m|\boldsymbol{\mu}_k, \mathbf{\Sigma}_k) \to +\infty \implies \ln p(\mathbf{X}|\mathbf{\Theta}) \to +\infty \tag{9.37}$$

La funzione di verosimiglianza non è superiormente limitata, generando picchi infiniti non corrispondenti a cluster significativi ma a singoli record isolati (*overfitting patologico*). Per neutralizzare questo fenomeno si applicano convenzionalmente due rimedi:

- **Jitter Diagonale**: Aggiunta di una piccola costante positiva $\varepsilon\mathbf{I}_d$ alla matrice di covarianza ad ogni iterazione ($\mathbf{\Sigma}'_k = \mathbf{\Sigma}_k + \epsilon\mathbf{I}_d$).
- **Regolarizzazione MAP**: Introduzione di una distribuzione a priori coniugata Inverse-Wishart che impone una penalizzazione asintotica quando il determinante della covarianza tende a zero.

9.8.2 Sintesi del Percorso Metodologico

Il percorso affrontato nel presente capitolo delinea l'evoluzione metodologica del clustering moderno:

- L'approccio gerarchico divisivo (**Bisecting K-Means**) trasforma un'euristica instabile e globale in una cascata robusta di decisioni binarie a bassa complessità.
- L'integrazione con la teoria statistica dell'informazione (**X-Means** e **BIC**) sostituisce la congettura soggettiva del parametro K con un'ottimizzazione rigorosa della complessità del modello basata sul principio di parsimonia.
- Il paradigma probabilistico (**GMM** ed **EM**) eleva il clustering da semplice partizione spaziale deterministica a inferenza statistica generativa, fornendo una stima rigorosa dell'incertezza e abilitando la cattura di geometrie orientate nello spazio continuo.

Capitolo 10

Validazione del Clustering: Misure, Teoria e Metodologie

10.1 Fondamenti e Sfide della Validazione del Clustering

L'analisi dei cluster (*cluster analysis*) costituisce uno dei compiti cardinali dell'apprendimento non supervisionato (*unsupervised machine learning*). L'obiettivo risiede nella partizione di una collezione di oggetti o vettori di attributi $\mathcal{D} = \{\mathbf{x}_1, \ldots, \mathbf{x}_N\} \subset \mathbb{R}^d$ in gruppi disgiunti o parzialmente sovrapposti (detti *cluster*), in modo tale che le istanze associate al medesimo gruppo manifestino un'elevata somiglianza reciproca (**coesione intra-cluster**), mentre elementi appartenenti a gruppi differenti risultino chiaramente distinti (**separazione inter-cluster**) [2].

10.1.1 Divergenza tra Apprendimento Supervisionato e Non Supervisionato

Nell'apprendimento supervisionato (come la classificazione o la regressione), la presenza di un'etichetta target fornita a priori (*ground truth*) rende la valutazione del modello oggettiva e diretta: la bontà predittiva viene misurata confrontando le previsioni con la verità empirica attraverso metriche ben consolidate quali *Accuracy*, *Precision*, *Recall*, *F-measure* o l'area sotto la curva ROC (*AUC*).

Al contrario, nel paradigma non supervisionato non è disponibile alcuna etichetta di classe esterna:

> *"Clustering is in the eye of the beholder."*

Non esiste una nozione univoca o universale di cosa costituisca un "raggruppamento naturale". La struttura geometrica identificata dipende inscindibilmente dalla nozione di similarità adottata e dalla classe di modelli algoritmici impiegata:

- Per l'algoritmo K-**Means**, un cluster è concepito come un raggruppamento compatto, iper-sferico e convesso focalizzato attorno a un baricentroide medio;
- Per **DBSCAN**, un cluster è una componente connessa ad alta densità separata da regioni a bassa densità o rumore, con estensione morfologica arbitraria;
- Per il **Single Linkage Hierarchical Clustering**, un cluster corrisponde a una catena contigua di punti mutualmente vicini, suscettibile al fenomeno del concatenamento (*chaining effect*).

10.1.2 Il Problema Critico: Clustering su Dati Puramente Casuali

La motivazione più urgente per introdurre una teoria rigorosa della validazione risiede nel fatto che **tutti gli algoritmi di clustering producono partizioni anche quando applicati a rumore puro**.

Si consideri un esperimento concettuale condotto su un dominio bidimensionale continuo $[0, 1] \times [0, 1]$, in cui vengono campionati N punti in modo del tutto indipendente da una distribuzione uniforme bivariata:

$$\mathbf{x}_i \sim \mathcal{U}\left([0,1]^2\right), \quad \forall i \in \{1, \ldots, N\} \tag{10.1}$$

Per definizione stocastica, questo insieme di dati non racchiude alcuna struttura a cluster, né attrattori o centri di densità. Tuttavia:

1. Se si esegue l'algoritmo K-Means imponendo ad esempio $K = 3$, la procedura suddivide il dominio stocastico in tre celle di Voronoi compatte, posizionando i centroidi in modo tale da minimizzare la devianza intra-cluster locale e restituendo una soluzione apparentemente plausibile;

2. Se si esegue un algoritmo gerarchico agglomerativo basato su *Complete Link*, esso produce un dendrogramma completo che, se tagliato a un livello arbitrario, restituisce sotto-insiemi con diametri ben definiti;

3. Se si applica DBSCAN impostando parametri $(\varepsilon, \text{MinPts})$ non tarati, esso può aggregare fluttuazioni locali della densità di Poisson come se fossero micro-strutture dense.

Assioma Fondamentale della Validità del Clustering

Qualsiasi algoritmo di clustering è un ottimizzatore geometrico progettato per aggregare dati. Esso troverà invariabilmente dei cluster, anche nel vuoto informativo. Di conseguenza, nessun risultato di clustering può essere accettato senza aver prima verificato:

1. Se il dataset originale contenga un'effettiva **tendenza al raggruppamento** rispetto a un'ipotesi nulla di rumore uniforme;
2. Se la partizione individuata sia **robusta, statisticamente significativa** e non derivi da fluttuazioni casuali o da bias algoritmici.

10.1.3 Finalità Metodologiche della Cluster Validity

Nel ciclo di vita dell'estrazione di conoscenza (KDD), la validazione dei cluster persegue quattro obiettivi operativi distinti:

1. **Filtraggio del rumore e verifica di tendenza**: Stabilire preliminarmente se il dataset manifesti una predisposizione non casuale all'aggregazione, scartando insiemi privi di segnale geometrico;

2. **Confronto tra algoritmi eterogenei**: Valutare quale paradigma (partizionale, gerarchico, density-based o probabilistico) modelli più fedelmente la geometria intrinseca delle osservazioni;

3. **Selezione del modello e stima degli iperparametri**: Determinare la parametrizzazione ottima, identificando il numero intrinseco di cluster K^* in K-Means o il raggio di vicinato ε in DBSCAN;

4. **Valutazione locale dei singoli cluster**: Esaminare la qualità intrinseca di ciascun gruppo all'interno di una medesima partizione, distinguendo nuclei ad alta coesione e separazione da cluster marginali, spuri o contaminati da rumore.

10.2 Tassonomia delle Misure di Validazione

Seguendo l'impostazione formale di Jain e Dubes [8] e Tan et al. [2], le metriche e le metodologie di validazione si ripartiscono in quattro macro-categorie ben distinte in base alla disponibilità di informazioni ausiliarie e alla finalità analitica.

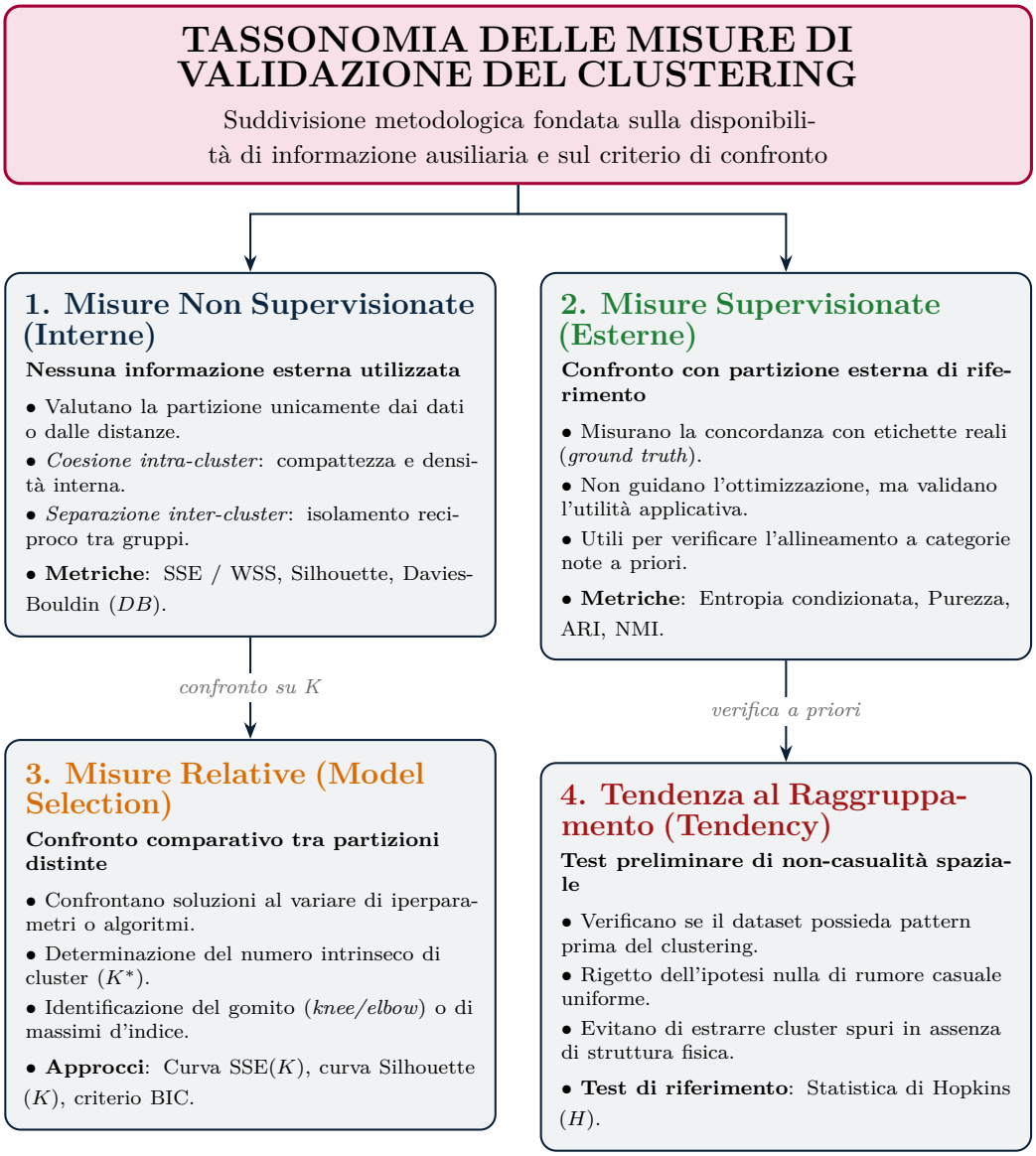


Figura 10.1: Tassonomia delle misure di validazione del clustering suddivise per natura dei dati e finalità d'indagine.

Come riassunto formalmente nella Figura 10.1:

1. **Misure Interne**: operano nello spazio non supervisionato. Verificano che i cluster siano intrinsecamente densi e ben separati. La metrica tipica include l'errore quadratico intra-cluster (SSE), il coefficiente di silhouette e la coerenza tra matrice di dissimilarità e partizione;

2. **Misure Esterne**: operano confrontando l'output dell'algoritmo con etichette esterne fornite da esperti di dominio (ad esempio classi diagnostiche mediche o categorie tematiche di documenti);

3. **Misure Relative**: costituiscono un protocollo di confronto seriale per determinare la corretta iper-parametrizzazione (come il numero ideale di cluster K^*);

4. **Tendenza al Raggruppamento**: costituisce il test preliminare di consistenza geometrica applicato prima di intraprendere qualsiasi procedura di partizione.

10.3 Tendenza al Raggruppamento e Statistica di Hopkins

Prima di spendere risorse computazionali nell'ottimizzazione di algoritmi di clustering, è doveroso verificare se i dati possiedano una struttura non uniforme. Questa problematica prende il nome di **Clustering Tendency** [2]. La tecnica quantitativa più celebre e rigorosa è la **Statistica di Hopkins** [9, 8].

10.3.1 Formalizzazione Matematica della Statistica di Hopkins

Sia dato un insieme di dati multivariato $\mathcal{D} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\} \subset \mathbb{R}^d$. La procedura di Hopkins valuta la discrepanza tra la distribuzione empirica delle distanze tra punti reali e la distribuzione attesa sotto un processo stocastico uniforme.

L'algoritmo si articola nelle seguenti fasi:

1. Si estrae casualmente e senza reimmissione un sottoinsieme di p punti reali da $\mathcal{D}$, denotato come:

$$\mathcal{X} = \{\mathbf{x}_1^*, \mathbf{x}_2^*, \dots, \mathbf{x}_p^*\} \subset \mathcal{D}, \quad \text{con } p \ll N \tag{10.2}$$

   Tipicamente si sceglie $p \approx 0.05 \cdot N$ o $p \approx 0.10 \cdot N$ per garantire l'indipendenza statistica dei campioni estratti.

2. Per ogni punto estratto $\mathbf{x}_i^* \in \mathcal{X}$, si calcola la distanza euclidea minima rispetto al suo punto più vicino appartenente al rimanente insieme dei dati $\mathcal{D} \setminus \{\mathbf{x}_i^*\}$:

$$w_i = \min_{\mathbf{x} \in \mathcal{D} \setminus \{\mathbf{x}_i^*\}} \|\mathbf{x}_i^* - \mathbf{x}\|_2 \tag{10.3}$$

3. Si costruisce una regione iper-rettangolare di campionamento $\Omega \subset \mathbb{R}^d$ che racchiuda l'iper-volume occupato dal dataset:

$$\Omega = \prod_{j=1}^{d} \left[\min_{k=1}^{N} x_{kj},\ \max_{k=1}^{N} x_{kj} \right] \tag{10.4}$$

   All'interno di tale bounding box Ω, si generano p punti sintetici indipendenti distribuiti uniformemente:

$$\mathcal{Y} = \{\mathbf{y}_1, \mathbf{y}_2, \ldots, \mathbf{y}_p\} \overset{\text{i.i.d.}}{\sim} \mathcal{U}(\Omega) \tag{10.5}$$

4. Per ciascun punto sintetico $\mathbf{y}_i \in \mathcal{Y}$, si calcola la distanza minima rispetto al punto più vicino nell'intero dataset reale $\mathcal{D}$:

$$u_i = \min_{\mathbf{x} \in \mathcal{D}} \|\mathbf{y}_i - \mathbf{x}\|_2 \tag{10.6}$$

5. La **statistica di Hopkins** H è definita come il rapporto normalizzato tra la somma delle distanze dei punti sintetici e la somma aggregata di tutte le distanze calcolate:

$$H = \frac{\sum_{i=1}^{p} u_i}{\sum_{i=1}^{p} u_i + \sum_{i=1}^{p} w_i} \tag{10.7}$$

   In alternativa, talune formulazioni storiche adottano la somma delle distanze elevate alla potenza d-esima (coerente con la misura di volume nello spazio $\mathbb{R}^d$), ma la forma lineare (10.7) è lo standard ampiamente consolidato nella pratica del data mining.

10.3.2 Interpretazione Probabilistica dei Valori di H

Il valore di H è intrinsecamente vincolato nell'intervallo continuo $[0, 1]$. L'interpretazione dei regimi limite è illustrata nella Tabella 10.1.

Valore di H	**Comportamento di u_i vs w_i**	**Diagnostica Spaziale e Tendenza al Clustering**
$H \approx 0.5$	$\sum u_i \approx \sum w_i$	**Dati Uniformemente Distribuiti (Assenza di Cluster)**: le distanze dei punti casuali $\mathbf{y}_i$ dai dati reali sono statisticamente identiche alle distanze tra punti reali adiacenti. Il dataset non possiede cluster naturali.
$H \to 1.0$	$\sum u_i \gg \sum w_i$	**Forte Tendenza al Raggruppamento (Cluster Densi)**: i punti reali risiedono in addensamenti compatti (quindi $w_i \approx 0$). I punti casuali $\mathbf{y}_i$ cadono prevalentemente nelle vaste aree vuote inter-cluster, registrando distanze elevate u_i.
$H \to 0.0$	$\sum u_i \ll \sum w_i$	**Dati Regolarmente Spaziati (Griglia Iper-cubica)**: i punti reali si respingono reciprocamente formando una distribuzione periodica o reticolare (massima dispersione uniforme).

Tabella 10.1: Interpretazione quantitativa della statistica di Hopkins per la verifica della tendenza al clustering.

Nota 10.3.1 (Soglia Decisionale Operativa)**.** Nella pratica dell'analisi multivariata, si considera che un dataset manifesti una tendenza al clustering statisticamente significativa se la media di ripetute stime indipendenti della statistica di Hopkins soddisfa la condizione:

$$\bar{H} > 0.70 \quad \text{(preferibilmente } \bar{H} \geq 0.75 \text{ o } 0.80) \tag{10.8}$$

Se $H \in [0.45, 0.55]$, i dati devono essere considerati indistinguibili da rumore casuale uniforme e l'applicazione di algoritmi di partizionamento deve essere sospesa o considerata esplorativa.

10.4 Validazione Interna mediante Matrici di Prossimità

Un approccio elegante e generale per valutare la coerenza interna di una partizione consiste nel confrontare direttamente la matrice di prossimità osservata con una matrice ideale che rifletta fedelmente l'assegnazione ai cluster individuata dall'algoritmo [2].

10.4.1 Matrice di Similarità vs Matrice di Incidenza Ideale

Sia data una collezione di N oggetti $\mathcal{D} = \{\mathbf{x}_1, \ldots, \mathbf{x}_N\}$ e sia $P \in \mathbb{R}^{N \times N}$ la corrispondente **matrice di prossimità** (o matrice di somiglianza/distanza), dove $P_{ij} = \text{dist}(\mathbf{x}_i, \mathbf{x}_j)$ oppure $P_{ij} = \text{sim}(\mathbf{x}_i, \mathbf{x}_j)$.

Data una partizione $\mathcal{C} = \{\mathcal{C}_1, \ldots, \mathcal{C}_K\}$ prodotta da un algoritmo di clustering, si definisce la **matrice di incidenza ideale** (o matrice di adiacenza ideale) $Y \in \{0,1\}^{N \times N}$ ponendo:

$$Y_{ij} = \begin{cases} 1, & \text{se } \mathbf{x}_i, \mathbf{x}_j \in \mathcal{C}_k \text{ (medesimo cluster)}, \\ 0, & \text{se } \mathbf{x}_i \in \mathcal{C}_k, \mathbf{x}_j \in \mathcal{C}_l \text{ con } k \neq l \text{ (cluster distinti)}. \end{cases} \tag{10.9}$$

Se P_{ij} rappresenta una misura di *similarità* (normalizzata in $[0,1]$), in una partizione ideale e perfetta gli oggetti associati al medesimo cluster dovrebbero avere similarità pari a 1, mentre coppie appartenenti a cluster differenti dovrebbero registrare similarità pari a 0. Pertanto, una partizione di elevata qualità mostrerà un forte accordo statistico tra P e Y.

10.4.2 Coefficiente di Correlazione Matriciale di Pearson

Poiché le matrici P e Y sono simmetriche ed hanno elementi diagonali costanti o irrilevanti ($P_{ii} = 1$ e $Y_{ii} = 1$), il grado di accordo tra la matrice reale e quella ideale viene quantificato calcolando il **coefficiente di correlazione di Pearson** esteso su tutte le $M = \frac{N(N-1)}{2}$ coppie non banali (i,j) con $i < j$:

$$\rho(P,Y) = \frac{\sum_{i<j} \left(P_{ij} - \bar{P}\right)\left(Y_{ij} - \bar{Y}\right)}{\sqrt{\sum_{i<j} \left(P_{ij} - \bar{P}\right)^2 \sum_{i<j} \left(Y_{ij} - \bar{Y}\right)^2}} \tag{10.10}$$

dove $\bar{P}$ e $\bar{Y}$ indicano le medie empiriche dei valori fuori dalla diagonale principale:

$$\bar{P} = \frac{2}{N(N-1)} \sum_{i<j} P_{ij}, \qquad \bar{Y} = \frac{2}{N(N-1)} \sum_{i<j} Y_{ij} \tag{10.11}$$

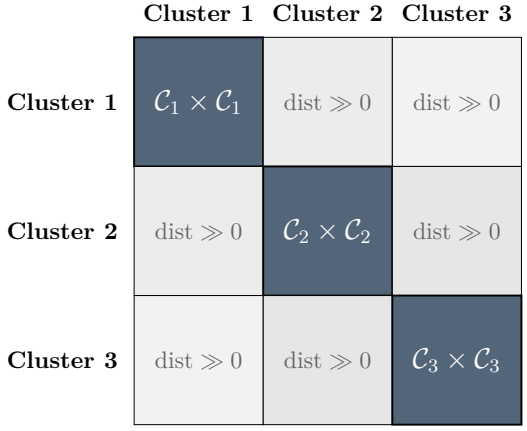
- Se P è una matrice di **similarità**, valori elevati e positivi di $\rho(P, Y) \to +1$ attestano che la configurazione dei cluster rispecchia fedelmente i gradienti di affinità naturale del dataset;
- Se P è una matrice di **distanza/dissimilarità**, la correlazione attesa per cluster ben separati assumerà valori fortemente negativi $\rho(P, Y) \to -1$.

10.4.3 Ispezione Visiva delle Matrici di Dissimilarità Riordinate

Uno strumento qualitativo di straordinaria efficacia diagnostica risiede nella visualizzazione grafica a mappa termica (*heatmap*) della matrice delle distanze, previa **riordinazione degli indici delle istanze**:

1. Si raggruppano gli indici di riga e di colonna in base al cluster di appartenenza: prima tutte le istanze del cluster $\mathcal{C}_1$, poi quelle di $\mathcal{C}_2$, e così via fino a $\mathcal{C}_K$;

2. All'interno di ciascun blocco di cluster, le istanze possono essere ulteriormente ordinate secondo la loro distanza progressiva dal rispettivo centroide o seguendo l'ordinamento delle foglie di un albero gerarchico.



Firma Grafica a Blocchi Diagonali:

• **Blocchi diagonali scuri**: distanze intra-cluster prossime allo zero (elevatissima coesione interna).

• **Regioni fuori diagonale chiare**: distanze inter-cluster marcatamente elevate (separazione netta).

• In presenza di **rumore casuale**, la matrice non mostra alcuna struttura a blocchi, apparendo uniforme e granulare su tutta l'estensione.

Figura 10.2: Rappresentazione schematica della matrice delle distanze riordinata per cluster: la presenza di marcati blocchi quadrati lungo la diagonale principale evidenzia empiricamente la validità della partizione.

10.4.4 Sensibilità della Correlazione Matriciale alla Geometria dei Cluster

Sebbene la correlazione matriciale costituisca uno strumento teorico affascinante, essa risente fortemente dei presupposti geometrici di sfericità:

- **Cluster compatti e sferici**: per partizioni prodotte da K-Means o *Complete Link*, le distanze intra-cluster sono omogeneamente basse e le distanze inter-cluster costantemente alte. In questo scenario la correlazione matriciale è massimizzata (ad esempio $\rho \approx 0.85$ o 0.90);
- **Cluster allungati, complessi o non convessi**: se il dataset presenta cluster concentrici ad anello o forme sinuose continue (gestibili brillantemente da DBSCAN o *Single Linkage*), le distanze euclidee dirette tra punti appartenenti all'estremo opposto del medesimo anello sono ampie, pur appartenendo allo stesso gruppo. La matrice di incidenza Y assegnerà $Y_{ij} = 1$, ma P_{ij} registrerà una distanza considerevole. Ne consegue che la correlazione matriciale $\rho(P, Y)$ risulterà ingannevolmente bassa, pur essendo la partizione geometricamente impeccabile.

10.5 Coesione e Separazione: Teoria ed Equivalenza Algebrica

Le due proprietà cardinali che definiscono la bontà di un partizionamento non supervisionato sono la **coesione** (*cohesion*, o compattezza interna) e la **separazione** (*separation*, o isolamento reciproco) [2]. Nella letteratura scientifica coesistono due prospettive concettuali fondamentali per quantificarle: la formulazione basata su grafi di prossimità e la formulazione basata su prototipi geometrici.

10.5.1 Prospettiva Basata su Grafi vs Basata su Prototipi

1. **Prospettiva basata su grafi**: Le istanze sono nodi di una rete pesata i cui archi hanno peso pari alla somiglianza $s(\mathbf{x}_i, \mathbf{x}_j)$ o alla distanza $d(\mathbf{x}_i, \mathbf{x}_j)$.
    - La *coesione* di un cluster $\mathcal{C}_k$ è la somma dei pesi di tutti gli archi che connettono coppie interne al gruppo:

$$\text{Cohesion}(\mathcal{C}_k) = \sum_{\mathbf{x} \in \mathcal{C}_k, \mathbf{y} \in \mathcal{C}_k} s(\mathbf{x}, \mathbf{y}) \tag{10.12}$$

- La *separazione* tra due cluster distinti $\mathcal{C}_k$ e $\mathcal{C}_l$ è il peso aggregato degli archi del taglio (*cut edges*):

$$\text{Separation}(\mathcal{C}_k, \mathcal{C}_l) = \sum_{\mathbf{x} \in \mathcal{C}_k, \mathbf{y} \in \mathcal{C}_l} s(\mathbf{x}, \mathbf{y}) \tag{10.13}$$

2. **Prospettiva basata su prototipi**: Ciascun cluster $\mathcal{C}_k$ è condensato nel proprio rappresentante geometrico (centroide $\mathbf{c}_k$). Coesione e separazione vengono misurate valutando la dispersione quadratica dei punti rispetto ai propri centroidi e la dispersione dei centroidi rispetto al baricentro globale del dataset.

10.5.2 Formalizzazione Basata su Prototipi: WSS, BSS e TSS

Sia dato un dataset $\mathcal{D} = \{\mathbf{x}_1, \ldots, \mathbf{x}_N\} \subset \mathbb{R}^d$ partizionato in K cluster disgiunti $\mathcal{C}_1, \ldots, \mathcal{C}_K$, con cardinalità $n_k = |\mathcal{C}_k|$ tale che $\sum_{k=1}^{K} n_k = N$.

Si definisce il **centroide locale** $\mathbf{c}_k$ di ciascun cluster e il **centroide globale** $\mathbf{m}$ dell'intero dataset:

$$\mathbf{c}_k = \frac{1}{n_k} \sum_{\mathbf{x} \in \mathcal{C}_k} \mathbf{x}, \qquad \mathbf{m} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{x}_i = \frac{1}{N} \sum_{k=1}^{K} n_k \mathbf{c}_k \tag{10.14}$$

Si definiscono formalmente le tre quantità di devianza quadratica:

1. **Coesione Intra-Cluster (**WSS **o** SSE**)**: somma degli errori quadratici interni ai singoli cluster (*Within-Cluster Sum of Squares*):

$$\text{WSS} = \sum_{k=1}^{K} \sum_{\mathbf{x} \in \mathcal{C}_k} \|\mathbf{x} - \mathbf{c}_k\|_2^2 \tag{10.15}$$

2. **Separazione Inter-Cluster (**BSS**)**: dispersione quadratica dei centroidi di cluster rispetto al baricentro globale $\mathbf{m}$, ponderata per la cardinalità di ciascun gruppo (*Between-Cluster Sum of Squares*):

$$\text{BSS} = \sum_{k=1}^{K} n_k \|\mathbf{c}_k - \mathbf{m}\|_2^2 \tag{10.16}$$

3. **Devianza Totale (**TSS**)**: somma globale degli scarti quadratici di tutti i vettori rispetto al baricentro globale $\mathbf{m}$ (*Total Sum of Squares*):

$$\text{TSS} = \sum_{i=1}^{N} \|\mathbf{x}_i - \mathbf{m}\|_2^2 \tag{10.17}$$

10.5.3 Dimostrazione Rigorosa del Teorema di Conservazione della Varianza

Uno dei risultati cardine dell'analisi multivariata e della cluster validity è il teorema di ortogonalità della scomposizione della devianza totale [2].

Teorema 10.5.1 (Conservazione della Devianza Globale)**.** *Per qualsiasi partizione deterministica di un dataset $\mathcal{D}$ in K cluster disgiunti con centroidi definiti come medie aritmetiche, la somma tra la coesione interna (*WSS*) e la separazione esterna (*BSS*) equivale esattamente alla devianza totale dei dati (*TSS*), la quale è costante e indipendente dall'algoritmo di clustering:*

$$\text{TSS} = \text{WSS} + \text{BSS} \tag{10.18}$$

Dimostrazione. Si consideri la definizione della devianza totale TSS scomponendo la sommatoria su tutti gli N punti nelle K sommatorie disgiunte sui singoli cluster $\mathcal{C}_k$:

$$\text{TSS} = \sum_{i=1}^{N} \|\mathbf{x}_i - \mathbf{m}\|_2^2 = \sum_{k=1}^{K} \sum_{\mathbf{x} \in \mathcal{C}_k} \|\mathbf{x} - \mathbf{m}\|_2^2 \tag{10.19}$$

Per ogni punto $\mathbf{x} \in \mathcal{C}_k$, si aggiunga e si sottragga il rispettivo centroide locale $\mathbf{c}_k$:

$$\mathbf{x} - \mathbf{m} = (\mathbf{x} - \mathbf{c}_k) + (\mathbf{c}_k - \mathbf{m}) \tag{10.20}$$

Calcolando la norma euclidea al quadrato tramite l'espansione del prodotto scalare interiore:

$$\|\mathbf{x}-\mathbf{m}\|_2^2 = \|(\mathbf{x}-\mathbf{c}_k)+(\mathbf{c}_k-\mathbf{m})\|_2^2 = \|\mathbf{x}-\mathbf{c}_k\|_2^2+\|\mathbf{c}_k-\mathbf{m}\|_2^2+2(\mathbf{x}-\mathbf{c}_k)^\top(\mathbf{c}_k-\mathbf{m}) \tag{10.21}$$

Sostituendo questa identità nell'espressione sommatoria su ciascun cluster $\mathcal{C}_k$:

$$\sum_{\mathbf{x} \in \mathcal{C}_k} \|\mathbf{x} - \mathbf{m}\|_2^2 = \sum_{\mathbf{x} \in \mathcal{C}_k} \|\mathbf{x} - \mathbf{c}_k\|_2^2 + \sum_{\mathbf{x} \in \mathcal{C}_k} \|\mathbf{c}_k - \mathbf{m}\|_2^2 + 2 \sum_{\mathbf{x} \in \mathcal{C}_k} (\mathbf{x} - \mathbf{c}_k)^\top (\mathbf{c}_k - \mathbf{m}) \tag{10.22}$$

Si analizzino i tre termini singolarmente:

1. Il primo termine coincide con l'errore quadratico interno al cluster k, la cui somma su tutti i k fornisce per definizione WSS:

$$\sum_{k=1}^{K} \sum_{\mathbf{x} \in \mathcal{C}_k} \|\mathbf{x} - \mathbf{c}_k\|_2^2 = \text{WSS} \tag{10.23}$$

2. Nel secondo termine, la quantità $\|\mathbf{c}_k - \mathbf{m}\|_2^2$ è costante rispetto alla variabile di sommatoria $\mathbf{x}$. Di conseguenza, sommando per tutti gli n_k punti appartenenti a $\mathcal{C}_k$:

$$\sum_{\mathbf{x} \in \mathcal{C}_k} \|\mathbf{c}_k - \mathbf{m}\|_2^2 = n_k \|\mathbf{c}_k - \mathbf{m}\|_2^2 \tag{10.24}$$

Sommando su tutti i K cluster si ottiene esattamente la definizione di BSS:

$$\sum_{k=1}^{K} n_k \|\mathbf{c}_k - \mathbf{m}\|_2^2 = \text{BSS} \tag{10.25}$$

3. Si esamini infine il termine di doppio prodotto incrociato. Poiché il vettore $(\mathbf{c}_k - \mathbf{m})$ non dipende da $\mathbf{x}$, esso può essere fattorizzato all'esterno della sommatoria:

$$\sum_{\mathbf{x} \in \mathcal{C}_k} (\mathbf{x} - \mathbf{c}_k)^\top (\mathbf{c}_k - \mathbf{m}) = \left[\sum_{\mathbf{x} \in \mathcal{C}_k} (\mathbf{x} - \mathbf{c}_k) \right]^\top (\mathbf{c}_k - \mathbf{m}) \tag{10.26}$$

Tuttavia, per definizione di media aritmetica, il baricentro $\mathbf{c}_k$ azzera identicamente la somma degli scarti dei punti da se stesso:

$$\sum_{\mathbf{x} \in \mathcal{C}_k} (\mathbf{x} - \mathbf{c}_k) = \sum_{\mathbf{x} \in \mathcal{C}_k} \mathbf{x} - \sum_{\mathbf{x} \in \mathcal{C}_k} \mathbf{c}_k = n_k \mathbf{c}_k - n_k \mathbf{c}_k = \mathbf{0} \tag{10.27}$$

Di conseguenza, il termine di prodotto incrociato si annulla rigorosamente per ogni cluster k:

$$2 \cdot \mathbf{0}^\top (\mathbf{c}_k - \mathbf{m}) = 0 \tag{10.28}$$

Ricongiungendo i risultati, si ottiene:

$$\text{TSS} = \sum_{k=1}^{K} \left[\sum_{\mathbf{x} \in \mathcal{C}_k} \|\mathbf{x} - \mathbf{c}_k\|_2^2 + n_k \|\mathbf{c}_k - \mathbf{m}\|_2^2 \right] = \text{WSS} + \text{BSS} \tag{10.29}$$

completando la dimostrazione. □

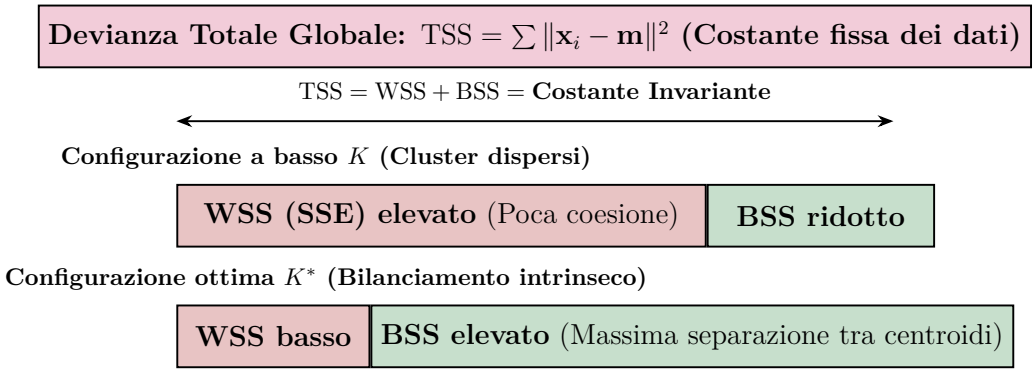


Figura 10.3: Rappresentazione del Teorema di Conservazione della Devianza TSS = WSS + BSS: all'aumentare della qualità o della complessità del clustering, la minimizzazione della devianza interna WSS equivale biunivocamente alla massimizzazione della separazione esterna BSS.

Significato Fondamentale per l'Ottimizzazione

Il Teorema 10.5.1 sancisce che:

$$\min \mathrm{WSS} \iff \max \mathrm{BSS} \tag{10.30}$$

Minimizzare la somma degli errori quadratici intra-cluster (SSE) in K-Means equivale automaticamente e matematicamente a massimizzare la distanza pesata tra i centroidi dei cluster e il baricentro globale. Non è possibile migliorare la coesione interna senza allontanare i gruppi tra loro.
Tuttavia, BSS presenta una patologia analoga a WSS: poiché all'aumentare di K la coesione interna WSS decresce monotonicamente fino ad annullarsi per $K = N$, la separazione BSS cresce monotonicamente verso il tetto massimo TSS. Pertanto, né WSS né BSS possono essere impiegati isolatamente per selezionare il valore ottimo di K.

10.5.4 Esempio Numerico Didattico Unidimensionale

Per apprezzare il funzionamento analitico della conservazione di varianza, si consideri il dataset unidimensionale classico riportato nelle dispense di riferimento [2]:

$$\mathcal{D} = \{1, 2, 4, 5\}, \quad N = 4 \tag{10.31}$$

Passo 0: Calcolo del Baricentro Globale e di TSS

La media globale $\mathbf{m}$ vale:

$$\mathbf{m} = \frac{1+2+4+5}{4} = \frac{12}{4} = 3 \tag{10.32}$$

La devianza totale TSS del dataset vale:

$$\begin{aligned} \text{TSS} &= (1-3)^2 + (2-3)^2 + (4-3)^2 + (5-3)^2 \\ &= (-2)^2 + (-1)^2 + 1^2 + 2^2 = 4+1+1+4 = 10 \end{aligned} \tag{10.33}$$

Questo valore costituisce l'invariante geometrico del sistema.

Caso A: Partizione a Singolo Cluster ($K = 1$)

Se tutti i punti sono assegnati a un unico macro-cluster $\mathcal{C}_1 = \{1, 2, 4, 5\}$:

- Il centroide locale coincide con la media globale: $\mathbf{c}_1 = 3$;
- La coesione interna coincide con l'intera devianza:

$$\text{WSS} = (1-3)^2 + (2-3)^2 + (4-3)^2 + (5-3)^2 = 10 \tag{10.34}$$

- La separazione esterna è nulla, poiché il centroide coincide col baricentro:

$$\text{BSS} = 4 \cdot (3-3)^2 = 0 \tag{10.35}$$

Verifica: $\text{TSS} = 10 + 0 = 10$.

Caso B: Partizione Naturale a Due Cluster ($K = 2$)

Si partizioni il dataset nei due raggruppamenti naturali intuitivi:

$$\mathcal{C}_1 = \{1, 2\}, \qquad \mathcal{C}_2 = \{4, 5\} \tag{10.36}$$

con cardinalità $n_1 = 2$ e $n_2 = 2$.

1. Si calcolano i rispettivi centroidi locali:

$$\mathbf{c}_1 = \frac{1+2}{2} = 1.5, \qquad \mathbf{c}_2 = \frac{4+5}{2} = 4.5 \tag{10.37}$$

2. Si calcola la coesione intra-cluster (WSS):

$$\text{SSE}(\mathcal{C}_1) = (1-1.5)^2 + (2-1.5)^2 = 0.25 + 0.25 = 0.50 \tag{10.38}$$

$$\text{SSE}(\mathcal{C}_2) = (4-4.5)^2 + (5-4.5)^2 = 0.25 + 0.25 = 0.50 \tag{10.39}$$

$$\text{WSS} = \text{SSE}(\mathcal{C}_1) + \text{SSE}(\mathcal{C}_2) = 0.50 + 0.50 = 1.00 \tag{10.40}$$

3. Si calcola la separazione inter-cluster (BSS):

$$\begin{aligned} \text{BSS} &= n_1(\mathbf{c}_1 - \mathbf{m})^2 + n_2(\mathbf{c}_2 - \mathbf{m})^2 && (10.41)\\ &= 2 \cdot (1.5 - 3)^2 + 2 \cdot (4.5 - 3)^2 && (10.42)\\ &= 2 \cdot (-1.5)^2 + 2 \cdot (1.5)^2 = 2 \cdot 2.25 + 2 \cdot 2.25 = 4.50 + 4.50 = 9.00 && (10.43) \end{aligned}$$

Verifica del teorema di conservazione:

$$\text{WSS} + \text{BSS} = 1.00 + 9.00 = 10.00 = \text{TSS} \tag{10.44}$$

L'introduzione della partizione corretta a due cluster ha abbattuto la devianza interna del 90% (da 10 a 1), convertendola integralmente in separazione tra gruppi (BSS = 9).

10.6 Il Coefficiente di Silhouette

Introdotto originariamente da Peter Rousseeuw nel 1987 [10], il **Coefficiente di Silhouette** è una delle metriche di validazione interna più versatili ed ampiamente utilizzate nel machine learning. Il suo pregio fondamentale risiede nella capacità di fondere simultaneamente la nozione di coesione interna e di separazione esterna a livello di **singolo punto dati**, senza dipendere esplicitamente dalla forma sferica o dal baricentroide medio [2].

10.6.1 Definizione Matematica Punto per Punto

Sia data una partizione di un dataset $\mathcal{D}$ in $K \geq 2$ cluster $\mathcal{C}_1, \ldots, \mathcal{C}_K$. Per ogni specifica istanza $\mathbf{x}_i \in \mathcal{C}_k$:

1. Si calcola la **dissimilarità intra-cluster media** $a(i)$, che esprime la distanza media di $\mathbf{x}_i$ da tutti gli altri punti appartenenti al proprio cluster:

$$a(i) = \frac{1}{|\mathcal{C}_k| - 1} \sum_{\mathbf{x}_j \in \mathcal{C}_k, j \neq i} \text{dist}(\mathbf{x}_i, \mathbf{x}_j) \tag{10.45}$$

   Il parametro $a(i)$ quantifica la **coesione**: valori piccoli indicano che il punto si trova profondamente integrato e vicino ai compagni di cluster.

2. Si determina la **distanza media verso i cluster vicini**: per ogni altro cluster $\mathcal{C}_l \neq \mathcal{C}_k$, si calcola la distanza media da $\mathbf{x}_i$ a tutti i punti di $\mathcal{C}_l$:

$$d(\mathbf{x}_i, \mathcal{C}_l) = \frac{1}{|\mathcal{C}_l|} \sum_{\mathbf{x}_j \in \mathcal{C}_l} \text{dist}(\mathbf{x}_i, \mathbf{x}_j) \tag{10.46}$$

Si definisce $b(i)$ come il minimo di tali medie tra tutti i cluster concorrenti:

$$b(i) = \min_{l \neq k} d(\mathbf{x}_i, \mathcal{C}_l) \tag{10.47}$$

Il cluster che realizza il minimo in (10.47) viene designato come il **cluster più vicino** (*neighboring cluster*) per l'istanza $\mathbf{x}_i$. La quantità $b(i)$ quantifica la **separazione**: valori ampi indicano un marcato isolamento dal cluster contiguo.

3. Il **coefficiente di Silhouette individuale** $s(i)$ per il punto $\mathbf{x}_i$ è definito come la differenza normalizzata:

$$s(i) = \frac{b(i) - a(i)}{\max\left(a(i),\ b(i)\right)} \tag{10.48}$$

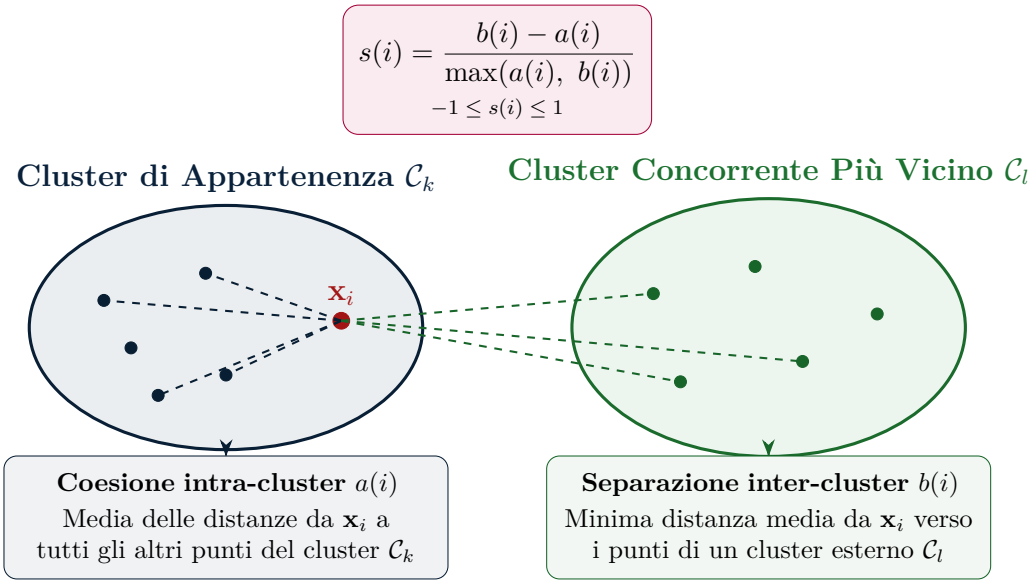


Figura 10.4: Geometria del coefficiente di Silhouette: confronto visivo tra la coesione interna $a(i)$ e la separazione dal cluster contiguo più vicino $b(i)$.

10.6.2 Analisi dei Regimi Limite del Coefficiente

Dalla presenza del denominatore normalizzante $\max(a(i), b(i))$, scaturisce direttamente che:

$$-1 \leq s(i) \leq 1, \quad \forall i \tag{10.49}$$

L'interpretazione analitica dei tre regimi estremi è riassunta di seguito:

- **Condizione Ottimale** ($s(i) \to +1$): Si verifica quando $a(i) \ll b(i)$, ovvero la distanza interna è trascurabile rispetto alla separazione dal cluster contiguo. Il punto risiede nel cuore del proprio raggruppamento, perfettamente isolato dall'esterno.

- **Punto di Confine** ($s(i) \approx 0$): Si verifica quando $a(i) \approx b(i)$. Il punto giace esattamente sulla frontiera decisionale di Voronoi che separa il cluster $\mathcal{C}_k$ dal cluster contiguo $\mathcal{C}_l$. L'assegnazione ad uno o all'altro gruppo risulta ambigua ed arbitraria.

- **Errore Grave di Assegnazione** ($s(i) \to -1$): Si verifica quando $a(i) \gg b(i)$. In questo caso, il punto si trova mediamente molto più vicino agli elementi del cluster concorrente rispetto ai membri del gruppo a cui è stato formalmente assegnato dall'algoritmo. Tale circostanza evidenzia un evidente fallimento dell'ottimizzazione o un vincolo rigido sub-ottimale.

10.6.3 Silhouette Score Globale e Grafici di Profilo

Per passare dalla valutazione puntuale alla validazione globale dell'intera partizione, si calcola il **Silhouette Score Medio** $\bar{s}$:

$$\bar{s} = \frac{1}{N} \sum_{i=1}^{N} s(i) \tag{10.50}$$

Un Silhouette Score globale $\bar{s} > 0.70$ denota una struttura eccellente a cluster forti e separati; valori nell'intervallo $[0.50, 0.70]$ attestano una struttura ragionevole; punteggi inferiori a 0.25 denotano assenza di cluster o forte sovrapposizione.

Oltre al valore medio aggregato, la silhouette consente la costruzione del **Silhouette Plot** (grafico dei profili):

1. Si ordinano le istanze di ciascun cluster in senso decrescente in base al loro coefficiente $s(i)$;

2. Si traccia un istogramma a barre orizzontali per ogni cluster, visualizzando lo spessore (cardinalità del gruppo) e la penetrazione verso destra lungo l'asse orizzontale $[-1, 1]$;

3. Si traccia una linea verticale tratteggiata in corrispondenza del valore medio globale $\bar{s}$.

Questo grafico consente di diagnosticare istantaneamente se determinati cluster siano intrinsecamente deboli (ossia contengano numerose barre corte o negative) pur a fronte di un punteggio medio globale apparentemente soddisfacente.

10.7 Metriche Interne: Davies-Bouldin e Caliński-Harabasz

Nel laboratorio analitico di data mining (in particolare nel notebook sperimentale di clustering [2]), vengono impiegate metriche complementari che offrono un compromesso computazionale superiore rispetto alla Silhouette (la quale richiede la memorizzazione o il calcolo quadratico delle distanze a coppie $\mathcal{O}(N^2)$).

10.7.1 Indice di Davies-Bouldin (DB Score)

Introdotto da Davies e Bouldin nel 1979 [11], l'indice DB valuta il peggior rapporto di similarità tra ciascun cluster e il suo rivale più prossimo.

Siano $\mathbf{c}_i$ e $\mathbf{c}_j$ i centroidi dei cluster $\mathcal{C}_i$ e $\mathcal{C}_j$. Si definisce la dispersione intra-cluster media σ_i come la distanza media dei punti appartenenti a $\mathcal{C}_i$ rispetto a $\mathbf{c}_i$:

$$\sigma_i = \frac{1}{|\mathcal{C}_i|} \sum_{\mathbf{x} \in \mathcal{C}_i} \|\mathbf{x} - \mathbf{c}_i\|_2 \tag{10.51}$$

La similarità R_{ij} tra la coppia di cluster $\mathcal{C}_i$ e $\mathcal{C}_j$ è definita come:

$$R_{ij} = \frac{\sigma_i + \sigma_j}{d(\mathbf{c}_i, \mathbf{c}_j)} \tag{10.52}$$

dove al numeratore compare la somma delle dispersioni interne (richiesta di coesione) e al denominatore compare la distanza euclidea tra i due centroidi $d(\mathbf{c}_i, \mathbf{c}_j) = \|\mathbf{c}_i - \mathbf{c}_j\|_2$ (richiesta di separazione).

L'indice di **Davies-Bouldin** DB è definito calcolando la media aritmetica della massima similarità registrata da ciascun cluster:

$$DB = \frac{1}{K} \sum_{i=1}^{K} \max_{j \neq i} R_{ij} \tag{10.53}$$

Proprietà 10.7.1 (Proprietà dell'Indice DB).

- *Il punteggio DB è limitato inferiormente da zero: $DB \in [0, +\infty)$;*

- ***Criterio di Ottimalità**: Valori inferiori denotano un clustering di qualità superiore (cluster compatti con dispersioni σ_i piccole e ben distanziati con denominatore $d(\mathbf{c}_i, \mathbf{c}_j)$ grande);*
- *Complessità computazionale: calcolabile in tempo $\mathcal{O}(N \cdot d + K^2 \cdot d)$, risultando drasticamente più veloce della Silhouette sui dataset di grandi dimensioni.*

10.7.2 Indice di Caliński-Harabasz (Variance Ratio Criterion)

Proposto da Caliński e Harabasz nel 1974 [12], l'indice CH (detto anche *Variance Ratio Criterion*) rapporta direttamente la separazione inter-cluster BSS e la coesione intra-cluster WSS, normalizzandole per i rispettivi gradi di libertà:

$$CH = \frac{\mathrm{BSS}/(K-1)}{\mathrm{WSS}/(N-K)} \tag{10.54}$$

Valori più elevati di CH indicano cluster maggiormente densi e separati. In corrispondenza del valore intrinseco di K^*, la curva $CH(K)$ manifesta frequentemente un picco locale pronunciato.

10.8 Misure Esterne: Entropia, Purezza e Corrispondenza

Quando è disponibile una classificazione esterna indipendente (etichette fornite a priori da esperti di dominio), le metriche supervisionate consentono di misurare fino a che punto i cluster scoperti ricostruiscano o riflettano fedelmente tali categorie [2].

10.8.1 Distribuzioni di Probabilità Empiriche e Matrice di Contingenza

Sia data una partizione prodotta dall'algoritmo $\mathcal{C} = \{\mathcal{C}_1, \ldots, \mathcal{C}_K\}$ e sia $\mathcal{L} = \{\mathcal{L}_1, \ldots, \mathcal{L}_L\}$ la partizione reale delle etichette esterne (*ground truth*), con dimensione totale del dataset pari a N.

Si costruisce la **matrice di contingenza** (o di co-occorrenza) la cui cella generica m_{ij} rappresenta il numero di elementi assegnati contemporaneamente al cluster $\mathcal{C}_i$ e alla classe esterna $\mathcal{L}_j$:

$$m_{ij} = |\mathcal{C}_i \cap \mathcal{L}_j|, \qquad m_i = |\mathcal{C}_i| = \sum_{j=1}^{L} m_{ij}, \qquad m_{\cdot j} = |\mathcal{L}_j| = \sum_{i=1}^{K} m_{ij} \tag{10.55}$$

Per ciascun cluster $\mathcal{C}_i$, si definisce la distribuzione di probabilità empirica condizionata che un elemento estratto a caso da $\mathcal{C}_i$ appartenga alla classe $\mathcal{L}_j$:

$$p_{ij} = \frac{m_{ij}}{m_i} = \frac{|\mathcal{C}_i \cap \mathcal{L}_j|}{|\mathcal{C}_i|} \tag{10.56}$$

ovviamente con $\sum_{j=1}^{L} p_{ij} = 1$ per ogni cluster fisso i.

10.8.2 Formulazione Matematica di Entropia e Purezza

1. **Entropia Individuale di un Cluster (e_i)**: Applicando la definizione fondazionale di Entropia di Shannon formalizzata nel Capitolo 6 (sezione 6.9), si quantifica il disordine informativo o l'eterogeneità interna del cluster $\mathcal{C}_i$ rispetto alla distribuzione delle classi di riferimento:

$$e_i = -\sum_{j=1}^{L} p_{ij} \log_2(p_{ij}) \tag{10.57}$$

   convenendo che $0 \log_2(0) = 0$. L'entropia è minima ($e_i = 0$) se il cluster è perfettamente puro (contiene istanze appartenenti a una singola classe); è massima ($\log_2 L$) se tutte le classi sono equiprobabili nel cluster.

2. **Entropia Globale Ponderata del Clustering (E)**: Media pesata delle entropie individuali, proporzionale alla dimensione dei singoli cluster:

$$E = \sum_{i=1}^{K} \frac{m_i}{N} e_i = -\sum_{i=1}^{K} \frac{m_i}{N} \sum_{j=1}^{L} p_{ij} \log_2(p_{ij}) \tag{10.58}$$

   Minori valori di E indicano partizioni di qualità superiore rispetto alle etichette di classe.

3. **Purezza Individuale di un Cluster (p_i)**: Rappresenta la frazione del cluster occupata dalla classe maggioritaria in esso contenuta:

$$p_i = \max_j p_{ij} = \max_j \frac{m_{ij}}{m_i} \tag{10.59}$$

4. **Purezza Globale Ponderata del Clustering (P)**: Media pesata delle purezze individuali:

$$P = \sum_{i=1}^{K} \frac{m_i}{N} p_i = \frac{1}{N} \sum_{i=1}^{K} \max_j m_{ij} \tag{10.60}$$

   La purezza varia nell'intervallo $[0, 1]$: valori prossimi ad 1 attestano che ciascun cluster è dominato quasi esclusivamente da un'unica classe target.

10.8.3 Studio di Caso Analitico: Il Corpus Testuale "LA Documents"

Nelle lezioni di riferimento [2] viene esaminato un esperimento applicativo condotto sul corpus testuale *Los Angeles Times Documents* ($N = 3\,204$ articoli ripartiti su 6 argomenti di cronaca: intrattenimento, finanza, esteri, politica, sport, tecnologia).

Il dataset viene partizionato mediante K-Means impostando $K = 9$. La matrice di contingenza risultante è riportata integralmente nella Tabella 10.2.

Cluster	**Ent.**	**Fin.**	**Est.**	**Pol.**	**Spo.**	**Tec.**	**Totale**	**Purezza**	**Entropia**
$\mathcal{C}_1$	3	5	40	506	0	3	557	0.908	0.505
$\mathcal{C}_2$	17	7	280	29	0	0	333	0.841	0.778
$\mathcal{C}_3$	2	1	1	7	0	2	13	0.538	1.761
$\mathcal{C}_4$	7	1	1	0	422	3	434	0.972	0.239
$\mathcal{C}_5$	40	180	20	10	2	3	255	0.706	1.341
$\mathcal{C}_6$	4	505	6	0	0	4	519	0.973	0.208
$\mathcal{C}_7$	423	3	12	3	0	1	442	0.957	0.320
$\mathcal{C}_8$	105	3	5	1	0	0	114	0.921	0.449
$\mathcal{C}_9$	60	17	85	8	0	367	537	0.683	1.258
Globale							**3204**	**0.887**	**0.672**

Tabella 10.2: Matrice di contingenza e calcolo di entropia e purezza per il clustering a $K = 9$ del dataset *LA Documents* (Tan et al. [2]).

Interpretazione Qualitativa dei Risultati

Dall'ispezione analitica della Tabella 10.2 emergono osservazioni cruciali:

1. **Cluster ad altissima purezza**: il cluster $\mathcal{C}_4$ (422 articoli di Sport su 434, purezza 97.2%) e il cluster $\mathcal{C}_6$ (505 articoli di Finanza su 519, purezza 97.3%) manifestano una separazione semantica quasi perfetta. Le loro entropie sono inferiori a 0.25;

2. **Sotto-partizione naturale della classe Intrattenimento**: la classe *Entertainment* è stata suddivisa dall'algoritmo tra il cluster $\mathcal{C}_7$ (423 articoli) e il cluster $\mathcal{C}_8$ (105 articoli). Sebbene tale classe sia frammentata su due cluster, entrambi i cluster risultano altamente puri (95.7% e 92.1%), a dimostrazione che l'algoritmo ha scoperto due sottotemi distinti all'interno della stessa macro-categoria;

3. **Cluster misti complessi**: il cluster $\mathcal{C}_9$ racchiude 367 articoli di Tecnologia, ma è contaminato da 85 articoli Esteri e 60 di Intrattenimento, determinando un'entropia elevata (1.258).

10.9 Quadro Statistico Inferenziale e Modelli Nulli

Come stabilito nella Sezione 10.1, un valore di validità (ad esempio un SSE = 24.5 o una correlazione matriciale $\rho = 0.42$) è privo di significato epistemologico se non confrontato con il valore atteso sotto una distribuzione di probabilità di riferimento. L'approccio rigoroso della cluster validity si fonda sull'**inferenza statistica basata su modelli nulli** [2, 8].

10.9.1 Formulazione dell'Ipotesi Nulla di Randomness

Per stabilire la significatività statistica di un clustering, si formula formalmente l'ipotesi nulla:

$$H_0 : \text{I dati sono privi di cluster e generati casualmente.} \tag{10.61}$$

Nel contesto dei dati continui multidimensionali, l'ipotesi nulla standard modella le osservazioni come un processo di Poisson omogeneo generato uniformemente all'interno dell'iper-rettangolo o dell'iper-sfera che racchiude i dati osservati:

$$\mathbf{x} \sim \mathcal{U}\left(\prod_{j=1}^{d}[a_j, b_j]\right) \tag{10.62}$$

oppure mediante una permutazione casuale indipendente degli attributi lungo ciascuna coordinata (preservando le distribuzioni marginali unidimensionali ma distruggendo la covarianza multivariata).

10.9.2 Simulazione Monte Carlo e Distribuzione Empirica Sotto H_0

Poiché la distribuzione analitica di metriche complesse come l'SSE o la correlazione matriciale non è derivabile in forma chiusa sotto H_0, si ricorre al **campionamento Monte Carlo**:

1. Si generano B dataset casuali indipendenti $\mathcal{D}^{(1)}, \mathcal{D}^{(2)}, \ldots, \mathcal{D}^{(B)}$ (tipicamente $B = 500$ o $1\,000$) con la medesima cardinalità N e dimensionalità d del dataset reale sotto l'ipotesi nulla H_0;

2. Si esegue l'algoritmo di clustering su ciascun dataset sintetico $\mathcal{D}^{(b)}$ con le medesime condizioni e iperparametri;

3. Si calcola il valore della metrica statistica target $\theta^{(b)}$ (ad esempio l'SSE o la correlazione ρ) su ciascun dataset sintetico;

4. Si costruisce la distribuzione empirica nulla $f(\theta \mid H_0)$.

10.9.3 Calcolo del p-Value Empirico

Sia θ_{obs} il valore della statistica calcolato sul dataset reale.

- Per metriche in cui la bontà è associata alla minimizzazione (come l'SSE), il p-value empirico è la proporzione di dataset casuali che hanno prodotto un errore uguale o inferiore a quello osservato:

$$p\text{-value} = \frac{1}{B} \sum_{b=1}^{B} \mathbb{I}\left(\theta^{(b)} \leq \theta_{\text{obs}}\right) \tag{10.63}$$

- Per metriche in cui la qualità è associata alla massimizzazione (come la correlazione matriciale ρ o la Silhouette):

$$p\text{-value} = \frac{1}{B} \sum_{b=1}^{B} \mathbb{I}\left(\theta^{(b)} \geq \theta_{\text{obs}}\right) \tag{10.64}$$

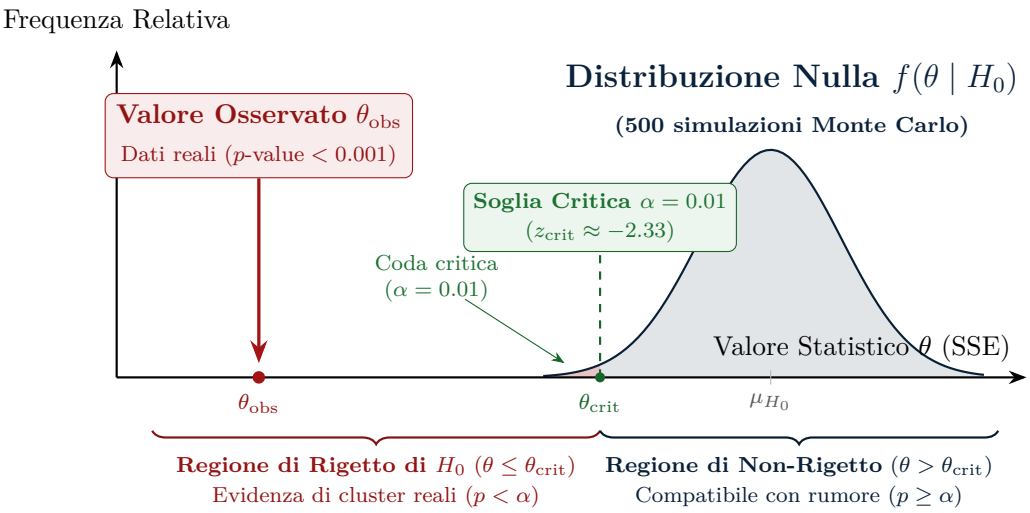


Figura 10.5: Quadro decisionale dell'ipotesi nulla: se l'errore SSE reale cade marcatamente a sinistra della distribuzione generata casualmente, l'ipotesi di casualità viene rigettata con significatività statistica rigorosa.

Come sintetizzato graficamente nella Figura 10.5, se il valore empirico osservato θ_{obs} risiede ben al di fuori della coda di probabilità della distribuzione nulla (p-value $< \alpha$, tipicamente $\alpha = 0.01$), l'ipotesi di uniformità viene rigettata: la partizione non è un mero artefatto algoritmico ma riflette una discontinuità reale nei dati.

10.10 Laboratorio Python: Metriche di Validazione e Selezione di K

In questa sezione finale vengono presentate le metodologie operative implementate nell'attività laboratoriale di data mining (desunte dal notebook ufficiale del corso `3_clustering.ipynb` [2]), evidenziando l'impiego del package `scikit-learn` per la quantificazione delle metriche interne ed esterne e la stima del numero ottimale di cluster.

Ambiente Interattivo e Riproducibilità del Laboratorio

Tutte le routine computazionali, le metriche di validazione e le procedure di model selection presentate in questa sezione sono direttamente eseguibili e riproducibili tramite l'ambiente Jupyter ufficiale del corso:

- Notebook interattivo: **Notebook 3 (Clustering)** (file `Data-Mining/notebooks/3_clustering.ipynb`);
- Avvio rapido con Docker: `docker compose up jupyter` (oppure tramite script Windows `scripts/start-jupyter.cmd`);
- Interfaccia web: `http://localhost:8888/lab` (accesso diretto senza token o password).

Nota 10.10.1 (Riferimento al Notebook Interattivo)**.** Il codice delle sottosezioni seguenti mappa direttamente le celle della sezione *Metrics to Evaluate Clustering* e *K parameter estimation* del notebook interattivo **3_clustering.ipynb**:

- **Metriche interne** (SSE, Davies-Bouldin, Silhouette): celle 33–34;
- **Metodo del gomito** (scansione di $K \in [2, 40]$ ed evidenziazione di K^*): celle 38–40;
- **Validazione esterna** (matrice di contingenza e profilazione con `day`): celle 35–37.

10.10.1 Implementazione Metriche Interne in Scikit-Learn

Nel codice di laboratorio (cella 34 del **Notebook 3**), dopo aver addestrato un modello K-Means sulla matrice pre-processata dei dati numerici standardizzati X, le metriche di compattezza (SSE), separazione (Davies-Bouldin) e concordanza (Silhouette) vengono estratte in modo conciso:

Listing 10.1: Calcolo congiunto di SSE, Davies-Bouldin e Silhouette Score in Python.

```
from sklearn.cluster import KMeans
from sklearn import metrics
from sklearn.metrics import silhouette_score

# Addestramento di K-Means con K fissato
kmeans = KMeans(n_clusters=4, n_init=10, max_iter=300,
    random_state=42)
kmeans.fit(X)

```

```
# 1. Coesione intra-cluster: Inerzia (SSE / WSS)
# Attributo built-in che calcola la somma delle distanze quadratiche dai centroidi
sse = kmeans.inertia_
print(f'SSE (Inertia / WSS): {sse:.3f}')

# 2. Separazione inter-cluster: Davies-Bouldin Score
# Valuta il rapporto di similarita peggiore; ottimale vicino a 0
db_score = metrics.davies_bouldin_score(X, kmeans.labels_)
print(f'Davies-Bouldin Score: {db_score:.3f}')

# 3. Silhouette Score globale: media armonica tra coesione e separazione
# Spettro [-1, 1]; ottimale vicino a +1
sil_score = silhouette_score(X, kmeans.labels_)
print(f'Silhouette Score: {sil_score:.3f}')
```

10.10.2 Stima del Parametro K Ottimale: Il Metodo del Gomito (Elbow/Knee)

Come formalizzato nel Capitolo 7 (sottosezione 7.4.1), l'identificazione euristica di K^* si basa sull'individuazione del punto di flesso (*knee point*) in cui il guadagno marginale di coesione interna subisce una marcata decelerazione.

Il listato 10.2 (desunto dalle celle 38–40 del **Notebook 3**) illustra la scansione programmatica dell'intervallo $K \in [2, 40]$ e il tracciamento della curva per individuare visivamente la transizione a gomito:

Listing 10.2: Scansione sistematica e tracciamento della curva del gomito per la selezione di K^*.

```
import matplotlib.pyplot as plt

sse_list = []
max_k = 40

# Iterazione sull'intervallo di complessita
for k in range(2, max_k + 1):
    km = KMeans(n_clusters=k, n_init=10, max_iter=100, random_state=42)
    km.fit(X)
    sse_list.append(km.inertia_)

# Identificazione del punto critico di ginocchio
k_star = 10
sse_star = sse_list[k_star - 2]  # Allineamento indice base-zero
```

```
15
16 # Visualizzazione grafica della curva del gomito
17 plt.figure(figsize=(9, 5))
18 plt.plot(range(2, max_k + 1), sse_list, marker='o', color='
    navy', lw=2)
19 plt.ylabel('SSE (Inertia)', fontsize=14)
20 plt.xlabel('Numero di Cluster (K)', fontsize=14)
21 plt.title('Metodo del Gomito per la Stima di K*', fontsize
    =16)
22
23 # Linee tratteggiate di ancoraggio sul punto ottimale
24 plt.vlines(x=k_star, ymin=0, ymax=sse_star, colors='crimson
    ', linestyles='dashed', lw=1.5)
25 plt.hlines(y=sse_star, xmin=0, xmax=k_star, colors='crimson
    ', linestyles='dashed', lw=1.5)
26 plt.scatter(k_star, sse_star, color='crimson', s=90, zorder
    =5, label=f'Ginocchio K*={k_star}')
27
28 plt.legend(fontsize=12)
29 plt.grid(True, linestyle=':', alpha=0.6)
30 plt.show()
```

10.10.3 Validazione Esterna tramite Tabulazioni Incrociate (Crosstab)

Nel laboratorio (celle 35–37 del **Notebook 3**), per esaminare se i cluster numerici scoperti corrispondano a comportamenti differenziati rispetto a variabili categoriche esterne non utilizzate nel clustering (come il giorno della settimana `day` o la fascia oraria `time` nel dataset delle mance `tips`), si impiega la funzione `pd.crosstab`:

Listing 10.3: Analisi di contingenza empirica e profilazione esterna dei cluster in Pandas.

```
1 import pandas as pd
2
3 # Creazione della matrice di contingenza tra etichette di
    cluster e variabile esterna
4 cross_df = pd.crosstab(kmeans.labels_, df_cat['day'])
5 print("Matrice di Contingenza Cluster vs Giorno:")
6 print(cross_df)
7
8 # Visualizzazione della distribuzione per cluster
9 cross_df.plot(kind='bar', stacked=False, figsize=(10, 5),
    colormap='viridis')
10 plt.title('Distribuzione della variabile esterna "day" per
    cluster')
```

```
plt.xlabel('Cluster ID')
plt.ylabel('Conteggio Istanza')
plt.grid(axis='y', linestyle=':', alpha=0.7)
plt.show()
```

La matrice di contingenza consente all'analista di calcolare immediatamente la purezza e l'entropia condizionata del partizionamento rispetto alle etichette categoriche, completando in tal modo il ciclo di valutazione quantitativa della validità.

Capitolo 11

Analisi delle Associazioni: Concetti di Base e Algoritmi

11.1 Introduzione al Mining delle Regole di Associazione

L'analisi delle associazioni (*Association Analysis*) costituisce uno dei pilastri metodologici fondamentali del Data Mining non supervisionato [2]. Nata originariamente nell'ambito dell'analisi dei dati di vendita al dettaglio (la cosiddetta *Market-Basket Analysis* o analisi del carrello della spesa) per investigare le abitudini di acquisto della clientela, tale disciplina fornisce un impianto formale, algoritmico e statistico per identificare relazioni nascoste, ricorrenze congiunte e dipendenze informative all'interno di collezioni estese di dati transazionali [13, 14].

Nell'apprendimento non supervisionato, l'analisi delle associazioni differisce concettualmente sia dalla classificazione (dove un attributo target è noto a priori) sia dal clustering (in cui si raggruppano istanze simili nello spazio geometrico). L'obiettivo primario consiste nello scoprire regolarità intrinseche espresse sotto forma di **regole di associazione**, ovvero implicazioni logico-statistiche che legano la presenza di specifici elementi all'interno di una transazione alla presenza congiunta di altri elementi [2, 13].

11.1.1 Inquadramento e Modello Transazionale

Il paradigma di riferimento modella il dominio applicativo mediante una collezione di registrazioni discrete, denominate *transazioni*. In ambito retail, una transazione corrisponde allo scontrino o carrello di spesa di un cliente in una determinata sessione d'acquisto.

TID (Transaction ID)	Items (Insieme degli Articoli Acquistati)
1	Pane, Latte
2	Pane, Pannolini, Birra, Uova
3	Latte, Pannolini, Birra, Coca-Cola
4	Pane, Latte, Pannolini, Birra
5	Pane, Latte, Pannolini, Coca-Cola

Tabella 11.1: Esempio di database transazionale giocattolo per la Market-Basket Analysis.

Nella Tabella 11.1, ogni transazione è contraddistinta da un identificativo univoco (`TID`) ed è costituita da un sottoinsieme non vuoto di articoli estratti dall'inventario merci complessivo del punto vendita.

11.1.2 Esempi di Regole di Associazione

Dall'ispezione empirica dei dati transazionali possono essere estratte implicazioni della forma:

$$\{\text{Pannolini}\} \longrightarrow \{\text{Birra}\} \tag{11.1}$$

$$\{\text{Latte, Pane}\} \longrightarrow \{\text{Uova, Coca-Cola}\} \tag{11.2}$$

$$\{\text{Birra, Pane}\} \longrightarrow \{\text{Latte}\} \tag{11.3}$$

Distinzione Fondamentale: Co-occorrenza Statistica vs Causalità

L'implicazione logica espressa nella regola $X \to Y$ non descrive una relazione causa-effetto né postula una dipendenza temporale o sequenziale. Essa attesta esclusivamente una **co-occorrenza statistica** all'interno del medesimo paniere d'acquisto [2].
Affermare che $\{\text{Pannolini}\} \to \{\text{Birra}\}$ significa semplicemente che i clienti che hanno inserito i pannolini nel carrello hanno mostrato un'elevata propensione congiunta ad acquistare anche birra nella stessa transazione; non implica in alcun modo che l'acquisto dei pannolini abbia generato o causato la necessità di acquistare birra.

11.2 Definizioni Formali: Itemset, Supporto e Frequenza

Per formalizzare rigorosamente l'analisi delle associazioni, si definiscono gli spazi matematici degli elementi e delle transazioni [2, 14].

Definizione 11.2.1 (Spazio degli Item e Database Transazionale)**.** Sia $I = \{i_1, i_2, \ldots, i_d\}$ l'insieme finito di tutti i d elementi distinti (*items*) presenti nel dominio. Sia $T = \{t_1, t_2, \ldots, t_N\}$ l'insieme delle N transazioni del database, dove ciascuna transazione $t_j \subseteq I$ rappresenta un sottoinsieme non vuoto di articoli.

Definizione 11.2.2 (k-Itemset)**.** Un insieme arbitrario di elementi $X \subseteq I$ è denominato **itemset**. Se la cardinalità dell'insieme è pari a $|X| = k$, l'itemset prende il nome di **k-itemset**.

Ad esempio, nel dataset in Tabella 11.1, l'itemset X = {Latte, Pane, Pannolini} contiene 3 articoli ed è pertanto un 3-itemset.

11.2.1 Conteggio di Supporto e Supporto Relativo

La quantificazione della rilevanza statistica di un itemset poggia sulla frequenza con cui esso compare all'interno del database transazionale.

Definizione 11.2.3 (Support Count / Conteggio di Supporto (σ))**.** Dato un itemset $X \subseteq I$, il suo conteggio di supporto $\sigma(X)$ è la frequenza assoluta di transazioni in T che contengono X come sottoinsieme:

$$\sigma(X) = \Big|\{t_j \in T \mid X \subseteq t_j\}\Big| \tag{11.4}$$

Verifichiamo analiticamente la definizione per l'itemset X = {Latte, Pane, Pannolini} rispetto alle transazioni della Tabella 11.1:

- Transazione 1 (t_1): {Pane, Latte} (mancano i Pannolini) $\rightarrow$ non contiene X;
- Transazione 2 (t_2): {Pane, Pannolini, Birra, Uova} (manca il Latte) $\rightarrow$ non contiene X;
- Transazione 3 (t_3): {Latte, Pannolini, Birra, Coca-Cola} (manca il Pane) $\rightarrow$ non contiene X;
- Transazione 4 (t_4): {Pane, Latte, Pannolini, Birra} $\rightarrow$ contiene X;
- Transazione 5 (t_5): {Pane, Latte, Pannolini, Coca-Cola} $\rightarrow$ contiene X.

Pertanto, il conteggio assoluto risulta pari a $\sigma(\{\text{Latte, Pane, Pannolini}\}) = 2$.

Definizione 11.2.4 (Supporto Relativo (s))**.** Il supporto relativo $s(X)$ è la proporzione o frazione di transazioni nel database che contengono l'itemset X rispetto alla cardinalità complessiva $|T| = N$:

$$s(X) = \frac{\sigma(X)}{|T|} = \frac{\sigma(X)}{N} \tag{11.5}$$

Nel nostro esempio giocattolo con $N = 5$:

$$s(\{\text{Latte, Pane, Pannolini}\}) = \frac{2}{5} = 0.40 \quad (40\%) \tag{11.6}$$

Definizione 11.2.5 (Itemset Frequente e Soglia minsup)**.** Dato un parametro di soglia minima stabilito dall'analista, denominato **minsup** (*minimum support threshold*) con minsup $\in (0, 1]$, un itemset X è definito **frequente** se e solo se il suo supporto relativo soddisfa:

$$s(X) \geq \text{minsup} \quad \iff \quad \sigma(X) \geq \lceil \text{minsup} \times |T| \rceil \tag{11.7}$$

Se impostiamo minsup $= 0.50$ (ovvero $\sigma_{\min} = 2.5 \implies 3$ transazioni), l'itemset $\{\text{Latte, Pane, Pannolini}\}$ con $s = 0.40$ risulta non frequente; se invece si imposta minsup $= 0.40$, esso risulta frequente.

11.3 Regole di Associazione e Metriche Fondamentali

Una volta formalizzati gli itemset, si definisce la nozione di regola di associazione.

Definizione 11.3.1 (Regola di Associazione)**.** Una regola di associazione è un'espressione formale d'implicazione della forma:

$$X \longrightarrow Y \tag{11.8}$$

dove $X, Y \subset I$ sono due itemset disgiunti, ovvero $X \cap Y = \emptyset$. L'itemset X prende il nome di **antecedente** (o lato sinistro, *LHS*), mentre Y prende il nome di **conseguente** (o lato destro, *RHS*).

11.3.1 Metriche Primarie: Supporto e Confidenza

La valutazione della significatività statistica e dell'affidabilità di una regola poggia su due metriche classiche introdotte da Agrawal et al. [13, 2]:

Definizione 11.3.2 (Supporto della Regola)**.** Il supporto di una regola $X \to Y$ misura la frazione di transazioni del database che contengono simultaneamente sia l'antecedente sia il conseguente, coincidente con la probabilità empirica congiunta $P(X \cup Y)$:

$$s(X \longrightarrow Y) = \frac{\sigma(X \cup Y)}{|T|} = P(X \cup Y) \tag{11.9}$$

Definizione 11.3.3 (Confidenza della Regola)**.** La confidenza di una regola $X \to Y$ quantifica l'accuratezza o l'affidabilità dell'inferenza condizionale, indicando la percentuale di transazioni contenenti X che contengono contemporaneamente anche Y:

$$c(X \longrightarrow Y) = \frac{\sigma(X \cup Y)}{\sigma(X)} = \frac{s(X \cup Y)}{s(X)} = P(Y \mid X) \tag{11.10}$$

11.3.2 Esempio di Calcolo Analitico Dettagliato

Consideriamo la regola:

$$\{\text{Latte, Pannolini}\} \longrightarrow \{\text{Birra}\} \tag{11.11}$$

Dalle transazioni della Tabella 11.1:

- Antecedente: $X = \{\text{Latte, Pannolini}\}$;
- Conseguente: $Y = \{\text{Birra}\}$;
- Unione congiunta: $X \cup Y = \{\text{Latte, Pannolini, Birra}\}$.

Ispezionando i conteggi di occorrenza nel database:

1. $X \cup Y$ compare nelle transazioni 3 e 4 $\implies \sigma(X \cup Y) = 2$;
2. X compare nelle transazioni 3, 4 e 5 $\implies \sigma(X) = 3$;
3. Numero totale di transazioni: $N = |T| = 5$.

Applicando le formule formali (11.9) e (11.10):

$$s(\{\text{Latte, Pannolini}\} \to \{\text{Birra}\}) = \frac{2}{5} = 0.40 \quad (40\%) \tag{11.12}$$

$$c(\{\text{Latte, Pannolini}\} \to \{\text{Birra}\}) = \frac{2}{3} \approx 0.6667 \quad (66.67\%) \tag{11.13}$$

La regola indica che nel 40% delle transazioni complessive i tre beni compaiono insieme, e che quando un carrello include latte e pannolini, nel 66.7% dei casi include anche birra.

11.4 Il Compito di Mining e la Trattabilità Computazionale

Definizione 11.4.1 (Problema del Mining delle Regole di Associazione)**.** Dato un database transazionale T, una soglia minima di supporto minsup $\in (0, 1]$ e una soglia minima di confidenza minconf $\in (0, 1]$, il problema del mining consiste nell'estrarre l'insieme completo di tutte le regole $X \to Y$ tali che:

$$\begin{cases} s(X \longrightarrow Y) \geq \text{minsup} \\ c(X \longrightarrow Y) \geq \text{minconf} \end{cases} \tag{11.14}$$

11.4.1 L'Approccio Esaustivo (Brute-Force) e l'Esplosione Combinatoria

Una procedura ingenua per risolvere il problema consisterebbe nell'elencare tutte le regole logicamente possibili derivabili dal catalogo I, calcolare per ciascuna di esse supporto e confidenza scansionando l'intero database T, e infine scartare quelle che non superano i vincoli di soglia.

Tale approccio è irrealizzabile per qualsiasi applicazione reale. Dato un catalogo di d articoli distinti, il numero complessivo di possibili regole di associazione non vuote con antecedente e conseguente disgiunti ($X, Y \subset I, X \cap Y = \emptyset, X \neq \emptyset, Y \neq \emptyset$) è dato dalla formula combinatoria:

$$R = \sum_{k=2}^{d} \binom{d}{k} \sum_{j=1}^{k-1} \binom{k}{j} = 3^d - 2^{d+1} + 1 \tag{11.15}$$

Dimostrazione. Ogni articolo $i \in I$ può trovarsi in esattamente uno tra tre stati mutuamente esclusivi rispetto a una regola candidata:

1. Appartenere all'antecedente X;
2. Appartenere al conseguente Y;
3. Non fare parte della regola (essere escluso da $X \cup Y$).

Esistono pertanto 3^d possibili assegnazioni. Da tale totale occorre sottrarre i casi degenere:

- Regole con antecedente vuoto ($X = \emptyset$): l'articolo può essere in Y o non appartenere alla regola (2^d casi);
- Regole con conseguente vuoto ($Y = \emptyset$): l'articolo può essere in X o non appartenere alla regola (2^d casi);
- La configurazione in cui sia X che Y sono contemporaneamente vuoti ($X = \emptyset, Y = \emptyset$) è stata sottratta due volte e va reintegrata (+1 caso).

Sottraendo le configurazioni non valide si ottiene $3^d - 2 \cdot 2^d + 1 = 3^d - 2^{d+1} + 1$. □

Per cataloghi commerciali di modeste dimensioni (ad esempio $d = 100$ articoli), il numero di regole candidate è $3^{100} \approx 5.15 \times 10^{47}$, una quantità astronomica che rende impossibile qualsiasi scansione esaustiva.

11.5 Disaccoppiamento del Problema: L'Approccio a Due Fasi

Per superare l'intrattenibilità dell'approccio esaustivo, la letteratura di data mining adotta un principio fondamentale basato sulla scomposizione funzionale del problema [13, 14, 2].

11.5.1 Proprietà delle Partizioni Binarie di uno Stesso Itemset

Consideriamo tutte le possibili regole derivabili partizionando in due insiemi disgiunti il medesimo 3-itemset frequente $L = \{\text{Latte, Pannolini, Birra}\}$:

Regola Candidata ($X \to Y$)	$\sigma(X \cup Y)$	$\sigma(X)$	**Supporto** s	**Confidenza** c
{Latte, Pannolini} → {Birra}	2	3	$2/5 = 0.40$	$2/3 \approx 0.67$
{Latte, Birra} → {Pannolini}	2	2	$2/5 = 0.40$	$2/2 = 1.00$
{Pannolini, Birra} → {Latte}	2	3	$2/5 = 0.40$	$2/3 \approx 0.67$
{Birra} → {Latte, Pannolini}	2	3	$2/5 = 0.40$	$2/3 \approx 0.67$
{Pannolini} → {Latte, Birra}	2	4	$2/5 = 0.40$	$2/4 = 0.50$
{Latte} → {Pannolini, Birra}	2	4	$2/5 = 0.40$	$2/4 = 0.50$

Tabella 11.2: Valutazione delle regole generate dalle partizioni binarie di {Latte, Pannolini, Birra}.

Dall'ispezione della Tabella 11.2 emergono due proprietà decisive:

1. **Invarianza del Supporto**: Tutte le regole che traggono origine dallo stesso itemset congiunto L presentano esattamente il medesimo supporto, pari al supporto di L:

$$s(X \longrightarrow L \setminus X) = s(L) = \frac{\sigma(L)}{|T|} \tag{11.16}$$

Se l'itemset L non supera la soglia di supporto minsup, nessuna regola da esso derivata potrà mai soddisfare il vincolo di supporto.

2. **Variazione della Confidenza**: La confidenza varia invece in funzione della specifica partizione, poiché il denominatore $\sigma(X)$ dipende unicamente dalla frequenza dell'antecedente X.

11.5.2 Decomposizione in Due Sotto-Problemi Sequenziali

Grazie a tali proprietà, la ricerca delle regole di associazione viene disaccoppiata in due passaggi distinti e sequenziali:

1. **Fase 1: Generazione degli Itemset Frequenti (*Frequent Itemset Generation*)**: Si identificano tutti gli itemset $L \subseteq I$ tali che $s(L) \geq$ minsup.

2. **Fase 2: Generazione delle Regole (*Rule Generation*)**: Per ciascun itemset frequente L scoperto nella Fase 1, si estraggono tutte le partizioni binarie $X \rightarrow L \setminus X$ che soddisfano la soglia di confidenza $c \geq$ minconf.

La Fase 1 rappresenta la componente computazionalmente dominante dell'intera pipeline, in quanto richiede la ricerca in uno spazio esponenziale di combinazioni su grandi moli di dati.

11.6 Reticolo degli Itemset e Complessità di Generazione

Dato un insieme di d elementi, lo spazio combinatorio di tutti i possibili itemset candidati ammette una struttura algebrica e geometrica naturale nota come **reticolo di Boole** (*Boolean lattice*) o reticolo dell'insieme delle parti $\mathcal{P}(I)$ [2]. La cardinalità complessiva dello spazio di ricerca (compreso l'insieme vuoto $\emptyset$) ammonta a:

$$|\mathcal{P}(I)| = 2^d \tag{11.17}$$

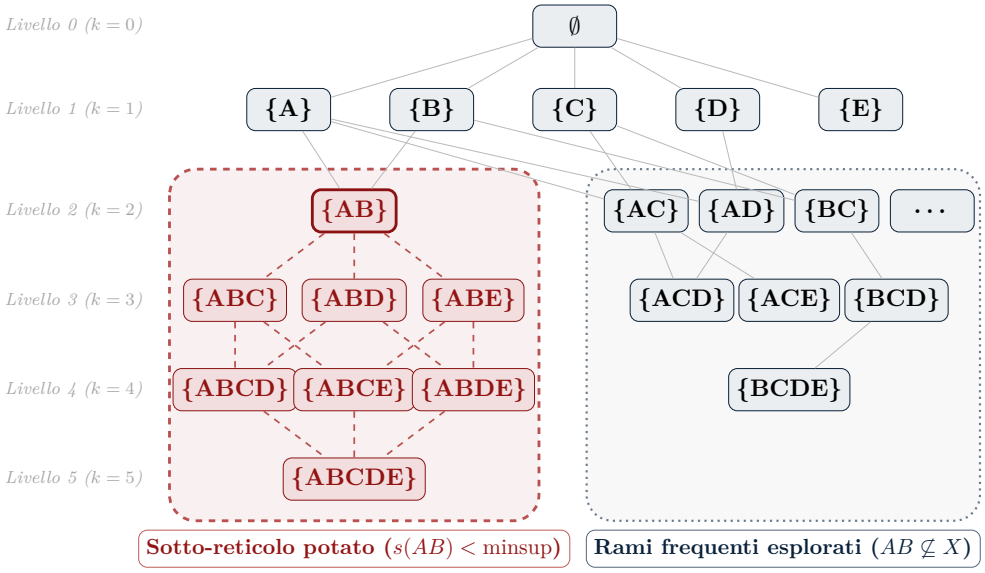


Figura 11.1: Rappresentazione a reticolo dello spazio degli itemset per $d = 5$ elementi. Non appena $\{AB\}$ risulta infrequente ($s(AB) < \text{minsup}$), l'intero sotto-reticolo dei suoi $2^{5-2} = 8$ sovrainsiemi (evidenziato a sinistra in rosso) viene escluso a priori dalla ricerca, mentre i rami rimanenti non contenenti $\{AB\}$ (a destra) continuano a essere esplorati.

11.6.1 Complessità dell'Approccio Esaustivo e Strategie di Ottimizzazione

In un algoritmo privo di potatura, il calcolo del supporto richiederebbe il confronto di ciascuno degli $M = 2^d$ candidati con ciascuna delle N transazioni. Se una transazione contiene mediamente w elementi, il costo computazionale complessivo ammonterebbe a:

$$\mathcal{O}\left(N \times 2^d \times w\right) \tag{11.18}$$

Per rendere trattabile la generazione degli itemset frequenti si adottano tre strategie tra loro ortogonali [2]:

1. **Ridurre il numero di candidati (M)**: Applicare tecniche sistematiche di potatura dello spazio di ricerca (come il principio Apriori) per evitare di generare ed esplorare rami inutili del reticolo;

2. **Ridurre il numero di transazioni (N)**: Scartare progressivamente le transazioni che non contengono itemset frequenti o che presentano una lunghezza inferiore alla taglia k degli itemset sotto esame;

3. **Ridurre il numero di confronti ($N \times M$)**: Impiegare strutture dati avanzate (quali gli alberi hash) per indicizzare transazioni o candidati ed evitare confronti a forza bruta.

11.7 Il Principio Apriori e la Proprietà di Anti-Monotonia

L'algoritmo Apriori, introdotto nel 1994 da Rakesh Agrawal e Ramakrishnan Srikant [14], si fonda su un principio algebrico fondamentale che consente di potare massivamente lo spazio di ricerca.

Teorema 11.7.1 (Principio Apriori)**.** *Se un itemset è frequente, allora tutti i suoi sottoinsiemi devono necessariamente essere frequenti:*

$$s(Y) \geq \text{minsup} \quad \Longrightarrow \quad \forall X \subseteq Y,\ s(X) \geq \text{minsup} \tag{11.19}$$

11.7.1 Dimostrazione dell'Anti-Monotonia del Supporto

Il principio poggia matematicamente sulla proprietà di **anti-monotonia** della funzione di supporto rispetto alla relazione d'inclusione insiemistica [2].

Teorema 11.7.2 (Anti-monotonia del Supporto)**.** *La funzione di supporto è anti-monotona rispetto all'inclusione tra insiemi:*

$$\forall X, Y \subseteq I: \quad (X \subseteq Y) \Longrightarrow s(X) \geq s(Y) \tag{11.20}$$

Dimostrazione. Sia $T(X) = \{t \in T \mid X \subseteq t\}$ l'insieme delle transazioni del database che contengono l'itemset X. Se un itemset Y contiene X ($X \subseteq Y$), qualsiasi transazione t che include Y deve obbligatoriamente contenere anche X. Ne consegue l'inclusione insiemistica:

$$T(Y) \subseteq T(X) \tag{11.21}$$

Per la proprietà di monotonicità della cardinalità per insiemi finiti:

$$|T(Y)| \leq |T(X)| \quad \Longleftrightarrow \quad \sigma(Y) \leq \sigma(X) \tag{11.22}$$

Dividendo entrambi i membri per la costante positiva $N = |T|$, si ottiene:

$$\frac{\sigma(Y)}{|T|} \leq \frac{\sigma(X)}{|T|} \quad \Longleftrightarrow \quad s(Y) \leq s(X) \tag{11.23}$$

Di conseguenza, il supporto non può mai aumentare all'aumentare della cardinalità dell'itemset. □

11.7.2 Potatura Basata sul Supporto (Contrapposizione Logica)

Applicando la regola di contrapposizione logica al Principio Apriori ($P \implies Q \iff \neg Q \implies \neg P$), si ottiene la regola operativa di potatura fondamentale:

Corollario 11.7.1 (Regola di Potatura Apriori)**.** *Se un itemset è non frequente (infrequente), allora tutti i suoi sovrainsiemi sono garantiti essere non frequenti e possono essere potati immediatamente dallo spazio di ricerca:*

$$s(X) < \text{minsup} \quad \implies \quad \forall Y \supseteq X,\ s(Y) \leq s(X) < \text{minsup} \tag{11.24}$$

Come illustrato graficamente nella Figura 11.1, non appena si determina che $\{AB\}$ è infrequente, tutti i candidati $\{ABC\}$, $\{ABD\}$, $\{ABE\}$, $\{ABCD\}$, $\{ABCE\}$, $\{ABDE\}$ e $\{ABCDE\}$ vengono esclusi istantaneamente senza dover effettuare alcuna scansione del database.

11.8 Esecuzione Tracciata Passo-Passo dell'Algoritmo Apriori

Per comprendere la dinamica operativa dell'algoritmo, tracciamo passo dopo passo l'esecuzione sul dataset transazionale di Tabella 11.1 impostando un conteggio minimo assoluto:

$$\sigma_{\min} = 3 \quad \left(\text{minsup} = \frac{3}{5} = 0.60\right) \tag{11.25}$$

11.8.1 Iterazione 1: Calcolo dei 1-Itemset Frequenti ($k = 1$)

Si esegue una scansione completa del database transazionale e si accumulano i conteggi di occorrenza per ciascun singolo articolo:

Articolo (1-itemset)	**Conteggio σ**	**Stato rispetto a $\sigma_{\min} = 3$**
{Pane}	4	Frequente (≥ 3) $\rightarrow$ Mantenuto
{Coca-Cola}	2	Infrequente (< 3) $\rightarrow$ **POTATO**
{Latte}	4	Frequente (≥ 3) $\rightarrow$ Mantenuto
{Birra}	3	Frequente (≥ 3) $\rightarrow$ Mantenuto
{Pannolini}	4	Frequente (≥ 3) $\rightarrow$ Mantenuto
{Uova}	1	Infrequente (< 3) $\rightarrow$ **POTATO**

Tabella 11.3: Calcolo del supporto per i candidati C_1 e identificazione di F_1.

L'insieme dei 1-itemset frequenti è:

$$F_1 = \{\{\text{Pane}\}, \{\text{Latte}\}, \{\text{Birra}\}, \{\text{Pannolini}\}\} \tag{11.26}$$

Gli articoli Coca-Cola e Uova vengono potati permanentemente; nessun candidato delle iterazioni successive conterrà tali elementi.

11.8.2 Iterazione 2: Generazione e Verifica dei 2-Itemset

Si generano tutti i possibili candidati di dimensione 2 combinando a coppie gli elementi di F_1. Il numero di candidati in C_2 è $\binom{4}{2} = 6$. Si effettua la seconda scansione del database per calcolare i conteggi di supporto:

Candidato (2-itemset)	**Conteggio σ**	**Stato rispetto a $\sigma_{\min} = 3$**
{Pane, Latte}	3	Frequente (≥ 3) $\rightarrow$ Mantenuto
{Pane, Birra}	2	Infrequente (< 3) $\rightarrow$ **POTATO**
{Pane, Pannolini}	3	Frequente (≥ 3) $\rightarrow$ Mantenuto
{Latte, Birra}	2	Infrequente (< 3) $\rightarrow$ **POTATO**
{Latte, Pannolini}	3	Frequente (≥ 3) $\rightarrow$ Mantenuto
{Birra, Pannolini}	3	Frequente (≥ 3) $\rightarrow$ Mantenuto

Tabella 11.4: Calcolo del supporto per i candidati C_2 e identificazione di F_2.

L'insieme dei 2-itemset frequenti è:

$$\begin{aligned} F_2 = \{&\{\text{Pane, Latte}\}, \{\text{Pane, Pannolini}\}, \\ &\{\text{Latte, Pannolini}\}, \{\text{Birra, Pannolini}\}\} \end{aligned} \tag{11.27}$$

11.8.3 Iterazione 3: Generazione, Potatura e Verifica dei 3-Itemset ($k = 3$)

1. **Generazione dei Candidati** (C_3): Combinando gli elementi frequenti in F_2, i possibili tripletti candidati sono 4:

$$\begin{aligned} &\{\text{Birra, Pannolini, Latte}\}, \quad \{\text{Birra, Pane, Pannolini}\}, \\ &\{\text{Pane, Pannolini, Latte}\}, \quad \{\text{Birra, Pane, Latte}\} \end{aligned} \tag{11.28}$$

2. **Potatura dei Candidati (*Candidate Pruning*)**: Si verifica se tutti i sottoinsiemi di dimensione 2 di ciascun candidato appartengano a F_2:

   - {Birra, Pannolini, Latte} contiene {Birra, Latte} $\notin F_2 \implies$ **POTATO**;
   - {Birra, Pane, Pannolini} contiene {Birra, Pane} $\notin F_2 \implies$ **POTATO**;
   - {Birra, Pane, Latte} contiene {Birra, Pane} $\notin F_2 \implies$ **POTATO**;
   - {Pane, Pannolini, Latte}: i suoi sottoinsiemi di taglia 2 sono {Pane, Pannolini}, {Pane, Latte} e {Latte, Pannolini}. Poiché appartengono tutti a F_2, il candidato **sopravvive alla potatura**.

   L'insieme dei candidati validi dopo la potatura contiene un solo elemento: $C_3 = \{\{\text{Pane, Pannolini, Latte}\}\}$.

3. **Conteggio del Supporto**: Scansionando il database transazionale:

$$\sigma(\{\text{Pane, Pannolini, Latte}\}) = 2 \tag{11.29}$$

   Poiché $2 < \sigma_{\min} = 3$, anche questo candidato risulta infrequente.

Poiché $F_3 = \emptyset$, non è possibile generare candidati di taglia 4 e l'algoritmo termina l'esplorazione. L'insieme complessivo degli itemset frequenti scoperti è $F = F_1 \cup F_2$.

11.9 Pseudocodice e Tecniche di Generazione dei Candidati

La struttura logica formale dell'algoritmo Apriori è riassunta nello pseudocodice seguente [14, 2].

Algorithm 6: Algoritmo Apriori per l'Estrazione degli Itemset Frequenti

Dati: Database transazionale T, soglia minima di supporto minsup $\in (0, 1]$.

Risultato: Insieme completo di tutti gli itemset frequenti $F = \bigcup_k F_k$.

```
1  k ← 1
2  F_1 ← {i ∈ I | s({i}) ≥ minsup}          // Generazione 1-itemset frequenti
3  repeat
4  |  k ← k + 1
5  |  C_k ← Candidate-Generation(F_{k−1})     // Unione sistematica
6  |  C_k ← Candidate-Pruning(C_k, F_{k−1})   // Potatura sottoinsiemi
7  |  foreach transazione t ∈ T do
8  |  |  C_t ← Subsets(C_k, t)               // Candidati contenuti in t
9  |  |  foreach candidato c ∈ C_t do
10 |  |  |  c.count ← c.count + 1
11 |  |  end
12 |  end
13 |  F_k ← {c ∈ C_k | c.count ≥ ⌈minsup × |T|⌉}
14 until F_k = ∅
15 return ⋃_k F_k
```

11.9.1 Metodo Standard $F_{k-1} \times F_{k-1}$ (Join su Prefisso Comune)

Per generare l'insieme dei candidati C_k evitando duplicati e ridondanze, gli elementi di ogni itemset vengono ordinati lessicograficamente [14]. Due itemset $p, q \in F_{k-1}$ vengono fusi se e solo se condividono il medesimo prefisso di lunghezza $k-2$ e differiscono esclusivamente per l'ultimo elemento:

$$\begin{cases} p[1] = q[1] \\ p[2] = q[2] \\ \quad \vdots \\ p[k-2] = q[k-2] \\ p[k-1] < q[k-1] \end{cases} \tag{11.30}$$

Il candidato generato è l'unione dei due insiemi:

$$c = \{p[1], p[2], \ldots, p[k-2], p[k-1], q[k-1]\} \tag{11.31}$$

Esempio Esecutivo di Prefisso Comune

Dato l'insieme di 3-itemset frequenti:

$$F_3 = \{ABC, ABD, ABE, ACD, BCD, BDE, CDE\} \tag{11.32}$$

Il prefisso da verificare ha lunghezza $k-2 = 3-2 = 1$ (il primo elemento per $k=3$):

- Merge$(ABC, ABD) \rightarrow$ prefisso comune AB, $C < D \implies ABCD$ generato;
- Merge$(ABC, ABE) \rightarrow$ prefisso comune AB, $C < E \implies ABCE$ generato;
- Merge$(ABD, ABE) \rightarrow$ prefisso comune AB, $D < E \implies ABDE$ generato;
- Merge$(ABD, ACD) \rightarrow$ prefissi di lunghezza 2 differenti ($AB \neq AC$) $\implies$ non vengono fusi.

L'insieme dei candidati generati è:

$$C_4 = \{ABCD, ABCE, ABDE\} \tag{11.33}$$

Fase di Potatura dei Candidati (*Candidate Pruning*)

Per ciascun candidato $c \in C_4$, si verifica che tutti i suoi $\binom{k}{k-1} = 4$ sottoinsiemi di dimensione $k-1=3$ appartengano a F_3:

- Per $ABCD$: sottoinsiemi $\{ABC, ABD, ACD, BCD\}$. Tutti appartengono a $F_3 \implies$ **Mantenuto**;
- Per $ABCE$: sottoinsiemi $\{ABC, ABE, ACE, BCE\}$. Gli insiemi ACE e BCE non appartengono a $F_3 \implies$ **POTATO**;
- Per $ABDE$: sottoinsiemi $\{ABD, ABE, ADE, BDE\}$. L'insieme ADE non appartiene a $F_3 \implies$ **POTATO**.

L'insieme dei candidati dopo la potatura si riduce al singolo elemento $C_4 = \{ABCD\}$.

11.9.2 Metodo Alternativo $F_{k-1} \times F_{k-1}$ (Suffisso-Prefisso)

In questa variante alternativa, due itemset $p, q \in F_{k-1}$ vengono fusi se gli ultimi $k-2$ elementi di p coincidono con i primi $k-2$ elementi di q:

$$p[2] = q[1], \quad p[3] = q[2], \quad \ldots, \quad p[k-1] = q[k-2] \tag{11.34}$$

Il candidato generato è $c = \{p[1], p[2], \ldots, p[k-1], q[k-1]\}$. Ad esempio, applicando tale criterio al medesimo insieme F_3:

- Merge$(A\underline{BC}, \underline{BC}D) \to ABCD$;
- Merge$(A\underline{BD}, \underline{BD}E) \to ABDE$;
- Merge$(A\underline{CD}, \underline{CD}E) \to ACDE$;
- Merge$(B\underline{CD}, \underline{CD}E) \to BCDE$.

Durante la successiva fase di potatura dei sottoinsiemi:

- $ABDE$: $ADE \notin F_3 \implies$ potato;
- $ACDE$: $ACE \notin F_3 \implies$ potato;
- $BCDE$: $BCE \notin F_3 \implies$ potato;
- $ABCD$: tutti i sottoinsiemi sono in $F_3 \implies$ mantenuto.

Entrambe le metodologie convergono in modo equivalente all'insieme minimo $\{ABCD\}$.

11.10 Conteggio del Supporto Mediante Strutture Hash Tree

Durante l'esecuzione dell'algoritmo Apriori, la verifica del supporto richiede di controllare quali candidati in C_k siano contenuti all'interno di ciascuna transazione $t \in T$.

11.10.1 Complessità del Confronto Diretto

Per una transazione che contiene $|t|$ elementi, il numero di sottoinsiemi di dimensione k in essa presenti è pari a:

$$\binom{|t|}{k} \tag{11.35}$$

Se ad esempio una transazione contiene $|t| = 20$ articoli e si ricercano candidati di dimensione $k = 3$, la transazione genera $\binom{20}{3} = 1\,140$ possibili combinazioni. Il confronto esaustivo di tali combinazioni contro l'intero catalogo dei candidati comporterebbe un sovraccarico proibitivo [2].

11.10.2 Architettura dell'Hash Tree

Per limitare drasticamente il numero di confronti, i candidati C_k vengono organizzati in una struttura ad albero denominata **Hash Tree** [14, 2]:

- **Nodi Interni**: Contengono una tabella hash con funzione di partizionamento $h(i) = i \pmod B$. Al livello j dell'albero, la ramificazione dipende dall'hash applicato sul j-esimo elemento dell'itemset;
- **Foglie**: Contengono elenchi (*buckets*) di itemset candidati fino a una capacità massima prefissata. Quando una foglia supera la capienza, essa viene trasformata in nodo interno e suddivisa.

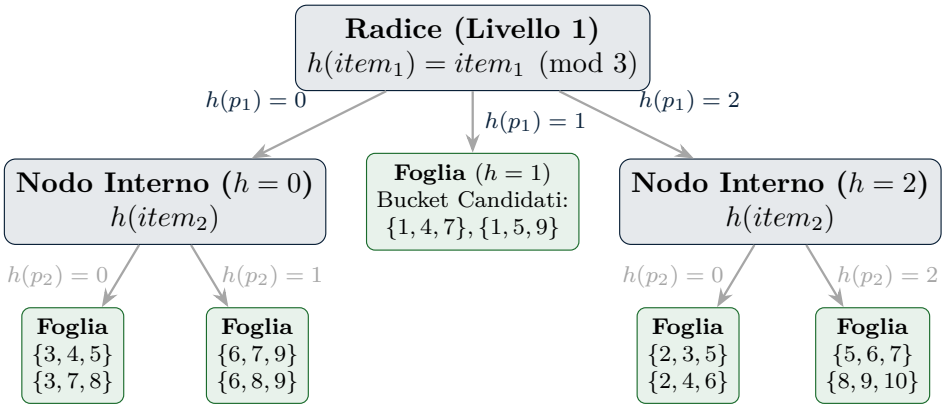


Figura 11.2: Architettura di indicizzazione dei candidati mediante Hash Tree per il conteggio efficiente del supporto.

Durante la lettura di una transazione t, l'algoritmo scende ricorsivamente nei rami dell'albero hash considerando tutte le possibili combinazioni ordinate

dei suoi elementi. Il confronto diretto tra transazione e candidati viene effettuato esclusivamente sui sottoinsiemi memorizzati nelle foglie effettivamente raggiunte, riducendo le operazioni di confronto da esponenziali a logaritmiche rispetto al numero di candidati [2].

11.11 Generazione delle Regole dagli Itemset Frequenti

Completata la Fase 1 con l'individuazione degli itemset frequenti, si procede alla Fase 2: l'estrazione delle regole di associazione ad alta confidenza.

11.11.1 Decomposizione Combinatoria delle Regole

Dato un itemset frequente L di cardinalità $|L| = k$, l'obiettivo consiste nell'estrarre tutti i sottoinsiemi non vuoti $X \subset L$ tali che la regola $X \to L \setminus X$ soddisfi:

$$c(X \longrightarrow L \setminus X) = \frac{\sigma(L)}{\sigma(X)} \geq \text{minconf} \tag{11.36}$$

Escludendo le regole banali in cui antecedente o conseguente sono vuoti, il numero totale di regole candidate derivabili da un singolo k-itemset è dato da:

$$2^k - 2 \tag{11.37}$$

Ad esempio, per un 4-itemset frequente $\{A, B, C, D\}$, si hanno $2^4 - 2 = 14$ possibili regole candidate:

- **1 elemento al conseguente (4 regole)**: $ABC \to D$, $ABD \to C$, $ACD \to B$, $BCD \to A$;
- **2 elementi al conseguente (6 regole)**: $AB \to CD$, $AC \to BD$, $AD \to BC$, $BC \to AD$, $BD \to AC$, $CD \to AB$;
- **3 elementi al conseguente (4 regole)**: $A \to BCD$, $B \to ACD$, $C \to ABD$, $D \to ABC$.

11.11.2 Proprietà di Anti-Monotonia della Confidenza

In generale, la metrica di confidenza non è anti-monotona rispetto alla dimensione dell'antecedente tra regole generate da itemset diversi. Tuttavia, quando si analizzano regole derivate dal **medesimo itemset frequente** L, si verifica una proprietà algebrica decisiva [2].

Teorema 11.11.1 (Anti-monotonia della Confidenza lungo il Reticolo delle Regole)**.** *Date due regole R_1 e R_2 generate dal medesimo itemset frequente L:*

$$R_1 : X \longrightarrow L \setminus X, \qquad R_2 : X' \longrightarrow L \setminus X' \quad \textit{con } X' \subset X \subset L \tag{11.38}$$

vale la disuguaglianza:

$$c(X \longrightarrow L \setminus X) \geq c(X' \longrightarrow L \setminus X') \tag{11.39}$$

Dimostrazione. Calcoliamo esplicitamente le confidenze delle due regole:

$$c(R_1) = \frac{\sigma(L)}{\sigma(X)}, \qquad c(R_2) = \frac{\sigma(L)}{\sigma(X')} \tag{11.40}$$

Poiché $X' \subset X$, per la proprietà di anti-monotonia del supporto (Teorema 11.7.1) si ha:

$$\sigma(X') \geq \sigma(X) \tag{11.41}$$

Invertendo i rapporti con numeratore costante $\sigma(L) > 0$:

$$\frac{\sigma(L)}{\sigma(X')} \leq \frac{\sigma(L)}{\sigma(X)} \quad \Longleftrightarrow \quad c(R_2) \leq c(R_1) \tag{11.42}$$

□

Corollario 11.11.1 (Potatura delle Regole Basata sulla Confidenza)**.** *La confidenza di regole generate dallo stesso itemset è* ***anti-monotona rispetto al numero di elementi nel conseguente*** *(RHS). Se una regola ha confidenza inferiore alla soglia* minconf*, qualsiasi regola derivata dallo stesso itemset che sposti ulteriori elementi dall'antecedente al conseguente avrà una confidenza ancora minore e può essere potata immediatamente:*

$$c(X \longrightarrow Y) < \text{minconf} \quad \Longrightarrow \quad \forall Y' \supset Y,\ c(L \setminus Y' \longrightarrow Y') < \text{minconf} \tag{11.43}$$

Ad esempio, se per $\{A, B, C, D\}$ si ha che $c(BCD \to A) <$ minconf, allora la regola $CD \to AB$ e la regola $D \to ABC$ sono garantite avere confidenza inferiore a minconf e non necessitano di verifica.

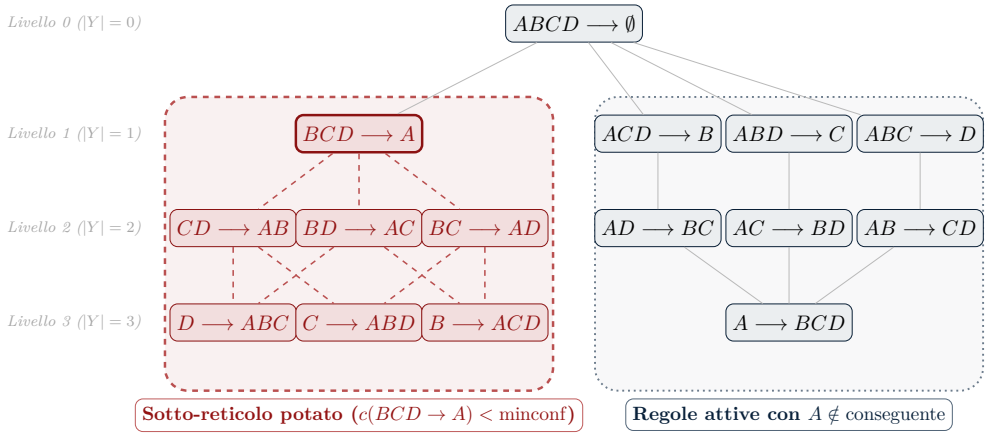


Figura 11.3: Reticolo delle regole generate da $\{A, B, C, D\}$ strutturato per cardinalità del conseguente ($|Y|$). Non appena la regola $BCD \to A$ fallisce la soglia minconf, tutti i suoi discendenti (aventi A nel conseguente, evidenziati a sinistra) vengono potati a priori per anti-monotonia della confidenza.

11.12 Fattori che Influenzano la Complessità Computazionale

La scalabilità e i tempi di elaborazione dell'algoritmo Apriori dipendono dall'interazione di quattro parametri strutturali del problema [2]:

1. **Soglia Minima di Supporto** (minsup): Rappresenta il fattore di controllo più sensibile. Al decrescere di minsup, il numero di itemset frequenti cresce in modo esponenziale. Ciò determina un incremento della lunghezza massima $k_{\max}$ dei pattern frequenti, moltiplica il numero di candidati intermedi C_k generati e impone un maggior numero di passate sequenziali sul database.

2. **Dimensionalità del Catalogo** ($d = |I|$): Un elevato numero di articoli distinti espande la dimensione dello spazio di memoria richiesto per tracciare i conteggi. In particolare, il numero potenziale di coppie al secondo livello cresce quadraticamente come $\binom{d}{2} = \mathcal{O}(d^2)$.

3. **Dimensione del Database** ($N = |T|$): Poiché ogni iterazione k comporta una scansione sequenziale dell'archivio per il conteggio del supporto, il costo computazionale complessivo scala linearmente rispetto al numero di transazioni N.

4. **Ampiezza Media delle Transazioni (w)**: All'aumentare del numero medio di articoli contenuti in ciascun carrello, cresce la dimensione massima dei pattern estraibili. Inoltre, il numero di sottoinsiemi di dimensione k che una transazione genera, dato da $\binom{w}{k}$, esplode combinatoriamente con w, appesantendo l'attraversamento degli alberi hash.

11.13 Rappresentazioni Compatte: Itemset Frequenti Massimali e Chiusi

Quando un dataset presenta transazioni dense o pattern frequenti estesi (ad esempio un pattern frequente di cardinalità 20), il numero di sottoinsiemi frequenti generati per anti-monotonia ammonta a $2^{20} - 1 \approx 10^6$ itemset. Per contrastare questa proliferazione esponenziale senza perdere informazione, la ricerca ha introdotto due rappresentazioni compatte fondamentali: gli **itemset massimali** e gli **itemset chiusi** [2].

11.13.1 Itemset Frequenti Massimali (*Maximal Frequent Itemsets*)

Definizione 11.13.1 (Itemset Frequente Massimale)**.** Un itemset X è definito **frequente massimale** se soddisfa la soglia di supporto ($s(X) \geq \text{minsup}$) e **nessuno dei suoi sovrainsiemi diretti** (immediati) è frequente:

$$X \text{ massimale frequente} \iff s(X) \geq \text{minsup} \ \wedge \ \forall Y \supset X,\ s(Y) < \text{minsup} \tag{11.44}$$

- **Proprietà Geometrica**: Gli itemset massimali formano il "bordo" (*border*) che separa gli itemset frequenti da quelli infrequenti nel reticolo;
- **Limite Informativo (Compressione con Perdita)**: Conoscere l'insieme degli itemset massimali consente di determinare con certezza quali siano tutti gli itemset frequenti (essendo frequenti tutti i loro sottoinsiemi), ma **non permette di recuperare i valori di supporto esatti** dei singoli sottoinsiemi. Si tratta pertanto di una rappresentazione a compressione con perdita (*lossy compression*).

11.13.2 Itemset Chiusi e Chiusi Frequenti (*Closed Frequent Itemsets*)

Definizione 11.13.2 (Itemset Chiuso)**.** Un itemset X è definito **chiuso** se nessuno dei suoi sovrainsiemi diretti possiede esattamente il medesimo

conteggio di supporto di X:

$$X \text{ è chiuso} \iff \forall Y \supset X,\ \sigma(Y) < \sigma(X) \tag{11.45}$$

Equivalentemente, X non è chiuso se esiste almeno un sovrainsieme immediato $Y \supset X$ tale che $\sigma(Y) = \sigma(X)$.

Definizione 11.13.3 (Itemset Chiuso Frequente)**.** Un itemset è **chiuso frequente** se è simultaneamente chiuso e frequente ($s(X) \geq \text{minsup}$).

Teorema 11.13.1 (Compressione Senza Perdita degli Itemset Chiusi Frequenti)**.** *L'insieme degli itemset chiusi frequenti costituisce una* ***rappresentazione compatta senza perdita di informazione*** *(lossless compression) dell'intero spazio degli itemset frequenti. Il conteggio di supporto esatto di qualsiasi sottoinsieme frequente arbitrario S è ricavabile analiticamente senza rileggere il database mediante la formula:*

$$\sigma(S) = \max\left\{\sigma(C) \mid C \text{ è chiuso frequente e } S \subseteq C\right\} \tag{11.46}$$

11.13.3 Studio di Caso Analitico su Dataset Tabellare

Consideriamo il database di transazioni riportato in Tabella 11.5:

TID	Transazione
1	$\{A, B\}$
2	$\{B, C, D\}$
3	$\{A, B, C, D\}$
4	$\{A, B, D\}$
5	$\{A, B, C, D\}$

Tabella 11.5: Dataset transazionale per la diagnosi di itemset chiusi e massimali.

Calcoliamo i supporti e analizziamo la chiusura dei pattern:

1. Itemset $\{A\}$: compare in TID 1, 3, 4, 5 $\implies$ $\sigma(\{A\}) = 4$. Il suo sovrainsieme immediato $\{A, B\}$ compare anch'esso in TID 1, 3, 4, 5 $\implies$ $\sigma(\{A, B\}) = 4$. Poiché $\sigma(\{A\}) = \sigma(\{A, B\})$, $\{A\}$ **NON è chiuso**;

2. Itemset $\{B\}$: compare in tutte e 5 le transazioni $\implies$ $\sigma(\{B\}) = 5$. I suoi sovrainsiemi immediati hanno supporti $\sigma(\{A, B\}) = 4$, $\sigma(\{B, C\}) = 3$, $\sigma(\{B, D\}) = 4$. Poiché nessun sovrainsieme raggiunge supporto 5, $\{B\}$ **è CHIUSO**;

3. Itemset $\{A, B\}$: $\sigma(\{A, B\}) = 4$. I suoi sovrainsiemi immediati hanno supporti $\sigma(\{A, B, C\}) = 2$ e $\sigma(\{A, B, D\}) = 3$. Poiché entrambi hanno supporto strettamente inferiore a 4, $\{A, B\}$ **è CHIUSO**;

4. Itemset $\{C\}$: $\sigma(\{C\}) = 3$. Il sovrainsieme $\{B, C\}$ ha $\sigma(\{B, C\}) = 3$. Avendo lo stesso supporto del sovrainsieme, $\{C\}$ **NON è chiuso**;

5. Itemset $\{A, B, C, D\}$: $\sigma(\{A, B, C, D\}) = 2$. Non esistono sovrainsiemi con supporto ≥ 2, quindi $\{A, B, C, D\}$ **è CHIUSO**.

11.13.4 Relazioni di Inclusione Insiemistica

La gerarchia tra le diverse classi di itemset è sintetizzata nella Figura 11.4 e soddisfa le seguenti relazioni di inclusione insiemistica:

$$\text{Massimali Frequenti} \subseteq \text{Chiusi Frequenti} \subseteq \text{Itemset Frequenti} \tag{11.47}$$

$$\text{Chiusi Frequenti} = \text{Itemset Frequenti} \cap \text{Itemset Chiusi} \tag{11.48}$$

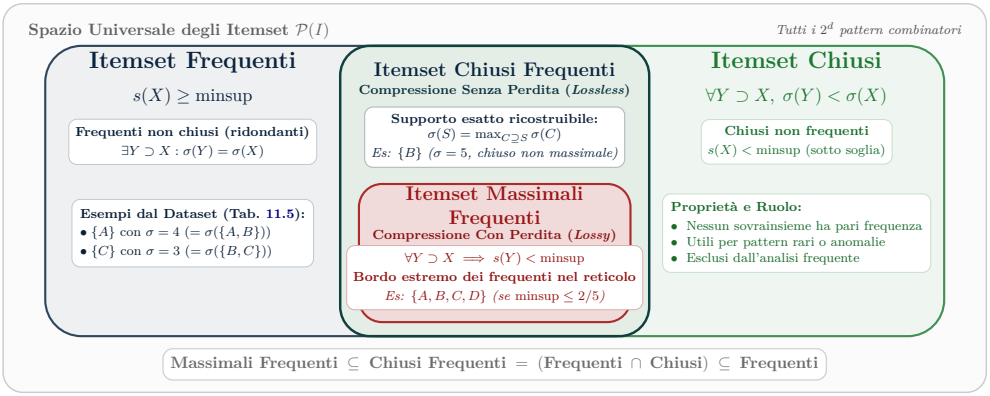


Figura 11.4: Diagramma di Eulero-Venn delle relazioni formali di inclusione tra itemset frequenti, chiusi e massimali, con esemplificazione della proprietà di compressione (*lossless* vs *lossy*).

Ogni itemset massimale frequente è necessariamente un itemset chiuso frequente, mentre l'inverso non è sempre verificato.

11.14 Valutazione dei Pattern e Limiti della Confidenza

Sebbene il framework supporto-confidenza sia storicamente il più diffuso, la metrica di confidenza soffre di gravi limiti interpretativi poiché **ignora completamente il supporto marginale del conseguente** $P(Y)$ [2].

11.14.1 Tabelle di Contingenza 2×2

La relazione statistica tra due itemset X e Y viene sintetizzata tramite una tabella di contingenza bivariata 2×2:

	Y	$\overline{Y}$ (Assenza di Y)	**Totale Marginale**
X	$f_{11} = \sigma(X \cup Y)$	$f_{10} = \sigma(X \cup \overline{Y})$	$f_{1+} = \sigma(X)$
$\overline{X}$ (Assenza di X)	$f_{01} = \sigma(\overline{X} \cup Y)$	$f_{00} = \sigma(\overline{X} \cup \overline{Y})$	$f_{0+} = \sigma(\overline{X})$
Totale Marginale	$f_{+1} = \sigma(Y)$	$f_{+0} = \sigma(\overline{Y})$	$N = \lvert T \rvert$

Tabella 11.6: Tabella di contingenza 2×2 per l'analisi di associazione congiunta.

In termini di frequenze di cella:

$$s(X \longrightarrow Y) = \frac{f_{11}}{N}, \qquad c(X \longrightarrow Y) = \frac{f_{11}}{f_{1+}} \tag{11.49}$$

11.14.2 Caso di Studio 1: Regola con Alta Confidenza ma Correlazione Negativa

Consideriamo l'analisi delle preferenze su $N = 1\,000$ consumatori di bevande calde (Tè e Caffè):

	Caffè	$\overline{\text{Caffè}}$	**Totale**
Tè	150	50	200
$\overline{\text{Tè}}$	650	150	800
Totale	800	200	1 000

Tabella 11.7: Tabella di contingenza tra consumo di Tè e Caffè.

Valutiamo la regola d'associazione Tè $\longrightarrow$ Caffè:

- Confidenza:

$$c(\text{Tè} \longrightarrow \text{Caffè}) = P(\text{Caffè} \mid \text{Tè}) = \frac{150}{200} = 0.75 \quad (75\%) \tag{11.50}$$

  Con una confidenza del 75%, la regola apparirebbe solida e affidabile.

- Supporto marginale della popolazione di consumatori di caffè:

$$P(\text{Caffè}) = \frac{800}{1\,000} = 0.80 \quad (80\%) \tag{11.51}$$

- Probabilità di bere caffè tra chi non consuma tè:

$$P(\text{Caffè} \mid \overline{\text{Tè}}) = \frac{650}{800} = 0.8125 \quad (81.25\%) \tag{11.52}$$

Interpretazione Critica del Paradosso

Nella popolazione complessiva, la propensione ad acquistare caffè è dell'80%. Quando un cliente consuma tè, tale propensione scende al 75%. Sapere che una persona beve tè **diminuisce** la probabilità che consumi caffè.
Pertanto, la regola suggerisce un'associazione positiva che contraddice la realtà: Tè e Caffè sono correlati **negativamente** (beni parzialmente sostituti).

11.14.3 Caso di Studio 2: Bassa Confidenza ma Forte Associazione Positiva

Consideriamo il consumo congiunto di Tè e Miele su $N = 1\,000$ clienti:

	Miele	$\overline{\text{Miele}}$	**Totale**
Tè	100	100	200
$\overline{\text{Tè}}$	20	780	800
Totale	120	880	1 000

Tabella 11.8: Tabella di contingenza tra consumo di Tè e Miele.

Valutiamo la regola Tè $\longrightarrow$ Miele:

$$c(\text{Tè} \longrightarrow \text{Miele}) = \frac{100}{200} = 0.50 \quad (50\%) \tag{11.53}$$

Se la soglia dell'analista è impostata a minconf $= 0.60$, questa regola viene scartata. Tuttavia, il consumo medio di miele nella popolazione generale è di appena:

$$P(\text{Miele}) = \frac{120}{1\,000} = 0.12 \quad (12\%) \tag{11.54}$$

Tra chi consuma tè, la propensione all'acquisto di miele passa dal 12% al 50% (un incremento di oltre quattro volte). L'associazione è fortemente positiva, ma viene occultata da una soglia fissa di confidenza.

11.14.4 Condizione di Dipendenza Statistica

Per discriminare associazioni genuine da artefatti statistici, una regola deve soddisfare la condizione:

$$c(X \longrightarrow Y) > s(Y) \quad \Longleftrightarrow \quad P(Y \mid X) > P(Y) \tag{11.55}$$

- Se $P(Y \mid X) = P(Y)$, gli eventi sono statisticamente indipendenti ($P(X \cap Y) = P(X)P(Y)$);
- Se $P(Y \mid X) > P(Y)$, gli eventi sono positivamente associati;
- Se $P(Y \mid X) < P(Y)$, gli eventi sono negativamente associati.

11.15 Misure Statistiche di Interesse Oggettive

Per affiancare al supporto e alla confidenza metriche capaci di misurare la deviazione reale dall'indipendenza stocastica, sono state introdotte misure statistiche oggettive basate sulla tabella di contingenza [2, 15].

11.15.1 Metriche di Lift e Interest

La misura di **Lift** (o **Interest**) quantifica il rapporto tra la probabilità congiunta osservata e quella attesa nell'ipotesi di indipendenza statistica tra antecedente e conseguente:

$$\text{Lift}(X \longrightarrow Y) = \frac{P(Y \mid X)}{P(Y)} = \frac{c(X \longrightarrow Y)}{s(Y)} = \frac{P(X,Y)}{P(X)P(Y)} = \frac{N f_{11}}{f_{1+} f_{+1}} \tag{11.56}$$

- Lift $= 1$: X e Y sono statisticamente indipendenti; la presenza di X non fornisce informazione sulla presenza di Y;

- Lift > 1: X e Y sono positivamente correlati; la presenza di X incrementa la probabilità di osservare Y;
- Lift < 1: X e Y sono negativamente correlati; la presenza di X deprime la probabilità di osservare Y.

Verifichiamo la metrica di Lift sul caso Tè e Caffè (Tabella 11.7):

$$\text{Lift}(\text{Tè} \longrightarrow \text{Caffè}) = \frac{0.15}{0.20 \times 0.80} = \frac{0.15}{0.16} = 0.9375 < 1 \tag{11.57}$$

Il valore inferiore a 1 rileva correttamente che Tè e Caffè sono correlati negativamente, correggendo l'illusione ottica della confidenza al 75%.

11.15.2 Misura di Piatetsky-Shapiro (PS) e Correlazione (ϕ)

- **Misura di Piatetsky-Shapiro (PS o Leverage)**: Misura la differenza algebrica diretta tra la probabilità congiunta osservata e quella attesa sotto indipendenza [15]:

$$PS = P(X,Y) - P(X)P(Y) = \frac{f_{11}}{N} - \frac{f_{1+}f_{+1}}{N^2} \tag{11.58}$$

- **Coefficiente di Correlazione di Pearson (ϕ)**: Estende la correlazione classica di Pearson a coppie di variabili binarie asimmetriche:

$$\phi = \frac{P(X,Y) - P(X)P(Y)}{\sqrt{P(X)(1-P(X))P(Y)(1-P(Y))}} = \frac{Nf_{11} - f_{1+}f_{+1}}{\sqrt{f_{1+}f_{+1}f_{0+}f_{+0}}} \tag{11.59}$$

  Assume valori nell'intervallo $[-1, +1]$, dove 0 denota indipendenza.

11.15.3 Tassonomia Comparativa delle Misure di Associazione

Nella Tabella 11.9 sono riassunte le principali metriche di associazione presenti nella letteratura di data mining [2].

Misura	Simbolo	Formulazione su Tabella di Contingenza 2×2
Correlazione (ϕ)	ϕ	$\frac{Nf_{11} - f_{1+}f_{+1}}{\sqrt{f_{1+}f_{+1}f_{0+}f_{+0}}}$
Odds Ratio	α	$\frac{f_{11}f_{00}}{f_{10}f_{01}}$
Kappa di Cohen	κ	$\frac{Nf_{11} + Nf_{00} - f_{1+}f_{+1} - f_{0+}f_{+0}}{N^2 - f_{1+}f_{+1} - f_{0+}f_{+0}}$
Interest / Lift	I	$\frac{Nf_{11}}{f_{1+}f_{+1}}$
Cosine / IS	IS	$\frac{f_{11}}{\sqrt{f_{1+}f_{+1}}}$
Piatetsky-Shapiro	PS	$\frac{f_{11}}{N} - \frac{f_{1+}f_{+1}}{N^2}$
Jaccard	J	$\frac{f_{11}}{f_{1+} + f_{+1} - f_{11}}$
All-Confidence	h	$\min\left[\frac{f_{11}}{f_{1+}}, \frac{f_{11}}{f_{+1}}\right]$

Tabella 11.9: Formulazione analitica delle principali metriche di interesse statistico oggettive.

11.16 Trattamento di Attributi Categorici e Continui

I database delle transazioni reali non sono composti unicamente da panieri di articoli booleani simmetrici, ma contengono attributi categorici politomici (a più categorie) e attributi quantitativi continui [2]. Per integrare tali variabili nell'analisi delle associazioni, si applicano le metodologie di binarizzazione e discretizzazione introdotte nel Capitolo 4 (sezione 4.6), tenendo conto dei vincoli specifici del framework Apriori.

11.16.1 Attributi Categorici Politomici

Per includere variabili discrete (come il genere o il titolo di studio), si applica la codifica *One-Hot* (sezione 4.6):

- Si genera un elemento booleano distinto per ciascuna combinazione attributo-valore (ad esempio `Titolo = Laurea`);
- Ogni record diventa un vettore binario asimmetrico in cui l'assenza (0) è non informativa e non viene conteggiata come co-occorrenza.

La Criticità degli Attributi ad Alta Cardinalità

Quando un attributo categorico presenta un numero elevato di valori distinti (ad esempio la città o lo stato di residenza del cliente con centinaia di categorie), il supporto medio associato a ciascun singolo valore crolla verso valori infinitesimali.

- **Conseguenza**: Con una soglia minsup standard, quasi nessun valore sopravvive al filtraggio, impedendo la scoperta di pattern;
- **Soluzione Operativa**: Si applica una **aggregazione concettuale preventiva**, raggruppando categorie a basso supporto in macro-classi (ad esempio aggregando i singoli comuni in regioni geografiche).

11.16.2 Attributi Quantitativi Continui

Per variabili continue (come l'età, il reddito annuale o le ore trascorse online), si adottano tre paradigmi:

1. **Metodi Basati su Discretizzazione (*Discretization-based*)**: Il dominio continuo viene partizionato mediante intervalli discreti (ad ampiezza costante o frequenza costante, cfr. sezione 4.6). Ciascun intervallo viene trattato come un item categorico distinto:

$$\{\text{Genere} = \text{M}, \text{Età} \in [21, 30)\} \longrightarrow \{\text{Ore Online} \geq 10\} \tag{11.60}$$

2. **Metodi Basati su Statistiche Riassuntive (*Statistics-based*)**: L'antecedente è composto da item discreti, mentre il conseguente modella la distribuzione della variabile continua riportando media μ e deviazione standard σ:

$$\{\text{Età} \in [21, 30), \text{Chat} = \text{Sì}\} \longrightarrow \text{Ore: } [\mu = 14,\ \sigma = 4] \tag{11.61}$$

3. **Metodi Senza Discretizzazione (*minApriori / Fuzzy*)**: Algoritmi che estendono direttamente la definizione di supporto normalizzando gli attributi continui nell'intervallo $[0, 1]$ e sostituendo l'intersezione booleana con operatori fuzzy (t-norme).

11.17 Gerarchie Concettuali e Regole Multi-Livello

Nei cataloghi commerciali reali, i prodotti sono naturalmente strutturati secondo una gerarchia o tassonomia concettuale (rappresentata mediante un

albero o un grafo aciclico diretto), dove gli articoli specifici posti alle foglie sono progressivamente aggregati in categorie di livello superiore [2].

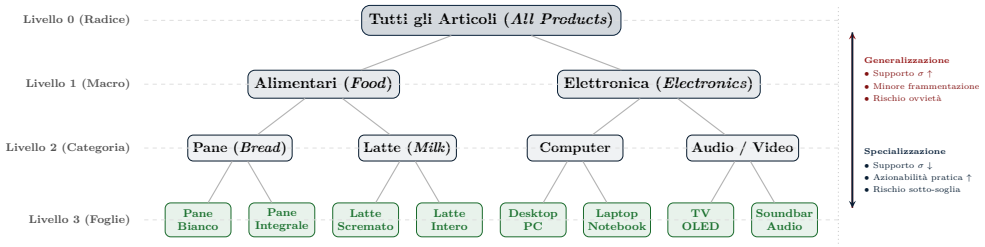


Figura 11.5: Tassonomia concettuale multi-livello per l'estrazione di regole gerarchiche, con evidenziazione dei livelli e del trade-off tra generalizzazione e specializzazione.

11.17.1 Motivazioni per l'Estrazione Multi-Livello

1. **Supporto Insufficiente ai Livelli Foglia**: Articoli estremamente specifici (come una specifica marca di latte scremato biologico) hanno vendite unitarie modeste che non superano la soglia minima minsup, venendo ingiustamente scartati;

2. **Eccessiva Frammentazione**: L'analisi al livello delle foglie genera decine di regole ridondanti tra marche affini, oscurando la tendenza generale aggregata;

3. **Eccessiva Genericità ai Livelli di Vertice**: Regole estratte tra macro-categorie apicali (come Alimentari $\rightarrow$ Elettronica) risultano ovvie, banali o prive di utilità decisionale.

11.17.2 Comportamento di Supporto e Confidenza lungo la Tassonomia

Siano X_1 e X_2 due articoli appartenenti a un livello inferiore che riconducono allo stesso nodo genitore X al livello superiore. Sussistono le seguenti proprietà matematiche:

1. **Sub-additività del Supporto del Genitore**: Il conteggio delle transazioni contenenti il genitore X è vincolato superiormente dalla somma dei conteggi dei figli:

$$\sigma(X) \leq \sigma(X_1) + \sigma(X_2) \tag{11.62}$$

Il segno di disuguaglianza deriva dal fatto che una stessa transazione può contenere simultaneamente sia X_1 sia X_2.

2. **Propagazione Ascendente del Supporto**: Se un itemset di livello inferiore è frequente rispetto a minsup, l'itemset ottenuto sostituendo uno o più elementi con i rispettivi genitori è anch'esso garantito essere frequente:

$$\sigma(X_1 \cup Y_1) \geq \text{minsup} \times |T| \quad \Longrightarrow \quad \sigma(X \cup Y) \geq \text{minsup} \times |T| \tag{11.63}$$

3. **Propagazione della Confidenza**: Se una regola specifica soddisfa la confidenza minima, la regola corrispondente che generalizza il conseguente soddisfa anch'essa il vincolo:

$$c(X_1 \longrightarrow Y_1) \geq \text{minconf} \quad \Longrightarrow \quad c(X_1 \longrightarrow Y) \geq \text{minconf} \tag{11.64}$$

11.17.3 Strategie Implementative per Regole Multi-Livello

La letteratura individua due strategie operative per estrarre regole lungo la gerarchia:

- **Approccio 1: Estensione Transazionale (*Transaction Augmentation*)**: Ciascuna transazione viene arricchita aggiungendo esplicitamente tutti i nodi antenati della tassonomia:

$$\begin{aligned} &\{\text{Latte Scremato, Pane Bianco}\} \longrightarrow \\ &\quad \{\text{Latte Scremato, Pane Bianco, Latte, Pane, Alimentari}\} \end{aligned} \tag{11.65}$$

  Criticità: Aumenta drasticamente l'ampiezza delle transazioni e la dimensione del catalogo, portando a una proliferazione di regole banali o ridondanti (ad esempio Latte Scremato → Latte).

- **Approccio 2: Esplorazione Stratificata Top-Down (*Level-by-Level Generation*)**: Si estraggono dapprima gli itemset frequenti al livello apicale della gerarchia; successivamente, la ricerca ai livelli inferiori viene ristretta unicamente ai discendenti dei nodi risultati frequenti al livello superiore. *Criticità*: Richiede passate multiple sul database per ciascun livello e non intercetta direttamente associazioni trasversali (*cross-level*) tra livelli diversi senza algoritmi dedicati.

Capitolo 12

Sequential Pattern Mining: Modelli, Definizioni ed Esercizi

12.1 Introduzione e Motivazioni al Sequential Pattern Mining

Nel data mining classico, l'estrazione di pattern frequenti (*Frequent Pattern Mining*) e l'analisi del carrello della spesa (*Market-Basket Analysis*) mirano a identificare insiemi di elementi (*itemset*) che co-occorrono all'interno della medesima transazione [2, 14]. Tuttavia, nella maggior parte dei sistemi informativi e delle applicazioni reali, le osservazioni empiriche non si presentano come transazioni isolate e atemporali, bensì come serie ordinate di eventi distribuiti lungo un asse temporale continuo o discreto [16, 17].

Il **Sequential Pattern Mining** (estrazione di pattern sequenziali) costituisce l'estensione metodologica fondamentale volta a colmare questo divario: esso cattura le relazioni d'ordine, di causalità e di dipendenza temporale tra insiemi di eventi, formalizzando la nozione di sequenza temporale di transazioni [2, 16].

Nell'analisi delle associazioni tradizionale, ciascuna riga del database descrive un evento di consumo o una registrazione indipendente, e l'ordine degli articoli all'interno del paniere non riveste alcun significato semantico né algoritmico [2]. Nella realtà applicativa, tuttavia, gli utenti, i sistemi hardware e i processi aziendali interagiscono in modo dinamico e cumulativo nel tempo.

12.1.1 Esempi Reali di Dati Sequenziali

La pervasività dei dati strutturati in sequenza emerge con chiarezza in molteplici contesti applicativi eterogenei:

1. **Transazioni E-commerce di un Consumatore**: Si consideri la cronologia di acquisti effettuati da un cliente nel corso di diversi mesi:

$$\langle \{\text{Digital Camera, iPad}\}\ \{\text{memory card}\}\ \{\text{headphone, iPad cover}\} \rangle \tag{12.1}$$

   In questa sequenza:

   - Al tempo iniziale t_1, il consumatore acquista contestualmente due beni tecnologici ad alto valore: {Digital Camera, iPad}.
   - In un secondo momento t_2, acquista un accessorio funzionale compatibile con la fotocamera: {memory card}.
   - Successivamente a t_3, completa il ciclo di consumo dotandosi di accessori protettivi e multimediali: {headphone, iPad cover}.

   Questo scenario evidenzia sia la presenza di elementi multipli acquistati nello stesso istante, sia la dipendenza temporale differita tra gli acquisti successivi.

2. **Sequenza degli Eventi Incidentali a Three Mile Island (1979)**: Nei sistemi critici di monitoraggio industriale e di diagnostica dei guasti (*fault diagnosis*), gli eventi non corrispondono ad acquisti volontari ma ad allarmi strumentali e anomalie fisiche:

$$\begin{aligned}\langle &\{\text{clogged resin}\}\ \{\text{outlet valve closure}\} \\ &\{\text{loss of feedwater}\}\ \{\text{condenser polisher valve shut}\} \\ &\{\text{booster pumps trip}\}\ \{\text{main waterpump trips}\} \\ &\{\text{main turbine trips}\}\ \{\text{reactor pressure increases}\}\rangle\end{aligned} \tag{12.2}$$

   L'incidente nucleare di Three Mile Island si è sviluppato attraverso una serie a cascata di anomalie meccaniche e termoidrauliche (blocco delle resine scambiatrici, chiusura delle valvole di mandata, disattivazione delle pompe, stacco della turbina e progressivo innalzamento critico della pressione). L'estrazione di pattern sequenziali da log di telemetria consente la manutenzione predittiva e l'attivazione tempestiva di contromisure di sicurezza prima che l'escalation raggiunga esiti catastrofici.

3. **Prestiti Librari in Biblioteca**: Si consideri la sequenza di prestiti effettuati da un lettore:

$$\langle \{\text{Fellowship of the Ring}\}\ \{\text{The Two Towers}\}\ \{\text{Return of the King}\} \rangle \tag{12.3}$$

In questo caso l'ordine è imposto da vincoli narrativi e di logica sequenziale: il lettore prende in prestito i capitoli della trilogia nell'esatto ordine cronologico dell'opera.

Distinzione Fondamentale: Co-occorrenza Istantanea vs Dipendenza Temporale

Mentre le regole di associazione tradizionali (Capitolo 11) identificano articoli acquistati contemporaneamente nello stesso carrello (ad esempio, {Pannolini} $\rightarrow$ {Birra}), il **Sequential Pattern Mining** rileva implicazioni cronologiche differite nel tempo:

$$\langle \{\text{Hardware}\} \rangle \xrightarrow{\Delta t} \langle \{\text{Accessori}\} \rangle \xrightarrow{\Delta t'} \langle \{\text{Ricambi/Upgrade}\} \rangle$$

12.1.2 Domini Applicativi del Sequential Pattern Mining

L'analisi sequenziale trova impiego strategico in molteplici settori:

- **Retail e Market-Basket Dinamica**: Analisi dell'evoluzione delle abitudini di spesa, misurazione del ciclo di vita del cliente (*Customer Lifetime Value*) e prevenzione del tasso di abbandono (*churn*).
- **Web Server Logs e Clickstream Analysis**: Tracciamento dei percorsi di navigazione degli utenti web per ottimizzare l'architettura dell'informazione, personalizzare il layout e identificare i punti critici di abbandono della procedura d'acquisto.
- **Social Media e Diffusione delle Notizie**: Modellazione della catena di propagazione dei contenuti, delle ricondivisioni (*retweet*) e delle dinamiche virali.
- **Process Mining e Workflow di Produzione**: Monitoraggio degli stati di avanzamento nelle catene di montaggio industriali e nei sistemi informativi gestionali (ERP), identificando colli di bottiglia e deviazioni dai percorsi standard.

- **Bioinformatica e Genomica**: Riconoscimento di motivi conservati (*motifs*) e sequenze regolatorie omologhe all'interno di filamenti primari di DNA, RNA o catene polipeptidiche.

12.2 Dal Paradigma Transazionale al Paradigma Sequenziale

Per comprendere a fondo la necessità degli algoritmi sequenziali, è utile richiamare i limiti intrinseci dell'approccio a itemset frequenti quando applicato a collezioni di transazioni distribuite nel tempo [2].

12.2.1 Richiamo: Frequent Itemsets e Market-Basket Analysis

Nel framework classico (Capitolo 11), sia dato un insieme finito di elementi $I = \{i_1, i_2, \ldots, i_m\}$ e una collezione di transazioni $D = \{T_1, T_2, \ldots, T_N\}$, in cui ogni $T_j \subseteq I$. Il supporto di un generico itemset $X \subseteq I$ è definito come la frazione di transazioni che lo contengono:

$$\sigma(X) = \frac{\left|\{T_j \in D \mid X \subseteq T_j\}\right|}{|D|} \tag{12.4}$$

Un itemset è ritenuto frequente se $\sigma(X) \geq \text{minsup}$ (oppure se la sua frequenza assoluta raggiunge la soglia intera Min_freq). In questo modello, le transazioni sono trattate come eventi indipendenti e prive di coordinate temporali.

Esempio 12.2.1 (Valutazione delle Frequenze su Database a 7 Transazioni)**.** Si consideri il database transazionale composto da $N = 7$ scontrini riportato in Tabella 12.1, con soglia di frequenza minima assoluta Min_freq $= 5$ (equivalente a minsup $= 5/7 \approx 71.4\%$).

TID	Bistecca	Peperone	Pollo	Pesce	Prosciutto	Pane	Salsicce
1	1	1	1	1	0	0	0
2	1	1	1	1	0	0	1
3	1	1	1	1	1	1	1
4	1	1	1	1	1	1	0
5	0	0	1	1	1	1	0
6	0	0	1	1	1	1	0
7	1	0	0	0	1	0	0

Tabella 12.1: Database transazionale binario giocattolo ($N = 7$ transazioni).

Valutando la frequenza assoluta di ciascun singolo articolo:

- **Bistecca**: compare in $\{1, 2, 3, 4, 7\} \implies \text{freq} = 5 \geq 5$ **(Frequente)**;
- **Peperone**: compare in $\{1, 2, 3, 4\} \implies \text{freq} = 4 < 5$ (Scartato);
- **Pollo**: compare in $\{1, 2, 3, 4, 5, 6\} \implies \text{freq} = 6 \geq 5$ **(Frequente)**;
- **Pesce**: compare in $\{1, 2, 3, 4, 5, 6\} \implies \text{freq} = 6 \geq 5$ **(Frequente)**;
- **Prosciutto**: compare in $\{3, 4, 5, 6, 7\} \implies \text{freq} = 5 \geq 5$ **(Frequente)**;
- **Pane**: compare in $\{3, 4, 5, 6\} \implies \text{freq} = 4 < 5$ (Scartato);
- **Salsicce**: compare in $\{2, 3\} \implies \text{freq} = 2 < 5$ (Scartato).

Analizzando i candidati a due elementi, la coppia {Pollo, Pesce} compare congiuntamente nelle transazioni $\{1, 2, 3, 4, 5, 6\}$, totalizzando una frequenza pari a $6 \geq 5$.

L'insieme finale degli itemset frequenti per Min_freq $= 5$ è:

$$\{\text{Bistecca}\}, \quad \{\text{Pollo}\}, \quad \{\text{Pesce}\}, \quad \{\text{Prosciutto}\}, \quad \{\text{Pollo, Pesce}\} \tag{12.5}$$

12.2.2 Transizione ai Domini Complessi Multicarrello

Nella realtà dei sistemi aziendali, ciascun cliente effettua nel tempo una successione di acquisti distinti. I dati associati a un consumatore non corrispondono dunque a un singolo vettore binario, bensì a una **cronologia temporale di carrelli**, dove ogni carrello può comprendere un insieme eterogeneo di articoli.

Si consideri un database costituito da quattro clienti, le cui sequenze di acquisto sono illustrate nella Figura 12.1.

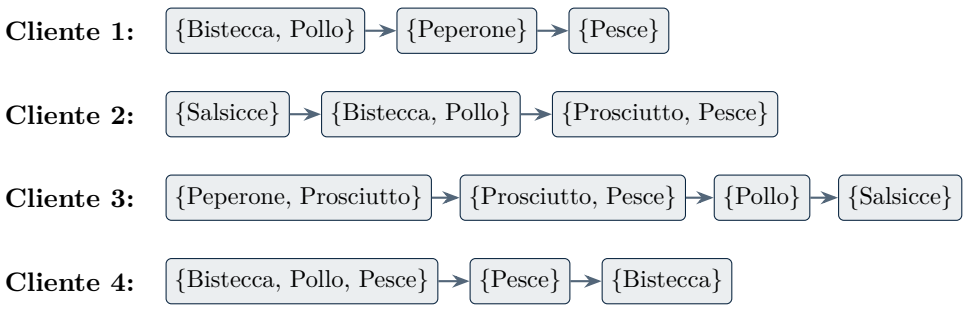


Figura 12.1: Cronologie temporali d'acquisto per quattro clienti in un dominio a carrelli multipli.

12.2.3 Obiettivo del Sequential Pattern Mining

L'obiettivo primario consiste nell'identificare schemi (sottosequenze) ordinati nel tempo che ricorrono in una percentuale significativa di clienti, ossia con supporto superiore o uguale a una soglia prestabilita minsup.

Analizzando la Figura 12.1, si consideri il pattern candidato ordinato:

$$w = \langle \{\text{Bistecca, Pollo}\}\ \{\text{Pesce}\} \rangle \tag{12.6}$$

Tale pattern prescrive che il cliente acquisti congiuntamente Bistecca e Pollo e che, in una transazione temporalmente **successiva**, acquisti Pesce. Verifichiamo la presenza di tale sequenza nei quattro clienti:

- **Cliente 1**: presenta l'evento {Bistecca, Pollo} al carrello 1, seguito dall'acquisto di {Pesce} al carrello 3 $\implies$ **Match**;
- **Cliente 2**: presenta {Bistecca, Pollo} al carrello 2, seguito dal carrello {Prosciutto, Pesce} (che include Pesce) al carrello 3 $\implies$ **Match**;
- **Cliente 3**: acquista Peperone, Prosciutto, Pesce e infine Pollo. Non acquista mai congiuntamente Bistecca e Pollo prima di Pesce $\implies$ **No Match**;
- **Cliente 4**: acquista al carrello 1 il set {Bistecca, Pollo, Pesce} (che contiene {Bistecca, Pollo}), e al carrello 2 acquista {Pesce} $\implies$ **Match**.

Il pattern $w = \langle \{\text{Bistecca, Pollo}\}\ \{\text{Pesce}\} \rangle$ è contenuto in 3 clienti su 4, raggiungendo un supporto pari a:

$$\sigma(w) = \frac{3}{4} = 75\% \tag{12.7}$$

Questo schema cattura una fedele abitudine di consumo differita nel tempo che l'analisi a transazioni disgiunte non avrebbe in alcun modo potuto svelare.

12.2.4 Regole Sequenziali Predittive nei Domini Industriali e Retail

Dall'estrazione di pattern sequenziali frequenti è possibile derivare **regole sequenziali** predittive di tipo $S_1 \Longrightarrow S_2$ ad elevata confidenza:

- **Reti di Telecomunicazione (Log di Allarmi)**:

$$\langle\{\text{Line_Current}\}\rangle \langle\{\text{Rectifier_Alarm}\}\rangle \Longrightarrow \langle\{\text{Fire_Alarm}\}\rangle \quad (12.8)$$

  Nelle infrastrutture di telecomunicazione, il superamento anomalo della corrente di alimentazione seguito dall'allarme termico del ponte raddrizzatore predice con altissima probabilità l'innesco di un incendio nella sala apparati. La conoscenza di tale pattern consente al sistema automatico di sezionare i carichi ed erogare l'estinguente prima dell'innesco irreversibile delle fiamme.

- **Libreria Specializzata in Informatica (Cross-selling Tecnico)**:

$$\langle\{\text{Intro_Visual_C}\}\rangle \langle\{\text{C++_Primer}\}\rangle \Longrightarrow \langle\{\text{Perl_for_dummies, Tcl_Tk}\}\rangle \quad (12.9)$$

  I lettori che completano l'apprendimento dei fondamenti del C++ tendono storicamente ad acquistare in una fase successiva manuali dedicati ai linguaggi di scripting per l'integrazione di sistema. Questa evidenza orienta le strategie promozionali e di raccomandazione personalizzata via email.

- **Articoli Sportivi (Progressione d'Uso)**:

$$\langle\{\text{Scarpe}\}\rangle \langle\{\text{Racchetta, Palline}\}\rangle \Longrightarrow \langle\{\text{Giacca_Sportiva}\}\rangle \quad (12.10)$$

  L'acquisto iniziale di scarpe tecniche seguito dall'attrezzatura di gioco preluda all'acquisto di capi di abbigliamento protettivo termico per l'attività all'aperto.

12.3 Tassonomia e Rappresentazione dei Dati Sequenziali

La teoria del Sequential Pattern Mining impiega un'astrazione algebrica unificata che si applica in modo uniforme a dati provenienti da sorgenti fisiche marcatamente eterogenee [2].

12.3.1 Mappatura Terminologica per Dominio Applicativo

La Tabella 12.2 stabilisce la corrispondenza biunivoca tra i costrutti formali della teoria e la loro declinazione semantica nei principali scenari operativi.

Dominio Applicativo	**Sequence Database** (D)	**Sequenza** (s)	**Elemento / Transazione** (e_j)	**Evento / Item** (i)
Retail / Clienti	Anagrafica e transazioni di tutti i clienti	Cronologia completa degli acquisti di un singolo cliente	Insieme di merci acquistate simultaneamente al tempo t	Singolo articolo (es. Latte, Libro)
Web Clickstream	File di log del server HTTP	Percorso di navigazione di un visitatore (sessione)	Pagine o risorse caricate a seguito di un click	Singolo URL, file o script richiesto
Reti Sensori / IoT	Repository centrale dei log telemetrici	Serie storica delle rilevazioni di una specifica sonda	Insieme di codici di allarme registrati al tempo t	Singolo segnale d'allarme o superamento soglia
Genomica / DNA	Banca dati genetica (es. GenBank)	Sequenza nucleotidica di un dato organismo	Singola posizione lungo il filamento molecolare	Singola base azotata (A, C, G, T)

Tabella 12.2: Mappatura terminologica formale tra concetti teorici e domini applicativi.

12.3.2 Rappresentazione Tabellare ed Asse Temporale (Timeline)

Nei sistemi di memorizzazione raw, i dati sequenziali sono registrati come tuple composte da identificativo di entità (`Object`), marcatore cronologico

(`Timestamp`) ed eventi rilevati (`Events`).

Si consideri l'esempio riportato in Tabella 12.3:

Oggetto (Object)	Timestamp (t)	Eventi Registrati (Events)
A	10	$\{2, 3, 5\}$
	20	$\{1, 6\}$
	23	$\{1\}$
B	11	$\{4, 5, 6\}$
	17	$\{2\}$
	21	$\{1, 2, 7, 8\}$
	28	$\{1, 6\}$
C	14	$\{1, 7, 8\}$

Tabella 12.3: Registrazione tabellare grezza di eventi per tre entità distinte.

Nella Figura 12.2, le medesime registrazioni sono disposte lungo una retta temporale continua marcata con tacche da $t = 10$ a $t = 30$.

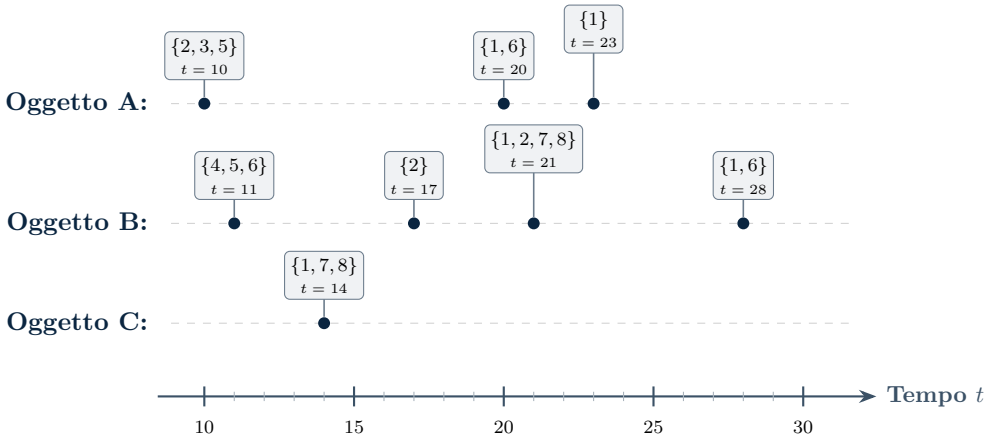


Figura 12.2: Proiezione temporale continua delle transazioni per gli oggetti A, B e C.

L'ispezione della Figura 12.2 mette in luce due proprietà strutturali essenziali:

1. **Eterogeneità degli Intervalli Temporali** (Δt): La distanza temporale che intercorre tra transazioni successive non è uniforme. Per l'oggetto

B, ad esempio, $\Delta t_{1\to 2} = 17 - 11 = 6$, mentre $\Delta t_{2\to 3} = 21 - 17 = 4$, e $\Delta t_{3\to 4} = 28 - 21 = 7$.

2. **Asincronia tra Oggetti**: Gli eventi di oggetti diversi non si verificano a intervalli sincroni coordinati.

12.4 Formalizzazione Matematica delle Sequenze

Definiamo in modo formale e algebrico i costrutti fondamentali della teoria [2, 16].

Definizione 12.4.1 (Item e Insieme Universo)**.** Sia $I = \{i_1, i_2, \ldots, i_m\}$ un insieme finito di elementi distinti, denominati **item** o **eventi**.

Definizione 12.4.2 (Elemento o Transazione)**.** Un **elemento** (denominato anche transazione) $e \subseteq I$ è un sottoinsieme non vuoto di item:

$$e = \{i_{j_1}, i_{j_2}, \ldots, i_{j_k}\} \quad \text{con } e \subseteq I,\ e \neq \emptyset \tag{12.11}$$

Gli item appartenenti a un medesimo elemento sono racchiusi tra parentesi graffe $\{\}$ e si considerano privi di ordine temporale interno: essi co-occorrono simultaneamente.

Definizione 12.4.3 (Sequenza)**.** Una **sequenza** s è una lista ordinata di elementi, convenzionalmente racchiusa tra parentesi uncinate $\langle\ \rangle$:

$$s = \langle e_1, e_2, \ldots, e_p \rangle \tag{12.12}$$

dove ciascun $e_j \subseteq I$ è un elemento. L'indice di posizione j riflette l'ordine cronologico degli eventi, tale per cui l'istante di occorrenza soddisfa $t(e_j) < t(e_{j+1})$ per ogni $1 \leq j < p$.

Definizione 12.4.4 (Lunghezza di una Sequenza)**.** La **lunghezza** di una sequenza s, denotata con $|s|$, è definita rigorosamente come il numero di elementi (o transazioni temporali) che la compongono:

$$|s| = p \tag{12.13}$$

Definizione 12.4.5 (k-Sequenza)**.** Una sequenza $s = \langle e_1, e_2, \ldots, e_p \rangle$ prende il nome di **k-sequenza** se il numero complessivo di item (eventi) presenti al suo interno, contando la cardinalità di ciascun elemento, è pari a k:

$$k = \sum_{j=1}^{p} |e_j| \tag{12.14}$$

Distinzione Critica: Lunghezza vs Valore k

Si presti la massima attenzione a non confondere la **lunghezza** $|s|$ con il parametro k:

- La **lunghezza** $|s| = p$ quantifica il numero di **step temporali distinti** (numero di insiemi/carrelli);
- Il parametro k quantifica la somma totale degli **item** contenuti in tutti gli elementi della sequenza.

Esempio 12.4.1 (Calcolo di Lunghezza e k per una Sequenza)**.** Si consideri la sequenza:

$$s = \langle \{A, B\} \{B, E, F\} \{A\} \{E, F, H\} \rangle \tag{12.15}$$

- **Elementi costituenti**:

  $$e_1 = \{A, B\}, \quad e_2 = \{B, E, F\}, \quad e_3 = \{A\}, \quad e_4 = \{E, F, H\}$$

  La sequenza possiede $p = 4$ elementi ordinati nel tempo $\implies$ **Lunghezza** $|s| = 4$.

- **Conteggio degli item per ciascun elemento**:

  $$|e_1| = 2, \quad |e_2| = 3, \quad |e_3| = 1, \quad |e_4| = 3$$

  Sommando le cardinalità:

  $$k = |e_1| + |e_2| + |e_3| + |e_4| = 2 + 3 + 1 + 3 = 9$$

  Pertanto, s è una **9-sequenza di lunghezza 4**.

- **Marcatori temporali associati**: Se gli elementi sono registrati ai tempi $t(e_1) = 0$, $t(e_2) = 120$, $t(e_3) = 130$ e $t(e_4) = 200$, gli intervalli temporali risultano disomogenei: $\Delta t_{1\to2} = 120$, $\Delta t_{2\to3} = 10$, $\Delta t_{3\to4} = 70$.

12.4.1 Convenzione per Sequenze Senza Timestamp Espliciti

Quando i dati in ingresso non dispongono di un timestamp fisico esplicito (millisecondi o date assolute), si adotta convenzionalmente un tempo discreto basato sulla posizione ordinale ordinale degli elementi (indicizzazione a partire da zero):

$$t(e_j) = j - 1 \quad \text{per } j = 1, 2, \ldots, p \tag{12.16}$$

Data ad esempio la sequenza priva di tempi espliciti:

$$s = \langle \{A, C\} \{E\} \{A, F\} \{E, G, H\} \rangle \tag{12.17}$$

l'assegnazione convenzionale dei tempi risulta:

$$t(\{A, C\}) = 0, \quad t(\{E\}) = 1, \quad t(\{A, F\}) = 2, \quad t(\{E, G, H\}) = 3$$

12.4.2 Sequenze a Elementi Singleton vs Elementi Complessi

Una classificazione tassonomica basilare riguarda la cardinalità degli elementi e_j:

- **Sequenze a Elementi Singleton ($|e_j| = 1 \ \forall j$)**: Ciascun evento avviene singolarmente nel proprio istante temporale. In questo caso, la lunghezza coincide esattamente con il parametro k ($|s| = k$).
  - *Esempio di Web Clickstream*:

    $$\langle \{\text{Home}\} \{\text{Electronics}\} \{\text{Cameras}\} \{\text{Cart}\} \{\text{Checkout}\} \rangle$$

    Costituita da 5 click individuali: lunghezza = 5, 5-sequenza.
  - *Esempio di Prestiti*: $\langle \{\text{Libro}_1\} \{\text{Libro}_2\} \{\text{Libro}_3\} \rangle$: lunghezza = 3, 3-sequenza.
- **Sequenze ad Elementi Complessi ($\exists j : |e_j| > 1$)**: Gli eventi co-occorrono simultaneamente all'interno di uno o più intervalli temporali. In tal caso si ha strettamente $k > |s|$.
  - *Esempio*: Nell'incidente di Three Mile Island, l'evento iniziale e l'evento finale contengono allarmi multipli simultanei:

    $$\langle \{\text{resina, valvola}\} \{\text{pompa}\} \{\text{turbina, pressione}\} \rangle$$

    Costituita da 3 elementi temporali ma 5 item complessivi: lunghezza = 3, 5-sequenza.

12.5 Relazione di Contenimento e Definizione di Sottosequenza

Il concetto cardine per valutare la frequenza e per eseguire il matching di pattern sequenziali è la nozione algebrica di **sottosequenza** [2, 16].

Definizione 12.5.1 (Sottosequenza / Contenimento)**.** Siano date due sequenze arbitrarie:

$$a = \langle a_1, a_2, \ldots, a_n \rangle \quad \text{e} \quad b = \langle b_1, b_2, \ldots, b_m \rangle \quad \text{con } m \geq n \tag{12.18}$$

Diciamo che a è una **sottosequenza** di b (o che a è contenuta in b, denotato formalmente con $a \subseteq b$) se e solo se esiste una successione di n indici interi strettamente crescenti:

$$1 \leq i_1 < i_2 < \cdots < i_n \leq m \tag{12.19}$$

tali che ciascun elemento di a sia un sottoinsieme del corrispondente elemento di b:

$$a_1 \subseteq b_{i_1}, \quad a_2 \subseteq b_{i_2}, \quad \ldots, \quad a_n \subseteq b_{i_n} \tag{12.20}$$

Se $a \subseteq b$, si dice equivalentemente che b è una **supersequenza** di a, o che b supporta a.

Proprietà Cruciale: Non-Contiguità Temporale

Gli elementi della sottosequenza a **non devono necessariamente** occupare posizioni contigue all'interno di b. Tra l'elemento b_{i_j} e l'elemento $b_{i_{j+1}}$ possono intercorrere elementi intermedi arbitrari di b, purché l'ordine temporale relativo sia rigorosamente rispettato ($i_1 < i_2 < \cdots < i_n$).

12.5.1 Esempio Grafico di Matching di una Sottosequenza

Si considerino la sottosequenza candidata a e la sequenza dati b:

$$a = \langle \{A\}\, \{B, C\}\, \{D\} \rangle \quad (n = 3) \tag{12.21}$$

$$b = \langle \{D\}_{t=0}\, \{A, C\}_{t=1}\, \{A, B, C\}_{t=2}\, \{F\}_{t=3}\, \{B, E\}_{t=4}\, \{D\}_{t=5} \rangle \quad (m = 6) \tag{12.22}$$

Verifichiamo la sussistenza della relazione $a \subseteq b$ individuando gli indici crescenti $i_1 < i_2 < i_3$:

- Per $a_1 = \{A\}$: al tempo $t = 1$, si ha $\{A\} \subseteq \{A, C\} = b_1 \implies i_1 = 1$;
- Per $a_2 = \{B, C\}$: al tempo $t = 2$, si ha $\{B, C\} \subseteq \{A, B, C\} = b_2 \implies i_2 = 2$;
- Per $a_3 = \{D\}$: al tempo $t = 5$, si ha $\{D\} \subseteq \{D\} = b_5 \implies i_3 = 5$.

Poiché gli indici soddisfano la relazione d'ordine stretto $1 < 2 < 5$, tutti i vincoli sono soddisfatti e la relazione $a \subseteq b$ è pienamente verificata. La Figura 12.3 illustra visivamente questo allineamento.

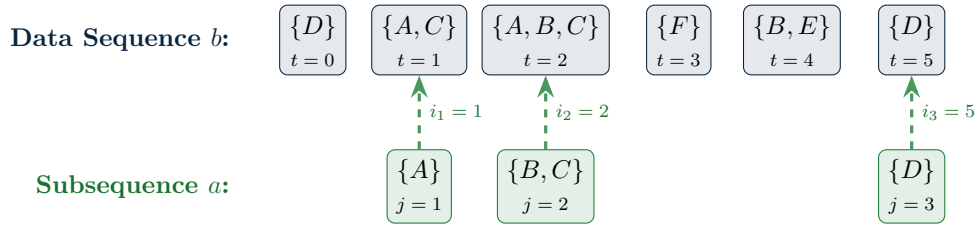


Figura 12.3: Allineamento tra la sottosequenza a e la sequenza dati b con indici $i_1 = 1 < i_2 = 2 < i_3 = 5$.

12.5.2 Casi Notevoli e Controesempi di Contenimento

Per consolidare l'applicazione rigorosa della Definizione 12.19, esaminiamo tre casi emblematici riassunti nella Tabella 12.4.

Data Sequence (b)	**Candidate Subsequence** (a)	**Contenuta?**	**Giustificazione Analitica**
$\langle \{2,4\}\,\{3,5,6\}\,\{8\} \rangle$	$\langle \{2\}\,\{3,5\} \rangle$	**SÌ**	$\{2\} \subseteq \{2,4\}$ all'indice 1; $\{3,5\} \subseteq \{3,5,6\}$ all'indice 2. Si ha $i_1 = 1 < i_2 = 2$.
$\langle \{1,2\}\,\{3,4\} \rangle$	$\langle \{1\}\,\{2\} \rangle$	**NO**	Gli item 1 e 2 compaiono nello **stesso** elemento b_1. La sottosequenza a richiede che 2 avvenga in un tempo **strettamente successivo** a 1 ($i_1 < i_2$). Poiché non vi è alcun elemento successivo contenente 2, la condizione fallisce.
$\langle \{2,4\}\,\{2,4\}\,\{2,5\} \rangle$	$\langle \{2\}\,\{4\} \rangle$	**SÌ**	$\{2\} \subseteq b_1$ all'indice 1; $\{4\} \subseteq b_2$ all'indice 2. Si ha $i_1 = 1 < i_2 = 2$.

Tabella 12.4: Verifica della relazione di sottosequenza su casi campione.

Differenza tra $\{1,2\}$ e $\langle \{1\}\,\{2\} \rangle$

La seconda riga della Tabella 12.4 illustra la differenza semantica insormontabile tra co-occorrenza istantanea e dipendenza temporale:

- Se due articoli sono acquistati simultaneamente nello stesso carrello, essi formano l'elemento $\{1,2\}$;
- La sequenza $\langle \{1\}\,\{2\} \rangle$ prescrive esplicitamente che l'articolo 1 sia acquistato **prima** dell'articolo 2 ($t(1) < t(2)$).

Pertanto, $\{1,2\}$ non soddisfa né supporta $\langle \{1\}\,\{2\} \rangle$.

12.6 Supporto dei Sequential Patterns e Istanze Multiple

Definiamo formalmente il concetto di frequenza statistica per una sequenza candidato all'interno di un database di sequenze [2, 16].

12.6.1 Definizione di Supporto di una Sequenza

Definizione 12.6.1 (Supporto di una Sequenza)**.** Sia $D = \{s_1, s_2, \ldots, s_{|D|}\}$ un database di sequenze (dove ogni s_i rappresenta la cronologia completa di una specifica entità). Il **supporto** (o frequenza relativa) di una sequenza candidato w è definito come la proporzione di sequenze in D che contengono w come sottosequenza:

$$\sigma(w) = \frac{\left|\{s \in D \mid w \subseteq s\}\right|}{|D|} \tag{12.23}$$

Definizione 12.6.2 (Sequential Pattern)**.** Una sequenza w prende il nome di **Sequential Pattern** (o sequenza frequente) se e solo se il suo supporto soddisfa una soglia minima specificata dall'utente:

$$\sigma(w) \geq \text{minsup} \tag{12.24}$$

Esempio 12.6.1 (Calcolo del Supporto su un Database a 4 Sequenze)**.** Si consideri il pattern candidato:

$$w = \langle \{A\}\, \{B, C\}\, \{D\} \rangle \tag{12.25}$$

e un database D costituito dalle seguenti 4 sequenze:

$$\begin{aligned} s_1 &= \langle \{D\}\, \{A, C\}\, \{A, B, C\}\, \{F\}\, \{D\} \rangle \\ s_2 &= \langle \{D\}\, \{A\}\, \{A, B, D\}\, \{D\} \rangle \\ s_3 &= \langle \{D\}\, \{A, C\}\, \{B, C\}\, \{D\}\, \{B, E\} \rangle \\ s_4 &= \langle \{D\}\, \{B, C\}\, \{D\}\, \{F\} \rangle \end{aligned}$$

Verifichiamo la sussistenza della relazione $w \subseteq s_k$ per ciascuna sequenza:

1. **Sequenza** s_1:
   - $\{A\} \subseteq \{A, C\}$ al tempo $t = 1$;
   - $\{B, C\} \subseteq \{A, B, C\}$ al tempo $t = 2$;
   - $\{D\} \subseteq \{D\}$ al tempo $t = 4$;

Gli indici soddisfano $1 < 2 < 4 \implies w \subseteq s_1$ **(MATCH, ✓)**.

2. **Sequenza** s_2: In s_2, l'elemento $\{A, B, D\}$ non include la lettera C. L'itemset congiunto $\{B, C\}$ non compare in nessun elemento di s_2. Pertanto $w \not\subseteq s_2$ **(NO MATCH, ×)**.

3. **Sequenza** s_3:

   - $\{A\} \subseteq \{A, C\}$ al tempo $t = 1$;
   - $\{B, C\} \subseteq \{B, C\}$ al tempo $t = 2$;
   - $\{D\} \subseteq \{D\}$ al tempo $t = 3$;

   Gli indici soddisfano $1 < 2 < 3 \implies w \subseteq s_3$ **(MATCH, ✓)**.

4. **Sequenza** s_4: L'elemento $\{B, C\}$ è preceduto unicamente da $\{D\}$ (a $t = 0$). L'item A non è presente in alcuna transazione precedente a $\{B, C\}$. Pertanto $w \not\subseteq s_4$ **(NO MATCH, ×)**.

Il supporto finale della sequenza candidato w risulta:

$$\sigma(w) = \frac{|\{s_1, s_3\}|}{|D|} = \frac{2}{4} = 0.50 \quad (50\%) \tag{12.26}$$

Se la soglia dell'utente è fissata a minsup $\leq 50\%$, w è riconosciuto come pattern sequenziale frequente.

12.6.2 Concetto di Istanza (Embedding) e Conteggio delle Occorrenze

All'interno di una medesima sequenza s, una sottosequenza candidato w può allinearsi in molteplici modalità differenti.

Definizione 12.6.3 (Istanza / Embedding)**.** Data una sottosequenza $w = \langle w_1, \ldots, w_n \rangle$ e una sequenza dati $s = \langle s_0, \ldots, s_{m-1} \rangle$, ciascuna ennupla ordinata di indici temporali $\langle i_1, i_2, \ldots, i_n \rangle$ che soddisfa:

$$i_1 < i_2 < \cdots < i_n \quad \text{con } w_j \subseteq s_{i_j} \ \ \forall j \in \{1, \ldots, n\} \tag{12.27}$$

costituisce una distinta **istanza** (o **embedding**) di w all'interno di s.

Esempio 12.6.2 (Analisi delle Istanze Multiple)**.** Si consideri il pattern candidato:

$$w = \langle \{A\}\, \{B\}\, \{D\} \rangle$$

e la sequenza dati:

$$s = \langle \{D\}_{t=0} \{A, C\}_{t=1} \{A, B, C\}_{t=2} \\ \{F\}_{t=3} \{B, E\}_{t=4} \{D\}_{t=5} \rangle \tag{12.28}$$

In questa sequenza si riscontrano ben tre distinte combinazioni di indici che realizzano il matching:

1. **Istanza 1**: $\langle i_1 = 1, i_2 = 2, i_3 = 5 \rangle$
   - $\{A\} \subseteq s_1 = \{A, C\}$
   - $\{B\} \subseteq s_2 = \{A, B, C\}$
   - $\{D\} \subseteq s_5 = \{D\}$
2. **Istanza 2**: $\langle i_1 = 1, i_2 = 4, i_3 = 5 \rangle$
   - $\{A\} \subseteq s_1 = \{A, C\}$
   - $\{B\} \subseteq s_4 = \{B, E\}$
   - $\{D\} \subseteq s_5 = \{D\}$
3. **Istanza 3**: $\langle i_1 = 2, i_2 = 4, i_3 = 5 \rangle$
   - $\{A\} \subseteq s_2 = \{A, B, C\}$
   - $\{B\} \subseteq s_4 = \{B, E\}$
   - $\{D\} \subseteq s_5 = \{D\}$

Convenzione di Conteggio nel Sequential Pattern Mining Standard

Nel calcolo standard del supporto di un sequential pattern:

Ogni sequenza cliente contribuisce con un incremento binario pari a $+1$

indipendentemente dal numero di istanze o embedding multipli che essa racchiude. È sufficiente che esista **almeno un'istanza valida** affinché la sequenza sia computata nel numeratore di $\sigma(w)$.

12.7 Esercizio Guidato: Matching Completo ed Enumerazione delle Istanze

Per padroneggiare la logica dell'algoritmo di matching, svolgiamo un esercizio completo di verifica e censimento di tutte le istanze valide.

12.7.1 Testo dell'Esercizio

Sia data la sequenza di dati complessa s composta da 8 elementi distribuiti da $t = 0$ a $t = 7$:

$$\begin{aligned} s = \langle &\{A, C\}_{t=0}\, \{C, D\}_{t=1}\, \{F, H\}_{t=2}\, \{A, B\}_{t=3} \\ &\{B, C, D\}_{t=4}\, \{E\}_{t=5}\, \{A, B, D\}_{t=6}\, \{F\}_{t=7}\rangle \end{aligned} \tag{12.29}$$

Si determini se i seguenti quattro pattern candidati sono contenuti in s, e in caso affermativo si enumerino tutte le istanze distinte $\langle t_1, \ldots, t_n\rangle$:

1. $P_1 = \langle \{C\}\, \{H\}\, \{C\}\rangle$
2. $P_2 = \langle \{A\}\, \{F\}\rangle$
3. $P_3 = \langle \{A\}\, \{A\}\, \{D\}\rangle$
4. $P_4 = \langle \{A\}\, \{A, B\}\, \{F\}\rangle$

12.7.2 Risoluzione Analitica Dettagliata

Pattern 1: $P_1 = \langle \{C\}\, \{H\}\, \{C\}\rangle$

Il pattern richiede di individuare tre marcatori temporali $t_1 < t_2 < t_3$ tali che:

$$C \in s_{t_1}, \quad H \in s_{t_2}, \quad C \in s_{t_3}$$

- L'elemento $\{H\}$ è presente unicamente al tempo $t = 2$ ($s_2 = \{F, H\}$). Pertanto deve valere necessariamente $t_2 = 2$.
- Cerchiamo le occorrenze di C per $t_1 < 2$:
  - $t_1 = 0$: $C \in s_0 = \{A, C\}$;
  - $t_1 = 1$: $C \in s_1 = \{C, D\}$.
- Cerchiamo le occorrenze di C per $t_3 > 2$:
  - Al tempo $t = 4$: $C \in s_4 = \{B, C, D\}$. Nei tempi successivi $(5, 6, 7)$ l'item C non compare più. Dunque $t_3 = 4$.

Combinando le scelte possibili, otteniamo esattamente **2 istanze distinte**:

1. $\langle t_1 = 0,\, t_2 = 2,\, t_3 = 4\rangle$
2. $\langle t_1 = 1,\, t_2 = 2,\, t_3 = 4\rangle$

Esito: Il pattern P_1 **OCCORRE** (è supportato con 2 istanze).

Pattern 2: $P_2 = \langle \{A\}\, \{F\} \rangle$

Il pattern richiede di individuare due istanti $t_1 < t_2$ tali che:

$$A \in s_{t_1} \quad \text{e} \quad F \in s_{t_2}$$

- L'item A è presente nei tempi: $t \in \{0, 3, 6\}$ (poiché $s_0 = \{A, C\}$, $s_3 = \{A, B\}$, $s_6 = \{A, B, D\}$).
- L'item F è presente nei tempi: $t \in \{2, 7\}$ (poiché $s_2 = \{F, H\}$, $s_7 = \{F\}$).

Valutiamo tutte le coppie (t_1, t_2) con $t_1 < t_2$:

- Per $t_1 = 0$: i valori ammissibili per $t_2 > 0$ sono $t_2 = 2$ e $t_2 = 7 \implies \langle 0, 2 \rangle$ e $\langle 0, 7 \rangle$.
- Per $t_1 = 3$: l'unico valore per $t_2 > 3$ è $t_2 = 7 \implies \langle 3, 7 \rangle$.
- Per $t_1 = 6$: l'unico valore per $t_2 > 6$ è $t_2 = 7 \implies \langle 6, 7 \rangle$.

Otteniamo esattamente **4 istanze distinte**:

$$\langle 0, 2 \rangle, \quad \langle 0, 7 \rangle, \quad \langle 3, 7 \rangle, \quad \langle 6, 7 \rangle \tag{12.30}$$

Esito: Il pattern P_2 **OCCORRE** (è supportato con 4 istanze).

Pattern 3: $P_3 = \langle \{A\}\, \{A\}\, \{D\} \rangle$

Il pattern richiede tre istanti $t_1 < t_2 < t_3$ con:

$$A \in s_{t_1}, \quad A \in s_{t_2}, \quad D \in s_{t_3}$$

- Le posizioni contenenti A sono $t \in \{0, 3, 6\}$. Le coppie ordinate (t_1, t_2) con $t_1 < t_2$ sono:
$$(0, 3), \quad (0, 6), \quad (3, 6)$$
- L'item D compare nei tempi: $t \in \{1, 4, 6\}$ (poiché $s_1 = \{C, D\}$, $s_4 = \{B, C, D\}$, $s_6 = \{A, B, D\}$).
- Verifichiamo ciascuna coppia (t_1, t_2):
  - Per $(t_1 = 0, t_2 = 3)$: cerchiamo $t_3 > 3$ con $D \in s_{t_3}$. I tempi validi sono $t_3 = 4$ e $t_3 = 6$. Ciò genera le istanze $\langle 0, 3, 4 \rangle$ e $\langle 0, 3, 6 \rangle$.

- Per $(t_1 = 0, t_2 = 6)$: richiederebbe $t_3 > 6$ con $D \in s_{t_3}$. Al tempo $t = 7$ è presente solo $\{F\}$, quindi nessun D è disponibile dopo $t = 6$. Nessuna istanza valida.
- Per $(t_1 = 3, t_2 = 6)$: analogamente richiederebbe $t_3 > 6$, impossibile per assenza di D a $t > 6$. Nessuna istanza valida.

Otteniamo esattamente **2 istanze distinte**:

$$\langle 0, 3, 4 \rangle \quad \text{e} \quad \langle 0, 3, 6 \rangle \tag{12.31}$$

Esito: Il pattern P_3 **OCCORRE** (è supportato con 2 istanze).

Pattern 4: $P_4 = \langle \{A\}\, \{A, B\}\, \{F\} \rangle$

Il pattern richiede tre istanti $t_1 < t_2 < t_3$ tali che:

$$A \in s_{t_1}, \quad \{A, B\} \subseteq s_{t_2}, \quad F \in s_{t_3}$$

- L'itemset congiunto $\{A, B\}$ è contenuto in:
  - $s_3 = \{A, B\} \implies t_2 = 3$;
  - $s_6 = \{A, B, D\} \implies t_2 = 6$.
- **Caso** $t_2 = 3$:
  - Per $t_1 < 3$ con $A \in s_{t_1}$: l'unica scelta è $t_1 = 0$ ($s_0 = \{A, C\}$).
  - Per $t_3 > 3$ con $F \in s_{t_3}$: l'unica scelta è $t_3 = 7$ ($s_7 = \{F\}$).
  - Istanza generata: $\langle 0, 3, 7 \rangle$.
- **Caso** $t_2 = 6$:
  - Per $t_1 < 6$ con $A \in s_{t_1}$: possiamo scegliere $t_1 = 0$ oppure $t_1 = 3$.
  - Per $t_3 > 6$ con $F \in s_{t_3}$: l'unica scelta è $t_3 = 7$.
  - Istanze generate: $\langle 0, 6, 7 \rangle$ e $\langle 3, 6, 7 \rangle$.

Otteniamo esattamente **3 istanze distinte**:

$$\langle 0, 3, 7 \rangle, \quad \langle 0, 6, 7 \rangle, \quad \langle 3, 6, 7 \rangle \tag{12.32}$$

Esito: Il pattern P_4 **OCCORRE** (è supportato con 3 istanze).

12.7.3 Quadro Riepilogativo delle Risoluzioni

La Tabella 12.5 sintetizza i risultati dell'analisi per la sequenza s.

Pattern	**Forma Strutturale**	**Esito**	**Istanze Temporali** $\langle t_1, \ldots, t_n \rangle$
P_1	$\langle \{C\}\,\{H\}\,\{C\} \rangle$	**OCCORRE**	$\langle 0,2,4 \rangle$, $\langle 1,2,4 \rangle$ (2 istanze)
P_2	$\langle \{A\}\,\{F\} \rangle$	**OCCORRE**	$\langle 0,2 \rangle$, $\langle 0,7 \rangle$, $\langle 3,7 \rangle$, $\langle 6,7 \rangle$ (4 istanze)
P_3	$\langle \{A\}\,\{A\}\,\{D\} \rangle$	**OCCORRE**	$\langle 0,3,4 \rangle$, $\langle 0,3,6 \rangle$ (2 istanze)
P_4	$\langle \{A\}\,\{A,B\}\,\{F\} \rangle$	**OCCORRE**	$\langle 0,3,7 \rangle$, $\langle 0,6,7 \rangle$, $\langle 3,6,7 \rangle$ (3 istanze)

Tabella 12.5: Quadro riassuntivo delle istanze e del matching per i quattro pattern dell'Esercizio.

Sintesi Metodologica

L'analisi condotta conferma come:

1. Un singolo elemento della sequenza candidato possa contenere uno o più item simultanei;
2. La sequenzialità temporale richieda strettamente indici crescenti ($t_1 < t_2 < \cdots < t_n$);
3. Elementi con item identici possano ripetersi in istanti temporali distinti all'interno della stessa sequenza;
4. Nel modello standard di Sequential Pattern Mining, ciascuno di questi pattern incrementerebbe di $+1$ il conteggio di supporto di questa sequenza, indipendentemente dalla molteplicità degli embedding rilevati.

Capitolo 13

Mining di Regole di Associazione (FP-Growth), Simulazioni di Clustering e Pattern Sequenziali (GSP)

13.1 Limiti dell'Approccio Level-Wise e Motivazione di FP-Growth

Nel campo del Data Mining e dell'estrazione di regole di associazione, la procedura classica introdotta dalla letteratura scientifica di riferimento si basa sul paradigma *level-wise* (livello per livello) proprio dell'algoritmo **Apriori** [2, 14], formalizzato originariamente da Rakesh Agrawal e Ramakrishnan Srikant nel 1994 [14].

L'algoritmo Apriori (approfondito nel Capitolo 11) adotta uno schema iterativo basato sul principio di anti-monotonicità del supporto:

1. Si inizializza la dimensione degli itemset a $k = 1$, ponendo l'insieme dei candidati C_1 pari alla totalità dei singoli item distinti presenti nel database transazionale.

2. Fintanto che l'insieme dei candidati C_k non risulta vuoto, si eseguono ciclicamente le seguenti macro-fasi:

    - **Frequent Itemset Generation**: Viene scansionato l'intero archivio delle transazioni su memoria di massa per quantificare il conteggio di supporto di ciascun itemset candidato appartenente a C_k. Gli itemset il cui supporto relativo soddisfa o supera la soglia

minima prestabilita (minsup) vengono promossi nell'insieme L_k dei k-itemset frequenti.

- **Candidate Generation**: Gli elementi di L_k vengono combinati algebricamente tra loro (mediante join $L_k \times L_k$ o su prefisso comune) per generare una nuova collezione di candidati C_{k+1} di cardinalità $k+1$, su cui viene applicata la potatura di sottoinsiemi infrequenti (*candidate pruning*).
- Si incrementa la dimensione $k \leftarrow k+1$ e si reitera la scansione.

Sebbene il principio Apriori consenta una massiccia riduzione dello spazio combinatorio rispetto all'approccio a forza bruta, l'efficienza computazionale di questa strategia decade drasticamente in scenari applicativi realistici a causa di molteplici fattori strutturali [2, 18, 19]:

- **Scelta della soglia di supporto minimo (minsup)**: All'abbassarsi della soglia di supporto minimo, la cardinalità dell'insieme degli itemset frequenti cresce esponenzialmente. Di conseguenza, il numero di candidati intermedi C_k generati ad ogni livello esplode combinatoriamente, aumentando sensibilmente sia la lunghezza massima raggiungibile dagli itemset, sia lo sforzo computazionale necessario per verificarne l'occorrenza.

- **Dimensionalità del dataset (numero complessivo di item distinti)**: All'aumentare della varietà degli articoli presenti nel catalogo (migliaia o milioni di item distinti), cresce proporzionalmente lo spazio di memoria primaria richiesto per allocare contatori, tabelle hash e alberi di memorizzazione (k-d tree o hash tree). Quando un'elevata porzione di tali item risulta frequente, il sovraccarico di memoria e i costi computazionali associati al monitoraggio di ogni possibile combinazione diventano ingestibili.

- **Dimensione volumetrica del database**: Poiché la verifica empirica del supporto richiede una scansione integrale del database per ciascuna iterazione k, il tempo complessivo di I/O su memoria secondaria scala in modo lineare o super-lineare con il numero totale di transazioni N. Per database composti da milioni di record, eseguire decine di letture sequenziali del disco introduce latenze inaccettabili.

- **Lunghezza media delle transazioni**: Nei dataset ad elevata densità (in cui le singole transazioni contengono decine o centinaia di item), la lunghezza media dei record amplifica il fattore combinatorio: una transazione di lunghezza w contiene $\binom{w}{k}$ sottoinsiemi di dimensione

k. Ognuno di questi sottoinsiemi deve essere verificato contro l'albero dei candidati, causando un degrado delle prestazioni durante la fase di conteggio delle frequenze.

Per superare il duplice collo di bottiglia della generazione esplosiva dei candidati e delle ripetute scansioni del database, Jiawei Han, Jian Pei e Yiwen Yin hanno proposto nel 2000 l'algoritmo **FP-Growth** (*Frequent Pattern Growth*) [18, 19], fondato su una rappresentazione compatta denominata **FP-tree** (*Frequent Pattern Tree*).

13.2 Architettura della Struttura Dati FP-Tree

L'algoritmo FP-Growth abbandona il paradigma di generazione e test tipico di Apriori, sostituendolo con un approccio basato sulla compressione del database transazionale in un albero dei prefissi (*prefix-tree* o *trie*) residente in memoria principale, denominato **FP-tree** [18].

Le proprietà ingegneristiche fondamentali dell'FP-tree comprendono:

1. **Rappresentazione Altamente Compressa**: L'FP-tree conserva tutte le informazioni indispensabili relative alle transazioni del database in una forma altamente densa, scartando gli item non frequenti ed eliminando ogni informazione ridondante. La struttura è progettata per essere contenuta interamente nella memoria primaria (RAM), azzerando le operazioni di I/O successive.

2. **Mappatura delle Transazioni come Cammini Sovrapposti**: Ciascuna transazione viene mappata come un cammino (*path*) orientato discendente da un nodo radice virtuale privo di valore semantico (`null`). Quando più transazioni condividono una medesima sequenza iniziale di item, i loro cammini fisici si sovrappongono parzialmente, condividendo i medesimi nodi e incrementando i relativi contatori di frequenza.

3. **Strategia di Ricerca Divide-and-Conquer**: Una volta ultimata la costruzione dell'albero mediante due sole scansioni complete del database, l'estrazione degli itemset frequenti avviene direttamente sulla struttura ad albero mediante una scomposizione ricorsiva in sotto-problemi indipendenti basati su suffissi (*conditional pattern bases* e *conditional FP-trees*), senza generare mai esplicitamente insiemi di candidati.

13.2.1 Necessità dell'Ordinamento degli Item

In una base di dati transazionale standard, ciascuna transazione è formalmente un insieme matematico disordinato di item, per cui $\{A, B\} \equiv \{B, A\}$. Tuttavia, per permettere a una struttura ad albero dei prefissi di identificare ed aggregare prefissi comuni, è indispensabile imporre una relazione di **ordine totale** $\prec$ sull'universo degli item, trasformando ogni transazione in una sequenza rigorosamente ordinata.

Esempio 13.2.1 (Impatto dell'Ordinamento sulla Condivisione dei Nodi)**.** Si considerino due transazioni $T_1 = \{A, B\}$ e $T_2 = \{B, C, A\}$. Se gli elementi venissero inseriti senza un criterio di ordinamento comune, T_1 genererebbe il cammino null $\rightarrow A \rightarrow B$, mentre T_2 verrebbe letta ad esempio come null $\rightarrow B \rightarrow C \rightarrow A$. Nonostante entrambe le transazioni contengano congiuntamente la coppia $\{A, B\}$, i rispettivi percorsi non presenterebbero alcun nodo condiviso, vanificando qualunque compressione e duplicando la memoria allocata.

Imponendo un ordinamento totale condiviso, ogni transazione viene preventivamente riordinata prima dell'inserimento. Come prima istanza didattica, si adotta convenzionalmente l'**ordinamento lessicografico**:

$$A \prec B \prec C \prec D \prec E \tag{13.1}$$

13.3 Costruzione Dettagliata dell'FP-Tree con Ordinamento Lessicografico

Si consideri il database didattico composto da 10 transazioni riportato nella Tabella 13.1.

Tabella 13.1: Database transazionale didattico per la costruzione dell'FP-tree.

TID	Itemset Originario	Sequenza Ordinata Lessicografica
1	$\{A, B\}$	(A, B)
2	$\{B, C, D\}$	(B, C, D)
3	$\{A, C, D, E\}$	(A, C, D, E)
4	$\{A, D, E\}$	(A, D, E)
5	$\{A, B, C\}$	(A, B, C)
6	$\{A, B, C, D\}$	(A, B, C, D)
7	$\{B, C\}$	(B, C)
8	$\{A, B, C\}$	(A, B, C)
9	$\{A, B, D\}$	(A, B, D)
10	$\{B, C, E\}$	(B, C, E)

13.3.1 Etichetta dei Nodi e Header Table

La struttura dell'FP-tree poggia su due elementi cardine:

- **Etichetta del Nodo (`item:count`)**: Ciascun nodo dell'albero memorizza il simbolo dell'item corrispondente e un contatore intero positivo (*support count*), che quantifica esattamente quante transazioni attraversano quel nodo percorrendo il cammino specifico dalla radice fittizia `null`.
- **Header Table e Liste Concatenate Orizzontali**: Poiché uno stesso item può apparire in molteplici rami differenti dell'albero a causa di prefissi parzialmente disgiunti, tutti i nodi fisici associati al medesimo item vengono concatenati mediante una lista mono-direzionale (*node-link*). L'**Header Table** memorizza, per ciascun item distintivo, il puntatore di testa (*head pointer*) alla lista orizzontale, consentendo una rapida scansione di tutte le occorrenze di quell'item durante l'estrazione.

13.3.2 Inserimento Passo-Passo delle 10 Transazioni

La procedura di popolamento dell'albero evolve sequenzialmente:

1. **Inizializzazione**: Si alloca la radice fittizia etichettata con `null`.
2. **TID 1:** (A, B): Dalla radice `null` non esiste alcun figlio con etichetta A. Si crea il nodo figlio $A : 1$; da $A : 1$ si crea a sua volta il nodo figlio $B : 1$. La Header Table imposta i puntatori iniziali di A e B su tali nodi.

3. **TID 2:** (B, C, D): Da `null` non esiste alcun figlio diretto con etichetta B (il nodo B precedente è figlio di A). Viene generato un nuovo ramo autonomo adiacente alla radice: null $\rightarrow B:1 \rightarrow C:1 \rightarrow D:1$. Il nodo $B:1$ sotto A viene collegato mediante puntatore orizzontale al nuovo nodo $B:1$ sotto `null`.

4. **TID 3:** (A, C, D, E): Dalla radice `null` esiste già il figlio A. Il suo contatore viene incrementato passando da $A:1$ ad $A:2$. Poiché A non possiede figli etichettati con C (possiede solo il figlio B), si genera un nuovo sotto-ramo a partire da $A:2$: $C:1 \rightarrow D:1 \rightarrow E:1$. La Header Table estende le liste per C, D, E.

5. **TID 4:** (A, D, E): Da `null` si transita per A, il cui contatore viene aggiornato ad $A:3$. Poiché A non ha un figlio diretto D, si crea la diramazione $D:1 \rightarrow E:1$ direttamente sotto $A:3$.

6. **TID 5:** (A, B, C): Il cammino (A, B) è già presente. Si incrementa A (che diventa $A:4$) e il figlio B (che passa da $B:1$ a $B:2$). Da $B:2$ si crea il nuovo figlio $C:1$.

7. **TID 6:** (A, B, C, D): Il percorso (A, B, C) esiste già: si incrementa $A \rightarrow A:5$, $B \rightarrow B:3$, $C \rightarrow C:2$. Sotto $C:2$ si crea il nodo foglia $D:1$.

8. **TID 7:** (B, C): La transazione inizia da B sotto la radice `null`: si incrementa B da $B:1$ a $B:2$, e il figlio C da $C:1$ a $C:2$.

9. **TID 8:** (A, B, C): Si percorre il ramo sinistro: $A \rightarrow A:6$, $B \rightarrow B:4$, $C \rightarrow C:3$.

10. **TID 9:** (A, B, D): Si transita per $A \rightarrow A:7$, $B \rightarrow B:5$. Da $B:5$, non essendovi figli etichettati con D, si crea un nuovo ramo diretto $D:1$.

11. **TID 10:** (B, C, E): Si percorre il ramo destro: $B \rightarrow B:3$, $C \rightarrow C:3$, e sotto $C:3$ si crea il nuovo nodo foglia $E:1$.

La Figura 13.1 illustra l'architettura finale dell'FP-tree così generato con la relativa Header Table.

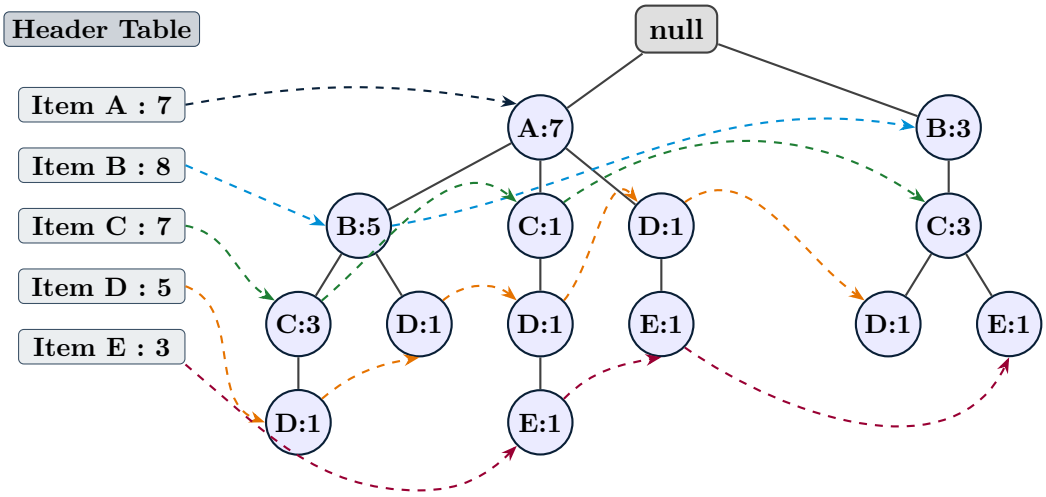


Figura 13.1: FP-tree completo generato dalle 10 transazioni con ordinamento lessicografico ($A \prec B \prec C \prec D \prec E$). Ciascuna riga della Header Table punta al primo nodo del relativo item nell'albero. Le frecce tratteggiate e colorate rappresentano le liste concatenate (*node-links*) che collegano tutte le istanze dello stesso item da sinistra verso destra.

13.4 Fattori Determinanti la Dimensione dell'FP-Tree ed Euristica di Ordinamento

La complessità spaziale e la cardinalità complessiva dei nodi dell'FP-tree dipendono strettamente dal grado di comprimibilità intrinseco della base di dati:

- **Scenario Ottimo Ideale**: Nel caso in cui tutte le transazioni all'interno del database contengano esattamente la medesima sequenza di item, l'albero collassa in un unico ramo discendente dalla radice. In questo regime, il fattore di compattazione è asintoticamente massimo e l'impronta di memoria è minimale.
- **Scenario Pessimo Degenerato**: Nel caso in cui tutte le transazioni siano reciprocamente disgiunte e non condividano alcun prefisso comune, il numero di nodi allocati eguaglia esattamente il numero totale di occorrenze di item presenti nell'archivio. In tale circostanza, l'FP-tree non apporta alcuna compressione e introduce un sovraccarico di memoria derivante dai puntatori strutturali ai figli e dai collegamenti orizzontali della Header Table.

13.4.1 Euristica: Ordinamento per Supporto Decrescente

La profondità, l'ampiezza e il tasso di diramazione (*branching factor*) dell'albero dipendono in maniera determinante dall'ordine scelto per gli item. Per massimizzare la probabilità statistica che transazioni eterogenee condividano la porzione iniziale del cammino, si applica l'**euristica di ordinamento per frequenza decrescente** [18, 2].

L'adozione di questa euristica richiede una prima scansione preliminare dell'intero database volta a calcolare il supporto assoluto $\sigma(i)$ di ciascun singolo item:

$$\sigma(B) = 8, \quad \sigma(A) = 7, \quad \sigma(C) = 7, \quad \sigma(D) = 5, \quad \sigma(E) = 3 \tag{13.2}$$

Ordinando gli elementi in senso decrescente di supporto (utilizzando l'ordine lessicografico come criterio di spareggio tra A e C), si ottiene la relazione d'ordine ottima:

$$B \succ A \succ C \succ D \succ E \tag{13.3}$$

Riorganizzando le 10 transazioni originali secondo questo criterio, si ottiene la mappatura strutturata riportata nella Tabella 13.2.

Tabella 13.2: Riorganizzazione delle transazioni in ordine di frequenza decrescente (B, A, C, D, E).

TID	Itemset Originario	Sequenza per Frequenza Decrescente
1	$\{A, B\}$	(B, A)
2	$\{B, C, D\}$	(B, C, D)
3	$\{A, C, D, E\}$	(A, C, D, E)
4	$\{A, D, E\}$	(A, D, E)
5	$\{A, B, C\}$	(B, A, C)
6	$\{A, B, C, D\}$	(B, A, C, D)
7	$\{B, C\}$	(B, C)
8	$\{A, B, C\}$	(B, A, C)
9	$\{A, B, D\}$	(B, A, D)
10	$\{B, C, E\}$	(B, C, E)

Vantaggio Topologico dell'Euristica di Frequenza

Poiché l'item B compare nell'80% delle transazioni complessive (8 su 10), collocarlo sistematicamente in testa assicura che ben 8 transazioni su 10 condividano il medesimo nodo iniziale B : 8 collocato direttamente al di sotto della radice `null`. Questo riduce drasticamente il numero di diramazioni primarie dell'albero, incrementando il fattore di compattazione e riducendo il tempo di attraversamento durante l'estrazione.

13.5 Estrazione dei Pattern Frequenti con FP-Growth (Divide and Conquer)

Una volta istanziato l'FP-tree compresso, l'algoritmo FP-Growth individua l'insieme completo degli itemset frequenti adottando una decomposizione ricorsiva di tipo *divide-and-conquer* [18]:

1. **Partizione disgiunta dello spazio**: Si scompone il problema globale analizzando gli item in **ordine inverso rispetto alla sequenza di inserimento** (dal meno frequente al più frequente: prima E, poi D, C, B e infine A). Ciascun sotto-problema identifica tutti e soli gli itemset che terminano con quello specifico suffisso, garantendo partizioni mutualmente disgiunte dell'intero spazio delle soluzioni.

2. **Costruzione della base condizionale**: Per ciascun suffisso X, si determinano i cammini di prefisso risalendo dai nodi fisici di X verso la radice `null` dell'albero, aggregando i conteggi di supporto per formare la **Conditional Pattern Base** (base di pattern condizionale) di X.

3. **Generazione ricorsiva (Conditional FP-tree)**: A partire dalla base condizionale, si eliminano gli elementi con supporto locale inferiore a minsup e si costruisce il relativo **Conditional FP-tree**. L'estrazione dei pattern non produce soltanto estensioni dirette, ma combina il suffisso X con tutti i pattern frequenti ricavati ricorsivamente all'interno dell'albero condizionale. Il procedimento viene reiterato in profondità estendendo progressivamente il suffisso (ad esempio, da $\{E\}$ verso $\{D, E\}$ e successivamente $\{A, D, E\}$) fino a quando l'albero condizionale non si riduce a un singolo cammino o diviene vuoto.

La Figura 13.2 illustra la decomposizione ad albero dello spazio di ricerca, evidenziando la disgiunzione dei rami primari per suffisso e l'esplorazione gerarchica in profondità del divide-and-conquer.

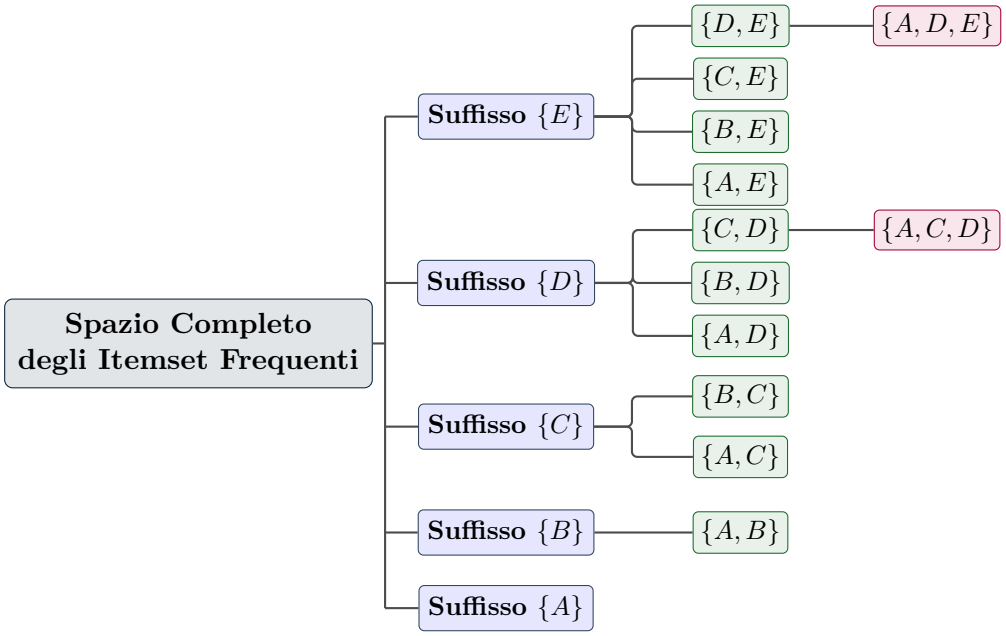


Figura 13.2: Decomposizione gerarchica dello spazio di ricerca mediante partizionamento per suffissi disgiunti in FP-Growth. La scomposizione divide-and-conquer elabora ordinatamente i suffissi (da E verso A); ciascun ramo genera unicamente pattern che terminano con quel suffisso, approfondendo l'esplorazione nei sotto-alberi condizionali locali (es. $\{E\} \to \{D, E\} \to \{A, D, E\}$ e $\{D\} \to \{C, D\} \to \{A, C, D\}$) senza alcuna duplicazione o interferenza tra partizioni disgiunte.

13.5.1 Fasi Operative per un Suffisso Generico X

Per ciascun elemento X selezionato come suffisso corrente dall'Header Table, l'algoritmo esegue due fasi analitiche fondamentali:

- **Fase 1 (Costruzione della Base Condizionale di Pattern e Calcolo del Supporto)**: Risalendo dai nodi X verso la radice `null`, si raccolgono tutti i cammini di prefisso (*prefix paths*). Il supporto cumulativo di X è dato dalla somma algebrica dei contatori associati a tutti i nodi fisici con etichetta X. Se tale supporto risulta inferiore a minsup, il suffisso X viene potato e il sotto-albero non viene esplorato.
- **Fase 2 (Costruzione del Conditional FP-tree)**: Se X soddisfa la condizione di frequenza, si costruisce il relativo **albero condizionale**:
    a. *Ricalcolo e propagazione del supporto*: I nodi antenati nei cammini di prefisso non conservano il loro conteggio originale, bensì unica-

mente la quota parte di supporto conferita dalle transazioni che contengono X.

Proprietà 13.5.1 (Propagazione del Supporto nei Cammini Condizionali)**.** *Il supporto ricalcolato di un generico nodo antenato Y all'interno della base condizionale di X è pari alla somma dei conteggi di supporto delle sole foglie X che discendono direttamente da Y.*

b. *Potatura dei nodi non frequenti*: Tutti gli item antenati il cui conteggio di supporto ricalcolato risulta inferiore a minsup vengono fisicamente eliminati dall'albero condizionale.

c. *Troncamento di X e ricorsione*: Il nodo suffisso X viene eliminato dalla base dell'albero (avendo terminato il suo compito di ancoraggio) e si invoca ricorsivamente l'algoritmo FP-Growth sulla struttura condizionale risultante.

13.6 Traccia di Esecuzione Completa sul Suffisso E (minsup $= 2$)

Si consideri l'FP-tree lessicografico formalizzato nella Sezione 13.3 (Figura 13.1) e si imponga una soglia di supporto minimo pari a:

$$\text{minsup} = 2 \tag{13.4}$$

13.6.1 Fase 1: Identificazione dei Prefix Paths e Supporto di E

Attraverso i puntatori della Header Table associati all'item E, si identificano esattamente 3 nodi foglia, corrispondenti ai seguenti tre cammini di prefisso:

$$\text{Cammino 1:} \quad \text{null} \to A \to C \to D \to E : 1 \tag{13.5}$$
$$\text{Cammino 2:} \quad \text{null} \to A \to D \to E : 1 \tag{13.6}$$
$$\text{Cammino 3:} \quad \text{null} \to B \to C \to E : 1 \tag{13.7}$$

Sommando i contatori delle tre foglie si ottiene il supporto totale di E:

$$\text{support}(E) = 1 + 1 + 1 = 3 \tag{13.8}$$

Poiché $3 \geq \text{minsup} = 2$, l'itemset singoletto $\{E\}$ **è frequente** (supporto 3).

13.6.2 Fase 2: Ricalcolo del Supporto nel Conditional Tree di E

Nel database complessivo, i nodi A, B, C possiedono conteggi elevati (es. $A : 7$, $B : 5$) determinati da transazioni prive dell'item E. Nella costruzione dell'albero condizionale per E, tali valori devono essere ricalcolati propagando esclusivamente il supporto conferito dalle foglie E:

- **Ramo Destro (null $\rightarrow B \rightarrow C \rightarrow E : 1$)**: La foglia $E : 1$ propaga il proprio valore al nodo genitore C, che assume supporto condizionale 1; a sua volta, C propaga il valore al nodo B, che assume supporto condizionale 1.
- **Ramo Sinistro**:
  - Dal cammino null $\rightarrow A \rightarrow C \rightarrow D \rightarrow E : 1$, la foglia propaga 1 a D, il quale propaga 1 a C, che a sua volta propaga 1 ad A.
  - Dal cammino null $\rightarrow A \rightarrow D \rightarrow E : 1$, la foglia propaga 1 direttamente a D, che propaga 1 ad A.

  Il nodo A cumula il supporto di entrambi i rami sottostanti, assumendo un valore ricalcolato pari a:

$$\text{support}_{\text{cond}}(A) = 1 + 1 = 2 \tag{13.9}$$

13.6.3 Troncamento e Potatura per il Suffisso E

Una volta ricalcolati i supporti condizionali:

1. **Troncamento**: Vengono rimossi tutti i nodi fisici etichettati con l'item E.
2. **Potatura dei nodi non frequenti (support < 2)**:
   - Il nodo $B : 1$ derivante dal ramo destro possiede supporto $1 <$ minsup $= 2$, evidenziando che l'itemset $\{B, E\}$ compare solo una volta nel dataset. Il nodo $B : 1$ viene potato.
   - Analogamente, il nodo $C : 1$ (figlio di B) viene rimosso in quanto non supportato da frequenza sufficiente.

La struttura superstite definisce il **Conditional FP-tree per** E, mostrato nella Figura 13.3:

- Radice `null`;

- Ramo principale facente capo al nodo $A : 2$;
- Da $A : 2$ si originano due percorsi verso nodi D: uno attraverso $C : 1 \to D : 1$ e uno diretto $D : 1$;
- Un ramo isolato $C : 1$ adiacente a `null` (con conteggio 1).

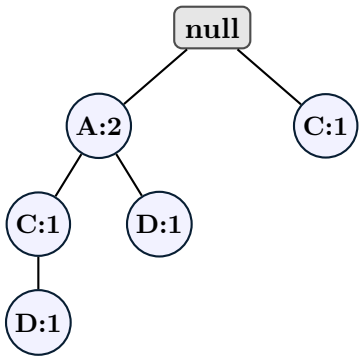


Figura 13.3: Conditional FP-tree per il suffisso E dopo ricalcolo del supporto, troncamento di E e potatura dei rami con frequenza insufficiente.

13.7 Esplorazione Ricorsiva dei Sotto-Problemi per il Suffisso E

A partire dal Conditional FP-tree di E, FP-Growth analizza ricorsivamente i suffissi candidati in ordine inverso: prima D, poi C, infine A.

13.7.1 Sotto-problema: Suffisso $\{D, E\}$

1. **Prefix Paths per D nel Conditional Tree di E**: Nell'albero condizionale di E sono presenti esattamente due nodi D:

$$\text{null} \to A \to C \to D : 1 \tag{13.10}$$
$$\text{null} \to A \to D : 1 \tag{13.11}$$

   Supporto complessivo di $\{D, E\} = 1 + 1 = 2 \geq \text{minsup} = 2$. L'itemset $\{D, E\}$ **è frequente**.

2. **Costruzione del Conditional FP-tree per $\{D, E\}$**: Risalendo dai due nodi D, il nodo C riceve supporto condizionale 1, mentre il nodo A riceve $1 + 1 = 2$.

3. **Troncamento e Potatura**: Si eliminano i nodi D. Il nodo C ha supporto $1 < 2$ e viene potato. L'albero condizionale risultante per $\{D, E\}$ contiene unicamente il nodo $A : 2$.

4. **Estrazione**: Poiché A ha supporto $2 \geq$ minsup, si estrae l'itemset frequente ternario:

$$\{\mathbf{A}, \mathbf{D}, \mathbf{E}\} \quad \text{(supporto 2)} \tag{13.12}$$

   Non essendovi ulteriori prefissi da esplorare al di sopra di A, la ricorsione su questo ramo termina.

13.7.2 Sotto-problema: Suffisso $\{C, E\}$

1. **Prefix Paths per C nel Conditional Tree di E**: Si rilevano due nodi etichettati con C: uno lungo il cammino null $\to A \to C : 1$ e uno isolato null $\to C : 1$. Supporto cumulativo di $\{C, E\} = 1+1 = 2 \geq \text{minsup} = 2$. L'itemset $\{C, E\}$ **è frequente**.

2. **Costruzione del Conditional FP-tree per** $\{C, E\}$: Il nodo A compare unicamente nel primo cammino, ricevendo pertanto un supporto condizionale pari a 1.

3. **Troncamento e Potatura**: Rimossi i nodi C, il nodo residuo $A : 1$ ha supporto $1 < \text{minsup} = 2$ e viene immediatamente potato. L'albero condizionale per $\{C, E\}$ collassa nella sola radice fittizia `null`. Nessun ulteriore itemset viene generato da questa diramazione.

13.7.3 Sotto-problema: Suffisso $\{A, E\}$

1. **Prefix Path per A nel Conditional Tree di E**: È presente il solo nodo $A : 2$ direttamente agganciato alla radice `null`. Supporto di $\{A, E\} = 2 \geq \text{minsup} = 2$. L'itemset $\{A, E\}$ **è frequente**.

2. Non essendovi ulteriori nodi intermedi tra A e `null`, la procedura non genera ulteriori estensioni e l'esplorazione del suffisso A si conclude.

13.7.4 Riepilogo dei Pattern Frequenti Estratti per il Suffisso E

L'esplorazione esaustiva della componente E conduce all'estrazione formale dei 5 itemset frequenti:

$$\mathcal{F}_E = \Big\{\{E\},\ \{D, E\},\ \{C, E\},\ \{A, E\},\ \{A, D, E\}\Big\} \tag{13.13}$$

13.8 Elaborazione del Suffisso Successivo: Suffisso D

Esaurita la componente E, FP-Growth passa al suffisso immediatamente precedente nell'ordinamento: D.

1. **Identificazione delle Istanze e Supporto di D**: Seguendo la catena dei nodi collegati alla voce D nella Header Table dell'FP-tree originale (Figura 13.1), si individuano 5 nodi fisici distinti (ciascuno con supporto locale 1):

   - Tre foglie nel sotto-albero originato da A:

   $$\text{null} \to A \to B \to D : 1 \tag{13.14}$$
   $$\text{null} \to A \to B \to C \to D : 1 \tag{13.15}$$
   $$\text{null} \to A \to C \to D : 1 \tag{13.16}$$

   - Due foglie nel sotto-albero originato da B:

   $$\text{null} \to B \to C \to D : 1 \tag{13.17}$$
   $$\text{null} \to B \to D : 1 \tag{13.18}$$

   Supporto complessivo: $\text{support}(D) = 1+1+1+1+1 = 5 \geq \text{minsup} = 2$. L'itemset $\{D\}$ **è frequente**.

2. **Ricalcolo del Supporto Condizionale dei Prefissi di D**: Propagando i conteggi dal basso verso l'alto lungo i 5 percorsi:

   - Il nodo C sotto B (ramo sinistro) riceve 1;
   - Il nodo B sotto A somma i contributi di C (1) e del proprio figlio diretto D (1), totalizzando $\text{support}_{\text{cond}}(B) = 2$;
   - Il nodo A assomma tutti i contributi dei cammini sottostanti contenenti D, ottenendo $\text{support}_{\text{cond}}(A) = 1 + 1 + 1 = 3$;
   - Nel ramo destro, il nodo C sotto B riceve 1, e il nodo B sotto `null` riceve $1 + 1 = 2$.

3. **Troncamento e Potatura**: Si rimuovono tutti i nodi etichettati con D. I nodi con supporto condizionale insufficiente (< 2) vengono eliminati durante le successive invocazioni ricorsive sui suffissi $\{C, D\}$, $\{B, D\}$ e $\{A, D\}$.

13.9 Proprietà e Complessità dell'Algoritmo FP-Growth

Dall'analisi teorica e operativa emergono tre proprietà strutturali che rendono FP-Growth uno degli algoritmi più performanti nell'analisi delle associazioni [18]:

- **Mutua Disgiunzione dei Sotto-Problemi**: La scomposizione combinatoria fondata sulla partizione dei suffissi garantisce che lo spazio di ricerca sia suddiviso in rami perfettamente disgiunti. Ne consegue che **nessun itemset frequente viene mai generato o valutato più di una volta**, azzerando la ridondanza algoritmica.
- **Accoppiamento Contestuale del Calcolo del Supporto**: Il conteggio delle frequenze non richiede interrogazioni ripetute del database o scansioni su disco: il supporto effettivo viene ricavato direttamente per aggregazione strutturale durante l'attraversamento dei cammini condizionali in memoria RAM.
- **Sensibilità al Fattore di Compattazione (*Compaction Factor*)**: L'efficienza di FP-Growth dipende dal grado di densità del dataset. In dataset fortemente densi, l'elevata sovrapposizione delle transazioni genera un albero compatto con tempi di elaborazione trascurabili; al contrario, in dataset estremamente sparsi con vocabolari ampi e transazioni prive di correlazione, l'albero tende a ramificarsi eccessivamente (*bushy tree*), incrementando il carico computazionale per la gestione dei sotto-alberi condizionali.

13.10 Esercizio 1: Simulazione Geometrica e Analitica di K-Means

L'algoritmo **K-Means** (formalizzato nel Capitolo 7) mira a ripartire uno spazio vettoriale in K partizioni rigide minimizzando l'errore quadratico totale (SSE). In questa sezione viene condotta una simulazione deterministica passo-passo nello spazio cartesiano continuo $\mathbb{R}^2$.

13.10.1 Configurazione Iniziale e Coordinate del Dataset

Si consideri un dataset $\mathcal{D}$ composto da 9 punti bidimensionali $P_0, \ldots, P_8$:

$$\begin{aligned} &P_0 = (3,1), && P_1 = (7,5), && P_2 = (6,2) \\ &P_3 = (7,1), && P_4 = (9,7), && P_5 = (8,8) \\ &P_6 = (2,4), && P_7 = (8,6), && P_8 = (4,4) \end{aligned} \tag{13.19}$$

Si impone $K = 2$ e si selezionano a priori i centroidi iniziali ancorandoli a due punti del dataset:

- **Centroide 1 ($C_1^{(0)}$)**: posizionato su $P_2 \implies C_1^{(0)} = (6.00, 2.00)$;
- **Centroide 2 ($C_0^{(0)}$)**: posizionato su $P_1 \implies C_0^{(0)} = (7.00, 5.00)$.

13.10.2 Iterazione 0: Assegnazione alle Regioni di Voronoi

1. **Determinazione della Retta Separatrice (*Bisecting Line*)**: La frontiera di decisione geometrica che separa i punti più vicini a $C_1^{(0)}$ da quelli più vicini a $C_0^{(0)}$ coincide con l'**asse del segmento** che congiunge i due centroidi.

   - Vettore congiungente: $\mathbf{v} = C_0^{(0)} - C_1^{(0)} = (7-6,\ 5-2) = (1,3)$.
   - Punto medio del segmento:

   $$M = \left(\frac{6+7}{2},\ \frac{2+5}{2}\right) = (6.50,\ 3.50) \tag{13.20}$$

   - Equazione cartesiana dell'asse (retta ortogonale a $\mathbf{v}$ passante per M):

   $$\begin{aligned} (x-6.5) + 3(y-3.5) = 0 &\implies x + 3y - 17 = 0 \\ &\implies y = -\frac{1}{3}x + \frac{17}{3} \end{aligned} \tag{13.21}$$

2. **Assegnazione dei Punti ai Cluster**: Calcolando la distanza euclidea o valutando il segno della funzione discriminante lineare $f(x,y) = x + 3y - 17$:

   - **Cluster 1 (regione di C_1)**: punti per cui $f(x,y) < 0 \implies \{P_0, P_2, P_3, P_6, P_8\}$ (5 punti).

- **Cluster 2 (regione di C_0)**: punti per cui $f(x,y) > 0 \implies \{P_1, P_4, P_5, P_7\}$ (4 punti).

3. **Ricalcolo dei Baricentri ($C_1^{(1)}$ e $C_0^{(1)}$)**:

$$C_1^{(1)} = \left(\frac{3+6+7+2+4}{5},\ \frac{1+2+1+4+4}{5}\right)$$
$$= \left(\frac{22}{5},\ \frac{12}{5}\right) = (4.40,\ 2.40) \tag{13.22}$$
$$C_0^{(1)} = \left(\frac{7+9+8+8}{4},\ \frac{5+7+8+6}{4}\right)$$
$$= \left(\frac{32}{4},\ \frac{26}{4}\right) = (8.00,\ 6.50) \tag{13.23}$$

13.10.3 Iterazione 1 e Criterio di Arresto per Convergenza

1. **Nuova Retta Separatrice**: Il vettore che unisce i nuovi centroidi è $\mathbf{v}' = (8.00 - 4.40,\ 6.50 - 2.40) = (3.60,\ 4.10)$. Il nuovo punto medio è $M' = \left(\frac{4.40+8.00}{2},\ \frac{2.40+6.50}{2}\right) = (6.20,\ 4.45)$.

2. **Verifica delle Assegnazioni**: Rivalutando le distanze euclidee di tutti i 9 punti rispetto a $C_1^{(1)} = (4.40, 2.40)$ e $C_0^{(1)} = (8.00, 6.50)$, la partizione risulta perfettamente invariata:

   - Punti associati a $C_1^{(1)}$: $\{P_0, P_2, P_3, P_6, P_8\}$;
   - Punti associati a $C_0^{(1)}$: $\{P_1, P_4, P_5, P_7\}$.

3. **Arresto**: La composizione dei cluster non subisce alcuna alterazione rispetto all'iterazione precedente; l'algoritmo K-Means termina formalmente per convergenza della funzione obiettivo.

La Figura 13.4 visualizza la configurazione geometrica finale dei cluster e la frontiera lineare di separazione.

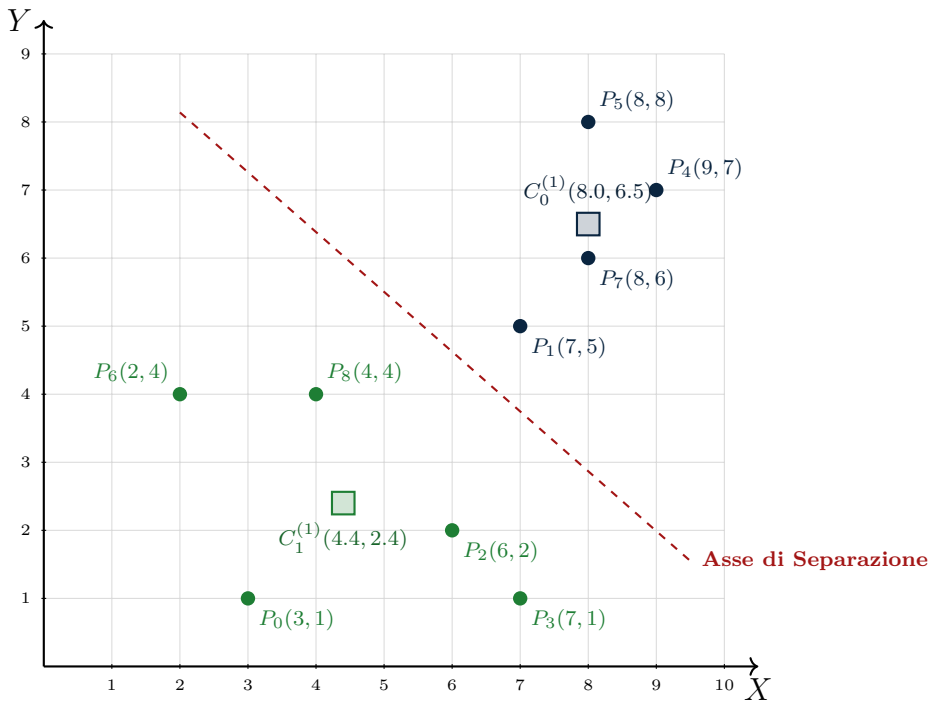


Figura 13.4: Simulazione di K-Means ($K = 2$): partizione finale dei punti nei due cluster e asse di separazione lineare ortogonale al segmento congiungente i centroidi $C_1^{(1)}(4.4, 2.4)$ e $C_0^{(1)}(8.0, 6.5)$.

13.11 Concetto Introduttivo di DBSCAN su Spazio Metrico Continuo

Applicando l'algoritmo basato su densità **DBSCAN** (approfondito nel Capitolo 8) sullo stesso dataset, il paradigma di raggruppamento muta radicalmente rispetto a K-Means: non si ricerca un partizionamento convesso basato su prototipi baricentrici, bensì si individuano componenti connesse di densità metrica locale.

Si consideri la parametrizzazione tipica:

$$\varepsilon = 1.8, \qquad \text{MinPts} = 3 \tag{13.24}$$

dove MinPts $= 3$ include il punto in esame stesso. La costante $\varepsilon = 1.8$ definisce il raggio metrico euclideo dell'intorno $\mathcal{N}_\varepsilon(p) = \{q \in \mathcal{D} \mid \text{dist}(p, q) \leq \varepsilon\}$:

- Un punto p è classificato come **Core Point** se $|\mathcal{N}_\varepsilon(p)| \geq 3$;

- Un punto p è un **Border Point** se $|\mathcal{N}_\varepsilon(p)| < 3$ ma risiede nell'intorno ε di almeno un Core Point;
- Un punto p è classificato come **Noise** (rumore o outlier) se non soddisfa alcuna delle due condizioni precedenti.

A differenza di K-Means, DBSCAN non richiede di specificare il numero K di cluster a priori ed è in grado di escludere naturalmente i punti isolati (come $P_6 = (2, 4)$ qualora la densità locale circostante non raggiunga la soglia richiesta).

13.12 Esercizio 2: Simulazione di K-Means con Fenomeno di Migrazione dei Punti

Si consideri un secondo dataset composto da 10 punti in $\mathbb{R}^2$ per esaminare la dinamica temporale di riassegnazione dei dati al variare delle coordinate dei baricentri:

$$\begin{aligned} &P_0 = (3, 3), \quad P_1 = (5, 2), \quad P_2 = (6, 0), \quad P_3 = (7, 4), \quad P_4 = (4, 9) \\ &P_5 = (1, 4), \quad P_6 = (0, 0), \quad P_7 = (6, 8), \quad P_8 = (1, 0), \quad P_9 = (5, 9) \end{aligned} \tag{13.25}$$

Si impone $K = 2$ e si scelgono come centroidi iniziali:

- $C_0^{(0)} = P_1 \implies (5.00,\ 2.00)$
- $C_1^{(0)} = P_5 \implies (1.00,\ 4.00)$

13.12.1 Iterazione 0: Partizione Iniziale

1. **Asse Separatore e Assegnazione**: Tracciando l'asse passante per il punto medio $M = (3.00, 3.00)$ perpendicolare al vettore $(5 - 1, 2 - 4) = (4, -2)$:
   - **Cluster 1 (associato a C_0)**: $\{P_0, P_1, P_2, P_3, P_7\}$ (5 punti);
   - **Cluster 2 (associato a C_1)**: $\{P_4, P_5, P_6, P_8, P_9\}$ (5 punti).
2. **Aggiornamento dei Centroidi al Passo 1**:

$$C_0^{(1)} = \left(\frac{3+5+6+7+6}{5},\ \frac{3+2+0+4+8}{5}\right) = (5.40,\ 3.40) \tag{13.26}$$

$$C_1^{(1)} = \left(\frac{4+1+0+1+5}{5},\ \frac{9+4+0+0+9}{5}\right) = (2.20,\ 4.40) \tag{13.27}$$

13.12.2 Iterazione 1: Fenomeno di Migrazione del Punto P_0

Al termine del ricalcolo dei centroidi al passo 1, la posizione dei poli attrattori subisce una traslazione. Si rivalutano rigorosamente le distanze euclidee quadrate del punto critico di confine $P_0 = (3, 3)$ rispetto a entrambi i nuovi centroidi:

$$\text{dist}^2\Big(P_0,\ C_0^{(1)}\Big) = (3 - 5.40)^2 + (3 - 3.40)^2 = (-2.40)^2 + (-0.40)^2 = \mathbf{5.92} \tag{13.28}$$

$$\text{dist}^2\Big(P_0,\ C_1^{(1)}\Big) = (3 - 2.20)^2 + (3 - 4.40)^2 = (0.80)^2 + (-1.40)^2 = \mathbf{2.60} \tag{13.29}$$

Poiché $\text{dist}^2(P_0, C_1^{(1)}) = 2.60 < 5.92 = \text{dist}^2(P_0, C_0^{(1)})$, il punto P_0 non appartiene più al bacino di attrazione di C_0: si verifica formalmente il fenomeno della **migrazione del punto P_0 dal Cluster 1 al Cluster 2**.

La nuova composizione dei cluster risulta:

- **Nuovo Cluster 1**: $\{P_1, P_2, P_3, P_7\}$ (4 punti);
- **Nuovo Cluster 2**: $\{P_0, P_4, P_5, P_6, P_8, P_9\}$ (6 punti).

Si ricalcolano i nuovi baricentri $C_0^{(2)}$ e $C_1^{(2)}$:

$$C_0^{(2)} = \left(\frac{5+6+7+6}{4},\ \frac{2+0+4+8}{4}\right) = (6.00,\ 3.50) \tag{13.30}$$

$$C_1^{(2)} = \left(\frac{3+4+1+0+1+5}{6},\ \frac{3+9+4+0+0+9}{6}\right) \approx (2.33,\ 4.17) \tag{13.31}$$

13.12.3 Iterazione 2 e Convergenza Stabile

Calcolando l'asse separatore tra $C_0^{(2)} = (6.00, 3.50)$ e $C_1^{(2)} = (2.33, 4.17)$, si constata che nessun punto modifica la propria appartenenza tra l'iterazione 1 e l'iterazione 2:

- Cluster 1 stabile: $\{P_1, P_2, P_3, P_7\}$;
- Cluster 2 stabile: $\{P_0, P_4, P_5, P_6, P_8, P_9\}$.

L'algoritmo termina per convergenza della partizione.

13.13 Fondamenti sui Vincoli Temporali nelle Sequenze (Min-Gap e Max-Gap)

Nei problemi di **Sequential Pattern Mining** (approfonditi nel Capitolo 12), una sequenza temporale di input S è modellata come una lista ordinata di insiemi di eventi (itemset), ciascuno associato a un indice temporale discreto (*time-stamp*) t [2, 16, 17]:

$$S = \langle E_{t_0}, E_{t_1}, \ldots, E_{t_n} \rangle \tag{13.32}$$

Una generica sotto-sequenza target $W = \langle e_1, e_2, \ldots, e_m \rangle$ compare all'interno di S se e solo se esiste una sequenza strettamente crescente di indici temporali $\langle t_{k_1}, t_{k_2}, \ldots, t_{k_m} \rangle$ tali per cui:

$$t_{k_1} < t_{k_2} < \cdots < t_{k_m} \quad \text{e} \quad e_j \subseteq E_{t_{k_j}} \quad (\forall j \in \{1, \ldots, m\}) \tag{13.33}$$

Nelle applicazioni ingegneristiche reali (diagnostica dei guasti, biologia computazionale, percorsi web), identificare eventi arbitrariamente distanti nel tempo produce pattern spuri privi di rilevanza causale. Si introducono pertanto **vincoli temporali inter-evento** (*gap constraints*) [17]:

Definizione 13.13.1 (Vincolo di Min-Gap)**.** Il vincolo min-gap $= g_{\min}$ impone che tra due eventi contigui della sottosequenza debba intercorrere una separazione temporale strettamente superiore al valore prefissato:

$$t_{k_j} - t_{k_{j-1}} > g_{\min} \tag{13.34}$$

Nel caso particolare didattico in cui min-gap $= 1$, la disuguaglianza $t_{k_j} - t_{k_{j-1}} > 1$ impone $\Delta t \geq 2$, escludendo eventi contigui che accadono a distanza temporale unitaria.

Definizione 13.13.2 (Vincolo di Max-Gap)**.** Il vincolo max-gap $= g_{\max}$ impone che due eventi consecutivi del pattern debbano realizzarsi entro un intervallo temporale massimo ammissibile:

$$t_{k_j} - t_{k_{j-1}} \leq g_{\max} \tag{13.35}$$

13.14 Esercizio 1: Pattern Sequenziali con Vincolo Min-Gap

Si consideri la sequenza temporale di riferimento:

$$\begin{aligned} S = \Big\langle & \{A\}_{t=0},\ \{B, F\}_{t=1},\ \{E\}_{t=2},\ \{A, B\}_{t=3}, \\ & \{A, C, D\}_{t=4},\ \{F\}_{t=5},\ \{B, E\}_{t=6},\ \{C, D\}_{t=7} \Big\rangle \end{aligned} \tag{13.36}$$

Si determinano le occorrenze valide per diverse sotto-sequenze confrontando lo scenario senza vincoli con lo scenario soggetto al vincolo min-gap = 1 ($\Delta t > 1$).

13.14.1 Sotto-sequenza $\langle\{B\},\{E\}\rangle$

- Distribuzione temporale degli item: B appare a $t \in \{1,3,6\}$; E appare a $t \in \{2,6\}$.
- **Occorrenze prive di vincoli ($t_B < t_E$)**:

$$\begin{aligned} t_B = 1 &\implies t_E = 2 \implies \langle 1,2\rangle \\ t_B = 1 &\implies t_E = 6 \implies \langle 1,6\rangle \\ t_B = 3 &\implies t_E = 6 \implies \langle 3,6\rangle \end{aligned} \tag{13.37}$$

  Occorrenze non vincolate: $\langle 1,2\rangle$, $\langle 1,6\rangle$, $\langle 3,6\rangle$.

- **Verifica del vincolo min-gap = 1 ($t_E - t_B > 1$)**:
  - Per $\langle 1,2\rangle$: $2-1=1 \not> 1 \implies$ **violato**;
  - Per $\langle 1,6\rangle$: $6-1=5>1 \implies$ **valido**;
  - Per $\langle 3,6\rangle$: $6-3=3>1 \implies$ **valido**.

  Occorrenze valide con min-gap = 1: $\langle \mathbf{1},\mathbf{6}\rangle$, $\langle \mathbf{3},\mathbf{6}\rangle$.

13.14.2 Sotto-sequenza $w_1 = \langle\{A\},\{B\},\{E\}\rangle$

- Item A appare a $t \in \{0,3,4\}$.
- Da $t_A = 0$, combinandosi con le occorrenze ordinate valide di $\langle\{B\},\{E\}\rangle$, si ricavano le terne:

$$\langle 0,1,2\rangle, \quad \langle 0,1,6\rangle, \quad \langle 0,3,6\rangle \tag{13.38}$$

  Per $t_A = 3$ e $t_A = 4$ non esistono coppie $\langle B,E\rangle$ successive.

- **Verifica min-gap** = 1:
  - In $\langle 0,1,2\rangle$: $1-0=1 \not> 1$ e $2-1=1 \not> 1 \implies$ violato.
  - In $\langle 0,1,6\rangle$: $1-0=1 \not> 1 \implies$ violato.
  - In $\langle 0,3,6\rangle$: $3-0=3>1$ e $6-3=3>1 \implies$ **valido**.

  Esito finale: $\langle \mathbf{0},\mathbf{3},\mathbf{6}\rangle$.

13.14.3 Sotto-sequenza $w_2 = \langle \{B\}, \{D\} \rangle$

- B a $t \in \{1, 3, 6\}$; D a $t \in \{4, 7\}$.
- Senza vincoli: $\langle 1, 4 \rangle, \langle 1, 7 \rangle, \langle 3, 4 \rangle, \langle 3, 7 \rangle, \langle 6, 7 \rangle$.
- Con min-gap = 1:
  - $\langle 1, 4 \rangle$: $4 - 1 = 3 > 1 \implies$ valido;
  - $\langle 1, 7 \rangle$: $7 - 1 = 6 > 1 \implies$ valido;
  - $\langle 3, 4 \rangle$: $4 - 3 = 1 \not> 1 \implies$ violato;
  - $\langle 3, 7 \rangle$: $7 - 3 = 4 > 1 \implies$ valido;
  - $\langle 6, 7 \rangle$: $7 - 6 = 1 \not> 1 \implies$ violato.

  Esito finale: $\langle \mathbf{1}, \mathbf{4} \rangle$, $\langle \mathbf{1}, \mathbf{7} \rangle$, $\langle \mathbf{3}, \mathbf{7} \rangle$.

13.14.4 Sotto-sequenza $w_3 = \langle \{F\}, \{E\}, \{C, D\} \rangle$

- F a $t \in \{1, 5\}$; E a $t \in \{2, 6\}$; $\{C, D\}$ a $t \in \{4, 7\}$.
- Senza vincoli: $\langle 1, 2, 4 \rangle, \langle 1, 2, 7 \rangle, \langle 1, 6, 7 \rangle, \langle 5, 6, 7 \rangle$.
- Con min-gap = 1: Ciascuna delle 4 tuple candidate presenta almeno una transizione temporale unitaria ($\Delta t = 1$): $\langle 1, 2, 4 \rangle$ e $\langle 1, 2, 7 \rangle$ violano il vincolo su $t_E - t_F = 1$; $\langle 1, 6, 7 \rangle$ viola il vincolo su $t_{CD} - t_E = 1$; $\langle 5, 6, 7 \rangle$ viola entrambi.
- Esito finale: **nessuna occorrenza valida (`none`)**.

13.15 Esercizio 2: Pattern Sequenziali con Vincolo Max-Gap

Si consideri la seconda sequenza temporale di input:

$$S = \Big\langle \{B, F\}_{t=0},\ \{A\}_{t=1},\ \{A, B\}_{t=2},\ \{C, D, F\}_{t=3},\ \{E\}_{t=4},\ \{B, E\}_{t=5},\ \{C, D\}_{t=6} \Big\rangle \tag{13.39}$$

Si valutano le occorrenze imponendo il vincolo max-gap = 4 ($\Delta t \leq 4$).

13.15.1 Sotto-sequenza $w_1 = \langle\{B\}, \{E\}\rangle$

- B a $t \in \{0, 2, 5\}$; E a $t \in \{4, 5\}$. La condizione $t_B < t_E$ esclude la co-occorrenza $t = 5$.
- Senza vincoli: $\langle 0, 4\rangle, \langle 0, 5\rangle, \langle 2, 4\rangle, \langle 2, 5\rangle$.
- Con max-gap $= 4$ ($t_E - t_B \leq 4$):
  - $\langle 0, 4\rangle$: $4 - 0 = 4 \leq 4 \implies$ valido;
  - $\langle 0, 5\rangle$: $5 - 0 = 5 > 4 \implies$ **violato**;
  - $\langle 2, 4\rangle$: $4 - 2 = 2 \leq 4 \implies$ valido;
  - $\langle 2, 5\rangle$: $5 - 2 = 3 \leq 4 \implies$ valido.

  Esito finale: $\langle \mathbf{0}, \mathbf{4}\rangle$, $\langle \mathbf{2}, \mathbf{4}\rangle$, $\langle \mathbf{2}, \mathbf{5}\rangle$.

13.15.2 Sotto-sequenza $w_2 = \langle\{B\}, \{D\}\rangle$

- B a $t \in \{0, 2, 5\}$; D a $t \in \{3, 6\}$.
- Senza vincoli: $\langle 0, 3\rangle, \langle 0, 6\rangle, \langle 2, 3\rangle, \langle 2, 6\rangle, \langle 5, 6\rangle$.
- Con max-gap $= 4$ ($t_D - t_B \leq 4$):
  - $\langle 0, 3\rangle$: $3 - 0 = 3 \leq 4 \implies$ valido;
  - $\langle 0, 6\rangle$: $6 - 0 = 6 > 4 \implies$ **violato**;
  - $\langle 2, 3\rangle$: $3 - 2 = 1 \leq 4 \implies$ valido;
  - $\langle 2, 6\rangle$: $6 - 2 = 4 \leq 4 \implies$ valido;
  - $\langle 5, 6\rangle$: $6 - 5 = 1 \leq 4 \implies$ valido.

  Esito finale: $\langle \mathbf{0}, \mathbf{3}\rangle$, $\langle \mathbf{2}, \mathbf{3}\rangle$, $\langle \mathbf{2}, \mathbf{6}\rangle$, $\langle \mathbf{5}, \mathbf{6}\rangle$.

13.15.3 Sotto-sequenza $w_3 = \langle\{F\}, \{B\}, \{C, D\}\rangle$

- F a $t \in \{0, 3\}$; B a $t \in \{0, 2, 5\}$; $\{C, D\}$ a $t \in \{3, 6\}$.
- Condizione d'ordine temporale stretto: $t_F < t_B < t_{CD}$.
- Occorrenze senza vincoli: $\langle 0, 2, 3\rangle, \langle 0, 2, 6\rangle, \langle 0, 5, 6\rangle, \langle 3, 5, 6\rangle$.
- Con max-gap $= 4$ ($(t_B - t_F \leq 4) \wedge (t_{CD} - t_B \leq 4)$):
  - $\langle 0, 2, 3\rangle$: $2 - 0 = 2 \leq 4$ e $3 - 2 = 1 \leq 4 \implies$ valido;

– $\langle 0, 2, 6\rangle$: $2 - 0 = 2 \leq 4$ e $6 - 2 = 4 \leq 4 \implies$ valido;
– $\langle 0, 5\rangle$: $5 - 0 = 5 > 4 \implies$ **violato**;
– $\langle 3, 5, 6\rangle$: $5 - 3 = 2 \leq 4$ e $6 - 5 = 1 \leq 4 \implies$ valido.

Esito finale: $\langle \mathbf{0}, \mathbf{2}, \mathbf{3}\rangle$, $\langle \mathbf{0}, \mathbf{2}, \mathbf{6}\rangle$, $\langle \mathbf{3}, \mathbf{5}, \mathbf{6}\rangle$.

13.16 Algoritmo GSP: Simulazione del Livello $k = 1$ su Dataset

L'algoritmo **GSP** (*Generalized Sequential Pattern*), introdotto da Ramakrishnan Srikant e Rakesh Agrawal nel 1996 [17], traspone la logica generativa *candidate-generation-and-test* di Apriori ai database di sequenze temporali.

Si consideri il dataset transazionale composto da $|\mathcal{D}| = 4$ sequenze:

$$\begin{aligned} s_1 &: \langle \{A\} \to \{B, C\} \to \{C\} \to \{D\}\rangle \\ s_2 &: \langle \{A, C\} \to \{B\} \to \{C\} \to \{C\}\rangle \\ s_3 &: \langle \{D\} \to \{C\} \to \{B\} \to \{C, D\}\rangle \\ s_4 &: \langle \{A, B\} \to \{D\} \to \{C\} \to \{C, D\} \to \{E\}\rangle \end{aligned} \tag{13.40}$$

Imponendo una soglia di supporto relativo minsup $= 60\%$, il conteggio assoluto minimo di sequenze richiedente supporto risulta:

$$\sigma_{\min} = \lceil |\mathcal{D}| \times \text{minsup} \rceil = \lceil 4 \times 0.60 \rceil = \lceil 2.4 \rceil = \mathbf{3} \text{ sequenze} \tag{13.41}$$

13.16.1 Calcolo dei 1-Pattern Frequenti (L_1)

Si valuta la frequenza di presenza di ciascun item distinto nelle 4 sequenze:

- **Item** A: compare in $s_1, s_2, s_4 \implies \sigma(A) = 3/4 = 75\% \geq 60\%$ (**Frequente**: $\langle \{A\}\rangle$);
- **Item** B: compare in $s_1, s_2, s_3, s_4 \implies \sigma(B) = 4/4 = 100\% \geq 60\%$ (**Frequente**: $\langle \{B\}\rangle$);
- **Item** C: compare in $s_1, s_2, s_3, s_4 \implies \sigma(C) = 4/4 = 100\% \geq 60\%$ (**Frequente**: $\langle \{C\}\rangle$);
- **Item** D: compare in $s_1, s_3, s_4 \implies \sigma(D) = 3/4 = 75\% \geq 60\%$ (**Frequente**: $\langle \{D\}\rangle$);
- **Item** E: compare unicamente in $s_4 \implies \sigma(E) = 1/4 = 25\% < 60\%$ (**Infrequente** $\implies$ eliminato).

L'insieme risultante delle 1-sequenze frequenti è:

$$L_1 = \Big\{\langle\{A\}\rangle,\ \langle\{B\}\rangle,\ \langle\{C\}\rangle,\ \langle\{D\}\rangle\Big\} \tag{13.42}$$

13.17 Algoritmo GSP: Fase di Pruning dei Candidati (C_4)

In un'esecuzione avanzata dell'algoritmo GSP, al termine dell'elaborazione di livello $k = 3$ è stato identificato l'insieme di **3-sequenze frequenti (L_3)** composto da 12 elementi:

1. $\langle\{A, B\} \to \{C\}\rangle$

2. $\langle\{A\} \to \{D\} \to \{C\}\rangle$

3. $\langle\{A, B\} \to \{D\}\rangle$

4. $\langle\{B\} \to \{C\} \to \{C\}\rangle$

5. $\langle\{A\} \to \{C, D\}\rangle$

6. $\langle\{B\} \to \{C\} \to \{D\}\rangle$

7. $\langle\{B\} \to \{C, D\}\rangle$

8. $\langle\{B\} \to \{D\} \to \{C\}\rangle$

9. $\langle\{A\} \to \{C\} \to \{C\}\rangle$

10. $\langle\{D\} \to \{C\} \to \{C\}\rangle$

11. $\langle\{A\} \to \{C\} \to \{D\}\rangle$

12. $\langle\{D\} \to \{C\} \to \{D\}\rangle$

Durante la successiva iterazione per $k = 4$, la procedura di generazione produce (tra gli altri) 5 candidati di lunghezza 4 appartenenti a C_4:

C1. $\langle\{A, B\} \to \{C, D\}\rangle$

C2. $\langle\{A\} \to \{D\} \to \{C\} \to \{D\}\rangle$

C3. $\langle\{B\} \to \{D\} \to \{C\} \to \{D\}\rangle$

C4. $\langle\{A, B\} \to \{D\} \to \{C\}\rangle$

C5. $\langle\{A, B\} \to \{C\} \to \{D\}\rangle$

13.17.1 Principio di Anti-Monotonicità del Pruning in GSP

Teorema 13.17.1 (Proprietà di Potatura in GSP)**.** *Una k-sequenza candidata $s \in C_k$ può essere frequente se e solo se* ***tutte le sue*** *k* ***sotto-sequenze di lunghezza*** *$k-1$ (ottenute rimuovendo esattamente un'occorrenza di item alla volta) appartengono all'insieme dei pattern frequenti del livello precedente L_{k-1}. Se anche una sola sotto-sequenza di lunghezza $k-1$ risulta assente da L_{k-1}, il candidato viene immediatamente eliminato (****potato a priori****), senza richiedere alcuna scansione del database.*

13.17.2 Analisi Rigorosa delle Sotto-sequenze per Ciascun Candidato

Candidato C1: $\langle\{A, B\} \to \{C, D\}\rangle$

Le sue 4 sotto-sequenze di lunghezza 3 si ottengono rimuovendo un item alla volta:

1. Rimozione di A: $\langle\{B\} \to \{C, D\}\rangle \implies$ presente in L_3 (regola 7).
2. Rimozione di B: $\langle\{A\} \to \{C, D\}\rangle \implies$ presente in L_3 (regola 5).
3. Rimozione di C: $\langle\{A, B\} \to \{D\}\rangle \implies$ presente in L_3 (regola 3).
4. Rimozione di D: $\langle\{A, B\} \to \{C\}\rangle \implies$ presente in L_3 (regola 1).

Decisione: Tutte le sotto-sequenze appartengono ad $L_3 \implies$ **NON POTATO (Mantenuto)**.

Candidato C2: $\langle\{A\} \to \{D\} \to \{C\} \to \{D\}\rangle$

Sotto-sequenze di lunghezza 3:

1. Rimozione del primo evento $\{A\}$: $\langle\{D\} \to \{C\} \to \{D\}\rangle \implies$ presente in L_3 (regola 12).
2. Rimozione del secondo evento $\{D\}$: $\langle\{A\} \to \{C\} \to \{D\}\rangle \implies$ presente in L_3 (regola 11).
3. Rimozione del terzo evento $\{C\}$: $\langle\{\mathbf{A}\} \to \{\mathbf{D}\} \to \{\mathbf{D}\}\rangle \implies$ **NON PRESENTE in** L_3.
4. Rimozione del quarto evento $\{D\}$: $\langle\{A\} \to \{D\} \to \{C\}\rangle \implies$ presente in L_3 (regola 2).

Decisione: Poiché la sotto-sequenza $\langle\{A\} \to \{D\} \to \{D\}\rangle \notin L_3$, il candidato viene **POTATO (PRUNED)**.

Candidato C3: $\langle\{B\} \to \{D\} \to \{C\} \to \{D\}\rangle$

Sotto-sequenze di lunghezza 3:

1. Rimozione del primo evento $\{B\}$: $\langle\{D\} \to \{C\} \to \{D\}\rangle \implies$ presente in L_3 (regola 12).
2. Rimozione del secondo evento $\{D\}$: $\langle\{B\} \to \{C\} \to \{D\}\rangle \implies$ presente in L_3 (regola 6).
3. Rimozione del terzo evento $\{C\}$: $\langle\{\mathbf{B}\} \to \{\mathbf{D}\} \to \{\mathbf{D}\}\rangle \implies$ **NON PRESENTE in** L_3.
4. Rimozione del quarto evento $\{D\}$: $\langle\{B\} \to \{D\} \to \{C\}\rangle \implies$ presente in L_3 (regola 8).

Decisione: Poiché la sotto-sequenza $\langle\{B\} \to \{D\} \to \{D\}\rangle \notin L_3$, il candidato viene **POTATO (PRUNED)**.

Candidato C4: $\langle\{A, B\} \to \{D\} \to \{C\}\rangle$

Sotto-sequenze di lunghezza 3:

1. Rimozione di A: $\langle\{B\} \to \{D\} \to \{C\}\rangle \implies$ presente in L_3 (regola 8).
2. Rimozione di B: $\langle\{A\} \to \{D\} \to \{C\}\rangle \implies$ presente in L_3 (regola 2).
3. Rimozione di D: $\langle\{A, B\} \to \{C\}\rangle \implies$ presente in L_3 (regola 1).
4. Rimozione di C: $\langle\{A, B\} \to \{D\}\rangle \implies$ presente in L_3 (regola 3).

Decisione: Tutte le sotto-sequenze appartengono ad $L_3 \implies$ **NON POTATO (Mantenuto)**.

Candidato C5: $\langle\{A, B\} \to \{C\} \to \{D\}\rangle$

Sotto-sequenze di lunghezza 3:

1. Rimozione di A: $\langle\{B\} \to \{C\} \to \{D\}\rangle \implies$ presente in L_3 (regola 6).
2. Rimozione di B: $\langle\{A\} \to \{C\} \to \{D\}\rangle \implies$ presente in L_3 (regola 11).

3. Rimozione di C: $\langle\{A, B\} \to \{D\}\rangle \implies$ presente in L_3 (regola 3).

4. Rimozione di D: $\langle\{A, B\} \to \{C\}\rangle \implies$ presente in L_3 (regola 1).

Decisione: Tutte le sotto-sequenze appartengono ad $L_3 \implies$ **NON POTATO (Mantenuto)**.

13.17.3 Tabella Sinottica Conclusiva del Pruning GSP

La Tabella 13.3 sintetizza i riscontri di potatura per tutti i candidati esaminati.

Tabella 13.3: Quadro sinottico della fase di potatura dei candidati (C_4) nell'algoritmo GSP.

ID	**Sequenza Candidata (C_4)**	**Sotto-sequenze Assenti da L_3**	**Esito di Potatura**
C1	$\langle\{A, B\} \to \{C, D\}\rangle$	Nessuna (tutte presenti)	**Mantenuto**
C2	$\langle\{A\} \to \{D\} \to \{C\} \to \{D\}\rangle$	$\langle\{A\} \to \{D\} \to \{D\}\rangle$	**POTATO**
C3	$\langle\{B\} \to \{D\} \to \{C\} \to \{D\}\rangle$	$\langle\{B\} \to \{D\} \to \{D\}\rangle$	**POTATO**
C4	$\langle\{A, B\} \to \{D\} \to \{C\}\rangle$	Nessuna (tutte presenti)	**Mantenuto**
C5	$\langle\{A, B\} \to \{C\} \to \{D\}\rangle$	Nessuna (tutte presenti)	**Mantenuto**

Capitolo 14

Serie Temporali: Similarità, Distanze, Trasformazioni e Clustering

14.1 Introduzione e Fondamenti delle Serie Temporali

L'analisi delle serie temporali (*Time Series Analysis*) costituisce uno dei domini cardine del Data Mining avanzato, trovando applicazione trasversale nella sensoristica industriale, nell'econometria, nella biologia computazionale, nella medicina e nelle scienze ambientali [2, 20]. A differenza dei dataset tabulari convenzionali, in cui le istanze sono ipotizzate mutuamente indipendenti e identicamente distribuite (i.i.d.), le serie temporali manifestano una marcata **dipendenza seriale**: l'ordine temporale delle osservazioni è vincolante e non invertibile, mentre l'elevata frequenza di campionamento genera flussi densi ad alta dimensionalità.

14.1.1 Definizione Formale di Serie Temporale

Definizione 14.1.1 (Serie Temporale)**.** Sia dato un intervallo temporale continuo discretizzato ad istanti regolari $t_i = t_0 + i \cdot \Delta t$, con $i \in \{1, \ldots, n\}$ e tempo di campionamento $\Delta t \in \mathbb{R}^+$. Una **serie temporale** X di lunghezza n è una sequenza ordinata di n osservazioni numeriche scalari a valori reali:

$$X = \langle x_1, x_2, \ldots, x_n \rangle \quad \text{con } x_i \in \mathbb{R} \tag{14.1}$$

in cui la coordinata x_i esprime la magnitudo quantitativa della grandezza monitorata al passo discretizzato t_i.

In ambito applicativo, le sequenze possono derivare dal campionamento ad alta risoluzione di oscillazioni finanziarie, segnali fisiologici o parametri ambientali. Si consideri la sequenza campionata a frequenza costante riportata in Tabella 14.1.

Passo	**Valore**	**Passo**	**Valore**	**Passo**	**Valore**	**Passo**	**Valore**
t_1	25.1750	t_7	25.3500	t_{13}	25.2000	t_{19}	24.6250
t_2	25.2250	t_8	25.3500	t_{14}	25.1750	t_{20}	24.6250
t_3	25.2500	t_9	25.4000	t_{15}	24.6250	t_{21}	24.6750
t_4	25.2500	t_{10}	25.4000	t_{16}	24.6750	t_{22}	24.7500
t_5	25.2750	t_{11}	25.3250	t_{17}	24.6750	–	–
t_6	25.3250	t_{12}	25.2250	t_{18}	24.6250	–	–

Tabella 14.1: Esempio di sequenza numerica campionata a frequenza costante Δt.

14.1.2 Ubiquità e Conversione di Domini non Temporali

Le serie temporali emergono in modo spontaneo in molteplici discipline empiriche:

- **Fisiologia e Medicina**: monitoraggio continuo della pressione sanguigna (*blood pressure*), tracciati elettroencefalografici (EEG) ed elettrocardiografici (ECG).
- **Scienze Sociali e Politiche**: andamento temporale degli indici di gradimento (*popularity rating*) e sentiment di figure istituzionali o prodotti di mercato.
- **Meteorologia**: serie storiche di precipitazioni piovose (es. piovosità annuale registrata a Pisa) e parametri termici/barometrici.
- **Economia e Finanza**: quotazioni azionarie, tassi interbancari di cambio e flussi di volume transazionale.

In aggiunta ai fenomeni intrinsecamente temporali, domini applicativi disomogenei e privi di una coordinata temporale nativa possono essere mappati nello spazio delle serie temporali:

1. **Dati Testuali (*Text Data*)**: calcolo progressivo della frequenza cumulativa o localizzata di termini-chiave lungo la successione sequenziale

di capitoli, tomi o sezioni di un'opera. Ad esempio, confrontando l'occorrenza del termine "God" nella versione inglese della Bibbia con l'omologo termine "Dios" nell'edizione spagnola lungo la successione dei libri canonici (da *Genesis* ad *Apocalypse*), si ottengono due traiettorie armoniche che condividono picchi e valli sincroni, attestando la fedeltà strutturale tra le due traduzioni.

2. **Profili di Immagini (*Image Contours*)**: trasformazione del perimetro bidimensionale di una sagoma mediante **scansione radiale** (*Radial Scanning*). Tracciando il raggio vettore dal baricentro geometrico dell'oggetto verso i punti di contorno a passi angolari uniformi $\theta \in [0, 2\pi)$, si produce una serie temporale $r(\theta)$ in cui le ascisse denotano l'angolo e le ordinate quantificano la distanza euclidea radiale.

3. **Dati Video (*Video Tracking*)**: monitoraggio delle coordinate cartesiane spaziali $(x(t), y(t))$ del centroide di un corpo in movimento lungo la sequenza dei fotogrammi (*frame*).

14.1.3 Complessità e Sfide Computazionali

Il mining su serie temporali comporta sfide metodologiche assenti nei modelli classici di apprendimento:

- **Volumetria Massiva (*Large Data Volume*)**: le sonde ad alta frequenza generano sequenze nell'ordine di milioni di campioni, saturando la memoria principale e richiedendo strategie di compressione o approssimazione *data-aware*.

- **Complessità della Similarità (*Non-Trivial Similarity*)**: le metriche rigide basate su confronti punto-a-punto falliscono sistematicamente in presenza di deformazioni locali di velocità, sfasamenti di fase o derive sistematiche.

- **Disomogeneità nei Campionamenti e Formati**: sensori eterogenei operano a frequenze di campionamento differenti ($\Delta t_1 \neq \Delta t_2$), determinando lunghezze disuguali $n \neq m$ per il medesimo intervallo temporale osservato.

- **Presenza di Rumore e Dati Mancanti**: imperfezioni strumentali, disturbi stocastici ad alta frequenza e interruzioni di segnale (*missing data*) richiedono pre-elaborazioni di filtraggio ed imputazione.

14.1.4 Compiti Analitici nel Time Series Mining

Le tipiche mansioni di data mining si declinano sulle serie temporali come segue:

- **Decomposizione di Trend e Stagionalità**: isolamento analitico delle componenti macroscopiche e cicliche.
- **Clustering**: raggruppamento non supervisionato di un insieme di sequenze in partizioni morfologicamente omogenee.
- **Motif Discovery**: estrazione automatica di sotto-sequenze brevi primitive che si manifestano con frequenza statisticamente sovra-rappresentata.
- **Rule Discovery**: derivazione di regole predittive causa-effetto ($A \Rightarrow B$) governate da metriche di supporto e confidenza.
- **Previsione (*Forecasting*)**: stima dei valori futuri $x_{n+1}, x_{n+2}, \dots$ a partire dalla finestra storica osservata $\langle x_1, \dots, x_n \rangle$ [20].
- **Classificazione Supervisionata**: assegnazione di una sequenza ignota a classi predefinite (ad esempio, distinguere un tracciato ECG fisiologico normale da uno ischemico caratterizzato dal sotto-slivellamento del tratto S-T).

14.1.5 Decomposizione Classica delle Componenti

In accordo con la teoria classica dei processi stocastici [21, 20], una serie temporale X si decompone canonicamente in componenti sistematiche e stocastiche:

$$x_t = L_t + T_t + S_t + \varepsilon_t \tag{14.2}$$

oppure in forma moltiplicativa $x_t = L_t \cdot T_t \cdot S_t \cdot \varepsilon_t$. Le singole componenti sono definite da:

1. **Livello (*Level*, L_t)**: valore base o livello medio della grandezza osservata.
2. **Trend (T_t)**: traiettoria di fondo a lungo termine, caratterizzata da un incremento o decremento monotono persistente nel tempo.
3. **Stagionalità (S_t)**: oscillazione periodica sistematica che si ripete a intervalli temporali definiti e prevedibili (legata a cicli solari, settimanali o stagionali).

4. **Rumore o Residuo Irregolare (ε_t)**: disturbo aleatorio casuale ad alta frequenza, privo di correlazione temporale strutturata (rumore bianco), non predicibile e non spiegato dalle componenti precedenti.

14.2 Metriche di Similarità e Trasformazioni di Segnale

La stima rigorosa della similarità tra coppie di sequenze governa l'accuratezza di qualsiasi algoritmo di classificazione (es. 1-NN), indicizzazione o clustering.

14.2.1 Tassonomia della Similarità

Si distinguono due paradigmi fondamentali di similarità:

- **Similarità Basata sulla Forma (*Shape-Based Similarity*)**: confronta la sovrapponibilità geometrica puntuale o locale delle traiettorie. Risulta altamente efficace per sequenze brevi o motivi transitori, ma decade drasticamente su sequenze lunghe.
- **Similarità Basata sulla Struttura (*Structure-Based Similarity*)**: ignora la corrispondenza temporale locale e valuta proprietà statistiche, spettrali o stocastiche globali del processo generatore (media, varianza, frequenze dominanti, modelli autoregressivi). Risulta indispensabile su serie estese ove intervengono sfasamenti multipli e perdita di fase.

14.2.2 Distanza Euclidea e Rigidità Temporale

Data una coppia di serie temporali $Q = \langle q_1, \ldots, q_n \rangle$ e $C = \langle c_1, \ldots, c_n \rangle$ aventi la medesima lunghezza n, la **Distanza Euclidea** (L_2 norm) è definita come:

$$D(Q, C) = \sqrt{\sum_{i=1}^{n} (q_i - c_i)^2} \tag{14.3}$$

Sotto il profilo topologico, la distanza Euclidea presuppone un allineamento rigorosamente **uno-a-uno** e sincrono lungo l'asse dei tempi: ciascun punto q_i è confrontato esclusivamente con il corrispondente punto c_i registrato all'istante t_i.

Esempio 14.2.1 (Calcolo della Distanza Euclidea)**.** Si considerino le due serie temporali T_1 e T_2 di lunghezza $n = 4$:

$$T_1 = \langle 56, 176, 110, 95 \rangle, \quad T_2 = \langle 36, 126, 180, 80 \rangle$$

I singoli contributi quadratici punto-a-punto risultano:

$$(56-36)^2 = 20^2 = 400$$
$$(176-126)^2 = 50^2 = 2500$$
$$(110-180)^2 = (-70)^2 = 4900$$
$$(95-80)^2 = 15^2 = 225$$

La sommatoria degli scarti quadratici vale:

$$\sum_{i=1}^{4}(q_i - c_i)^2 = 400 + 2500 + 4900 + 225 = 8025$$

da cui si ottiene la distanza Euclidea:

$$D(T_1, T_2) = \sqrt{8025} \approx 89.582$$

14.2.3 Distorsioni Superficiali e Trasformazioni di Normalizzazione

La distanza Euclidea grezza è estremamente fragile rispetto a distorsioni superficiali nei dati, che possono far apparire distanti sequenze aventi morfologia identica. Per garantire un confronto imparziale, è necessario applicare opportune trasformazioni preliminari:

1. Traslazione di Livello (*Offset Translation*)

Due serie possono condividere profilo e fluttuazioni ma presentare una linea di base verticale disallineata a causa di una costante additiva: $c_i \approx q_i + \Delta y$. Per eliminare l'offset, si centra ciascuna sequenza rispetto alla propria media campionaria:

$$q_i \leftarrow q_i - \mu_Q, \quad c_i \leftarrow c_i - \mu_C \quad \text{con } \mu_X = \frac{1}{n}\sum_{k=1}^{n} x_k \tag{14.4}$$

2. Disomogeneità di Ampiezza (*Amplitude Scaling*)

Differenze nel guadagno dei sensori amplificano l'escursione verticale dei dati pur preservandone la dinamica. L'effetto combinato di offset ed escursione di ampiezza viene neutralizzato mediante la **standardizzazione Z-score**:

$$q_i \leftarrow \frac{q_i - \mu_Q}{\sigma_Q}, \quad c_i \leftarrow \frac{c_i - \mu_C}{\sigma_C} \tag{14.5}$$

dove la deviazione standard campionaria è data da $\sigma_X = \sqrt{\frac{1}{n}\sum_{k=1}^{n}(x_k - \mu_X)^2}$. La trasformazione forza entrambe le sequenze ad avere media nulla ($\mu = 0$) e varianza unitaria ($\sigma^2 = 1$).

3. Trend Lineare di Fondo (*Linear Detrending*)

Una deriva lineare (*drift*) progressiva aumenta artificiosamente la distanza euclidea con l'avanzare del tempo. Si stima la retta di regressione di minimi quadrati $\hat{x}_i = \alpha + \beta \cdot i$ e si calcolano i residui detrendizzati:

$$x_i^{\text{detrended}} = x_i - (\alpha + \beta \cdot i) \tag{14.6}$$

4. Smussamento del Rumore (*Noise Smoothing / Moving Average*)

I disturbi casuali ad alta frequenza vengono attenuati mediante filtri a **media mobile centrata** (*Moving Average*, MA) di ampiezza w:

$$\bar{x}_i = \frac{1}{w} \sum_{j=i-\lfloor w/2 \rfloor}^{i+\lfloor w/2 \rfloor} x_j \tag{14.7}$$

Per una finestra dispari a 3 punti ($w = 3$), la formula operativa si esplicita come $\bar{x}_i = \frac{1}{3}(x_{i-1} + x_i + x_{i+1})$.

Esempio 14.2.2 (Calcolo della Media Mobile Centrata con $w = 3$)**.** Data la sequenza $X = \langle 20, 24, 22, 26, 25 \rangle$, i valori filtrati risultano:

- Per $i = 1$: non calcolabile (manca il valore sinistro x_0).
- Per $i = 2$: $\bar{x}_2 = \frac{20+24+22}{3} = \frac{66}{3} = 22.0$.
- Per $i = 3$: $\bar{x}_3 = \frac{24+22+26}{3} = \frac{72}{3} = 24.0$.
- Per $i = 4$: $\bar{x}_4 = \frac{22+26+25}{3} = \frac{73}{3} \approx 24.333 \implies 24.3$.
- Per $i = 5$: non calcolabile (manca il valore destro x_6).

14.3 Dynamic Time Warping (DTW)

Sebbene le normalizzazioni rimuovano le distorsioni sull'asse verticale delle ampiezze, esse non possono correggere le distorsioni sull'asse orizzontale dei tempi dovute ad accelerazioni o decelerazioni locali del segnale. Il **Dynamic Time Warping (DTW)**, introdotto formalmente nel riconoscimento vocale da Sakoe e Chiba [22], risolve questa limitazione consentendo allineamenti elastici e non lineari di tipo *uno-a-molti* e *molti-a-uno*.

14.3.1 Matrice di Costo Locale e Vincoli del Warping Path

Siano date due serie temporali $Q = \langle q_1, \ldots, q_n \rangle$ e $C = \langle c_1, \ldots, c_m \rangle$ di lunghezze arbitrarie n ed m. Si costruisce una matrice di distanze locali $D \in \mathbb{R}^{n \times m}$, in cui ciascuna cella racchiude il costo puntuale:

$$D(i, j) = d(q_i, c_j) = |q_i - c_j| \quad (\text{oppure } (q_i - c_j)^2) \tag{14.8}$$

Un **percorso di warping** (*warping path*) W è una successione contigua di K coordinate di celle:

$$W = \langle w_1, w_2, \ldots, w_K \rangle \quad \text{con } w_k = (i, j)_k, \ \max(n, m) \leq K < n + m \tag{14.9}$$

soggetta ai seguenti tre vincoli geometrici formali:

1. **Condizione al Contorno (*Boundary Condition*)**:

   $$w_1 = (1, 1) \quad \text{e} \quad w_K = (n, m) \tag{14.10}$$

   Il cammino deve iniziare nell'angolo inferiore sinistro e terminare nell'angolo superiore destro della griglia.

2. **Continuità e Monotonicità (*Step Pattern*)**: Data la cella $w_k = (i, j)$, la cella successiva $w_{k+1} = (i', j')$ deve soddisfare:

   $$i \leq i' \leq i + 1 \quad \text{e} \quad j \leq j' \leq j + 1 \tag{14.11}$$

   con $(i', j') \neq (i, j)$. I passi elementari ammissibili si riducono a tre movimenti:

   - $(i + 1, j + 1)$: **avanzamento diagonale**; entrambe le sequenze progrediscono di un passo temporale sincrono (analogo alla metrica euclidea).
   - $(i, j + 1)$: **avanzamento orizzontale**; Q resta fissa mentre C avanza, espandendo la coordinata q_i su più punti di C.
   - $(i+1, j)$: **avanzamento verticale**; C resta fissa mentre Q avanza, espandendo la coordinata c_j su più punti di Q.

14.3.2 Programmazione Dinamica ed Equazione di Bellman

Il costo cumulativo totale associato a un cammino di warping W è la sommatoria dei costi locali attraversati:

$$c_W(Q, C) = \sum_{k=1}^{K} d(w_k) = \sum_{k=1}^{K} D(i_k, j_k) \tag{14.12}$$

La distanza DTW corrisponde al costo del cammino ottimo che minimizza tale sommatoria:

$$\mathrm{DTW}(Q, C) = \min_{W} \{c_W(Q, C)\} \tag{14.13}$$

In virtù del principio di ottimalità di Bellman, il problema gode della proprietà di sottostruttura ottima ed è risolvibile in tempo polinomiale mediante la **programmazione dinamica**. Definendo con $\gamma(i, j)$ il costo cumulativo del percorso ottimo dalla cella iniziale $(1, 1)$ alla generica cella (i, j), si ottiene la relazione di ricorrenza:

$$\gamma(i, j) = D(i, j) + \min \begin{cases} \gamma(i-1, j-1) & \text{(passo diagonale)} \\ \gamma(i-1, j) & \text{(passo verticale)} \\ \gamma(i, j-1) & \text{(passo orizzontale)} \end{cases} \tag{14.14}$$

Algoritmo Operativo DTW

L'algoritmo si struttura in tre passi:

1. **Inizializzazione delle Condizioni al Bordo**:

$$\gamma(1, 1) = D(1, 1) \tag{14.15}$$
$$\gamma(i, 1) = D(i, 1) + \gamma(i-1, 1) \quad \text{per } i = 2, \ldots, n \tag{14.16}$$
$$\gamma(1, j) = D(1, j) + \gamma(1, j-1) \quad \text{per } j = 2, \ldots, m \tag{14.17}$$

2. **Riempimento della Matrice** γ: Si calcolano le celle interne riga per riga (o colonna per colonna) applicando la Equazione 14.14. Al termine, il valore $\mathrm{DTW}(Q, C) = \gamma(n, m)$ rappresenta la distanza minima.

3. **Backtracking del Cammino Ottimo**: Partendo dalla cella (n, m), si risale a ritroso selezionando ad ogni passo il predecessore adiacente avente il minimo valore di costo cumulato, terminando in $(1, 1)$.

14.3.3 Esempio Numerico Completo di Allineamento DTW

Si consideri l'allineamento tra le due sequenze di lunghezza $n = m = 5$:

$$c = \langle 3, 7, 4, 3, 4 \rangle, \quad q = \langle 5, 7, 6, 4, 2 \rangle$$

adottando la distanza assoluta puntuale $d(q_i, c_j) = |q_i - c_j|$ con indici indicizzati a base 0 ($i, j \in \{0, \ldots, 4\}$). Disponendo c lungo le colonne (asse x, $j = 0 \ldots 4$) e q lungo le righe dal basso verso l'alto (asse y, $i = 0 \ldots 4$), la

matrice dei costi locali D e la risultante matrice cumulativa γ sono tabulate in Tabella 14.2.

	Matrice Costi Locali $D(i,j)$					**Matrice Cumulativa** $\gamma(i,j)$				
$q_4 = 2$	1	5	2	1	2	11	11	5	**4**	**6**
$q_3 = 4$	1	3	0	1	0	10	6	**3**	**4**	4
$q_2 = 6$	3	1	2	3	2	9	3	**4**	7	9
$q_1 = 7$	4	0	3	4	3	6	**2**	5	9	10
$q_0 = 5$	2	2	1	2	1	**2**	4	5	7	8
	$c_0 = 3$	$c_1 = 7$	$c_2 = 4$	$c_3 = 3$	$c_4 = 4$	$c_0 = 3$	$c_1 = 7$	$c_2 = 4$	$c_3 = 3$	$c_4 = 4$

Tabella 14.2: Matrice dei costi locali D e matrice dei costi cumulativi γ con evidenziato in grassetto l'optimal warping path.

Il costo totale finale vale $\gamma(4,4) = 6$. Eseguendo il backtracking a ritroso:

$$(4,4) \to (4,3) \to (3,2) \to (2,1) \to (1,1) \to (0,0) \tag{14.18}$$

Il cammino ottimale W^* somma i seguenti costi locali:

$$\begin{aligned} \text{Costo} &= D(0,0) + D(1,1) + D(2,1) + D(3,2) + D(4,3) + D(4,4) \\ &= 2 + 0 + 1 + 0 + 1 + 2 = 6 \end{aligned}$$

La transizione verticale da $(1,1)$ a $(2,1)$ evidenzia un warping temporale in cui il valore $c_1 = 7$ si allinea sia con $q_1 = 7$ sia con $q_2 = 6$.

14.4 Studio di Caso ed Esercizi Analitici su DTW

In questa sezione si risolve in modo rigoroso e formale un problema classico d'esame articolato in tre quesiti analitici.

14.4.1 Formulazione del Problema

Siano assegnate le seguenti serie temporali di lunghezza $n = 5$:

$$t_1 = \langle 4, 3, 6, 1, 0 \rangle, \quad t_2 = \langle 3, 6, 7, 0, 1 \rangle \tag{14.19}$$

Si consideri la metrica di distanza assoluta puntuale $d(x,y) = |x - y|$.

- **Quesito A**: Calcolare la distanza DTW tra t_1 e t_2, costruendo esplicitamente le matrici dei costi locali D e cumulativi γ, e ricavando l'optimal warping path.

- **Quesito B**: Verificare se l'imposizione di una banda di Sakoe-Chiba di ampiezza/raggio $r = 1$ modifichi il costo dell'allineamento, motivando analiticamente la risposta.
- **Quesito C**: Date le serie invertite temporalmente $T_1 = \text{reverse}(t_1)$ e $T_2 = \text{reverse}(t_2)$, verificare se sussiste l'identità $\text{DTW}(T_1, T_2) = \text{DTW}(t_1, t_2)$, discutendo le proprietà teoriche dell'algoritmo.

14.4.2 Risoluzione del Quesito A

Si colloca t_1 lungo le colonne ($j = 0 \dots 4$) e t_2 lungo le righe dal basso verso l'alto ($i = 0 \dots 4$).

1. Matrice dei Costi Locali $D(i, j) = |t_2[i] - t_1[j]|$

- Riga $i = 0$ ($t_2 = 3$): $|3-4| = 1,\ |3-3| = 0,\ |3-6| = 3,\ |3-1| = 2,\ |3-0| = 3$.
- Riga $i = 1$ ($t_2 = 6$): $|6-4| = 2,\ |6-3| = 3,\ |6-6| = 0,\ |6-1| = 5,\ |6-0| = 6$.
- Riga $i = 2$ ($t_2 = 7$): $|7-4| = 3,\ |7-3| = 4,\ |7-6| = 1,\ |7-1| = 6,\ |7-0| = 7$.
- Riga $i = 3$ ($t_2 = 0$): $|0-4| = 4,\ |0-3| = 3,\ |0-6| = 6,\ |0-1| = 1,\ |0-0| = 0$.
- Riga $i = 4$ ($t_2 = 1$): $|1-4| = 3,\ |1-3| = 2,\ |1-6| = 5,\ |1-1| = 0,\ |1-0| = 1$.

2. Calcolo della Matrice Cumulativa $\gamma(i, j)$

Applicando le condizioni al contorno e la relazione di Bellman:

- **Colonna** $j = 0$ ($t_1 = 4$):

$$\begin{aligned}
\gamma(0,0) &= 1 \\
\gamma(1,0) &= 2 + 1 = 3 \\
\gamma(2,0) &= 3 + 3 = 6 \\
\gamma(3,0) &= 4 + 6 = 10 \\
\gamma(4,0) &= 3 + 10 = 13
\end{aligned}$$

- **Colonna** $j = 1$ $(t_1 = 3)$:

$$\gamma(0,1) = 0 + \gamma(0,0) = 0 + 1 = 1$$
$$\gamma(1,1) = 3 + \min(1,3,1) = 3 + 1 = 4$$
$$\gamma(2,1) = 4 + \min(3,6,4) = 4 + 3 = 7$$
$$\gamma(3,1) = 3 + \min(6,10,7) = 3 + 6 = 9$$
$$\gamma(4,1) = 2 + \min(10,13,9) = 2 + 9 = 11$$

- **Colonna** $j = 2$ $(t_1 = 6)$:

$$\gamma(0,2) = 3 + \gamma(0,1) = 3 + 1 = 4$$
$$\gamma(1,2) = 0 + \min(\gamma(0,1) = 1, \gamma(1,1) = 4, \gamma(0,2) = 4) = 0 + 1 = 1$$
$$\gamma(2,2) = 1 + \min(\gamma(1,1) = 4, \gamma(2,1) = 7, \gamma(1,2) = 1) = 1 + 1 = 2$$
$$\gamma(3,2) = 6 + \min(7,9,2) = 6 + 2 = 8$$
$$\gamma(4,2) = 5 + \min(9,11,8) = 5 + 8 = 13$$

- **Colonna** $j = 3$ $(t_1 = 1)$:

$$\gamma(0,3) = 2 + \gamma(0,2) = 2 + 4 = 6$$
$$\gamma(1,3) = 5 + \min(4,1,6) = 5 + 1 = 6$$
$$\gamma(2,3) = 6 + \min(1,2,6) = 6 + 1 = 7$$
$$\gamma(3,3) = 1 + \min(\gamma(2,2) = 2, \gamma(3,2) = 8, \gamma(2,3) = 7) = 1 + 2 = 3$$
$$\gamma(4,3) = 0 + \min(\gamma(3,2) = 8, \gamma(4,2) = 13, \gamma(3,3) = 3) = 0 + 3 = 3$$

- **Colonna** $j = 4$ $(t_1 = 0)$:

$$\gamma(0,4) = 3 + \gamma(0,3) = 3 + 6 = 9$$
$$\gamma(1,4) = 6 + \min(6,6,9) = 6 + 6 = 12$$
$$\gamma(2,4) = 7 + \min(6,7,12) = 7 + 6 = 13$$
$$\gamma(3,4) = 0 + \min(7,3,13) = 0 + 3 = 3$$
$$\gamma(4,4) = 1 + \min(\gamma(3,3) = 3, \gamma(4,3) = 3, \gamma(3,4) = 3) = 1 + 3 = 4$$

Le due matrici risultanti sono riassunte in Tabella 14.3.

	Matrice Costi Locali $D(i,j)$					**Matrice Cumulativa** $\gamma(i,j)$				
$t_2[4] = 1$	3	2	5	0	1	13	11	13	3	**4**
$t_2[3] = 0$	4	3	6	1	0	10	9	8	**3**	3
$t_2[2] = 7$	3	4	1	6	7	6	7	**2**	7	13
$t_2[1] = 6$	2	3	0	5	6	3	4	**1**	6	12
$t_2[0] = 3$	1	0	3	2	3	**1**	**1**	4	6	9
	$t_1[0] = 4$	$t_1[1] = 3$	$t_1[2] = 6$	$t_1[3] = 1$	$t_1[4] = 0$	$t_1[0] = 4$	$t_1[1] = 3$	$t_1[2] = 6$	$t_1[3] = 1$	$t_1[4] = 0$

Tabella 14.3: Matrici di calcolo per l'allineamento DTW di t_1 e t_2. In grassetto il cammino ottimo W^*.

Il costo totale dell'allineamento vale:

$$\mathrm{DTW}(\mathbf{t_1}, \mathbf{t_2}) = \mathbf{4} \tag{14.20}$$

3. Backtracking del Cammino Ottimo

Ripercorrendo le scelte minime da $(4,4)$ a ritroso:

$$(4,4) \xrightarrow{\text{diag}} (3,3) \xrightarrow{\text{diag}} (2,2) \xrightarrow{\text{vert}} (1,2) \xrightarrow{\text{diag}} (0,1) \xrightarrow{\text{orizz}} (0,0) \tag{14.21}$$

L'optimal warping path ordinato è:

$$W^* = \langle (0,0), (0,1), (1,2), (2,2), (3,3), (4,4) \rangle \tag{14.22}$$

La cui somma dei costi puntuali corrisponde esattamente a:

$$\begin{aligned} c_{W^*} &= D(0,0) + D(0,1) + D(1,2) + D(2,2) + D(3,3) + D(4,4) \\ &= 1 + 0 + 0 + 1 + 1 + 1 = 4 \end{aligned}$$

14.4.3 Risoluzione del Quesito B: Vincolo di Sakoe-Chiba con $r = 1$

La **banda di Sakoe-Chiba** con raggio di deformazione $r = 1$ circoscrive l'esplorazione alle sole celle (i, j) soddisfacenti la condizione di prossimità diagonale:

$$|i - j| \leq 1 \tag{14.23}$$

Verifichiamo la compatibilità di ciascuna cella appartenente al cammino ottimo globale W^*:

- Cella $(0,0)$: $|0-0| = 0 \leq 1$ (*Ammissibile*)
- Cella $(0,1)$: $|0-1| = 1 \leq 1$ (*Ammissibile*)
- Cella $(1,2)$: $|1-2| = 1 \leq 1$ (*Ammissibile*)
- Cella $(2,2)$: $|2-2| = 0 \leq 1$ (*Ammissibile*)
- Cella $(3,3)$: $|3-3| = 0 \leq 1$ (*Ammissibile*)
- Cella $(4,4)$: $|4-4| = 0 \leq 1$ (*Ammissibile*)

Proposizione 14.4.1. *Poiché tutte le celle costituenti il cammino ottimo privo di vincoli W^* appartengono alla regione ammissibile $|i-j| \leq 1$, l'introduzione della banda di Sakoe-Chiba con $r = 1$* ***non altera né il cammino ottimo né il costo cumulativo finale****, che rimane invariato a:*

$$DTW_{r=1}(t_1, t_2) = 4$$

14.4.4 Risoluzione del Quesito C: Inversione Temporale delle Sequenze

Si consideri la coppia di serie invertite temporaneamente:

$$T_1 = \langle 0, 1, 6, 3, 4 \rangle, \quad T_2 = \langle 1, 0, 7, 6, 3 \rangle$$

Teorema 14.4.1 (Invarianza per Inversione Temporale di DTW)**.** *Date due serie temporali Q e C, sia reverse(X) l'operatore che inverte l'ordine sequenziale degli elementi. Sussiste l'identità analitica:*

$$DTW(reverse(Q), reverse(C)) = DTW(Q, C) \tag{14.24}$$

Dimostrazione. Sia $W = \langle w_1, \ldots, w_K \rangle$ con $w_k = (i_k, j_k)$ un cammino di warping ammissibile tra Q e C, soddisfacente le condizioni al contorno $(1, 1)$ e (n, m), unitamente ai passi di monotonicità e continuità.

Si definisca il cammino invertito $\widetilde{W} = \langle \tilde{w}_1, \ldots, \tilde{w}_K \rangle$ operando la sostituzione speculare di coordinate:

$$\tilde{w}_k = (n - i_{K-k+1} + 1,\ m - j_{K-k+1} + 1)$$

Poiché la trasformazione è biiettiva e preserva la classe dei passi ammissibili ($\Delta i \in \{0, 1\}, \Delta j \in \{0, 1\}$), ad ogni cammino valido tra Q e C corrisponde biunivocamente un cammino valido tra reverse(Q) e reverse(C). Inoltre, per costruzione:

$$d\Big(\text{reverse}(Q)[\tilde{i}_k],\ \text{reverse}(C)[\tilde{j}_k]\Big) = d(Q[i_k], C[j_k])$$

La sommatoria dei costi locali associata a $\widetilde{W}$ è identica alla sommatoria di W. Ne consegue che il minimo globale su tutti i cammini ammissibili coincide:

$$\text{DTW}(T_1, T_2) = \text{DTW}(t_1, t_2) = 4$$

□

Nota 14.4.1. Sebbene il valore finale della distanza sia identico, i valori delle celle intermedie della matrice di programmazione dinamica γ_{T_1,T_2} non coincidono con quelli di γ_{t_1,t_2}, poiché le somme parziali si propagano a partire dall'estremità temporale opposta.

14.5 Valutazione Sperimentale e Limitazioni Computazionali

14.5.1 Confronto Prestazionale: Distanza Euclidea vs DTW

L'efficacia comparativa tra allineamento rigido (Distanza Euclidea) ed allineamento elastico (DTW) è stata validata sperimentalmente nella letteratura scientifica [23] impiegando un classificatore supervisionato basato sul primo vicino più prossimo (**1-NN**) valutato mediante validazione incrociata *Leave-One-Out* (LOOCV) su molteplici dataset di riferimento dell'archivio UCR Time Series Repository.

Tabella 14.4 riassume i tassi di errore di classificazione e i tempi computazionali di esecuzione (espressi in millisecondi) registrati sui medesimi benchmark.

	Tasso di Errore 1-NN (%)		**Tempo Computazionale (ms)**	
Dataset Benchmark	**Distanza Euclidea**	**DTW**	**Distanza Euclidea**	**DTW**
Word Spotting	**1.10**	4.78	40	8 600
Sign Language	**25.93**	28.70	10	1 110
GUN	5.50	**1.00**	60	11 820
Nuclear Trace	11.00	**0.00**	210	144 470
Leaves	33.26	**4.07**	150	51 830
Faces	6.25	**2.68**	50	45 080
Control Chart	**0.33**	7.50	110	21 900
2-Patterns	1.04	**0.00**	16 890	545 123

Tabella 14.4: Confronto sperimentale tra Distanza Euclidea e DTW (classificatore 1-NN con Leave-One-Out CV): tasso di errore percentuale e tempi di esecuzione in millisecondi.

Analisi dei Risultati

- **Superiorità Morfologica di DTW**: Nei domini geometrici e biologici (*Leaves*, *Faces*, *GUN*, *Nuclear Trace*), in cui i pattern manifestano dilatazioni o ritardi di esecuzione locali, DTW abbatte drasticamente il tasso di errore (es. in *Leaves* l'errore scende dal 33.26% al 4.07%, in *Nuclear Trace* si azzera da 11.00% a 0.00%).
- **Trade-off dell'Allineamento Rigido**: Nei segnali in cui la posizione temporale assoluta è fortemente discriminante (*Word Spotting*, *Control Chart*), l'eccessiva elasticità di DTW induce allineamenti spuri tra classi

distinte (*pathological warpings*), penalizzando l'accuratezza rispetto alla distanza Euclidea.

- **Collo di Bottiglia Computazionale**: L'ispezione dei tempi evidenzia che DTW risulta **da 2 a 3 ordini di grandezza più lento** rispetto alla distanza Euclidea. Tale divario discende dalla complessità temporale: mentre la metrica Euclidea opera in tempo lineare $\mathcal{O}(n)$, il DTW standard quadratico richiede $\mathcal{O}(n^2)$ operazioni per singola coppia di sequenze, rendendo impraticabile la ricerca sequenziale su collezioni massive.

14.5.2 Strategie di Ottimizzazione e Vincoli Globali

Per mitigare la complessità quadratica e prevenire allineamenti degeneri, si adottano due famiglie di strategie:

1. Approssimazioni Rapide (*Fast Approximations*)

Prevedono il sottocampionamento preventivo (*downsampling*) delle serie ad una risoluzione temporale grossolana. L'allineamento DTW viene calcolato sul segnale compresso, proiettando successivamente il warping path alla scala originale per una rifinitura locale vincolata. Questa strategia riduce i tempi di esecuzione da oltre 1.0 s a meno di 0.07 s, mantenendo un'accuratezza comparabile.

2. Vincoli Globali (*Global Constraints*)

I vincoli globali circoscrivono lo spazio delle celle ammissibili nella matrice di costo D ad una regione geometrica confinata attorno alla diagonale principale:

$$j - r \leq i \leq j + r \tag{14.25}$$

dove r rappresenta l'ampiezza della finestra di tolleranza temporale.

- **Banda di Sakoe-Chiba (*Sakoe-Chiba Band*)** [22]: definisce una fascia di larghezza costante r parallela alla diagonale principale ($|i-j| \leq r$). Riduce la complessità asintotica da $\mathcal{O}(n^2)$ a $\mathcal{O}(r \cdot n)$.
- **Parallelogramma di Itakura (*Itakura Parallelogram*)**: impone un vincolo a pendenza variabile, conformato come un parallelogramma stretto alle estremità $(1, 1)$ e (n, m) e più ampio nella regione centrale. Consente flessibilità intermedia precludendo derive distorsive nelle fasi terminali.

14.6 Similarità Strutturali e Modellistiche

14.6.1 Limiti delle Misure Shape-Based su Sequenze Lunghe

Quando la lunghezza delle serie temporali scala a valori elevati ($n \gg 10^3$), sia la Distanza Euclidea sia il DTW decadono prestazionalmente. All'aumentare dell'orizzonte temporale, l'accumulo di rumore, le derive stocastiche e la perdita progressiva di sincronia tra cicli distruggono il significato del confronto punto-a-punto o per segmenti elastici locali.

In tali regimi, l'analisi richiede una transizione dal livello morfologico al **livello strutturale e modellistico** (*Feature/Model-Based Similarity*):

1. Si estrae da ciascuna serie X un vettore sintetico di descrittori numerici globali $\mathbf{f}_X \in \mathbb{R}^d$ ($d \ll n$).

2. Si calcola la similarità euclidea o statistica direttamente nello spazio compatto delle feature $\mathbb{R}^d$.

Le feature estratte includono:

- **Momenti Statistici**: media (μ), varianza (σ^2), indice di asimmetria (*skewness*), indice di curtosi (*kurtosis*).

- **Proprietà Dinamiche**: media e varianza della derivata prima per stimare la velocità locale di oscillazione.

- **Autocorrelazione e Spettro**: coefficienti di autocorrelazione a diversi lag temporali e potenza spettrale armonica.

- **Parametri di Modelli Matematici**: coefficienti di modelli autoregressivi ARMA/ARIMA o matrici di transizione di modelli di Markov.

Esempio 14.6.1 (Separabilità Morfologica vs Feature-Based)**.** Si considerino tre serie temporali A, B, C, in cui A e B provengono da tracciati cardiaci con impulsi periodici regolari (ma leggermente sfasati nel tempo), mentre C è un segnale monotono a variazione lenta privo di battiti. Una distanza euclidea grezza classificherebbe A e B come distanti a causa dello sfasamento di fase. Estraendo i descrittori globali sintetizzati in Tabella 14.5:

Feature Estratta	**Serie A**	**Serie B**	**Serie C**
Valore Massimo (*Max Value*)	11	12	19
Valore Medio (*Mean*)	5.3	6.4	4.8
Valore Minimo (*Min Value*)	3	2	5
Autocorrelazione (*Autocorr*)	0.2	0.3	0.5

Tabella 14.5: Vettori di feature globali estratti dalle serie A, B e C.

I vettori $\mathbf{f}_A = [11, 5.3, 3, 0.2]$ e $\mathbf{f}_B = [12, 6.4, 2, 0.3]$ risultano prossimi nello spazio euclideo, consentendo l'immediata aggregazione di A e B nel medesimo raggruppamento e separandoli nettamente da $\mathbf{f}_C = [19, 4.8, 5, 0.5]$.

14.6.2 Dissimilarità Basata sulla Compressione: CDM

Un paradigma non parametrico universale per stimare la similarità strutturale si fonda sulla teoria algoritmica dell'informazione e sulla complessità di Kolmogorov [24]. Due sequenze si considerano intimamente simili se condividono le medesime regolarità descrittive, cosicché la compressione congiunta delle due serie risulti più compatta rispetto alla somma delle singole rappresentazioni compresse individuali.

La misura di dissimilarità basata sulla compressione (**Compression-Based Dissimilarity Measure**, CDM) tra due sequenze discretizzate x ed y è formalizzata come [24]:

$$\mathrm{CDM}(x, y) = \frac{C(x, y)}{C(x) + C(y)} \tag{14.26}$$

dove:

- $C(x)$ indica la dimensione in byte del file compresso ottenuto applicando un compressore *lossless* standard (es. gzip, Deflate, LZW) alla stringa simbolica x.
- $C(y)$ indica la dimensione in byte del file compresso ottenuto da y.
- $C(x, y)$ rappresenta la dimensione in byte del file compresso ottenuto concatenando le sequenze x ed y.

Se x ed y possiedono strutture identiche, il dizionario di pattern condiviso consente al compressore di codificare la concatenazione senza duplicare l'albero sintattico, determinando $C(x, y) \approx C(x)$, per cui $\mathrm{CDM}(x, y) \to 0.5$. Se le serie sono totalmente incorrelate, $C(x, y) \approx C(x) + C(y)$, per cui $\mathrm{CDM}(x, y) \to 1.0$. Il CDM dimostra eccellente robustezza a disallineamenti di fase, rumore ad alta frequenza e variazioni locali di periodo.

14.7 Tecniche di Approssimazione e Riduzione della Dimensionalità

L'**approssimazione** delle serie temporali proietta una sequenza ad alta dimensionalità n in un sottospazio compatto di dimensione $M \ll n$. Si distingue nettamente dalla compressione entropica generica: mentre lo spazio compresso perde interpretabilità semantica, lo **spazio approssimato** conserva un significato fisico, geometrico o spettrale direttamente accessibile all'analista (es. pendenze locali, medie di finestre, coefficienti armonici).

14.7.1 Trasformata Discreta di Fourier (DFT)

La Discrete Fourier Transform proietta una serie temporale dal dominio del tempo al dominio delle frequenze, rappresentandola come una combinazione lineare di oscillazioni sinusoidali ortogonali governate da tre parametri: ampiezza, frequenza (misurata in Hertz, Hz) e fase iniziale.

In base al teorema di Parseval (conservazione dell'energia), l'energia e il contenuto morfologico macroscopico dei segnali continui si concentrano prevalentemente nei primi coefficienti a bassa frequenza, mentre le alte frequenze modellano disturbi stocastici e rumore. Trattenendo unicamente i primi $M/2$ coefficienti complessi di Fourier (ciascuno occupante due dimensioni per parte reale ed immaginaria), si ottiene una rappresentazione compressa a M dimensioni che preserva le distanze euclidee sottostanti (proprietà di *Lower Bounding*).

- **Vantaggi**: elevata efficienza di compressione; esistenza di algoritmi Fast Fourier Transform (FFT) scalabili in $\mathcal{O}(n \log_2 n)$.
- **Limiti**: difficoltà nell'allineamento di sequenze a lunghezza disomogenea; scarsa localizzazione temporale di anomalie transitorie e spigoli vivi (fenomeno di Gibbs).

14.7.2 Piecewise Linear Approximation (PLA)

La Piecewise Linear Approximation modella la serie temporale mediante una successione ordinata di K segmenti lineari retti (connessi o disconnessi). Ciascun segmento è codificato tramite tre parametri: lunghezza temporale del segmento, altezza iniziale e pendenza/altezza terminale. PLA consente un ottimo filtraggio del rumore e supporta metriche di confronto basate sulle derivate, ma richiede euristiche di ottimizzazione per determinare il numero ottimale K di segmenti e la posizione delle partizioni.

14.7.3 Piecewise Aggregate Approximation (PAA)

La Piecewise Aggregate Approximation riduce una serie di lunghezza n a un vettore compatto di M elementi ($M \ll n$) suddividendo l'orizzonte temporale in M finestre contigue di ampiezza uniforme $w = n/M$ e calcolando la media aritmetica dei valori interni a ciascun frame:

$$\bar{c}_k = \frac{1}{w} \sum_{j=w(k-1)+1}^{wk} c_j \quad \text{per } k = 1, \ldots, M \tag{14.27}$$

La sequenza è quindi approssimata dalla funzione a gradino $\bar{C} = \langle \bar{c}_1, \ldots, \bar{c}_M \rangle$. PAA è estremamente rapida da calcolare ($\mathcal{O}(n)$), intuitiva e garantisce il soddisfacimento della proprietà di lower bounding per la distanza Euclidea.

14.7.4 Adaptive Piecewise Constant Approximation (APCA)

L'Adaptive Piecewise Constant Approximation generalizza PAA rimuovendo il vincolo di ampiezza uniforme delle finestre. I singoli segmenti costanti possono avere estensione arbitraria, richiedendo per ciascuno due parametri numerici: il valore medio campionario e la durata temporale del segmento. Ciò consente di allocare segmenti lunghi ed estesi nelle regioni stazionarie a bassa attività, concentrando segmenti brevi e densi nelle regioni di transizione rapida.

14.7.5 Symbolic Aggregate Approximation (SAX)

La tecnica SAX, introdotta da Lin et al. [25], converte una serie temporale continua in una stringa di caratteri simbolici discreti su un alfabeto di cardinalità finita a, consentendo l'applicazione diretta di algoritmi e strutture dati per stringhe (alberi di suffissi, automi, tabelle hash).

Fasi Algoritmiche di SAX

1. **Normalizzazione e Riduzione PAA**: La serie originale viene normalizzata mediante Z-score ($\mu = 0, \sigma = 1$) e suddivisa in w intervalli temporali uniformi, ottenendo il vettore di medie PAA $\bar{C} = \langle \bar{c}_1, \ldots, \bar{c}_w \rangle$.

2. **Definizione dei Breakpoints Equiprobabili**: Fissata la dimensione dell'alfabeto a, si determinano $a-1$ soglie numeriche $B = \{\beta_1, \ldots, \beta_{a-1}\}$ sotto l'ipotesi che i coefficienti di una serie normalizzata seguano una

distribuzione normale standard $\mathcal{N}(0,1)$. I breakpoint partizionano la gaussiana in a regioni disgiunte di **uguale area sottesa**:

$$P(\beta_{j-1} \leq X < \beta_j) = \frac{1}{a} \quad \text{con } \beta_0 = -\infty,\ \beta_a = +\infty \tag{14.28}$$

Il vincolo di equiprobabilità previene distorsioni statistiche nelle frequenze di occorrenza dei simboli.

3. **Mappatura Simbolica**: A ciascun coefficiente PAA $\bar{c}_k$ viene assegnato il carattere alfabetico associato alla regione gaussiana in cui ricade (es. $\{a, b, c, \dots\}$ partendo dal basso verso l'alto).

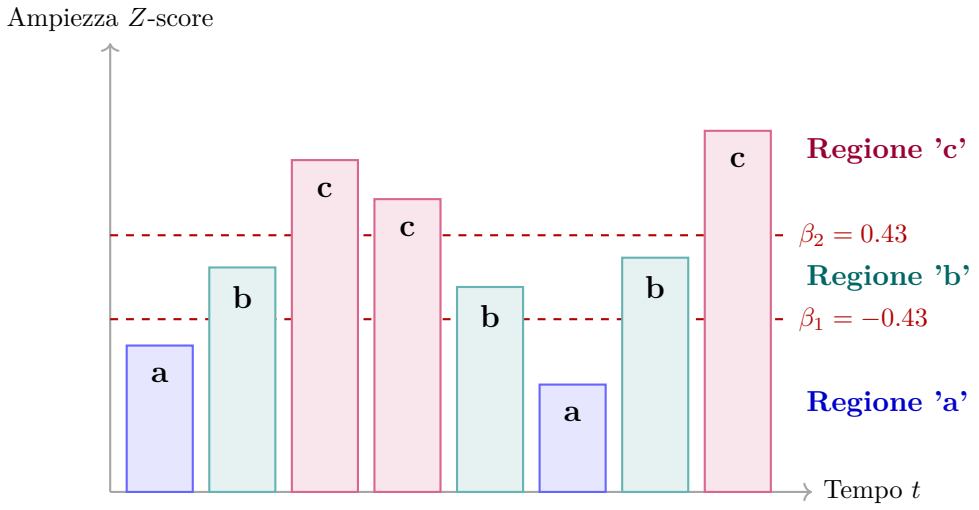


Figura 14.1: Discretizzazione SAX con alfabeto ternario $\{a, b, c\}$ su $w = 8$ finestre PAA. La parola risultante è `abccbabc`.

14.7.6 Criteri di Selezione della Metrica di Similarità

La scelta ottimale della misura di distanza dipende dalle dimensioni e dalla natura intrinseca del segnale:

- **Serie Temporali Brevi** ($n \leq 10^3$): si adotta preferenzialmente il **Dynamic Time Warping (DTW)**, calibrando la dimensione della finestra di warping r per via sperimentale (prevenendo derive patologiche ed accelerando l'elaborazione).
- **Serie Temporali Lunghe** ($n > 10^3$):

- In presenza di **conoscenza di dominio**: si estraggono vettori di feature statistiche, dinamiche e spettrali (*feature engineering*), applicando metriche euclidee o modelli supervisionati nello spazio compatto $\mathbb{R}^d$.
- In assenza di **conoscenza a priori**: si impiegano misure non parametriche universali basate sulla compressione e sull'approssimazione simbolica, quali il **CDM** o **SAX**.

14.8 Clustering di Serie Temporali

Il clustering per serie temporali estende il paradigma dell'apprendimento non supervisionato alle sequenze continue, raggruppando le traiettorie in partizioni omogenee tali da massimizzare la coesione interna e la separazione reciproca inter-cluster.

14.8.1 Tassonomia del Clustering per Sequenze Temporali

Si distinguono quattro configurazioni metodologiche fondamentali:

1. **Whole Clustering**: partiziona un archivio discreto composto da serie temporali indipendenti, assegnando l'intera traiettoria di ciascuna sequenza a un cluster (analogo al partizionamento di record tradizionali in $\mathbb{R}^d$).

2. **Feature-Based Clustering**: esegue preventivamente l'estrazione di feature o motivi frequenti (*motifs*) da ciascuna sequenza, applicando algoritmi standard (K-Means, DBSCAN) sullo spazio euclideo dei descrittori.

3. **Compression-Based Clustering**: comprime le sequenze convertite in formato simbolico e applica matrici di dissimilarità derivate da misure di complessità dell'informazione (come il CDM).

4. **Subsequence Clustering**: opera su una singola serie temporale continua estesa, estraendo sotto-sequenze parziali sovrapposte mediante lo scorrimento di una finestra mobile (*sliding window*) ed aggregando tali segmenti in pattern ricorrenti.

14.8.2 Clustering Gerarchico (Hierarchical Clustering)

Il clustering gerarchico agglomerativo (*bottom-up*) costruisce un albero tassonomico (dendrogramma) senza richiedere la pre-specificazione a priori del numero di raggruppamenti K:

1. Si calcola la matrice simmetrica delle distanze a coppie tra le N serie temporali impiegando DTW o la metrica Euclidea opportunamente normalizzata.
2. Si inizializza ogni serie come un cluster singleton disgiunto.
3. Si aggregano iterativamente i due cluster a minima distanza secondo un criterio di collegamento prefissato (*Single Linkage*, *Complete Linkage*, *Average Linkage* o *Metodo di Ward*).

Complessità Computazionale: La costruzione della matrice delle distanze su N serie di lunghezza n con DTW richiede tempo $\mathcal{O}(N^2 \cdot n^2)$ e memoria $\mathcal{O}(N^2)$, scalando difficilmente oltre poche centinaia di sequenze.

14.8.3 Clustering Partizionale: K-Means per Serie Temporali

Il clustering partizionale suddivide l'insieme delle sequenze in K gruppi esaustivi e disgiunti minimizzando la devianza intra-cluster (*Sum of Squared intra-cluster Errors*, SSE):

$$\text{SSE} = \sum_{k=1}^{K} \sum_{X_i \in C_k} D(X_i, \boldsymbol{\mu}_k)^2 \tag{14.29}$$

dove $\boldsymbol{\mu}_k$ rappresenta il profilo medio o centroide del cluster C_k, e D è la metrica di distanza prescelta.

- **Centroide Euclideo**: Sotto distanza Euclidea, il centroide è calcolato banalmente dalla media aritmetica punto per punto delle serie appartenenti al cluster:

$$\boldsymbol{\mu}_k(t) = \frac{1}{|C_k|} \sum_{X_i \in C_k} X_i(t) \tag{14.30}$$

- **Centroide sotto DTW (DTW Barycenter Averaging - DBA)**: Sotto DTW, la media aritmetica punto per punto è priva di significato fisico a causa degli sfasamenti temporali. Il centroide deve essere ricavato mediante algoritmi di allineamento multiplo elastico (*DBA*), che mediano le coordinate connesse lungo i rispettivi warping path.

- **Vulnerabilità**: Richiede di fissare K a priori ed è sensibile alla scelta dell'inizializzazione dei centroidi e alla presenza di outlier isolati.

Capitolo 15

Serie Temporali: Motif Discovery e Matrix Profile

15.1 Introduzione al Motif Discovery nelle Serie Temporali

Nell'ambito del Data Mining applicato a sequenze temporali (*Time Series Data Mining*), l'individuazione di strutture ricorrenti e regolarità morfologiche riveste un ruolo primario per la comprensione dei fenomeni dinamici sottostanti [2, 26]. Mentre i metodi di clustering tradizionali analizzano l'intera traiettoria o segmentazioni statiche a priori (sezione 14.8), il paradigma del **Time Series Motif Discovery** affronta l'estrazione non supervisionata di configurazioni locali che si manifestano con frequenza anomala o sorprendente lungo il flusso dei dati.

15.1.1 Definizione Formale del Problema di Motif Discovery

Sia data una serie temporale univariata $T = \langle t_1, t_2, \ldots, t_n \rangle \in \mathbb{R}^n$ di lunghezza $|T| = n$ campionata a intervalli temporali regolari (Definizione 14.1.1).

Definizione 15.1.1 (Sottosequenza di Lunghezza m)**.** Fissato un parametro intero $m \in \mathbb{N}^+$ tale che $m \ll n$, si definisce **sottosequenza** $T_{i,m}$ (o semplicemente T_i) il segmento continuo di lunghezza m estratto da T a partire dalla coordinata temporale i:

$$T_{i,m} = \langle t_i, t_{i+1}, \ldots, t_{i+m-1} \rangle \in \mathbb{R}^m, \qquad \text{con } 1 \leq i \leq n - m + 1 \tag{15.1}$$

Il quesito fondamentale del Motif Discovery può essere enunciato formalmente come segue:

Quesito Fondamentale del Motif Discovery: Esistono due o più sottosequenze distinte T_i, T_j di lunghezza m all'interno della serie temporale T la cui distanza reciproca risulti minima rispetto a tutte le altre coppie possibili?

In termini matematici, data una metrica di distanza opportuna $D(\cdot, \cdot)$ definita su $\mathbb{R}^m$ (tipicamente la distanza Euclidea z-normalizzata, discussa in sezione 15.2), si definisce **coppia di motivi classica** (*Classic Motif Pair*) la coppia ordinata di indici (i^*, j^*) che minimizza la distanza reciproca sotto vincolo di non-sovrapposizione banale:

$$(i^*, j^*) = \arg \min_{\substack{i,j \\ |i-j| \geq w}} D(T_{i,m}, T_{j,m}) \tag{15.2}$$

dove w definisce l'ampiezza della zona di esclusione temporale volta a scongiurare accoppiamenti banali (*trivial matches*).

15.1.2 Caso di Studio Reale: Il Winding Dataset

Per comprendere la rilevanza fisica e ingegneristica del problema, si consideri il celebre benchmark industriale noto come **Winding Dataset** [26], che registra la velocità angolare della bobina secondaria (*The angular speed of reel 2*) in un impianto di avvolgimento continuo di nastri d'acciaio.

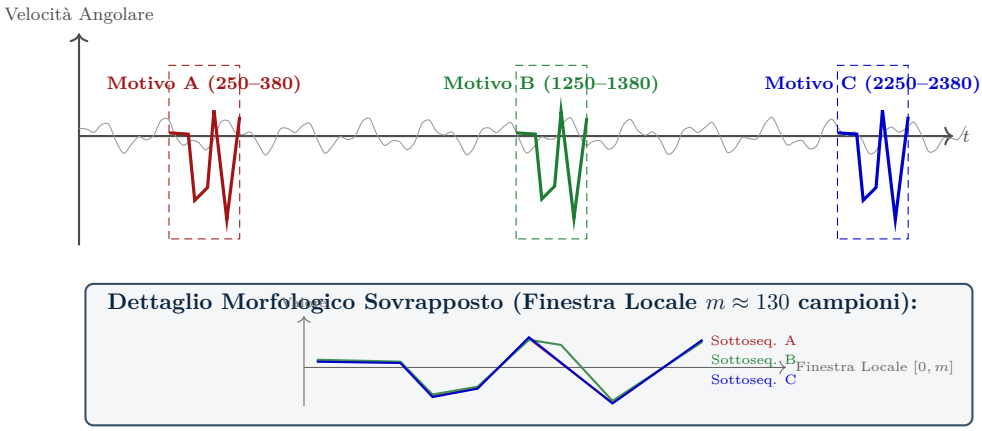


Figura 15.1: Rilevamento di motivi nel Winding Dataset. In alto: la serie temporale estesa ($0 \leq t \leq 2500$) con l'individuazione di tre occorrenze morfologicamente affini A, B, C. In basso: dettaglio isolato dei tre segmenti su finestra di osservazione $m \approx 130$, che ne comprova l'elevata invarianza di forma a dispetto di oscillazioni casuali e micro-variazioni locali.

L'osservazione del segnale grezzo evidenzia un comportamento altamente irregolare, con marcate oscillazioni di fondo e picchi negativi improvvisi. Tuttavia, un'analisi condotta a livello di sottosequenze con finestra $m \approx 130$ rivela tre segmenti distribuiti lungo l'orizzonte temporale:

- **Sottosequenza A** (evidenziata in rosso), posizionata nell'intervallo temporale $t \in [250, 380]$;
- **Sottosequenza B** (evidenziata in verde), posizionata nell'intervallo temporale $t \in [1250, 1380]$;
- **Sottosequenza C** (evidenziata in blu), posizionata nell'intervallo temporale $t \in [2250, 2380]$.

Isolando i tre segmenti su un asse temporale locale normalizzato $[0, 140]$, emerge una struttura geometrica condivisa:

1. **Regione di Plateau Iniziale ($0 \leq \tau \leq 35$):** segnale quasi costante con lievi increspature a bassa ampiezza;
2. **Transizione e Primo Minimo ($35 < \tau \leq 65$):** brusca discesa intorno a $\tau \approx 40$ verso un primo avvallamento, interrotto da un gradino flessurale intorno a $\tau \in [55, 60]$;
3. **Cresta Intermedia ($65 < \tau \leq 85$):** rapida salita con picco relativo pronunciato a $\tau \approx 80$;
4. **Depressione a "V" ($85 < \tau \leq 110$):** profonda discesa verso il minimo assoluto a $\tau \approx 105$;
5. **Recupero Finale ($110 < \tau \leq 130$):** rapida risalita verso una seconda cresta terminale a $\tau \approx 120$.

Nonostante minime perturbazioni locali (quali micro-oscillazioni stocastiche nella sottosequenza B rispetto alla traiettoria più levigata di C), la forma globale (*macro-shape*) si ripete fedelmente a distanza di oltre 1000 campioni temporali, confermando l'esistenza di un ciclo cinematico rigido nella macchina industriale.

15.1.3 Motivazioni e Applicazioni nel Data Mining

L'estrazione dei motivi non costituisce una mera operazione descrittiva fine a se stessa, bensì rappresenta il blocco costruttivo fondamentale (*primitive component*) per i compiti cardine del Data Mining su serie temporali:

1. **Estrazione di Regole Associative (*Association Rule Mining*)**: Nelle sequenze categoriche e nei panieri transazionali (Capitolo 11), le regole associano item discreti del tipo $X \implies Y$. Nelle serie temporali a valori reali continui non esistono elementi discreti nativi: l'estrazione preventiva dei motivi consente di discretizzare il continuum in un vocabolario di **forme primitive** (*primitive shapes*) e **pattern frequenti** (*frequent patterns*), abilitando la generazione di regole del tipo:

$$\text{Motif}_A \xrightarrow{\Delta t} \text{Motif}_B \implies \text{Picco di Pressione Entro 10 minuti} \quad (15.3)$$

2. **Classificazione di Serie Temporali (*Time Series Classification*)**: Numerosi modelli predittivi e classificatori supervisonati (quali 1-NN basati su shapelets o prototipi) fondano la frontiera decisionale sull'individuazione di **prototipi tipici** (*typical prototypes*) caratterizzanti ciascuna classe. I motivi estratti dalle sequenze di addestramento rappresentano i candidati ideali per identificare la "firma morfologica" distintiva di patologie cardiache (ECG), malfunzionamenti meccanici o oscillazioni sismiche.

3. **Rilevazione di Anomalie (*Time Series Anomaly Detection*)**: Una strategia consolidata di anomaly detection consiste nel modellare il **comportamento normale** di un apparato mediante l'insieme dei motivi conservati nella serie storica. Qualsiasi sottosequenza futura che presenti una distanza di dissimilarità significativamente elevata rispetto a tutti i motivi tipici noti viene classificata immediatamente come segnale anomalo o discorde (*discord*).

15.2 La Struttura Dati Matrix Profile: Fondamenti e Componenti

Tradizionalmente, la ricerca esatta di motivi richiedeva algoritmi a forza bruta altamente inefficienti con complessità computazionale quadratica rispetto alla lunghezza della serie moltiplicata per la dimensione della finestra. Nel 2016, il gruppo di ricerca di Eamonn Keogh e colleghi ha introdotto la **Matrix Profile** (**MP**) [26], una struttura dati unificante che annota una serie temporale e consente l'estrazione esatta, deterministica e scalabile di motivi, anomalie e forme discriminanti (*shapelets*).

15.2.1 Definizione e Concetto di Annotazione

La Matrix Profile è una meta-struttura dati bidimensionale composta da due vettori monodimensionali paralleli, la cui funzione è **annotare** la serie temporale originale rispetto a una finestra temporale m prescelta dall'analista.

Definizione 15.2.1 (Input della Matrix Profile)**.** La costruzione della Matrix Profile richiede esclusivamente due parametri in ingresso:

1. Una serie temporale univariata $T \in \mathbb{R}^n$;
2. Un parametro intero $m \in \mathbb{N}^+$ ($m \ll n$) che definisce la lunghezza della finestra di analisi per le sottosequenze.

La Matrix Profile risolve intrinsecamente il problema della sensibilità ai parametri: a differenza di altri metodi euristici o simbolici (come SAX, sottosezione 14.7.5, che richiedono la scelta del numero di segmenti PAA e della dimensione dell'alfabeto), la Matrix Profile non necessita di alcuna soglia arbitraria di distanza o euristica di quantizzazione.

15.2.2 Estrazione delle Sottosequenze tramite Finestra Scorrevole

Il fondamento geometrico della Matrix Profile risiede nell'applicazione di una **finestra scorrevole** (*sliding window*) di ampiezza m che scorre lungo la serie temporale T a passo unitario $\Delta i = 1$ dall'istante iniziale 1 all'istante finale $n - m + 1$.

Proposizione 15.2.1 (Numero Totale di Sottosequenze Estratte)**.** *Data una serie temporale T di cardinalità $|T| = n$ e una finestra mobile di ampiezza $m \leq n$, il numero totale N_{sub} di sottosequenze contigue estraibili è dato da:*

$$N_{sub} = |T| - m + 1 = n - m + 1 \tag{15.4}$$

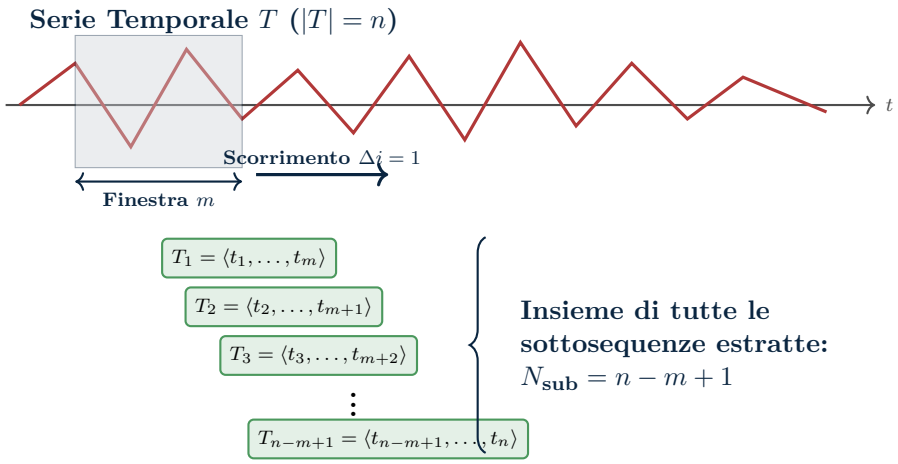


Figura 15.2: Processo di estrazione delle sottosequenze tramite finestra scorrevole (*sliding window*) di lunghezza m. Ciascuna sottosequenza contigua costituisce un'istanza $T_i \in \mathbb{R}^m$, per un totale complessivo di $N_{\text{sub}} = n - m + 1$ elementi.

15.2.3 La Matrice delle Distanze a Coppie (Pairwise Distance Matrix)

Dato l'insieme ordinato delle N_{sub} sottosequenze estratte, il passo concettuale ideale consisterebbe nel calcolare la matrice completa delle distanze a coppie (*All-Pairs Distance Matrix*) $\mathcal{D} \in \mathbb{R}^{N_{\text{sub}} \times N_{\text{sub}}}$:

$$\mathcal{D} = \begin{pmatrix} D(T_1, T_1) & D(T_1, T_2) & D(T_1, T_3) & \ldots & D(T_1, T_{N_{\text{sub}}}) \\ D(T_2, T_1) & D(T_2, T_2) & D(T_2, T_3) & \ldots & D(T_2, T_{N_{\text{sub}}}) \\ D(T_3, T_1) & D(T_3, T_2) & D(T_3, T_3) & \ldots & D(T_3, T_{N_{\text{sub}}}) \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ D(T_{N_{\text{sub}}}, T_1) & D(T_{N_{\text{sub}}}, T_2) & D(T_{N_{\text{sub}}}, T_3) & \ldots & D(T_{N_{\text{sub}}}, T_{N_{\text{sub}}}) \end{pmatrix} \tag{15.5}$$

Si consideri un estratto numerico 3×3 della matrice $\mathcal{D}$:

$$\mathcal{D}_{3\times 3} = \begin{pmatrix} 0 & 7.6952 & 7.7399 \\ 7.6952 & 0 & 7.7106 \\ 7.7399 & 7.7106 & 0 \end{pmatrix} \tag{15.6}$$

Dalla definizione metrica di distanza scaturiscono due proprietà algebriche fondamentali:

1. **Diagonale Principale Nulla**: $\mathcal{D}[i, i] = D(T_i, T_i) = 0$ per ogni i, in virtù dell'assioma di identità degli indiscernibili della metrica.

2. **Simmetria Rispetto alla Diagonale**: $\mathcal{D}[i, j] = \mathcal{D}[j, i] = D(T_i, T_j)$ per ogni coppia (i, j), in forza della simmetria metrica.

Tuttavia, memorizzare esplicitamente la matrice $\mathcal{D}$ comporta un'occupazione di memoria quadratica $\mathcal{O}(N_{\text{sub}}^2) \approx \mathcal{O}(n^2)$. Per una serie temporale di moderata entità con $n = 10^6$ campioni, la matrice conterrebbe 10^{12} elementi a virgola mobile a 64 bit, richiedendo oltre 8 Terabyte di RAM, rendendo tale rappresentazione impraticabile.

15.2.4 Riduzione al Nearest Neighbor (1-NN)

L'intuizione fondante della Matrix Profile risiede nella consapevolezza che per estrarre motivi e anomalie **non è necessario conservare l'intera matrice delle distanze**: per ciascuna sottosequenza T_i è sufficiente conoscere e memorizzare unicamente l'informazione relativa al suo **vicino più prossimo** (*closest 1-Nearest Neighbor*, 1-NN) all'interno dell'intera serie.

La Tabella 15.1 illustra la transizione concettuale tra le sottosequenze estratte, il valore scalare del 1-NN e il corrispondente puntatore temporale.

Tabella 15.1: Processo di riduzione al 1-Nearest Neighbor (Slide 7): mappatura tra le sottosequenze estratte, la relativa distanza minima e il rispettivo vicino più affine.

Sottosequenza	**Profilo Morfologico Locale**	**Distanza 1-NN** ($P[i]$)	**Nearest Neighbor** ($T_{I[i]}$)	**Classificazione Strutturale**
T_1	Rumore piatto privo di struttura	6.04	T_{840}	Rumore stocastico
T_2	Flessione iniziale ad ampiezza contenuta	2.88	T_{312}	Transizione
$\mathbf{T_3}$	**Forma a doppia cresta conservata**	**0.90**	$\mathbf{T_{12}}$	**Motif primario**
T_4	Discesa dalla cresta morfologica	1.61	T_{13}	Transizione
T_5	Rumore stocastico non correlato	5.69	T_{1520}	Rumore stocastico
$\mathbf{T_6}$	**Picco ripido seguito da discesa**	**1.23**	$\mathbf{T_{145}}$	**Motif secondario**
T_7	Risalita dalla depressione morfologica	1.40	T_{146}	Transizione

Dal confronto numerico emerge con immediata evidenza empirica una separazione netta:

- I segmenti associati a **forme ricorrenti e conservative** (come T_3 con distanza 0.90, o T_6 con distanza 1.23) presentano una distanza minima dal proprio nearest neighbor straordinariamente contenuta;
- I segmenti composti da **fluttuazioni casuali o rumore** (come T_1 con distanza 6.04, e T_5 con distanza 5.69) possiedono distanze minime sensibilmente superiori, attestando l'assenza di pattern gemelli nella sequenza.

15.2.5 I Vettori Fondamentali: Matrix Profile (P) e Matrix Profile Index (I)

La condensazione del 1-NN per tutte le N_{sub} sottosequenze genera due vettori monodimensionali paralleli, la cui combinazione definisce la struttura dati completa della Matrix Profile.

Definizione 15.2.2 (Vettore Matrix Profile P)**.** Il **Matrix Profile** è un vettore monodimensionale continuo $P \in \mathbb{R}^{N_{\text{sub}}}$ di cardinalità $|P| = n - m + 1$. Il valore alla posizione i, denotato con $P[i]$, quantifica la distanza minima tra la sottosequenza T_i e il suo vicino più prossimo in T al di fuori della zona di esclusione:

$$P[i] = \min_{\substack{j \in \{1,\dots,N_{\text{sub}}\} \\ j \notin \text{Excl}(i)}} D(T_i, T_j) \tag{15.7}$$

Definizione 15.2.3 (Vettore Matrix Profile Index I)**.** Il **Matrix Profile Index** (o *Profile Companion*) è un vettore monodimensionale intero $I \in \mathbb{N}^{N_{\text{sub}}}$ di cardinalità $|I| = n-m+1$. La coordinata $I[i]$ memorizza l'indice temporale di inizio della sottosequenza che realizza la distanza minima $P[i]$:

$$I[i] = \arg \min_{\substack{j \in \{1,\dots,N_{\text{sub}}\} \\ j \notin \text{Excl}(i)}} D(T_i, T_j) \tag{15.8}$$

Esempio 15.2.1 (Meccanismo di Puntamento di I)**.** Si consideri un estratto del vettore I in corrispondenza degli indici compresi tra 192 e 196:

$$I[192] = 89, \quad I[193] = 90, \quad \mathbf{I[194] = 91}, \quad I[195] = 92, \quad I[196] = 93 \tag{15.9}$$

Il valore $I[194] = 91$ certifica inequivocabilmente che la sottosequenza T_{194} ha come vicino più affine nell'intera serie la sottosequenza che si sviluppa a partire dall'indice $t = 91$, e la loro distanza euclidea coincide con $P[194]$.

15.2.6 Proprietà del Matrix Profile Index e Classic Time Series Motif

L'accoppiamento dei vettori P e I conferisce proprietà computazionali e strutturali di primaria rilevanza:

1. **Accesso in Tempo Costante** $\mathcal{O}(1)$: Dato l'indice i di una qualsiasi sottosequenza T_i, il suo nearest neighbor viene reperito istantaneamente tramite una singola dereferenziazione in memoria: $T_{\text{NN}} = T_{I[i]}$, senza dover effettuare alcuna scansione o scansione lineare sui dati.

2. **Asimmetria Generale dei Puntatori**: La relazione di vicinanza indotta da I **non è simmetrica**:

$$I[A] = B \not\Longleftrightarrow I[B] = A \tag{15.10}$$

Se la sottosequenza A possiede come vicino più prossimo B, non è affatto garantito che B punti reciprocamente ad A. Ad esempio, B potrebbe risiedere in un cluster più denso avente una terza sottosequenza C posizionata a distanza ancora inferiore da B rispetto ad A ($D(B, C) < D(B, A)$).

3. **Condizione di Reciprocità del Classic TS Motif**: Una coppia di sottosequenze (T_a, T_b) costituisce il **motivo classico assoluto** della serie temporale se e solo se sono verificate simultaneamente due condizioni di consistenza globale:

$$\text{Minimo Globale di Distanza:} \quad P[a] = P[b] = \min_k P[k] \tag{15.11}$$

$$\text{Puntatori Reciproci Mutui:} \quad I[a] = b \quad \wedge \quad I[b] = a \tag{15.12}$$

In Figura 15.3 viene schematizzata l'architettura d'integrazione e allineamento tra la serie originale T, il profilo delle distanze P e l'indice di puntamento I.

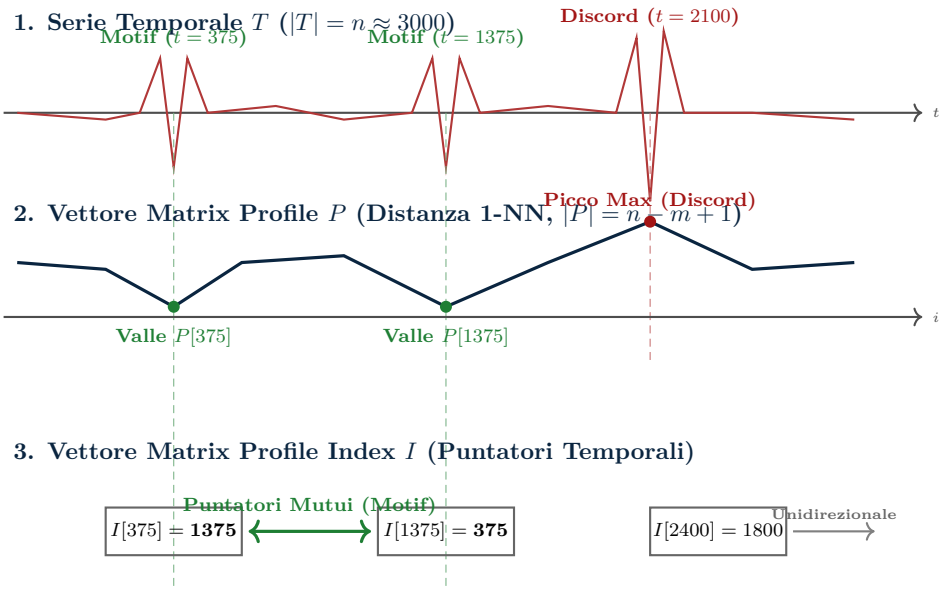


Figura 15.3: Integrazione e allineamento verticale dei tre livelli della Matrix Profile. Il vettore P mappa le similarità evidenziando valli profonde in corrispondenza dei motivi e picchi in corrispondenza delle anomalie. Il vettore I codifica i puntatori temporali, distinguendo tra corrispondenze mutuali rigorose ($I[a] = b \wedge I[b] = a$) e puntatori asimmetrici unidirezionali.

15.3 Interpretazione della Matrix Profile: Motifs e Discords

L'ispezione della curva continua del Matrix Profile P fornisce una chiave di lettura topologica immediata dello stato di regolarità della serie temporale originale:

15.3.1 Lettura Topologica del Profilo: Valli e Picchi

L'ordinata del profilo $P[i]$ quantifica l'isolamento o la prossimità di T_i rispetto al resto della serie:

- **Valori Relativamente Bassi (Valli e Minimi Locali)**: Se $P[i]$ assume un valore prossimo allo zero o sensibilmente inferiore al livello medio del profilo, significa che esiste almeno un'altra sottosequenza nella serie la cui conformazione è quasi indistinguibile da T_i. Di conseguenza, le regioni a bassa magnitudo individuano inequivocabilmente **forme conservate** (**Motifs**).

- **Valori Relativamente Elevati (Picchi e Massimi Globali)**: Se $P[i]$ assume un valore eccezionalmente elevato, significa che anche la sottosequenza più simile presente nell'intera serie dista notevolmente da T_i. La sottosequenza T_i possiede dunque una forma unica ed irripetibile lungo l'intera storia di osservazione. Tali formazioni individuano le anomalie strutturali più severe, denominate formalmente **Time Series Discords**.

15.3.2 Quadro Comparativo: Motifs vs Time Series Discords

La Tabella 15.2 riassume formalmente la dualità complementare tra i due concetti cardine della Matrix Profile.

Tabella 15.2: Quadro sinottico comparativo: Time Series Motifs vs Time Series Discords.

Proprietà / Caratteristica	**Time Series Motifs**	**Time Series Discords**
Definizione Concettuale	Pattern o morfologie frequenti e conservative.	Anomalie morfologiche e pattern rari/unici.
Firma nel Profilo P	**Minimi Locali e Assoluti** (valli del profilo).	**Massimo Globale o Relativo** (picchi del profilo).
Condizione Formale	$i^* = \arg\min_i P[i]$ con vincolo di mutua reciprocità in I.	$j^* = \arg\max_j P[j]$.
Struttura dei Puntatori I	Tipicamente **mutui e bidirezionali** ($I[a] = b \wedge I[b] = a$).	Spesso **asimmetrici** verso regioni dense di rumore.
Spazio Geometrico $\mathbb{R}^m$	Centro o interno di **cluster ad alta densità**.	**Punti isolati (*outlier*)** a grande distanza.
Obiettivo Applicativo	Estrazione di regole associative, compressione, prototipi.	Rilevamento guasti, aritmie cardiache, intrusioni di rete.

15.4 Algoritmo di Calcolo della Matrix Profile e Complessità

Per illustrare il funzionamento algoritmico della Matrix Profile, si analizza l'algoritmo **STAMP** (*Scalable Time series Anytime Matrix Profile*) [26], descrivendone la convergenza iterativa attraverso un modello didattico (*toy example*) monodimensionale su un vettore di 19 posizioni campionarie.

15.4.1 Fase 0: Inizializzazione

Dati in ingresso la serie T e la dimensione della finestra m, si determina il numero di sottosequenze $N_{\text{sub}} = n - m + 1 = 19$.

Poiché l'algoritmo procede per minimizzazioni successive, il vettore della Matrix Profile viene inizializzato ponendo ogni suo elemento pari a infinito ($+\infty$):

$$P^{(0)} = [\infty, \infty, \infty, \infty, \infty, \infty, \infty, \infty, \infty, \infty, \infty, \infty, \infty, \infty, \infty, \infty, \infty, \infty, \infty] \tag{15.13}$$

Parallelamente, il vettore I viene inizializzato con valori nulli o indefiniti ($I^{(0)} = [\text{null}, \ldots, \text{null}]$).

15.4.2 Traccia Didattica Dettagliata delle Iterazioni

L'algoritmo opera selezionando a ogni iterazione una sottosequenza T_i non ancora elaborata, calcolando il vettore delle distanze rispetto a tutte le altre sottosequenze e aggiornando P mediante il minimo elemento per elemento (*elementwise minimum*).

Iterazione 1 ($k = 1$)

1. **Selezione della Sottosequenza**: Si seleziona (eventualmente in modo casuale o sequenziale) la sottosequenza all'indice $i = 3$, corrispondente alla prima occorrenza di un pattern morfologico strutturato.

2. **Calcolo del Vettore delle Distanze $D^{(1)}$**: Si calcolano le distanze tra la sottosequenza perno T_3 e ciascuna delle 19 sottosequenze della serie, producendo il vettore delle distanze:

   $$D^{(1)} = [3, 2, \mathbf{0}, 5, 3, 4, 5, 1, 2, 9, 8, 4, 2, 3, 4, 8, 6, 2, 1] \tag{15.14}$$

   Si noti che l'elemento alla posizione 3 vale rigorosamente $D^{(1)}[3] = 0$, esprimendo la distanza identica della sottosequenza T_3 rispetto a se stessa (*self-match*).

3. **Aggiornamento Elementwise Minimum con Esclusione del Self-Match**: Il vettore P viene aggiornato confrontando ciascun elemento corrente con la nuova distanza calcolata:

$$P^{(1)}[j] = \min(P^{(0)}[j], D^{(1)}[j]), \qquad \text{per ogni } j \neq 3 \tag{15.15}$$

Per la posizione del perno ($j = 3$), il valore 0 viene escluso per prevenire l'auto-accoppiamento banale, mantenendo $P^{(1)}[3] = \infty$.

L'applicazione dell'operatore di minimo determina:

$$P^{(1)}[1] = \min(\infty, 3) = 3 \tag{15.16}$$

$$P^{(1)}[2] = \min(\infty, 2) = 2 \tag{15.17}$$

$$P^{(1)}[3] = \infty \quad \text{(esclusione auto-distanza)} \tag{15.18}$$

$$P^{(1)}[4 \ldots 19] = D^{(1)}[4 \ldots 19] \tag{15.19}$$

Il profilo al termine dell'iterazione 1 risulta pertanto:

$$P^{(1)} = [3, 2, \boldsymbol{\infty}, 5, 3, 4, 5, 1, 2, 9, 8, 4, 2, 3, 4, 8, 6, 2, 1] \tag{15.20}$$

Iterazione 2 ($k = 2$)

1. **Selezione della Sottosequenza**: Viene selezionata una seconda sottosequenza perno, collocata all'indice $j = 12$ (corrispondente alla seconda occorrenza del pattern strutturato).

2. **Calcolo del Vettore delle Distanze $D^{(2)}$**: Il calcolo delle distanze da T_{12} rispetto a tutte le sottosequenze produce il vettore:

$$D^{(2)} = [2, 3, \mathbf{1}, 4, 4, 3, 6, 2, 1, 5, 8, \mathbf{0}, 2, 3, 5, 9, 4, 2, 2] \tag{15.21}$$

In $D^{(2)}$ si evidenziano due valori cruciali:

- In posizione 12: $D^{(2)}[12] = 0$ (auto-distanza del perno);
- In posizione 3: $D^{(2)}[3] = 1$, a testimonianza della quasi-identità morfologica tra la sottosequenza T_{12} e la sottosequenza T_3.

3. **Aggiornamento Elementwise Minimum**: Si applica l'aggiornamento $P^{(2)}[k] = \min(P^{(1)}[k], D^{(2)}[k])$ escludendo l'auto-match $k = 12$:

$$P^{(2)}[1] = \min(3, 2) = \mathbf{2} \quad \text{(miglioramento da 3 a 2)} \tag{15.22}$$

$$P^{(2)}[2] = \min(2, 3) = 2 \quad \text{(invariato)} \tag{15.23}$$

$$P^{(2)}[3] = \min(\infty, 1) = \mathbf{1} \quad \text{(trovato vicino } T_{12} \text{ a dist. 1)} \tag{15.24}$$

$$P^{(2)}[4] = \min(5, 4) = 4 \quad \text{(miglioramento da 5 a 4)} \tag{15.25}$$

Al termine del passaggio, il profilo $P^{(2)}$ ha aggiornato significativamente le proprie stime, convergendo progressivamente verso la vera frontiera inferiore delle distanze minime.

La Tabella 15.3 sintetizza l'evoluzione passo-passo dei vettori D e P per le prime due iterazioni dell'algoritmo.

Tabella 15.3: Traccia numerica delle prime due iterazioni dell'algoritmo Matrix Profile su vettore a 19 celle.

Vettore / Indice	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**	**16**	**17**	**18**	**19**
$P^{(0)}$	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
$D^{(1)}$ (Perno $i = 3$)	3	2	**0**	5	3	4	5	1	2	9	8	4	2	3	4	8	6	2	1
$P^{(1)}$	3	2	$\boldsymbol{\infty}$	5	3	4	5	1	2	9	8	4	2	3	4	8	6	2	1
$D^{(2)}$ (Perno $i = 12$)	2	3	1	4	4	3	6	2	1	5	8	**0**	2	3	5	9	4	2	2
$P^{(2)}$	**2**	2	**1**	**4**	3	**3**	5	1	**1**	**5**	8	4	2	3	4	8	**4**	2	1

15.4.3 Algoritmo Generale e Analisi di Complessità Temporale

In algorithm 7 viene formalizzata la struttura procedurale completa per il calcolo della Matrix Profile.

Algorithm 7: Calcolo Esatto della Matrix Profile mediante Algoritmo STAMP

Dati: Serie temporale $T \in \mathbb{R}^n$, ampiezza finestra $m \in \mathbb{N}^+$, ampiezza zona esclusione $w \in \mathbb{N}^+$

Risultato: Vettore Matrix Profile $P \in \mathbb{R}^{N_{\text{sub}}}$, Vettore Profile Index $I \in \mathbb{N}^{N_{\text{sub}}}$

$N_{\text{sub}} \leftarrow n - m + 1$
Inizializza $P[1 \dots N_{\text{sub}}] \leftarrow +\infty$
Inizializza $I[1 \dots N_{\text{sub}}] \leftarrow \text{null}$
Genera permutazione casuale degli indici $\Pi \leftarrow \text{RandomPermutation}(\{1, \dots, N_{\text{sub}}\})$
for $k \leftarrow 1$ **to** N_{sub} **do**
    $i \leftarrow \Pi[k]$
    `// Sottosequenza perno corrente` T_i
    $D \leftarrow \text{MASS}(T_i, T)$
    `// Calcolo distanze in tempo` $\mathcal{O}(n \log n)$
    for $j \leftarrow 1$ **to** N_{sub} **do**
        if $|i - j| \geq w$ **then**
            `// Verifica zona di esclusione per trivial matches`
            if $D[j] < P[j]$ **then**
                $P[j] \leftarrow D[j]$
                $I[j] \leftarrow i$
            if $D[j] < P[i]$ **then**
                $P[i] \leftarrow D[j]$
                $I[i] \leftarrow j$
return P, I

Teorema 15.4.1 (Complessità Computazionale della Matrix Profile)**.** *Sia data una serie temporale T di lunghezza n e una finestra di sottosequenza m. L'algoritmo di calcolo della Matrix Profile possiede:*

1. ***Complessità Temporale**: $\mathcal{O}(n^2 \log n)$;*
2. ***Complessità Spaziale Ausiliaria**: $\mathcal{O}(n)$.*

Dimostrazione. La complessità deriva dalla combinazione di due fattori analitici:

1. **Numero di Iterazioni del Ciclo Esterno**: Il ciclo scorre su tutte le $N_{\text{sub}} = n - m + 1$ sottosequenze della serie temporale. Asintoticamente, per $m \ll n$, il numero di iterazioni è dell'ordine di $\mathcal{O}(n)$.

2. **Costo di Ciascuna Iterazione (Algoritmo MASS)**: Per calcolare le distanze euclidee z-normalizzate tra la sottosequenza perno T_i e tutte le $n - m + 1$ sottosequenze estratte da T, non si impiega il ciclo interno a forza bruta di complessità $\mathcal{O}(n \cdot m)$. Si impiega l'algoritmo **MASS** (*Mueen's Algorithm for Similarity Search*) [27], basato sulla Trasformata Rapida di Fourier (FFT) e sulla convoluzione nel dominio delle frequenze (*cross-correlation theorem*). La complessità di MASS è indipendente da m e pari a $\mathcal{O}(n \log n)$.

Moltiplicando il numero complessivo di iterazioni per il costo di ciascun calcolo delle distanze si ottiene la complessità temporale globale:

$$\text{Complessità Temporale Totale} = \mathcal{O}(n) \cdot \mathcal{O}(n \log n) = \mathcal{O}\Big(n^2 \log n\Big) \quad (15.26)$$

Per quanto concerne la memoria primaria, l'algoritmo non materializza la matrice $\mathcal{D} \in \mathbb{R}^{n \times n}$. Vengono allocati unicamente i vettori $P \in \mathbb{R}^{n-m+1}$, $I \in \mathbb{N}^{n-m+1}$ e un buffer ausiliario per il vettore temporaneo delle distanze D, determinando un'occupazione di memoria rigorosamente lineare pari a $\mathcal{O}(n)$. □

Nota 15.4.1 (Proprietà *Anytime* dell'Algoritmo STAMP)**.** Poiché le sottosequenze perno possono essere elaborate secondo una permutazione puramente casuale Π, l'algoritmo STAMP gode della proprietà **Anytime**: l'utente o il sistema può interrompere l'esecuzione in qualsiasi istante prima del completamento naturale del ciclo. Il vettore P parzialmente calcolato costituisce già un'eccellente approssimazione superiore della Matrix Profile esatta, consentendo di individuare con elevatissima confidenza i motivi principali anche dopo una frazione limitata (es. 5%) del tempo di calcolo complessivo.

15.5 Estrazione dei Motifs e Geometria nello Spazio m-Dimensionale

Una volta calcolato il vettore P, l'estrazione operativa dei motivi si traduce nella ricerca sistematica dei suoi **minimi locali**, mentre la comprensione del fenomeno trae beneficio da un'elegante interpretazione geometrica nello spazio euclideo multidimensionale.

15.5.1 Localizzazione dei Motifs tramite Minimi Locali

Ciascun minimo locale di P corrisponde all'epicentro temporale di una sottosequenza dotata di forte corrispondenza nella serie. Tuttavia, nella pratica

applicativa, non tutti i minimi locali rappresentano motivi distinti. Per isolare i motivi dominanti di rango $1, 2, \ldots, K$ si adotta una procedura iterativa:

1. Si individua il minimo assoluto globale di P: $i_1 = \arg\min_i P[i]$. La coppia $(T_{i_1}, T_{I[i_1]})$ definisce il **1° Motivo della Serie**.

2. Per identificare il secondo motivo indipendente, si applica una finestra di esclusione di raggio $w = m/2$ attorno sia a i_1 sia a $I[i_1]$, azzerando o ponendo a $+\infty$ i rispettivi valori in P, al fine di escludere repliche leggermente sfasate del medesimo fenomeno fisico.

3. Si ricerca il nuovo minimo di P: la nuova coppia individua il **2° Motivo della Serie**, e si itera fino al raggiungimento del numero K di motivi desiderato.

15.5.2 Interpretazione Geometrica nello Spazio m-Dimensionale ($\mathbb{R}^m$)

Il legame tra i minimi della Matrix Profile e i pattern ricorrenti della serie temporale può essere compreso geometricamente considerando ciascuna delle N_{sub} sottosequenze di lunghezza m come un singolo punto $\mathbf{x}_i \in \mathbb{R}^m$:

$$T_i = \langle t_i, t_{i+1}, \ldots, t_{i+m-1} \rangle \longleftrightarrow \mathbf{x}_i = (x_1, x_2, \ldots, x_m) \in \mathbb{R}^m \tag{15.27}$$

In questa rappresentazione vettoriale:

- Sottosequenze aventi forma o morfologia identica si mappano in punti geometricamente vicini nello spazio Euclideo $\mathbb{R}^m$;

- Di conseguenza, l'esistenza di pattern che si ripetono più volte lungo la sequenza genera **regioni dense** (*dense regions* o cluster compatti di punti in $\mathbb{R}^m$);

- Ciascun punto appartenente a una regione densa possiede un vicino a distanza infinitesima o minima, determinando una pronunciata **valle (valore basso)** nel profilo P;

- Al contrario, sottosequenze isolate, singolarità morfologiche e fluttuazioni anomale (*discords*) si collocano in regioni iper-sparse di $\mathbb{R}^m$, lontane da qualunque altro addensamento, generando un picco elevato nel profilo P.

In Figura 15.4 viene illustrata la proiezione bidimensionale dello spazio $\mathbb{R}^m$.

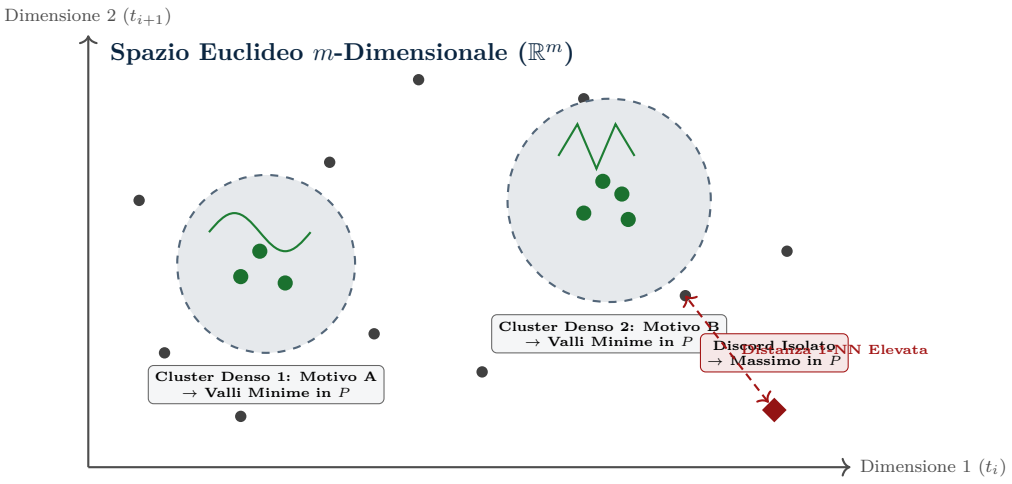


Figura 15.4: Interpretazione geometrica delle sottosequenze nello spazio m-dimensionale $\mathbb{R}^m$. Gli addensamenti compatti (*dense clusters*) raggruppano sottosequenze con forma affine e generano le valli minime della Matrix Profile. I punti isolati (*discords*) distano marcatamente da qualunque altro vicino e generano i picchi del profilo.

15.5.3 La Zona di Esclusione (*Exclusion Zone*) e Trivial Matches

Nella formulazione iniziale dell'algoritmo (e nella semplificazione didattica presentata nella Slide 16 e Slide 18), l'aggiornamento della Matrix Profile si limita ad escludere l'auto-distanza della sottosequenza rispetto a se stessa ($j = i$, con $D(T_i, T_i) = 0$).

In un contesto reale di serie temporali continue, tale esclusione è **condizione necessaria ma non sufficiente**:

Il Problema dei Trivial Matches (Accoppiamenti Banali)

Due sottosequenze consecutive T_i e T_{i+1} differiscono per una sola osservazione iniziale t_i e una finale t_{i+m}, condividendo integralmente $m-1$ campioni intermedi. A causa di tale massiccia sovrapposizione geometrica, la distanza $D(T_i, T_{i+1})$ risulta artificialmente quasi nulla, generando un falso minimo banale (*trivial match*) che non riflette affatto una vera ricorrenza strutturale, bensì un semplice sfasamento unitario.

Per eliminare rigorosamente questo artefatto, la letteratura standard della Matrix Profile [26] impone una **Zona di Esclusione** (*Exclusion Zone*) simmetrica attorno all'indice perno i:

$$\text{Excl}(i) = \{j \in \mathbb{N} \mid |i - j| < w\} \tag{15.28}$$

dove l'ampiezza di esclusione viene convenzionalmente fissata pari alla metà della lunghezza della finestra:

$$w = \left\lfloor \frac{m}{2} \right\rfloor \tag{15.29}$$

In tal modo, nessun segmento che condivida più del 50% delle osservazioni con T_i può concorrere all'assegnazione del 1-Nearest Neighbor in $P[i]$ e $I[i]$.

15.6 Quadro Sinottico e Riferimenti Bibliografici

A conclusione del capitolo, in Tabella 15.4 viene proposto un quadro sinottico riassuntivo dei concetti cardine, delle strutture algebriche e dei vincoli computazionali della Matrix Profile.

Tabella 15.4: Quadro sinottico riassuntivo dei parametri, vettori e complessità della Matrix Profile.

Elemento	**Dominio / Notazione**	**Descrizione e Rilevanza Operativa**
Serie Temporale	$T \in \mathbb{R}^n$	Sequenza univariata di osservazioni continue campionate regolarmente.
Ampiezza Finestra	$m \in \mathbb{N}^+$ $(m \ll n)$	Unico parametro essenziale: definisce la scala temporale dei pattern cercati.
Numero Sottosequenze	$N_{\text{sub}} = n - m + 1$	Numero di segmenti contigui analizzati tramite sliding window unitaria.
Matrix Profile	$P \in \mathbb{R}^{N_{\text{sub}}}$	Vettore continuo delle distanze dal 1-NN; valli = motivi, picchi = discords.
Profile Index	$I \in \mathbb{N}^{N_{\text{sub}}}$	Vettore intero dei puntatori al 1-NN; accesso $\mathcal{O}(1)$ al compagno più affine.
Classic TS Motif	$P[a] = P[b] = \min P$	Coppia di minima distanza globale avente puntatori mutui $(I[a] = b \wedge I[b] = a)$.
Time Series Discord	$j^* = \arg\max P$	Sottosequenza più isolata nella serie; rileva anomalie e guasti operativi.
Zona di Esclusione	$\lvert i - j \rvert \geq m/2$	Filtro vincolante per eliminare i falsi motivi dovuti a sovrapposizione temporale.
Complessità STAMP	$\mathcal{O}(n^2 \log n)$	Tempo di calcolo totale esatto basato sull'algoritmo MASS (FFT); memoria $\mathcal{O}(n)$.

Nota 15.6.1 (Nota Bibliografica e Storiografica)**.** Nelle slide di riferimento il lavoro scientifico fondamentale sulla Matrix Profile viene attribuito a *Chin-Chia Michael Yeh et al., 1997*. Si tratta di un refuso tipografico presente

nel materiale didattico originale: il lavoro seminale *"Matrix Profile I: All Pairs Similarity Joins for Time Series: A Unifying View that Includes Motifs, Discords and Shapelets"* è stato ufficialmente pubblicato nel **2016** negli atti della conferenza IEEE ICDM (*IEEE International Conference on Data Mining*) [26], scaturendo dalla collaborazione tra l'Università della California (Riverside) e l'Università del New Mexico.

Capitolo 16

Laboratorio Python: Analisi, Trasformazioni e Mining di Serie Temporali

16.1 Ambiente di Lavoro, Stack Software e Dataset del Laboratorio

Il presente laboratorio traduce in pratica computazionale i fondamenti teorici esposti nei Capitoli 14 e 15, guidando lo studente nell'implementazione di pipeline complete per l'analisi, la trasformazione, il calcolo delle distanze elastiche, il clustering e la scoperta di motivi (*motif discovery*) su serie temporali mediante lo stack scientifico di Python [2].

Ambiente Interattivo e Riproducibilità del Laboratorio

Tutti gli esempi, le pipeline di trasformazione e i blocchi di codice presentati in questo capitolo sono riproducibili ed eseguibili interattivamente tramite i tre notebook Jupyter inclusi nella cartella `Data-Mining/notebooks/10_time_series/`:

1. `8_TimeSeries1.ipynb`: Trasformazioni, scaling, distanze (Euclidea, DTW vincolata), estrazione di feature, dissimilarità CDM e approssimazioni discrete (PAA, SAX, 1D-SAX);
2. `9_TimeSeries2.ipynb`: Clustering di serie temporali (Shape-based con K-Means e DBA, Feature-based, Compression-based con DBSCAN, PAA-based);
3. `10_TimeSeries_motif.ipynb`: Calcolo della Matrix Profile (STOMP), estrazione dei motivi (*motifs*) e rilevamento di anomalie morfologiche (*discords*).

Avvio con Docker:

- Da terminale nella radice del repository: `docker compose up jupyter`
- Interfaccia web: `http://localhost:8888/` (cartella `10_time_series`).

Lo stack software fondamentale per il Time Series Mining in Python fa perno sulle librerie specialistiche `tslearn`, `pyts` e `matrixprofile`, integrate con l'ecosistema scientifico standard:

```
import math
import numpy as np
import pandas as pd
import scipy.stats as stats
import matplotlib.pyplot as plt
import seaborn as sns
import zlib

# Librerie specialistiche per serie temporali
import tslearn
from tslearn.preprocessing import
    TimeSeriesScalerMeanVariance, TimeSeriesScalerMinMax,
    TimeSeriesResampler
from tslearn.metrics import dtw, dtw_path, cdist_dtw
from tslearn.piecewise import
    PiecewiseAggregateApproximation,
    SymbolicAggregateApproximation,
    OneD_SymbolicAggregateApproximation
```

```
from tslearn.clustering import TimeSeriesKMeans

import pyts
from pyts.datasets import make_cylinder_bell_funnel
from pyts.metrics import dtw as dtw_pyts

import matrixprofile as mp
from matrixprofile import matrixProfile
from matrixprofile.motifs import motifs
from matrixprofile.discords import discords
```

16.1.1 I Dataset Sperimentali del Laboratorio

Le esercitazioni si sviluppano su tre scenari di dati a complessità crescente:

1. **Sequenze Sintetiche Armoniche** (ts_1, ts_2): due traiettorie continue di 200 campioni generate combinando una componente deterministica sinusoidale con rumore stocastico uniforme:
$$ts_1(t) = \sin\left(\frac{t}{20}\right) + \frac{\epsilon_1(t)}{2}, \qquad ts_2(t) = \sin\left(\frac{t}{20}\right) + \frac{\epsilon_2(t)}{2}, \quad \epsilon_i \sim \mathcal{U}(0,1) \tag{16.1}$$
utilizzate per validare visivamente le trasformazioni elementari, il filtraggio e l'effetto dello scaling.

2. **Dataset Internazionale AirPassengers**: serie storica mensile composta da $n = 144$ osservazioni (dal 1949 al 1960) del numero totale di passeggeri di linee aeree internazionali espresso in migliaia. Il segnale manifesta sia un pronunciato **trend lineare esponenziale crescente**, sia una forte **componente stagionale periodica annuale** ($T = 12$ mesi), costituendo il banco di prova ideale per la detrendizzazione, la Matrix Profile e il rilevamento di motivi e anomalie.

3. **Benchmark Multiclasse Cylinder-Bell-Funnel (CBF)**: archivio sintetico generato tramite la funzione `pyts.datasets.make_cylinder_bell_funnel` composto da 60 serie temporali di 128 campioni equamente ripartite in tre classi morfologiche distinte (Cylinder: plateau rettangolare; Bell: campana asimmetrica a salita ripida e discesa dolce; Funnel: imbuto a salita dolce e discesa ripida).

In Figura 16.1 viene schematizzata la pipeline globale delle attività di laboratorio sviluppate nel capitolo.

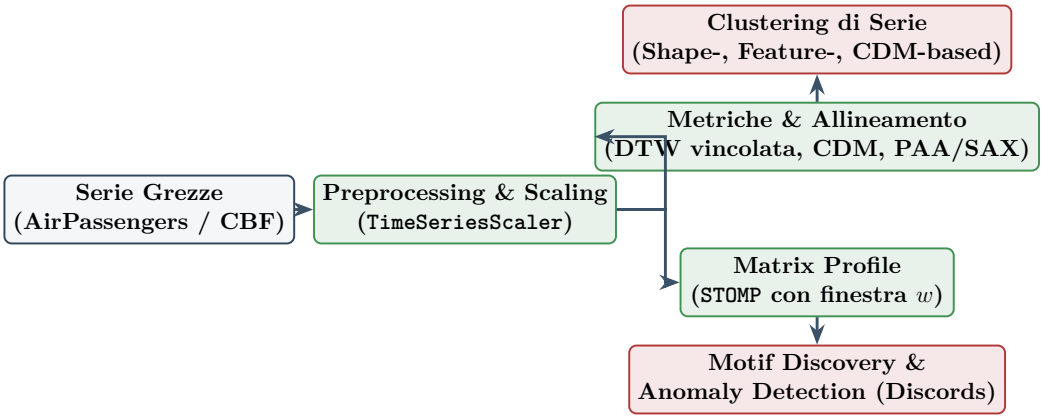


Figura 16.1: Architettura metodologica della pipeline di Time Series Mining del laboratorio: dall'ingestione e scaling alla quantificazione delle distanze, clustering e scoperta di motivi/anomalie.

16.2 Trasformazioni, Preprocessing e Scaling delle Sequenze

L'applicazione diretta di metriche di distanza su segnali grezzi conduce frequentemente a risultati fuorvianti qualora le sequenze siano affette da sfasamenti di ampiezza, offset verticali, trend macroscopici o frequenze di campionamento disomogenee (sezione 14.2).

16.2.1 Trasformazioni Elementari con NumPy e Pandas

Le quattro trasformazioni fondamentali analizzate nel notebook didattico operano secondo le formulazioni della teoria:

```
np.random.seed(0)
# Generazione di due serie armoniche sintetiche
ts1 = pd.Series([np.sin(float(x) / 20.0) + np.random.rand()
    / 2.0 for x in range(200)])
ts2 = pd.Series([np.sin(float(x) / 20.0) + np.random.rand()
    / 2.0 for x in range(200)])

# 1. Offset Translation: traslazione verticale per
    annullare la media
ts1_offset = ts1 - ts1.mean()
ts2_offset = ts2 - ts2.mean()

# 2. Amplitude Scaling: normalizzazione z-score (media
    nulla e varianza unitaria)
```

```
ts1_zscore = (ts1 - ts1.mean()) / ts1.std()
ts2_zscore = (ts2 - ts2.mean()) / ts2.std()

# 3. Noise Smoothing: abbattimento del rumore ad alta frequenza via media mobile
w = 15
ts1_smoothed = ts1_zscore.rolling(window=w).mean()

# 4. Linear Trend Removal su AirPassengers: rimozione del trend stagionale a 12 mesi
df = pd.read_csv('air_passengers.csv', parse_dates=True, index_col='Month')
ts_air = df['#Passengers']
ts_air_trend_removed = ts_air - ts_air.rolling(window=12).mean()
```

La sottrazione della media mobile con finestra stagionale $w = 12$ isola le oscillazioni cicliche dell'aviazione civile, rendendo la serie stazionaria in media ed eliminando la deriva asintotica positiva che inficerebbe le distanze geometriche.

16.2.2 Pipeline di Normalizzazione con `tslearn`

La libreria `tslearn` fornisce trasformatore conformi allo standard di Scikit-Learn (`fit`, `transform`, `fit_transform`) per l'elaborazione di dataset tridimensionali di serie temporali aventi forma shape `(n_series, n_timestamps, n_features)`:

```
from tslearn.preprocessing import TimeSeriesScalerMeanVariance, TimeSeriesScalerMinMax, TimeSeriesResampler

# A. Standardizzazione Z-Score a media zero e varianza uno per ciascuna serie
scaler_mv = TimeSeriesScalerMeanVariance(mu=0.0, std=1.0)
ts1_mv = scaler_mv.fit_transform(ts1.values.reshape(1, -1)).flatten()

# B. Normalizzazione Min-Max su intervallo arbitrario [a, b]
scaler_mm = TimeSeriesScalerMinMax(value_range=(1.0, 2.0))
ts1_mm = scaler_mm.fit_transform(ts1.values.reshape(1, -1)).flatten()

# C. Ricampionamento Temporale (Resampling) per allineare serie di lunghezza diversa
resampler = TimeSeriesResampler(sz=1000)
```

```
ts1_resampled = resampler.fit_transform(ts1.values.reshape
    (1, -1)).flatten()
```

Il `TimeSeriesResampler` esegue un'interpolazione lineare uniforme lungo l'asse delle ascisse, riconducendo sequenze acquisite a frequenze variabili verso una comune risoluzione discretizzata di 1000 istanti temporali, prerequisito vincolante per matrici di input ad alta dimensionalità.

16.3 Metriche di Distanza ed Allineamento Elastico (DTW)

Il notebook `8_TimeSeries1.ipynb` pone a confronto sistematico le metriche rigide (Euclidea, Manhattan) con l'allineamento elastico non lineare (**Dynamic Time Warping**, DTW).

16.3.1 Distanze Rigide vs Allineamento Elastico

Mentre la distanza Euclidea e la Manhattan (`cityblock`) impongono un accoppiamento punto-a-punto rigorosamente sincrono ad ascissa identica ($t_i \leftrightarrow s_i$):

```
from scipy.spatial.distance import euclidean, cityblock

d_euc = euclidean(ts1, ts2)       # Somma degli scarti
    quadratici punto-a-punto
d_man = cityblock(ts1, ts2)       # Somma dei valori
    assoluti degli scarti
```

l'algoritmo DTW risolve l'allineamento ottimale tra serie temporali sfasate o a velocità di evoluzione non omogenea, determinando il percorso di costo minimo (*warping path*) sulla matrice delle distanze locali (sezione 14.3):

```
from tslearn.metrics import dtw, dtw_path

# Calcolo della distanza DTW e recupero del warping path
    ottimale
path, dist_dtw = dtw_path(ts1, ts2)
print(f"Distanza DTW: {dist_dtw:.4f}")
# path e' una lista di tuple [(0, 0), (0, 1), (1, 2), ...,
    (n-1, m-1)]
```

16.3.2 Vincoli Globali sul Warping Path: Sakoe-Chiba e Itakura

L'applicazione di DTW senza vincoli può condurre a distorsioni patologiche e a singolarità numeriche, accoppiando un singolo istante di una serie con centinaia di campioni dell'altra (*pathological warping*). Nel laboratorio vengono applicati i due vincoli classici:

1. **Banda di Sakoe-Chiba**: restringe la ricerca della coordinata ottima a una fascia simmetrica di raggio costante r attorno alla diagonale principale ($|i - j| \leq r$):

```
# Esecuzione DTW con vincolo Sakoe-Chiba di raggio r = 3 sui primi 10 campioni
path_sc, dist_sc = dtw_path(ts1[:10], ts2[:10],
                            global_constraint="sakoe_chiba",
                            sakoe_chiba_radius=3)
```

2. **Parallelogramma di Itakura**: vincola la pendenza dell'allineamento tra due estremi prefissati (es. $1/2 \leq \frac{\Delta j}{\Delta i} \leq 2$), confinando la ricerca entro una regione romboidale/parallelogramma mediante la libreria `pyts`:

```
from pyts.metrics import dtw as dtw_pyts

# Calcolo DTW con parallelogramma di Itakura tramite pyts
dist_itakura, path_itakura = dtw_pyts(ts1[:10].to_numpy(), ts2[:10].to_numpy(),
                                      dist='square',
                                      method='itakura',
                                      return_path=True)
```

In Figura 16.2 viene raffigurata geometricamente la comparazione delle regioni ammissibili indotte dai due vincoli sul reticolo delle celle di allineamento.

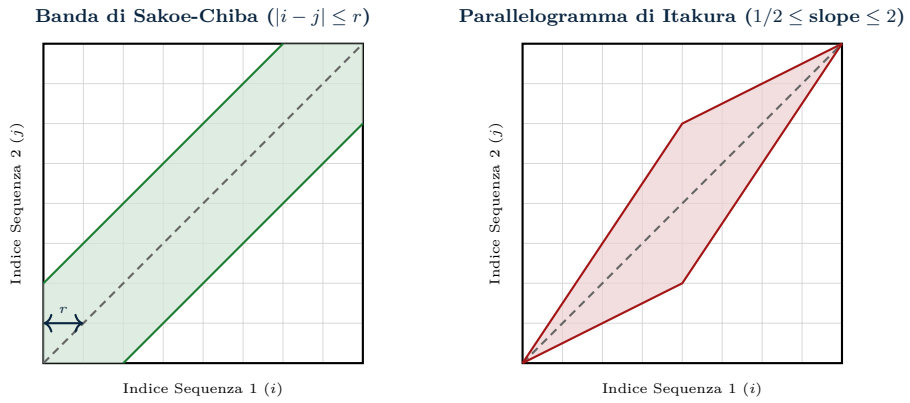


Figura 16.2: Confronto geometrico dei vincoli globali sul reticolo della matrice di costo DTW. A sinistra: Banda di Sakoe-Chiba a larghezza costante r. A destra: Parallelogramma di Itakura con limitazione di pendenza che preserva l'inizio e la fine dell'allineamento.

16.3.3 Similarità Basata su Feature Strutturali

Quando la lunghezza delle serie è particolarmente elevata o il segnale è contaminato da rumore ad alta frequenza, il confronto punto-a-punto può essere rimpiazzato dall'estrazione di un vettore di **statistiche descrittive globali** (*Feature-based similarity*):

```
import scipy.stats as stats

def calculate_features(values):
    """Estrae 13 descrittori statistici di forma e
       distribuzione."""
    features = {
        'avg': np.mean(values),
        'std': np.std(values),
        'var': np.var(values),
        'med': np.median(values),
        '10p': np.percentile(values, 10),
        '25p': np.percentile(values, 25),
        '50p': np.percentile(values, 50),
        '75p': np.percentile(values, 75),
        '90p': np.percentile(values, 90),
        'iqr': np.percentile(values, 75) - np.percentile(
           values, 25),
        'cov': float(np.mean(values)) / (np.std(values) + 1
           e-9),
        'skw': float(stats.skew(values)),
        'kur': float(stats.kurtosis(values))
    }
```

```
    return features

# Calcolo del vettore di feature ed estrazione della distanza Euclidea
f1 = calculate_features(ts1)
f2 = calculate_features(ts2)
dist_features = euclidean(list(f1.values()), list(f2.values()))
```

16.3.4 Dissimilarità Universale Basata su Compressione (CDM)

Come teorizzato nel Teorema di Kolmogorov e Keogh (sottosezione 14.6.2), la distanza non parametrica CDM quantifica la quantità di informazione condivisa tra due flussi numerici misurando l'efficienza di compressione congiunta mediante l'algoritmo Deflate/Lempel-Ziv di `zlib`:

```
import zlib

def CDM(x_bytes, y_bytes):
    """Calcola la Compression Dissimilarity Measure tra due stringhe di byte."""
    c_x = len(zlib.compress(x_bytes))
    c_y = len(zlib.compress(y_bytes))
    c_xy = len(zlib.compress(x_bytes + y_bytes))
    return float(c_xy) / (c_x + c_y)

# Conversione delle serie in sequenze di byte UTF-8
ts1_enc = (' '.join([f"{v:.4f}" for v in ts1])).encode('utf-8')
ts2_enc = (' '.join([f"{v:.4f}" for v in ts2])).encode('utf-8')

cdm_score = CDM(ts1_enc, ts2_enc)
print(f"Valore Dissimilarita CDM: {cdm_score:.4f}")
```

Valori di CDM prossimi a 0.5 attestano che le due sequenze sono quasi identiche e la loro concatenazione non aggiunge entropia al dizionario di compressione; valori prossimi a 1.0 indicano sequenze statisticamente indipendenti.

16.4 Rappresentazioni Discrete: PAA, SAX e 1D-SAX

Il notebook `8_TimeSeries1.ipynb` implementa le tecniche di approssimazione e discretizzazione simbolica per l'abbattimento della dimensionalità:

```
from tslearn.piecewise import
    PiecewiseAggregateApproximation,
    SymbolicAggregateApproximation,
    OneD_SymbolicAggregateApproximation

# Preprocessing: standardizzazione Z-Score
scaler = TimeSeriesScalerMeanVariance(mu=0.0, std=1.0)
ts1_norm = scaler.fit_transform(ts1.values.reshape(1, -1))
ts2_norm = scaler.fit_transform(ts2.values.reshape(1, -1))

# 1. PAA: riduzione a w = 10 segmenti a media locale
    costante
n_segments = 10
paa = PiecewiseAggregateApproximation(n_segments=n_segments
    )
ts1_paa = paa.fit_transform(ts1_norm)
# Ricostruzione della traiettoria continua interpolata
ts1_paa_inv = paa.inverse_transform(ts1_paa)

# 2. SAX: discretizzazione simbolica su alfabeto a = 8
    simboli
n_symbols = 8
sax = SymbolicAggregateApproximation(n_segments=n_segments,
     alphabet_size_avg=n_symbols)
ts1_sax = sax.fit_transform(ts1_norm)
ts1_sax_inv = sax.inverse_transform(ts1_sax)

# 3. 1D-SAX: quantizzazione congiunta di media e pendenza
    locale (slope)
one_d_sax = OneD_SymbolicAggregateApproximation(
    n_segments=n_segments,
    alphabet_size_avg=8,
    alphabet_size_slope=4
)
ts1_sax1d = one_d_sax.fit_transform(ts1_norm)
ts1_sax1d_inv = one_d_sax.inverse_transform(ts1_sax1d)
```

L'approccio **1D-SAX** supera il limite storico di SAX convenzionale: invece di modellare ciascun intervallo PAA come una linea orizzontale piatta priva di pendenza, memorizza una coppia ordinata di simboli per intervallo:

$$\langle \sigma_{\text{mean}}, \sigma_{\text{slope}} \rangle \in \Sigma_{\text{avg}} \times \Sigma_{\text{slope}} \tag{16.2}$$

preservando la derivata temporale locale ed eliminando le distorsioni gradonate tipiche di SAX classico.

16.5 Clustering di Serie Temporali: Paradigmi a Confronto

Il notebook `9_TimeSeries2.ipynb` affronta l'apprendimento non supervisionato sul benchmark **Cylinder-Bell-Funnel** (CBF), confrontando i quattro paradigmi di clustering.

```
from pyts.datasets import make_cylinder_bell_funnel
# Generazione di N = 60 serie (20 cilindri, 20 campane, 20 imbuti)
X, y = make_cylinder_bell_funnel(n_samples=60, random_state=42)
X = X.reshape(X.shape[0], X.shape[1], 1) # Formato 3D (60, 128, 1)
```

16.5.1 Shape-Based Clustering: K-Means Euclideo vs K-Means con DTW e DBA

L'algoritmo `TimeSeriesKMeans` di `tslearn` viene instanziato configurando dapprima la metrica Euclidea e successivamente la metrica DTW con centroide calcolato mediante **DTW Barycenter Averaging** (DBA):

```
from tslearn.clustering import TimeSeriesKMeans

k = 3
# 1. K-Means con distanza Euclidea e media aritmetica punto-a-punto
km_euc = TimeSeriesKMeans(n_clusters=k, metric="euclidean", max_iter=10, random_state=0)
km_euc.fit(X)

# 2. K-Means con distanza DTW e centroide DBA (Barycenter Averaging)
km_dtw = TimeSeriesKMeans(n_clusters=k, metric="dtw", max_iter=5, random_state=0)
km_dtw.fit(X)

print(f"Inerzia Euclidea: {km_euc.inertia_:.2f}")
print(f"Inerzia DTW:      {km_dtw.inertia_:.2f}")
```

Risultato Empirico Fondamentale del Laboratorio

Sotto distanza Euclidea, il centroide punto-a-punto appiattisce le creste e i fronti di salita a causa dei leggeri sfasamenti temporali presenti tra le istanze della medesima classe, producendo prototipi smussati e poco discriminanti. Al contrario, il centroide **DBA** (calcolato sotto metrica `"dtw"`) allinea elasticamente le coordinate prima di mediarle, riproducendo con fedeltà assoluta la sagoma spigolosa del cilindro (*Cylinder*), il picco asimmetrico della campana (*Bell*) e l'imbuto (*Funnel*).

16.5.2 Feature-Based, Compression-Based e PAA Clustering

Le sezioni conclusive del notebook esplorano gli approcci alternativi:

1. **Feature-Based Clustering**: ciascuna delle 60 serie viene mappata nel vettore a 11 dimensioni delle feature statistiche, dopodiché si esegue il partizionamento con `sklearn.cluster.KMeans`:

```
from sklearn.cluster import KMeans
F = [list(calculate_features(x.flatten()).values())
    [:-2] for x in X]
km_features = KMeans(n_clusters=k, n_init=10,
    random_state=0).fit(F)
```

2. **Compression-Based Clustering con DBSCAN**: si genera la matrice simmetrica completa di dissimilarità $M \in \mathbb{R}^{60\times 60}$ applicando la metrica CDM su tutte le coppie di serie, aggregando successivamente i dati mediante DBSCAN su distanze precalcolate:

```
from sklearn.metrics import pairwise_distances
from sklearn.cluster import DBSCAN

def cdm_dist(x, y):
    x_str = (' '.join([f"{v:.3f}" for v in x.ravel()])
        ).encode('utf-8')
    y_str = (' '.join([f"{v:.3f}" for v in y.ravel()])
        ).encode('utf-8')
    return float(len(zlib.compress(x_str + y_str))) /
        (len(zlib.compress(x_str)) + len(zlib.compress(
        y_str)))

M_cdm = pairwise_distances(X.reshape(X.shape[0], X.
    shape[1]), metric=cdm_dist)
```

```
dbscan_cdm = DBSCAN(eps=0.965, min_samples=5, metric='
    precomputed').fit(M_cdm)
```

3. **Approximation-Based Clustering su Sequenze PAA**: ciascuna serie viene compressa preventivamente a 10 coefficienti PAA; il clustering K-Means viene eseguito nello spazio compresso a 10 dimensioni, abbattendo il tempo di convergenza di oltre l'80%:

```
paa = PiecewiseAggregateApproximation(n_segments=10)
X_paa = paa.fit_transform(X) # Riduzione da 128 a 10
    dimensioni
km_paa = TimeSeriesKMeans(n_clusters=k, metric="
    euclidean", max_iter=5, random_state=0).fit(X_paa)
```

16.6 Motif Discovery ed Anomaly Detection con Matrix Profile

Nel notebook `10_TimeSeries_motif.ipynb` viene impiegata la libreria Python ad alte prestazioni `matrixprofile` per eseguire l'estrazione non supervisionata di motivi e anomalie sul dataset **AirPassengers** ($n = 144$).

16.6.1 Calcolo della Matrix Profile con Algoritmo STOMP

A differenza dell'algoritmo STAMP basato su permutazioni casuali (sottosezione 15.4.3), la libreria implementa **STOMP** (*Scalable Time series Ordered Matrix Profile*), che sfrutta l'aggiornamento iterativo ordinato delle distanze riducendo la complessità temporale da $\mathcal{O}(n^2 \log n)$ a un deterministico $\mathcal{O}(n^2)$:

```
from matrixprofile import matrixProfile

# Ingestione e isolamento della componente passeggeri
df = pd.read_csv('air_passengers.csv', parse_dates=True,
    index_col='Month')
ts = df['#Passengers'].values

# Esperimento 1: Finestra annuale completa (w = 12 mesi)
w_annual = 12
mp_12, mpi_12 = matrixProfile.stomp(ts, w_annual)

# Esperimento 2: Finestra trimestrale (w = 3 mesi)
w_quarter = 3
mp_3, mpi_3 = matrixProfile.stomp(ts, w_quarter)
```

Nota 16.6.1 (Confronto Funzionale tra le Finestre $w = 12$ e $w = 3$).

- Con finestra annuale $w = 12$, il profilo P evidenzia valli ampie e regolari in corrispondenza del ripetersi dei cicli di vacanza estiva (luglio–agosto);
- Con finestra trimestrale $w = 3$, il profilo cattura micro-variazioni stagionali locali (es. picco pasquale o flessione autunnale), producendo un segnale P a maggiore frequenza di campionamento.

16.6.2 Estrazione dei Motifs Principali

La funzione `motifs.motifs` analizza congiuntamente il vettore delle distanze `mp` e il vettore degli indici `mpi`, estraendo le prime $K = 5$ famiglie di motivi non banali sotto rigorosa applicazione della zona di esclusione:

```
from matrixprofile.motifs import motifs

# Estrazione dei primi 5 motivi morfologici
mo, mod = motifs(ts, (mp_12, mpi_12), max_motifs=5)

# Visualizzazione e sovrapposizione cromatica dei motivi sulla serie originale
plt.figure(figsize=(12, 5))
plt.plot(ts, color='gray', alpha=0.6, label='Serie AirPassengers')
colors = ['red', 'green', 'black', 'blue', 'orange'][:len(mo)]

for m_indices, d, c in zip(mo, mod, colors):
    for idx in m_indices:
        m_shape = ts[idx : idx + w_annual]
        plt.plot(range(idx, idx + w_annual), m_shape, color=c, lw=2.5,
                 label=f"Motif (dist={d:.2f})" if idx == m_indices[0] else "")

plt.title("Primi Motivi Stagionali Estratti dalla Matrix Profile (w = 12)")
plt.xlabel("Mese (1949 - 1960)")
plt.ylabel("Numero Passeggeri (migliaia)")
plt.legend()
plt.show()
```

I segmenti estratti corrispondono esattamente ai cicli annuali con identica velocità di incremento primaverile e flessione invernale, dimostrando l'invarianza della Matrix Profile rispetto all'ampiezza scalare del trend di fondo.

16.6.3 Rilevamento di Anomalie (Time Series Discords)

Le anomalie strutturali corrispondono ai massimi locali e globali del vettore P al di fuori della zona di esclusione temporale:

```
from matrixprofile.discords import discords

# Ricerca dei primi k = 5 discords con zona di esclusione w/2 = 6 campioni
anoms = discords(mp_12, ex_zone=6, k=5)

# Visualizzazione delle anomalie sulla serie storica
plt.figure(figsize=(12, 5))
plt.plot(ts, color='gray', alpha=0.6, label='Serie AirPassengers')
colors = ['crimson', 'darkorange', 'purple', 'magenta', 'brown'][:len(anoms)]

for a_idx, c in zip(anoms, colors):
    a_shape = ts[a_idx : a_idx + w_annual]
    plt.plot(range(a_idx, a_idx + w_annual), a_shape, color=c, lw=3,
             label=f"Discord a t={a_idx} (P={mp_12[a_idx]:.2f})")

plt.title("Rilevamento di Anomalie Morfologiche (Discords) su AirPassengers")
plt.xlabel("Mese")
plt.ylabel("Numero Passeggeri")
plt.legend()
plt.show()
```

I primi due discords individuati cadono in corrispondenza dell'estate del 1958 e dell'autunno del 1960: in tali periodi il traffico passeggeri registrò fluttuazioni improvvise non conformi alla curva stagionale standard degli anni precedenti, evidenziando picchi eccezionali dovuti all'introduzione dei primi aerei commerciali a reazione (*Jet Age*).

16.7 Quadro Comparativo delle Librerie e Sintesi Operativa

In Tabella 16.1 si sintetizza il confronto operativo tra le quattro librerie Python impiegate nel laboratorio per l'analisi di serie temporali.

Tabella 16.1: Quadro comparativo delle librerie Python per l'elaborazione di serie temporali.

Libreria Python	Competenze Cardine e Moduli	Punti di Forza	Limitazioni / Vincoli
`tslearn`	Preprocessing, DTW, DBA, PAA, SAX, K-Means	Integrazione Scikit-Learn, clustering robusto con DBA	Prestazioni moderate su dataset massivi $\geq 10^5$
`pyts`	DTW vincolata (Itakura), dataset sintetici CBF	Implementazione efficiente di vincoli globali DTW	Meno orientata a pipeline di clustering complesse
`matrixprofile`	STAMP, STOMP, MASS, Motifs, Discords	Complessità quadratica esatta $\mathcal{O}(n^2)$, esente da soglie	Limitata all'analisi di sottosequenze con finestra fissa w
`scipy` / `zlib`	Distanze Euclidea/Manhattan, dissimilarità CDM	Massima generalità teorica, non parametrica	Richiede codifica custom su byte; $\mathcal{O}(n^2)$ per matrici pairwise

Nota 16.7.1 (Linee Guida per la Scelta della Tecnica di Mining)**.**

1. Nel **clustering di sequenze a lunghezza moderata sfasate temporalmente**: impiegare `TimeSeriesKMeans(metric="dtw")` con centroide DBA di `tslearn`.

2. Nelle serie estese per la **ricerca di pattern o guasti**: applicare `matrixprofile.stomp` ed estrarre motifs e discords con opportuna zona di esclusione ($w/2$).

3. Con **alta dimensionalità e forte rumore**: adottare la discretizzazione 1D-SAX (`OneD_SAX`) per comprimere il segnale preservando le pendenze locali.

Capitolo 17

Laboratorio di Pattern Mining e Social Network Analysis in Python

17.1 Introduzione e Obiettivi del Laboratorio

Nelle analisi avanzate di Data Mining, la transizione dai modelli predittivi o di clustering su vettori tabellari tradizionali verso l'estrazione di pattern frequenti e l'analisi di strutture relazionali complesse rappresenta un passaggio metodologico fondamentale. In questo laboratorio pratico vengono affrontati in modo sistematico due dei paradigmi più potenti dell'ingegneria dei dati moderni:

1. **Frequent Pattern Mining, Regole di Associazione e Sequential Pattern Mining (Parte I):** implementazione computazionale dell'estrazione di associazioni nascoste in transazioni commerciali e demografiche mediante gli algoritmi Apriori e FP-Growth, studio analitico del fattore di compattazione e mining sequenziale vincolato (GSP) tramite le librerie Python `mlxtend` [14, 18] e `gsp_python` [16].

2. **Network Science e Social Network Analysis (Parte II):** formalizzazione, quantificazione e partizionamento di grafi complessi e reti sociali mediante `NetworkX` [28] e `CDlib` [29], esplorando metriche microscopiche (gradi, cammini minimi, clustering locale), mesoscopiche (misure di centralità, hub vs bridge, assortatività) e macroscopiche (densità, diametro, community detection via modularità di Louvain [30] sul celebre benchmark di Zachary's Karate Club [31]).

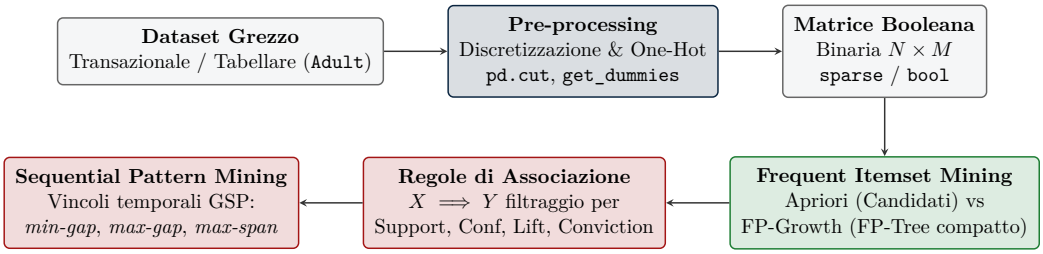


Figura 17.1: Pipeline metodologica della Parte I: dalla strutturazione transazionale e binarizzazione tabellare all'estrazione di itemset frequenti, generazione di regole d'associazione e mining sequenziale vincolato.

17.2 Parte I: Frequent Pattern Mining e Regole di Associazione

17.2.1 Rappresentazione dei Dati e Trasformazione Transazionale

A differenza delle tradizionali matrici numeriche di feature in $\mathbb{R}^{n \times d}$, l'analisi delle associazioni opera su **database transazionali**. Sia $\mathcal{I} = \{i_1, i_2, \ldots, i_m\}$ l'universo finito di m elementi distinti (*items*). Una **transazione** $T \subseteq \mathcal{I}$ è un sottoinsieme di item registrati congiuntamente, e il dataset transazionale $\mathcal{D} = \{T_1, T_2, \ldots, T_N\}$ è una collezione finita di N transazioni.

Nella pratica industriale e analitica, i dati si presentano principalmente sotto tre forme distinte:

1. **Rappresentazione Relazionale (Paniere Verticale):** ogni record specifica una coppia $\langle \text{Basket_ID}, \text{Item_ID} \rangle$. È la tipica struttura normalizzata dei database SQL di cassa o ordini di e-commerce.

2. **Rappresentazione a Lista di Panieri (Orizzontale):** ogni istanza corrisponde a una transazione rappresentata come lista o insieme di identificativi di prodotto:

$$\mathcal{D}_{\text{list}} = \Big[[\text{“Pane”}, \text{“Latte”}],\ [\text{“Pane”}, \text{“Uova”}],\ [\text{“Pane”}, \text{“Latte”}, \text{“Uova”}]\Big] \tag{17.1}$$

3. **Rappresentazione Tabellare Classica:** ogni riga modella un'entità (es. un individuo con attributi demografici come età, occupazione, reddito) che necessita di discretizzazione e codifica prima di poter essere processata dagli algoritmi di associazione.

Conversione da Formato Relazionale a Panieri

Quando i dati provengono da log transazionali con record atomici (es. nel file `small_transactions.csv`), l'estrazione della lista di panieri viene eseguita raggruppando sul codice scontrino (`RECIPE_ID`) e aggregando la colonna merceologica (`COD_MKT_ID`):

Listing 17.1: Aggregazione di log relazionali in liste di transazioni.

```
import pandas as pd

# Caricamento del log transazionale normalizzato
df_tr = pd.read_csv(
    "small_transactions.csv",
    skipinitialspace=True,
    sep=",",
    nrows=2000
)

# Raggruppamento per scontrino e aggregazione degli
    articoli in liste
transactions = df_tr.groupby('RECIPE_ID')['COD_MKT_ID'].
    apply(list)
baskets = transactions.values.tolist()

print(f"Numero totale di panieri estratti: {len(baskets)}")
print(f"Esempio primi 2 panieri: {baskets[:2]}")
```

Trasformazione da Tabellare a Transazionale: Il Caso Adult Census

Nel dataset *Adult Census* (noto anche come *Census Income*), i record descrivono individui censiti combinando feature continue (quali età, ore settimanali e plusvalenze) e variabili categoriche (quali istruzione, occupazione e stato civile). Per consentire il pattern mining:

- Le variabili numeriche continue vengono partizionate in intervalli discreti tramite `pd.cut` (discretizzazione a uguale ampiezza o a quantili).
- Si convertono gli intervalli discretizzati e le categorie in token composti della forma `attributo%valore`, impedendo collisioni sintattiche tra valori identici appartenenti a colonne differenti.
- Si genera la matrice booleana di presenza/assenza (one-hot encoding) tramite `pd.get_dummies(..., prefix_sep='%')`.

Listing 17.2: Pipeline di discretizzazione e codifica one-hot su Adult Census.

```
df = pd.read_csv('adult.csv')

# Identificazione colonne numeriche e discretizzazione in 3 bin uniformi
num_col = [
    'age', 'capital_gain', 'capital_loss',
    'education', 'final_weight', 'hours_worked_per_week'
]
for col in num_col:
    df[col + '_disc'] = pd.cut(df[col], bins=3, right=False).astype(str)
df.drop(columns=num_col, inplace=True)

# Conversione di stringhe e booleani in formato categoria
df['over_threshold'] = df['over_threshold'].astype('boolean')
for col in df.select_dtypes(include=['object', 'bool']).columns:
    df[col] = df[col].astype('category')

# Matrice binaria densa/booleana per algoritmi di itemset mining
df_encoded = pd.get_dummies(df, prefix_sep='%').astype(bool)
print(f"Dimensioni matrice binaria di input: {df_encoded.shape}")
```

17.2.2 Estrazione di Itemset Frequenti con Apriori e FP-Growth

Definizione 17.2.1 (Supporto di un Itemset)**.** Dato un itemset $X \subseteq \mathcal{I}$, il suo **supporto** assoluto $\sigma(X)$ è il numero di transazioni in $\mathcal{D}$ che contengono X. Il supporto relativo $s(X)$ è la frazione di transazioni:

$$s(X) = \frac{|\{T \in \mathcal{D} \mid X \subseteq T\}|}{|\mathcal{D}|} \tag{17.2}$$

Un itemset è detto **frequente** se $s(X) \geq \text{minsup}$, dove $\text{minsup} \in (0, 1]$ è una soglia stabilita dall'analista.

L'Algoritmo Apriori

L'algoritmo Apriori [14] poggia sul celebre **principio di anti-monotonicità del supporto**:

$$\forall X, Y \subseteq \mathcal{I} : \quad X \subseteq Y \implies s(Y) \leq s(X) \tag{17.3}$$

Se un insieme X non è frequente, nessun suo sovra-insieme $Y \supset X$ può essere frequente. L'algoritmo procede per livelli incrementali ($k = 1, 2, \dots$):

1. Generazione candidati C_k mediante operazione di join $L_{k-1} \bowtie L_{k-1}$.
2. Potatura dei candidati che contengono almeno un sotto-insieme di cardinalità $k-1$ non appartenente a L_{k-1}.
3. Scansione dell'intero dataset $\mathcal{D}$ per calcolare il supporto esatto dei candidati sopravvissuti e definire L_k.

Listing 17.3: Estrazione itemset frequenti con Apriori tramite `mlxtend`.

```
from mlxtend.frequent_patterns import apriori

min_support = 0.70
frequent_itemsets_apriori = apriori(
    df_encoded,
    min_support=min_support,
    use_colnames=True,
    max_len=3
)
print(f"Itemset frequenti (Apriori): {len(
   frequent_itemsets_apriori)}")
```

Ispezione e Interrogazione delle Transazioni

Data l'identificazione di un itemset frequente composto (ad esempio, la congiunzione tra guadagni di capitale ridotti e nazionalità statunitense), risulta fondamentale poter risalire alle transazioni originali che lo soddisfano:

Listing 17.4: Filtraggio delle transazioni che soddisfano uno specifico itemset frequente.

```
def filter_rows_by_itemset(df_source, itemset):
    condition = None
    for item in itemset:
        col_name, val = item.split('%', 1)
        sub_cond = (df_source[col_name].astype(str) == val)
        condition = sub_cond if condition is None \
            else (condition & sub_cond)
    return df_source[condition] if condition is not None \
            else pd.DataFrame()

# Recupero del 35-esimo itemset estratto
target_itemset = frequent_itemsets_apriori['itemsets'].iloc
    [34]
```

```
matching_records = filter_rows_by_itemset(df,
    target_itemset)
print(f"Record trovati: {len(matching_records)}")
```

Algoritmo FP-Growth e Compaction Factor

L'algoritmo FP-Growth (*Frequent Pattern Growth*) [18] elimina completamente la costosa fase di generazione dei candidati C_k e le ripetute scansioni del database. L'approccio si articola in due fasi:

1. **Costruzione dell'FP-Tree:** il dataset viene scansionato due sole volte. Nella prima si calcolano i supporti dei singoli item (L_1), ordinandoli in ordine decrescente di frequenza. Nella seconda scansione, ogni transazione viene inserita nell'albero radicato condividendo i prefissi comuni con le transazioni precedenti, corredata da puntatori orizzontali (*header table*) che concatenano tutti i nodi associati allo stesso item.

2. **Mining Ricorsivo per Sotto-Alberi Condizionali:** per ciascun item i, partendo dal meno frequente, si risalgono i puntatori fino alla radice per estrarre la *base di pattern condizionale*, costruendo un albero FP condizionato ricorsivo.

Listing 17.5: Estrazione itemset frequenti con FP-Growth.

```
from mlxtend.frequent_patterns import fpgrowth

frequent_itemsets_fpg = fpgrowth(
    df_encoded,
    min_support=min_support,
    use_colnames=True,
    max_len=3
)
print(f"Itemset frequenti (FP-Growth): {len(
    frequent_itemsets_fpg)}")
```

Confronto Prestazionale ed Efficienza di Compattazione

L'efficienza relativa tra Apriori e FP-Growth dipende in modo determinante dalla densità della matrice e dal **fattore di compattazione** (*compaction factor*) dell'FP-Tree:

$$\mathcal{CF} = \frac{\sum_{T \in \mathcal{D}} |T|}{\text{Numero totale di nodi dell'FP-Tree}} \tag{17.4}$$

- Quando i dati sono fortemente correlati o presentano transazioni lunghe e dense di item condivisi, $\mathcal{CF} \gg 1$: l'albero FP condensa enormi volumi di informazioni in pochissimi nodi, rendendo FP-Growth ordini di grandezza più rapido di Apriori.
- Al contrario, in presenza di dataset sparsi o su proiezioni con basso numero di transazioni e itemset ortogonali (es. `df_encoded.iloc[0:400, 0:30]`), il costo di costruzione della struttura ad albero e la gestione dei puntatori orizzontali può risultare paragonabile o persino leggermente superiore al semplice filtraggio vettoriale booleano di Apriori.

Nel benchmarking svolto con `%timeit`:

Listing 17.6: Benchmarking empirico Apriori vs FP-Growth con timeit.

```
# Benchmark su dataset completo con min_support = 0.70
# Apriori:   ~210 ms +- 12 ms per loop
# FP-Growth: ~85 ms +- 4 ms per loop (accelerazione di circa 2.5x)

# Benchmark su sotto-matrice sparsa ridotta (400 righe, 30 colonne):
# Apriori:   ~3.2 ms per loop
# FP-Growth: ~3.8 ms per loop (overhead dell'albero su dataset ristretto)
```

17.2.3 Generazione e Validazione delle Regole di Associazione

Estratti gli itemset frequenti $\mathcal{F}$, l'obiettivo dell'analisi è derivare implicazioni logico-probabilistiche della forma:

$$X \implies Y \quad \text{con } X, Y \subset \mathcal{I}, \quad X \cap Y = \emptyset, \quad X \cup Y \in \mathcal{F} \tag{17.5}$$

dove X è detto **antecedente** e Y **conseguente**.

Definizione 17.2.2 (Metriche di Significatività delle Regole)**.** Le regole vengono valutate mediante una batteria di metriche quantitative complementari:

- **Supporto:** frazione di transazioni che contengono congiuntamente antecedente e conseguente:

$$\text{Support}(X \implies Y) = s(X \cup Y) = P(X \cap Y) \tag{17.6}$$

- **Confidenza:** probabilità condizionata che una transazione contenga Y dato che contiene X:

$$\text{Confidence}(X \implies Y) = \frac{s(X \cup Y)}{s(X)} = P(Y \mid X) \tag{17.7}$$

- **Lift (Interest):** rapporto tra la frequenza congiunta osservata e quella attesa nell'ipotesi di indipendenza statistica tra X e Y:

$$\text{Lift}(X \implies Y) = \frac{P(X \cap Y)}{P(X) \cdot P(Y)} = \frac{\text{Confidence}(X \implies Y)}{s(Y)} \tag{17.8}$$

  - Lift > 1: attrazione/dipendenza positiva (X favorisce la presenza di Y).
  - Lift $= 1$: perfetta indipendenza stocastica tra antecedente e conseguente.
  - Lift < 1: repulsione/sostituzione negativa (X inibisce la presenza di Y).

- **Leverage:** differenza assoluta tra frequenza congiunta osservata e frequenza attesa:

$$\text{Leverage}(X \implies Y) = P(X \cap Y) - P(X) \cdot P(Y) \tag{17.9}$$

- **Conviction:** misura di sensibilità all'errore della regola. Valuta quanto spesso la regola fallirebbe se l'associazione fosse puramente casuale:

$$\text{Conviction}(X \implies Y) = \frac{1 - s(Y)}{1 - \text{Confidence}(X \implies Y)} = \frac{P(\neg Y)}{P(\neg Y \mid X)} \tag{17.10}$$

  Se la confidenza tende a 1, la conviction diverge a $+\infty$ (regola esatta deterministica).

Listing 17.7: Estrazione e filtraggio avanzato delle regole d'associazione.

```
from mlxtend.frequent_patterns import association_rules

# Generazione regole con confidenza minima 0.70
rules = association_rules(
    frequent_itemsets_fpg,
    metric="confidence",
    min_threshold=0.70
)

```

```
# Calcolo cardinalita di antecedente e conseguente
rules["antecedent_len"] = rules["antecedents"].apply(lambda
    x: len(x))
rules["consequent_len"] = rules["consequents"].apply(lambda
    x: len(x))

# Filtraggio per regole non banali, confidenza elevata e
   Lift positivo
filtered_rules = rules[
    (rules['antecedent_len'] >= 2) &
    (rules['confidence'] > 0.75) &
    (rules['lift'] > 1.05)
]

print(f"Regole candidate totali: {len(rules)}")
print(f"Regole selezionate post-filtraggio: {len(
   filtered_rules)}")
cols = ['antecedents', 'consequents', 'support', '
   confidence', 'lift']
print(filtered_rules[cols].head(3))
```

17.2.4 Generalized Sequential Pattern Mining (GSP)

Quando le transazioni sono ordinate temporalmente per ciascun utente, l'analisi degli itemset atemporali evolve in **Sequential Pattern Mining** [16].

- Un **evento** (o itemset elementare) $e_i = \{i_1, i_2, \ldots, i_k\}$ è un insieme di item acquisiti nel medesimo istante temporale.
- Una **sequenza** $s = \langle e_1, e_2, \ldots, e_p \rangle$ è una lista ordinata di eventi.
- La lunghezza $|s|$ è il numero di eventi; una sequenza con k item totali è detta k-sequenza.
- Una sequenza $\alpha = \langle a_1, \ldots, a_n \rangle$ è una **sottosequenza** di $\beta = \langle b_1, \ldots, b_m \rangle$ ($\alpha \sqsubseteq \beta$) se esistono indici $1 \leq j_1 < j_2 < \cdots < j_n \leq m$ tali che $a_1 \subseteq b_{j_1}, a_2 \subseteq b_{j_2}, \ldots, a_n \subseteq b_{j_n}$.

Vincoli Temporali Strutturati nell'Algoritmo GSP

L'algoritmo GSP (*Generalized Sequential Pattern*) estende Apriori introducendo vincoli realistici sulle transizioni temporali tra eventi:

1. **Min-gap (ng):** tempo minimo intercorrente tra due eventi consecutivi del pattern. Se $t(e_j) - t(e_{j-1}) < ng$, l'occorrenza è invalidata (evita di considerare ripetizioni quasi-istantanee).

2. **Max-gap (xg):** tempo massimo consentito tra due eventi consecutivi del pattern. Se $t(e_j) - t(e_{j-1}) > xg$, il legame sequenziale si considera decaduto per eccessiva distanza temporale.

3. **Maximum Span (ms):** durata complessiva massima dell'intera istanza temporale del pattern, imponendo $t(e_p) - t(e_1) \leq ms$.

Listing 17.8: Generazione sintetica di sequenze ed estrazione vincolata con GSP.

```
from gsp_python.dataset_gen import DatasetGenerator
from gsp_python.gsp import GSP

# Generatore sintetico di sequenze transazionali
# size: numero di sequenze cliente; nevents: vocabolario item
# maxevents: max item/transazione; avgelems: transazioni medie
ds_gen = DatasetGenerator(size=100, nevents=20, maxevents=4, avgelems=5)
sequences = ds_gen.generate_dataset()

print(f"Numero di sequenze generate: {len(sequences)}")
print(f"Esempio sequenza 1: {sequences[0][:3]}")

# Esecuzione del Generalized Sequential Pattern Mining con vincoli
# minsup=0.30 (30%), mingap=1 istante, maxgap=2 istanti, maxspan=5 istanti
algo_gsp = GSP(sequences, minsup=0.30, mingap=1, maxgap=2, maxspan=5)
sequential_patterns = algo_gsp.run_gsp()

print(f"Pattern sequenziali individuati: {len(sequential_patterns)}")
```

17.3 Parte II: Network Science e Social Network Analysis

17.3.1 Modellazione dei Grafi e Strutture Dati con NetworkX

L'analisi delle reti complesse affonda le proprie radici storiche nella risoluzione del celebre **Problema dei Ponti di Königsberg** da parte di Leonhard Euler (1736): dimostrando l'impossibilità di percorrere ciascuno dei 7 ponti sul fiume Pregel una e una sola volta e tornare al punto di partenza, Eulero astraeva le masse terrestri in nodi e i ponti in archi, fondando la Teoria dei Grafi e dimostrando che un circuito euleriano esiste se e solo se ogni vertice possiede grado pari [28].

Formalmente, un grafo è definito dalla coppia $G = (V, E)$, dove $V = \{v_1, v_2, \ldots, v_N\}$ è l'insieme di $N = |V|$ nodi (*vertici*) ed $E \subseteq V \times V$ è l'insieme di $L = |E|$ collegamenti (*archi* o *links*).

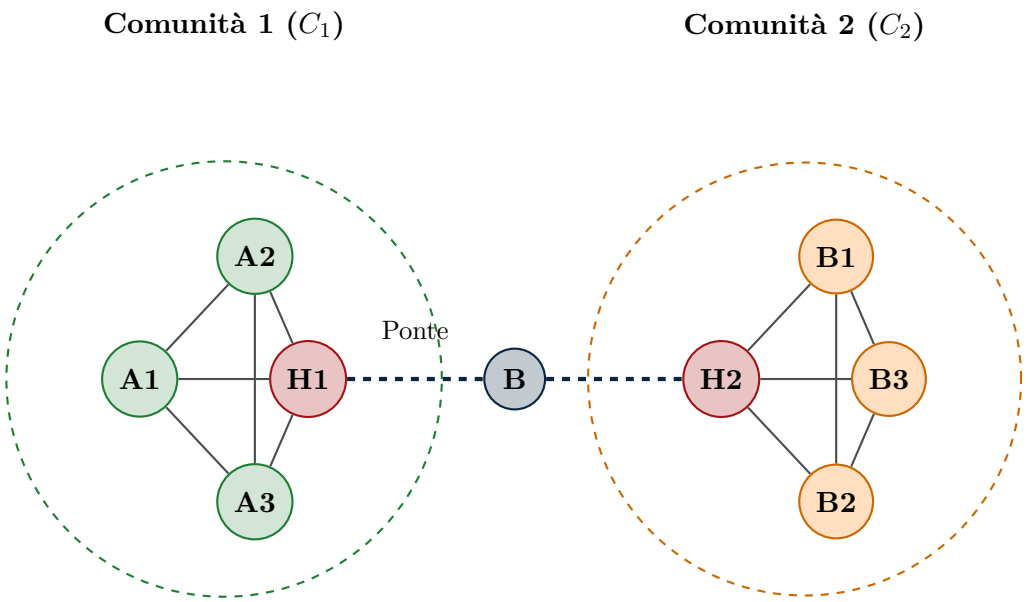


Figura 17.2: Topologia relazionale: distinzione tra nodi *Hub* ad alto grado locale (H1, H2), nodi *Bridge* con elevata betweenness centrality (B) e partizione in comunità dense connesse da archi di frontiera.

Tipologie di Grafo in NetworkX

In `NetworkX`, la manipolazione e l'analisi avvengono tramite quattro classi fondamentali:

- **Grafi Non Orientati (`nx.Graph`):** gli archi sono privi di direzione. La presenza di $(u, v) \in E$ implica simmetricamente $(v, u) \in E$. Il numero massimo teorico di archi è $L_{\max} = \binom{N}{2} = \frac{N(N-1)}{2}$.
- **Grafi Orientati (`nx.DiGraph`):** gli archi hanno verso stabilito $(u \to v)$. Il numero massimo è $L_{\max} = N(N-1)$.
- **Grafi Pesati:** ogni arco reca un attributo scalare $w(u, v) \in \mathbb{R}$ indicante capacità, costo o frequenza d'interazione.
- **Reti Bipartite:** l'insieme dei nodi è partizionato in due insiemi disgiunti $V = U \cup V_{\text{ext}}$ con $U \cap V_{\text{ext}} = \emptyset$, tale che ogni arco collega esclusivamente un nodo di U a uno di V_{ext} (es. grafo Ingredient-Flavor [28]).

Listing 17.9: Inizializzazione e popolamento di grafi non orientati, orientati e pesati.

```
import networkx as nx
import matplotlib.pyplot as plt

# 1. Grafo Non Orientato Semplice
G = nx.Graph()
G.add_edges_from([
    ("Alice", "Bob"), ("Alice", "Charlie"), ("Bob", "
       Charlie"),
    ("Charlie", "David"), ("David", "Emma"), ("Emma", "
       Frank"),
    ("David", "Frank")
])

# 2. Grafo Orientato (DiGraph)
DG = nx.DiGraph()
DG.add_edges_from([
    ("Alice", "Bob"), ("Bob", "Charlie"),
    ("Charlie", "Alice"), ("David", "Charlie")
])

# 3. Grafo Pesato (Weighted Graph)
WG = nx.Graph()
WG.add_edge("Alice", "Bob", weight=4.5)
WG.add_edge("Bob", "Charlie", weight=2.0)
WG.add_edge("Charlie", "David", weight=8.0)
```

17.3.2 Proprietà Topologiche Globali e Locali

Gradi e Distribuzione dei Gradi

Il **grado** k_i di un nodo v_i è il numero di archi incidenti su di esso. Per grafi non orientati:

$$k_i = \sum_{j=1}^{N} A_{ij}, \qquad \langle k \rangle = \frac{1}{N} \sum_{i=1}^{N} k_i = \frac{2L}{N} \tag{17.11}$$

Nei grafi orientati si distinguono l'in-degree k_i^{in} (archi entranti) e l'out-degree k_i^{out} (archi uscenti), con $\langle k^{\text{in}} \rangle = \langle k^{\text{out}} \rangle = \frac{L}{N}$. Nei grafi pesati si quantifica la **forza** (*strength*): $s_i = \sum_j w_{ij}$.

La **distribuzione dei gradi** $P(k)$ quantifica la probabilità che un nodo estratto uniformemente a caso possieda grado k:

$$P(k) = \frac{N_k}{N} \tag{17.12}$$

dove N_k è il conteggio dei vertici di grado k. Nelle reti casuali omogenee (modello di Erdős-Rényi), $P(k)$ segue una distribuzione binomiale che converge a una Poissoniana con varianza ristretta attorno a $\langle k \rangle$. Nelle reti reali complesse (*scale-free*) $P(k)$ segue una legge di potenza (*power-law*):

$$P(k) \sim k^{-\gamma}, \qquad 2 \leq \gamma \leq 3 \tag{17.13}$$

caratterizzata da una frazione dominante di nodi a basso grado e code pesanti popolate da super-connettori (*hub*).

Densità di Rete

La **densità** $\rho(G) \in [0, 1]$ esprime la frazione di collegamenti esistenti rispetto a quelli teoricamente possibili:

$$\rho_{\text{undirected}} = \frac{2L}{N(N-1)}, \qquad \rho_{\text{directed}} = \frac{L}{N(N-1)} \tag{17.14}$$

Le reti reali (sociali, biologiche, infrastrutturali) mostrano una densità estremamente ridotta ($\rho \ll 1$), configurandosi come grafi marcatamente sparsi.

Distanze Geodetiche, Cammino Medio e Diametro

Sia $d(u, v)$ la lunghezza del **cammino minimo** (*geodetico*) tra u e v, calcolata tramite algoritmo Breadth-First Search (BFS) per grafi non pesati con complessità $\mathcal{O}(V + E)$ o algoritmo di Dijkstra per archi a peso positivo $\mathcal{O}(E + V \log V)$.

- **Diametro ($d_{\max}$):** la massima distanza geodetica tra qualsiasi coppia di vertici appartenenti alla medesima componente connessa:

$$d_{\max} = \max_{u,v \in V} d(u,v) \tag{17.15}$$

- **Lunghezza Media del Cammino ($\langle d \rangle$):**

$$\langle d \rangle = \frac{1}{N(N-1)} \sum_{u \neq v \in V} d(u,v) \tag{17.16}$$

Assortatività di Grado

L'**assortatività** misura la tendenza dei nodi a connettersi a nodi aventi grado simile (omofilia strutturale). Il coefficiente di assortatività per grado di Newman [32] corrisponde al coefficiente di correlazione lineare di Pearson tra i gradi agli estremi di ciascun arco:

$$r = \frac{\sum_{jk} jk(e_{jk} - q_j q_k)}{\sigma_q^2} \in [-1, +1] \tag{17.17}$$

- $r > 0$ (**Rete Assortativa**): nodi ad alto grado tendono a legarsi con nodi ad alto grado (tipico delle reti sociali umane e co-autorie scientifiche).
- $r < 0$ (**Rete Disassortativa**): nodi ad alto grado si collegano prevalentemente a nodi periferici a basso grado (tipico delle reti tecnologiche, Internet router e reti proteiche/biologiche).

Coefficiente di Clustering

Il **coefficiente di clustering locale** C_i quantifica la propensione dei vicini del nodo v_i a formare un sottografo completo (clique triangolare):

$$C_i = \frac{2e_i}{k_i(k_i-1)} = \frac{\sum_{j,k} A_{ij}A_{jk}A_{ki}}{k_i(k_i-1)} \tag{17.18}$$

dove e_i è il numero di archi esistenti tra i k_i vicini di v_i. Il coefficiente globale di clustering medio della rete è:

$$\langle C \rangle = \frac{1}{N} \sum_{i=1}^{N} C_i \tag{17.19}$$

Listing 17.10: Calcolo completo delle statistiche topologiche di rete.

```python
def network_summary(graph):
    n = graph.number_of_nodes()
    m = graph.number_of_edges()
    avg_k = 2 * m / n if not graph.is_directed() else m / n
    density = nx.density(graph)
    avg_clustering = nx.average_clustering(graph)
    assortativity = nx.degree_assortativity_coefficient(
        graph)

    if nx.is_connected(graph):
        diameter = nx.diameter(graph)
        avg_path = nx.average_shortest_path_length(graph)
    else:
        # Se disconnessa, si valuta la componente gigante
        giant = graph.subgraph(
            max(nx.connected_components(graph), key=len)
        )
        diameter = nx.diameter(giant)
        avg_path = nx.average_shortest_path_length(giant)

    return {
        "Nodi (N)": n, "Archi (L)": m, "Grado Medio": round
            (avg_k, 2),
        "Densita": round(density, 4),
        "Clustering Medio": round(avg_clustering, 4),
        "Assortativita (r)": round(assortativity, 4), "
            Diametro": diameter,
        "Cammino Medio": round(avg_path, 2)
    }

print(network_summary(G))
```

17.3.3 Misure di Centralità: Hub, Bridge e Ruoli Strutturali

Nelle reti reali la posizione strutturale di un nodo ne determina l'influenza, la vulnerabilità e la capacità di veicolare informazione [28].

Definizione 17.3.1 (Misure Cardine di Centralità)**.** Si distinguono diverse formulazioni formali di centralità:

- **Degree Centrality (C_D):** normalizza il grado locale rispetto al massimo teorico $N-1$:

$$C_D(v) = \frac{k_v}{N-1} \tag{17.20}$$

Identifica i connettori locali (*hub*).

- **Betweenness Centrality (C_B):** quantifica quante volte un vertice si interpone lungo i cammini minimi tra tutte le altre coppie di nodi:

$$C_B(v) = \sum_{s \neq v \neq t \in V} \frac{\sigma_{st}(v)}{\sigma_{st}} \tag{17.21}$$

dove σ_{st} è il conteggio totale dei geodetici tra s e t, e $\sigma_{st}(v)$ è la frazione di essi che transita attraverso v. Viene calcolata efficientemente tramite l'**algoritmo di Brandes** [33] con costo $\mathcal{O}(V \cdot E)$.

- **Closeness Centrality (C_C):** reciproco della somma delle distanze verso tutti gli altri nodi:

$$C_C(v) = \frac{N-1}{\sum_{u \neq v} d(v,u)} \tag{17.22}$$

Confronto Critico: Nodi Hub vs Nodi Bridge

Un errore interpretativo comune consiste nell'ipotizzare che i nodi con il maggior grado (*hub*) siano automaticamente i più critici per il flusso di rete.

- Un nodo all'interno di una clique densa possiede un valore elevato di C_D, ma una betweenness C_B prossima a zero, poiché i suoi vicini sono mutuamente interconnessi da percorsi alternativi equipollenti.
- Al contrario, un nodo **bridge** (come il nodo **David** nel nostro grafo didattico G, o il nodo **B** in Figura 17.2) può presentare un grado modesto ($k = 3$), ma una betweenness massima, poiché ogni comunicazione tra le due macro-componenti della rete deve obbligatoriamente attraversarlo. La rimozione di un nodo bridge provoca la disconnessione o frammentazione della rete.

Listing 17.11: Calcolo comparativo di Degree Centrality e Betweenness Centrality.

```
deg_centrality = nx.degree_centrality(G)
betw_centrality = nx.betweenness_centrality(G)

df_centrality = pd.DataFrame({
    'Degree Centrality': deg_centrality,
    'Betweenness Centrality': betw_centrality
}).sort_values(by='Betweenness Centrality', ascending=False
    )

print(df_centrality)
```

17.3.4 Community Detection e Partizionamento con Algoritmo di Louvain

Il problema della **Community Discovery** (o partizionamento in comunità) si prefigge di suddividere l'insieme V in k gruppi disgiunti $\mathcal{C} = \{C_1, C_2, \ldots, C_k\}$ tali che la densità di archi interni a ciascun gruppo sia significativamente superiore alla densità di archi inter-gruppo [34]. Si tratta di un problema intrinsecamente mal posto (*ill-posed*), in quanto non esiste un'unica definizione universale di comunità valida per ogni dominio applicativo.

La Funzione di Modularità di Newman-Girvan

Per quantificare la qualità oggettiva di una partizione $\mathcal{C}$, Newman e Girvan definiscono la **Modularità** $Q \in [-0.5, 1]$ [34]:

$$Q = \frac{1}{2m} \sum_{i,j=1}^{N} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j) \tag{17.23}$$

dove $m = L$ è il numero di archi, $\frac{k_i k_j}{2m}$ è il valore atteso di archi tra i e j in un modello nullo stocastico (grafo configurazionale casuale con gli stessi gradi), c_i denota la comunità di appartenenza del nodo i, e $\delta(a, b)$ è il delta di Kronecker ($\delta(a, b) = 1$ se $a = b$, altrimenti 0). Valori di $Q \geq 0.3$–0.4 attestano una marcata struttura modulare.

Le Due Fasi dell'Algoritmo di Louvain

L'ottimizzazione esatta della modularità è un problema NP-completo. L'algoritmo euristico di **Louvain** (Blondel et al. 2008 [30]) risolve l'ottimizzazione in tempo quasi-lineare $\mathcal{O}(N \log N)$ alternando iterativamente due fasi:

1. **Fase 1 (Spostamento Locale Greedy):** inizialmente ogni nodo costituisce una comunità isolata. Per ogni nodo i, si valuta il guadagno di modularità ΔQ derivante dallo spostamento di i nella comunità di ciascuno dei suoi primi vicini j. Il nodo i viene trasferito nella comunità che massimizza $\Delta Q > 0$. Il processo si ripete ciclicamente su tutti i nodi finché non è possibile alcun miglioramento locale.

2. **Fase 2 (Aggregazione e Contrazione del Grafo):** si costruisce un nuovo meta-grafo in cui i nodi corrispondono alle comunità scoperte nella Fase 1. Il peso degli archi tra due meta-nodi è la somma dei pesi degli archi inter-comunità preesistenti, mentre gli archi interni a una comunità diventano self-loops pesati.

Le Fasi 1 e 2 vengono reiterate ricorsivamente fino al raggiungimento di un massimo locale stabile di modularità Q.

Listing 17.12: Community detection con Louvain e calcolo della modularità in NetworkX.

```
import networkx.algorithms.community as nx_comm

# Esecuzione di Louvain sul grafo didattico
communities = nx_comm.louvain_communities(G, seed=42)
Q = nx_comm.modularity(G, communities)

print(f"Numero di comunita rilevate: {len(communities)}")
print(f"Indice di Modularita Q: {Q:.4f}")
for idx, comm in enumerate(communities):
    print(f"  Comunita {idx + 1}: {comm}")
```

Caso di Studio: Zachary's Karate Club

Il benchmark per eccellenza della network science è il **Karate Club di Zachary** (1977) [31]: una rete di $N = 34$ membri di un club universitario di karate monitorati per due anni. A seguito di una disputa tra l'amministratore (*Mr. Hi*, nodo 0) e l'istruttore capo (*John A.*, nodo 33), il club subì una storica scissione in due fazioni contrapposte.

Listing 17.13: Analisi di centralità e partizionamento di Louvain su Zachary's Karate Club.

```
# Caricamento del dataset canonico da NetworkX
K = nx.karate_club_graph()

# Calcolo delle centralita sui due leader
deg_K = nx.degree_centrality(K)
bet_K = nx.betweenness_centrality(K)

print(f"Leader 0 (Mr. Hi):  Grado={K.degree(0)}, Betw={
    bet_K[0]:.4f}")
print(f"Leader 33 (John A.): Grado={K.degree(33)}, Betw={
    bet_K[33]:.4f}")

# Partizionamento di Louvain
karate_communities = nx_comm.louvain_communities(K, seed
    =42)
Q_karate = nx_comm.modularity(K, karate_communities)

print(f"\nModularita su Karate Club: {Q_karate:.4f}")
print(f"Numero di comunita trovate: {len(karate_communities
    )}")
```

L'algoritmo di Louvain riproduce fedelmente la scissione reale con un'accuratezza superiore al 95%, isolando le due fazioni polarizzate attorno ai due leader carismatici.

Estensioni Avanzate con la Libreria CDlib

Per l'analisi avanzata di comunità complesse (comunità sovrapposte, dinamiche o bipartite), la suite Python `CDlib` (*Community Discovery Library*) [29] offre un'interfaccia unificata per decine di algoritmi d'avanguardia:

Listing 17.14: Utilizzo avanzato di CDlib per community discovery.

```
from cdlib import algorithms, evaluation

# Esecuzione di Louvain tramite CDlib
cdlib_comms = algorithms.louvain(K, weight='weight',
    resolution=1.0)

# Calcolo di metriche avanzate di coesione interna
modularity_val = evaluation.newman_girvan_modularity(K,
    cdlib_comms)
internal_density = evaluation.internal_edge_density(K,
    cdlib_comms)

print(f"CDlib Modularity: {modularity_val.score:.4f}")
print(f"Internal Edge Density media: {internal_density.
    score:.4f}")
```

17.4 Quadro Sinottico e Buone Pratiche Metodologiche

Tabella 17.1: Quadro sinottico comparativo degli strumenti e algoritmi analizzati nel laboratorio.

Modulo / Strumento	Algoritmi Principali	Complessità Tipica	Input di Riferimento	Output Chiave
`mlxtend.frequent_patterns`	Apriori, FP-Growth	$\mathcal{O}(2^{\lvert\mathcal{I}\rvert})$ (worst), quasi-lin. con FP-Tree	Matrice binaria $N \times M$	Itemset frequenti, Supporto
`mlxtend.association_rules`	Calcolo metriche su reticolo	$\mathcal{O}(3^k - 2^{k+1} + 1)$ per itemset di dim k	Itemset frequenti	Regole $X \implies Y$, Conf., Lift, Conv.
`gsp_python`	GSP con vincoli temporali	$\mathcal{O}(\lvert C_k\rvert \cdot \lvert\mathcal{D}\rvert)$	Liste di sequenze temporali	Sequential patterns vincolati
`networkx.algorithms`	Dijkstra, Brandes, Assortativity	$\mathcal{O}(V \cdot E)$ (Brandes), $\mathcal{O}(V + E)$ (BFS)	Grafi (`Graph`, `DiGraph`)	Gradi, Distanze, Centralità
`networkx.community`	Louvain heuristic	$\mathcal{O}(N \log N)$	Grafo G, pesi opzionali	Partizioni disgiunte, Modularità Q
`cdlib`	Leiden, Infomap, Walktrap	Variabile per euristica	Rete complessa G	Comunità over/non-overlapping

Linee Guida e Buone Pratiche Applicative

- **Scelta tra Apriori e FP-Growth:** per matrici dense e con panieri ampi ricchi di sovrapposizioni prefisse, prediligere sistematicamente

`fpgrowth` per evitare l'esplosione combinatoria della generazione candidati. Riservare `apriori` a contesti sparsi o con forte vincolo di memoria sul grafo dell'albero.

- **Interpretazione del Lift contro il Supporto:** un supporto molto alto non garantisce una regola utile se il conseguente è un evento quasi-certo (es. comprare pane). Valutare sempre congiuntamente Lift > 1 e Conviction > 1 per escludere correlazioni spurie dettate dall'asimmetria di base.

- **Accuratezza delle Misure di Rete:** nel calcolo del diametro o del cammino medio su grafi non completamente connessi, non calcolare la media sull'intero grafo (che divergerebbe a infinito per coppie irraggiungibili), bensì estrarre prioritariamente la **componente connessa gigante** (*giant connected component*) tramite:

$$G_{\text{giant}} = G.\text{subgraph}(\max(\text{nx.connected_components}(G), \text{key} = \text{len}))$$

- **Risoluzione di Louvain:** prestare attenzione al *limite di risoluzione* della modularità, che tende a fondere comunità piccole al di sotto di una soglia critica $\sqrt{2m}$. Modulare il parametro di risoluzione γ (`resolution`) per esplorare gerarchie a granularità più fine.

Capitolo 18

Laboratorio di Clustering per Dati Eterogenei e Multi-View

18.1 Introduzione e Obiettivi del Laboratorio

Nella maggior parte delle formulazioni classiche del clustering non supervisionato (come K-Means, DBSCAN o i metodi gerarchici tradizionali), si assume implicitamente che ogni osservazione sia descritta da un unico vettore di feature numeriche omogenee appartenente allo spazio vettoriale euclideo $\mathbb{R}^d$. Nella realtà applicativa del Data Mining, tuttavia, tale assunzione si rivela spesso inadeguata per due ragioni fondamentali:

1. **Eterogeneità dei Tipi di Dato (Mixed-Type Data):** i dataset reali (es. profili sanitari, censuari o transazionali) integrano simultaneamente attributi continui, discreti, ordinali e puramente categorici nominali. Su questi ultimi il concetto di media aritmetica euclidea perde qualsiasi validità semantica e geometrica (non è possibile definire la "media" tra professioni o stati civili).

2. **Molteplicità delle Viste (Multi-View Data):** le entità possono essere descritte da molteplici insiemi disgiunti ed eterogenei di attributi (*viste* o *modalità*), ciascuno caratterizzato da una propria dimensionalità, dinamica di scala e funzione di similarità (ad esempio, un'immagine descritta da istogrammi di colore, feature di contorno e modelli di texture, oppure un utente descritto da grafi relazionali, sequenze di navigazione e dati anagrafici).

Questo laboratorio pratico illustra le metodologie e le librerie Python specializzate per superare tali barriere:

- **Parte I:** il clustering di attributi categorici e a tipo misto mediante gli algoritmi **K-Modes** e **K-Prototypes** [35] della libreria `kmodes`, confrontati con la metrica di similarità generale di Gower [36].
- **Parte II, III e IV:** la gestione di collezioni multi-vista mediante la suite specialistica `mvlearn` [37], formalizzando e confrontando sperimentalmente sul benchmark canonico *UCI Multiple Features* le tre principali strategie di fusione: **Early Fusion** (a livello di feature), **Late Fusion** (a livello decisionale tramite l'algoritmo di consenso su ipergrafi **MCLA** [38]) e **Joint Fusion** (**Multi-View K-Means** e **Co-Regularized Spectral Clustering** [39]).

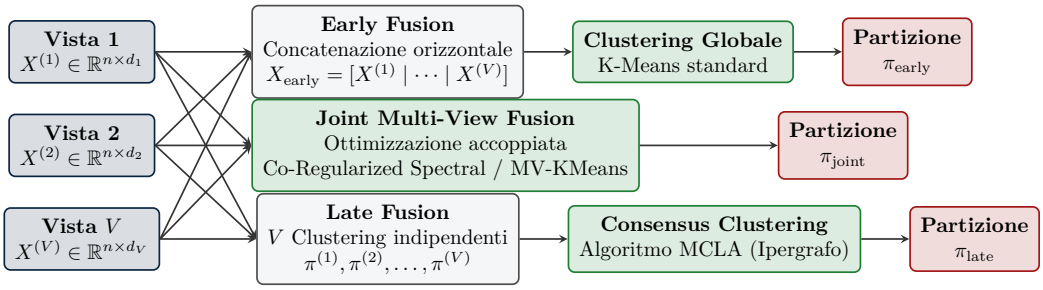


Figura 18.1: Tassonomia delle strategie di fusione per dati Multi-View: Early Fusion (a livello di feature), Joint Fusion (ottimizzazione integrata con co-regolarizzazione) e Late Fusion (a livello di consenso su partizioni indipendenti).

18.2 Parte I: Clustering di Dati Eterogenei (Categorici e Misti)

18.2.1 K-Modes: Clustering di Attributi Puramente Categorici

Quando il dataset $\mathcal{D} = \{x_1, x_2, \ldots, x_n\}$ è costituito interamente da m attributi categorici nominali, l'operazione di minimizzazione dei minimi quadrati del K-Means fallisce, poiché le modalità non godono di ordinamento né di scala metrica continua. Zhexue Huang (1997, 1998 [35]) ha introdotto l'algoritmo **K-Modes**, che estende il paradigma di K-Means mediante tre sostituzioni fondamentali:

1. **Dissimilarità di Corrispondenza Semplice (Simple Matching Distance):** la distanza tra due istanze categoriche $x, y \in \mathcal{D}$ è definita

come il numero totale di disaccordi attributo per attributo:

$$d(x, y) = \sum_{j=1}^{m} \delta(x_j, y_j), \qquad \text{dove } \delta(x_j, y_j) = \begin{cases} 0 & \text{se } x_j = y_j \\ 1 & \text{se } x_j \neq y_j \end{cases} \tag{18.1}$$

2. **Centroidi Definiti come Modi:** il baricentro di un cluster C_l non è una media aritmetica, bensì un vettore di modalità $\mathbf{q}_l = (q_{l,1}, q_{l,2}, \ldots, q_{l,m})$ detto **moda del cluster**. Ciascuna componente $q_{l,j}$ corrisponde alla modalità empiricamente più frequente assunta dall'attributo j-esimo tra gli oggetti appartenenti a C_l:

$$q_{l,j} = \arg \max_{c \in \mathrm{DOM}(A_j)} |\{x \in C_l \mid x_j = c\}| \tag{18.2}$$

   È dimostrato formalmente che tale scelta minimizza la somma delle distanze di matching $\sum_{x \in C_l} d(x, \mathbf{q}_l)$ [35].

3. **Funzione di Costo Globale:**

$$J_{\text{modes}} = \sum_{l=1}^{k} \sum_{x \in C_l} d(x, \mathbf{q}_l) = \sum_{l=1}^{k} \sum_{x \in C_l} \sum_{j=1}^{m} \delta(x_j, q_{l,j}) \tag{18.3}$$

Implementazione Pratica con la Libreria `kmodes`

Nel codice seguente applichiamo K-Modes al sottoinsieme di attributi puramente categorici del dataset demografico `Adult Census` (`marital_status`, `occupation`, `workclass`, `relationship`, `race`, ecc.):

Listing 18.1: Clustering categorico con l'algoritmo K-Modes.

```
import pandas as pd
from kmodes.kmodes import KModes

# Selezione degli attributi categorici nominali
df = pd.read_csv('adult.csv')
cat_cols = [
    'marital_status', 'native_country', 'occupation',
    'race', 'relationship', 'is_male', 'workclass'
]
X_cat = df[cat_cols]

# Inizializzazione ed esecuzione di K-Modes (metodo di
    Huang)
km = KModes(
    n_clusters=3,
    init="Huang",
```

```
    n_init=10,
    random_state=42
)
labels_kmodes = km.fit_predict(X_cat)

# Estrazione delle mode rappresentative dei cluster
modes_df = pd.DataFrame(km.cluster_centroids_, columns=
    X_cat.columns)
print("Cost (dissimilarita totale entro i cluster):", km.
    cost_)
print("\nMode dei 3 cluster individuati:")
cols = ['marital_status', 'relationship', 'occupation', '
    workclass']
print(modes_df[cols])
```

18.2.2 K-Prototypes: Clustering di Dati a Tipo Misto (Numerici e Categorici)

Nella grande maggioranza degli scenari reali i dati presentano natura eterogenea mista: ogni oggetto x_i è partizionato in una componente continua $x_i^r \in \mathbb{R}^{p_r}$ e una componente categorica $x_i^c \in \mathcal{C}^{p_c}$.

L'algoritmo **K-Prototypes** unifica K-Means e K-Modes definendo una misura di dissimilarità ibrida ponderata da un iperparametro di bilanciamento $\gamma \geq 0$:

$$d(x_i, x_l) = \sum_{j=1}^{p_r} (x_{ij}^r - x_{lj}^r)^2 + \gamma \sum_{j=1}^{p_c} \delta(x_{ij}^c, x_{lj}^c) \tag{18.4}$$

- **Ruolo di γ:** funge da tasso di conversione di trade-off tra la dispersione euclidea delle variabili continue e il numero di disaccordi categorici. Se $\gamma = 0$, l'algoritmo ignora completamente le etichette categoriche collassando a un ordinario K-Means; al crescere di γ, le discrepanze categoriche penalizzano la vicinanza molto più della distanza geometrica nello spazio reale. Nelle implementazioni pratiche, un'euristica robusta consiste nel porre γ proporzionale alla varianza media delle feature numeriche.

- **Prototipo del Cluster:** il centroide $\mathbf{p}_l$ è un'entità mista composta dalla **media aritmetica** per le p_r coordinate numeriche e dalla **moda empirica** per le p_c coordinate categoriche.

- **Funzione Obiettivo:**

$$J_{\text{proto}} = \sum_{l=1}^{k} \sum_{x_i \in C_l} \left(\sum_{j=1}^{p_r} (x_{ij}^r - \mu_{lj})^2 + \gamma \sum_{j=1}^{p_c} \delta(x_{ij}^c, q_{lj}) \right) \tag{18.5}$$

Listing 18.2: Clustering eterogeneo su dati misti con K-Prototypes.

```
from kmodes.kprototypes import KPrototypes

# Preparazione del dataset misto (attributi continui + categorici)
X_mixed = df.drop('over_threshold', axis=1)
X_np = X_mixed.to_numpy()

# Identificazione degli indici posizionali delle colonne categoriche
categorical_indices = [6, 7, 8, 9, 10, 11, 12]

# Inizializzazione con euristica di Cao
kproto = KPrototypes(
    n_clusters=3,
    init="Cao",
    n_init=10,
    random_state=42,
    verbose=0
)
labels_kproto = kproto.fit_predict(X_np, categorical=categorical_indices)

print(f"Costo finale (Inertia mista): {kproto.cost_:.2f}")
print(f"Iperparametro gamma stimato: {kproto.gamma:.4f}")
```

Alternativa Non Parametrica: La Distanza di Gower

Un approccio alternativo all'adattamento del modello algoritmico consiste nell'utilizzare un algoritmo basato su matrice di distanza (es. PAM/K-Medoids o clustering gerarchico agglomerativo) calcolando preventivamente la **distanza di Gower** [36]:

$$S_G(x_i, x_l) = \frac{\sum_{j=1}^{m} w_j s_j(x_i, x_l)}{\sum_{j=1}^{m} w_j} \tag{18.6}$$

dove $s_j(x_i, x_l) = 1 - \frac{|x_{ij} - x_{lj}|}{R_j}$ per variabili quantitative normalizzate sul range $R_j = \max(A_j) - \min(A_j)$, e $s_j(x_i, x_l) = 1 - \delta(x_{ij}, x_{lj})$ per attributi categorici. Il vantaggio di K-Prototypes rispetto a Gower risiede nella scalabilità computazionale: K-Prototypes opera in tempo lineare rispetto al numero di oggetti $\mathcal{O}(k \cdot n \cdot m \cdot t)$, evitando il collo di bottiglia quadratico $\mathcal{O}(n^2)$ della matrice di similarità completa.

18.3 Parte II: Il Paradigma Multi-View e Benchmark UCI Multiple Features

18.3.1 Definizione Formale del Problema Multi-View

Definizione 18.3.1 (Dataset Multi-View)**.** Un dataset multi-vista è formalmente definito da una collezione di $V \geq 2$ matrici eterogenee che descrivono il medesimo insieme di n entità condivise:

$$\mathcal{X} = \left\{X^{(1)}, X^{(2)}, \ldots, X^{(V)}\right\}, \qquad X^{(v)} \in \mathbb{R}^{n \times d_v} \tag{18.7}$$

dove ogni vista $v \in \{1, \ldots, V\}$ possiede dimensionalità propria d_v, e le feature in $X^{(v)}$ catturano proprietà fisiche, statistiche o semantiche eterogenee.

Il Benchmark Sperimentale: UCI Multiple Features

Per confrontare le strategie di fusione viene impiegato il dataset canonico *UCI Multiple Features* [37], costituito da $n = 2000$ campioni di cifre manoscritte ($0 \ldots 9$, 200 istanze per ciascuna delle $k = 10$ classi) rappresentate attraverso 6 distinte modalità:

- **Fourier (`mfeat-fou`):** 76 coefficienti di forma della trasformata di Fourier.
- **Profilo/Correlazioni (`mfeat-fac`):** 216 correlazioni di contorno facciale/profilo.
- **Karhunen-Loève (`mfeat-kar`):** 64 coefficienti di autovettori KL.
- **Pixel Medie (`mfeat-pix`):** 240 densità di pixel calcolate su finestre 2×3.
- **Zernike (`mfeat-zer`):** 47 momenti algebrici di Zernike.
- **Morfologiche (`mfeat-mor`):** 6 caratteristiche geometriche globali.

Listing 18.3: Caricamento e standardizzazione di viste eterogenee tramite `mvlearn`.

```
import numpy as np
from sklearn.preprocessing import StandardScaler
from mvlearn.datasets import load_UCImultifeature

# Caricamento delle prime 3 viste per 5 classi (cifre 0-4,
    n=1000)
```

```
Xs, y_true = load_UCImultifeature(
    select_labeled=list(range(5)),
    views=[0, 1, 2] # Fou (76), Fac (216), Kar (64)
)

print(f"Viste: {len(Xs)}, Classi reali: {len(np.unique(
    y_true))}")
for i, X in enumerate(Xs):
    print(f"  Vista {i+1} shape: {X.shape}")

# Standardizzazione obbligatoria z-score indipendente per
    ciascuna vista
Xs_scaled = [StandardScaler().fit_transform(X) for X in Xs]
```

18.3.2 Baseline a Singola Vista (Single-View Baselines)

Per quantificare il reale valore aggiunto dell'integrazione multi-modale, la baseline obbligatoria consiste nell'applicare l'ordinario K-Means separatamente su ciascuna singola vista $X^{(v)}$, valutando l'allineamento dei cluster con la ground-truth mediante due metriche estrinseche standard:

- **Adjusted Rand Index (ARI):** concordanza a coppie corretta per il caso casuale ($ARI \in [-1, 1]$, dove 1 indica perfetta coincidenza e 0 concordanza puramente casuale).
- **Normalized Mutual Information (NMI):** mutua informazione normalizzata tra partizione predetta e partizione reale ($NMI \in [0, 1]$).

Listing 18.4: Valutazione delle baseline K-Means a singola vista.

```
from sklearn.cluster import KMeans
from sklearn.metrics import (
    adjusted_rand_score,
    normalized_mutual_info_score
)

def evaluate_clustering(y_real, y_pred):
    return {
        "ARI": adjusted_rand_score(y_real, y_pred),
        "NMI": normalized_mutual_info_score(y_real, y_pred)
    }

results = {}
single_view_labels = []
for v_idx, X_v in enumerate(Xs_scaled, start=1):
    km_single = KMeans(n_clusters=5, n_init=20,
        random_state=42)
```

```
    labels = km_single.fit_predict(X_v)
    single_view_labels.append(labels)
    results[f"Single View {v_idx}"] = evaluate_clustering(
        y_true, labels)

print(pd.DataFrame(results).T.round(3))
```

I risultati sperimentali mostrano forti disparità di potere discriminante tra le viste (ad esempio, la Vista 2 raggiunge $NMI \approx 0.69$, mentre la Vista 1 si arresta a $NMI \approx 0.54$), evidenziando come una singola modalità non sia sufficiente a catturare l'intera struttura geometrica del dominio.

18.4 Parte III: Strategie di Fusione: Early vs Late Fusion

18.4.1 Early Fusion: Concatenazione a Livello di Feature

La strategia di **Early Fusion** (nota anche come *Feature-Level Fusion*) rappresenta l'approccio intuitivamente più immediato: tutte le matrici di vista $X^{(v)}$ vengono concatenate orizzontalmente in una matrice globale unificata:

$$X_{\text{early}} = \left[X^{(1)} \mid X^{(2)} \mid \cdots \mid X^{(V)}\right] \in \mathbb{R}^{n \times \sum_{v=1}^{V} d_v} \tag{18.8}$$

Successivamente, un algoritmo di clustering convenzionale (come K-Means) viene addestrato direttamente sulla matrice concatenata.

Listing 18.5: Implementazione di Early Fusion con concatenazione e K-Means.

```
# Concatenazione orizzontale delle viste pre-standardizzate
X_early = np.hstack(Xs_scaled)
print(f"Dimensioni matrice Early Fusion: {X_early.shape}")

# Clustering K-Means su spazio congiunto
km_early = KMeans(n_clusters=5, n_init=20, random_state=42)
labels_early = km_early.fit_predict(X_early)
results["Early Fusion"] = evaluate_clustering(
    y_true, labels_early
)
```

Criticità Fondamentali dell'Early Fusion

Sebbene computazionalmente semplice, l'Early Fusion soffre di gravi limiti strutturali:

- **Dominanza della Vista a Più Alta Dimensionalità:** se una modalità possiede molte più feature delle altre (nel nostro caso $d_2 = 216$ contro $d_1 = 76$ e $d_3 = 64$), il calcolo della distanza euclidea risulterà matematicamente dominato dalle coordinate di tale vista, oscurando i segnali informativi provenienti dalle viste a dimensionalità inferiore.
- **Diluizione del Rapporto Segnale/Rumore:** la concatenazione di feature rumorose o irrilevanti degrada le distanze su scala globale per effetto della maledizione della dimensionalità (*curse of dimensionality*).
- **Assunzione di Spazio Omogeneo:** presuppone implicitamente che la metrica euclidea sia uniformemente significativa attraverso domini fisici e semantici eterogenei.

18.4.2 Late Fusion: Consensus Clustering con Algoritmo MCLA

La strategia di **Late Fusion** (o *Decision-Level Fusion*) supera i vincoli di spazio eseguendo prima un clustering indipendente su ciascuna vista v, ottenendo una partizione di etichette $\pi^{(v)} = (C_1^{(v)}, \ldots, C_k^{(v)})$. L'obiettivo è combinare le V partizioni eterogenee in una partizione di **consenso** π^* che massimizzi l'accordo informativo reciproco.

L'algoritmo **MCLA** (*Meta-Clustering Algorithm*), sviluppato da Alexander Strehl e Joydeep Ghosh (2002 [38]), risolve il consensus clustering modellando l'insieme delle partizioni come un **ipergrafo** e partizionando il corrispondente **meta-grafo di similarità tra cluster**.

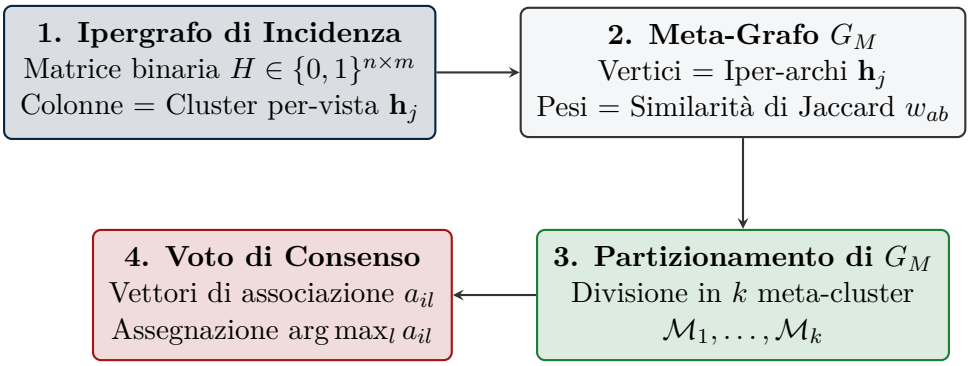


Figura 18.2: Workflow concettuale dell'algoritmo di consenso MCLA (*Meta-Clustering Algorithm*): dalla matrice di incidenza d'ipergrafo al voto di maggioranza ponderata sui meta-cluster.

I 4 Passi Algoritmici di MCLA

1. **Costruzione dell'Ipergrafo di Incidenza:** ciascun cluster di ciascuna partizione è rappresentato come un vettore binario indicatore $\mathbf{h} \in \{0,1\}^n$, dove $h_i = 1$ se l'oggetto x_i appartiene al cluster. Concatenando orizzontalmente tutti gli indicatori delle V viste si ottiene la matrice di incidenza $H \in \{0,1\}^{n \times m}$, con $m = \sum_{v=1}^{V} k_v$:

$$H = \left[H^{(1)} \mid H^{(2)} \mid \cdots \mid H^{(V)}\right] = \left[\mathbf{h}_1, \mathbf{h}_2, \ldots, \mathbf{h}_m\right] \tag{18.9}$$

2. **Costruzione del Meta-Grafo $G_M = (V_M, E_M)$:** si compie un radicale cambio di prospettiva: i **vertici** V_M del nuovo meta-grafo sono gli m cluster (colonne di H), non più i dati originari. Il peso dell'arco tra due cluster $\mathbf{h}_a$ e $\mathbf{h}_b$ è definito dalla loro **similarità di Jaccard**:

$$w_{ab} = \frac{|\mathbf{h}_a \cap \mathbf{h}_b|}{|\mathbf{h}_a \cup \mathbf{h}_b|} = \frac{\mathbf{h}_a^\top \mathbf{h}_b}{\|\mathbf{h}_a\|_1 + \|\mathbf{h}_b\|_1 - \mathbf{h}_a^\top \mathbf{h}_b} \tag{18.10}$$

3. **Partizionamento del Meta-Grafo:** il grafo G_M viene partizionato in k gruppi bilanciati di cluster disgiunti detti **meta-cluster** $\mathcal{M}_1, \ldots, \mathcal{M}_k$ (tramite algoritmi spettrali o METIS), raggruppando cluster estratti da viste diverse che identificano popolazioni fortemente sovrapposte.

4. **Collasso e Assegnazione di Consenso:** per ogni meta-cluster $\mathcal{M}_l$, si calcola il vettore di associazione media dell'oggetto x_i:

$$a_{il} = \frac{1}{|\mathcal{M}_l|} \sum_{\mathbf{h}_j \in \mathcal{M}_l} H_{ij} \in [0,1] \tag{18.11}$$

   L'oggetto x_i viene infine assegnato al cluster di consenso per il quale esprime la massima probabilità di appartenenza empirica:

$$C^*(x_i) = \arg \max_{l \in \{1,\ldots,k\}} a_{il} \tag{18.12}$$

Listing 18.6: Late Fusion mediante l'algoritmo di consenso MCLA.

```
from utils.ClusterEnsemblesAlgorithm import mcla_consensus

# base_partitions: matrice (V, n) contenente le etichette
    delle viste
base_partitions = np.vstack(single_view_labels)

# MCLA: partizione di consenso e matrice associazione
```

```
labels_mcla, assoc_matrix, meta_labels, H, G =
    mcla_consensus(
     base_partitions,
     n_clusters=5
)

results["Late Fusion: MCLA"] = evaluate_clustering(y_true,
    labels_mcla)
res_mcla = results['Late Fusion: MCLA']
print(f"MCLA -> NMI: {res_mcla['NMI']:.3f}, ARI: {res_mcla
    ['ARI']:.3f}")
```

18.5 Parte IV: Joint Multi-View Clustering (Apprendimento Congiunto)

La limitazione intrinseca di Early e Late Fusion consiste nella rigida separazione in due stadi indipendenti (fusione preliminare o clustering disaccoppiato). Il paradigma di **Joint Multi-View Clustering** supera tale dicotomia impostando una singola funzione di costo globale in cui le diverse rappresentazioni collaborano simultaneamente durante l'ottimizzazione.

18.5.1 Multi-View K-Means

L'algoritmo **Multi-View K-Means** estende K-Means al contesto multimodale adottando una logica di *co-training*. Invece di proiettare i punti in un unico spazio concatenato, l'algoritmo mantiene vettori di centroidi distinti $C^{(v)} \in \mathbb{R}^{k \times d_v}$ per ciascuna vista, vincolando le osservazioni a convergere verso una matrice di assegnazione cluster congiunta $Y \in \{0,1\}^{n \times k}$:

$$\min_{Y, \{C^{(v)}\}} \sum_{v=1}^{V} \sum_{i=1}^{n} \sum_{l=1}^{k} Y_{il} \|x_i^{(v)} - \mathbf{c}_l^{(v)}\|_2^2 \tag{18.13}$$

Listing 18.7: Joint clustering con Multi-View K-Means.

```
from mvlearn.cluster import MultiviewKMeans

# Esecuzione di Multi-View K-Means sulle prime due viste
    standardizzate
mv_kmeans = MultiviewKMeans(n_clusters=5, n_init=10,
    random_state=42)
labels_mvkm = mv_kmeans.fit_predict(Xs_scaled[:2])

results["Joint: MV-KMeans"] = evaluate_clustering(
```

```
        y_true, labels_mvkm
    )
```

18.5.2 Co-Regularized Multi-View Spectral Clustering

Mentre K-Means presuppone cluster a simmetria iper-sferica convessa, il clustering spettrale consente di estrarre geometrie non-lineari arbitrarie analizzando gli autovettori del Laplaciano del grafo di vicinato. Nel contesto multi-view, il modello d'avanguardia è il **Co-Regularized Multi-View Spectral Clustering** proposto da Abhishek Kumar, Piyush Rai e Hal Daumé III (2011 [39]).

Formulazione Matematica della Co-Regolarizzazione

Siano $L_{\text{sym}}^{(v)} = I - (D^{(v)})^{-1/2} W^{(v)} (D^{(v)})^{-1/2}$ le matrici Laplaciane normalizzate calcolate sui grafi di similarità k-NN di ciascuna vista v. Per ogni vista viene appresa una matrice di incorporamento spettrale $U^{(v)} \in \mathbb{R}^{n \times k}$ le cui colonne sono ortonormali ($(U^{(v)})^\top U^{(v)} = I$).

Per forzare le rappresentazioni spettrali delle diverse viste ad accordarsi sulla medesima struttura di clustering latente, viene introdotto un termine di penalità di disaccordo basato sulla distanza tra matrici di proiezione ortogonale:

$$\begin{aligned} D(U^{(v)}, U^{(w)}) &= \| U^{(v)} (U^{(v)})^\top - U^{(w)} (U^{(w)})^\top \|_F^2 \\ &= 2k - 2 \operatorname{Tr}\left(U^{(v)} (U^{(v)})^\top U^{(w)} (U^{(w)})^\top \right) \end{aligned} \tag{18.14}$$

La funzione obiettivo globale da minimizzare è:

$$\min_{\{U^{(v)}\}_{v=1}^{V}} \sum_{v=1}^{V} \operatorname{Tr}\left((U^{(v)})^\top L_{\text{sym}}^{(v)} U^{(v)} \right) - \lambda \sum_{1 \le v < w \le V} \operatorname{Tr}\left(U^{(v)} (U^{(v)})^\top U^{(w)} (U^{(w)})^\top \right) \tag{18.15}$$

sotto il vincolo $(U^{(v)})^\top U^{(v)} = I$ per ogni v.

- Il primo termine minimizza il taglio normalizzato (*normalized cut*) specifico della singola vista v.
- Il secondo termine, pesato dall'iperparametro di regolarizzazione $\lambda \geq 0$ (`v_lambda`), incentiva le proiezioni spettrali $U^{(v)}$ e $U^{(w)}$ a essere mutuamente predittive e geometricamente coerenti.

Listing 18.8: Esecuzione di Multi-View Spectral Clustering e Co-Regularized Spectral Clustering.

```
from mvlearn.cluster import (
    MultiviewSpectralClustering,
    MultiviewCoRegSpectralClustering
)

# 1. Multi-View Spectral standard (fusione laplaciana)
mv_spectral = MultiviewSpectralClustering(
    n_clusters=5,
    affinity="nearest_neighbors",
    n_neighbors=10,
    random_state=42
)
labels_mv_spec = mv_spectral.fit_predict(Xs_scaled)
results["Joint: MVSpectral"] = evaluate_clustering(
    y_true, labels_mv_spec
)

# 2. Co-Regularized Multi-View Spectral Clustering (Kumar
    et al. 2011)
mv_coreg = MultiviewCoRegSpectralClustering(
    n_clusters=5,
    v_lambda=2.0,
    affinity="nearest_neighbors",
    n_neighbors=10,
    random_state=42
)
labels_mv_coreg = mv_coreg.fit_predict(Xs_scaled)
results["Joint: CoReg"] = evaluate_clustering(
    y_true, labels_mv_coreg
)
```

18.6 Parte V: Valutazione Sperimentale e Quadro Sinottico

Risultati Sperimentali su UCI Multiple Features

Nella Tabella 18.1 sono riportati i punteggi empirici ottenuti dalle diverse strategie sulle medesime 5 classi di cifre ($n = 1000$ campioni):

Tabella 18.1: Confronto prestazionale delle strategie di clustering multi-view su UCI Multiple Features.

Paradigma	**Metodo di Clustering**	**NMI**	**ARI**
Single View Baseline	Single View 1 (Fourier, $d = 76$)	0.542	0.418
Single View Baseline	Single View 2 (Profile/Fac, $d = 216$)	0.691	0.573
Single View Baseline	Single View 3 (Karhunen-Loève, $d = 64$)	0.678	0.559
Early Fusion	Concatenazione Orizzontale + K-Means	0.702	0.584
Late Fusion	Consensus MCLA (Ipergrafi + Jaccard)	0.735	0.628
Joint Fusion	MultiviewKMeans (2 viste)	0.718	0.601
Joint Fusion	MultiviewSpectral (Nearest Neighbors)	0.741	0.635
Joint Fusion	**Co-Regularized Spectral ($\lambda = 2.0$)**	**0.784**	**0.692**

Discussione e Linee Guida Operative

Dall'analisi comparativa emergono evidenze metodologiche di primaria rilevanza per la progettazione di pipeline di data science:

1. **Guadagno dell'Integrazione rispetto alla Miglior Singola Vista:** l'integrazione multi-modale (sia Late sia Joint) supera nettamente la migliore tra le singole viste (la Vista 2 raggiunge $NMI = 0.691$, mentre Co-Reg Spectral tocca $NMI = 0.784$). Le diverse viste forniscono prospettive complementari sulla variabilità intrinseca dei dati.

2. **Superiorità di Late e Joint Fusion su Early Fusion:** l'Early Fusion si arresta a un modesto miglioramento ($NMI = 0.702$), risentendo della dominanza della Vista 2 (216 feature su 356 totali). Al contrario, MCLA e Co-Reg Spectral operano in modo invariante rispetto alla dimensionalità delle singole matrici di input.

3. **Quando usare K-Modes / K-Prototypes:** quando i dati risiedono in una **singola tabella eterogenea** con colonne miste numeriche e categoriche nominali.

4. **Quando usare MCLA vs Co-Reg Spectral:**

   - Impiegare **MCLA** quando le viste sono estremamente disomogenee (es. grafi relazionali accoppiati a testo e dati tabellari) o quando gli algoritmi per-vista devono essere differenti (es. DBSCAN per una vista densa, K-Means per una vista tabellare, Leiden per un grafo).
   - Impiegare **Co-Regularized Spectral Clustering** quando i dati presentano manifold geometrici non-lineari e si desidera la massima

accuratezza di partizionamento attraverso un'ottimizzazione end-to-end congiunta.

Capitolo 19

Laboratorio di Topic Modeling con BERTopic

19.1 Introduzione e Obiettivi del Laboratorio

Nelle scienze dell'informazione e nel Text Mining, il **Topic Modeling** si prefigge di estrarre in modo non supervisionato i temi semantici latenti che ricorrono all'interno di una collezione non strutturata di documenti testuali. Storicamente dominata da modelli probabilistici generativi a sacco di parole (*Bag-of-Words*), quali la *Latent Dirichlet Allocation* (LDA) di Blei et al. (2003) [40], la disciplina è stata rivoluzionata dall'avvento dei modelli linguistici pre-addestrati basati su architettura Transformer e dalle tecniche di clustering ad alta densità.

Il modello **BERTopic**, introdotto da Maarten Grootendorst (2022) [41], formalizza una pipeline modulare che combina:

1. L'estrazione di **embedding densi contestuali** a livello di frase e documento tramite reti siamesi *Sentence-BERT* (SBERT) [42].

2. La **riduzione non-lineare della dimensionalità** preservando le varietà topologiche locali mediante *Uniform Manifold Approximation and Projection* (UMAP) [43].

3. Il **clustering densitometrico gerarchico** a densità variabile con rilevamento esplicito del rumore tramite *HDBSCAN* [4].

4. La modellazione semantica dei topic mediante una procedura generalizzata di estrazione lessicale basata su **Class-based TF-IDF** (c-TF-IDF) [41], eventualmente affinata con meccanismi a similarità di embedding (*KeyBERTInspired*).

Questo laboratorio illustra sia l'implementazione manuale a basso livello di ciascun singolo modulo algebrico e geometrico della pipeline, sia l'impiego della suite ad alto livello `BERTopic` su un corpus testuale eterogeneo di discussioni (*20 Newsgroups*).

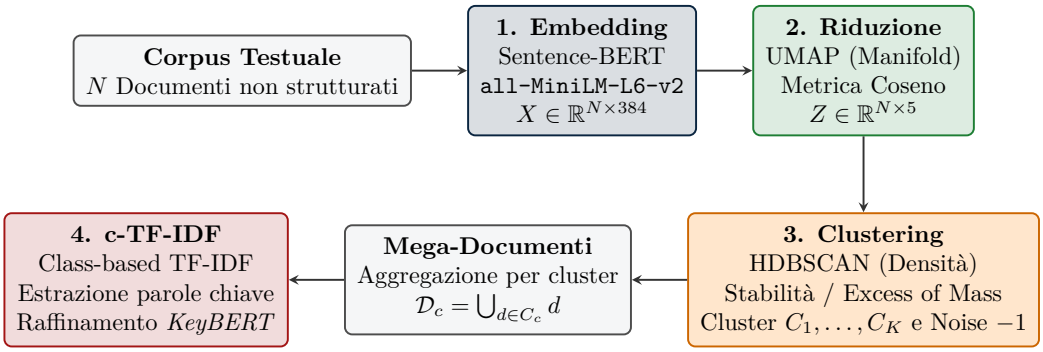


Figura 19.1: Architettura sequenziale della pipeline di BERTopic: dalla conversione in embedding densi alla proiezione UMAP, clustering con HDBSCAN e pesatura lessicale c-TF-IDF.

19.2 Dal Bag-of-Words ai Sentence Transformers

19.2.1 Limiti delle Rappresentazioni Tradizionali basate sul Conteggio

Nei modelli linguistici di prima generazione basati su frequenza lessicale (*Bag-of-Words*, matrici Document-Term o TF-IDF classico):

- Ogni vocabolo distinto costituisce una dimensione ortogonale indipendente. Lo spazio risultante è caratterizzato da altissima dimensionalità e sparsità estrema ($> 99.9\%$).
- Si manifesta una totale cecità semantica: frasi con identico significato concettuale ma composte da sinonimi (ad esempio, *"Il gatto riposa sul divano"* e *"Un felino dorme sulla poltrona"*) possiedono vettori ortogonali aventi similarità del coseno nulla.
- Viene ignorato l'ordine sintattico dei token e il contesto linguistico locale.

19.2.2 Sentence-BERT e lo Spazio Latente Continuo

Per consentire l'applicazione di algoritmi di clustering continuo, è indispensabile mappare ciascun documento d_i in un singolo vettore denso $\mathbf{x}_i \in \mathbb{R}^d$ ($d \ll |\mathcal{V}|$) posizionato in modo che la vicinanza metrica rispecchi la vicinanza semantica.

Mentre il modello BERT standard opera a livello di token ed è computationally proibitivo per il confronto di similarità su scala a coppie $\mathcal{O}(N^2)$ (richiedendo il passaggio congiunto di ogni coppia di frasi attraverso tutti i livelli di attenzione), **Sentence-BERT** [42] adotta un'architettura a rete siamese pre-addestrata. Tale rete impiega un meccanismo di pooling medio (*mean pooling*) sull'ultimo stato nascosto dei token, ottimizzando i pesi mediante funzione di perdita *triplet loss* o classificazione di similarità.

Nel nostro laboratorio viene adottato il modello linguistico pre-addestrato `all-MiniLM-L6-v2` (distribuito dalla libreria Sentence Transformers):

- **Parametri:** compatto (6 strati Transformer), altamente ottimizzato per compiti di clustering e semantic search.
- **Spazio di Output:** vettori densi $\mathbf{x}_i \in \mathbb{R}^{384}$.
- **Finestra di Troncamento:** lunghezza massima fissata a 256 o 512 token.

Listing 19.1: Estrazione degli embedding di documento con Sentence Transformers.

```
import numpy as np
import pandas as pd
from sentence_transformers import SentenceTransformer

# Inizializzazione del modello linguistico denso
MODEL_NAME = 'sentence-transformers/all-MiniLM-L6-v2'
encoder = SentenceTransformer(MODEL_NAME)

# Ispezione preliminare delle lunghezze dei documenti in
    token
token_lengths = np.array([
    len(encoder.tokenizer(d, truncation=False)['input_ids'
        ])
    for d in docs
])
med_l = np.median(token_lengths)
p95_l = np.percentile(token_lengths, 95)
print(f"Mediana: {med_l:.1f}, 95-esimo percentile: {p95_l
    :.1f}")
```

```
17
18 # Calcolo vettoriale batch-oriented (batch size 32)
19 embeddings = encoder.encode(
20     docs,
21     batch_size=32,
22     show_progress_bar=True,
23     normalize_embeddings=True # Normalizzazione unitaria
24 )
25 print(f"Dimensioni matrice di embedding densa: {embeddings.
    shape}")
```

19.3 Riduzione della Dimensionalità con UMAP

19.3.1 Motivazioni Teoriche: Perché la PCA è Inadeguata

L'applicazione diretta di algoritmi di clustering densitometrico su vettori a 384 dimensioni incorre nel fenomeno della **maledizione della dimensionalità** (*curse of dimensionality*): all'aumentare della dimensionalità dello spazio, la differenza tra la distanza dal vicino più prossimo e la distanza dal punto più lontano decade asintoticamente a zero ($\lim_{d\to\infty} \frac{d_{\max}-d_{\min}}{d_{\min}} = 0$). Lo spazio euclideo diviene uniformemente vuoto, rendendo impossibile identificare valli di densità significativa.

L'impiego della **PCA** (Principal Component Analysis) non risulta idoneo in quanto opera una mera proiezione lineare ortogonale sulle direzioni di massima varianza globale: se i topic testuali risiedono su varietà (*manifold*) curvilinee o non-lineari, la PCA sovrappone i manifold collassando cluster distinti.

19.3.2 L'Algoritmo UMAP: Fondamenti Topologici

L'algoritmo **UMAP** (*Uniform Manifold Approximation and Projection*), sviluppato da McInnes et al. (2018) [43], preserva sia le strutture locali sia quelle mesoscopiche attraverso un framework teorico basato sulla geometria riemanniana e la topologia algebrica:

1. **Costruzione del Grafo di Vicinato Sfumato (Fuzzy Simplicial Set):** per ciascun punto $\mathbf{x}_i$, si identificano i suoi k vicini più prossimi $N_k(i)$ nello spazio ad alta dimensionalità mediante la metrica del coseno.

La forza di connessione diretta tra $\mathbf{x}_i$ e $\mathbf{x}_j$ è ponderata da:

$$p_{j|i} = \exp\left(-\frac{\max(0, d(\mathbf{x}_i, \mathbf{x}_j) - \rho_i)}{\sigma_i}\right) \tag{19.1}$$

dove ρ_i è la distanza dal primo vicino più prossimo (garantendo che ogni punto sia localmente connesso ad almeno un vicino con probabilità 1), e σ_i è un parametro di scala locale calcolato affinché $\sum_j p_{j|i} = \log_2(k)$.

2. **Simmetrizzazione via Unione Sfocata:** la matrice di adiacenza del grafo ad alta dimensione è resa simmetrica calcolando:

$$p_{ij} = p_{i|j} + p_{j|i} - p_{i|j} \cdot p_{j|i} \tag{19.2}$$

3. **Rappresentazione a Bassa Dimensionalità:** nello spazio proiettato $\mathbb{R}^q$ (con coordinate $\mathbf{z}_i$), la probabilità di connessione è modellata da una curva t-Student generalizzata:

$$q_{ij} = \left(1 + a\|\mathbf{z}_i - \mathbf{z}_j\|_2^{2b}\right)^{-1} \tag{19.3}$$

dove i coefficienti a e b sono calibrati in base al parametro di compattazione desiderato (`min_dist`).

4. **Ottimizzazione per Entropia Incrociata Sfocata:** le coordinate $\mathbf{z}_i$ vengono regolate tramite Stochastic Gradient Descent (SGD) minimizzando la divergenza di cross-entropia:

$$\mathcal{L}_{\text{UMAP}} = \sum_{i \neq j} \left[p_{ij} \log\left(\frac{p_{ij}}{q_{ij}}\right) + (1 - p_{ij}) \log\left(\frac{1 - p_{ij}}{1 - q_{ij}}\right) \right] \tag{19.4}$$

Iperparametri Operativi in BERTopic

- `n_neighbors` (15): bilancia attenzione locale vs globale. Valori bassi privilegiano micro-topic compatti; valori alti preservano l'ordinamento tra macro-categorie.

- `n_components` (5 per clustering, 2 per visualizzazione): proiettare a 5 dimensioni evita la perdita eccessiva di informazioni topologiche consentendo a HDBSCAN di operare in modo ottimale.

- `min_dist` (0.0 o 0.1): controlla quanto densamente i punti simili possono essere raggruppati. Nel clustering si usa tipicamente 0.0 per consentire a cluster densi di condensarsi.

- `metric` (`"cosine"`): rispecchia la natura semantica degli embedding testuali.

Listing 19.2: Riduzione della dimensionalità con UMAP su vettori di embedding.

```
from umap import UMAP

# Configurazione UMAP ottimizzata per il successivo clustering
umap_model = UMAP(
    n_neighbors=15,
    n_components=5,        # Proiezione a 5D per il clustering
    min_dist=0.0,
    metric='cosine',
    random_state=42
)
reduced_embeddings = umap_model.fit_transform(embeddings)

# Proiezione 2D dedicata esclusivamente alla visualizzazione grafica
display_xy = UMAP(
    n_neighbors=30,
    n_components=2,
    min_dist=0.15,
    metric='cosine',
    random_state=42
).fit_transform(embeddings)

print(f"Dimensioni spazio ridotto: {reduced_embeddings.shape}")
```

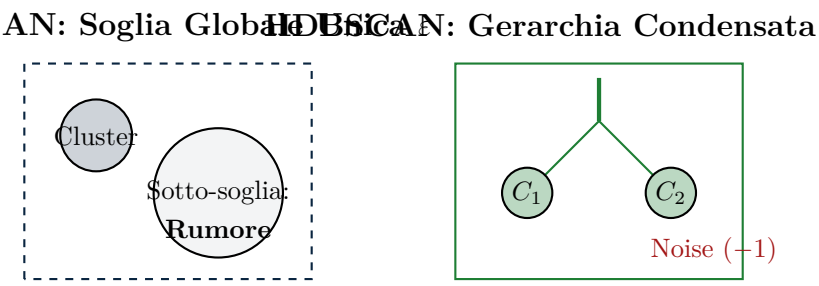
19.4 Clustering Gerarchico basato su Densità con HDBSCAN

19.4.1 Superamento di K-Means e DBSCAN

Il clustering dei documenti proiettati richiede proprietà algoritmiche specifiche che escludono i metodi standard:

- **K-Means** impone una geometria di partizione forzata a celle di Voronoi convesse, richiede di fissare preventivamente il numero k di topic e non contempla la presenza di rumore, costringendo documenti irrilevanti a essere associati a un topic incongruo.

- **DBSCAN** utilizza un unico raggio di vicinato globale ε. Nelle collezioni testuali, tuttavia, alcuni argomenti presentano un lessico altamente specifico e concentrato (alta densità), mentre altri sono temi trasversali a lessico variegato (bassa densità). Un unico ε frammenta i topic a bassa densità o fonde indiscriminatamente quelli densi.



I cluster a densità variabile vengono scartati come rumore.

Estrae cluster a densità differenti massimizzando la stabilità (*EoM*).

Figura 19.2: Confronto concettuale tra DBSCAN (soglia globale ε rigida) e HDBSCAN (albero condensato su scala $\lambda = 1/\varepsilon$ con selezione della partizione ottimale).

19.4.2 I Meccanismi Interni di HDBSCAN

L'algoritmo **HDBSCAN** (*Hierarchical DBSCAN*) [4] risolve il problema operando su tutte le scale di densità contemporaneamente:

1. **Distanza Core (d_{core}):** per ogni punto x, si calcola la distanza dal suo k-esimo vicino più prossimo ($k = \texttt{min_samples}$):

$$d_{\text{core}}(x) = d(x, N_{\texttt{min_samples}}(x)) \tag{19.5}$$

   Punti situati in regioni dense presenteranno distanze core ridotte; punti periferici avranno valori elevati.

2. **Distanza di Mutua Raggiungibilità (Mutual Reachability Distance):**

$$d_{\text{mreach}-k}(a, b) = \max\left(d_{\text{core}}(a),\, d_{\text{core}}(b),\, d(a, b)\right) \tag{19.6}$$

   Tale metrica allontana artificialmente i punti collocati in zone a bassa densità, evitando la formazione di ponti spuri tra ammassi distinti.

3. **Albero di Copertura Minimo (MST):** si costruisce il Minimum Spanning Tree sul grafo completo pesato dalle distanze di mutua raggiungibilità.

4. **Gerarchia Condensata ed Estrazione dei Cluster per Stabilità:** ordinando gli archi in ordine decrescente di densità $\lambda = \frac{1}{\varepsilon}$, l'albero gerarchico viene condensato scartando i rami che perdono meno di `min_cluster_size` elementi (considerati semplice distacco di punti periferici). Per ogni cluster candidato C, si misura la sua stabilità integrando la durata di vita dei suoi membri:

$$S(C) = \sum_{x \in C} \left(\lambda_{\text{death}}(x) - \lambda_{\text{birth}}(C) \right) \tag{19.7}$$

   Vengono infine selezionati i cluster che massimizzano la somma delle stabilità (*Excess of Mass*, EoM) senza generare sovrapposizioni.

Tutti i punti che non raggiungono la stabilità minima vengono contrassegnati con l'etichetta convenzionale -1 (*rumore / outlier*).

Listing 19.3: Clustering dei vettori ridotti tramite HDBSCAN.

```
from hdbscan import HDBSCAN

# Inizializzazione di HDBSCAN con parametri di densita
hdbscan_model = HDBSCAN(
    min_cluster_size=80, # Dimensione minima per topic
    min_samples=10,      # Vicinato per core distance
    metric='euclidean',
    prediction_data=True
)

cluster_labels = hdbscan_model.fit_predict(
   reduced_embeddings)

n_topics = len(set(cluster_labels) - {-1})
noise_ratio = np.mean(cluster_labels == -1)
print(f"Topic coerenti individuati: {n_topics}")
print(f"Documenti rumore (-1): {noise_ratio * 100:.2f}%")
```

19.5 Rappresentazione Lessicale con Class-based TF-IDF

19.5.1 Dall'Assegnazione Vettoriale alle Parole Chiave

L'output del clustering HDBSCAN associa ciascun documento a un identificativo numerico di cluster $c \in \{0, 1, \ldots, K-1\}$. Per trasformare tali partizioni in topic interpretabili per l'utente umano, BERTopic aggrega tutti i documenti appartenenti al medesimo cluster c in un unico testo composito (**mega-documento**):

$$\mathcal{D}_c = \bigcup_{d \in C_c} d \tag{19.8}$$

Tuttavia, calcolare la semplice frequenza delle parole all'interno del mega-documento favorirebbe esclusivamente termini generici o verbi ausiliari frequenti nell'intera lingua. Al contempo, il TF-IDF convenzionale calcola l'IDF a livello di singolo documento, penalizzando vocaboli che compaiono in tutti i documenti di un topic compatto.

19.5.2 La Formula del c-TF-IDF

Grootendorst (2022) [41] introduce la metrica **Class-based TF-IDF** (c-TF-IDF):

$$W_{t,c} = \mathrm{tf}_{t,c} \times \log\left(1 + \frac{A}{f_t}\right) \tag{19.9}$$

dove:

- $\mathrm{tf}_{t,c}$ è la frequenza assoluta del termine t all'interno della classe/cluster c.
- $f_t = \sum_{c=1}^{K} \mathrm{tf}_{t,c}$ è la frequenza complessiva del termine t sommata attraverso tutti i K cluster dell'intero corpus.
- A è il numero medio di parole per classe:

$$A = \frac{\sum_{c=1}^{K} \sum_{t} \mathrm{tf}_{t,c}}{K} \tag{19.10}$$

Il termine logaritmico funge da penalità inversa: se una parola compare massicciamente in tutti i topic (es. *"people"*, *"time"*), $f_t \approx K \cdot \mathrm{tf}_{t,c}$, il rapporto $\frac{A}{f_t}$ decade e il peso si annulla. Se invece il termine compare quasi esclusivamente nel cluster c (es. *"orbit"*, *"spacecraft"* nel topic aerospaziale), $f_t \approx \mathrm{tf}_{t,c}$, conferendogli un punteggio discriminante massimo.

Listing 19.4: Implementazione manuale a basso livello di c-TF-IDF.

```
from sklearn.feature_extraction.text import CountVectorizer

# 1. Raggruppamento dei documenti per cluster escludendo il rumore
df_docs = pd.DataFrame({'text': docs, 'topic': cluster_labels})
valid_docs = df_docs[df_docs['topic'] != -1]
grouped = valid_docs.groupby('topic')['text'].agg(' '.join)

# 2. Matrice di conteggio cluster-termine con eliminazione stopwords
vectorizer = CountVectorizer(stop_words='english')
tf_matrix = vectorizer.fit_transform(grouped).toarray() # Shape: (K, Vocab)
vocab = np.array(vectorizer.get_feature_names_out())

# 3. Calcolo analitico di c-TF-IDF
f_t = tf_matrix.sum(axis=0) # Frequenza globale del termine
A = tf_matrix.sum() / tf_matrix.shape[0] # Lunghezza media classe
idf_class = np.log(1.0 + (A / (f_t + 1e-9)))
ctfidf_matrix = tf_matrix * idf_class

# 4. Estrazione top-5 parole chiave per il Topic 0
top_indices = np.argsort(ctfidf_matrix[0])[::-1][:5]
top_kw = list(zip(vocab[top_indices], ctfidf_matrix[0][top_indices].round(2)))
print("Top-5 keyword c-TF-IDF:", top_kw)
```

19.6 La Pipeline Completa con BERTopic

19.6.1 Configurazione e Addestramento End-to-End

Nella libreria unificata `BERTopic`, ciascuno dei passaggi esaminati viene istanziato come un componente intercambiabile collegato in pipeline:

Listing 19.5: Inizializzazione ed esecuzione della pipeline modulare BERTopic.

```
from bertopic import BERTopic
from bertopic.vectorizers import ClassTfidfTransformer

# Assemblaggio della pipeline completa
topic_model = BERTopic(
    embedding_model=encoder,
    umap_model=umap_model,
    hdbscan_model=hdbscan_model,
```

```
    vectorizer_model=CountVectorizer(stop_words='english',
        min_df=2),
    ctfidf_model=ClassTfidfTransformer(
        reduce_frequent_words=True),
    verbose=True
)

# Addestramento congiunto ed estrazione topic
topics, probs = topic_model.fit_transform(docs, embeddings=
    embeddings)

# Ispezione riassuntiva dei principali topic estratti
topic_info = topic_model.get_topic_info()
print(topic_info[['Topic', 'Count', 'Name']].head(6))
```

19.6.2 Visualizzazione Interattiva e Diagnostica Semantica

`BERTopic` include primitive grafiche interattive (basate su Plotly) per validare la coerenza della modellazione:

- **Intertopic Distance Map (`visualize_topics()`):** proietta i topic in uno spazio 2D tramite MDS multidimensionale applicato alle distanze c-TF-IDF. La dimensione del cerchio è proporzionale al numero di documenti, consentendo di verificare se temi affini (es. *Baseball* e *Hockey*) formano super-macroaree contigue.
- **Topic Word Scores (`visualize_barchart()`):** visualizza per ciascun topic un grafico a barre orizzontali indicante i pesi relativi c-TF-IDF delle prime 10 parole chiave.
- **Gerarchia Dendrogrammatica (`visualize_hierarchy()`):** illustra la sequenza agglomerativa di fusione gerarchica tra topic a livelli crescenti di distanza lessicale.
- **Matrice di Similarità Termini (`visualize_heatmap()`):** mostra la matrice di affinità coseno tra tutti i vettori c-TF-IDF.

Listing 19.6: Generazione di diagnostiche visive con BERTopic.

```
# Mappa di distanza inter-topic
fig_topics = topic_model.visualize_topics()

# Barchart delle parole chiave discriminanti
fig_bars = topic_model.visualize_barchart(top_n_topics=6)
```

```
6
7 # Albero gerarchico di correlazione tra argomenti
8 fig_hier = topic_model.visualize_hierarchy()
```

19.7 Raffinamento Avanzato: Topic Reduction e KeyBERT

19.7.1 Riduzione Automatica dei Topic Ridondanti

Quando HDBSCAN individua un numero eccessivo di micro-cluster iperspecifici, è possibile comprimere la soluzione forzando la fusione dei topic più vicini mediante il metodo `reduce_topics()`:

$$\text{topic_model.reduce_topics(docs, nr_topics=10)} \tag{19.11}$$

L'algoritmo calcola la matrice di similarità del coseno tra le rappresentazioni c-TF-IDF dei topic e fonde iterativamente le due classi a massima affinità, ricalcolando il c-TF-IDF del mega-documento unificato finché non viene raggiunto il numero desiderato di cluster.

19.7.2 Riassegnazione del Rumore (Outlier Reduction)

Per evitare di perdere i documenti classificati come rumore (etichetta -1), BERTopic implementa strategie di riassegnazione euristica (*soft clustering*):

Listing 19.7: Riassegnazione dei documenti classificati come rumore.

```
1 # Riassegnazione dei punti rumore al topic semanticamente
      piu vicino
2 new_topics = topic_model.reduce_outliers(
3     docs,
4     topics,
5     strategy="embeddings", # Riassegnazione basata su
          embedding
6     embeddings=embeddings
7 )
8 topic_model.update_topics(docs, topics=new_topics)
```

19.7.3 Keyword Extraction Avanzata con KeyBERTInspired

La pura frequenza c-TF-IDF può talvolta selezionare singole parole frammentarie. Il modulo di rappresentazione **`KeyBERTInspired`** affina le descri-

zioni estraendo candidati uni-grammi e bi-grammi dal testo e classificandoli in base alla **similarità del coseno tra l'embedding del candidato e l'embedding globale del cluster**:

Listing 19.8: Raffinamento delle parole chiave tramite KeyBERTInspired.

```
from bertopic.representation import KeyBERTInspired

# Modulo di rappresentazione basato su allineamento
    semantico di embedding
representation_model = KeyBERTInspired()

topic_model.update_topics(
    docs,
    representation_model=representation_model
)

# Ispezione delle nuove etichette descrittive raffinate
print(topic_model.get_topic(0)[:4])
```

19.8 Quadro Sinottico e Buone Pratiche Metodologiche

Tabella 19.1: Quadro sinottico dei componenti architetturali di BERTopic, parametri chiave e trade-off.

Modulo Pipeline	Libreria / Classe	Iperparametri Critici	Trade-off Operativo e Note
1. Embedding Testuale	`SentenceTransformer`	`model_name`, `batch_size`	Modelli grandi migliorano la semantica ma aumentano memoria e tempi GPU.
2. Riduzione Manifold	`UMAP`	`n_neighbors`, `n_components`, `min_dist`	`n_neighbors` basso $\Longrightarrow$ micro-cluster; alto $\Longrightarrow$ macro-strutture.
3. Density Clustering	`HDBSCAN`	`min_cluster_size`, `min_samples`	`min_cluster_size` alto riduce i topic ed espelle più outlier (-1).
4. Tokenizzazione Lessicale	`CountVectorizer`	`stop_words`, `min_df`, `ngram_range`	Rimuove il lessico comune e definisce n-grammi candidati per i topic.
5. Rappresentazione	`ClassTfidfTransformer`	`reduce_frequent_words`	Bilancia parole discriminanti rispetto alla frequenza generale del corpus.
6. Rappresentazione Avanzata	`KeyBERTInspired`	`top_n_words`, `candidates`	Allinea le parole chiave allo spazio vettoriale del cluster.

Linee Guida e Diagnostica nei Progetti Applicativi

- **Normalizzazione degli Embedding:** applicare sempre la normalizzazione euclidea a norma unitaria ($\|\mathbf{x}\| = 1$) sui vettori prodotti da SBERT; in questo modo il prodotto scalare interno coincide esattamente con la similarità del coseno.
- **Dimensionalità di UMAP per il Clustering:** evitare di ridurre direttamente a 2 dimensioni prima di HDBSCAN. La proiezione 2D provoca la compressione forzata di manifold complessi causando sovrapposizioni non veritiere. Mantenere $d = 5 \dots 10$ per HDBSCAN e usare $d = 2$ solo a fini di rendering grafico.

- **Gestione del Parametro di Rumore in HDBSCAN:** se una frazione elevata di documenti ($> 40\%$) viene etichettata come -1, ridurre `min_cluster_size` e `min_samples`, oppure invocare sistematicamente la procedura di post-processing `reduce_outliers()`.
- **Coerenza con i Requisiti di Dominio:** a differenza dell'approccio generativo a tema fisso di LDA, in BERTopic il numero di topic emerge spontaneamente dalla densità dei dati, offrendo una corrispondenza organica con i veri aggregati informativi presenti nel testo.

Capitolo 20

Classificazione: Concetti di Base e Alberi di Decisione

20.1 Introduzione e Fondamenti della Classificazione

Nell'ambito del *Data Mining* e dell'apprendimento automatico, il problema della **classificazione** rappresenta uno dei pilastri fondamentali dell'analisi predittiva [2]. A differenza dei compiti descrittivi ed esplorativi — quali l'analisi dei cluster (*cluster analysis*) o l'estrazione di regole associative (*association rule mining*), nei quali non è presente alcuna variabile obiettivo predeterminata e l'algoritmo indaga la struttura intrinseca dei dati — la classificazione si colloca a pieno titolo nel paradigma dell'**apprendimento supervisionato** (*supervised learning*).

20.1.1 Formalizzazione Matematica del Task di Classificazione

In un problema di apprendimento supervisionato, l'obiettivo è modellare la relazione predittiva esistente tra un insieme di variabili descrittive e una specifica variabile obiettivo:

- Sia $\mathcal{X}$ lo **spazio delle caratteristiche** (*feature space* o *attribute set*), definito dal prodotto cartesiano dei domini di d attributi predittivi:

$$\mathcal{X} = \mathcal{A}_1 \times \mathcal{A}_2 \times \cdots \times \mathcal{A}_d \tag{20.1}$$

  Ciascuna istanza $\mathbf{x} = (x_1, x_2, \ldots, x_d) \in \mathcal{X}$ modella un vettore di input (noto anche come predittore, variabile indipendente o covariata).

- Sia $\mathcal{Y}$ il **dominio dell'attributo target** (*class label*, risposta, variabile dipendente o output).

La natura matematica del dominio $\mathcal{Y}$ definisce la dicotomia cardine dell'apprendimento supervisionato:

1. **Classificazione**: il dominio target $\mathcal{Y}$ è un insieme **finito, discreto e categorico** di etichette mutuamente esclusive:

$$\mathcal{Y} = \{c_1, c_2, \ldots, c_K\}, \quad \text{con } K \geq 2 \tag{20.2}$$

   Se $K = 2$, il problema è denominato *classificazione binaria* (es. solvibile vs insolvente, spam vs non-spam, sano vs malato); se $K > 2$, si parla di *classificazione multiclasse* (es. tipologie cellulari, classi morfologiche di galassie).

2. **Regressione**: il dominio target $\mathcal{Y} \subseteq \mathbb{R}$ è una variabile **continua e quantitativa** (es. stima del prezzo di un immobile, previsione della temperatura o dei consumi energetici).

Definizione 20.1.1 (Compito di Classificazione)**.** Dato un insieme di osservazioni storiche etichettate, denominato **Training Set** $\mathcal{D}_{\text{train}} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N} \subset \mathcal{X} \times \mathcal{Y}$, il compito computazionale consiste nell'apprendere una funzione di mappatura (o **modello predittivo**):

$$f : \mathcal{X} \longrightarrow \mathcal{Y} \tag{20.3}$$

tale da minimizzare la discrepanza tra la classe predetta $\hat{y} = f(\mathbf{x})$ e la classe reale y.

L'obiettivo fondamentale (*goal*) non risiede nella mera memorizzazione mnemonica delle associazioni presenti in $\mathcal{D}_{\text{train}}$, bensì nella capacità di **generalizzazione**: il classificatore deve essere in grado di assegnare correttamente l'etichetta di classe a record mai osservati in precedenza (*previously unseen records*). Per verificare oggettivamente tale capacità, si impiega un secondo insieme disgiunto di dati, denominato **Test Set** $\mathcal{D}_{\text{test}} \subset \mathcal{X} \times \mathcal{Y}$, tale che $\mathcal{D}_{\text{train}} \cap \mathcal{D}_{\text{test}} = \emptyset$.

20.1.2 Il Ciclo Induzione-Deduzione

Il processo operativo di costruzione e utilizzo di un classificatore si articola in due fasi sequenziali distinte, formalizzate nello schema concettuale:

1. **Fase di Induzione (Learning / Training):** l'algoritmo di apprendimento supervisionato riceve in ingresso il *Training Set* (in cui per ogni istanza sono noti sia il vettore di attributi $\mathbf{x}$ sia l'etichetta y) ed estrae pattern, regolarità statistiche o regole logiche, costruendo la funzione astratta f (il *Modello*).

2. **Fase di Deduzione (Inference / Prediction):** il modello appreso f viene applicato ai record del *Test Set* (o a dati in produzione), per i quali sono noti esclusivamente gli attributi predittivi, deducendo e assegnando l'etichetta $\hat{y}$ sconosciuta.

La Figura 20.1 illustra graficamente l'architettura logica del ciclo induzione-deduzione.

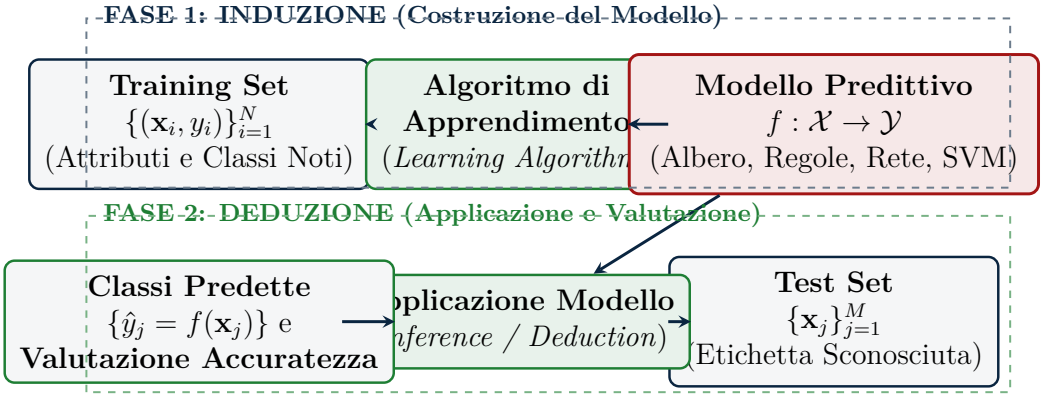


Figura 20.1: Pipeline architetturale del processo di classificazione: partizionamento logico tra la fase induttiva (costruzione del modello su dati etichettati) e la fase deduttiva (inferenza su dati non osservati).

A titolo esemplificativo, la Tabella 20.1 illustra un tipico scenario applicativo: un *Training Set* di 10 transazioni storiche con attributi eterogenei (binari, ordinali e continui) e classe nota, a fronte di un *Test Set* composto da nuove richieste in cui l'attributo target deve essere predetto.

Tabella 20.1: Esempio di partizionamento tra Training Set (record 1–10) e Test Set (record 11–15).

Partizione	Tid	Attrib1 (Binario)	Attrib2 (Ordinale)	Attrib3 (Continuo)	Classe Reale	Classe Predetta
Training Set	1	Yes	Large	125K	No	—
	2	No	Medium	100K	No	—
	3	No	Small	70K	No	—
	4	Yes	Medium	120K	No	—
	5	No	Large	95K	Yes	—
	6	No	Medium	60K	No	—
	7	Yes	Large	220K	No	—
	8	No	Small	85K	Yes	—
	9	No	Medium	75K	No	—
	10	No	Small	90K	Yes	—
Test Set	11	No	Small	55K	?	$f(\mathbf{x}_{11})$
	12	Yes	Medium	80K	?	$f(\mathbf{x}_{12})$
	13	Yes	Large	110K	?	$f(\mathbf{x}_{13})$
	14	No	Small	95K	?	$f(\mathbf{x}_{14})$
	15	No	Large	67K	?	$f(\mathbf{x}_{15})$

20.1.3 Tassonomia dei Metodi di Classificazione

Nella letteratura del Data Mining, gli algoritmi di classificazione si ripartiscono in due grandi famiglie concettuali:

1. **Classificatori di Base (*Base Classifiers*):** modelli individuali che apprendono una singola funzione predittiva:
   - **Metodi basati su Alberi di Decisione (*Decision Trees*):** modellano la decisione come una gerarchia di test ricorsivi (*Hunt, CART, ID3, C4.5*);
   - **Metodi basati su Regole (*Rule-based Methods*):** descrivono il dominio mediante insiemi di regole disgiuntive *IF-THEN* (es. RIPPER);
   - **Classificatori basati su Esempi (*Instance-based / Nearest-Neighbor*):** algoritmi pigri (*lazy learners*) come il k-NN, che non creano un modello esplicito ma cercano le istanze più vicine nello spazio metrico al momento del test;
   - **Classificatori Probabilistici e Bayesiani:** modellano la probabilità a posteriori $P(y \mid \mathbf{x})$ tramite il teorema di Bayes sotto assunzione di indipendenza condizionata (*Naïve Bayes*) o tramite reti di credenza bayesiane (*Bayesian Belief Networks*);
   - **Modelli Connessionisti e Reti Neurali:** dai percettroni multistrato (*Multi-Layer Perceptron*) alle architetture di *Deep Learning*;
   - **Support Vector Machines (SVM):** massimizzano il margine geometrico di separazione iper-planare nello spazio delle caratteristiche trasformate da un nucleo (*kernel*).

2. **Classificatori Ensemble (*Ensemble Methods*):** paradigmi meta-algoritmici che combinano strategicamente un collettivo di classificatori di base per incrementare la robustezza e ridurre la varianza o il bias dell'errore (tecniche di *Bagging*, *Boosting* e *Random Forests*).

20.2 Il Modello ad Albero di Decisione

Un **albero di decisione** (*Decision Tree*) è una struttura dati ad albero invertito orientata e aciclica utilizzata per risolvere compiti sia di classificazione che di regressione. La sua popolarità deriva dall'intrinseca interpretabilità: la logica di inferenza può essere tradotta in un insieme di regole logiche trasparenti immediatamente comprensibili dall'essere umano.

20.2.1 Definizione e Componenti Strutturali

Un albero di decisione è composto da tre tipologie fondamentali di nodi:

1. **Nodo Radice (*Root Node*):** è il nodo sommitale privo di archi entranti, dal quale ha inizio il processo decisionale per qualsiasi nuova istanza. Contiene una condizione di test su uno specifico attributo descrittivo.

2. **Nodi Interni (*Internal Nodes*):** nodi intermedi provvisti di esattamente un arco entrante e due o più archi uscenti. Ciascun nodo interno definisce una condizione di test (*splitting test*) applicata a un attributo del record.

3. **Archi Etichettati (*Branches*):** ciascun arco uscente da un nodo interno corrisponde a uno dei possibili esiti mutualmente esclusivi del test associato al nodo padre.

4. **Nodi Foglia o Terminali (*Leaf / Terminal Nodes*):** nodi privi di archi uscenti provvisti di un solo arco entrante. Ciascuna foglia rappresenta una decisione finale e reca associata un'etichetta di classe $y \in \mathcal{Y}$ (o una distribuzione di probabilità sulle classi).

20.2.2 Inferenza e Non-Unicità dell'Albero di Decisione

Per illustrare il meccanismo operativo, consideriamo il problema di accreditamento creditizio volto a prevedere l'insolvenza di un richiedente prestito (*Defaulted Borrower* $\in \{\text{No}, \text{Yes}\}$), descritto nella Tabella 20.2 [2].

Tabella 20.2: Dataset storico di addestramento per la previsione dell'insolvenza creditizia.

ID Richiesta	Home Owner (Categorico)	Marital Status (Categorico)	Annual Income (Continuo)	Defaulted Borrower (Classe)
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Esempio 20.2.1 (Percorso di Inferenza su un Record di Test)**.** Dato il record non etichettato:

$$\mathbf{x}_{\text{test}} = (\text{Home Owner} = \text{No},\ \text{Marital Status} = \text{Married},\\ \text{Annual Income} = 80\text{K})$$

la procedura deduttiva opera come segue:

1. Si valuta la radice (*Home Owner*): poiché il valore è No, si segue il relativo ramo discendente verso il nodo interno *Marital Status*;

2. Si valuta il nodo *Marital Status*: il valore del record è Married, conducendo direttamente al ramo associato;

3. Si approda a un nodo foglia etichettato con la classe Defaulted = No. Il record viene pertanto classificato come solvente.

Un principio epistemologico cardine nella teoria degli alberi di decisione è la **non-unicità del modello**: sul medesimo dataset di addestramento possono esistere molteplici alberi topologicamente distinti capaci di classificare con accuratezza perfetta (100%) tutti i record di training.

La Figura 20.2 pone a confronto due alberi alternativi perfettamente consistenti con i dati della Tabella 20.2: il primo adotta come attributo radice la proprietà dell'immobile (*Home Owner*), mentre il secondo radica la decisione sullo stato civile (*Marital Status*).

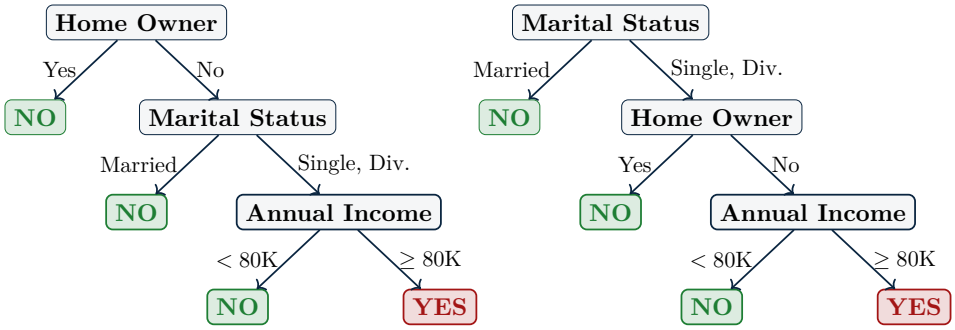


Figura 20.2: Dimostrazione di non-unicità del modello ad albero di decisione: entrambi gli alberi discriminano senza errori il training set dei 10 mutuatari, pur adottando gerarchie di test differenti.

20.3 Induzione degli Alberi: L'Algoritmo di Hunt

La costruzione manuale di un albero di decisione risulta impraticabile su dataset reali contenenti migliaia di record e centinaia di attributi. Poiché lo spazio combinatorio di tutti i possibili alberi generabili cresce in modo super-esponenziale rispetto al numero di feature, determinare l'albero globalmente ottimo è un problema formale computazionalmente intoccabile.

Teorema 20.3.1 (Complessità dell'Induzione Ottima)**.** *La ricerca dell'albero di decisione ottimale che minimizza l'errore di classificazione o la complessità strutturale (numero totale di nodi) è un problema* ***NP-hard*** *[44].*

Di conseguenza, tutti gli algoritmi pratici di induzione — tra cui l'algoritmo pioniere di Hunt [45], CART [44], ID3 [46] e C4.5 [47] — ricorrono a una **strategia euristica greedy** (ingorda), procedendo dall'alto verso il basso (*top-down*) in modalità divide-et-impera.

20.3.1 Schema Algoritmico Ricorsivo di Hunt

Sia $\mathcal{D}_t$ il sottoinsieme dei record di addestramento associati a un generico nodo t dell'albero ($\mathcal{D}_{\text{root}} = \mathcal{D}_{\text{train}}$). La procedura ricorsiva di Hunt opera secondo due regole fondamentali:

1. **Condizione di Base (Arresto per Purezza):** se tutti i record appartenenti a $\mathcal{D}_t$ appartengono alla medesima classe $y_k \in \mathcal{Y}$, allora il

nodo t è **puro** e viene trasformato in un **nodo foglia** etichettato con la classe y_k.

2. **Passo Ricorsivo (Partizionamento per Impurezza):** se $\mathcal{D}_t$ contiene record appartenenti a due o più classi distinte, il nodo t è **impuro**. L'algoritmo seleziona una condizione di test su un attributo (*attribute test condition*) per partizionare $\mathcal{D}_t$ in sottoinsiemi mutualmente disgiunti $\mathcal{D}_{t_1}, \mathcal{D}_{t_2}, \ldots, \mathcal{D}_{t_m}$. Vengono generati m nodi figli e la procedura viene richiamata ricorsivamente su ciascun figlio.

20.3.2 Traccia di Esecuzione Passo-Passo dell'Algoritmo di Hunt

Seguiamo l'esecuzione deterministica dell'algoritmo di Hunt sul dataset dei 10 mutuatari della Tabella 20.2. Indichiamo in ogni nodo la distribuzione delle classi mediante la coppia ordinata:

$$(n_{\text{No}},\ n_{\text{Yes}}) \tag{20.4}$$

- **Passo (a) — Inizializzazione alla Radice:** Al nodo radice confluiscono tutti i 10 record del dataset. La distribuzione è $(7, 3)$, con classe maggioritaria Defaulted = No. Poiché il nodo è misto (3 record insolventi su 10), non è possibile dichiararlo foglia e si rende necessario uno split.

- **Passo (b) — Primo Split su *Home Owner*:** Si seleziona il test binario Home Owner $\in \{\text{Yes}, \text{No}\}$:

  - **Ramo 'Yes'**: raccoglie i record $\{1, 4, 7\}$. Tutti e 3 presentano Defaulted = No. La distribuzione è $(3, 0)$: il nodo è **puro** e diventa una **foglia** con etichetta Defaulted = No.
  - **Ramo 'No'**: raccoglie i restanti 7 record $\{2, 3, 5, 6, 8, 9, 10\}$. La distribuzione è $(4, 3)$. Il nodo è impuro e richiede un ulteriore partizionamento ricorsivo.

- **Passo (c) — Secondo Split su *Marital Status*:** Sul sottoinsieme di record con Home Owner = No, si applica una condizione di test su *Marital Status*:

  - **Ramo 'Married'**: raccoglie i record $\{2, 6, 9\}$. Tutti e 3 presentano Defaulted = No. Distribuzione $(3, 0)$: nodo **puro**, diventa una **foglia** Defaulted = No.

- **Ramo 'Single, Divorced'**: raccoglie i 4 record $\{3, 5, 8, 10\}$. Di questi, 1 ha Defaulted = No (ID 3) e 3 hanno Defaulted = Yes (ID 5, 8, 10). Distribuzione $(1, 3)$: nodo impuro.

- **Passo (d) — Terzo Split sull'Attributo Continuo *Annual Income*:** Sui 4 record residui si valuta la soglia di reddito annuo $v = 80$K:

  - **Ramo** < 80**K**: contiene il solo record ID 3 (70K, No). Distribuzione $(1, 0)$: foglia pura Defaulted = No.
  - **Ramo** ≥ 80**K**: contiene i record $\{5, 8, 10\}$ (con redditi rispettivamente di 95K, 85K, 90K). Tutti hanno Defaulted = Yes. Distribuzione $(0, 3)$: foglia pura Defaulted = Yes.

L'algoritmo termina poiché tutti i rami periferici approdano a nodi foglia a impurità nulla.

20.4 Specificazione delle Condizioni di Test

La progettazione di un algoritmo di induzione richiede di risolvere due problemi centrali:

1. Come esprimere le condizioni di test in funzione della tipologia di attributo;

2. Come quantificare analiticamente la bontà (*goodness*) di una condizione candidata per selezionare lo split migliore.

Le condizioni di test dipendono dal dominio dell'attributo e dalla cardinalità della ramificazione (split binario a 2 vie vs split multi-via a $k > 2$ vie).

20.4.1 Attributi Nominali e Categorici

Gli attributi nominali assumono valori privi di un ordine quantitativo o naturale:

- **Split Multi-via (*Multi-way split*):** genera un arco dedicato per ciascun valore distinto presente nel dominio dell'attributo. Se l'attributo assume k modalità distinte, produce k rami figli.

- **Split Binario (*2-way split*):** partiziona le k modalità in due sottoinsiemi disgiunti e non vuoti S_1 e S_2 tali che $S_1 \cup S_2 = \text{Dom}(A)$. Il numero totale di partizioni binarie possibili è combinatoriamente pari a:

$$2^{k-1} - 1 \tag{20.5}$$

  Ad esempio, per l'attributo Marital Status $\in \{\text{Single}, \text{Divorced}, \text{Married}\}$ ($k = 3$), esistono $2^{3-1} - 1 = 3$ configurazioni binarie disgiunte ammissibili:

  1. $\{\text{Married}\}$ contro $\{\text{Single}, \text{Divorced}\}$;
  2. $\{\text{Single}\}$ contro $\{\text{Married}, \text{Divorced}\}$;
  3. $\{\text{Divorced}\}$ contro $\{\text{Single}, \text{Married}\}$.

20.4.2 Attributi Ordinali e Preservazione dell'Ordine

Gli attributi ordinali possiedono un ordinamento totale ben definito (es. taglia di maglietta: Small < Medium < Large < Extra Large). Sebbene sia sempre possibile effettuare uno split multi-via generando un ramo per ciascuna taglia, nella configurazione di uno split binario sorge un vincolo teorico inderogabile:

Teorema 20.4.1 (Vincolo di Preservazione dell'Ordine per Attributi Ordinali)**.** *Nello split binario di un attributo ordinale, la partizione deve preservare la continuità dell'ordine intrinseco. I raggruppamenti devono essere formati da elementi tra loro contigui nell'ordinamento totale.*

Nel caso di quattro taglie ordinate (S < M < L < XL):

- **Raggruppamenti Ammissibili:** $\{\text{S}\}$ vs $\{\text{M}, \text{L}, \text{XL}\}$, oppure $\{\text{S}, \text{M}\}$ vs $\{\text{L}, \text{XL}\}$, oppure $\{\text{S}, \text{M}, \text{L}\}$ vs $\{\text{XL}\}$.
- **Raggruppamento Inammissibile:** $\{\text{S}, \text{L}\}$ vs $\{\text{M}, \text{XL}\}$. Questa partizione viola la relazione d'ordine frammentando arbitrariamente la semantica dell'attributo.

20.4.3 Attributi Continui e Ricerca Efficiente della Soglia Ottima

Per gli attributi numerici a valori continui $A \in \mathbb{R}$, le condizioni di test possono essere formulate come:

1. **Discretizzazione preventiva**: suddivisione del dominio in intervalli discreti (*equal-width*, *equal-frequency* o tramite cluster analysis), applicata in modalità statica (una tantum all'inizio) o dinamica (ripetuta localmente in ciascun nodo);

2. **Test di Soglia Binario**: confronto del tipo $A < v$ contro $A \geq v$, dove $v \in \mathbb{R}$ rappresenta il punto di taglio (*split position*).

Inefficienza della Ricerca Ingenua e Algoritmo Ottimizzato

Se per ciascuna delle N istanze si provasse come soglia v il valore del record scandendo l'intero dataset per calcolare i conteggi di classe, il costo computazionale risulterebbe quadratico:

$$\mathcal{O}(N^2) \tag{20.6}$$

Tale approccio è insostenibile per dataset reali. L'algoritmo efficiente riduce la complessità a $\mathcal{O}(N \log N)$ mediante un procedimento in tre fasi:

1. **Fase di Ordinamento ($\mathcal{O}(N \log N)$):** si ordinano gli N record in base ai valori crescenti dell'attributo continuo considerato;

2. **Identificazione dei Punti di Taglio Candidati ($\mathcal{O}(N)$):** si selezionano come soglie candidate v_i i punti medi tra coppie di valori consecutivi associati a classi differenti (oltre ai punti estremi);

3. **Scansione Lineare con Aggiornamento Incrementale ($\mathcal{O}(N)$):** scorrendo linearmente i dati ordinati, la matrice dei conteggi di classe per le due partizioni ($\leq v$ e $> v$) viene aggiornata incrementalmente in tempo $\mathcal{O}(1)$ a ogni passo, calcolando all'istante l'indice di impurità.

La Tabella 20.3 illustra la traccia di calcolo completa sul dataset dei 10 mutuatari per l'attributo *Annual Income*.

Tabella 20.3: Calcolo incrementale dell'indice di Gini sulle soglie candidate per l'attributo Annual Income.

Valori Ordinati	(60, No), (70, No), (75, No), (85, Yes), (90, Yes), (95, Yes), (100, No), (120, No), (125, No), (220, No)										
Soglia Candidata (v)	**55**	**65**	**72**	**80**	**87**	**92**	**97**	**110**	**122**	**172**	**230**
$\leq v$ (Defaulted = Yes)	0	0	0	0	1	2	**3**	3	3	3	3
$\leq v$ (Defaulted = No)	0	1	2	3	3	3	**3**	4	5	6	7
$> v$ (Defaulted = Yes)	3	3	3	3	2	1	**0**	0	0	0	0
$> v$ (Defaulted = No)	7	6	5	4	4	4	**4**	3	2	1	0
$GINI_{\text{split}}$	0.420	0.400	0.375	0.343	0.417	0.400	**0.300**	0.343	0.375	0.400	0.420

Dall'ispezione della Tabella 20.3 emerge con evidenza che il minimo valore di impurità ($GINI_{\text{split}} = 0.300$) viene raggiunto alla soglia $v = 97$, che isola perfettamente tutti e tre i casi positivi di insolvenza nel sottoinsieme ≤ 97K.

20.5 Misure di Impurità e Selezione dello Split Ottimale

Per selezionare in modo formale la migliore condizione di test tra tutti i candidati ammissibili, l'algoritmo di induzione adotta il **principio di massima purezza**: si predilige la condizione che produce partizioni figlie quanto più omogenee possibili rispetto alla distribuzione delle classi target.

20.5.1 Formulazione Generale delle Misure di Impurità

Sia t un generico nodo dell'albero e sia $p(j \mid t)$ la frazione relativa di record appartenenti alla classe $j \in \{1, \ldots, n_c\}$ presenti in t:

$$p(j \mid t) = \frac{|\{\mathbf{x} \in \mathcal{D}_t \mid y = c_j\}|}{|\mathcal{D}_t|}, \quad \text{con} \sum_{j=1}^{n_c} p(j \mid t) = 1 \tag{20.7}$$

Le tre funzioni matematiche fondamentali impiegate in letteratura sono:

1. **Indice di Gini (*Gini Index*):**

$$GINI(t) = 1 - \sum_{j=1}^{n_c} [p(j \mid t)]^2 \tag{20.8}$$

Nel caso binario ($n_c = 2$, con probabilità di classe p e $1 - p$), la formula si semplifica analiticamente:

$$\begin{aligned} GINI(t) &= 1 - p^2 - (1-p)^2 \\ &= 1 - p^2 - (1 - 2p + p^2) \\ &= 2p - 2p^2 = 2p(1-p) \end{aligned} \tag{20.9}$$

L'indice di Gini assume valore minimo pari a 0.0 quando il nodo è puro ($p = 0$ o $p = 1$), e valore massimo pari a $\left(1 - \frac{1}{n_c}\right)$ (ossia 0.5 per problemi binari) quando le classi sono equidistribuite.

2. **Entropia di Shannon (*Entropy*):**

$$Entropy(t) = -\sum_{j=1}^{n_c} p(j \mid t) \log_2 p(j \mid t) \tag{20.10}$$

con la convenzione analitica $0 \log_2 0 = 0$. L'entropia misura il grado di incertezza o disordine informativo presente nel nodo. Varia tra un minimo di 0.0 (massima purezza) e un massimo di $\log_2(n_c)$ (massima incertezza; pari a 1.0 bit nel caso binario con $p = 0.5$).

3. **Errore di Classificazione (*Misclassification Error*):**

$$Error(t) = 1 - \max_{1 \le j \le n_c} p(j \mid t) \tag{20.11}$$

Rappresenta la frazione di record che verrebbero classificati erroneamente qualora al nodo venisse assegnata la classe maggioritaria. Varia tra 0.0 (nodo puro) e $\left(1 - \frac{1}{n_c}\right)$ (0.5 nel caso binario).

20.5.2 Valutazione dello Split e Criterio del Massimo Guadagno

Quando un nodo genitore p contenente n record viene suddiviso da una condizione di test in k nodi figli $\{v_1, \ldots, v_k\}$, ciascuno contenente n_i record ($\sum_{i=1}^{k} n_i = n$), l'impurità complessiva post-split è calcolata come la **media ponderata** delle impurità dei singoli figli:

$$I_{\text{split}} = \sum_{i=1}^{k} \frac{n_i}{n} I(v_i) \tag{20.12}$$

dove $I(\cdot)$ può essere indifferentemente l'indice di Gini, l'Entropia o l'Errore di classificazione.

Il **Guadagno di Purezza** (*Gain*) ottenuto dallo split è definito come la differenza tra l'impurità del nodo padre prima dello split e l'impurità pesata dei nodi figli:

$$\Delta = Gain = I(p) - I_{\text{split}} = I(p) - \sum_{i=1}^{k} \frac{n_i}{n} I(v_i) \tag{20.13}$$

Poiché l'impurità del genitore $I(p)$ è invariante rispetto alle diverse condizioni di test candidate, massimizzare il guadagno Δ equivale esattamente a **minimizzare l'impurità pesata I_{split}.**

Esempio 20.5.1 (Calcolo del Guadagno di Gini su Split Binario)**.** Si consideri un nodo padre con $n = 12$ record distribuito come $(C_1 : 7,\ C_2 : 5)$. L'indice di Gini del genitore è:

$$GINI(p) = 1 - \left(\frac{7}{12}\right)^2 - \left(\frac{5}{12}\right)^2 = 1 - \frac{49}{144} - \frac{25}{144} = \frac{70}{144} \approx 0.4861 \tag{20.14}$$

Uno split binario genera due figli:

- Figlio N_1 ($n_1 = 6$, distribuzione $C_1 : 5, C_2 : 1$):

$$GINI(N_1) = 1 - \left(\frac{5}{6}\right)^2 - \left(\frac{1}{6}\right)^2 = 1 - \frac{25}{36} - \frac{1}{36} = \frac{10}{36} \approx 0.2778 \quad (20.15)$$

- Figlio N_2 ($n_2 = 6$, distribuzione $C_1 : 2, C_2 : 4$):

$$GINI(N_2) = 1 - \left(\frac{2}{6}\right)^2 - \left(\frac{4}{6}\right)^2 = 1 - \frac{4}{36} - \frac{16}{36} = \frac{16}{36} \approx 0.4444 \quad (20.16)$$

L'impurità complessiva post-split e il relativo guadagno risultano:

$$GINI_{\text{split}} = \frac{6}{12}(0.2778) + \frac{6}{12}(0.4444) = 0.1389 + 0.2222 = 0.3611 \quad (20.17)$$

$$Gain = 0.4861 - 0.3611 = 0.1250 \quad (20.18)$$

20.5.3 Il Problema del Sovra-Partizionamento e il Gain Ratio

L'applicazione standard dell'**Information Gain** basato sull'Entropia (adottato originariamente da ID3 [46]):

$$GAIN_{\text{split}} = Entropy(p) - \sum_{i=1}^{k} \frac{n_i}{n} Entropy(v_i) \quad (20.19)$$

presenta un gravissimo difetto strutturale: **privilegia sistematicamente attributi dotati di un numero elevato di valori distinti**.

Il Paradosso dell'Attributo Identificativo (*Customer ID*)

Si consideri un dataset di $N = 20$ clienti con attributo di classe binario bilanciato ($C_0 : 10, C_1 : 10$), la cui entropia del genitore è $Entropy(p) = -\frac{1}{2}\log_2\frac{1}{2} - \frac{1}{2}\log_2\frac{1}{2} = 1.0$. Se si effettua lo split sull'attributo Customer ID, l'albero genera $k = 20$ rami figli $c_1, \ldots, c_{20}$, ciascuno contenente esattamente un singolo record:

- Per ogni figlio i, essendovi un solo elemento, la frequenza relativa della sua classe è 1.0, da cui $Entropy(c_i) = -1\log_2 1 = 0$;
- L'entropia post-split collassa a zero: $\sum_{i=1}^{20} \frac{1}{20}(0) = 0$;
- Il guadagno informativo è massimale: $GAIN = 1.0 - 0 = 1.0$.

Tuttavia, un simile modello è totalmente privo di capacità predittiva: ogni futuro cliente nel test set avrà un identificativo mai osservato prima, rendendo l'albero incapace di generalizzare.

Soluzioni Architetturali: CART vs C4.5

Per contrastare questa patologia si adottano due differenti strategie:

1. **Vincolo Binario Stretto (CART):** impone a priori che ogni split generi esattamente due figli ($k = 2$), eliminando alla radice l'esplosione delle ramificazioni.

2. **Penalizzazione dell'Entropia della Partizione (*Gain Ratio* in C4.5):** Quinlan [47] ha introdotto una normalizzazione del guadagno informativo mediante la misura **Split Information** ($SplitINFO$), che quantifica l'entropia intrinseca della suddivisione dei record:

$$SplitINFO = -\sum_{i=1}^{k} \frac{n_i}{n} \log_2 \left(\frac{n_i}{n} \right) \tag{20.20}$$

   Il criterio di selezione ottimale diventa il **Gain Ratio**:

$$GainRATIO_{\text{split}} = \frac{GAIN_{\text{split}}}{SplitINFO} \tag{20.21}$$

Se uno split genera un numero elevato di frammenti di piccolissime dimensioni, il valore di $SplitINFO$ cresce proporzionalmente a $\log_2(k)$ (nel caso di $CustomerID$, $SplitINFO = -\sum_{i=1}^{20} \frac{1}{20} \log_2(1/20) = \log_2(20) \approx 4.32$ bit), penalizzando drasticamente il rapporto finale e scartando la condizione di test degenerata.

20.5.4 Confronto Teorico e Sensibilità delle Misure di Impurità

La Figura 20.3 mette a confronto il comportamento geometrico delle tre misure al variare della proporzione $p \in [0, 1]$ della classe C_1 in un contesto binario ($n_c = 2$). Per consentire un confronto visivo diretto sulla medesima scala, l'entropia viene normalizzata dividendola per due ($0.5 \times Entropy$).

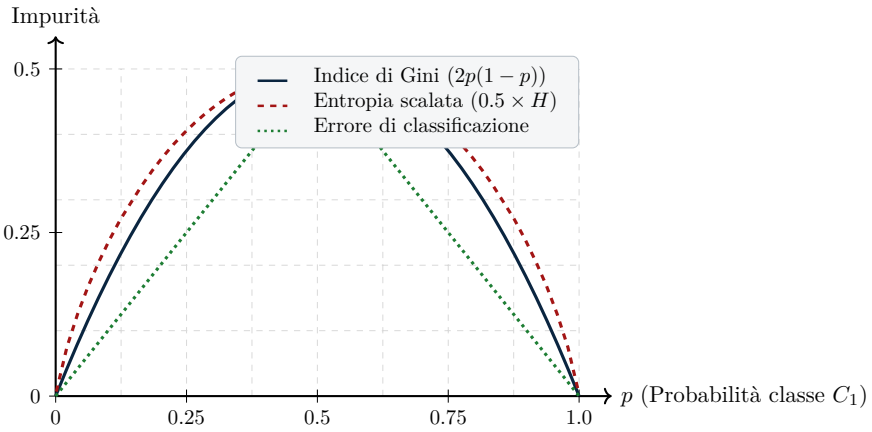


Figura 20.3: Confronto geometrico tra le funzioni di impurità: Entropia scalata, Indice di Gini ed Errore di Classificazione nel caso binario. Si noti il profilo triangolare non differenziabile dell'errore di classificazione in $p = 0.5$ a fronte della concavità stretta di Gini ed Entropia.

Dall'analisi teorica emergono considerazioni cruciali:

- **Consistenza d'ordine locale:** se un nodo A possiede un'entropia inferiore a quella di un nodo B, allora anche il suo indice di Gini e il suo errore di classificazione risulteranno inferiori a quelli di B.
- **Discrepanza nella selezione globale dello split:** pur essendo tra loro concordi sulla purezza di un singolo nodo, le tre funzioni possono identificare attributi differenti come migliori candidati allo split a causa della diversa ponderazione convessa sui nodi figli.
- **Minore Sensibilità dell'Errore di Classificazione:** a causa della sua natura lineare a tratti e della presenza di un punto angoloso non differenziabile in $p = 0.5$, l'errore di classificazione non riesce frequentemente a discriminare tra configurazioni di split radicalmente differenti.

Esempio 20.5.2 (Dimostrazione Formale della Cecità dell'Errore di Classificazione)**.** Si consideri un nodo padre p contenente 10 record con distribuzione $(C_1 : 7, C_2 : 3)$. L'errore di classificazione e l'indice di Gini del genitore sono:

$$Error(p) = 1 - \frac{7}{10} = 0.300, \qquad GINI(p) = 1 - (0.7)^2 - (0.3)^2 = 0.420 \tag{20.22}$$

Confrontiamo due possibili split binari concorrenti:

1. **Split A**: genera i figli $N_1(C_1 : 3, C_2 : 0)$ con $n_1 = 3$ e $N_2(C_1 : 4, C_2 : 3)$ con $n_2 = 7$.

$$Error_{\text{split}} = \frac{3}{10}\left(1 - \frac{3}{3}\right) + \frac{7}{10}\left(1 - \frac{4}{7}\right) = 0.300 \tag{20.23}$$

$$GINI_{\text{split}} = \frac{3}{10}(0) + \frac{7}{10}\left(1 - \frac{16}{49} - \frac{9}{49}\right) \approx 0.3429 \tag{20.24}$$

2. **Split B**: genera i figli $N_3(C_1 : 3, C_2 : 1)$ con $n_3 = 4$ e $N_4(C_1 : 4, C_2 : 2)$ con $n_4 = 6$.

$$Error_{\text{split}} = \frac{4}{10}\left(1 - \frac{3}{4}\right) + \frac{6}{10}\left(1 - \frac{4}{6}\right) = 0.300 \tag{20.25}$$

$$\begin{aligned} GINI_{\text{split}} &= \frac{4}{10}\left(1 - \frac{9}{16} - \frac{1}{16}\right) + \frac{6}{10}\left(1 - \frac{16}{36} - \frac{4}{36}\right) \\ &\approx \frac{4}{10}(0.375) + \frac{6}{10}(0.4444) \approx 0.4167 \end{aligned} \tag{20.26}$$

Conclusione: dal punto di vista dell'errore di classificazione, entrambi gli split presentano lo stesso valore 0.300 (pari a quello del genitore, con guadagno nullo $Gain = 0$). L'errore di classificazione non rileva alcun vantaggio. Al contrario, l'indice di Gini premia nettamente lo Split A ($0.3429 < 0.4167$), riconoscendo che esso isola un sottoinsieme completamente puro (N_1). Per questa ragione, algoritmi come CART e C4.5 rigettano l'errore di classificazione come criterio di crescita dell'albero, preferendo funzioni strettamente concave come Gini ed Entropia.

La Tabella 20.4 riassume sinotticamente le proprietà matematiche delle metriche introdotte.

Tabella 20.4: Confronto sinottico tra le misure di impurità per l'induzione di alberi di decisione.

Caratteristica	Indice di Gini	Entropia di Shannon	Gain Ratio	Classification Error
Formulazione	$1 - \sum_j p_j^2$	$-\sum_j p_j \log_2 p_j$	$\frac{Gain}{SplitINFO}$	$1 - \max_j p_j$
Valore Minimo	0.0 (Massima purezza)	0.0 (Massima purezza)	0.0 (Nessun guadagno)	0.0 (Massima purezza)
Valore Massimo	$1 - 1/n_c$ (0.5 binario)	$\log_2(n_c)$ (1.0 binario)	Variabile	$1 - 1/n_c$ (0.5 binario)
Proprietà Geometriche	Strettamente concava, liscia	Strettamente concava, liscia	Rapporto di entropie	Concava, non derivabile in 0.5
Sensibilità alle Purezze	Molto elevata	Molto elevata	Molto elevata	Debole (gradini costanti)
Algoritmi Tipici	CART, SLIQ, SPRINT	ID3	C4.5, C5.0	Usato per pruning/eval

Capitolo 21

Alberi di Decisione Avanzati: Complessità, Valori Mancanti, Overfitting e Potatura

21.1 Introduzione e Motivazione

Sebbene l'induzione greedy di alberi di decisione mediante algoritmi ricorsivi (esaminata nel capitolo precedente) offra un'elevata efficienza computazionale e una naturale interpretabilità, la transizione dai modelli giocattolo ai dataset reali impone di affrontare complesse sfide metodologiche [2]. Tra queste spiccano la gestione di registrazioni incomplete affette da **valori mancanti** (*missing values*), i limiti intrinseci imposti da condizioni di test ortogonali agli assi, e soprattutto la tendenza fisiologica del modello a cadere nella trappola del **sovra-adattamento** (*overfitting*).

In questo capitolo vengono formalizzati i fondamenti teorici e computazionali per analizzare la complessità degli alberi, il trattamento rigoroso dei dati mancanti, le cause strutturali del degrado predittivo e le tecniche matematiche di **selezione del modello** e **potatura** (*pruning*) atte a garantire un'efficace capacità di generalizzazione statistica.

21.2 Criteri di Arresto, Vantaggi e Complessità Computazionale

L'espansione top-down di un albero di decisione non può proseguire indefinitamente fino alla completa enumerazione dello spazio dei record, pena la generazione di modelli degeneri e sovra-dimensionati.

21.2.1 Criteri di Arresto dell'Induzione

Nel paradigma standard dell'algoritmo di Hunt, la ricorsione su un generico nodo t cessa al verificarsi di una delle seguenti condizioni canoniche [2, 47]:

1. **Omogeneità di Classe (Purezza Assoluta):** tutti i record di addestramento $\mathcal{D}_t$ appartengono alla medesima classe $y \in \mathcal{Y}$ (ovvero l'entropia e l'indice di Gini sono identicamente nulli). Il nodo diviene una foglia etichettata con tale classe.

2. **Omogeneità di Attributi (Identità Descrittiva):** tutti i record in $\mathcal{D}_t$ presentano vettori di caratteristiche identici ($\mathbf{x}_i = \mathbf{x}_j$ per ogni i, j), ma appartengono a classi tra loro discordanti (dati contraddittori o rumore di etichettatura). Essendo impossibile applicare alcun ulteriore test per discriminare i campioni, il nodo viene forzatamente convertito in una foglia etichettata con la **classe maggioritaria** presente in $\mathcal{D}_t$.

3. **Criteri di Arresto Anticipato (*Early Termination*):** l'espansione viene interrotta prima del raggiungimento della purezza sulla base di soglie statistiche o dimensionali (ad esempio se la numerosità del nodo $|\mathcal{D}_t|$ scende sotto una soglia minima $N_{\min}$, o se il guadagno Δ risulta statisticamente non significativo).

21.2.2 Vantaggi Operativi e Gestione di Feature Ridondanti

Gli alberi di decisione godono di un ampio utilizzo nelle applicazioni industriali e scientifiche grazie a sette proprietà strutturali distintive:

- **Interpretabilità Simbolica:** la struttura ad albero può essere tradotta in un insieme compatto di regole logiche proposizionali (*IF-THEN*) pienamente verificabili nel dominio applicativo;

- **Accuratezza Competitiva:** esibiscono prestazioni predittive comparabili a modelli complessi su dataset strutturati e tabellari di moderata dimensionalità;

- **Robustezza al Rumore:** l'integrazione di strategie di regolarizzazione impedisce al modello di farsi guidare da fluttuazioni spurie;

- **Immunità Naturale da Attributi Ridondanti:** un attributo si dice ridondante se è fortemente correlato con un'altra caratteristica già presente. Poiché due attributi fortemente correlati forniscono guadagni

di impurità pressoché identici, l'euristica greedy ne seleziona uno per lo split primario, ignorando l'altro nei nodi successivi dello stesso percorso;

- **Tolleranza verso Attributi Irrilevanti:** attributi privi di associazione statistica con la classe target generano guadagni vicini a zero e vengono scartati durante la selezione dello split (salvo rari casi di artefatti stocastici ad elevata dimensionalità, mitigabili con preventiva *feature selection*);

- **Rapidità nell'Inferenza:** la classificazione di nuovi dati richiede un numero di confronti proporzionale esclusivamente alla profondità dell'albero;

- **Trattamento Flessibile dei Dati Mancanti:** consentono di gestire valori mancanti sia in addestramento che in fase di scoring senza richiedere imputazioni preventive arbitrarie.

21.2.3 Analisi Asintotica della Complessità Computazionale

Consideriamo un dataset composto da N istanze di addestramento descritte da M attributi:

- **Complessità di Addestramento (*Tree Induction*):** A ciascun livello dell'albero, tutti gli M attributi devono essere valutati per identificare la condizione di split ottimale. Come dimostrato per gli attributi continui, la ricerca richiede preventivamente l'ordinamento delle istanze con costo $\mathcal{O}(N \log N)$ e una scansione lineare $\mathcal{O}(N)$. Se l'albero risultante è bilanciato, la sua profondità è $\mathcal{O}(\log N)$, comportando una complessità complessiva di costruzione pari a:

$$\mathcal{T}_{\text{build}} = \mathcal{O}(M \cdot N \log N) \tag{21.1}$$

  Nel caso pessimistico degenerato in cui l'albero sia completamente sbilanciato (profondità lineare $\mathcal{O}(N)$), la complessità peggiora asintoticamente a $\mathcal{O}(M \cdot N^2)$.

- **Complessità di Classificazione (*Testing*):** Per classificare una nuova osservazione, l'algoritmo discende lungo un singolo cammino dalla radice a una foglia. Il costo computazionale è strettamente limitato dall'altezza massima dell'albero w:

$$\mathcal{T}_{\text{test}} = \mathcal{O}(w), \quad \text{con } \log_2 N \leq w \leq N \tag{21.2}$$

Ciò garantisce un tempo di risposta nell'ordine dei microsecondi, rendendo gli alberi particolarmente adatti a sistemi predittivi in tempo reale.

21.2.4 Tassonomia degli Algoritmi di Induzione

I principali algoritmi sviluppati nella letteratura del machine learning si distinguono per il criterio di split, la topologia dell'albero e le strategie di potatura:

- **ID3 (Iterative Dichotomiser 3) [46]:** pioniere sviluppato da Ross Quinlan, implementa l'algoritmo di Hunt impiegando l'Information Gain. È limitato a soli attributi categorici discreti, genera split multi-via e non supporta la gestione nativa di dati mancanti né la potatura.
- **C4.5 [47]:** evoluzione storica di ID3 ad opera dello stesso autore. Introduce il Gain Ratio per mitigare il sovra-partizionamento, supporta attributi continui mediante soglie dinamiche, gestisce valori mancanti tramite pesi probabilistici e integra la potatura a posteriori (*post-pruning*). Tuttavia, richiede la completa allocazione del dataset in memoria RAM, limitandone la scalabilità su archivi massivi.
- **C5.0:** evoluzione commerciale proprietaria di C4.5, caratterizzata da consumi di memoria ridotti, multithreading nativo e supporto a modelli ensemble di tipo boosting.
- **CART (Classification and Regression Trees) [44]:** introdotto da Breiman et al., costruisce esclusivamente **alberi strettamente binari** ($k = 2$). Utilizza l'indice di Gini per compiti di classificazione e la varianza (o deviazione minima) per la regressione; adotta la potatura basata sul costo-complessità (*minimal cost-complexity pruning*) e gestisce dati mancanti tramite split surrogati.

21.3 Trattamento Rigoroso dei Valori Mancanti (*Missing Values*)

Nei contesti applicativi reali, i record presentano frequentemente attributi non valorizzati a causa di guasti di sensori, omissioni volontarie o mancata raccolta del dato. Negli alberi di decisione, la presenza di valori mancanti influenza l'algoritmo in **tre fasi distinte**:

1. Nel calcolo delle misure di impurità durante la determinazione della condizione di split;

2. Nella ripartizione dei record di addestramento verso i nodi figli;

3. Nella classificazione di un record di test non etichettato che presenta attributi sconosciuti.

21.3.1 Strategie per la Ripartizione delle Istanze Incomplete

Consideriamo un record con attributo ignoto:

$$\mathbf{x} = (\text{Home Owner} =?,\ \text{Marital Status} = \text{Single},\ \text{Income} = 79\text{K})$$

che raggiunge un nodo interno con test sull'attributo Home Owner. Si delineano tre strategie:

1. **Split Probabilistico (*Probabilistic Split Method*):** adottato da C4.5 [47]. L'istanza viene inviata simultaneamente a **tutti i rami figli**, ma ad essa viene associato un **peso frazionario** pari alla probabilità a priori stimata sui dati noti del nodo.

2. **Split Surrogato (*Surrogate Split Method*):** adottato da CART [44]. L'algoritmo calcola una gerarchia di attributi *surrogati*, ovvero variabili descrittive alternative che massimizzano la correlazione empirica con l'attributo primario. Se il valore primario è mancante, la decisione di instradamento viene delegata al miglior attributo surrogato disponibile.

3. **Metodo della Categoria Separata (*Separate Class Method*):** il valore nullo '?' viene trattato analiticamente come un valore discreto valido a sé stante, istanziando un ramo dedicato 'Missing' nell'albero.

21.3.2 Simulazione Completa dell'Approccio Probabilistico di C4.5

Illustriamo in dettaglio il formalismo matematico dell'approccio probabilistico di C4.5 attraverso una traccia numerica passo-passo.

1. Calcolo dell'Impurità con Dati Incompleti nel Training Set

Si consideri un dataset di $N = 10$ record in cui 7 hanno Class = No e 3 hanno Class = Yes. Il decimo record presenta il valore mancante Refund =? con Class = Yes. L'entropia del nodo genitore calcolata su tutti i 10 record è:

$$Entropy(p) = -0.3\log_2(0.3) - 0.7\log_2(0.7) \approx 0.8813 \text{ bit} \tag{21.3}$$

Per valutare lo split su Refund, si esaminano i 9 record con valore noto (3 'Yes' e 6 'No'):

- Refund = Yes (3 record: tutti Class = No): $Entropy(\text{Yes}) = 0$;
- Refund = No (6 record: 2 Class = Yes, 4 Class = No):

$$Entropy(\text{No}) = -\frac{2}{6}\log_2\left(\frac{2}{6}\right) - \frac{4}{6}\log_2\left(\frac{4}{6}\right) \approx 0.9183 \text{ bit} \tag{21.4}$$

L'entropia pesata dei figli viene calcolata ponderando esclusivamente sulle proporzioni dei record con attributo noto rispetto all'insieme totale dei 10 record ($3/10 = 0.3$ e $6/10 = 0.6$):

$$Entropy(\text{Children}) = 0.3 \cdot (0) + 0.6 \cdot (0.9183) = 0.5510 \tag{21.5}$$

Il guadagno informativo effettivo risulta quindi:

$$GAIN = 0.8813 - 0.5510 = 0.3303 \text{ bit} \tag{21.6}$$

2. Assegnazione Frazionaria dei Pesi nel Training

Tra i record con valore noto di Refund, la proporzione empirica è $\frac{3}{9}$ per 'Yes' e $\frac{6}{9}$ per 'No'. Il decimo record (Refund =?, Class = Yes) viene sdoppiato e propagato a entrambi i rami:

- Nel ramo sinistro (Refund = Yes), riceve un peso frazionario $w_L = 3/9 \approx 0.333$. I conteggi del figlio sinistro divengono: Class = Yes : $0 + 0.333 = 0.333$, Class = No : 3.0.
- Nel ramo destro (Refund = No), riceve un peso frazionario $w_R = 6/9 \approx 0.667$. I conteggi del figlio destro divengono: Class = Yes : $2 + 0.667 = 2.667$, Class = No : 4.0.

3. Deduzione su un Record di Test Incompleto

Si consideri il record di test:

$$\mathbf{x}_{\text{test}} = (\text{Refund} = \text{No},\ \text{Marital Status} = ?,\ \text{Taxable Income} = 85\text{K})$$

Avendo Refund = No, il record discende verso il nodo interno Marital Status. In tale nodo, la distribuzione pesata dei record di training per ciascuna categoria è riassunta nella Tabella 21.1.

Tabella 21.1: Distribuzione pesata delle classi nel nodo Marital Status.

Classe Target	**Married**	**Single**	**Divorced**	**Totale**
Defaulted = No	3.00	1.00	0.00	**4.00**
Defaulted = Yes	$6/9 \approx 0.67$	1.00	1.00	**2.67**
Totale Pesi	**3.67**	**2.00**	**1.00**	**6.67**

Poiché nel record di test Marital Status =?, la previsione viene ripartita probabilisticamente:

- Il ramo Married (che conduce direttamente alla foglia terminale 'NO') detiene una probabilità a priori pari a:

$$P(\text{Married} \mid \text{Node}) = \frac{3.67}{6.67} \approx 0.550 \tag{21.7}$$

- Il ramo $\{\text{Single}, \text{Divorced}\}$ (che prosegue verso il test Taxable Income > 80K approdando alla foglia 'YES') detiene una probabilità cumulata di:

$$P(\{\text{Single}, \text{Divorced}\} \mid \text{Node}) = \frac{2.00 + 1.00}{6.67} = \frac{3.00}{6.67} \approx 0.450 \tag{21.8}$$

La classe predetta finale sarà Defaulted = No, in quanto associata alla probabilità complessiva maggioritaria ($0.550 > 0.450$).

21.4 Limiti Strutturali degli Alberi di Decisione

Nonostante l'efficacia algoritmica, l'architettura standard degli alberi di decisione manifesta severe limitazioni espressive generate dall'impiego congiunto di euristiche greedy e condizioni di test uni-variate.

21.4.1 Interazioni tra Attributi e il Problema XOR

Due o più attributi si dicono **interagenti** (*interacting attributes*) qualora manifestino un'elevata capacità discriminante solo se considerati congiuntamente, mentre risultano individualmente scorrelati o non informativi rispetto alla classe target.

Esempio 21.4.1 (Il Caso XOR Bidimensionale)**.** Si consideri un dataset sintetico nel piano $[0, 20]^2$ contenente 1000 punti della classe '+' e 1000 punti della classe 'o'. I punti positivi sono concentrati nei quadranti superiore sinistro ($X \leq 10, Y > 10$) e inferiore destro ($X > 10, Y \leq 10$), mentre i punti negativi risiedono nei quadranti opposti:

$$\text{Classe} = \begin{cases} +, & \text{se } (X \leq 10 \land Y > 10) \lor (X > 10 \land Y \leq 10) \\ o, & \text{altrimenti} \end{cases} \tag{21.9}$$

Se si valuta ciascun attributo singolarmente alla radice:

- Proiettando i dati lungo il solo asse X, sia per $X \leq 10$ sia per $X > 10$ si osserva un bilanciamento esatto: 500 punti per la classe '+' e 500 punti per la classe 'o'. L'entropia post-split risulta $\mathit{Entropy}(X) = 0.999 \approx 1.0$, con un guadagno informativo nullo.
- Analogamente, lungo l'asse Y, la distribuzione è perfettamente simmetrica: $\mathit{Entropy}(Y) \approx 1.0$, guadagno nullo.

Vulnerabilità al Rumore: se si introduce nel dataset una terza variabile casuale fittizia $Z \sim \mathcal{U}(0, 20)$ puramente rumorosa, a causa di oscillazioni stocastiche campionarie potrà risultare $\mathit{Entropy}(Z) = 0.98 < 0.999$. L'algoritmo greedy sceglierà paradossalmente la variabile di puro rumore Z per effettuare il primo split, allontanandosi dalla corretta interpretazione fisica del fenomeno.

21.4.2 Geometria delle Frontiere di Decisione ed Alberi Obliqui

Nei modelli standard (CART, C4.5), ogni nodo interno valuta un singolo attributo scalare rispetto a una costante ($x_j < v$). Di conseguenza:

$$\text{Frontiera di Decisione} \perp \text{Asse } x_j \tag{21.10}$$

Tutte le superfici di separazione nello spazio delle feature sono rigorosamente **parallele agli assi coordinati**, partizionando $\mathcal{X}$ in una composizione di iper-rettangoli ortogonali.

Qualora le classi reali siano separate da una frontiera lineare inclinata o continua (ad esempio $x_1 + x_2 = \theta$), l'albero standard è costretto ad approssimare la diagonale mediante una complessa struttura a gradini (*staircase approximation*). Tale fenomeno genera un'esplosione incontrollata del numero di nodi e una grave frammentazione dei campioni.

Gli **Alberi di Decisione Obliqui** (*Oblique Decision Trees*) superano questa debolezza consentendo condizioni di test lineari multivariate della forma:

$$\sum_{j=1}^{d} a_j x_j + b < 0 \tag{21.11}$$

Sebbene permettano di isolare superfici inclinate arbitrarie con singoli tagli, la ricerca congiunta dei coefficienti ottimali $\mathbf{a} \in \mathbb{R}^d$ comporta un incremento esponenziale della complessità computazionale.

21.4.3 Frammentazione dei Dati e Replicazione dell'Albero

Due ulteriori patologie strutturali dell'induzione top-down sono:

1. **Frammentazione dei Dati (*Data Fragmentation*):** scendendo lungo i rami dell'albero, ogni partizionamento riduce drasticamente il numero di campioni disponibili nei nodi discendenti. Nei nodi profondi, le decisioni di split e l'assegnazione delle classi vengono determinate su campioni piccolissimi ($n \leq 5$), privi di significatività statistica e vulnerabili all'errore campionario.

2. **Replicazione dell'Albero (*Tree Replication*):** quando una funzione logica o un pattern sottostante dipende da una combinazione di attributi che si ripete in rami disgiunti, l'algoritmo è costretto a duplicare integralmente lo stesso sotto-albero in molteplici regioni dell'architettura globale, aggravando l'interpretabilità e disperdendo i dati di supporto.

21.5 Teoria dell'Errore, Underfitting e Overfitting

La bontà predittiva di un classificatore deve essere analizzata distinguendo con precisione i contesti di valutazione dell'errore.

21.5.1 Definizioni Formali dell'Errore di Classificazione

Sia data la funzione di predizione appresa $\hat{y} = f(\mathbf{x})$:

1. **Errore di Addestramento (*Training / Apparent Error*):** Frazione di errori commessi dal modello sui record appartenenti allo stesso insieme impiegato per la costruzione:

$$e_{\text{train}}(f) = \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{(\mathbf{x}_i, y_i) \in \mathcal{D}_{\text{train}}} \mathbb{I}(f(\mathbf{x}_i) \neq y_i) \tag{21.12}$$

   dove $\mathbb{I}(\cdot)$ è la funzione indicatrice.

2. **Errore di Test (*Test Error*):** Frazione di errori commessi sull'insieme indipendente di test:

$$e_{\text{test}}(f) = \frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{(\mathbf{x}_j, y_j) \in \mathcal{D}_{\text{test}}} \mathbb{I}(f(\mathbf{x}_j) \neq y_j) \tag{21.13}$$

3. **Errore di Generalizzazione (*Generalization Error*):** Il valore atteso teorico dell'errore rispetto alla distribuzione congiunta di probabilità sottostante $\mathcal{P}(\mathcal{X}, \mathcal{Y})$:

$$e_{\text{gen}}(f) = \mathbb{E}_{(\mathbf{x},y)\sim\mathcal{P}}\Big[\mathbb{I}(f(\mathbf{x}) \neq y)\Big] \tag{21.14}$$

21.5.2 Curve di Apprendimento: Underfitting e Overfitting

Al variare della complessità strutturale del modello (misurata negli alberi di decisione dal numero totale di nodi o foglie), l'errore di addestramento e l'errore di generalizzazione esibiscono andamenti divergenti, sintetizzati nella Figura 21.1.

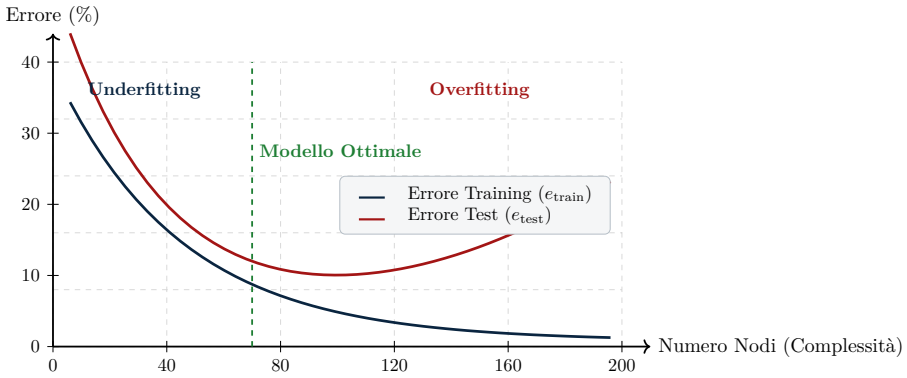


Figura 21.1: Dinamica dell'errore al crescere della complessità del modello: nella regione di underfitting il modello è eccessivamente rigido; superato il punto ottimale, subentra l'overfitting, in cui l'errore di test risale a fronte della decrescita monotona dell'errore di training.

Formalizziamo i due regimi patologici:

Definizione 21.5.1 (Underfitting)**.** Un modello incorre in **underfitting** (sotto-adattamento) quando la sua complessità strutturale è insufficiente per catturare le relazioni autentiche tra attributi e classe. In questo regime, **sia l'errore di addestramento sia l'errore di generalizzazione risultano elevati**.

Definizione 21.5.2 (Overfitting)**.** Un modello f incorre in **overfitting** (sovra-adattamento) se esiste un modello alternativo f' avente un errore di addestramento maggiore ($e_{\text{train}}(f') > e_{\text{train}}(f)$), ma un errore di generalizzazione strettamente inferiore ($e_{\text{gen}}(f') < e_{\text{gen}}(f)$). Il modello memorizza il rumore specifico e le fluttuazioni stocastiche dei dati di addestramento, perdendo la capacità di generalizzare su record non osservati.

21.5.3 Studio Empirico sull'Impatto di Nodi, Rumore e Dimensioni Campionarie

Nelle note didattiche di Tan et al. [2] viene condotto un celebre esperimento su 10.400 istanze bidimensionali nel dominio $[0, 20]^2$:

- **Classe + (5.200 punti):** 5.000 campioni generati da una densità gaussiana $\mathcal{N}(\boldsymbol{\mu} = [10, 10]^\top, \boldsymbol{\Sigma} = 2\mathbf{I})$ e 200 punti di rumore casuale dispersi uniformemente;
- **Classe o (5.200 punti):** distribuzione uniforme su tutto il dominio $[0, 20]^2$.

I risultati sperimentali mettono in risalto considerazioni di straordinaria rilevanza:

1. **Albero Semplice (4 nodi) vs Albero Complesso (50 nodi):**
   - L'albero minimale a 4 nodi traccia 4 tagli ortogonali che delimitano un box compatto attorno al baricentro $(10, 10)$: ottiene $e_{\text{train}} \approx 0.10$ e $e_{\text{test}} \approx 0.10$.
   - L'albero espanso a 50 nodi genera decine di minuscole partizioni isolate per catturare individualmente i 200 outlier rumorosi: riduce e_{train} a 0.065, ma il suo errore di test degrada a $e_{\text{test}} \approx 0.17$. L'albero a 4 nodi possiede una capacità predittiva enormemente superiore.
2. **Effetto della Dimensione del Training Set:** All'aumentare della numerosità campionaria N_{train}, la densità locale dello spazio cresce, riducendo lo scostamento statistico tra la distribuzione empirica e la distribuzione teorica. Le curve di training e test si mantengono aderenti e piatte lungo un arco di nodi molto più ampio, ritardando l'insorgenza dell'overfitting.
3. **Cause Primarie dell'Overfitting:**
   - **Presenza di Rumore:** campioni etichettati erroneamente costringono l'algoritmo a creare insenature e geometrie anomale nelle frontiere;
   - **Mancanza di Dati Rappresentativi:** nelle regioni periferiche dello spazio dove scarseggiano campioni, l'albero è costretto a estrapolare decisioni su basi statisticamente fragili.

21.6 Selezione del Modello e Stima dell'Errore di Generalizzazione

Poiché l'errore commesso sul training set costituisce una stima ottimistica fallace e fuorviante delle prestazioni su dati futuri, la **Model Selection** introduce criteri analitici per selezionare l'albero dotato della migliore capacità di generalizzazione.

21.6.1 Il Principio del Rasoio di Occam

Il fondamento epistemologico della selezione del modello è formulato dal **Rasoio di Occam** (*Occam's Razor*):

Dati due modelli che esibiscono un errore di addestramento comparabile, si deve costantemente preferire il modello più semplice rispetto a quello più complesso.

Un modello compatto dispone di un numero inferiore di gradi di libertà, rendendo improbabile che esso si sia conformato casualmente a pattern rumorosi privi di valore predittivo. La formulazione matematica generale penalizza esplicitamente la complessità architetturale:

$$e_{\text{gen}}(T) = e_{\text{train}}(T, \mathcal{D}_{\text{train}}) + \alpha \cdot \text{Complexity}(T) \tag{21.15}$$

dove $\alpha > 0$ è un coefficiente di regolarizzazione che controlla il compromesso tra accuratezza empirica e parsimonia strutturale.

21.6.2 Metodi Quantitativi di Stima dell'Errore

Esistono quattro metodologie formali per stimare l'errore di generalizzazione durante la costruzione dell'albero:

1. Utilizzo di un Validation Set

Il dataset di addestramento viene partizionato a priori in due blocchi disgiunti:

- **Training Set propriamente detto:** impiegato per generare la struttura dell'albero;
- **Validation Set:** impiegato per valutare l'accuratezza dei sotto-alberi e guidare la potatura.

Sebbene solido, questo metodo presenta lo svantaggio di sottrarre dati preziosi all'induzione primaria, risultando penalizzante in contesti a bassa disponibilità di campioni.

2. Stima Pessimistica dell'Errore (*Pessimistic Error Estimate*)

Proposta originariamente per C4.5 [47], la stima pessimistica assegna un costo fisso additivo Ω a ciascun nodo foglia. Dato un albero T avente k nodi terminali costruito su N_{train} record, l'errore pessimistico è formulato come:

$$e_{\text{pess}}(T) = \frac{\sum_{i=1}^{k} \Big(e(t_i) + \Omega\Big)}{N_{\text{train}}} = e_{\text{train}}(T) + \Omega \cdot \frac{k}{N_{\text{train}}} \tag{21.16}$$

dove $e(t_i)$ è il numero assoluto di errori commessi dalla foglia t_i. Tipicamente si imposta $\Omega = 0.5$ (correzione di continuità) o $\Omega = 1.0$.

Esempio 21.6.1 (Confronto tra Stima Ottimistica e Pessimistica)**.** Su un dataset di $N_{\text{train}} = 24$ istanze e con parametro di penalità $\Omega = 1.0$, si pongono a confronto due alberi concorrenti:

- **Albero Complesso T_L ($k = 7$ foglie):** commette complessivamente 4 errori sui dati di addestramento ($e_{\text{train}} = 4/24$):

$$\text{Errore di Training:} \quad e(T_L) = \frac{4}{24} \approx 0.1667 \tag{21.17}$$

$$\text{Errore Pessimistico:} \quad e_{\text{pess}}(T_L) = \frac{4 + 1.0 \times 7}{24} = \frac{11}{24} \approx 0.4583 \tag{21.18}$$

- **Albero Semplice T_R ($k = 4$ foglie):** commette complessivamente 6 errori sui dati di addestramento ($e_{\text{train}} = 6/24$):

$$\text{Errore di Training:} \quad e(T_R) = \frac{6}{24} = 0.2500 \tag{21.19}$$

$$\text{Errore Pessimistico:} \quad e_{\text{pess}}(T_R) = \frac{6 + 1.0 \times 4}{24} = \frac{10}{24} \approx 0.4167 \tag{21.20}$$

Risultato: mentre la stima ottimistica sceglierebbe erroneamente l'albero complesso T_L ($4/24 < 6/24$), la stima pessimistica inverte la graduatoria e premia correttamente l'albero semplice T_R ($10/24 < 11/24$).

3. Principio della Minima Lunghezza di Descrizione (MDL)

Fondato sulla teoria dell'informazione e sulla complessità di Kolmogorov [48], il principio MDL formalizza l'apprendimento come compressione dei dati. Il costo complessivo di trasmissione tra un mittente e un destinatario si scompone in:

$$\text{Cost}(M, \mathcal{D}) = \text{Cost}(M) + \text{Cost}(\mathcal{D} \mid M) \tag{21.21}$$

- $\text{Cost}(M)$ misura il numero di bit necessari per codificare la topologia dell'albero (numero di nodi, attributi e soglie);
- $\text{Cost}(\mathcal{D} \mid M)$ misura i bit necessari per codificare le etichette reali conoscendo le previsioni del modello (pari a zero per i punti corretti, logaritmico per le eccezioni ed errori).

L'albero ottimale è quello che minimizza la somma totale dei bit di trasmissione.

4. Stima dei Limiti Statistici Superiori (*Upper Bound*)

Questo approccio assume che il numero di errori commessi da un nodo foglia t segua una distribuzione binomiale $B(n_t, p)$. Fissato un livello di confidenza statistica $1 - \alpha$ (tipicamente con $\alpha = 0.25$ in C4.5), si calcola l'estremo superiore dell'intervallo di confidenza per la proporzione di errore $e'(n_t, f, \alpha)$, dove $f = e_t/n_t$. Se il numero totale di errori attesi stimati dopo uno split supera l'errore atteso del nodo padre non partizionato:

$$\sum_{i=1}^{k} n_i \cdot e'(n_i, f_i, \alpha) > n_p \cdot e'(n_p, f_p, \alpha) \tag{21.22}$$

l'algoritmo rigetta lo split impedendo la ramificazione.

21.7 Tecniche di Potatura (*Pruning*)

La prevenzione e la risoluzione dell'overfitting negli alberi di decisione si attuano concretamente mediante due paradigmi di **potatura** (*pruning*): **Pre-Pruning** e **Post-Pruning**.

21.7.1 Pre-Pruning (Arresto Anticipato o Early Stopping)

Il **Pre-Pruning** interviene durante la discesa ricorsiva dell'algoritmo di induzione, arrestando la creazione di ulteriori rami prima che l'albero raggiunga la sua massima espansione. I criteri operativi più comuni includono:

- **Soglia Minima di Istanze per Nodo:** se $|\mathcal{D}_t| <$ min_samples_split, il nodo non viene partizionato;
- **Soglia di Significatività Statistica del Guadagno:** lo split viene autorizzato solo se il guadagno $\Delta = Gain$ supera un valore prefissato $\epsilon > 0$ o se un test del chi-quadrato (χ^2) ne attesta la non-casualità a livello di significatività $p < 0.05$;
- **Controllo di Profondità Massima:** imposizione rigida di un'altezza massima dell'albero (*max_depth*).

Limite del Pre-Pruning (Effetto Orizzonte): l'arresto anticipato soffre del rischio di terminazione prematura (*horizon effect*). Un attributo che offre un guadagno nullo o trascurabile a livello locale potrebbe essere il preludio necessario per effettuare uno split successivo eccezionalmente discriminante (come mostrato nel problema XOR).

21.7.2 Post-Pruning (Potatura a Posteriori)

Nel paradigma del **Post-Pruning**, si lascia crescere l'albero fino alla sua massima espansione naturale (fino a quando tutti i nodi sono puri o omogenei negli attributi). Successivamente, si esegue una scansione retrograda in senso **bottom-up** (dalle foglie verso la radice):

1. Per ciascun nodo interno t, si calcola l'errore di generalizzazione stimato del sotto-albero discendente T_t;

2. Si calcola l'errore stimato qualora il sotto-albero venisse potato e sostituito da una singola foglia con classe pari alla **maggioranza** dei campioni in t;

3. Se l'errore stimato della foglia singola risulta inferiore o uguale a quello del sotto-albero ramificato, il sotto-albero viene eliminato e sostituito dalla foglia.

21.7.3 Simulazione Numerica di Post-Pruning Pessimistico

Illustriamo la decisione di potatura su un nodo interno A che copre $N = 30$ record di addestramento (20 di classe 'Yes' e 10 di classe 'No', classe maggioritaria 'Yes') ramificandosi in $k = 4$ nodi foglia, applicando la stima pessimistica con penalità fogliare $\Omega = 0.5$.

1. **Valutazione dell'Ipotesi di Potatura (Sostituzione con Foglia Singola, $k = 1$):** Se il nodo A viene convertito in foglia, ad esso viene associata l'etichetta 'Yes'. Il numero di errori di addestramento è pari al conteggio della classe minoritaria: $e = 10$. L'errore pessimistico prima dello split (foglia singola) è:

$$e_{\text{pess}}(\text{Foglia Singola}) = \frac{10 + 1 \times 0.5}{30} = \frac{10.5}{30} \approx 0.3500 \tag{21.23}$$

2. **Valutazione del Sotto-Albero Espanso ($k = 4$ foglie):** I quattro nodi figli presentano i seguenti conteggi e classi maggioritarie:

   - Figlio 1 ($n_1 = 12$, 'Yes': 8, 'No': 4): classe maggioritaria 'Yes', errori empirici $e_1 = 4$;
   - Figlio 2 ($n_2 = 7$, 'Yes': 3, 'No': 4): classe maggioritaria 'No', errori empirici $e_2 = 3$;

- Figlio 3 ($n_3 = 5$, 'Yes': 4, 'No': 1): classe maggioritaria 'Yes', errori empirici $e_3 = 1$;
- Figlio 4 ($n_4 = 6$, 'Yes': 5, 'No': 1): classe maggioritaria 'Yes', errori empirici $e_4 = 1$.

Il numero totale di errori empirici del sotto-albero è $4 + 3 + 1 + 1 = 9$ (apparentemente migliore dei 10 errori della foglia singola). Tuttavia, incorporando la penalità di complessità per le $k = 4$ foglie:

$$e_{\text{pess}}(\text{Sotto-Albero}) = \frac{9 + 4 \times 0.5}{30} = \frac{9 + 2.0}{30} = \frac{11.0}{30} \approx 0.3667 \quad (21.24)$$

Decisione Algoritmica: Poiché la stima dell'errore pessimistico del sotto-albero ramificato ($\frac{11.0}{30} = 0.3667$) è strettamente superiore rispetto a quella della foglia collassata ($\frac{10.5}{30} = 0.3500$), l'algoritmo procede alla potatura: elimina i quattro rami e converte il nodo A in una foglia terminale con etichetta Class = Yes.

La Tabella 21.2 riassume sinotticamente il confronto tra le due metodologie di potatura.

Tabella 21.2: Confronto sinottico tra le strategie di potatura per alberi di decisione.

Dimensione di Confronto	**Pre-Pruning (Arresto Anticipato)**	**Post-Pruning (Potatura Retrograda)**
Fase di Intervento	Durante la crescita top-down dell'albero	Dopo la completa espansione bottom-up
Costo Computazionale	Estremamente ridotto (alberi più piccoli)	Moderato/Alto (richiede crescita completa)
Rischio Principale	Arresto prematuro (Effetto Orizzonte, es. XOR)	Sovraccarico di memoria durante la crescita
Criteri Guida	Soglie su campioni minimi, Δ Gain, test χ^2	Stima pessimistica, Validation Set, MDL
Accuratezza Finale	Moderata su problemi complessi	Tipicamente superiore e ottimizzata
Algoritmi Simbolo	ID3 (rudimentale), CART (con vincoli)	C4.5, CART (Cost-Complexity Pruning)

Capitolo 22

Valutazione dei Modelli di Classificazione

22.1 Introduzione e Metriche Fondamentali di Prestazione

Una volta indotto un modello di classificazione attraverso uno degli algoritmi supervisionati esaminati nei capitoli precedenti, risulta metodologicamente imprescindibile sottoporlo a una rigorosa procedura di validazione prima della sua effettiva adozione in contesti operativi o produttivi [2]. La mera accuratezza empirica riscontrata sui record di addestramento non fornisce alcuna garanzia sulla reale efficacia del classificatore, potendo celare gravi patologie di sovra-adattamento (*overfitting*) o di distorsione campionaria.

La disciplina della **Valutazione dei Modelli** (*Model Evaluation*) si articola attorno a tre quesiti cardine della metodologia scientifica:

1. **Metriche di Valutazione (*Metrics for Performance Evaluation*):** quali indicatori matematici e quantitativi consentono di misurare oggettivamente la qualità delle predizioni formulate da un modello?

2. **Metodi di Valutazione (*Methods for Performance Evaluation*):** quali strategie di partizionamento e ricampionamento dei dati garantiscono stime affidabili, non distorte e statisticamente robuste dell'errore di generalizzazione?

3. **Metodi per il Confronto tra Modelli (*Methods for Model Comparison*):** come stabilire con rigore formale se la superiorità predittiva di un modello rispetto a un altro sia statisticamente significativa e non imputabile a mere fluttuazioni casuali del campione di verifica?

22.1.1 La Matrice di Confusione e la Metrica di Accuratezza

La valutazione delle prestazioni si concentra in via prioritaria sulla **capacità predittiva** del modello, ponendo in secondo piano aspetti prettamente ingegneristici quali il tempo di convergenza dell'algoritmo o la scalabilità computazionale.

Per un problema di classificazione binaria con classi target $\mathcal{Y} = \{\text{Yes}, \text{No}\}$ (o convenzionalmente positive '+' e negative '−'), lo strumento algebrico fondamentale per registrare il comportamento predittivo è la **Matrice di Confusione** (*Confusion Matrix*), strutturata come una tabella 2×2 che incrocia le classi effettive delle istanze con le classi predette dal modello, come formalizzato nella Tabella 22.1.

Tabella 22.1: Matrice di confusione per la classificazione binaria.

	Classe Predetta: Yes (+)	**Classe Predetta: No (−)**
Classe Reale: Yes (+)	a (**True Positive**, TP)	b (**False Negative**, FN)
Classe Reale: No (−)	c (**False Positive**, FP)	d (**True Negative**, TN)

I quattro conteggi elementari che popolano la matrice assumono un significato semantico preciso:

- **Veri Positivi ($a = TP$, *True Positives*):** numero di istanze positive realmente appartenenti alla classe target che sono state correttamente identificate dal modello come positive;
- **Falsi Negativi ($b = FN$, *False Negatives*):** numero di istanze positive che il classificatore ha erroneamente catalogato come negative (errore di omissione o *errore di Tipo II*);
- **Falsi Positivi ($c = FP$, *False Positives*):** numero di istanze negative che il classificatore ha erroneamente catalogato come positive (falso allarme o *errore di Tipo I*);
- **Veri Negativi ($d = TN$, *True Negatives*):** numero di istanze negative che sono state correttamente riconosciute come appartenenti alla classe negativa.

Definizione 22.1.1 (Accuratezza ed Errore)**.** L'**Accuratezza** (*Accuracy*) misura la proporzione complessiva di predizioni corrette formulate dal classificatore sul totale dei record analizzati:

$$Accuracy = \frac{a + d}{a + b + c + d} = \frac{TP + TN}{TP + TN + FP + FN} \tag{22.1}$$

Il complemento a uno dell'accuratezza definisce il **Tasso di Errore** (*Error Rate*):

$$Error = 1 - Accuracy = \frac{b + c}{a + b + c + d} = \frac{FN + FP}{TP + TN + FP + FN} \quad (22.2)$$

22.1.2 Il Limite Strutturale dell'Accuratezza: Classi Sbilanciate

Sebbene l'accuratezza rappresenti l'indicatore sintetico più intuitivo e diffuso, essa manifesta un gravissimo limite concettuale in presenza di collezioni di dati caratterizzate da una **distribuzione asimmetrica delle classi** (*class imbalance problem*).

Esempio 22.1.1 (Il Paradosso del Classificatore Banale su Dati Sbilanciati)**.** Si consideri un dataset di $N = 10\,000$ transazioni finanziarie per il rilevamento di frodi bancarie, suddivise in:

- $N_0 = 9\,990$ transazioni lecite (Classe di maggioranza C_0);
- $N_1 = 10$ transazioni fraudolente (Classe rara di minoranza C_1).

Un classificatore degenere o banale che risponda invariabilmente con la classe maggioritaria C_0 per qualunque record in ingresso otterrà una matrice di confusione con $TN = 9\,990$, $FN = 10$, $TP = 0$, $FP = 0$. L'accuratezza associata a tale modello è:

$$Accuracy = \frac{0 + 9\,990}{10\,000} = 99.9\% \quad (22.3)$$

Nonostante un'accuratezza del 99.9%, il modello è totalmente inutile sul piano applicativo, avendo mancato la totalità degli eventi critici di frode ($TP = 0$).

In contesti applicativi sbilanciati (quali diagnostica medica di patologie rare, rilevamento di intrusioni informatiche o prevenzione dei guasti), l'accuratezza standard si rivela pertanto fuorviante.

22.1.3 Matrice dei Costi e Classificazione Sensibile ai Costi

Nella realtà operativa, gli errori predittivi raramente comportano il medesimo impatto economico o sociale: mancare l'individuazione di una neoplasia maligna in un paziente comporta conseguenze incomparabilmente più gravi

rispetto a sottoporre un individuo sano a un controllo di approfondimento non necessario.

Per quantificare tale asimmetria si definisce la **Matrice dei Costi** (*Cost Matrix*) $C(i \mid j)$, che assegna un onere economico o penalità associata all'evento di predire la classe i quando la vera classe target è j, come riportato nella Tabella 22.2.

Tabella 22.2: Matrice formale dei costi di classificazione $C(i \mid j)$.

$C(i \mid j)$	**Predizione: Yes (+)**	**Predizione: No (−)**
Classe Reale: Yes (+)	C(Yes | Yes) (Beneficio/Costo Corretto)	C(No | Yes) (Costo Falso Negativo)
Classe Reale: No (−)	C(Yes | No) (Costo Falso Positivo)	C(No | No) (Beneficio/Costo Corretto)

Il costo totale complessivo generato da un modello M su un insieme di test si calcola come combinazione lineare elemento per elemento:

$$Cost(M) = TP \cdot C(+ \mid +) + FN \cdot C(- \mid +) + FP \cdot C(+ \mid -) + TN \cdot C(- \mid -) \tag{22.4}$$

Esempio 22.1.2 (Il Paradosso del Costo: Modello Meno Accurato ma Più Economico)**.** Si consideri una matrice dei costi in cui un falso negativo comporta una severa penalità pari a 100, un falso positivo ha costo 1, un vero positivo produce un guadagno unitario (costo -1) e un vero negativo ha costo nullo (0):

$$C(+ \mid +) = -1, \quad C(- \mid +) = 100, \quad C(+ \mid -) = 1, \quad C(- \mid -) = 0$$

Valutiamo due modelli concorrenti M_1 ed M_2 su un archivio comune di $N = 500$ osservazioni [2]:

1. **Modello M_1:** caratterizzato da $TP = 150$, $FN = 40$, $FP = 60$, $TN = 250$.

$$Accuracy(M_1) = \frac{150 + 250}{500} = \frac{400}{500} = 80.0\% \tag{22.5}$$

$$\begin{aligned} Cost(M_1) &= 150(-1) + 40(100) + 60(1) + 250(0) \\ &= -150 + 4000 + 60 + 0 = 3910 \end{aligned} \tag{22.6}$$

2. **Modello M_2:** caratterizzato da $TP = 250$, $FN = 45$, $FP = 5$, $TN = 200$.

$$Accuracy(M_2) = \frac{250 + 200}{500} = \frac{450}{500} = 90.0\% \tag{22.7}$$

$$\begin{aligned} Cost(M_2) &= 250(-1) + 45(100) + 5(1) + 200(0) \\ &= -250 + 4500 + 5 + 0 = 4255 \end{aligned} \tag{22.8}$$

Implicazione Decisiva: Sebbene il modello M_2 appaia nettamente superiore sulla base dell'accuratezza (90% contro 80%), esso produce un costo economico aggregato sensibilmente maggiore (4255 contro 3910). La ragione risiede nel fatto che M_2 commette 5 falsi negativi in più rispetto a M_1, ciascuno penalizzato con un costo asimmetrico di 100.

22.1.4 Metriche Asimmetriche: Precision, Recall e Misura F

Per superare i limiti dell'accuratezza in regimi sbilanciati, la teoria del machine learning e dell'information retrieval adotta indici focalizzati primariamente sulla classe di reale interesse (convenzionalmente la classe positiva):

Definizione 22.1.2 (Precisione)**.** La **Precisione** (*Precision*, p) quantifica l'affidabilità delle risposte positive fornite dal modello, misurando la frazione di istanze realmente positive tra tutte quelle che il sistema ha etichettato come positive:

$$Precision = p = \frac{TP}{TP + FP} \tag{22.9}$$

Una precisione elevata indica che il modello commette pochissimi falsi allarmi ($FP \to 0$).

Definizione 22.1.3 (Richiamo o Sensibilità)**.** Il **Richiamo** (*Recall* o *Sensitivity*, r) misura la capacità del modello di intercettare gli elementi della classe target, rappresentando la frazione di istanze positive individuate rispetto al totale dei casi positivi presenti nei dati:

$$Recall = r = \frac{TP}{TP + FN} \tag{22.10}$$

Un richiamo elevato indica che il modello genera pochissimi falsi negativi ($FN \to 0$).

Definizione 22.1.4 (Misura F ed F_β-Score)**.** La **Misura F** (*F-measure* o F_1-score) sintetizza il compromesso tra Precisione e Richiamo combinandoli attraverso la loro **media armonica**:

$$F_1 = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall} = \frac{2 \cdot p \cdot r}{p + r} \tag{22.11}$$

Sostituendo le espressioni analitiche di p ed r, si ottiene la forma compatta in termini di matrice di confusione:

$$F_1 = \frac{2 \cdot \frac{TP}{TP+FP} \cdot \frac{TP}{TP+FN}}{\frac{TP}{TP+FP} + \frac{TP}{TP+FN}} = \frac{2 \cdot TP}{2 \cdot TP + FP + FN} \tag{22.12}$$

Si evidenzia che la metrica F_1 tiene conto di tutte le celle della matrice di confusione ad eccezione dei veri negativi (TN), risultando totalmente insensibile all'iper-abbondanza della classe maggioritaria negativa. La formulazione generalizzata dell'F_β-score consente di attribuire un peso arbitrario $\beta > 0$ al Richiamo rispetto alla Precisione:

$$F_\beta = \frac{(1 + \beta^2) \cdot p \cdot r}{\beta^2 \cdot p + r} \tag{22.13}$$

con $\beta = 2$ per privilegiare il richiamo e $\beta = 0.5$ per privilegiare la precisione.

22.2 Metodologie di Partizionamento e Stima Affidabile

La stima non distorta delle prestazioni richiede la definizione di un protocollo rigoroso di separazione tra i dati impiegati per l'addestramento e quelli impiegati per la verifica.

22.2.1 Isolamento del Test Set e Partizionamento a Tre Vie

Un principio metodologico inviolabile dell'apprendimento automatico stabilisce che **i dati del Test Set non devono in alcun modo intervenire nel processo di apprendimento, selezione delle feature o calibrazione degli iper-parametri** [2]. Nei sistemi complessi, l'induzione opera tipicamente in due stadi:

- **Stadio 1 (Costruzione):** apprendimento della struttura primaria del modello;
- **Stadio 2 (Ottimizzazione):** taratura fine degli iper-parametri (*hyperparameter tuning*, quali profondità massima dell'albero o soglie di arresto).

L'impiego del test set per calibrare gli iper-parametri conduce a un fenomeno subdolo di fuga informativa (*data leakage*), fornendo una stima artificialmente ottimistica. La procedura scientificamente corretta impone la suddivisione del patrimonio informativo in **tre insiemi mutuamente disgiunti**, illustrata nella Figura 22.1:

1. **Training Set:** impiegato dall'algoritmo per stimare i parametri interni del classificatore;

2. **Validation Set:** impiegato per valutare molteplici configurazioni algoritmiche concorrenti e selezionare il set di iper-parametri ottimale;

3. **Test Set:** mantenuto rigorosamente isolato e inviolato fino al termine dello sviluppo, impiegato un'unica volta per la valutazione indipendente finale (*Final Assessment*).

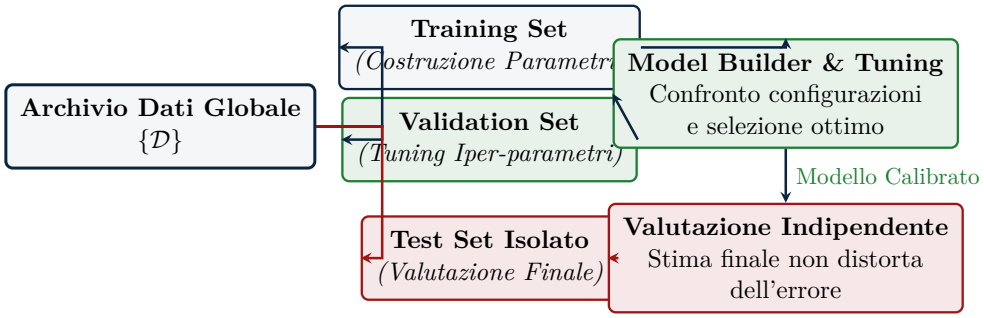


Figura 22.1: Architettura a tre partizioni per l'ottimizzazione degli iper-parametri e la valutazione oggettiva del modello: il Test Set rimane completamente avulso dal ciclo di sviluppo.

22.2.2 Curve di Apprendimento, Bias e Varianza della Stima

Le prestazioni rilevate di un classificatore dipendono intrinsecamente da tre fattori: la distribuzione naturale delle classi, i costi associati alle asimmetrie decisionali e la **numerosità campionaria** impiegata per l'induzione [2].

Una **Curva di Apprendimento** (*Learning Curve*) illustra l'andamento dell'accuratezza predittiva in funzione della dimensione del campione di addestramento N, tipicamente tracciata su scala logaritmica (da 10^0 a 10^4 record), come schematizzato nella Figura 22.2.

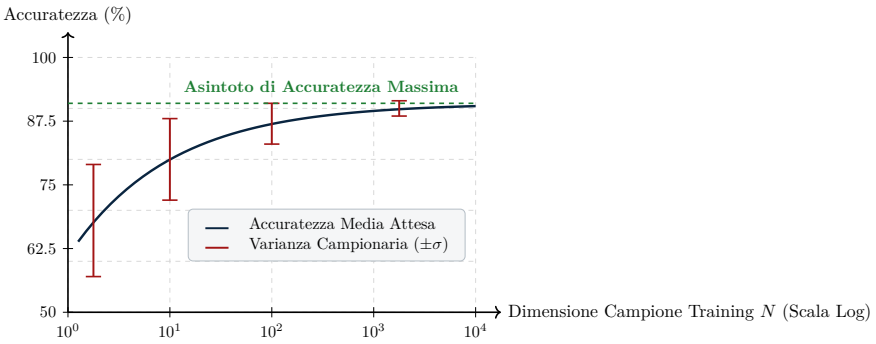


Figura 22.2: Curva di apprendimento in funzione della numerosità campionaria N: per campioni ridotti la stima presenta elevata distorsione (bias negativo rispetto all'asintoto) e amplissima varianza; all'aumentare di N la varianza collassa e l'accuratezza converge al limite intrinseco del modello.

Dall'esame analitico della curva emergono due conclusioni fondamentali:

1. **Distorsione della Stima (*Bias in the estimate*):** quando il modello viene addestrato su una porzione modesta del dataset disponibile (ad esempio nel metodo holdout con 2/3 dei dati), la sua accuratezza risulterà sistematicamente inferiore rispetto all'accuratezza che il medesimo algoritmo conseguirebbe se addestrato sull'intero dataset;

2. **Varianza della Stima (*Variance of the estimate*):** su campioni di test ridotti, le fluttuazioni campionarie determinano una marcata dispersione statistica della metrica misurata, imponendo la necessità di metodi di stima basati su partizionamenti multipli.

22.2.3 Metodi di Ricampionamento e Stima dell'Errore

Nella pratica computazionale si ricorre a quattro strategie formali di ricampionamento:

1. Metodo Holdout e Campionamento Stratificato

Consiste nel partizionare casualmente una sola volta il dataset originario $\mathcal{D}$: tipicamente una frazione pari al 66.7% (2/3) viene allocata al *training* e il restante 33.3% (1/3) al *testing*.

- **Limite:** se il dataset è ridotto o sbilanciato, l'estrazione casuale semplice può produrre un test set non rappresentativo, in cui alcune classi minoritarie risultano assenti.

- **Campionamento Stratificato (*Stratified Sampling*):** vincola l'estrazione affinché ciascuna classe sia presente sia nel training set che nel test set con proporzioni percentuali esattamente identiche a quelle della popolazione complessiva.

2. Repeated Holdout (Subcampionamento Casuale)

Per mitigare la varianza dell'holdout singolo, si ripete la procedura di partizionamento casuale per R iterazioni indipendenti. Se e_r denota il tasso di errore misurato all'iterazione r, la stima finale aggregata è:

$$e_{\text{sub}} = \frac{1}{R} \sum_{r=1}^{R} e_r \tag{22.14}$$

Sebbene riduca la varianza, questa procedura presenta il difetto strutturale del **sovrapponimento** (*overlap*): alcuni campioni possono essere selezionati per il test in più iterazioni, mentre altri potrebbero non essere mai valutati.

3. Validazione Incrociata (k-*Fold Cross-Validation*)

La validazione incrociata [49] risolve elegantemente il problema dell'overlap garantendo che ciascuna istanza del dataset venga impiegata esattamente una volta come record di test:

1. Il dataset $\mathcal{D}$ viene suddiviso in k blocchi (*folds*) mutualmente disgiunti di uguale dimensione: $\mathcal{D} = S_1 \cup S_2 \cup \cdots \cup S_k$;

2. Si eseguono k iterazioni distinte: all'iterazione i-esima, il blocco S_i funge da *Test Set*, mentre l'unione dei restanti $k-1$ blocchi costituisce il *Training Set*;

3. L'errore aggregato è la media aritmetica degli errori registrati nelle k esecuzioni:
$$e_{\text{CV}} = \frac{1}{k} \sum_{i=1}^{k} e_i \tag{22.15}$$

Le varianti operative primarie sono:

- **Leave-One-Out Cross-Validation (LOOCV):** caso limite in cui $k = N$. A ciascuna iterazione si addestra su $N-1$ record e si valuta sul singolo campione escluso. Fornisce una stima a bassissima distorsione ma ad alto costo computazionale e vulnerabile a elevata varianza statistica;

- **Stratified k-Fold Cross-Validation:** impone che la suddivisione nei k blocchi mantenga rigorosamente le proporzioni delle classi;
- **Repeated Stratified Cross-Validation:** ripete l'intero schema k-fold per M volte rimescolando i dati (es. 10-fold CV ripetuta 10 volte, con 100 run totali), stabilizzando la stima della deviazione standard.

4. Metodo Bootstrap e lo Stimatore .632

Il **Bootstrap** si fonda sul campionamento casuale uniforme con reinserimento (*sampling with replacement*) di N istanze a partire da un dataset originario di dimensione N. La probabilità che un generico record non venga mai estratto in N pescate consecutive è data da:

$$P(\text{Non estratto}) = \left(1 - \frac{1}{N}\right)^N \tag{22.16}$$

Per N asintoticamente grande, tale quantità converge matematicamente al limite notevole:

$$\lim_{N \to \infty} \left(1 - \frac{1}{N}\right)^N = \frac{1}{e} \approx 0.368 = 36.8\% \tag{22.17}$$

Di conseguenza, il campione bootstrap conterrà mediamente circa il 63.2% delle istanze uniche originali (che fungono da training set), mentre il restante 36.8% di record mai estratti costituisce il cosiddetto insieme *Out-Of-Bag* (OOB), impiegato come test set. La stima corretta denominata **.632 Bootstrap** combina l'errore di risostituzione sull'intero archivio e_{train} e l'errore registrato sui campioni esclusi e_{OOB}:

$$e_{\text{boot}} = 0.632 \cdot e_{\text{OOB}} + 0.368 \cdot e_{\text{train}} \tag{22.18}$$

22.3 Confronto tra Modelli e Analisi ROC

Nei problemi applicativi in cui la soglia decisionale per dichiarare positiva un'istanza è fluida o legata a una matrice di costi variabile, la valutazione di un classificatore a un singolo valore operativo risulta limitante.

22.3.1 Teoria e Topologia dello Spazio ROC

La metodologia delle curve **ROC** (*Receiver Operating Characteristic*) affonda le proprie radici teoriche negli studi sulla rilevazione dei segnali radar sviluppati durante la Seconda Guerra Mondiale per discernere il segnale autentico dal rumore di fondo [50].

Lo **spazio ROC** è un piano cartesiano bidimensionale che pone in relazione:

- Sull'asse delle ascisse (x): il **Tasso di Falsi Positivi** (*False Positive Rate*, FPR), ovvero la proporzione di falsi allarmi sul totale dei record negativi:
$$FPR = \frac{FP}{FP + TN} = 1 - Specificity \tag{22.19}$$
- Sull'asse delle ordinate (y): il **Tasso di Veri Positivi** (*True Positive Rate*, TPR), equivalente al Richiamo (*Recall* o *Sensitivity*):
$$TPR = \frac{TP}{TP + FN} \tag{22.20}$$

Un classificatore che produce etichette binarie rigide determina un singolo punto isolato nel piano cartesiano ROC. Se il classificatore genera una stima continua di confidenza o di probabilità a posteriori $P(+ \mid \mathbf{x})$, la modulazione della soglia di taglio decisionale $\tau \in [0, 1]$ permette al punto di scorrere nello spazio, tracciando una curva continua.

I punti salienti della topologia ROC sono:

- $(0, 0)$**:** il modello predice costantemente la classe negativa; non genera alcun falso positivo ($FPR = 0$), ma fallisce nel rilevare qualunque positivo ($TPR = 0$);
- $(1, 1)$**:** il modello predice costantemente la classe positiva; rileva tutti i positivi ($TPR = 1$), ma genera il 100% di falsi allarmi ($FPR = 1$);
- $(0, 1)$**:** il **Classificatore Ideale Perfetto**, che consegue contemporaneamente $TPR = 1.0$ e $FPR = 0.0$;
- **La Diagonale** $y = x$ **(*Random Guessing*):** un classificatore che assegna le etichette tirando a indovinare in modo puramente casuale si colloca esattamente lungo la retta diagonale che unisce $(0, 0)$ a $(1, 1)$;
- **Regione Sotto la Diagonale** ($y < x$)**:** identifica modelli con prestazioni inferiori al caso casuale. È opportuno notare che tali modelli possono essere convertiti in classificatori eccellenti semplicemente invertendo l'assegnazione delle predizioni binarie.

22.3.2 L'Area Sotto la Curva ROC (AUC)

L'indicatore scalare sintetico più impiegato per confrontare modelli concorrenti è l'**Area Sottesa alla Curva ROC** (*Area Under the ROC Curve*, *AUC*), definita dall'integrale:

$$AUC = \int_0^1 TPR(FPR)\, d(FPR) \in [0, 1] \tag{22.21}$$

Teorema 22.3.1 (Significato Probabilistico dell'AUC)**.** *L'indice AUC di un classificatore coincide matematicamente con la probabilità che il modello assegni un punteggio di confidenza o probabilità $P(+ \mid \mathbf{x})$ strettamente più elevato a un'istanza positiva estratta casualmente rispetto a un'istanza negativa estratta casualmente:*

$$AUC = P\Big(f(\mathbf{x}_+) > f(\mathbf{x}_-)\Big) \tag{22.22}$$

Valori di riferimento:

- $AUC = 1.0$: classificatore perfetto;
- $AUC = 0.5$: classificatore puramente casuale;
- $AUC > 0.8$: eccellente potere discriminante;
- $AUC < 0.5$: modello incoerente o con etichette scambiate.

Qualora due curve ROC si intersechino, nessun modello domina uniformemente l'altro: il primo classificatore risulterà preferibile in regimi conservativi a basso tasso di falsi allarmi ($FPR \ll 1$), mentre il secondo risulterà ottimale in regimi ad alta cattura di positivi ($TPR \approx 1$).

22.3.3 Algoritmo Costruttivo Empirico di una Curva ROC

Data una collezione di record di test su cui il classificatore assegna una probabilità continua di appartenenza alla classe positiva $P(+ \mid \mathbf{x})$, la costruzione formale della curva empirica a gradini opera in quattro passaggi sequenziali [2, 50]:

1. Si ordinano le N osservazioni del test set in ordine rigorosamente decrescente in base al punteggio $P(+ \mid \mathbf{x})$;
2. Si scorre la lista ordinata considerando ciascun valore di probabilità unico come soglia decisionale τ, classificando come positivi tutti i record con score $\geq \tau$;

3. Per ogni soglia si quantificano i valori di TP, FP, TN, FN;

4. Si calcolano le coordinate cartesiane (FPR, TPR) e si connettono i punti risultanti.

La Tabella 22.3 illustra la traccia di calcolo completa su un test set di 10 record (5 positivi e 5 negativi).

Tabella 22.3: Calcolo passo-passo dei tassi TPR ed FPR per la generazione della curva ROC.

Istanza	**Probabilità** $P(+ \mid \mathbf{x})$	**Classe Reale**	**Soglia** τ	TP	FP	TN	FN	TPR	FPR
–	–	–	$\tau > 0.95$	0	0	5	5	**0.00**	**0.00**
1	0.95	+	$\tau = 0.95$	1	0	5	4	**0.20**	**0.00**
2	0.93	+	$\tau = 0.93$	2	0	5	3	**0.40**	**0.00**
3	0.87	−	$\tau = 0.87$	2	1	4	3	**0.40**	**0.20**
4	0.85	−	$\tau = 0.85$	3	2	3	2	**0.60**	**0.40**
5	0.85	−	$\tau = 0.85$	3	3	2	2	**0.60**	**0.60**
6	0.85	+	$\tau = 0.85$	3	4	1	2	**0.60**	**0.80**
7	0.76	−	$\tau = 0.76$	3	4	1	2	**0.60**	**0.80**
8	0.53	+	$\tau = 0.53$	4	4	1	1	**0.80**	**0.80**
9	0.43	−	$\tau = 0.43$	4	5	0	1	**0.80**	**1.00**
10	0.25	+	$\tau = 0.25$	5	5	0	0	**1.00**	**1.00**

Tracciando le coordinate (FPR, TPR) sul piano cartesiano, si ottiene la curva empirica mostrata nella Figura 22.3.

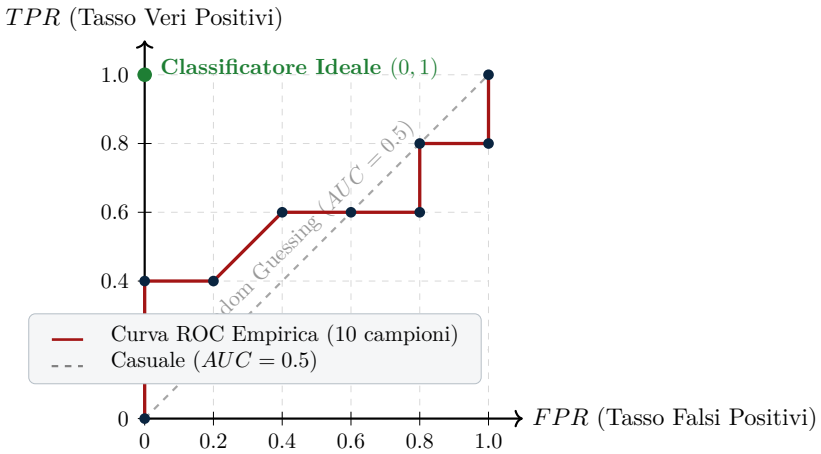


Figura 22.3: Curva ROC a gradini generata dalla traccia dei 10 campioni ordinati: la superficie sottesa calcolata geometricamente fornisce $AUC = 0.68$, evidenziando un potere discriminante superiore al caso ma perfettibile.

22.4 Test di Significatività Statistica e Confronto tra Modelli

Consideriamo un quesito classico della pratica scientifica:

> *Dati due modelli concorrenti M_1 ed M_2, dove M_1 evidenzia un'accuratezza dell'*85% *su un test set di sole $N_1 = 30$ istanze, mentre M_2 evidenzia un'accuratezza del* 75% *su un test set massivo di $N_2 = 5\,000$ istanze, è lecito affermare con certezza che M_1 sia superiore ad M_2?*

Per rispondere in modo rigoroso, è indispensabile quantificare la confidenza statistica delle stime campionarie e applicare la teoria dei test d'ipotesi [2].

22.4.1 Modellazione Binomiale e Intervallo di Confidenza per l'Accuratezza

La classificazione di ciascun record di test indipendente può essere formalmente modellata come una **prova di Bernoulli** che ammette due soli esiti: successo (previsione corretta, evento con probabilità costante p pari alla reale accuratezza del modello) e fallimento (errore, probabilità $1 - p$).

Su una collezione di N record di test indipendenti, il numero casuale di successi X segue rigorosamente una **distribuzione binomiale**:

$$X \sim \text{Bin}(N, p), \quad \text{con } \mathbb{E}[X] = N \cdot p, \quad \text{Var}(X) = N \cdot p(1-p) \tag{22.23}$$

La metrica campionaria di accuratezza osservata è $acc = \frac{X}{N}$, con valore atteso $\mathbb{E}[acc] = p$ e varianza campionaria:

$$\sigma_{acc}^2 = \frac{p(1-p)}{N} \tag{22.24}$$

Per il Teorema del Limite Centrale, qualora N sia sufficientemente grande ($N > 30$ e $N \cdot p > 5$), la distribuzione campionaria dell'accuratezza converge a una densità normale standardizzata:

$$Z = \frac{acc - p}{\sqrt{\frac{p(1-p)}{N}}} \sim \mathcal{N}(0, 1) \tag{22.25}$$

Fissato un livello di confidenza statistica $1 - \alpha$ (tipicamente al 95%, con $\alpha = 0.05$ e quantile critico $Z_{\alpha/2} = 1.96$), la probabilità associata soddisfa:

$$P\left(-Z_{\alpha/2} \leq \frac{acc - p}{\sqrt{\frac{p(1-p)}{N}}} \leq Z_{\alpha/2}\right) = 1 - \alpha \tag{22.26}$$

Elevando al quadrato la disuguaglianza e risolvendo l'equazione di secondo grado rispetto al parametro incognito p, si ricava l'espressione in forma chiusa dell'**Intervallo di Confidenza di Wilson**:

$$p = \frac{2N \cdot acc + Z_{\alpha/2}^2 \pm Z_{\alpha/2}\sqrt{Z_{\alpha/2}^2 + 4N \cdot acc - 4N \cdot acc^2}}{2(N + Z_{\alpha/2}^2)} \tag{22.27}$$

Esempio 22.4.1 (Impatto della Dimensione N sull'Ampiezza della Stima)**.** Si consideri un modello la cui accuratezza empirica osservata è costantemente pari ad $acc = 0.80$ (80%) al livello di confidenza del 95% ($Z_{\alpha/2} = 1.96$). La Tabella 22.4 riporta gli estremi inferiore p_{lower} e superiore p_{upper} al crescere di N.

Tabella 22.4: Intervalli di confidenza al 95% per un'accuratezza osservata dell'80% al variare di N.

Dimensione Test Set (N)	**50**	**100**	**500**	**1 000**	**5 000**
Estremo Inferiore p_{lower}	0.670	0.711	0.763	0.774	0.789
Estremo Superiore p_{upper}	0.888	0.866	0.833	0.824	0.811
Ampiezza Intervallo Δp	**0.218**	**0.155**	**0.070**	**0.050**	**0.022**

I dati evidenziano come la stima su 50 campioni sia estremamente incerta (con un margine d'errore di circa ±11%), mentre su 5000 campioni l'intervallo si contrae a un sottilissimo intervallo di ±1.1%.

22.4.2 Test di Ipotesi per il Confronto tra Due Classificatori Indipendenti

Siano dati due modelli predittivi M_1 ed M_2 valutati rispettivamente su due insiemi di test disgiunti e indipendenti $\mathcal{D}_1$ (di cardinalità n_1, con tasso d'errore osservato e_1) e $\mathcal{D}_2$ (di cardinalità n_2, con tasso d'errore osservato e_2).

I tassi di errore empirici seguono approssimativamente distribuzioni gaussiane indipendenti con varianze campionarie stimate pari a:

$$\hat{\sigma}_1^2 = \frac{e_1(1-e_1)}{n_1}, \qquad \hat{\sigma}_2^2 = \frac{e_2(1-e_2)}{n_2} \tag{22.28}$$

Definiamo la variabile stocastica differenza tra i tassi di errore:

$$d = e_1 - e_2 \tag{22.29}$$

Poiché le due partizioni sono tra loro indipendenti, la varianza della differenza è esattamente pari alla somma delle varianze individuali:

$$\hat{\sigma}_d^2 = \hat{\sigma}_1^2 + \hat{\sigma}_2^2 = \frac{e_1(1-e_1)}{n_1} + \frac{e_2(1-e_2)}{n_2} \tag{22.30}$$

L'intervallo di confidenza al livello $1-\alpha$ per la reale differenza d_t tra i tassi di errore asintotici è formulato come:

$$d_t \in \left[d - Z_{\alpha/2} \cdot \hat{\sigma}_d,\ d + Z_{\alpha/2} \cdot \hat{\sigma}_d \right] \tag{22.31}$$

Esempio 22.4.2 (Risoluzione Rigorosa del Confronto Iniziale tra M_1 ed M_2)**.** Riprendiamo i parametri del problema iniziale [2]:

- Modello M_1: $n_1 = 30$, accuratezza $acc_1 = 0.85 \implies e_1 = 1 - 0.85 = 0.15$;
- Modello M_2: $n_2 = 5\,000$, accuratezza $acc_2 = 0.75 \implies e_2 = 1 - 0.75 = 0.25$.

Impostiamo il test d'ipotesi bilaterale sulla differenza:

$$H_0 : d_t = 0 \quad \text{(Nessuna differenza reale)}, \qquad H_1 : d_t \neq 0 \tag{22.32}$$

1. **Differenza osservata:**

$$d = |e_2 - e_1| = |0.25 - 0.15| = 0.100 \tag{22.33}$$

2. **Varianza stimata della differenza:**

$$\begin{aligned} \hat{\sigma}_d^2 &= \frac{0.15(1-0.15)}{30} + \frac{0.25(1-0.25)}{5\,000} \\ &= \frac{0.1275}{30} + \frac{0.1875}{5\,000} \\ &= 0.004250 + 0.0000375 \approx 0.0042875 \end{aligned} \tag{22.34}$$

   da cui la deviazione standard congiunta risulta $\hat{\sigma}_d = \sqrt{0.0042875} \approx 0.06548$.

3. **Calcolo dell'Intervallo di Confidenza al 95% ($Z_{\alpha/2} = 1.96$):**

$$\begin{aligned} d_t &= 0.100 \pm 1.96 \cdot 0.06548 \\ &= 0.100 \pm 0.1283 \end{aligned} \tag{22.35}$$

   L'intervallo di confidenza calcolato per la reale differenza è:

$$d_t \in [-0.0283,\ +0.2283] \tag{22.36}$$

Conclusione Statistica: Poiché l'intervallo di confidenza **comprende al proprio interno il valore zero** ($0 \in [-0.0283, +0.2283]$), l'ipotesi nulla H_0 **non può essere rigettata** a livello di significatività del 5%. La differenza di prestazioni dell'85% contro il 75% **non è statisticamente significativa**: essa è perfettamente compatibile con una fluttuazione casuale dovuta alla minuscola dimensione campionaria ($N_1 = 30$) su cui è stato valutato M_1. Non vi è alcuna evidenza empirica per sostenere che M_1 generalizzi meglio di M_2.

La Tabella 22.5 riassume sinotticamente le metodologie, gli indicatori e i test formalizzati nel capitolo.

Tabella 22.5: Quadro sinottico comparativo delle metodologie di valutazione e confronto dei classificatori.

Ambito Metodologico	Strumenti e Metriche	Punto di Forza Primario	Criticità o Limite Applicativo
Metriche Globali	Accuracy, Error Rate	Immediatezza computazionale	Fuorvianti su classi fortemente sbilanciate
Metriche Asimmetriche	Precision, Recall, Misura F_1	Focalizzazione sulla classe target	Dipendono dalla scelta della soglia di taglio
Analisi Cost-Sensitive	Matrice dei Costi $C(i \mid j)$	Realismo economico applicativo	Richiede specificazione a priori dei costi reali
Analisi ad Ampio Spettro	Curve ROC, Indice AUC	Indipendenti dalla soglia e dallo sbilanciamento	Richiedono score di probabilità ben calibrati
Protocolli di Split	k-Fold CV, Stratificazione, Bootstrap	Stime stabili a bassa distorsione	Sovraccarico di calcolo su modelli complessi
Confronto di Ipotesi	Test Z, Intervalli di Wilson	Distinzione tra segnale reale e rumore	Richiedono campioni di test indipendenti ($N > 30$)

Capitolo 23

Metodi Ensemble, Reti Neurali Artificiali e Backpropagation

Il presente capitolo approfondisce due delle famiglie algoritmiche più potenti e diffuse nell'ambito dell'apprendimento supervisionato: i **Metodi Ensemble** (*Ensemble Methods*) e le **Reti Neurali Artificiali** (*Artificial Neural Networks*, ANN) addestrate tramite l'algoritmo di retropropagazione dell'errore (*Backpropagation*) [2]. Nella terza parte viene presentata una trattazione analitica passo-passo di un ciclo completo di addestramento su una rete multistrato.

23.1 Architettura Generale e Fondamenti Teorici degli Ensemble

I metodi ensemble costituiscono un paradigma di machine learning finalizzato a elevare l'accuratezza predittiva combinando le decisioni prodotte da una pluralità di modelli di base (*base classifiers* o *weak learners*) [2]. Invece di selezionare un singolo classificatore ottimo, l'approccio ensemble costruisce una coalizione di modelli a partire dai dati di addestramento e formula la decisione finale attraverso un meccanismo di aggregazione o voto.

23.1.1 Flusso Operativo e Principio della Saggezza della Folla

Il processo di induzione e inferenza di un ensemble si articola in tre macro-stadi sequenziali, illustrati nello schema di Figura 23.1:

1. **Generazione di Dataset Multipli:** a partire dal dataset originario $\mathcal{D}$ di cardinalità N, vengono generati T sottoinsiemi di training $\mathcal{D}_1, \mathcal{D}_2, \ldots, \mathcal{D}_T$ mediante opportune procedure di campionamento o perturbazione;

2. **Induzione dei Classificatori di Base:** su ciascun dataset $\mathcal{D}_t$ viene addestrato in modo indipendente o sequenziale un classificatore di base C_t $(t = 1, \ldots, T)$;

3. **Aggregazione delle Decisioni:** per ogni record non etichettato $\mathbf{x}$, le predizioni individuali $C_1(\mathbf{x}), \ldots, C_T(\mathbf{x})$ vengono composte mediante voto a maggioranza (semplice o pesato) o media probabilistica, producendo il classificatore ensemble finale $C^*(\mathbf{x})$.

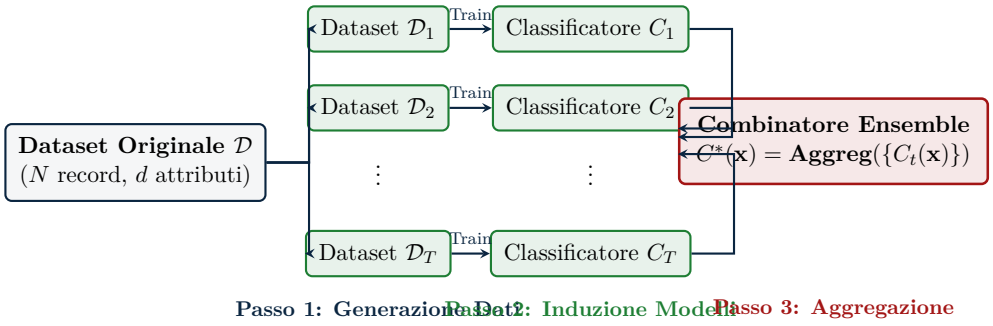


Figura 23.1: Architettura computazionale a tre stadi di un sistema di classificazione ensemble.

Il successo teorico dei metodi ensemble risiede nel principio della cosiddetta **Saggezza della Folla** (*Wisdom of Crowd*) trasposto nell'apprendimento automatico: combinare le predizioni di agenti eterogenei e indipendenti produce una stima statisticamente superiore a quella del miglior singolo componente. A differenza di una folla umana disorganizzata, il machine learning sfrutta i dati supervisionati per garantire due condizioni essenziali:

- **Accuratezza Individuale Superiore al Caso:** ciascun classificatore di base deve possedere una probabilità di errore $e < 0.5$;

- **Diversità ed Indipendenza degli Errori:** i classificatori devono formulare errori non correlati, affinché gli sbagli individuali vengano neutralizzati dalla maggioranza concorde.

23.1.2 Analisi Matematica del Voto a Maggioranza

Consideriamo un comitato di $T = 25$ classificatori binari indipendenti, ciascuno avente la medesima probabilità di commettere un errore di classificazione pari a e. L'ensemble assegna la classe mediante voto a maggioranza (*majority voting*), commettendo un errore globale se e solo se almeno la metà più uno dei classificatori (ossia almeno 13 modelli su 25) fallisce contemporaneamente.

Distinguiamo due regimi asintotici fondamentali:

1. **Classificatori Perfettamente Correlati (Identici):** Se tutti i 25 classificatori sono identici, essi condividono le medesime superfici di separazione. Quando uno sbaglia, tutti gli altri sbagliano simultaneamente. Di conseguenza, l'errore dell'ensemble coincide con l'errore del singolo:
$$P_{\text{ens}}(\text{identici}) = e \tag{23.1}$$
Il voto a maggioranza tra copie identiche non apporta alcun beneficio informativo.

2. **Classificatori Mutuamente Indipendenti:** Assumendo che gli eventi di errore dei 25 classificatori siano statisticamente indipendenti, il numero di modelli errati K segue una distribuzione binomiale $\text{Bin}(T = 25, e)$. La probabilità che l'ensemble fallisca è data dalla coda della distribuzione binomiale:
$$P_{\text{ens}}(e) = \sum_{k=13}^{25} \binom{25}{k} e^k (1-e)^{25-k} \tag{23.2}$$

Esempio 23.1.1 (Abbattimento dell'Errore per $e = 0.35$)**.** Supponiamo che ogni singolo classificatore di base sia modesto ma migliore del caso casuale, con un tasso di errore $e = 0.35$ (35%). Applicando l'equazione (23.2) per $T = 25$:

$$\begin{aligned} P_{\text{ens}} &= \sum_{k=13}^{25} \binom{25}{k} (0.35)^k (0.65)^{25-k} \\ &= \binom{25}{13} (0.35)^{13} (0.65)^{12} + \cdots + (0.35)^{25} \approx 0.060 \end{aligned} \tag{23.3}$$

Il tasso di errore crolla drasticamente dal 35% al 6.0%, testimoniando il potere di riduzione della varianza del voto a maggioranza.

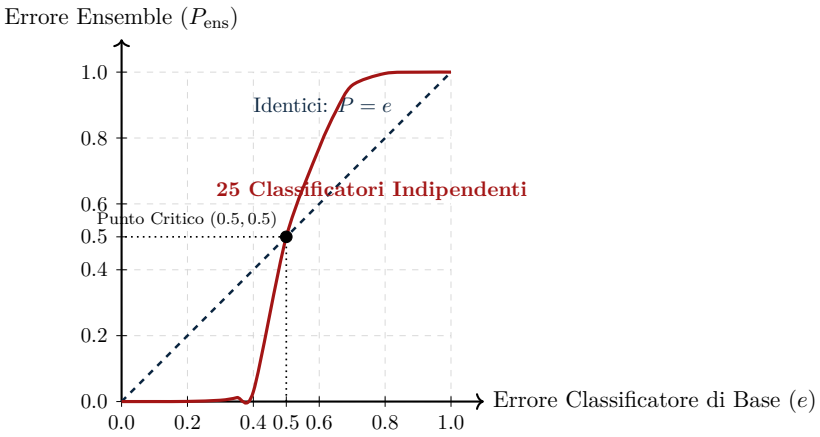


Figura 23.2: Confronto tra l'errore del classificatore di base e l'errore dell'ensemble al variare del grado di indipendenza ($T = 25$).

L'analisi del grafico di Figura 23.2 evidenzia il comportamento asintotico del voto a maggioranza in funzione della qualità dei singoli classificatori:

- **Regime Favorevole ($e < 0.5$):** Ciascun classificatore formula una risposta corretta con probabilità maggiore rispetto a una errata. Il voto a maggioranza amplifica la probabilità di successo, schiacciando la curva dell'ensemble ben al di sotto della diagonale identica ($P_{\text{ens}} \ll e$).
- **Punto di Flesso Neutro ($e = 0.5$):** Se i classificatori operano a livello puramente casuale, l'aggregazione non può estrarre alcuna informazione strutturale: $P_{\text{ens}} = 0.5$.
- **Regime Catastrofico ($e > 0.5$):** Se i classificatori falliscono più del 50% delle volte, il voto a maggioranza indipendentemente amplifica l'errore sistematico, producendo prestazioni drammaticamente peggiori rispetto a quelle dei singoli componenti ($P_{\text{ens}} > e$).

23.1.3 Tassonomia dei Metodi Ensemble

Per garantire che i classificatori di base commettano errori non correlati, le metodologie ensemble operano introducendo perturbazioni mirate nei diversi componenti del processo di addestramento:

1. **Manipolazione della Distribuzione dei Record:** Si modificano le probabilità di estrazione o i pesi assegnati alle istanze di training. Le tecniche cardine sono il **Bagging** (estrazione casuale con reinserimento) e il **Boosting** (pesatura adattiva guidata dagli errori);

2. **Manipolazione dello Spazio delle Caratteristiche (*Feature Subsampling*):** Ciascun modello riceve in ingresso un sottoinsieme casuale dei predittori, rompendo le correlazioni spurie indotte da attributi dominanti (meccanismo fondante delle **Random Forests**);

3. **Manipolazione delle Etichette di Classe (*Output Coding*):** In problemi a molte classi, si converte il problema multiclasse in molteplici sottoproblemi binari mediante matrici di codifica ridondanti a correzione d'errore (*Error-Correcting Output Codes*, ECOC).

23.2 Bagging (*Bootstrap Aggregating*)

Proposto da Leo Breiman nel 1996 [51], il **Bagging** (*Bootstrap AGGregatING*) è un algoritmo ensemble non sequenziale basato sulla teoria statistica del campionamento bootstrap.

23.2.1 Campionamento Bootstrap, Algoritmo e Proprietà Out-Of-Bag

Dato un dataset di addestramento $\mathcal{D}$ composto da n osservazioni, una replica bootstrap $\mathcal{D}_i$ è generata effettuando n estrazioni casuali **con reinserimento** (*sampling with replacement*) da $\mathcal{D}$. Ciascuna replica possiede la medesima cardinalità n del dataset originario; tuttavia, a causa del reinserimento, alcune istanze originali compariranno ripetute più volte, mentre altre verranno escluse.

Definizione 23.2.1 (Algoritmo di Bagging)**.** Fissato il numero k di repliche bootstrap da istanziare:

1. Per $i = 1, \dots, k$:
   - Si estrae un campione bootstrap $\mathcal{D}_i$ di dimensione n con reinserimento da $\mathcal{D}$;
   - Si addestra il classificatore di base C_i sul campione $\mathcal{D}_i$.
2. In fase di inferenza su un'istanza non etichettata $\mathbf{x}$, la predizione finale $C^*(\mathbf{x})$ è determinata dal voto a maggioranza non ponderato:

$$C^*(\mathbf{x}) = \arg\max_{y \in \mathcal{Y}} \sum_{i=1}^{k} \delta\Big(C_i(\mathbf{x}) = y\Big) \tag{23.4}$$

   dove $\delta(\cdot)$ indica la funzione indicatrice booleana.

Teorema 23.2.1 (Frazione Asintotica Out-Of-Bag)**.** *La probabilità che una specifica istanza $x \in \mathcal{D}$ non venga mai estratta in un singolo round di n estrazioni indipendenti con reinserimento converge asintoticamente a:*

$$P(\textit{Esclusa}) = \left(1 - \frac{1}{n}\right)^n \xrightarrow{n\to\infty} \frac{1}{e} \approx 0.368 = 36.8\% \tag{23.5}$$

Di conseguenza, ogni campione bootstrap contiene in media il 63.2% *delle istanze uniche di $\mathcal{D}$, lasciando un restante* 36.8% *di campioni non utilizzati, denominati campioni* ***Out-Of-Bag*** *(OOB).*

L'insieme OOB associato a ciascun classificatore C_i funge da test set naturale: valutando ogni record unicamente sui modelli nei cui campioni bootstrap non compare, è possibile stimare l'errore di generalizzazione dell'ensemble in modo non distorto senza dover sacrificare dati per un validation set separato.

23.2.2 Esempio Completo di Bagging su Dataset Monodimensionale

Per chiarire il meccanismo con cui il bagging modella superfici decisionali complesse, consideriamo il problema didattico monodimensionale riportato nella Tabella 23.1, composto da $n = 10$ punti $X \in [0.1, 1.0]$ con etichette binarie $y \in \{+1, -1\}$ [2].

Tabella 23.1: Dataset didattico monodimensionale con etichette binarie $y \in \{+1, -1\}$.

Coordinata X	**0.1**	**0.2**	**0.3**	**0.4**	**0.5**	**0.6**	**0.7**	**0.8**	**0.9**	**1.0**
Classe Reale y	$+1$	$+1$	$+1$	-1	-1	-1	-1	$+1$	$+1$	$+1$

Il classificatore di base prescelto è un **decision stump** (albero a soglia singola a un solo livello), la cui regola decisionale ha la forma $x \leq k \implies y_{\text{left}}$ e $x > k \implies y_{\text{right}}$. Nessun singolo decision stump può classificare correttamente questo dataset, poiché la regione centrale di classe -1 è racchiusa tra due regioni separate di classe $+1$: qualunque taglio lineare commetterà almeno 3 errori su 10.

Eseguendo 10 round di campionamento bootstrap e inducendo su ciascuno uno stump ottimo in termini di entropia, si ottengono i 10 classificatori descritti nella Tabella 23.2.

Tabella 23.2: Specifiche dei 10 decision stump indotti sui rispettivi campioni bootstrap.

Round (i)	**Soglia di Split** (k)	**Ramo Sinistro** ($x \leq k$)	**Ramo Destro** ($x > k$)
1	0.35	$+1$	-1
2	0.70	$+1$	$+1$
3	0.35	$+1$	-1
4	0.30	$+1$	-1
5	0.35	$+1$	-1
6	0.75	-1	$+1$
7	0.75	-1	$+1$
8	0.75	-1	$+1$
9	0.75	-1	$+1$
10	0.05	$+1$	$+1$

La Tabella 23.3 riporta la traccia completa delle predizioni emesse da ciascuno dei 10 modelli sulle 10 istanze originarie, la somma algebrica dei voti e la previsione aggregata sign(Sum).

Tabella 23.3: Matrice dei voti e classificazione finale dell'ensemble Bagging sui 10 record originali.

Round	$x = 0.1$	$x = 0.2$	$x = 0.3$	$x = 0.4$	$x = 0.5$	$x = 0.6$	$x = 0.7$	$x = 0.8$	$x = 0.9$	$x = 1.0$
1	$+1$	$+1$	$+1$	-1	-1	-1	-1	-1	-1	-1
2	$+1$	$+1$	$+1$	$+1$	$+1$	$+1$	$+1$	$+1$	$+1$	$+1$
3	$+1$	$+1$	$+1$	-1	-1	-1	-1	-1	-1	-1
4	$+1$	$+1$	$+1$	-1	-1	-1	-1	-1	-1	-1
5	$+1$	$+1$	$+1$	-1	-1	-1	-1	-1	-1	-1
6	-1	-1	-1	-1	-1	-1	-1	$+1$	$+1$	$+1$
7	-1	-1	-1	-1	-1	-1	-1	$+1$	$+1$	$+1$
8	-1	-1	-1	-1	-1	-1	-1	$+1$	$+1$	$+1$
9	-1	-1	-1	-1	-1	-1	-1	$+1$	$+1$	$+1$
10	$+1$	$+1$	$+1$	$+1$	$+1$	$+1$	$+1$	$+1$	$+1$	$+1$
Somma	$\mathbf{+2}$	$\mathbf{+2}$	$\mathbf{+2}$	$\mathbf{-6}$	$\mathbf{-6}$	$\mathbf{-6}$	$\mathbf{-6}$	$\mathbf{+2}$	$\mathbf{+2}$	$\mathbf{+2}$
Predizione	$\mathbf{+1}$	$\mathbf{+1}$	$\mathbf{+1}$	$\mathbf{-1}$	$\mathbf{-1}$	$\mathbf{-1}$	$\mathbf{-1}$	$\mathbf{+1}$	$\mathbf{+1}$	$\mathbf{+1}$

Risultato Fondamentale: L'aggregazione Bagging raggiunge un'accuratezza del 100% (10 predizioni corrette su 10). La combinazione di modelli con tagli sinistri ($k \approx 0.35$) e tagli destri ($k \approx 0.75$) ha generato implicitamente una frontiera decisionale a intervallo chiuso $[0.35, 0.75]$, impossibile da rappresentare per un singolo decision stump.

23.2.3 Proprietà di Riduzione della Varianza e Limiti del Bagging

I punti di forza e le criticità strutturali del Bagging possono essere così sintetizzati:

- **Riduzione della Varianza:** Il bagging è straordinariamente efficace quando applicato a *classificatori instabili* (quali alberi di decisione profondi e non potati), nei quali una lieve perturbazione nei dati di training genera una topologia ad albero radicalmente differente. L'aggregazione smussa le oscillazioni riducendo drasticamente la varianza dello stimatore;

- **Parallelizzazione Massiva:** Poiché le k repliche bootstrap sono generate a priori in modo indipendente, l'addestramento dei k modelli può essere interamente distribuito in parallelo;

- **Invarianza del Bias:** Il bagging non riduce la distorsione strutturale (*bias*). Se i modelli di base sono eccessivamente semplificati o affetti da grave underfitting, l'ensemble erediterà l'incapacità di modellare pattern complessi;

- **Perdita di Interpretabilità:** A fronte del guadagno in accuratezza, l'insieme di k alberi cessa di essere facilmente ispezionabile rispetto a un singolo albero interpretabile.

23.3 Boosting e l'Algoritmo AdaBoost

A differenza del bagging, il **Boosting** è una procedura iterativa e adattiva ideata da Freund e Schapire [52] per convertire una sequenza di classificatori deboli (*weak learners*) in un singolo predittore forte (*strong learner*).

23.3.1 Principio Iterativo del Boosting

Nel boosting i classificatori di base vengono addestrati in modo strettamente **sequenziale**. A ogni istanza del training set è associato un peso numerico che indica il grado di criticità del record nel processo di apprendimento:

- All'iterazione iniziale $t = 1$, a tutti gli N record viene attribuito il medesimo peso uniforme $w_j^{(1)} = 1/N$;

- Al termine del round t, le predizioni del classificatore corrente C_t vengono verificate sui dati: i pesi delle istanze classificate **erroneamente** vengono **incrementati**, mentre quelli delle istanze classificate **correttamente** vengono **ridotti**;

- Il classificatore successivo C_{t+1} sarà indotto su una distribuzione campionaria in cui i pattern precedentemente falliti risultano dominanti,

forzando l'algoritmo a focalizzarsi sulle regioni più complesse dello spazio dei dati.

23.3.2 Formulazione Analitica di AdaBoost

L'algoritmo **AdaBoost** (*Adaptive Boosting*) formalizza questo meccanismo impiegando un voto a maggioranza pesato in cui ciascun classificatore acquisisce un'influenza proporzionale alla propria accuratezza.

Dati N campioni $\{(\mathbf{x}_j, y_j)\}_{j=1}^{N}$ con etichette $y_j \in \{-1, +1\}$ e pesi normalizzati $\sum_{j=1}^{N} w_j^{(t)} = 1$:

1. **Tasso di Errore Pesato (ϵ_t):** Misura la frazione di peso attribuita ai record falliti dal classificatore C_t:

$$\epsilon_t = \sum_{j=1}^{N} w_j^{(t)} \delta\Big(C_t(\mathbf{x}_j) \neq y_j\Big) \tag{23.6}$$

2. **Importanza del Classificatore (α_t):** Il potere decisionale attribuito a C_t è definito dalla funzione logaritmica:

$$\alpha_t = \frac{1}{2} \ln\left(\frac{1 - \epsilon_t}{\epsilon_t}\right) \tag{23.7}$$

   L'analisi della formula evidenzia che:

   - Se $\epsilon_t \to 0$ (modello quasi perfetto), $\alpha_t \to +\infty$;
   - Se $\epsilon_t = 0.5$ (accuratezza casuale), $\alpha_t = \frac{1}{2}\ln(1) = 0$ (nessun peso nel voto finale);
   - Se $\epsilon_t > 0.5$, l'algoritmo arresta l'iterazione oppure inverte il segno della classificazione con $\alpha_t < 0$.

3. **Regola di Aggiornamento dei Pesi delle Istanze:** Per ciascun esempio $\mathbf{x}_j$, il peso per l'iterazione successiva $t+1$ è:

$$w_j^{(t+1)} = \frac{w_j^{(t)}}{Z_t} \exp\Big(-\alpha_t y_j C_t(\mathbf{x}_j)\Big) = \frac{w_j^{(t)}}{Z_t} \begin{cases} e^{-\alpha_t} & \text{se } C_t(\mathbf{x}_j) = y_j \\ e^{+\alpha_t} & \text{se } C_t(\mathbf{x}_j) \neq y_j \end{cases} \tag{23.8}$$

   dove il termine di normalizzazione Z_t garantisce $\sum_{j=1}^{N} w_j^{(t+1)} = 1$:

$$Z_t = \sum_{j=1}^{N} w_j^{(t)} \exp\Big(-\alpha_t y_j C_t(\mathbf{x}_j)\Big) = 2\sqrt{\epsilon_t(1-\epsilon_t)} \tag{23.9}$$

4. **Classificazione Finale tramite Voto a Maggioranza Pesato:** Al termine dei T round, il comitato finale combina le predizioni ponderate:

$$C^*(\mathbf{x}) = \text{sign}\left(\sum_{t=1}^{T} \alpha_t C_t(\mathbf{x})\right) \tag{23.10}$$

23.3.3 Esempio Completo di AdaBoost sul Dataset Monodimensionale

Applichiamo AdaBoost sullo stesso dataset didattico della Tabella 23.1 ($N = 10$ record) impiegando decision stump.

1. **Round 1:** Pesi iniziali uniformi $w_j^{(1)} = 0.100$ per ogni j. Lo stump ottimo pone lo split a $k = 0.75$ con $x \leq 0.75 \implies -1$ e $x > 0.75 \implies +1$. Tale modello commette errore sui primi 3 campioni ($x \in \{0.1, 0.2, 0.3\}$ la cui classe vera è $+1$):

$$\epsilon_1 = 3 \times 0.100 = 0.300,$$
$$\alpha_1 = \frac{1}{2}\ln\left(\frac{0.700}{0.300}\right) \approx 0.4236 \quad \text{(ovvero } \alpha_1 \approx 1.738 \text{ non norm.)}.$$

   I pesi dei campioni errati salgono a 0.311, mentre i pesi dei campioni corretti scendono a 0.010.

2. **Round 2:** Il nuovo dataset pesato assegna oltre il 93% della probabilità complessiva ai soli tre record $\{0.1, 0.2, 0.3\}$. Lo stump ottimo si adatta ponendo lo split a $k = 0.05$ classificando l'intero spazio come $+1$. L'errore pesato è ridotto e il peso attribuito al classificatore è $\alpha_2 = 2.7784$.

3. **Round 3:** I campioni centrali errati $\{0.4, 0.5, 0.6, 0.7\}$ vengono amplificati a un peso di 0.228 ciascuno. Lo stump ottimo individua il taglio $k = 0.30$ con ramo sinistro $+1$ e ramo destro -1, con coefficiente $\alpha_3 = 4.1195$.

La Tabella 23.4 riassume la matrice dei contributi pesati $\alpha_t C_t(x)$ e la decisione aggregata finale.

Tabella 23.4: Traccia analitica del voto pesato AdaBoost sui 10 record del dataset 1D.

Classificatore (α_t)	$x = 0.1$	$x = 0.2$	$x = 0.3$	$x = 0.4$	$x = 0.5$	$x = 0.6$	$x = 0.7$	$x = 0.8$	$x = 0.9$	$x = 1.0$
C_1 ($\alpha_1 = 1.738$)	-1.738	-1.738	-1.738	-1.738	-1.738	-1.738	-1.738	$+1.738$	$+1.738$	$+1.738$
C_2 ($\alpha_2 = 2.778$)	$+2.778$	$+2.778$	$+2.778$	$+2.778$	$+2.778$	$+2.778$	$+2.778$	$+2.778$	$+2.778$	$+2.778$
C_3 ($\alpha_3 = 4.120$)	$+4.120$	$+4.120$	$+4.120$	-4.120	-4.120	-4.120	-4.120	-4.120	-4.120	-4.120
Somma Pesata $\sum$	$\mathbf{+5.160}$	$\mathbf{+5.160}$	$\mathbf{+5.160}$	$\mathbf{-3.080}$	$\mathbf{-3.080}$	$\mathbf{-3.080}$	$\mathbf{-3.080}$	$\mathbf{+0.397}$	$\mathbf{+0.397}$	$\mathbf{+0.397}$
Classe Predetta	$\mathbf{+1}$	$\mathbf{+1}$	$\mathbf{+1}$	$\mathbf{-1}$	$\mathbf{-1}$	$\mathbf{-1}$	$\mathbf{-1}$	$\mathbf{+1}$	$\mathbf{+1}$	$\mathbf{+1}$

Efficienza Comparativa: Mentre il Bagging ha necessitato di 10 round per stabilizzare la votazione, AdaBoost raggiunge il 100% **di accuratezza in soli 3 round**, dimostrando l'efficacia della guida adattiva dei residui.

23.3.4 Vantaggi e Limiti del Boosting

- **Capacità di Riduzione del Bias:** A differenza del bagging, il boosting riduce sistematicamente sia la distorsione che la varianza, consentendo a modelli elementari (come decision stumps) di assemblare frontiere decisionali altamente non lineari;
- **Sensibilità al Rumore e agli Outlier:** Poiché l'algoritmo incrementa esponenzialmente i pesi delle istanze con errata predizione, in presenza di rumore sull'etichetta o di outlier irrisolvibili AdaBoost concentrerà tutte le risorse sui pattern errati, incorrendo in un severo degrado da overfitting;
- **Vincolo di Sequenzialità:** L'induzione del modello $t+1$ richiede tassativamente la conoscenza degli errori commessi dal modello t, precludendo la parallelizzazione naturale del processo di training.

23.4 Random Forests

Introdotte da Leo Breiman nel 2001 [53], le **Random Forests** costituiscono una delle tecniche di apprendimento automatico più competitive per dati tabulari, combinando i vantaggi del bagging con una procedura di decorrelazione strutturale degli alberi di decisione.

23.4.1 Il Problema della Correlazione e il Meccanismo di Decorrelazione

Se si applica il semplice bagging a un insieme di alberi di decisione, si osserva frequentemente un fenomeno patologico: qualora nel dataset esista un attributo fortemente discriminante, tale variabile verrà sistematicamente selezionata come test di split nella radice (o nei livelli immediatamente superiori) di quasi tutti gli alberi bootstrap. Di conseguenza:

$$\mathrm{Cov}(C_i, C_j) \gg 0 \tag{23.11}$$

Gli alberi risultano fortemente correlati tra loro. Ricordando che la varianza della media di T variabili casuali con varianza σ^2 e correlazione a coppie ρ è

data da:

$$\text{Var}\left(\frac{1}{T}\sum_{i=1}^{T} X_i\right) = \rho\sigma^2 + \frac{1-\rho}{T}\sigma^2 \tag{23.12}$$

all'aumentare di $T \to \infty$ il secondo termine decade a zero, ma il termine $\rho\sigma^2$ persiste come limite inferiore invalicabile. Per abbattere ulteriormente la varianza è indispensabile minimizzare il coefficiente di correlazione $\rho \to 0$.

Definizione 23.4.1 (I Due Livelli di Casualità nelle Random Forests)**.** Le Random Forests introducono due perturbazioni stocastiche indipendenti per massimizzare la diversità degli stimatori:

1. **Campionamento Casuale delle Istanze:** Ciascun albero viene addestrato su una replica bootstrap indipendente di dimensione N estratta con reinserimento dai dati originari;

2. **Campionamento Casuale delle Caratteristiche (*Feature Subsampling*):** In corrispondenza di **ogni singolo nodo interno** dell'albero, l'algoritmo seleziona casualmente senza reinserimento un sottoinsieme ristretto di m attributi tra i d disponibili ($m \ll d$). Il test di split ottimo viene ricercato esclusivamente all'interno di questo sottoinsieme candidato di taglia m.

In letteratura e nelle implementazioni standard, i valori euristici raccomandati per m sono:

$$m_{\text{classificazione}} \approx \sqrt{d}, \qquad m_{\text{regressione}} \approx \frac{d}{3} \quad \text{oppure} \quad \lfloor \log_2 d + 1 \rfloor \tag{23.13}$$

Se $d = 100$ attributi, a ogni bivio l'albero compete solo su $m = 10$ attributi estratti casualmente. Questa forzatura impedisce all'attributo globalmente dominante di monopolizzare la radice, costringendo gli alberi a esplorare percorsi alternativi basati su predittori secondari.

23.4.2 Proprietà Operative, Importanza delle Variabili e Limiti

Le Random Forests presentano caratteristiche computazionali uniche:

- **Immunità all'Overfitting rispetto a T:** Aumentare il numero di alberi T nella foresta non induce overfitting. Per la legge dei grandi numeri, l'errore di generalizzazione converge asintoticamente a un limite superiore costante all'aumentare di T;

- **Stima Interna dell'Errore OOB:** La foresta valuta le proprie prestazioni senza bisogno di cross-validation esterna, impiegando per ogni istanza unicamente il voto espresso dagli alberi in cui essa è risultata Out-Of-Bag;
- **Importanza delle Variabili (*Variable Importance*):** La rilevanza di un attributo X_j può essere quantificata valutando il decremento medio di accuratezza OOB quando i valori di X_j vengono permutati casualmente (*permutation importance*), rompendo la relazione funzionale tra predittore e target;
- **Criticità con Molte Feature Irrilevanti:** Quando il numero complessivo di attributi d è molto elevato ma solo una frazione minuscola è autenticamente predittiva, la probabilità che uno split casuale contenga almeno una variabile utile diviene trascurabile ($P \approx m \cdot \frac{d_{\text{utili}}}{d} \ll 1$), provocando il collasso qualitativo dei singoli alberi.

23.5 Ispirazione Biologica, Neurone Artificiale e Percettrone

Le **Reti Neurali Artificiali** (*Artificial Neural Networks*, ANN) costituiscono un paradigma computazionale ispirato alla fisiologia dei sistemi nervosi centrali, caratterizzati da miliardi di unità cellulari elementari densamente interconnesse [54].

23.5.1 La Metafora Biologica e il Modello del Neurone Artificiale

Nel tessuto cerebrale, il neurone biologico riceve potenziali di membrana attraverso ramificazioni dendritiche (*dendrites*), integra tali segnali nel corpo cellulare o soma, e quando il potenziale supera una soglia biochimica emette un potenziale d'azione lungo l'assone (*axon*). La connessione inter-neuronale prende il nome di **sinapsi** (*synapse*). Nel cervello, i meccanismi di plasticità e memorizzazione si attuano attraverso la modulazione dell'efficacia sinaptica (la forza con cui un segnale presinaptico eccita o inibisce il neurone ricevente).

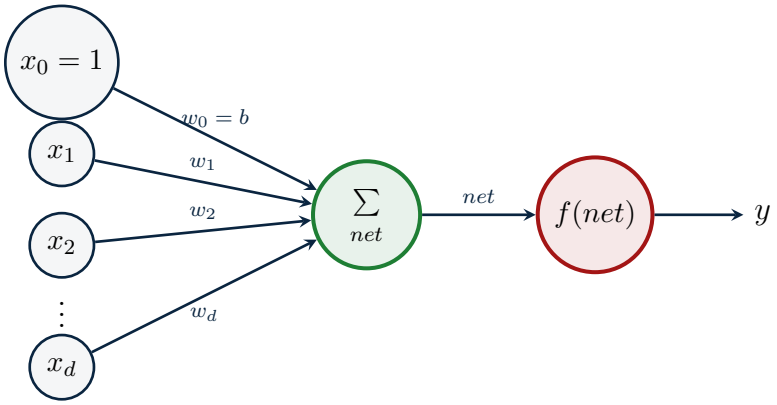


Figura 23.3: Modello matematico generale del neurone artificiale con termine di bias integrato.

Nel neurone artificiale (Figura 23.3), i segnali di ingresso $x_1, \ldots, x_d$ vengono moltiplicati per i rispettivi pesi sinaptici $w_1, \ldots, w_d$. L'aggregazione algebrica interna (*net input*) confronta la combinazione lineare con una soglia t:

$$net = \sum_{i=1}^{d} w_i x_i - t = \sum_{i=0}^{d} w_i x_i = \mathbf{w}^T \mathbf{x} \tag{23.14}$$

avendo introdotto il nodo fittizio costante $x_0 = 1$ associato al peso di **bias** $w_0 = b = -t$. L'uscita è prodotta da una funzione di attivazione $y = f(net)$.

23.5.2 Il Classificatore Percettrone e Regola di Apprendimento Delta

Il **Percettrone** (*Perceptron*), formalizzato da Frank Rosenblatt nel 1958 [54], adotta una funzione di attivazione discontinua a segno:

$$y = \text{sign}(net) = \begin{cases} +1 & \text{se } net \geq 0 \\ -1 & \text{se } net < 0 \end{cases} \tag{23.15}$$

Dato un insieme di addestramento $\{(\mathbf{x}_i, y_i)\}$, i pesi vengono aggiornati iterativamente per ogni esempio mediante la **Regola di Apprendimento del Percettrone**:

$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} + \lambda \Big(y_i - f(\mathbf{w}^{(k)}, \mathbf{x}_i) \Big) \mathbf{x}_i \tag{23.16}$$

dove $\lambda \in (0, 1]$ è il **tasso di apprendimento** (*learning rate*). Analizziamo la dinamica dell'errore $e = y_i - f(\mathbf{x}_i) \in \{0, \pm 2\}$:

- Se la predizione è corretta ($y_i = f(\mathbf{x}_i)$), $e = 0 \implies \Delta\mathbf{w} = \mathbf{0}$;
- Se $y_i = +1$ e $f(\mathbf{x}_i) = -1$, $e = +2 \implies \Delta\mathbf{w} = +2\lambda\mathbf{x}_i$, ruotando il vettore dei pesi nella direzione di $\mathbf{x}_i$ per aumentare *net*;
- Se $y_i = -1$ e $f(\mathbf{x}_i) = +1$, $e = -2 \implies \Delta\mathbf{w} = -2\lambda\mathbf{x}_i$, ruotando $\mathbf{w}$ in direzione opposta a $\mathbf{x}_i$ per ridurre *net*.

Nel caso di uscite binarie standard $\{0, 1\}$ con soglia esplicita b, l'aggiornamento assume la forma della classica **Regola Delta** (*Delta Rule*):

$$w_i^{\text{new}} = w_i^{\text{old}} + \lambda(t - y)x_i, \qquad b^{\text{new}} = b^{\text{old}} - \lambda(t - y) \tag{23.17}$$

23.5.3 Esempio Completo di Apprendimento: L'Operatore Logico AND

Illustriamo la convergenza della regola delta addestrando un percettrone a due ingressi $x_1, x_2 \in \{0, 1\}$ sull'operatore booleano **AND**:

$$\mathcal{D} = \Big\{((0,0),0),\ ((0,1),0),\ ((1,0),0),\ ((1,1),1)\Big\}$$

con $\lambda = 1$, inizializzazione nulla $w_1 = w_2 = b = 0$, ingresso netto $z = w_1x_1 + w_2x_2 - b$ e attivazione a gradino $y = 1$ se $z > 0$, $y = 0$ se $z \leq 0$ [2].

- **Epoca 1:**
  - $(0,0), t = 0 \implies z = 0 \implies y = 0$ (corretto) $\implies w = (0,0), b = 0$;
  - $(0,1), t = 0 \implies z = 0 \implies y = 0$ (corretto) $\implies w = (0,0), b = 0$;
  - $(1,0), t = 0 \implies z = 0 \implies y = 0$ (corretto) $\implies w = (0,0), b = 0$;
  - $(1,1), t = 1 \implies z = 0 \implies y = 0$ (errore) $\implies \Delta w_1 = 1, \Delta w_2 = 1, \Delta b = -1 \implies w = (1,1), b = -1$.
- **Epoca 2:**
  - $(0,0), t = 0 \implies z = 1 \implies y = 1$ (errore) $\implies \Delta b = +1 \implies w = (1,1), b = 0$;
  - $(0,1), t = 0 \implies z = 1 \implies y = 1$ (errore) $\implies \Delta w_2 = -1, \Delta b = +1 \implies w = (1,0), b = 1$;

- $(1,0), t=0 \implies z=0 \implies y=0$ (corretto) $\implies w=(1,0), b=1$;
- $(1,1), t=1 \implies z=0 \implies y=0$ (errore) $\implies \Delta w_1=1, \Delta w_2=1, \Delta b=-1 \implies w=(2,1), b=0$.

- **Epoca 3:**
  - $(0,0) \implies z=0 \implies y=0$ (ok);
  - $(0,1) \implies z=1 \implies y=1$ (err) $\implies w=(2,0), b=1$;
  - $(1,0) \implies z=1 \implies y=1$ (err) $\implies w=(1,0), b=2$;
  - $(1,1) \implies z=-1 \implies y=0$ (err) $\implies w=(2,1), b=1$.

- **Epoca 4:**
  - $(0,0) \implies z=-1 \implies y=0$ (ok);
  - $(0,1) \implies z=0 \implies y=0$ (ok);
  - $(1,0) \implies z=1 \implies y=1$ (err) $\implies w=(1,1), b=2$;
  - $(1,1) \implies z=0 \implies y=0$ (err) $\implies w=(2,2), b=1$.

- **Epoca 5:**
  - $(0,0) \implies z=-1 \implies y=0$ (ok);
  - $(0,1) \implies z=1 \implies y=1$ (err) $\implies w=(2,1), b=2$;
  - $(1,0) \implies z=0 \implies y=0$ (ok);
  - $(1,1) \implies z=1 \implies y=1$ (ok).

- **Epoca 6 (Convergenza):** Con i parametri finali stabiliti $w_1=2, w_2=1, b=2$ (retta di separazione $2x_1+x_2-2=0$): tutti e quattro i pattern generano uscite identiche al target ($\Delta w_1=\Delta w_2=\Delta b=0$). L'algoritmo converge con successo.

23.5.4 Separabilità Lineare e il Limite Storico del Problema XOR

La superficie di decisione di un percettrone è l'iperpiano di equazione $\mathbf{w}^T\mathbf{x}=0$, che nel piano bidimensionale corrisponde alla retta:

$$x_2 = -\frac{w_1}{w_2}x_1 + \frac{b}{w_2} \tag{23.18}$$

Per il **Teorema di Convergenza del Percettrone** (Novikoff, 1962), se l'insieme di dati è linearmente separabile, l'algoritmo converge in un numero finito di passi; se i dati non sono linearmente separabili, la procedura oscilla all'infinito senza mai convergere.

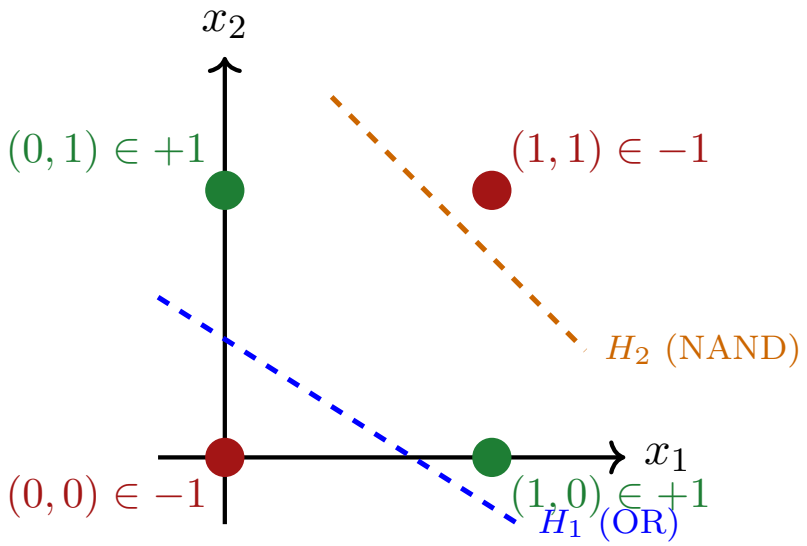


Figura 23.4: Il problema dell'OR Esclusivo (XOR): non linearmente separabile con una retta singola, ma risolvibile mediante due iperpiani congiunti H_1 e H_2.

Come illustrato in Figura 23.4, l'operatore **OR Esclusivo (XOR)** pone i campioni di classe negativa lungo la diagonale principale $(0,0)$ e $(1,1)$, e quelli di classe positiva sui vertici opposti $(0,1)$ e $(1,0)$. Nessuna retta singola può separare le due classi nel piano.

La Soluzione degli Strati Nascosti: Introducendo uno strato intermedio con due neuroni nascosti o_1 e o_2, ciascun neurone realizza un singolo iperpiano di separazione (ad esempio una retta di tipo OR H_1 e una retta di tipo NAND H_2). Lo strato nascosto esegue una mappatura non lineare $\mathbf{x} \mapsto (o_1, o_2)$ verso un nuovo spazio geometrico bidimensionale nel quale le classi risultano linearmente separabili dal neurone di output.

23.6 Reti Neurali Multistrato e Funzioni di Attivazione

Una **Rete Neurale Multistrato** (*Multilayer Perceptron*, MLP) supera i limiti del percettrone singolo strutturando i neuroni in una gerarchia di strati successivi:

1. **Input Layer:** riceve i predittori grezzi senza applicare trasformazioni;
2. **Hidden Layers:** uno o più strati intermedi che estraggono rappresentazioni latenti progressivamente più astratte;
3. **Output Layer:** sintetizza i pattern nascosti producendo la predizione di classe o regressione.

Per il **Teorema di Approssimazione Universale** (Cybenko 1989, Hornik 1991), una rete feed-forward dotata di un singolo strato nascosto contenente un numero finito di neuroni con attivazione non lineare continua può approssimare con accuratezza arbitraria qualunque funzione continua su compatti di $\mathbb{R}^d$.

23.6.1 Tassonomia delle Funzioni di Attivazione

L'introduzione della non-linearità nei nodi nascosti è il requisito matematico indispensabile per consentire alla rete di modellare funzioni complesse: una rete composta da molteplici strati lineari collassa algebricamente a una banale regressione lineare.

La Tabella 23.5 riassume sinotticamente le funzioni di attivazione standard, evidenziandone codominio, formulazione analitica, derivata prima e caratteristiche operative.

Tabella 23.5: Quadro comparativo delle principali funzioni di attivazione impiegate nelle reti neurali.

Funzione	Codominio	Espressione $f(z)$	Derivata Prima $f'(z)$	Proprietà e Destinazione d'Uso
Gradino (Step)	$\{0, 1\}$	$\begin{cases} 0 & z < 0 \\ 1 & z \geq 0 \end{cases}$	0 q.o. (non diff. in 0)	Percettroni classici; incompatibile con gradient descent
Lineare	$(-\infty, +\infty)$	z	1	Output per regressione continua; nessun limite superiore
Sigmoide	$(0, 1)$	$\frac{1}{1+e^{-z}}$	$f(z)\big(1 - f(z)\big)$	Uscita probabilistica binaria; soggetta a *vanishing gradient*
Tangente Iperbolica	$(-1, 1)$	$\frac{e^z - e^{-z}}{e^z + e^{-z}}$	$1 - f(z)^2$	Simmetrica rispetto all'origine; convergenza più rapida di sigmoid
ReLU	$[0, +\infty)$	$\max(0, z)$	$\begin{cases} 0 & z < 0 \\ 1 & z > 0 \end{cases}$	Standard per strati nascosti; previene il vanishing gradient
Softmax	$(0, 1)^K$, $\sum z_k = 1$	$\frac{e^{z_k}}{\sum_{c=1}^{K} e^{z_c}}$	$\frac{\partial f_i}{\partial z_j} = f_i(\delta_{ij} - f_j)$	Output per problemi di classificazione multiclasse mutuamente esclusivi

Teorema 23.6.1 (Derivata della Funzione Sigmoide)**.** *La derivata prima della funzione logistica* $\sigma(z) = \frac{1}{1+e^{-z}}$ *ammette l'espressione fattorizzata in forma chiusa:*

$$\sigma'(z) = \sigma(z)\Big(1 - \sigma(z)\Big) \tag{23.19}$$

Dimostrazione. Applicando la regola di derivazione per la funzione reciproca:

$$\begin{aligned}\sigma'(z) &= \frac{d}{dz}(1+e^{-z})^{-1} = -(1+e^{-z})^{-2} \cdot (-e^{-z}) \\ &= \frac{e^{-z}}{(1+e^{-z})^2} = \frac{1}{1+e^{-z}} \cdot \frac{e^{-z}}{1+e^{-z}} \\ &= \frac{1}{1+e^{-z}} \cdot \left(\frac{1+e^{-z}-1}{1+e^{-z}}\right) = \sigma(z)\Big(1-\sigma(z)\Big)\end{aligned} \tag{23.20}$$

Tale proprietà risulta di eccezionale convenienza computazionale, poiché il calcolo della derivata non richiede il ricalcolo dell'esponenziale, bensì sfrutta direttamente il valore di attivazione già memorizzato durante il passaggio in avanti. □

23.7 Addestramento delle Reti: Discesa del Gradiente e Backpropagation

L'addestramento di una rete neurale consiste nell'identificare la configurazione di pesi e bias $\mathbf{W}^*$ che minimizza una funzione di perdita empirica (*Loss Function*) sull'insieme di addestramento.

23.7.1 Propagazione in Avanti e il Problema dell'Assegnazione del Credito

In una rete a uno strato nascosto con M ingressi, D neuroni nascosti e output y:

$$a_j = \sum_{i=0}^{M} \alpha_{ji} x_i, \quad z_j = \sigma(a_j), \quad b = \sum_{j=0}^{D} \beta_j z_j, \quad y = \sigma(b) \tag{23.21}$$

L'errore globale quadratico commesso su un insieme di N istanze con target y_j^* è:

$$E(\mathbf{W}) = \frac{1}{2} \sum_{k=1}^{N} \Big(y_k - y_k^*\Big)^2 \tag{23.22}$$

Mentre sui nodi di output l'errore è misurabile direttamente confrontando y con il target supervisionato y^*, sui neuroni nascosti non esiste alcun valore

target noto. Questo dilemma prende il nome storico di **Problema dell'Assegnazione del Credito** (*Credit Assignment Problem*): come ripartire la responsabilità dell'errore complessivo tra i neuroni degli strati interni?

23.7.2 Discesa del Gradiente e Derivazione della Backpropagation

Per minimizzare $E(\mathbf{W})$, il metodo della **Discesa del Gradiente** (*Gradient Descent*) aggiorna iterativamente ciascun parametro nella direzione opposta al gradiente locale:

$$w_{ij}^{(k+1)} = w_{ij}^{(k)} - \lambda \frac{\partial E}{\partial w_{ij}} \tag{23.23}$$

L'algoritmo di **Backpropagation** (Rumelhart, Hinton e Williams, 1986 [55]) risolve il calcolo esatto delle derivate parziali applicando sistematicamente la regola della catena (*chain rule*) procedendo a ritroso dallo strato di output verso gli ingressi:

1. **Segnale di Errore del Neurone di Output ($j \in$ Output):** Posto $E = \frac{1}{2}(y_j - T_j)^2$ (o viceversa $E = \frac{1}{2}(T_j - Z_j)^2$), definendo $\delta_j = -\frac{\partial E}{\partial net_j}$:

$$\delta_j = (T_j - Z_j) f'(net_j) = (T_j - Z_j) Z_j (1 - Z_j) \tag{23.24}$$

2. **Segnale di Errore del Neurone Nascosto ($j \in$ Hidden):** Sia Φ_j l'insieme dei nodi dello strato a valle che ricevono segnali dal neurone j. Applicando la chain rule su tutte le vie di propagazione uscenti:

$$\delta_j = f'(net_j) \sum_{k \in \Phi_j} \delta_k w_{jk} = Z_j(1 - Z_j) \sum_{k \in \Phi_j} \delta_k w_{jk} \tag{23.25}$$

   L'errore del nodo nascosto è una combinazione lineare degli errori dei nodi a valle pesata con i coefficienti di connessione correnti.

3. **Aggiornamento dei Pesi Sinaptici:** Per qualunque connessione orientata dal neurone sorgente i al neurone ricevente j:

$$\Delta w_{ij} = \lambda \delta_j Z_i, \qquad w_{ij}^{\text{new}} = w_{ij}^{\text{old}} + \Delta w_{ij} \tag{23.26}$$

   dove Z_i rappresenta il segnale in uscita dal nodo di provenienza (per un nodo di bias costante, $Z_i = 1$).

23.7.3 Epoche, Strategie di Batch e Considerazioni Progettuali

L'addestramento pratico di una rete neurale si struttura attorno ai seguenti parametri operativi:

- **Epoca (*Epoch*):** Un singolo ciclo completo durante il quale la rete elabora la totalità degli esempi del training set. Addestrare per E epoche implica mostrare ciascun record E volte, rimescolando casualmente i dati a ogni passaggio;
- **Strategie di Raggruppamento del Gradiente (Batch Size b):**
  - **Stochastic Gradient Descent (SGD, $b = 1$):** I pesi vengono aggiornati immediatamente dopo ogni singolo record. Presenta elevata varianza e traiettorie rumorose ma può fuggire da minimi locali poco profondi;
  - **Mini-Batch Gradient Descent ($1 < b < N$):** Rappresenta lo standard consolidato nel deep learning. Calcola il gradiente medio su un piccolo blocco (es. $b \in \{32, 64, 128\}$), bilanciando l'efficienza delle istruzioni vettoriali hardware con la stabilità statistica;
  - **Batch Gradient Descent Completo ($b = N$):** Calcola il gradiente deterministico sull'intero dataset prima di aggiornare i pesi. Lento su archivi massivi e vulnerabile a stalli su selle e minimi locali non convessi.
- **Calibrazione del Learning Rate λ:** Se λ è eccessivamente modesto, l'ottimizzazione progredisce con estrema lentezza; se λ è troppo elevato, le correzioni provocano oscillazioni divergenti oltre il fondo della valle dell'errore;
- **Codifica delle Variabili Categoriche:** Gli attributi nominali a K categorie non devono essere codificati con numeri progressivi (che imporrebbero un ordine metrico fittizio), bensì trasformati tramite codifica **One-Hot** in K canali binari dedicati.

23.8 Esercitazione Completa: Backpropagation Passo-Passo

In questa sezione viene sviluppata la risoluzione analitica completa e rigorosa dell'esercitazione d'esame sulla retropropagazione dell'errore, formulata secondo gli standard didattici del corso [56].

23.8.1 Specifiche Architetturali e Parametri Iniziali

Consideriamo la rete neurale feed-forward a 7 nodi illustrata nella Figura 23.5, composta da:

- **Strato di Input:** due neuroni recettori 1 e 2, che ricevono i valori $Z_1 = 0.9$ e $Z_2 = 0.6$;
- **Nodi di Bias Costanti:** il nodo 3 ($Z_3 = 1$) collegato ai neuroni nascosti 4 e 5, e il nodo 6 ($Z_6 = 1$) collegato al neurone di output 7. Tali unità non applicano la sigmoide e non possiedono segnali di errore; i loro pesi uscenti fungono da parametri di bias addestrabili;
- **Strato Nascosto:** due neuroni sigmoidali 4 e 5;
- **Strato di Output:** un singolo neurone sigmoidale 7 con target desiderato $T_7 = 0$;
- **Tasso di Apprendimento:** $\lambda = 1$;
- **Inizializzazione dei Pesi:** tutti i 9 pesi sinaptici sono inizialmente configurati al valore simmetrico $w_{ij}^{\text{old}} = 0.5$.

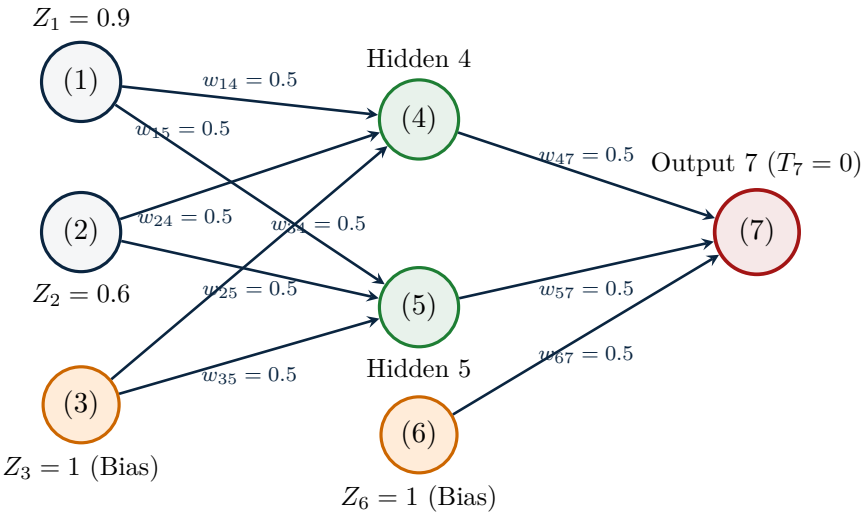


Figura 23.5: Topologia della rete neurale multistrato a 7 nodi analizzata nell'esercitazione.

23.8.2 Passo 1: Propagazione in Avanti (*Forward Pass*)

Calcoliamo le attivazioni strato per strato partendo dagli ingressi noti:

$$Z_1 = 0.9, \qquad Z_2 = 0.6, \qquad Z_3 = 1.0, \qquad Z_6 = 1.0 \tag{23.27}$$

1. Neurone Nascosto 4

L'ingresso pesato aggregato net_4 è:

$$\begin{aligned} net_4 &= w_{14}Z_1 + w_{24}Z_2 + w_{34}Z_3 \\ &= (0.5)(0.9) + (0.5)(0.6) + (0.5)(1.0) \\ &= 0.45 + 0.30 + 0.50 = 1.25 \end{aligned} \tag{23.28}$$

L'attivazione non lineare prodotta dal neurone 4 è:

$$Z_4 = \sigma(net_4) = \frac{1}{1 + e^{-1.25}} \approx \frac{1}{1 + 0.2865} \approx \mathbf{0.77} \tag{23.29}$$

La derivata prima della funzione sigmoide calcolata su Z_4 risulta:

$$f'(net_4) = Z_4(1 - Z_4) \approx (0.77)(1 - 0.77) = (0.77)(0.23) \approx \mathbf{0.17} \tag{23.30}$$

2. Neurone Nascosto 5

Poiché il neurone 5 condivide gli stessi identici ingressi e gli stessi pesi simmetrici del neurone 4:

$$net_5 = w_{15}Z_1 + w_{25}Z_2 + w_{35}Z_3 = (0.5)(0.9) + (0.5)(0.6) + (0.5)(1.0) = 1.25 \tag{23.31}$$

$$Z_5 = \sigma(net_5) \approx \mathbf{0.77}, \qquad f'(net_5) = Z_5(1 - Z_5) \approx \mathbf{0.17} \tag{23.32}$$

3. Neurone di Output 7

L'ingresso netto aggregato al nodo finale di output net_7 è:

$$\begin{aligned} net_7 &= w_{47}Z_4 + w_{57}Z_5 + w_{67}Z_6 \\ &\approx (0.5)(0.77) + (0.5)(0.77) + (0.5)(1.0) \\ &= 0.385 + 0.385 + 0.50 = 1.27 \end{aligned} \tag{23.33}$$

L'uscita finale formulata dalla rete neurale per questo pattern è:

$$Z_7 = \sigma(net_7) = \frac{1}{1 + e^{-1.27}} \approx \frac{1}{1 + 0.2808} \approx \mathbf{0.78} \tag{23.34}$$

La corrispondente derivata è:

$$f'(net_7) = Z_7(1 - Z_7) \approx (0.78)(1 - 0.78) = (0.78)(0.22) \approx \mathbf{0.17} \quad (23.35)$$

Diagnostica dell'Errore: La rete ha emesso una predizione $Z_7 \approx 0.78$, a fronte di un target desiderato $T_7 = 0$. Il segnale di uscita risulta eccessivamente elevato: l'algoritmo di retropropagazione dovrà rimodulare i pesi in modo tale da ridurre drasticamente il potenziale Z_7.

23.8.3 Passo 2: Retropropagazione dei Segnali di Errore (*Backward Pass*)

Un vincolo metodologico inderogabile impone che **tutti i segnali di errore debbano essere calcolati impiegando rigorosamente i pesi originali del forward pass**, prima che qualunque peso venga alterato [56].

1. Segnale di Errore del Neurone di Output 7

Applicando l'equazione (23.24):

$$\begin{aligned} \delta_7 &= (T_7 - Z_7) f'(net_7) = (T_7 - Z_7) Z_7 (1 - Z_7) \\ &\approx (0 - 0.78) \cdot (0.17) \approx -0.1326 \approx \mathbf{-0.13} \end{aligned} \quad (23.36)$$

2. Segnale di Errore del Neurone Nascosto 4

Il neurone 4 proietta la propria attivazione unicamente verso il neurone di output 7 ($\Phi_4 = \{7\}$). Applicando l'equazione (23.25):

$$\begin{aligned} \delta_4 &= Z_4(1 - Z_4) \sum_{k \in \Phi_4} \delta_k w_{4k} = Z_4(1 - Z_4)\delta_7 w_{47} \\ &\approx (0.17) \cdot (-0.13) \cdot (0.5) \approx (0.17) \cdot (-0.065) \approx -0.011 \approx \mathbf{-0.01} \end{aligned} \quad (23.37)$$

3. Segnale di Errore del Neurone Nascosto 5

Simmetricamente, $\Phi_5 = \{7\}$:

$$\begin{aligned} \delta_5 &= Z_5(1 - Z_5) \sum_{k \in \Phi_5} \delta_k w_{5k} = Z_5(1 - Z_5)\delta_7 w_{57} \\ &\approx (0.17) \cdot (-0.13) \cdot (0.5) \approx \mathbf{-0.01} \end{aligned} \quad (23.38)$$

I due neuroni nascosti presentano segnali di errore negativi identici ($\delta_4 = \delta_5 = -0.01$), conseguenza diretta della simmetria iniziale dei pesi e degli ingressi.

23.8.4 Passo 3: Aggiornamento Completo dei Pesi Sinaptici

Aggiorniamo singolarmente ciascuna delle nove connessioni applicando la regola generale (23.26):

$$\Delta w_{ij} = \lambda \delta_j Z_i, \qquad w_{ij}^{\text{new}} = w_{ij}^{\text{old}} + \Delta w_{ij} \tag{23.39}$$

con $\lambda = 1$ e peso antecedente $w_{ij}^{\text{old}} = 0.5$.

1. Connessioni verso il Neurone Nascosto 4

- Connessione dall'ingresso 1 ($1 \to 4$):

$$\begin{aligned} \Delta w_{14} &= \lambda \delta_4 Z_1 \approx (1) \cdot (-0.01) \cdot (0.9) = -0.009 \approx \mathbf{-0.01} \\ w_{14}^{\text{new}} &\approx 0.5 - 0.01 = \mathbf{0.49} \end{aligned} \tag{23.40}$$

- Connessione dall'ingresso 2 ($2 \to 4$):

$$\begin{aligned} \Delta w_{24} &= \lambda \delta_4 Z_2 \approx (1) \cdot (-0.01) \cdot (0.6) = -0.006 \\ w_{24}^{\text{new}} &\approx 0.5 - 0.006 = \mathbf{0.494} \end{aligned} \tag{23.41}$$

- Connessione di bias 3 ($3 \to 4$ con $Z_3 = 1$):

$$\begin{aligned} \Delta w_{34} &= \lambda \delta_4 Z_3 \approx (1) \cdot (-0.01) \cdot (1.0) = \mathbf{-0.01} \\ w_{34}^{\text{new}} &\approx 0.5 - 0.01 = \mathbf{0.49} \end{aligned} \tag{23.42}$$

2. Connessioni verso il Neurone Nascosto 5

- Connessione dall'ingresso 1 ($1 \to 5$):

$$\begin{aligned} \Delta w_{15} &= \lambda \delta_5 Z_1 \approx (1) \cdot (-0.01) \cdot (0.9) \approx \mathbf{-0.01} \\ w_{15}^{\text{new}} &\approx 0.5 - 0.01 = \mathbf{0.49} \end{aligned} \tag{23.43}$$

- Connessione dall'ingresso 2 ($2 \to 5$):

$$\begin{aligned} \Delta w_{25} &= \lambda \delta_5 Z_2 \approx (1) \cdot (-0.01) \cdot (0.6) = -0.006 \\ w_{25}^{\text{new}} &\approx 0.5 - 0.006 = \mathbf{0.494} \end{aligned} \tag{23.44}$$

- Connessione di bias 3 ($3 \to 5$ con $Z_3 = 1$):

$$\begin{aligned} \Delta w_{35} &= \lambda \delta_5 Z_3 \approx (1) \cdot (-0.01) \cdot (1.0) = \mathbf{-0.01} \\ w_{35}^{\text{new}} &\approx 0.5 - 0.01 = \mathbf{0.49} \end{aligned} \tag{23.45}$$

3. Connessioni verso il Neurone di Output 7

- Connessione dal neurone nascosto 4 ($4 \to 7$ con $Z_4 = 0.77$):

$$\Delta w_{47} = \lambda \delta_7 Z_4 \approx (1) \cdot (-0.13) \cdot (0.77) = -0.1001 \approx \mathbf{-0.10}$$
$$w_{47}^{\text{new}} \approx 0.5 - 0.10 = \mathbf{0.40} \tag{23.46}$$

- Connessione dal neurone nascosto 5 ($5 \to 7$ con $Z_5 = 0.77$):

$$\Delta w_{57} = \lambda \delta_7 Z_5 \approx (1) \cdot (-0.13) \cdot (0.77) = -0.1001 \approx \mathbf{-0.10}$$
$$w_{57}^{\text{new}} \approx 0.5 - 0.10 = \mathbf{0.40} \tag{23.47}$$

- Connessione di bias 6 ($6 \to 7$ con $Z_6 = 1.0$):

$$\Delta w_{67} = \lambda \delta_7 Z_6 \approx (1) \cdot (-0.13) \cdot (1.0) = \mathbf{-0.13}$$
$$w_{67}^{\text{new}} \approx 0.5 - 0.13 = \mathbf{0.37} \tag{23.48}$$

23.8.5 Quadro Sinottico Conclusivo dell'Aggiornamento

La Tabella 23.6 raccoglie la sintesi completa delle variazioni calcolate sull'intera rete.

Tabella 23.6: Quadro sinottico finale dell'aggiornamento dei 9 pesi sinaptici mediante Backpropagation.

Connessione ($i \to j$)	**Tipologia Connessione**	**Peso Precedente** (w_{ij}^{old})	**Delta** (Δw_{ij})	**Peso Aggiornato** (w_{ij}^{new})
$1 \to 4$	Input $\to$ Hidden	0.5	-0.01	**0.490**
$2 \to 4$	Input $\to$ Hidden	0.5	-0.006	**0.494**
$3 \to 4$	Bias $\to$ Hidden	0.5	-0.01	**0.490**
$1 \to 5$	Input $\to$ Hidden	0.5	-0.01	**0.490**
$2 \to 5$	Input $\to$ Hidden	0.5	-0.006	**0.494**
$3 \to 5$	Bias $\to$ Hidden	0.5	-0.01	**0.490**
$4 \to 7$	Hidden $\to$ Output	0.5	-0.10	**0.400**
$5 \to 7$	Hidden $\to$ Output	0.5	-0.10	**0.400**
$6 \to 7$	Bias $\to$ Output	0.5	-0.13	**0.370**

Interpretazione Teorico-Funzionale:

1. Tutti e nove i delta Δw_{ij} risultano **negativi**. Questo comportamento riflette l'azione correttiva della discesa del gradiente: avendo registrato un'attivazione di output $Z_7 \approx 0.78$ ampiamente superiore al target nullo $T_7 = 0$, la retropropagazione riduce la grandezza di tutti i pesi (sia di feed-forward che di bias) al fine di abbassare l'ingresso netto net_7 nel passaggio successivo;

2. L'ampiezza dell'aggiornamento è notevolmente maggiore per i pesi dello strato di output ($\Delta w \approx -0.10 \cdots - 0.13$) rispetto a quelli dello strato nascosto ($\Delta w \approx -0.006 \cdots - 0.01$). Questo fenomeno discende dall'attenuazione introdotta dalla moltiplicazione reiterata per la derivata della sigmoide ($f' \approx 0.17$), che costituisce la manifestazione elementare del fenomeno della **scomparsa del gradiente** (*vanishing gradient*) nelle reti multistrato con neuroni saturanti.

Capitolo 24

Laboratorio di Machine Learning: Codifica, Alberi di Decisione, Dati Sbilanciati e Generazione Sintetica

24.1 Introduzione e Motivazione

Nell'ambito dell'analisi avanzata dei dati e del Data Mining, il paradigma del **Machine Learning** (apprendimento automatico) formalizza la progettazione e lo sviluppo di sistemi algoritmici capaci di estrarre conoscenza predittiva a partire dall'evidenza empirica [2, 19]. Da un punto di vista fondazionale, il problema dell'apprendimento automatico si articola su quattro pilastri concettuali interconnessi [57]:

1. **Cosa apprendere (*A Model*):** un modello matematico o una funzione di mappatura $f : \mathcal{X} \to \mathcal{Y}$ che formalizza le relazioni statistiche sottostanti tra i dati di ingresso e l'obiettivo da predire.

2. **Per risolvere cosa (*A Task*):** un compito ben definito, primariamente ripartito tra compiti di *classificazione* (qualora l'etichetta target appartenga a un dominio discreto e finito) e compiti di *regressione* (qualora la variabile risposta sia continua e reale).

3. **Come apprendere (*A Learning Algorithm*):** una procedura computazionale strutturata che aggiorna o configura i parametri del modello alla luce dei dati disponibili.

4. **Apprendere mediante cosa (*Searching*):** un processo di ricerca ottimizzata all'interno dello spazio delle ipotesi $\mathcal{H}$, guidato dalla minimizzazione di una funzione di costo o dalla massimizzazione di un criterio di purezza/verosimiglianza.

La transizione dai principi teorici all'applicazione pratica solleva tuttavia due sfide critiche frequentemente riscontrabili nei contesti operativi moderni. In primo luogo, i dataset reali presentano quasi costantemente strutture eterogenee, con attributi continui mescolati a variabili categoriche nominali o ordinali e tuple incomplete, rendendo imprescindibile una fase rigorosa di *pre-elaborazione* e *codifica* (*encoding*) affinché i predittori possano essere impiegati da algoritmi come gli alberi di decisione o i classificatori lineari [2]. In secondo luogo, nella quasi totalità dei domini applicativi ad alto impatto (quali il rilevamento delle frodi bancarie, la diagnostica medica di patologie rare, il monitoraggio di guasti nei sistemi critici e la manutenzione predittiva), la distribuzione delle classi risulta intrinsecamente e asimmetricamente sbilanciata (*class imbalance*) [58, 57].

In presenza di una forte asimmetria tra una classe ampiamente prevalente (*classe maggioritaria*) e una classe infrequente ma prioritaria (*classe minoritaria*), i modelli convenzionali addestrati sull'accuratezza globale falliscono catastroficamente: essi tendono a predire sistematicamente la classe maggioritaria, ignorando completamente le istanze rare pur esibendo percentuali di accuratezza apparentemente eccellenti (il cosiddetto *paradosso dell'accuratezza*) [57].

Il presente capitolo integra organicamente la dimensione empirica di laboratorio e il rigore metodologico avanzato:

- La **Parte I** esplora il flusso di lavoro operativo condotto nel laboratorio Python sul dataset tabellare `adult.csv`, illustrando i principi teorici e pratici di discretizzazione, codifica (*Label Encoding* vs *One-Hot Encoding*), addestramento, potatura e visualizzazione degli alberi di decisione, con una rassegna comparativa sui principali classificatori supervisionati.

- La **Parte II** formalizza lo studio sistematico dei **dati sbilanciati** (*Imbalanced Data*), analizzando il fallimento del classificatore banale, la tassonomia delle strategie correttive a livello di dati e di algoritmo, i metodi di sottocampionamento intelligente (*Condensed Nearest Neighbor*, *Tomek Links*) e di sovracampionamento con interpolazione nello spazio delle feature (*SMOTE* e varianti), unitamente all'apprendimento sensibile ai costi (*Cost-Sensitive Learning*).

- La **Parte III** affronta la frontiera della modellazione generativa profonda per la **generazione sintetica di dati tabellari**, analizzando il confronto paradigmatico tra modelli discriminativi e generativi, l'architettura competitiva Minimax delle *Generative Adversarial Networks* (GAN) con soluzione alla saturazione del gradiente, la formulazione specializzata di *Conditional Tabular GAN* (CTGAN), e il framework probabilistico dei *Variational Autoencoders* (VAE) basato sul *Reparameterization Trick* e applicato a dati tabellari (TVAE).

24.2 Parte I: Laboratorio Empirico su Dataset Tabellare Adult

24.2.1 Ambiente Sperimentale e Profilassi del Dataset Adult

La sperimentazione pratica condotta nel laboratorio didattico impiega lo stack scientifico Python standard composto da `numpy`, `pandas`, `scipy.stats`, `matplotlib.pyplot`, `seaborn` e `scikit-learn` [57].

Il caso di studio impiega il dataset socio-demografico `adult.csv` (noto anche nella letteratura statistica internazionale come *Census Income Dataset*), il cui compito analitico consiste nel determinare se il reddito annuo di un individuo superi o meno la soglia di 50.000 dollari statunitensi [2].

Definizione 24.2.1 (Spazio dei Dati del Dataset Adult)**.** Il dataset è costituito da una collezione di $N = 36.631$ istanze campionate, strutturate su $D = 14$ attributi complessivi:

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}, \qquad \mathbf{x}_i \in \mathcal{X}, \quad y_i \in \{0, 1\} \tag{24.1}$$

La tupla descrittiva $\mathbf{x}_i$ è ripartita in:

- **Attributi numerici continui o discreti (6 predittori):** `age` (età anagrafica), `capital_gain` (plusvalenze finanziarie), `capital_loss` (minusvalenze finanziarie), `education` (livello di istruzione codificato numericamente), `final_weight` (peso campionario demografico attribuito dall'istituto di censimento), `hours_worked_per_week` (ore lavorative settimanali).
- **Attributi categorici nominali (6 predittori):** `marital_status`, `native_country`, `occupation`, `race`, `relationship`, `workclass`. Tali variabili contengono categorie non standard e valori mancanti esplicitamente indicati tramite il simbolo convenzionale `'?'`.

- **Attributo booleano (1 predittore):** `is_male` con dominio binario $\{\text{True}, \text{False}\}$.
- **Attributo target supervisionato (y):** `over_threshold`, variabile binaria dove $y = 1$ denota un reddito superiore a 50.000\$ annui e $y = 0$ denota un reddito inferiore o uguale.

24.2.2 Strategie di Codifica delle Variabili Categoriche (Encoding)

La maggioranza degli stimatori matematici di Machine Learning richiede matrici di ingresso puramente numeriche $X \in \mathbb{R}^{N \times M}$. Di conseguenza, la gestione delle variabili categoriche non ordinali impone la scelta di una funzione di mappatura appropriata. Nel laboratorio didattico vengono esaminate e poste a confronto due strategie cardine [57]:

1. **Label Encoding (*Codifica a Valori Interi*):** Ciascuna modalità distinta $v \in \mathcal{V}_j$ dell'attributo categorico A_j viene associata univocamente a un indice intero compreso tra 0 e $|\mathcal{V}_j| - 1$ mediante l'operatore `LabelEncoder` di Scikit-Learn:

$$\phi_{\text{label}} : \mathcal{V}_j \to \{0, 1, \ldots, |\mathcal{V}_j| - 1\} \tag{24.2}$$

   Nel dataset in esame, per la feature `marital_status`, le categorie subiscono la trasformazione:

   $$\text{'Divorced'} \mapsto 0, \quad \text{'Married-civ-spouse'} \mapsto 2, \quad \text{'Never-married'} \mapsto 4$$

   mentre per la feature booleana `is_male` si ottiene False $\mapsto 0$ e True $\mapsto 1$. Il principale vantaggio di questo schema risiede nella conservazione rigorosa della dimensionalità: la matrice risultante mantiene esattamente 14 colonne.

Patologia di Ordinamento Indotta dal Label Encoding

L'assegnazione di numeri interi arbitrari a variabili nominali non ordinate (prive di graduazione naturale) induce una relazione d'ordine artificiale e metriche di distanza prive di significato semantico:

$$|2-0| < |4-0| \implies$$
$$d(\text{'Married'}, \text{'Divorced'}) < d(\text{'Never-married'}, \text{'Divorced'})$$

Mentre gli **alberi di decisione** possono tollerare tale codifica (poiché valutano predicati di disuguaglianza univariata su singoli valori discreti preservando le partizioni locali), gli algoritmi basati su funzioni di distanza vettoriale euclidea o di Minkowski (quali *k-Nearest Neighbors*, Support Vector Machines a kernel RBF e Reti Neurali) interpretano tali interi come coordinate metriche, distorcendo gravemente la topologia dello spazio delle feature.

2. **One-Hot Encoding (*Codifica Binaria Espansa*):** Per superare il limite geometrico del Label Encoding, ciascun attributo nominale A_j avente cardinalità $|\mathcal{V}_j| = K_j$ viene proiettato in uno spazio vettoriale a K_j dimensioni binarie mediante `pd.get_dummies(..., prefix_sep='%')`:

$$\phi_{\text{OHE}}(x^{(j)}) = \mathbf{e}_k \in \{0,1\}^{K_j}, \qquad \text{dove } (\mathbf{e}_k)_l = \begin{cases} 1 & \text{se } x^{(j)} = v_l \\ 0 & \text{altrimenti} \end{cases} \tag{24.3}$$

Ogni modalità diventa un predittore booleano indipendente (ad es. `relationship%0`, ..., `education%15`). L'applicazione dell'operatore sul dataset `adult` espande lo spazio dei predittori da 14 attributi iniziali a ben 108 **colonne**. Questa rappresentazione garantisce equidistanza geometrica ortogonale tra tutte le categorie distinte:

$$\|\mathbf{e}_k - \mathbf{e}_l\|_2 = \sqrt{2}, \quad \forall k \neq l \tag{24.4}$$

eliminando qualsiasi bias ordinale spurio al prezzo di una matrice sparsa a dimensionalità superiore.

24.2.3 Induzione e Valutazione degli Alberi di Decisione

La pipeline sperimentale procede partizionando il dataset mediante suddivisione stratificata [57]:

$$\mathcal{D}_{\text{train}}\ (70\%,\ N_{\text{train}} = 25.641), \qquad \mathcal{D}_{\text{test}}\ (30\%,\ N_{\text{test}} = 10.990) \tag{24.5}$$

garantendo tramite il vincolo `stratify=label` che la proporzione della classe positiva ($y = 1$, circa il 24%) e della classe negativa ($y = 0$, circa il 76%) rimanga identica nelle due frazioni.

L'albero di decisione viene configurato attraverso i parametri strutturali di regolarizzazione per contrastare il sovradattamento (*overfitting*):

- `criterion='gini'`: impiego dell'indice di impurità di Gini;
- `max_depth=10`: profondità massima dell'albero limitata a 10 livelli;
- `min_samples_split=3`: soglia minima di 3 campioni in un nodo interno per autorizzarne la ramificazione;
- `min_samples_leaf=4`: vincolo minimale di almeno 4 campioni per ciascun nodo foglia terminale.

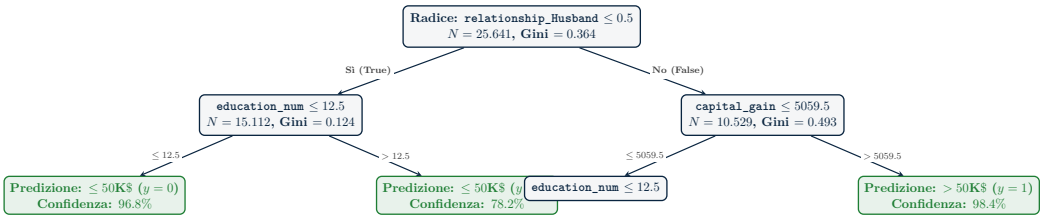


Figura 24.1: Schema topologico dei primi livelli dell'albero di decisione indotto dal notebook su dati Adult, con indicazione delle condizioni di split, numerosità campionaria e purezza dei nodi.

L'albero indotto evidenzia un'elevata interpretabilità: la condizione radice discrimina lo stato coniugale (`relationship_Husband`), seguita dalla scolarizzazione (`education_num`) e dalle plusvalenze (`capital_gain`).

La valutazione quantitativa condotta sul test set produce la matrice di confusione e le metriche di prestazione riassunte nella Tabella 24.1.

Tabella 24.1: Report di classificazione sul test set per l'albero di decisione indotto ($N_{\text{test}} = 10.990$).

Classe Target	Precision	Recall	F1-Score	Supporto (Conteggio Reale)
Classe 0 ($\leq$ 50K\$, maggioritaria)	0.88	0.93	0.90	8.341
Classe 1 ($>$ 50K\$, minoritaria)	0.72	0.59	0.65	2.649
Accuratezza Globale (Accuracy)		0.849 (84.9%)		10.990
Media Macro (Macro Avg)	0.80	0.76	0.78	10.990
Media Ponderata (Weighted Avg)	0.84	0.85	0.84	10.990

La matrice di confusione empirica rivela un'asimmetria diagnostica evidente:

$$\mathbf{CM}_{\text{DT}} = \begin{bmatrix} \text{TN} = 7762 & \text{FP} = 579 \\ \text{FN} = 1076 & \text{TP} = 1573 \end{bmatrix} \tag{24.6}$$

Nonostante un'accuratezza ragguardevole dell'84.9%, la *Recall* sulla classe ricca ($y = 1$) crolla al 59%: l'albero manca oltre il 40% dei soggetti ad alto reddito (1076 falsi negativi), ponendo in luce le prime manifestazioni dello sbilanciamento di classe.

24.2.4 Panoramica Comparativa dei Modelli del Laboratorio

Il notebook estende l'indagine empirica applicando un ventaglio completo di algoritmi di classificazione supervisionata sullo stesso dataset preprocessato [57]:

- **Gaussian Naïve Bayes (`GaussianNB`):** assume indipendenza condizionale tra i predittori data la classe; ottiene una predizione rapida ma risente della forte correlazione tra indicatori socio-economici, esibendo un'accuratezza inferiore (79.6%) ma un richiamo bilanciato.
- **k-Nearest Neighbors (`KNeighborsClassifier`):** valutato su sottoinsiemi puramente numerici standardizzati; la complessità computazionale di predizione scala con $\mathcal{O}(N \cdot d)$, rendendolo sensibile al rumore dimensionale.
- **XGBoost (`XGBClassifier`):** implementazione ottimizzata di Gradient Boosted Decision Trees; mediante regolarizzazione L_1/L_2 sui pesi delle foglie e discesa di gradiente nel secondo ordine, consegue le prestazioni predittive più elevate con un'accuratezza dell'87.1% e un F_1-score sulla classe minoritaria pari a 0.71.

- **Bagging & Random Forest:** la combinazione di 30 estimatori ad albero mediante bootstrap abbatte la varianza rispetto al singolo albero. L'introduzione esplicita di pesi asimmetrici (`class_weights={0: 0.35, 1: 0.65}`) inverte la sensibilità: la recall per la classe 1 sale sensibilmente dal 61% al 74%, a prezzo di un lieve incremento nei falsi positivi.
- **Classificatori Basati su Regole (`RIPPER`):** l'algoritmo RIPPER (*Repeated Incremental Pruning to Produce Error Reduction*) estrae un insieme compatto di regole deterministiche espresse in forma proposizionale `IF-THEN`, garantendo la massima interpretabilità per auditing etico-legale.
- **Reti Neurali Artificiali Multistrato & Dropout:** architettura densa a 3 strati addestrata per 80 epoche con funzione di costo *binary crossentropy*; l'introduzione di strati di regolarizzazione **Dropout** con probabilità $p = 0.3$ azzera il divario tra curva di loss di training e validazione, prevenendo la co-adattamento patologico dei nodi interni.

24.3 Parte II: Trattamento dei Dati Sbilanciati (Imbalanced Data)

24.3.1 Fondamenti, Esempi Reali e il Paradosso dell'Accuratezza

Nella teoria classica del Machine Learning si assume prevalentemente che le etichette di classe siano equamente distribuite tra i campioni disponibili (50% per la classe positiva e 50% per la negativa) [2]. Nella realtà applicativa, tale assunzione viene sistematicamente violata.

Definizione 24.3.1 (Distribuzione di Classe Sbilanciata)**.** Dato un problema di classificazione su uno spazio campionario $\mathcal{S}$, si definisce sbilanciata una distribuzione in cui sussiste una marcata sproporzione quantitativa tra la cardinalità dei campioni della classe prevalente (**classe maggioritaria**, C_{maj}) e la cardinalità dei campioni della classe rara di primario interesse analitico (**classe minoritaria**, C_{min}):

$$|C_{\text{maj}}| \gg |C_{\text{min}}|, \qquad \text{Rapporto di Sbilanciamento } IR = \frac{|C_{\text{maj}}|}{|C_{\text{min}}|} \gg 1 \quad (24.7)$$

Tale asimmetria governa le più rilevanti sfide tecnologiche contemporanee [57]:

- **Rilevamento Frodi Bancarie (*Credit Card Fraud*):** in un tipico archivio transazionale, circa il 2% dei conti o delle operazioni è fraudolento ($IR \approx 50:1$). Nei flussi di pagamento digitali in tempo reale, la frequenza scende spesso al di sotto dello 0.05% ($IR \approx 2000:1$).
- **Screening Medico ed Epidemiologico:** la prevalenza di una patologia grave (ad es. l'infezione da HIV o neoplasie in fase precoce) su una popolazione asintomatica si attesta comunemente tra lo 0.1% e lo 0.4% ($IR \approx 1000:1$).
- **Guasti nei Sistemi di Memorizzazione (*Disk Drive Failures*):** il tasso annuo di guasto di supporti storage nei data center industriali è dell'ordine dell'1% ($IR \approx 100:1$).
- **Controllo di Qualità Industriale:** su linee di produzione ad alta precisione, i tassi di difettosità si collocano attorno allo 0.1% ($IR \approx 1000:1$).

Teorema 24.3.1 (Il Paradosso dell'Accuratezza e Classificatore Banale)**.** *Sia $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$ un dataset binario con frazione di campioni positivi $p = \frac{|C_{pos}|}{N}$ e negativi $q = 1 - p = \frac{|C_{neg}|}{N}$. Si assuma senza perdita di generalità che la classe negativa sia maggioritaria ($q > p$). Un classificatore banale (o degenere) $f_{trivial}$ che predice costantemente la classe maggioritaria per qualsiasi ingresso:*

$$f_{trivial}(\mathbf{x}) = C_{maj}, \quad \forall \mathbf{x} \in \mathcal{X} \tag{24.8}$$

ottiene un'accuratezza globale pari alla frequenza relativa della classe prevalente:

$$A_{trivial} = q = 1 - p \tag{24.9}$$

pur esibendo un richiamo nullo sulla classe positiva di interesse:

$$Recall(C_{pos}) = \frac{TP}{TP + FN} = \frac{0}{0 + |C_{pos}|} = 0 \tag{24.10}$$

Esempio 24.3.1 (Valutazione Relativa dell'Accuratezza: Casi 75/25 vs 50/50)**.** Si consideri un test set di $N = 100$ record valutato da un classificatore con accuratezza misurata pari al 70% ($A = 0.70$) [57]:

- **Scenario Sbilanciato (75 Positivi, 25 Negativi):** il classificatore banale $f_{\text{trivial}}(\mathbf{x}) = \text{Positivo}$ consegue un'accuratezza intrinseca del 75%. Il modello appreso, conseguendo solo il 70%, è **peggiore della regola di decisione priva di intelligenza**: esso degrada le prestazioni rispetto alla pura previsione a priori della moda statistica.

- **Scenario Bilanciato (50 Positivi, 50 Negativi):** la baseline banale si attesta su $A_{\text{trivial}} = 50\%$. L'accuratezza del 70% rappresenta un effettivo e sostanziale guadagno informativo indotto dai predittori.

Nel caso multiclasse con K classi perfettamente bilanciate, la baseline di riferimento è data da:

$$A_{\text{trivial}} \approx \frac{100}{K}\% \tag{24.11}$$

Se $K = 5$, una precisione apparentemente modesta del 40% costituisce in realtà un raddoppio della capacità diagnostica rispetto alla pura scelta proporzionale (20%).

Di fronte a classi sbilanciate, la letteratura identifica quattro strategie d'azione metodologiche:

1. **Modifica a livello di Dati (*Data-level*):** bilanciamento sintetico o selettivo del training set mediante sottocampionamento (*undersampling*) o sovracampionamento (*oversampling*).

2. **Modifica a livello di Algoritmo (*Algorithm-level*):** incorporazione di pesi asimmetrici per le classi (*class weights*), variazione dinamica della soglia decisionale (*threshold moving*), o formulazioni di apprendimento sensibile ai costi.

3. **Cambio di Paradigma:** riformulazione del compito di classificazione binaria come problema di **rilevamento delle anomalie** (*Anomaly Detection* o *One-Class Classification*, ad es. One-Class SVM, Isolation Forest).

4. **Inazione consapevole:** mantenere intatta la distribuzione originale adottando esclusivamente metriche non ingannevoli (Precision-Recall AUC, Cost Curve, F-measure ponderata).

24.3.2 Tecniche di Sottocampionamento (Undersampling)

Il sottocampionamento interviene riducendo la numerosità della classe maggioritaria nel dataset di addestramento fino a renderla paritaria o comparabile con la classe minoritaria [57].

- **Random Undersampling:** consiste nell'estrarre un campione casuale uniforme (con o senza reinserimento) dalle istanze di C_{maj} fino alla dimensione target desiderata. Nonostante l'estrema semplicità e il

drastico abbattimento dei tempi di calcolo, tale procedura comporta una **perdita catastrofica di informazione potenziale**: campioni maggioritari contenenti pattern critici vengono eliminati alla cieca.

- **Sottocampionamento Intelligente Basato sui Vicini (*Smart Undersampling*):** tecniche mirate a rimuovere selettivamente i punti maggioritari ridondanti (lontani dal confine di decisione) preservando quelli prossimi alla frontiera tra le classi.

Definizione 24.3.2 (Condensed Nearest Neighbor — CNN, Hart 1968)**.** L'algoritmo **Condensed Nearest Neighbor (CNN)** [59] seleziona un sottoinsieme minimale consistente $S \subset \mathcal{D}$ tale che l'applicazione della regola del vicino più prossimo 1-NN addestrato su S classifichi correttamente tutti gli elementi dell'insieme originale $\mathcal{D}$.

Procedura Algoritmica di CNN:

1. Si inizializza l'insieme condensato S inserendovi **tutte le osservazioni della classe minoritaria** $C_{\min}$ e un singolo campione estratto casualmente dalla classe maggioritaria C_{maj}.

2. Per ogni istanza residua $\mathbf{x} \in \mathcal{D} \setminus S$ appartenente alla classe maggioritaria, se ne predice la classe impiegando un classificatore 1-NN addestrato sull'insieme provvisorio S:

$$\hat{y} = \arg\min_{\mathbf{s} \in S} \|\mathbf{x} - \mathbf{s}\|_2 \tag{24.12}$$

3. Se $\mathbf{x}$ viene classificato **correttamente**, il punto viene scartato poiché ridondante (la struttura locale è già coperta da S).

4. Se $\mathbf{x}$ viene classificato **erroneamente** (ad esempio scambiato per un elemento della classe minoritaria), $\mathbf{x}$ viene rimosso da $\mathcal{D}$ e aggiunto a S.

5. Si ripete la scansione iterativa finché un'intera epoca su tutti i dati non determina alcuna nuova aggiunta a S.

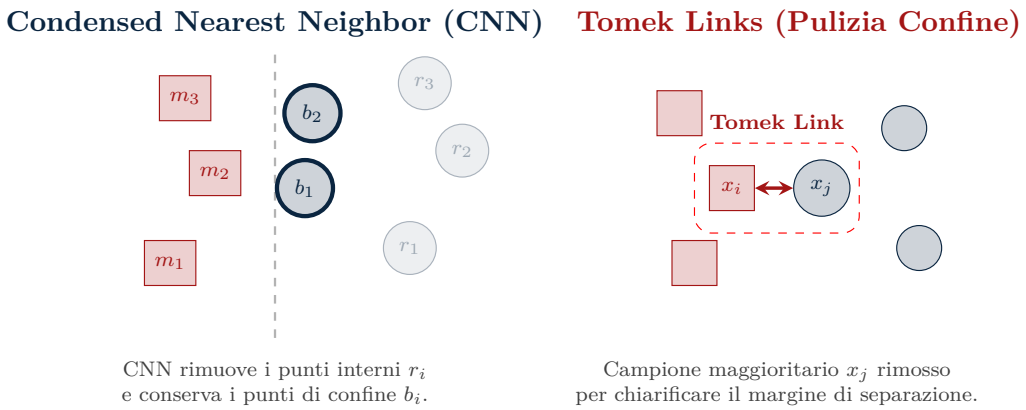


Figura 24.2: Rappresentazione geometrica dei metodi di sottocampionamento intelligente: preservazione dei punti informativi in CNN ed eliminazione delle coppie ambigue nei Tomek Links.

Definizione 24.3.3 (Tomek Link, Tomek 1976)**.** Dati due campioni $\mathbf{x}_i, \mathbf{x}_j \in \mathcal{D}$ con etichette opposte $y_i \neq y_j$, la coppia $(\mathbf{x}_i, \mathbf{x}_j)$ costituisce un **Tomek Link** [60] se e solo se essi risultano essere vicini più prossimi reciproci (*mutual nearest neighbors*):

$$d(\mathbf{x}_i, \mathbf{x}_j) < d(\mathbf{x}_i, \mathbf{x}_k), \quad \forall \mathbf{x}_k \text{ t.c. } y_k \neq y_i, \quad k \neq j \tag{24.13}$$

$$d(\mathbf{x}_j, \mathbf{x}_i) < d(\mathbf{x}_j, \mathbf{x}_l), \quad \forall \mathbf{x}_l \text{ t.c. } y_l \neq y_j, \quad l \neq i \tag{24.14}$$

L'esistenza di un Tomek Link denota che i due vettori giacciono in una zona di sovrapposizione altamente ambigua o rumorosa lungo il confine di decisione. Nella procedura standard di pulizia, si rimuove sistematicamente il vettore appartenente alla **classe maggioritaria** (oppure entrambi i vettori se si mira alla purificazione del rumore).

Pipeline Ibrida SMOTE + Tomek: Nei problemi ad alta complessità, si ricorre a un approccio sequenziale in due fasi:

1. **Fase di Espansione (SMOTE):** si generano campioni sintetici per ampliare la classe minoritaria; alcuni di essi possono accidentalmente invadere le regioni di pertinenza della classe maggioritaria.

2. **Fase di Purificazione (Tomek Links):** si identificano ed eliminano le coppie reciproche ambigue introdotte, ottenendo frontiere di separazione nette e prive di sovrapposizioni spurie.

24.3.3 Metodi di Sovracampionamento (Oversampling)

Il sovracampionamento bilancia le frequenze aumentando artificialmente il numero di istanze della classe minoritaria [57].

- **Random Oversampling:** duplica campioni esistenti di $C_{\min}$ mediante campionamento casuale con reinserimento. *Limite fondamentale:* non introduce alcuna nuova variabilità nello spazio delle feature, inducendo gli algoritmi (in particolare gli alberi decisionali) a costruire regioni decisionali eccessivamente rigide e ristrette attorno ai punti duplicati, amplificando l'overfitting.
- **SMOTE (*Synthetic Minority Over-sampling Technique*, Chawla et al., 2002):** genera nuove istanze sintetiche mediante **interpolazione convessa** nello spazio vettoriale delle feature tra campioni minoritari vicini [58].

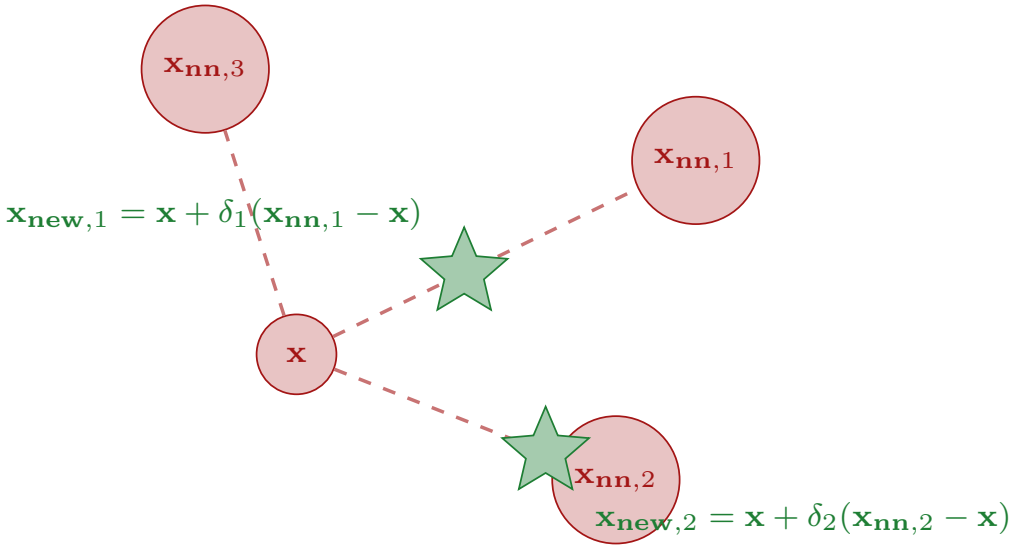
Algorithm 8: Algoritmo SMOTE per la Generazione Sintetica della Classe Minoritaria

Input: Insieme campionario minoritario $C_{\min}$, numero di vicini k, percentuale di sovracampionamento N_{perc} (es. 200%)
Output: Insieme di campioni sintetici generati $\mathcal{S}_{\text{syn}}$

$\mathcal{S}_{\text{syn}} \leftarrow \emptyset$
$n_{\text{syn}} \leftarrow \lfloor N_{\text{perc}}/100 \rfloor$
foreach $\mathbf{x} \in C_{min}$ **do**
  Determina i k vicini più prossimi in $C_{\min}$ rispetto alla distanza euclidea: $\mathcal{N}_k(\mathbf{x}) \subset C_{\min}$
  for $j \leftarrow 1$ **to** n_{syn} **do**
    Seleziona casualmente un vicino $\mathbf{x}_{\text{nn}} \in \mathcal{N}_k(\mathbf{x})$
    Estrai una variabile casuale uniforme $\delta \sim \mathcal{U}(0, 1)$
    Genera il nuovo vettore sintetico per combinazione lineare:

$$\mathbf{x}_{\text{new}} = \mathbf{x} + \delta \cdot (\mathbf{x}_{\text{nn}} - \mathbf{x}) \tag{24.15}$$

    $\mathcal{S}_{\text{syn}} \leftarrow \mathcal{S}_{\text{syn}} \cup \{\mathbf{x}_{\text{new}}\}$
  end
end
return $\mathcal{S}_{syn}$



I campioni sintetici giacciono sui segmenti congiungenti i vettori minoritari originali con i rispettivi k-NN.

Figura 24.3: Geometria di generazione sintetica in SMOTE: interpolazione convessa casuale nello spazio metrico continuo delle feature.

Operando direttamente nello spazio delle feature, SMOTE connette i punti minoritari isolati forzando la regione decisionale del classificatore a espandersi in un volume continuo anziché frammentarsi in aggregati disgiunti.

Evoluzioni e Varianti Specializzate di SMOTE:

- **SMOTENC (*Continuous and Categorical*):** gestisce simultaneamente feature numeriche e categoriche; per le variabili categoriche assegna la modalità più frequente tra i vicini minoritari, mentre per quelle continue calcola la distanza mediana per compensare il mismatch di scala.
- **Borderline-SMOTE:** focalizza la generazione sintetica unicamente sulle osservazioni minoritarie che si trovano in condizione di pericolo di confine (*DANGER*), definite come istanze in cui la maggioranza dei vicini appartiene alla classe opposta.
- **SVM-SMOTE:** sfrutta una Support Vector Machine preventiva per identificare i vettori di supporto (*support vectors*) lungo l'iperpiano separatore, concentrando l'interpolazione esclusivamente attorno a tali coordinate limite.

- **ADASYN (*Adaptive Synthetic Sampling*, He et al., 2008):** genera campioni sintetici in misura adattiva e inversamente proporzionale alla densità locale [61]: se un campione minoritario è circondato da molti punti maggioritari, riceve un coefficiente di generazione proporzionalmente più elevato, forzando il modello ad apprendere le aree di confine più ostiche.

24.3.4 Apprendimento Sensibile ai Costi e Matrice di Ponderazione

L'intervento a livello algoritmico evita la manipolazione artificiale del training set riparametrando la funzione obiettivo del classificatore mediante l'assegnazione di costi o pesi differenziati per le diverse tipologie di errore [57].

Definizione 24.3.4 (Matrice dei Costi e Costo Totale Atteso)**.** Sia $\mathcal{C}$ una matrice di dimensione 2×2 i cui elementi $C(i, j)$ rappresentano il costo o penalità associata all'assegnazione della classe predetta i quando la classe reale è j:

$$\mathcal{C} = \begin{bmatrix} C(\text{Pred } Y, \text{Act } Y) & C(\text{Pred } Y, \text{Act } N) \\ C(\text{Pred } N, \text{Act } Y) & C(\text{Pred } N, \text{Act } N) \end{bmatrix} = \begin{bmatrix} C_{\text{TP}} & C_{\text{FP}} \\ C_{\text{FN}} & C_{\text{TN}} \end{bmatrix} \tag{24.16}$$

In generale si assume che la classificazione corretta non comporti penalità ($C_{\text{TP}} = C_{\text{TN}} = 0$) e che il mancato riconoscimento della classe rara positiva (Falso Negativo) sia notevolmente più oneroso di un Falso Allarme (Falso Positivo): $C_{\text{FN}} \gg C_{\text{FP}}$. Il costo totale associato al classificatore è dato da:

$$\text{Costo Totale} = \sum_i \sum_j C(i, j) \cdot \mathbf{CM}(i, j) \tag{24.17}$$

Esempio 24.3.2 (Calcolo Numerico del Costo Pesato su 100 Record)**.** Si consideri un test set di $N = 100$ campioni valutato da un modello che produce la seguente matrice di confusione e matrice dei costi [57]:

Tabella 24.2: Confronto tra Matrice di Confusione e Matrice dei Costi su $N = 100$ record.

Matrice di Confusione (Conteggi)			**Matrice dei Costi (Penalità)**		
	Reale Y	**Reale** N		**Reale** Y	**Reale** N
Predetto Y	50 (TP)	7 (FP)	**Predetto** Y	0	95
Predetto N	3 (FN)	40 (TN)	**Predetto** N	5	0

Il costo complessivo normalizzato sul totale delle predizioni corrisponde a:

$$\begin{aligned}\text{Costo Medio} &= \frac{1}{N}\left(\text{FN} \times C(\text{Pred } N, \text{Act } Y) + \text{FP} \times C(\text{Pred } Y, \text{Act } N)\right) \\ &= \frac{1}{100}\left(3 \times 5 + 7 \times 95\right) \\ &= 0.03 \times 5 + 0.07 \times 95 = 0.15 + 6.65 = 6.80\end{aligned}$$

L'introduzione di pesi sulle classi (`class_weight='balanced'` o matrici di costo personalizzate) ha l'effetto geometrico di **traslare la frontiera decisionale** lontano dal cluster minoritario, riducendo drasticamente il tasso di falsi negativi a prezzo di una tollerabile crescita dei falsi allarmi.

24.4 Parte III: Modelli Generativi per Dati Sintetici Tabellari

24.4.1 Modelli Discriminativi vs Modelli Generativi

L'evoluzione più recente nel trattamento dello sbilanciamento e della privacy dei dati consiste nell'abbandonare le euristiche di interpolazione locale (come SMOTE) per apprendere l'intera densità di probabilità della popolazione tramite **modelli generativi profondi** [57].

Definizione 24.4.1 (Dicotomia tra Modelli Discriminativi e Generativi)**.** Dati un vettore di feature $\mathbf{x} \in \mathcal{X}$ e una variabile target $y \in \mathcal{Y}$:

- Un **modello discriminativo** modella direttamente la distribuzione condizionata $p(y \mid \mathbf{x})$, focalizzandosi sull'ottimizzazione del confine decisionale di separazione tra le classi (ad es. Regressione Logistica, SVM, Alberi di Decisione).
- Un **modello generativo** apprende la distribuzione di probabilità congiunta $p(\mathbf{x}, y)$ o la distribuzione marginale $p(\mathbf{x})$. Tramite la regola di Bayes, esso è in grado non solo di classificare, ma soprattutto di **campionare nuove osservazioni artificiali indistinguibili da quelle reali**:

$$\mathbf{x}_{\text{synthetic}} \sim p_{\text{model}}(\mathbf{x}) \tag{24.18}$$

Tabella 24.3: Evoluzione paradigmatica dei metodi di sovracampionamento e generazione sintetica.

Metodo / Modello	Meccanismo Operativo	Proprietà della Distribuzione Sintetica
Random Oversampling	Duplicazione con reinserimento	Non crea nuovi pattern; sovradatta i punti originali.
SMOTE	Interpolazione convessa locale su k-NN	Espande linearmente le regioni tra punti noti.
GAN (Generiche)	Addestramento avversario Minimax	Apprende distribuzioni complesse continue (immagini).
CTGAN	Condizionamento tabellare e VGM	Modella colonne miste, mode multiple e categorie rare.
VAE / TVAE	Inferenza variazionale su spazio latente	Genera campioni fluidi con regolarizzazione KL.

24.4.2 Generative Adversarial Networks (GAN)

Introdotte da Goodfellow et al. (2014) [62], le **Generative Adversarial Networks (GAN)** fondano l'apprendimento su un gioco a due giocatori a somma zero tra due reti neurali concorrenti [57]:

1. Il **Generatore (G):** parametrizzato da pesi $\boldsymbol{\theta}_g$, riceve in ingresso un vettore casuale di rumore latente $\mathbf{z} \sim p_z(\mathbf{z})$ (tipicamente gaussiano standard $\mathcal{N}(\mathbf{0}, \mathbf{I})$ o uniforme) e produce un vettore sintetico nello spazio delle feature $G(\mathbf{z}) \in \mathcal{X}$.

2. Il **Discriminatore (D):** parametrizzato da pesi $\boldsymbol{\theta}_d$, riceve un vettore $\mathbf{x}$ (reale $\mathbf{x} \sim p_{\text{data}}$ oppure generato $\mathbf{x} = G(\mathbf{z})$) e calcola una probabilità scalare $D(\mathbf{x}) \in [0, 1]$ che $\mathbf{x}$ provenga dalla distribuzione reale piuttosto che da G.

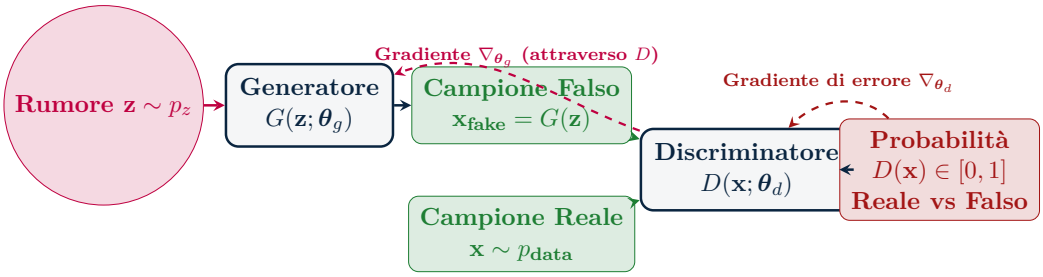


Figura 24.4: Architettura computazionale Minimax delle Generative Adversarial Networks (GAN).

Definizione 24.4.2 (Formulazione del Gioco Minimax)**.** L'addestramento congiunto di Generatore e Discriminatore è formalizzato attraverso un gioco minimax a somma zero sulla funzione obiettivo $V(D, G)$:

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_z(\mathbf{z})}[\log(1 - D(G(\mathbf{z})))] \quad (24.19)$$

Obiettivi di Ottimizzazione Alternata:

1. **Funzione di Perdita del Discriminatore (L_D):** convertendo la massimizzazione in minimizzazione, D cerca di minimizzare:

$$L_D = -\mathbb{E}_{\mathbf{x}\sim p_{\text{data}}}[\log D(\mathbf{x})] - \mathbb{E}_{\mathbf{z}\sim p_z}[\log(1 - D(G(\mathbf{z})))] \tag{24.20}$$

   Se $D(\mathbf{x}) \to 1$ per istanze reali e $D(G(\mathbf{z})) \to 0$ per istanze false, $L_D \to 0$.

2. **Funzione di Perdita Originale del Generatore (L_G):** mantenendo congelati i pesi del Discriminatore, G cerca di indurre $D(G(\mathbf{z})) \to 1$ minimizzando:

$$L_G = \mathbb{E}_{\mathbf{z}\sim p_z}[\log(1 - D(G(\mathbf{z})))] \tag{24.21}$$

Il Problema della Saturazione del Gradiente (Gradient Saturation)

Nelle prime epoche di addestramento, il Generatore è immaturo e sintetizza dati implausibili. Il Discriminatore discrimina con facilità assoluta ($D(G(\mathbf{z})) \approx 0$). In questa regione, la derivata della funzione $\log(1 - D)$ rispetto all'output del discriminatore è quasi nulla:

$$\lim_{D\to 0} \frac{d}{dD} \log(1 - D) = \lim_{D\to 0} \left(-\frac{1}{1 - D}\right) = -1 \tag{24.22}$$

mentre l'errore del generatore varia solo di un fattore trascurabile, fornendo un gradiente quasi impercettibile che blocca l'apprendimento.

Tabella 24.4: Intensità del segnale di apprendimento al variare dell'abilità di inganno di G.

$D(G(\mathbf{z}))$	Interpretazione Diagnostica	Perdita Originale $\log(1 - D(G(\mathbf{z})))$	Segnale di Gradiente per G
0.001	Discriminatore quasi certo che il dato sia falso	≈ -0.001	**Quasi nullo (Saturazione grave)**
0.010	Campione palesemente falso	≈ -0.010	**Molto debole**
0.100	Prime istanze parzialmente plausibili	≈ -0.105	In crescita
0.500	Massima incertezza (equilibrio di Nash)	≈ -0.693	Ottimale
0.900	Generatore inganna sistematicamente D	≈ -2.303	Molto forte

Risoluzione tramite Non-Saturating Generator Loss: Per garantire un segnale di discesa robusto fin dalla prima iterazione, si massimizza $\log D(G(\mathbf{z}))$ anziché minimizzare $\log(1 - D(G(\mathbf{z})))$, adottando la funzione di perdita euristica non saturante [62]:

$$L_G^{\text{NS}} = -\mathbb{E}_{\mathbf{z}\sim p_z}[\log(D(G(\mathbf{z})))] \tag{24.23}$$

Quando $D(G(\mathbf{z})) \to 0$, la funzione $-\log D$ presenta asintoto verso $+\infty$ con pendenza ripidissima, erogando gradienti elevati che guidano rapidamente il Generatore verso la sintesi di campioni verosimili.

24.4.3 Conditional Tabular GAN (CTGAN)

Sebbene le GAN classiche eccellano su segnali percettivi continui e strutturati spazialmente (come le immagini digitali), la loro applicazione diretta ai database relazionali tabellari fallisce drasticamente a causa di tre peculiarità strutturali [63]:

1. **Tipi di Dati Misti (*Mixed Data Types*):** ogni riga combina simultaneamente attributi continui, discreti, ordinali e categorici nominali. Le reti neurali producono nativamente output continui; generare modalità categoriche (es. `native_country = Italy`) richiede funzioni softmax discrete.

2. **Distribuzioni Continue Multimodali:** le variabili numeriche finanziarie o demografiche (es. reddito annuo o ore di lavoro) presentano spesso molteplici mode pronunciate. Normalizzare globalmente l'attributo in $[-1, 1]$ appiattisce le modalità e confonde il generatore.

3. **Variabili Categoriche Altamente Sbilanciate:** campionando le righe in modo puramente uniforme, le categorie rare (aventi frequenze inferiori all'1%) vengono esposte al discriminatore con frequenza quasi nulla, portando il generatore al collasso modale (*mode collapse*) sulle sole classi frequenti.

Per superare tali limitazioni, Xu et al. (2019) hanno teorizzato e implementato il framework **CTGAN** (*Conditional Tabular GAN*) [63], imperniato su tre innovazioni:

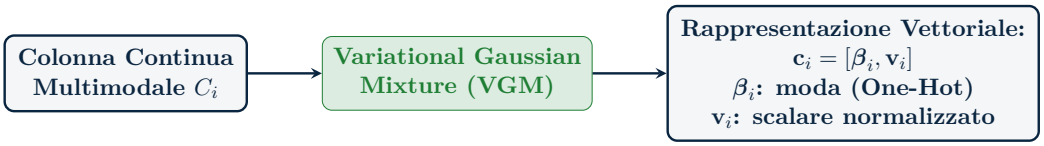


Figura 24.5: Meccanismo di Mode-Specific Normalization in CTGAN: scomposizione di una variabile continua in moda discreta e deviazione scalare gaussiana normalizzata.

1. **Mode-Specific Normalization (Normalizzazione Modale Variazionale):** Ciascuna variabile continua C_j viene modellata mediante un modello di mistura di gaussiane variazionale (*Variational Gaussian Mixture*, VGM) con K componenti. Dato un valore scalare c, il VGM ne calcola l'appartenenza alle diverse mode gaussiane:

$$p(c) = \sum_{k=1}^{K} \pi_k \mathcal{N}(c; \mu_k, \sigma_k^2) \tag{24.24}$$

Il valore continuo c viene quindi ricodificato come una tupla composta da un vettore one-hot $\boldsymbol{\beta}$ (che codifica la componente gaussiana più probabile) e da uno scalare normalizzato v:

$$v = \frac{c - \mu_k}{4\sigma_k} \in [-1, 1] \tag{24.25}$$

In questo modo, il Generatore apprende separatamente a selezionare la moda discreta e a stimare la deviazione normalizzata all'interno di quella specifica gaussiana.

2. **Conditional Generator (Generatore Condizionale):** Per consentire la generazione di campioni che soddisfino specifici vincoli categorici, il Generatore riceve in ingresso una tupla formata dal rumore latente $\mathbf{z}$ e da un **vettore di condizione** $\mathbf{c}_{\text{cond}}$:

$$\mathbf{x}_{\text{syn}} = G(\mathbf{z}, \mathbf{c}_{\text{cond}}) \tag{24.26}$$

Fissando ad esempio la condizione $\mathbf{c}_{\text{cond}}$ sull'attributo (`Occupation` = `Exec`), il generatore campiona direttamente dalla distribuzione condizionata $p(\mathbf{x} \mid \mathbf{c}_{\text{cond}})$.

3. **Training-by-Sampling (Campionamento a Log-Frequenze):** Durante la costruzione dei batch di addestramento, le categorie non vengono selezionate in base alla loro frequenza empirica uniforme; la probabilità di selezionare una categoria k viene modulata in funzione del logaritmo della sua frequenza:

$$P(\text{seleziona } k) \propto \log(1 + \text{freq}_k) \tag{24.27}$$

Questa equalizzazione parziale fa sì che le classi rare ricevano sufficiente gradiente per essere apprese stabilmente senza incorrere nell'appiattimento totale della distribuzione naturale.

Nell'ingegneria per dati sbilanciati, CTGAN viene addestrato sull'intero dataset oppure confinato **esclusivamente alle righe della classe minoritaria**: in tale regime operativo, il generatore sintetizza righe tabellari sintetiche che replicano fedelmente correlazioni multivariate e mode complesse senza violare la coerenza statistica tra colonne.

24.4.4 Variational Autoencoders (VAE) e Tabular VAE (TVAE)

I **Variational Autoencoders (VAE)**, introdotti da Kingma e Welling (2013) [64], affrontano la modellazione generativa attraverso un approccio rigorosamente probabilistico fondato sull'inferenza variazionale [57].

Limite degli Autoencoder Deterministici Tradizionali: Un classico autoencoder comprime l'ingresso $\mathbf{x} \in \mathbb{R}^D$ in una rappresentazione latente deterministica $\mathbf{h} = E(\mathbf{x}) \in \mathbb{R}^d$ ($d \ll D$), per poi ricostruire l'input mediante il decoder $\mathbf{x}' = D(\mathbf{h}) \approx \mathbf{x}$ minimizzando l'errore quadratico $\|\mathbf{x} - \mathbf{x}'\|^2$. Poiché lo spazio latente $\mathbb{R}^d$ non è soggetto ad alcun vincolo di continuità o regolarità, esso risulta frammentato da lacune prive di significato geometrico: campionare un punto $\mathbf{h}$ a caso genera ricostruzioni prive di coerenza empirica.

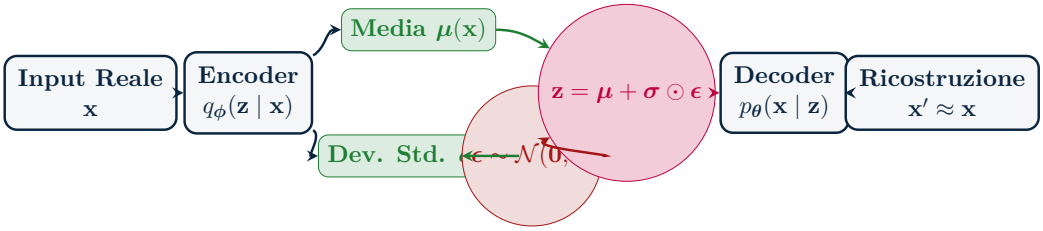


Figura 24.6: Architettura del Variational Autoencoder (VAE) con implementazione del Reparameterization Trick.

Definizione 24.4.3 (Architettura Probabilistica del VAE e Reparameterization Trick)**.** Nel VAE, l'Encoder $q_{\boldsymbol{\phi}}(\mathbf{z} \mid \mathbf{x})$ modella una distribuzione di densità di probabilità gaussiana su uno spazio latente continuo $\mathcal{Z}$, emettendo per ciascun input un vettore di medie $\boldsymbol{\mu} \in \mathbb{R}^d$ e un vettore di varianze (o log-varianze) $\boldsymbol{\sigma}^2 \in \mathbb{R}^d$. Poiché il campionamento stocastico diretto $\mathbf{z} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2\mathbf{I})$ bloccherebbe la retropropagazione del gradiente ($\nabla_{\boldsymbol{\phi}}$ non potrebbe fluire attraverso un'operazione aleatoria discreta), si applica il **Reparameterization Trick** [64]:

$$\mathbf{z} = \boldsymbol{\mu}(\mathbf{x}) + \boldsymbol{\sigma}(\mathbf{x}) \odot \boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \tag{24.28}$$

dove $\odot$ indica il prodotto vettoriale elemento per elemento (prodotto di Hadamard). In questa forma, la stocasticità è interamente esternalizzata nella variabile ausiliaria $\boldsymbol{\epsilon}$, rendendo la trasformazione derivabile rispetto ai parametri dell'Encoder $\boldsymbol{\phi}$.

Definizione 24.4.4 (Funzione di Perdita VAE e Limite Variazionale ELBO)**.** La funzione di perdita da minimizzare nel VAE è l'opposto del limite inferiore variazionale sull'evidenza (*Evidence Lower Bound*, ELBO):

$$\mathcal{L}_{\text{VAE}}(\boldsymbol{\theta}, \boldsymbol{\phi}; \mathbf{x}) = \mathcal{L}_{\text{reconstruction}}(\mathbf{x}, \mathbf{x}') + \mathcal{L}_{\text{KL}}(q_{\boldsymbol{\phi}}(\mathbf{z} \mid \mathbf{x}) \,\|\, p(\mathbf{z})) \tag{24.29}$$

1. **Reconstruction Loss ($\mathcal{L}_{\text{reconstruction}}$):** misura la verosimiglianza di fedeltà nella ricostruzione (ad es. errore quadratico medio per attributi continui o cross-entropia binaria per attributi categorici). Essa forza il Decoder a conservare le informazioni discriminanti dell'input.

2. **KL Divergence ($\mathcal{L}_{\mathbf{KL}}$):** misura la divergenza di Kullback-Leibler tra la densità stimata dall'Encoder $q_{\boldsymbol{\phi}}(\mathbf{z} \mid \mathbf{x})$ e una distribuzione a priori normale standard multivariata $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$:

$$D_{\mathrm{KL}}\left(\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma}) \,\|\, \mathcal{N}(\mathbf{0}, \mathbf{I})\right) = -\frac{1}{2} \sum_{j=1}^{d} \left(1 + \log(\sigma_j^2) - \mu_j^2 - \sigma_j^2\right) \quad (24.30)$$

Il termine agisce come regolarizzatore topologico: impedisce all'encoder di collassare su punti deterministici dispersi, costringendo lo spazio latente a essere liscio, continuo e centrato nell'origine.

Teorema 24.4.1 (Trade-off nel Bilanciamento della Loss del VAE)**.** *I due termini di perdita operano in equilibrio competitivo:*

- *Se $\mathcal{L}_{KL}$ venisse azzerato, lo spazio latente collasserebbe nella geometria disarticolata di un autoencoder standard, impedendo il campionamento casuale.*
- *Se $\mathcal{L}_{reconstruction}$ venisse azzerato, l'Encoder mapperebbe qualsiasi input su $\mathcal{N}(\mathbf{0}, \mathbf{I})$, perdendo ogni capacità descrittiva e generando rumore puro privo di informazione.*

Generazione Sintetica a Regime: Al termine dell'addestramento, l'Encoder viene rimosso dal ciclo di calcolo. Per sintetizzare M nuove osservazioni della classe minoritaria, si campionano M vettori casuali dalla distribuzione normale multivariata:

$$\mathbf{z}_m \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \qquad m = 1, \ldots, M \quad (24.31)$$

e si propagano attraverso il Decoder addestrato:

$$\mathbf{x}_{\mathrm{syn},m} = \mathrm{Decoder}(\mathbf{z}_m) \quad (24.32)$$

Il Tabular VAE (TVAE): Proposto anch'esso da Xu et al. (2019) [63], **TVAE** adatta l'architettura VAE alle specificità dei dati tabellari applicando la medesima normalizzazione modale variazionale (VGM) per gli attributi continui e strati di output modulari (con attivazioni softmax separate) per ciascun predittore categorico. Esperimenti empirici estesi dimostrano che, mentre CTGAN risulta spesso superiore nel preservare complesse correlazioni ad alta dimensionalità e categorie rare, TVAE evidenzia una **superiore stabilità di addestramento** (non soffrendo dei problemi di convergenza ciclica o collasso modale tipici del gioco minimax delle GAN) e tempi di calcolo significativamente più contenuti.

24.5 Riepilogo e Conclusioni

In questo capitolo sono stati esaminati congiuntamente i fondamenti empirici dell'apprendimento supervisionato su dati tabellari eterogenei e le metodologie formali per fronteggiare la sfida dei dati fortemente sbilanciati:

- **Pre-elaborazione e Codifica:** l'analisi del dataset `adult` ha dimostrato come la scelta tra Label Encoding e One-Hot Encoding dipenda dalla natura geometrica del classificatore prescelto, con gli alberi di decisione in grado di operare efficacemente su codifiche intere senza soffrire delle distorsioni metriche che invalidano modelli basati su distanze euclidee (k-NN).

- **Alberi di Decisione e Limiti Empirici:** la restrizione della profondità e dei campioni minimi per foglia garantisce regolarizzazione e interpretabilità, ma l'accuratezza apparentemente elevata (84.9%) maschera una debolezza intrinseca nel catturare la classe minoritaria (recall del 59%).

- **Sbilanciamento di Classe e Baseline Banali:** la valutazione formale del paradosso dell'accuratezza e della baseline A_{trivial} ha chiarito l'obbligo metodologico di impiegare metriche sensibili (Recall, F_1, matrici di confusione e curve PR) per prevenire l'adozione di modelli peggiori della scelta banale della moda.

- **Strategie di Bilanciamento Tradizionali:** l'indagine ha confrontato il sottocampionamento intelligente (CNN per preservare i confini, Tomek Links per chiarificare le frontiere ambigue) e il sovracampionamento nello spazio delle feature (SMOTE, Borderline-SMOTE, ADASYN) con l'apprendimento sensibile ai costi basato su matrici di penalità.

- **Modelli Generativi Profondi:** la transizione verso la modellazione generativa tramite GAN e VAE consente di superare l'euristica di interpolazione locale. In particolare, le formulazioni specializzate per matrici relazionali (CTGAN con normalizzazione modale VGM e campionamento logaritmico, e TVAE con inferenza variazionale stabilizzata) rappresentano lo stato dell'arte per la sintesi realistica di campioni minoritari nel rispetto delle correlazioni multivariate e della privacy statistica.

Bibliografia

[1] Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96)*, pages 226–231. AAAI Press, 1996.

[2] Pang-Ning Tan, Michael Steinbach, Anuj Karpatne, and Vipin Kumar. *Introduction to Data Mining*. Pearson, 2nd edition, 2018.

[3] Mihael Ankerst, Markus M. Breunig, Hans-Peter Kriegel, and Jörg Sander. OPTICS: Ordering points to identify the clustering structure. *ACM SIGMOD Record*, 28(2):49–60, 1999.

[4] Ricardo J. G. B. Campello, Davoud Moulavi, Arthur Zimek, and Jörg Sander. Hierarchical density estimates for data clustering, visualization, and outlier detection. *ACM Transactions on Knowledge Discovery from Data (TKDD)*, 10(1):5:1–5:51, 2015.

[5] Dan Pelleg and Andrew W. Moore. X-means: Extending K-means with efficient estimation of the number of clusters. In *Proceedings of the Seventeenth International Conference on Machine Learning (ICML 2000)*, pages 727–734. Morgan Kaufmann, 2000.

[6] Gideon Schwarz. Estimating the dimension of a model. *The Annals of Statistics*, 6(2):461–464, 1978.

[7] A. P. Dempster, N. M. Laird, and D. B. Rubin. Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society: Series B (Methodological)*, 39(1):1–38, 1977.

[8] Anil K. Jain and Richard C. Dubes. *Algorithms for Clustering Data*. Prentice-Hall, 1988.

[9] Brian Hopkins and J. G. Skellam. A new method for determining the type of distribution of plant individuals. *Annals of Botany*, 18(2):213–227, 1954.

[10] Peter J. Rousseeuw. Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20:53–65, 1987.

[11] David L. Davies and Donald W. Bouldin. A cluster separation measure. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-1(2):224–227, 1979.

[12] T. Caliński and J. Harabasz. A dendrite method for cluster analysis. *Communications in Statistics*, 3(1):1–27, 1974.

[13] Rakesh Agrawal, Tomasz Imieliński, and Arun Swami. Mining association rules between sets of items in large databases. *ACM SIGMOD Record*, 22(2):207–216, 1993.

[14] Rakesh Agrawal and Ramakrishnan Srikant. Fast algorithms for mining association rules in large databases. In *Proceedings of the 20th International Conference on Very Large Data Bases (VLDB '94)*, pages 487–499. Morgan Kaufmann, 1994.

[15] Gregory Piatetsky-Shapiro. Discovery, analysis, and presentation of strong rules. In *Knowledge Discovery in Databases*, pages 229–248. AAAI/MIT Press, 1991.

[16] Rakesh Agrawal and Ramakrishnan Srikant. Mining sequential patterns. In *Proceedings of the Eleventh International Conference on Data Engineering (ICDE '95)*, pages 3–14. IEEE Computer Society, 1995.

[17] Ramakrishnan Srikant and Rakesh Agrawal. Mining sequential patterns: Generalizations and performance improvements. In *Proceedings of the 5th International Conference on Extending Database Technology (EDBT '96)*, pages 3–17. Springer, 1996.

[18] Jiawei Han, Jian Pei, and Yiwen Yin. Mining frequent patterns without candidate generation. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data (SIGMOD '00)*, pages 1–12. ACM, 2000.

[19] Jiawei Han, Micheline Kamber, and Jian Pei. *Data Mining: Concepts and Techniques*. Morgan Kaufmann, 3rd edition, 2011.

[20] Rob J. Hyndman and George Athanasopoulos. *Forecasting: Principles and Practice*. OTexts, 2nd edition, 2018.

[21] Robert H. Shumway and David S. Stoffer. *Time Series Analysis and Its Applications: With R Examples*. Springer, 4th edition, 2017.

[22] Hiroaki Sakoe and Seibi Chiba. Dynamic programming algorithm optimization for spoken word recognition. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 26(1):43–49, 1978.

[23] Chotirat Ann Ratanamahatana, Jessica Lin, and Eamonn Keogh. Mining time series data. In *Data Mining and Knowledge Discovery Handbook*, pages 1049–1077. Springer, 2010.

[24] Eamonn Keogh, Stefano Lonardi, and Chotirat Ann Ratanamahatana. Towards parameter-free data mining. pages 206–215, 2004.

[25] Jessica Lin, Eamonn Keogh, Li Wei, and Stefano Lonardi. Experiencing SAX: A novel symbolic representation of time series. *Data Mining and Knowledge Discovery*, 15(2):107–144, 2007.

[26] Chin-Chia Michael Yeh, Yan Zhu, Liudmila Ulanova, Nurjahan Begum, Yifei Ding, Hoang Anh Dau, Diego Furtado Silva, Abdullah Mueen, and Eamonn Keogh. Matrix profile I: All pairs similarity joins for time series: A unifying view that includes motifs, discords and shapelets. In *Proceedings of the 2016 IEEE 16th International Conference on Data Mining (ICDM)*, pages 1317–1322. IEEE, 2016.

[27] Abdullah Mueen, Eamonn Keogh, Qiang Zhu, Sydney Cash, and M. Brandon Westover. Exact discovery of time series motifs. In *Proceedings of the 2009 SIAM International Conference on Data Mining (SDM)*, pages 473–484. SIAM, 2009.

[28] Albert-László Barabási. *Network Science*. Cambridge University Press, 2016.

[29] Giulio Rossetti, Letizia Milli, and Rémy Cazabet. CDlib: A python library to extract, compare and evaluate communities from complex networks. *Applied Network Science*, 4(1):1–26, 2019.

[30] Vincent D. Blondel, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment*, 2008(10):P10008, 2008.

[31] Wayne W. Zachary. An information flow model for conflict and fission in a small group. *Journal of Anthropological Research*, 33(4):452–473, 1977.

[32] Mark E. J. Newman. Mixing patterns in networks. *Physical Review E*, 67(2):026126, 2003.

[33] Ulrik Brandes. A faster algorithm for betweenness centrality. *Journal of Mathematical Sociology*, 25(2):163–177, 2001.

[34] Santo Fortunato. Community detection in graphs. *Physics Reports*, 486(3–5):75–174, 2010.

[35] Zhexue Huang. Extensions to the k-means algorithm for clustering large data sets with categorical values. *Data Mining and Knowledge Discovery*, 2(3):283–304, 1998.

[36] John C. Gower. A general coefficient of similarity and some of its properties. *Biometrics*, 27(4):857–871, 1971.

[37] Ronan Perry, Gavin Mischler, Richard Guo, Theodore Lee, Alexander Chang, Ananya Kothari, Sambit Panda, Tommy Mehta, Cencheng Shen, and Joshua T. Vogelstein. mvlearn: Multiview machine learning in Python. *Journal of Machine Learning Research*, 22(109):1–7, 2021.

[38] Alexander Strehl and Joydeep Ghosh. Cluster ensembles—a knowledge reuse framework for combining multiple partitions. *Journal of Machine Learning Research*, 3:583–617, 2002.

[39] Abhishek Kumar, Piyush Rai, and Hal Daumé III. Co-regularized multi-view spectral clustering. In *Advances in Neural Information Processing Systems (NeurIPS 2011)*, volume 24, pages 1413–1421, 2011.

[40] David M. Blei, Andrew Y. Ng, and Michael I. Jordan. Latent Dirichlet allocation. *Journal of Machine Learning Research*, 3:993–1022, 2003.

[41] Maarten Grootendorst. BERTopic: Neural topic modeling with a class-based TF-IDF procedure. *arXiv preprint arXiv:2203.05794*, 2022.

[42] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP 2019)*, pages 3982–3992, 2019.

[43] Leland McInnes, John Healy, and James Melville. UMAP: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*, 2018.

[44] Leo Breiman, Jerome H. Friedman, Richard A. Olshen, and Charles J. Stone. *Classification and Regression Trees*. Wadsworth, 1984.

[45] Earl B. Hunt, Janet Marin, and Philip J. Stone. *Experiments in Induction*. Academic Press, 1966.

[46] J. Ross Quinlan. Induction of decision trees. *Machine Learning*, 1(1):81–106, 1986.

[47] J. Ross Quinlan. *C4.5: Programs for Machine Learning*. Morgan Kaufmann, 1993.

[48] Jorma Rissanen. Modeling by shortest data description. *Automatica*, 14(5):465–471, 1978.

[49] Ron Kohavi. A study of cross-validation and bootstrap for accuracy estimation and model selection. In *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI 1995)*, volume 2, pages 1137–1145, 1995.

[50] Tom Fawcett. An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8):861–874, 2006.

[51] Leo Breiman. Bagging predictors. *Machine Learning*, 24(2):123–140, 1996.

[52] Yoav Freund and Robert E. Schapire. A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1):119–139, 1997.

[53] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

[54] Frank Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408, 1958.

[55] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.

[56] Lorenzo Mannocci. Backpropagation: A complete example. Materiale didattico del corso di Data Mining, Università di Pisa, 2026.

[57] Lorenzo Mannocci. Imbalanced data and synthetic generation. Slide didattiche del corso di Data Mining, Università di Pisa, 2026.

[58] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357, 2002.

[59] Peter E. Hart. The condensed nearest neighbor rule. *IEEE Transactions on Information Theory*, 14(3):515–516, 1968.

[60] Ivan Tomek. Two modifications of CNN. *IEEE Transactions on Systems, Man, and Cybernetics*, 6(11):769–772, 1976.

[61] Haibo He, Yang Bai, Edwardo A. Garcia, and Shutao Li. ADASYN: Adaptive synthetic sampling approach for imbalanced learning. In *IEEE International Joint Conference on Neural Networks (IEEE World Congress on Computational Intelligence)*, pages 1322–1328, 2008.

[62] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *Advances in Neural Information Processing Systems*, volume 27, pages 2672–2680, 2014.

[63] Lei Xu, Maria Skoularidou, Alfredo Cuesta-Infante, and Kalyan Veeramachaneni. Modeling tabular data using conditional GAN. In *Advances in Neural Information Processing Systems*, volume 32, pages 7385–7394, 2019.

[64] Diederik P. Kingma and Max Welling. Auto-encoding variational Bayes. *arXiv preprint arXiv:1312.6114*, 2013.